SPECIFICATION

大学駅伝 育成シミュレーション

(仮題)

仕様書 統合版

統合版 第 1 版(2026-10-29 時点)

第 0 部 全体像 ／ 第 1〜10 部 各仕様書(基本設計・データ・育成・駅伝本番・世界とスカウト・イベント・特別招聘・画面・セーブと技術・個性と素養)／ 付録 A〜G

原典のリポジトリ `ekiden-sim`(非公開)の Markdown から `specbook/build_spec_pdf.py` が生成。仕様の正本は原典。

文書情報

項目	内容
名称	大学駅伝 育成シミュレーション(仮題)仕様書 **統合版**
版	統合版 第 1 版(2026-10-29 時点)
対象	育成シミュレーション『大学駅伝(仮題)』のゲーム仕様・データ仕様・画面仕様・セーブと技術の仕様の全体
構成	第 0 部 全体像(本書のために整理した要約)/ 第 1〜10 部 各仕様書の全文(第 8 部は 08・08a・08b)/ 付録 A〜G
原典	リポジトリ `ekiden-sim`(非公開)の `01_basic_system_design.md` 〜 `10_individuality_spec.md`(08a・08b を含む)と `00_spec_index.md`。**仕様の正本は原典**で、本書はそれを 1 冊にまとめた編集版。数値・規則・表は原典から**加工せずに**収録している
作成方法	`specbook/build_spec_pdf.py` が原典(Markdown)から生成する。原典を直したら再生成する(手で PDF を直さない)
状態	仕様の作成(マイルストーン S)は一巡済み。M0(Domain の C# 実装と検算)は完了。M1 に向けた準備(画面向けの読み取り API・App 層)まで実装済み(第 0 部 §7)
前提	架空の大学・選手・大会名を使う。大会名は仮称(秋雲駅伝 / 全国大学駅伝 / 新春大学駅伝)で、実名化は商用化前に判断する

各部の文書番号と版

部	文書	原典のファイル	原典の版
第 1 部	基本システム設計書	`01_basic_system_design.md`	v1.0
第 2 部	データ仕様	`02_data_spec.md`	v1.1
第 3 部	育成ルール仕様	`03_training_rules_spec.md`	v0.3
第 4 部	駅伝本番仕様	`04_race_spec.md`	v0.2
第 5 部	世界・スカウト・名声・資金・施設 仕様	`05_world_scout_spec.md`	v0.3
第 6 部	イベント仕様	`06_event_spec.md`	v0.1
第 7 部	特別招聘・追加コンテンツ仕様	`07_special_recruit_spec.md`	v0.1
第 8 部	画面仕様(本編・カタログ・読み取り API)	`08_screen_spec.md` / `08a_screen_catalog.md` / `08b_screen_data.md`	v0.1 / v0.1 / M1 準備
第 9 部	セーブ・技術仕様	`09_save_tech_spec.md`	v0.1
第 10 部	個性・素養仕様	`10_individuality_spec.md`	v0.1
付録 A〜E	用語集・基準・未決・課題・検算ツール	`00_spec_index.md`	—

> 文書の版は、その文書を最後に大きく改めたときの版で、本書の版とは別。各文書の改版の経緯は付録 G にまとめた。

目 次

文書情報 2

目 次 3

この仕様書の読み方 7

第0部 全体像 8
1. 目的と範囲 8
2. ゲームの概要 8
3. ゲームの構造 9
4. 主要な仕組みの要約 9
5. 仕様書の体系 12
6. 検証の方法と受け入れ基準 12
7. 実装と検証の現況(2026-10-29 時点) 13
8. 決定済みの主要事項 13
9. 残課題と今後 14

第1部 基本システム設計書 15
0. 前提・決定事項 15
1. コンセプト 15
2. パワプロとの対応表(設計の拠り所) 16
3. ゲーム全体構造 16
4. 選手システム 18
5. 育成システム(ゲームの中心) 20
6. チーム運営と世代交代 21
7. 駅伝本番システム 24
8. ライバル校・世界シミュレーション 25
9. イベントシステム 26
10. 体験イメージ(中堅校スタート) 26
11. UI/UX・アート方針(Steam + スマホ / 2D) 26
12. 技術設計(Unity) 27
13. 開発ロードマップ 28
14. 決定済み事項と未決事項 29
付録: パワプロ要素を落とさないためのチェックリスト 29

第2部 データ仕様 30
1. 規約 30
2. 列挙(Enum) 30
3. マスタデータ 31
4. 実行時エンティティ 39
5. 不変条件(Invariants) 45
6. 出身県・留学生に関する生成規則(データ定義の範囲) 46
7. 次の文書への引き継ぎ 46

第3部 育成ルール仕様 48
1. 用語と単位 48
2. 定数表(初期値案。K_ 接頭辞) 48
3. 乱数 49
4. マスタ値 49
5. 週メニューの決定 51
6. 週の解決手順(WeekResolve、選手ごとに独立) 53
7. 月次処理(MonthEnd、各月の第4週の終わり) 54
8. 他文書から呼ばれる操作(フック) 55
9. 境界・例外 55
10. 計算例(テストベクタ) 56
11. 検算結果(設計意図の確認) 57

12. 他文書への反映事項(本書の確定による変更) 58
13. 未決・次の文書への引き継ぎ 58

第4部 駅伝本番仕様 60
1. 大会の流れ(週の時系列) 60
2. 定数表(初期値案) 60
3. 出場権 61
4. エントリー(登録) 62
5. 区間オーダー 62
6. 環境 63
7. 区間タイム 63
8. レース進行と順位 65
9. 監督の介入 66
10. 大会後の処理 67
11. 他校の方針と、検算用のプレイヤーの方針 67
12. 乱数のキー(03 §3 の PCG32) 68
13. 計算例(テストベクタ。verify/ref_model_04.py vectors) 68
14. 検算結果 70
15. 課題・未決・他文書への反映 73

第5部 世界・スカウト・名声・資金・施設 仕様 74
1. 前提と用語 74
2. 定数表(初期値案) 74
3. 名声と評価 75
4. 資金 76
5. 施設・スタッフ 76
6. 入学候補(プール) 77
7. スカウト活動(自校) 78
8. 勧誘の確定(3月の割り当て) 78
9. 他校(世界)の更新 79
10. 入学・卒業・OB 80
11. 特別招聘券との関係 81
12. 年度末処理の順序(3月) 81
13. 計算例(乱数を使わない部分。verify/ref_model_05.py vectors) 81
14. 検算結果 82
15. 課題・未決・他文書への反映 88

第6部 イベント仕様 90
1. 目的と原則 90
2. 定数表(初期値案。K_EV_ 接頭辞) 90
3. 条件式(DSL) 91
4. イベント定義(EventDef) 92
5. 効果 93
6. 補正(TimedModifier) 94
7. 発生の手順(週ごと) 95
8. OBの役割(05 §10.4) 96
9. 特殊能力の覚醒 96
10. カウンター(ctr)と更新のフック 97
11. 04 との接続 98
12. データ(02 への追加) 98
13. 検証ルール(データを読み込むときとテストで確認する) 98
14. 初期イベント一覧 99
15. 計算例(テストベクタ。verify/ref_model_06.py vectors) 103
16. 検算結果(ref_model_06.py sim) 104
17. 課題・未決・他文書への反映 106

第7部 特別招聘・追加コンテンツ仕様 108
1. 原則 108

2. 定数表(初期値案。K_TICKET_ / K_PLAN_ 接頭辞) 108
3. 特別招聘券 109
4. ゲーム内の獲得(TicketMilestone) 110
5. 黄金世代プラン(GoldenPlan) 110
6. 招聘選手の生成 111
7. 宣言の画面に出す範囲 112
8. 購入状態(エンタイトルメント) 113
9. 追加コンテンツ(ContentPack) 114
10. 開始状態(シナリオ) 115
11. 実績・共有との関係 116
12. 検算結果 116
13. 計算例(テストベクタ。verify/ref_model_07.py vectors) 117
14. 課題・未決・他文書への反映 119

第8部 画面仕様 — 本編 121
1. 原則 121
2. 定数表(初期値案。K_UI_ 接頭辞) 121
3. 入力モデル 122
4. 画面の基準 123
5. 画面の一覧と遷移 124
6. 進行制御(FlowController) 126
7. 共通部品 129
8. 画面のための規則(R-01〜R-17) 131
9. 設定項目と保存の場所 136
10. データへの追加(02_data_spec.md へ反映する内容) 137
11. 検算結果(verify/ref_model_08.py) 137
12. 計算例(テストベクタ。verify/ref_model_08.py vectors) 140
13. 課題・未決・他文書への反映 143

第8部 画面仕様 — 画面カタログ 147
A. 起動・設定 147
B. ホームと進行 149
C. 育成・部員 150
D. 大会 152
E. スカウト 154
F. 運営・歴史 155
G. ダイアログ・割り込み 156

第8部 画面仕様 — 画面ごとの必要データと読み取り API 159
1. 読み方 159
2. 実装済みの読み取り API 159
3. 画面ごとの対応表 162
4. 足りないもの(ギャップ) 165
5. 次の順序(提案) 167

第9部 セーブ・技術仕様 168
1. 設計の原則 168
2. 定数表(初期値案。K_ 接頭辞。ref_model_09.py の K と一致させる) 168
3. 保存するものと置き場所 169
4. コンテナ形式 EKSV 170
5. セーブの中身とサイズ 173
6. バージョニングとマイグレーション 178
7. 読み込み時の検証と修復 179
8. 乱数と再現性 181
9. クラウド同期 183
10. 購入状態の時計対策と LocalState 186
11. 実績・購入の通知のタイミング(U-14) 187
12. マスタデータと文字列テーブル 188

13. 数値・時間の整形(08 §7.4 の正本) 189
14. 性能とスレッド 190
15. 技術スタックとプロジェクト構成 191
16. テストと自動検証 194
17. 仕様完了のチェックと M0 の着手条件 195
18. 課題・未決・他文書への反映 200
19. テストベクタ(verify/ref_model_09.py) 201

第10部 個性・素養仕様 204
1. 要約 204
2. 定義 204
3. 設計の原則 206
4. 生成 207
5. 練習への影響 207
6. レースへの影響(04 §7.3 が確定) 209
7. 特殊能力の取得への影響(06 §9 が確定) 210
8. 見せ方 211
9. 検算結果 212
10. 計算例(テストベクタ。verify/ref_model_10.py vectors) 212
11. 課題・未決・他文書への反映 214

付録A 用語集 216

付録B 仕様完了の基準と書式規約 218
B.1 仕様完了の基準(各文書がすべて満たすこと) 218
B.2 書式規約 218

付録C 未決トラッカー(既定値付き) 219

付録D 課題トラッカーと拡張メモ 220
D.1 長期バランスの課題トラッカー 220
D.2 拡張メモ(未設計。採用は未決) 228

付録E 検算ツールと実行方法 231
E.1 参照モデル(Python)と検算の実行 231
E.2 C# の実装と実行コマンド 231

付録F 仕様と実装(C#)・試験の対応 233
F.1 仕様の各部と実装の対応 233
F.2 実装の層と依存 235
F.3 主要な型と入口 235
F.4 注意すべき実装上の約束 235

付録G 改版履歴 236
G.1 第1部 基本システム設計書(01) 236
G.2 第2部 データ仕様(02) 236
G.3 第3部 育成ルール仕様(03) 236
G.4 第4部 駅伝本番仕様(04) 237
G.5 第5部 世界・スカウト・名声・資金・施設 仕様(05) 237

この仕様書の読み方

1. 読む順番

- 初めて読む人は、**第 0 部(全体像)**だけで、ゲームの概要・仕組み・現在地が分かる(約 7 ページ)。
- 規則の詳細は第 1 部以降。**第 1 部(基本設計)**がコンセプトと全体構造、**第 2 部(データ)**が全データの定義、第 3〜7 部が規則、第 8 部が画面、第 9 部が保存と技術、第 10 部が個性・素養。
- 数値を調べたいときは、各部の **「2. 定数表」**(`K_` で始まる名前)を見る。計算例(テストベクタ)は各部の後ろの方にある。

2. 相互参照の書き方

原典の相互参照は、そのまま使えるように書き換えていない。

書き方	意味	例
`04 §7.4`	第 4 部の 7.4 節	`04 §7.4` (ペース配分)
`08 R-05`	第 8 部 8 章の規則 R-05	月末の予測
`08 SC-26`	第 8 部の画面カタログ(08a)の画面 SC-26	大会の結果
`00 §2` `00 §3`	付録 B(完了の基準・書式規約)	
`00 §4`	付録 A(用語集)	
`00 §5`	付録 C(未決トラッカー U-xx)	
`00 §6` `00 §7`	付録 D(課題トラッカー L-xx・拡張メモ)	
`K_…`	定数(各部の定数表。調整できる値)	`K_XP_SCALE`
`L-nn`	課題(付録 D。『解決』『一部解決』『新規』)	`L-59`
`U-nn`	未決・決定事項(付録 C)	`U-12`
`TV-n`	テストベクタ(入力と期待値が数値で書かれた計算例)	`03 TV-5`
`I-nn`	不変条件(02 §5)	`I-04`
`R-nn`	画面のための規則(08 §8)	`R-04`

3. 数値の意味

- 「初期値案」とある数値は、現時点の設計値で、プレイテストとシミュレーションで調整する前提。調整できる値は定数表(`K_`)に集約してある。
- 「検算」「計算例」は、Python の参照モデル(`verify/ref_model_NN.py`)の出力と一致する値。C# の実装は、そのまま自動テストにして一致を確かめている(付録 E・F)。

4. 原典に出てくる場所

- `verify/` は Python の参照モデル、`m0/` は C# の実装(Unity プロジェクトを兼ねる)。いずれもリポジトリ内の場所で、本書には含まれない。

5. 付録の使い方

付録	内容
A	用語集(日本語 ↔ 識別子 ↔ 意味)
B	仕様完了の基準と書式規約
C	未決トラッカー(U-nn。既定値つき)
D	課題トラッカー(L-nn)と拡張メモ
E	検算ツール(参照モデル・ゴールデン・SimCli)と実行方法
F	仕様と実装(C#)・試験の対応表
G	改版履歴(各文書の変更の経緯)

第0部 全体像

> この部は、本書のために原典から**要約・整理**した概要編。規則の正確な内容・数値は、各項の末尾に示した部を正とする。

1. 目的と範囲

- **目的**: 『大学駅伝(仮題)』を、実装の途中で迷わず・食い違わずに作れるだけの厳密さで定義する。Unity の環境が重いため、**実装より先に仕様を厳密化**し、仕様 → 計算例(テストベクタ)→ Unity 不要の純 C# での検証 → Unity、の順に進める(第 1 部 §13、付録 B)。
- **範囲**: ゲーム規則(育成・レース・世界・スカウト・イベント・特別招聘)、データ、画面、保存と技術、検算と受け入れの基準。美術・音・価格などの商品設計は含まない(未決は付録 C)。
- **現在地**: 仕様は一巡し、M0(Domain の C# 実装)は完了、M1 準備(画面向けの読み取り API と App 層)まで実装・検証済み。次は Unity 上での 1 年度の縦切り(M1)。

2. ゲームの概要

一行説明: 新任監督となり、卒業と入学を繰り返しながら大学を育て、**出雲・全日本・箱根に当たる三大駅伝をすべて制する。**制覇後も王朝を築き続けられる。

項目	内容
立場	監督モードのみ(選手一人称モードは作らない)
進行	任期なし。卒業・入学で世代交代しながら無期限に続く。解任なし
クリア	三大駅伝それぞれでの**優勝経験**(別の年度でも可)。以後も続行できる。同一年度の三冠・連覇などは上位目標
難易度	**開始校**で決まる(新規ゲームで選ぶ。既定は中堅)
対象	Steam + スマホ。横画面基準。タッチ / マウス / パッド共通の入力
収益	本体は買い切り。広告なし。**特別招聘券は永続解放(DLC・非消費型)**で有料提供(確率課金・消費型は採らない)
表現	2D。ドット絵の SD(ちびキャラ)+ パーツ合成。大学名・選手名は架空
プレイ時間	1 年度 ≒ 30 分(年度末を加えて約 32 分)、4 年 ≒ 2 時間
エンジン	Unity 6 LTS(6000.3.x)、UI は UI Toolkit が第一候補(M1 で検証)

開始校(難易度)

開始校	状況(戦力順位 / 36 校)	目安	クリアまでの年度(中央値)	頭打ち後の優勝率(1 大会)
強豪	優勝候補、層が厚い(3 位)	Easy	約 7	約 0.57
中堅	シード圏、エース不在(7 位)	Normal	約 12.5	約 0.51
弱小	予選会から(20 位)	Hard	約 23	約 0.46
同好会	部員 5 人、設備なし(33 位)	Very Hard	約 35.5	約 0.14

(検算値。方針は『おまかせ』、特別招聘券はゲーム内の獲得のみ。第 1 部 §12、第 5 部 §14)

デザインピラー

#	ピラー	具体化
1	育てる快感	練習を選ぶ → 経験値 → 能力アップ → 特殊能力・イベント。成長の可視化
2	「鍛える or 休ませる」のジレンマ	疲労・故障・調子(ピーキング)が育成の核
3	層の厚さと配置の妙	3 大会は区間数・距離・地形が違う(6 / 8 / 10 区間)。エース 1 人では勝てない
4	襷のドラマ	ブレーキ、補欠の覚醒、4 年生最後の駅伝をイベントで描く

#	ピラー	具体化
5	世代をつなぐ	卒業・入学・OB。黄金世代の後には谷間が来る。次の世代を育て続けることが終わりのない遊びになる
6	制覇は通過点	クリアの先に、連覇・同一年度の三冠・歴代記録という長期目標がある

3. ゲームの構造

時間軸: 解決の単位は **1週 = 1ターン**、1か月 = 4週、**1年度 = 48週**(4月第1週〜3月第4週)。プレイヤーの操作は週ごとではなく、**月単位の計画 + 割り込み対応**(1年度30分に収めるため)。

```
[年度初め 4月: 新入部員・主将・年間方針]
   |
   ▼
月ループ ― ① 月間計画(グループ別にテーマを選ぶ → 4週分のメニューを自動生成)
   |       ② 4週を自動解決(練習 → 疲労・経験値・故障 → 割り込み判定)
   |          └ 割り込み: 故障 / 疲労警告 / イベント / 記録会 / 大会週
   |       ③ 月末レポート(成長ハイライト・コンディション)
   ▼
[3月: 卒業・進路・スカウト確定] → 次年度へ(無期限に継続)
```

年間カレンダー(基準)

月	時期の性格	主な出来事
4月	新体制・基礎期	新入部員、主将指名、年間方針
5〜6月	春季記録会	能力の実測。スカウトの視察が始まる(6〜11月)
7月	強化期	走り込み
8月	**夏合宿**	1か月で最も伸びる(経験値 ×1.5。ただし故障率 ×1.3)
9月	調整期	第一戦のエントリー
10月2週	★第一戦(出雲型)	6区間 / 45.1 km
10月3週	第三戦の予選会	シード外校のみ。20 kmを同時に走る個人レース
11月1週	★第二戦(全日本型)	8区間 / 106.8 km
11〜12月	第三戦に向けた強化期	長距離 + 山対策。12月にエントリー
1月1週	★第三戦(箱根型)	10区間 / 217.1 km(往路5・復路5)
1月下旬〜2月	オフ	回復、補強、次年度計画。スカウトの勧誘(12〜2月)
3月	年度末	勧誘の確定、卒業、進路、入学者の発表

操作の階層

階層	操作	頻度
① 年間方針	重点(バランス / スピード寄り / ロング寄り)と目標大会	年1
② 月間計画	部員をA主力 / B準主力 / C育成の3グループに分け、**グループごとにテーマ**を選ぶ	月1
③ 個別指導	指導ポイント(月4)で、特定の選手のメニュー指定・面談	月1
④ 割り込み	故障・疲労の危険域・イベント・大会週など、必要なときだけ介入	随時

進行は『続ける』ただ1つの操作で前に進み、入力が必要な場所(止まる場所)でだけ止まる(第8部 §6)。

4. 主要な仕組みの要約

4.1 選手(第2・3・10部)

- **能力10種**(0〜100、表示は数値 + ランクG〜S): スピード SPD・スタミナ STA・ペース維持 PAC・スパート KCK・登坂 CLB・下り DWN・安定感 CON・勝負強さ MEN・回復力 REC・頑丈さ DUR。

- **状態**: 疲労(0〜100)・調子(絶不調 −2 〜 絶好調 +2)・やる気(1〜5)・故障(軽傷 1〜2 週 / 中傷 4〜8 週 / 重傷 12〜24 週)。
- **隠しパラメータ**: 素質(能力ごとの上限)・成長タイプ(早熟 / 標準 / 晩成。学年ごとの伸びが変わる)。スカウトの精度や育成の中で徐々に分かる。
- **個性(全員)**: 型(スピード / バランス / ロング)・走法(集団走◎ / 普通 / 単独走◎)・展開(追い上げ / 普通 / 逃げ)。練習の伸び方・絆の効き・レースの味付けに効き、『この子にはこのテーマが合う』を相性の印(◎○△)で見せる。
- **素養(まれ・隠し)**: 登坂 / 下り / エース。20% の選手が 1〜2 個持ち、入部から 12 週で判明。成長 ×1.15・素質の上限 +6・特殊能力への道。
- **特殊能力**(金 / 青 / 赤): 山の神・山下りの達人・エース区間○・勝負のアンカー・駅伝男・ブレーキ癖 など。練習の累積・実績で覚醒する。
- **出身**: 全部員に出身県(47)と出身高校(架空)。留学生は出身国。県による能力の固定補正は設けない。

4.2 育成(第 3 部)

- **週メニュー = ポイント練習 2 枠 + 走行量 1 つ**。月のテーマ(基礎ビルド・スピード強化・山対策・夏合宿・回復月・カスタム)が 4 週ぶんを自動生成する。
- **経験値(XP)→ 能力 +1**: 閾値 `T = 20 + 0.6 × 現在値`。効率は 疲労・やる気・成長タイプ・施設・コーチ・絆・合宿・素質への接近で決まる。
- **疲労**: `F(次週) = F × (1 - r) + L`。F が高いほど成長効率が下がり、故障率が上がる(F≥85 で ×4)。**安全網**(F≥85。設定で 75 / 無効)が自動で軽め週に差し替える。
- **調子とピーキング**: 調子スコア = (50 − F) × 0.5 + ピーキング補正 + 乱数。大会にエントリーした選手は大会の 3 週前から**自動で調整メニュー**になり、疲労が抜けて好調以上になりやすい。
- **月末の予測**: 乱数を除いた週の解決で、疲労・能力・経験値は正確に、故障と調子は確率で先に見せる(第 8 部 R-05)。

4.3 運営と世代交代(第 5 部)

- **名声**(0〜1000)・**資金**・**施設**(Lv1〜5)・**スタッフ**・**監督ランク**・**士気**。年度評価ポイント(優勝 10 / 2 位 6 / 3 位 4 / 4〜6 位 2 / 7〜10 位 1、三冠 +30)の累積で監督ランクが上がり、施設の上限・スタッフ枠・スカウト枠が増える。
- **世代交代**: 毎年 3 月に卒業(進路: 実業団 / 教員 / 一般就職 / 院進学)、4 月に入学。卒業生は **OB** として軽量データで保存され、コーチ・スカウト協力・寄付・スポンサーとして効く。県ごとの OB 数が**県別 OB ネットワーク**(Lv0〜5)を作る。
- **歴史**: 歴代優勝校・区間記録・大会記録・OB 殿堂。記録更新と殿堂入りが長期目標になる。

4.4 スカウト(第 5 部)

- 6〜11 月に視察、12〜2 月に勧誘、3 月に確定、4 月に入部。スカウトポイントは 6〜2 月に毎月 +4(年度内は繰り越し)。
- 候補の見える情報(持ちタイム・評価★・市場評価・競合度)と隠れた情報(素質・成長タイプ・故障しやすさ)。隠れた情報は段階的に判明する。
- **勧誘の確定は確率ではなく競争**: 市場評価 V の高い順に、定員 10 と奨学金枠の範囲で確定する。狙う人数の推奨は 8 人。画面には競合度と枠のメーターを出す。

4.5 駅伝本番(第 4 部)

	第一戦(出雲型)	第二戦(全日本型)	第三戦(箱根型)
時期	10 月 2 週	11 月 1 週	1 月 1 週(往路・復路)
区間 / 距離	6 区間 / 45.1 km	8 区間 / 106.8 km	10 区間 / 217.1 km
登録 / 出走	10 / 6	14 / 8	16 / 10(補欠 6)
当日変更(最大)/ 襷の上限	1 / 2	2 / 3	4 / 4
性格	スピード勝負	バランス、アンカー勝負	スタミナ + 山(5 区上り・6 区下り)
出場権	前年上位校	全校(簡略)	前年 10 位以内(シード)+ 予選会の通過 13 校(計 23 校)

- **本番までの流れ**: エントリー(締切は大会の 4 週前)→ 区間オーダー(区間適性 ◎○△ と予想タイム)→ 当日(調子・気温・風・雨が判明 → 当日変更 → 区間ごとの指示)→ レース進行(中継所ごとの順位・見どころ)→ 結果(評価ポイント・名声・選手の経験)。

- **区間タイム**: 区間適性値 P(能力の加重平均)から基準ペースを出し、調子・環境・ノイズ・展開・指示の補正を掛ける。ブレーキ(確率は CON と MEN で下がる)・失速(スタミナが足りないほど終盤に起きる)・好走がある。
- **監督の介入**: 指示(攻める / 維持 / 抑える)は『余裕の目安 ◎○△』を見て決める判断で、いつも攻めるのは得にならない。当日変更は不調以下の選手を補欠と入れ替える。檄はブレーキ・失速の損失を 2 割減らす(やる気と絆が高い走者ほど効く)。
- **観戦モード**: フル / ダイジェスト / 結果のみ。おまかせ(ダイジェスト・結果のみ)でも十分に戦え、手動で詰めるほど有利になる(上級の方針で優勝率 +0.15〜+0.17)。
- **故障と出走**: 登録は重傷だけ不可、配置は中傷以上が不可(軽傷は調子 −2 で出られる)。出走人数に足りなければ棄権(名声・評価ポイントは 0)。

4.6 世界シミュレーション(第 5 部)

- 36 校すべてが選手名簿と能力を持ち、**自校と同じレース式**で処理される。他校の育成は簡略化(校格を軸に成長・入学・卒業)。王朝の興亡があり、強豪は続きやすいが永続しない。
- **長期安定**: 100 年度以上続けても分布が発散しない(300 年度で確認)。入学時の能力分布は固定でインフレしない。セーブは年数に比例して膨らまない(刈り込み。100 年度で約 285 KB)。

4.7 イベント(第 6 部)

- 発生条件(条件式 DSL)→ 本文 → 選択肢 → 効果(能力・やる気・絆・特殊能力・資金・補正)のデータ駆動。**カタログは 52 件**(手書き 34 + 覚醒 18)。1 年度あたり約 13 件 + 覚醒約 5 件、すべて既定の選択肢があり『おまかせ』で自動解決できる。
- 効果は小さく(練習による成長の約 1%)、物語より小さな判断と手触りを足すもの。ブレーキからの再起・直前の軽傷・4 年生の最後の襷・県人会・OB の役割(コーチ・スカウト協力・スポンサー)など。

4.8 特別招聘と追加コンテンツ(第 7 部)

- **特別招聘券**を 1 枚使うと、**翌年度に留学生またはスーパールーキーが 1 名確定入学**する。物語や専用イベントはなく、確定するのは勧誘判定だけ。上限: 年 1 回・所持 2 枚・同時在籍 2 名・能力帯。
- 券はゲーム内の節目でも獲得できる(45 年度で 14 枚)。**黄金世代プラン**(永続解放。4 年度に 2 枚、失効なし)で有料の獲得手段が増える。購入しなくても全実績・クリアに到達できる。
- **公平性は実績で分ける**:『弱小・無招聘クリア』など、券を一度も使わない挑戦を別に評価する(購入状態は実績の判定に使わない)。
- 購入状態(True / False / 不明。不明は 30 日の猶予)と時計の巻き戻し対策、追加コンテンツ(見た目パック・シナリオ『古豪復活』)を定義。

4.9 画面と進行(第 8 部)

- **画面 42**(通常 22 / 進行 12 / 重なり 8)。入力モデルは『選択 → 決定』で、ホバー依存なし。長押し / X / F1 で説明。
- **進行制御**:『続ける』だけが前に進む。止まる場所は 年度初め・月間計画・エントリー・オーダー・当日・イベント・故障・疲労警告・月末レポート・年度末 など。
- **画面のための規則 R-01〜R-17**(能力のランク・推奨プラン・月末の予測・個別指導・スカウトの競合度と枠・前評判・月末レポート・効果の要約・性格タグ・個性の表示)。
- 画面は**表示して操作するだけ**で、規則は Domain にある。画面向けの読み取り API(`Views`)が状態を変えずに表示用の値を返す(第 8 部 08b)。文字列は文字列テーブルのキーで返す。

4.10 セーブと技術(第 9 部)

- **セーブ**: コンテナ `EKSV`(ヘッダ JSON + gzip の本体 + CRC32)、3 スロット × 自動 2 世代(A / B)+ 手動 1、原子的な書き込み、**中断セーブ**(週の途中・レース進行中も再開できる)、バージョニングとマイグレーション、読み込み時の**検証と修復**(I-01〜I-21)。
- **決定性**: 乱数は鍵つき PCG32(`derive(master_seed, label, a, b)`、状態を持たない)。時刻・ファイルを Domain が知らない。同じ入力は同じ結果、保存 → 読み込み → 続行は続行のみとバイト一致。
- **クラウド同期**:『どちらが変わったか』で判定し、削除しない。競合は退避して両方残す。購入状態は端末の内部状態として同期しない。

- **性能とスレッド**: 1 回の Domain 呼び出し 100 ms 以内、状態の取り出し 30 ms 以内などの予算(第 9 部 §14)。
- **構成**: Domain(純 C#・`UnityEngine` を参照しない)/ Persistence / Platform(ストアとの境界)/ App(`GameHost`)/ Presentation(UI)。マスタは `TextAsset`(JSON)。

5. 仕様書の体系

部	文書	内容	主な前提
1	基本システム設計書	コンセプト・全体構造・各システムの概要・UI 方針・技術設計・ロードマップ・決定事項	—
2	データ仕様	列挙・マスタ・実行時エンティティ・不変条件・セーブ構造・実績・シナリオ・端末の設定	1
3	育成ルール仕様	週メニュー・週の解決(XP・疲労・故障・調子・成長)・月次処理・個別指導	2
4	駅伝本番仕様	出場権・予選会・登録・オーダー・環境・区間タイム・順位・監督の介入・大会後の処理	2・3・5
5	世界・スカウト・名声・資金・施設	名声・資金・施設・候補生成・スカウト・勧誘の確定・他校の更新・入学卒業 OB・長期バランス	2〜4
6	イベント仕様	条件式 DSL・効果・補正・OB の役割・特殊能力の覚醒・カウンター・初期イベント	2〜5
7	特別招聘・追加コンテンツ	券・黄金世代プラン・招聘選手の生成・購入状態・追加コンテンツ・シナリオ	1・2・5・6
8	画面仕様	画面 42・入力モデル・進行制御・共通部品・画面のための規則・カタログ・読み取り API	1〜7
9	セーブ・技術仕様	コンテナ・世代・移行・検証と修復・乱数の鍵・同期・購入状態の時計対策・性能・構成・テスト・M0 の着手条件	1〜8
10	個性・素養仕様	型・走法・展開・素養の定義・生成・練習とレースへの影響・見せ方	1〜9(後から追加)

作成順: 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10(前の文書の数値・用語を後の文書が参照する)。10 は一巡のあとに『素養のない大多数の選手に育成の判断が無い』という指摘から足し、02〜09 へ反映済み。

6. 検証の方法と受け入れ基準

進め方: 仕様書 → **Python の参照モデル**で数値を検算 → 計算例(テストベクタ)を文書へ → **Unity 不要の C#**(`m0/`)で再現し自動テスト化 → 最後に Unity へ載せる。数値を変えるときの順序は『参照モデル → 仕様書 → C#(+ ゴールデンデータ)』。C# だけ変えて仕様と食い違わせない。

テストの層(第 9 部 §16)

層	内容
T1 ベクタ	各文書のテストベクタを C# に写した値の一致(完全一致または桁の指定)
T2 性質・ファズ	コンテナの切り詰め・ビット反転・電源断、移行、検証と修復、同期、`Q6`、整形
T3 決定性	同じ入力 2 回のバイト一致 / 任意の年度で保存 → 読み込み → 続行 = 続行のみ / 中断 → 再開の一致
T4 統計	世界シミュレーションが参照の帯に入る(3 開始校 × 120 シード × 45 年度)。T4c は実体の自校
T5 性能	各処理の予算(`bench`)
T6 画面 / T7 実機	画面の到達性・目視 / ストア API・フォント・性能(M1・M5)

壊してはいけない約束

1. 再現性の目印: リファクタの前後で、決定性テストの先頭 8 件の SHA-256(抽象の自校)が完全一致する。保存の形式や規則を**意図して**変えたときだけ更新し、理由を記録する。
2. 決定性の規則 D-1〜D-12(鍵つきストリームだけを使う・時刻を読まない・走査は ID 昇順・安定な整列・丸めは `floor(x+0.5)`・保存する `double` は `Q6`・浮動小数の和は仕様の順 など)。
3. 何も操作しない進行制御 =『おまかせ』の自動進行(バイト一致)。
4. 何も変えない当日 = 従来の『おまかせ』の当日。見込み = 実際の結果。
5. 保存 → 読み込み → 続行 = 続行のみ(バイト一致)。

7. 実装と検証の現況(2026-10-29 時点)

区分	状態
仕様(S)	01〜10 が一巡。仕様完了の基準(付録 B)を全文書が満たす(第 9 部 §17)
M0(Domain の C# 実装)	**段階 0〜10 完了**。参照モデルとの差分テスト(週の解決 5,500 件・レース 450 件・イベントの発生と効果・永続化 など)がすべて一致
M1 準備	実体の自校 + 世界(`GameSession`)・手動操作の入口・スカウト・進行制御(`GameFlow`)・区間オーダーと当日(`RaceDay`)・中断セーブ・画面向けの読み取り API(`Views`)・App 層の入口(`GameHost`)・実績・通知のログ・読み込み時の検証と修復
自動テスト	**685 件**(Domain 508 + Persistence 137 + App 40)がすべて合格
Unity	ゲート A(Domain / Persistence がエラー 0)・ゲート B(IL2CPP の Windows ビルドで T1 全行一致・T3 一致。2026-10-28)に合格
画面(UI)	未着手(M1)

統計の受け入れ(T4c: 実体の自校。120 シード × 45 年度。第 9 部 §16.3)

開始校	優勝率(参照の帯)	三冠率(参照の帯)	クリア年度の平均(参照の帯)
強豪	0.576(0.590 ± 0.044)	0.243(0.305 ± 0.053。下側は 0.03 まで許す)	8.76(6.35 ± 1.07。許容は帯 ± 1.5 年度)
中堅	0.530(0.553 ± 0.058)	0.202(0.275 ± 0.056)	14.80(12.99 ± 2.21)
弱小	0.461(0.441 ± 0.071)	0.144(0.185 ± 0.052)	23.66(25.48 ± 3.44)

- 第三戦は **23 校**(シード 10 + 予選会の通過 13)。モデルとした大会の出場枠の拡大(単独校 20 → 23 校)に合わせた(L-67)。帯は参照モデルで取り直した(上の表の括弧)。
- 新規ゲームの区間記録は、年度 1 の前に実在の水準の『初期の区間記録』へ置き換える(保持者 = 歴代の記録。L-68)。自校の走者の記録更新(`race.record`)が年度 1 から起こりうるようになるが、T4c への影響は誤差の内(優勝率 −0.002〜−0.005・三冠率 −0.003〜−0.009・クリア −0.11〜+0.09 年度)。
- 実体の自校は、故障・疲労・調子のばらつきのぶん抽象の自校より少し弱い(クリア +1〜2 年度。三冠率は 0.03〜0.06 低く、T4c は下側に 0.03 の余裕を見る)。T4c の緩和は『人が操作すれば改善する』前提(上級の方針で優勝率 +0.15〜+0.16)。
- 予選会に出た 12 名の疲労と予選会の週のピーキング(L-53 ①②)、スカウトを画面のボタンで行う方針(L-58。検算用の方針と同等)も、この条件で確かめた。操作の拒否の理由は文字列テーブルのキーで返す(L-63 ③)。中断セーブで戻らないもの(L-62 ①②)は、通常の読み込みと同じ性質なので受け入れた。
- **性能(T5)**: 世界の 1 年度の更新 約 8 ms(予算 100 ms)、1 レース 0.07 ms、週の解決 1.9 µs/件、状態の取り出し(100 年度)約 5 ms(予算 30 ms)。GC『0』は未達(実測を基準に退行を止めている。L-46)。

8. 決定済みの主要事項

項目	決定	出典
立場・進行	監督モードのみ。任期なし・解任なし	第 1 部 §0
クリア	三大駅伝それぞれでの優勝経験	第 1 部 §3.3
操作の単位	月間計画 + 割り込み(週ごとの入力は求めない)	第 1 部 §5.1
収益	買い切り + 特別招聘券は永続解放。確率課金・消費型・広告なし	第 1 部 §6.5、U-09
券の配布	4 年度に 2 枚・失効なし(繰り延べ)・所持 2・同時在籍 2・年 1 回	第 7 部 §3・§5、U-12
公平性	券の効果は制約にせず、実績『無招聘』で分ける	第 1 部 §6.5
乱数・決定性	鍵つき PCG32。端末をまたぐビット一致は求めない(環境内の一致を見る)	第 9 部 §8
セーブ	3 スロット、自動 2 世代 + 手動 1、中断セーブ。ストアごとに購入は独立	第 9 部 §3・§4
言語	v1 は日本語のみ(文字列はテーブル化)	第 8 部 §13.2、U-04
画面向き	横画面のみ(縦は将来)	U-15

項目	決定	出典
UI	Unity 6 LTS・UI Toolkit(M1 で基準を確かめ、満たせなければ uGUI)	U-17
予選会	当日の画面(当日変更・指示・檄)は作らない。結果の画面に通過ラインとの差を出す	L-61
第三戦の出場校	23 校 = シード 10 + 予選会の通過 13(26 校が走る)。学生連合は作らない	L-67
新規ゲームの区間記録	年度 1 の前に実在の水準の初期値へ置き換える(保持者 = 歴代の記録)。初年度に消える区間が出るのは受け入れる。大会記録(総合)は予熱のまま	L-68
実績	23 件。本体で判定・記録し、ストアへは通知(ミラー)	第 1 部 §3.3、U-14

9. 残課題と今後

残っている課題(付録 D が正)

課題	内容	見通し
L-60 ①	檄の先行使用(区間の前に使うと時間 −0.25% × f_k)	観戦の見え方を見てから決める
L-66	移行(`SaveMigrator`)の接続 / クラウド同期と購入状態 / 書き込みのスレッド	最初に版を上げるとき / Unity の層(M5)
L-53 ⑤	他校の先発の出身高校(`rival_same_hs`)が近似	必要になったとき
L-52・L-54・L-55	高校の強さの表 / 実体の自校が抽象より少し弱い / 資金の使い道(L-05)・型に合わせた育成の自動化(L-24)	必要になったとき
未決(付録 C)	本体の価格、美術の調達方法、実名化、ストア別の実機検証	M1 以降

ロードマップ(第 1 部 §13)

マイルストーン	内容	状態
S	仕様の確定	完了
M0	Domain の C# 実装 + テスト + SimCli(Unity 不要)	完了
M1	1 年度の縦切りを Unity 上で(月間計画 → 3 大会 → 結果)。UI Toolkit の技術検証を含む	**次**
M2	世代交代ループ(卒業・入学・スカウト・OB・主将)+ 長期シムの安定性	—
M3	世界シミュレーション(36 校 + 予選会)+ 歴史・殿堂	—
M4	イベント / 特殊能力 / 施設・スタッフ / クリア演出・実績	—
M5	スマホ最適化・Steam 対応・演出・難易度調整	—
M6(拡張・未決)	レジェンド保存とレジェンドオーダー、共有コード、非同期の対戦(付録 D の M-01・M-02)	—

第1部 基本システム設計書

原典: `01_basic_system_design.md`　　原典の版: v1.0　　改版の経緯: 付録 G.1

> 状態: ドラフト。数値はすべて「初期値案」で、ヘッドレスシミュレーションで調整する前提。

0. 前提・決定事項

項目	決定	設計への影響
立場	監督モードのみ	選手一人称モードは作らない(§1)
進行	任期なし。卒業・入学で世代交代しながら無期限に続く	長期でも破綻しない世界シム・セーブが必須(§6.4, §8)
クリア	三大駅伝の制覇(3大会それぞれでの優勝経験。別年度でも可)でクリア。以後も続行可。同一年度の三冠は上位目標	§3.3
解任	なし	目標未達ペナルティは設けない(§3.3)
エンジン	Unity	§12
対象	Steam + スマホ。横画面基準	タッチ/マウス/パッド共通(§11)
収益	本体は買い切り(Steam・スマホとも)。**特別招聘券は「永続解放」(DLC・非消費型)で有料提供する**	広告なし。確率課金・消費型なし。券の配布周期は未決(§6.5, §14.2)
表現	2D。ドット絵のSD(ちびキャラ)+ パーツ合成	描き手がいないため、美術を最小量で成立させる設計にする(§11)
名称	大学名・選手名は架空。大会名は仮称(秋雲駅伝 / 全国大学駅伝 / 新春大学駅伝)で進め、実名化は後で判断	名称は差し替え可能なテーブルで持つ(§7.1, §12)
出身県	全部員に出身県(47都道府県)と出身高校(架空)を設定。留学生は出身国	愛着の軸。スカウト・同郷・県別OBに効く(§4.5, §6.1, §6.4)
開発方針	**仕様を先に厳密化する**。Unityの実装は後回し	仕様→計算例(テストベクタ)→純C#で検証→Unityへ(§13, `00_spec_index.md`)
プレイ時間	1年度 ≒ 30分 / 4年 ≒ 2時間	**月間計画方式**(§3.4, §5.1)

---

1. コンセプト

一行説明 新任監督となり、卒業と入学を繰り返しながら大学を育て、**出雲・全日本・箱根の三大駅伝をすべて制する**。制覇後も王朝を築き続けられる。

デザインピラー

#	ピラー	具体化
1	育てる快感(パワプロ)	練習を選ぶ → 経験値 → 能力アップ → 特殊能力・イベント。成長の可視化
2	「鍛える or 休ませる」のジレンマ	疲労・故障・調子(ピーキング)が育成の核。やりすぎれば壊れ、休ませすぎればキレが落ちる
3	層の厚さと配置の妙	3大会は区間数・距離・地形が違う(6/8/10区間)。エース1人では勝てない
4	襷のドラマ	ブレーキ、補欠の覚醒、4年生最後の駅伝。数字の裏にある物語をイベントで描く
5	世代をつなぐ	卒業・入学・OB。黄金世代の後には谷間が来る。「次の世代を育て続ける」ことが終わりのない遊びになる
6	制覇は通過点	三大駅伝制覇でクリア。その先に、連覇・同一年度の三冠・歴代記録という長期目標がある

プレイヤーの立場: 監督 駅伝はチーム競技なので、監督としてチームを育てる。パワプロのサクセス的な個人育成は**選手個人レベルの成長・イベント**で担保する。

---

2. パワプロとの対応表(設計の拠り所)

パワプロ	本作
練習コマンド	月間計画(テーマ選択)+ 週メニュー(ポイント練習2枠 + 走行量)(§5)
経験点 → 能力アップ	経験値(XP)→ 能力+1。素質(上限)で頭打ち
調子・やる気	調子(5段階)・やる気(5段階)+ **疲労**(本作の核)
特殊能力(金/青/赤)	特殊能力(山の神、ブレーキ癖 など)
友人・彼女イベント	仲間・ライバル・先輩後輩・家族イベント
試合・采配	駅伝本番(区間オーダー・当日変更・走行中の指示)
栄冠ナイン(スカウト/予算)	スカウト・施設・スタッフ・部運営・**世代交代**
ペナント / ドラフト	三大駅伝 / 卒業後の進路
評価点・ランク	年度評価ポイント・監督ランク・実績
強くてニューゲーム	別の開始校で再挑戦(引き継ぎ要素は拡張)

3. ゲーム全体構造

3.1 時間軸と操作単位

- 内部の解決単位は **1週間 = 1ターン / 1ヶ月 = 4週 / 1年度 = 48ターン(4月第1週〜3月第4週)**。
- プレイヤーの操作は **月単位の計画 + 割り込み対応**(§5.1)。週ごとの入力は求めない。
- **任期なし。年度を無期限に進められる。** 開始時点で各学年に部員が在籍している。
- 年度は4月始まり。**出雲・全日本・箱根は同一年度に属する**(箱根は翌1月)。
- 毎年3月に卒業、4月に入学(§6.4)。

```
［年度初め 4月: 新入部員・主将・年間方針］
   │
   ▼
 月ループ ― ① 月間計画(グループ別にテーマを選ぶ → 4週分のメニューを自動生成)
   │        ② 4週を自動解決(練習 → 疲労・XP・故障 → 割り込み判定)
   │           └ 割り込み: 故障 / 疲労警告 / イベント / 記録会 / 大会週
   │        ③ 月末レポート(成長ハイライト・コンディション)
   ▼
［3月: 卒業・進路・スカウト確定］→ 次年度へ(無期限に継続)
```

3.2 年間カレンダー(基準)

月	時期の性格	主なイベント
4月	新体制・基礎期	新入部員、主将指名、年間方針、春のトラックシーズン開始
5〜6月	春季記録会	5000m/10000m記録会(能力の実測)。スカウト視察開始
7月	強化期	走り込み
8月	**夏合宿**	**1か月で最も伸びる**(XP ×1.5。通常月の約1.6倍。ただし故障率 ×1.3 で、故障確率は約2.2倍)
9月	調整期①	第一戦のエントリー、日本インカレ(任意)
10月2週	★第一戦(出雲型)	6区間 / 45.1km
10月3週	箱根型大会の予選会	シード外校のみ。ハーフ形式
11月1週	★第二戦(全日本型)	8区間 / 106.8km
11〜12月	第三戦に向けた強化期	長距離+山対策。12月2週に16名エントリー
1月1週	★第三戦(箱根型)	10区間 / 217.1km(往路5・復路5)
1月下旬〜2月	オフ	回復、補強、次年度計画

月	時期の性格	主なイベント
3月	年度末	卒業、進路決定、入部確定

3.3 クリアと目標

- **クリア = 三大駅伝のすべてで優勝経験を持つ(通算制覇。別の年度でも可)**。達成時にクリア演出(エンディング)を流し、「続ける」を選べば同じセーブで継続できる。
- **上位目標(クリア後の長期目標)**: 同一年度の**三冠** / 連覇 / 連続三冠 / 歴代記録の更新 / OB殿堂の充実。
- **年度評価ポイント**: 優勝10 / 2位6 / 3位4 / 4〜6位2 / 7〜10位1。同一年度の三冠 +30。
- **監督ランク**: 評価ポイントの累積でランクが上がり、スタッフ枠・施設の上限・スカウト枠が拡張される(長期の成長曲線)。
- **解任なし**: 大学からの期待目標は達成すると予算ボーナスが付くだけ。未達でもペナルティはない。
- **実績(Steam 実績を搭載。スマホは Game Center / Google Play Games に同じ内容を対応づける)**
  - 実績は**アカウント単位**で、セーブとは別に保存する(データは 02 §3.10)。解除は本体で判定し、ストアへは通知だけを送る。
  - **購入状態は実績に影響しない**。特別招聘券を使ったかどうかだけで、実績の条件が変わる(「無招聘」)。
  - 一覧は初期案(条件はすべて測れる形で書く)。

区分	実績	条件
大会	初シード	第三戦(箱根型)で10位以内
	初表彰台	いずれかの大会で3位以内
	初優勝	いずれかの大会で優勝
	出雲を制す / 全日本を制す / 箱根を制す	各大会で初優勝(3つ)
	連覇	同じ大会を2年連続で優勝
	王朝	いずれかの大会を5年連続で優勝
クリア	**三大駅伝制覇**	**3大会すべてで優勝経験(クリア)**
	強豪クリア / 中堅クリア / 弱小クリア / 同好会クリア	開始校がそれぞれ強豪 / 中堅 / 弱小 / 同好会で、クリア(4つ)
	弱小・無招聘クリア	**開始校が弱小で、クリア時点で特別招聘券を一度も使っていない**
	同好会・無招聘クリア	開始校が同好会で、クリア時点で特別招聘券を一度も使っていない(隠し)
	三冠	同一年度に3大会すべてで優勝
	連続三冠	2年連続で三冠
育成	47都道府県	卒業生の出身県が47県すべてにそろう
	殿堂	殿堂入りの卒業生が10人
	山の神	「山の神」を持つ選手が第三戦の5区で区間賞
	区間賞100	通算の区間賞が100回
継続	25年 / 50年	年度を25 / 50年度まで続ける

- 「無招聘」は、そのセーブで特別招聘券を使った(招聘を宣言した)ことが一度もないこと。券を持っているだけ、購入しているだけでは外れない。
- **難易度 = 開始校。新規ゲームの開始時にプレイヤーが選ぶ(既定: 中堅)**。弱小・同好会は上級者向け。

開始校	状況(戦力順位 / 36校)	目安
強豪	優勝候補、層が厚い(3位)	Easy
中堅	シード圏、エース不在(7位)	Normal
弱小	予選会から(20位)	Hard
同好会	部員5人、設備なし(最下位付近。33位)	Very Hard

- 開始校の施設・スタッフ・資金の初期値は `05_world_scout_spec.md` §5。

3.4 プレイ時間バジェット

目標: 1年度 ≒ 30分(年度末を加えて 約32分)、4年度 ≒ 2時間。

項目	時間(1年度)
年度初め(入部確認・主将・年間方針)	約3分
月間計画(約12回 × 40秒)	約8分
スカウト	約4分
大会(オーダー + 観戦。出雲型2.5 / 全日本型3.5 / 箱根型6)	約12分
イベント・成長演出	約3分
年度末(結果と精算・強化・卒業・入学者発表。08 で追加)	約2分
合計	**約32分**(機械的な最小時間は約21分。08 §11.3)

- 大会は「**フル観戦 / ダイジェスト(中継所ごとの要約)/ 結果のみ**」から選べる。最短なら 1年度 12〜15分。
- **セッションの区切り**: 年度を3つの節に分ける(①春〜夏合宿 ②第一戦〜第二戦 ③第三戦に向けた強化〜年度末)。1節 約10分でスマホの隙間時間に収まる。どこでもオートセーブ。
- **帰結**: 週単位で毎回入力する設計はこの時間に収まらない。そのため**月間計画方式**を採る(§5.1)。

---

4. 選手システム

4.1 能力(0〜100、表示は数値+ランク G/F/E/D/C/B/A/S)

分類	略称	名称	意味
走力	SPD	スピード	短〜中距離の速さ。短区間・トラック
走力	STA	スタミナ	長距離の持久力。長区間の核
走力	PAC	ペース維持	後半の失速しにくさ(粘り)
走力	KCK	スパート	ラストの切れ。接戦・アンカーで効く
走力	CLB	登坂	上り適性(山登り区間)
走力	DWN	下り	下り適性(山下り区間)
精神	CON	安定感	走りのブレにくさ。ブレーキ率に直結
精神	MEN	勝負強さ	大舞台・重圧への強さ
体質	REC	回復力	疲労の抜けやすさ
体質	DUR	頑丈さ	故障しにくさ

- 入部時の目安: 平均 40〜55、スカウト上位 60〜70、怪物級 75+。卒業時のエースは 85〜95。
- **記録(PB)は能力から導出**して表示する(後述の同一ペース式を使う)。10000m 27:22 ≒ 能力90相当。

4.2 状態

状態	範囲	説明
疲労 F	0〜100	週ごとに蓄積・減衰。成長効率と故障率に影響(§5.3)
調子	絶好調 / 好調 / 普通 / 不調 / 絶不調	疲労・調整・安定感から週ごとに決定(§5.4)。レースに直結
やる気	5段階	練習効率 ×0.85〜1.15。低いと退部リスク。起用・面談・勝敗で変動
故障	なし / 軽傷 / 中傷 / 重傷	軽傷1〜2週、中傷4〜8週、重傷(疲労骨折)12〜24週

4.3 素質と成長タイプ(隠しパラメータ)

- **素質(能力ごとの上限)**: 見えない。スカウト精度や育成の中で徐々に判明する。→「原石を見抜く」楽しさ。

- **成長タイプ**で学年ごとの伸びが変わる。

学年	早熟	標準	晩成
1年	×1.3	×1.0	×0.7
2年	×1.1	×1.1	×0.9
3年	×0.8	×1.0	×1.2
4年	×0.6	×0.9	×1.3

4.3a 個性と素養(10)

- **個性**(全員・見える): **型**(スピード型 / バランス型 / ロング型)・**走法**(集団走◎ / 普通 / 単独走◎)・**展開**(追い上げ型 / 普通 / 逃げ型)。練習での能力の伸び方(型・展開は得意の能力が +20%・苦手が −20%)、仲間との絆の効き方(集団走◎は 2 倍・単独走◎は 1/3)、個別指導の効き(単独走◎は ×1.25)に影響し、レースでは集団走・単独走・追う展開・先頭走の補正に味付けとして効く。**『この子にはこのテーマが合う』を相性の印(◎〇△)で見せる。**
- **素養**(まれ・隠し): 登坂 / 下り / エース。20% の選手が 1〜2 個持ち、入部から 12 週で判明する。該当する能力が伸びやすく(×1.15)素質の上限も高い。**特殊能力(山の神・山下りの達人・エース区間〇)への道**になる。素養のない選手は、標準の練習ではこれらにほとんど届かない。
- 個性・素養は**平均して中立**(チーム全体の強さの期待値は +0.1〜+0.3 P)。育成の判断の**数と手がかり**を増やすもので、判断の**価値**を大きくするものではない(登録 16 人から最適に選ぶため、個別の最適化の上限は +0.2〜+0.4 P)。詳細・検算は `10_individuality_spec.md`。

4.4 特殊能力

系統	例(正)	例(負)
区間	山の神(山登り区間で登坂+)、山下りの達人、エース区間〇、アンカー〇	—
展開	ごぼう抜き(追う展開で+)、逃げ切り、集団走〇	単独走×、ハイペース×
調整	調整力〇(ピーキングしやすい)、夏男	夏バテ、調整下手
精神	大舞台〇、駅伝男、リベンジ(前回ブレーキ後に+)	本番弱い、ブレーキ癖
体質	鉄の脚(故障率-)、回復力〇	故障癖

- 取得: 練習の累積・イベント・レース実績。強力なものは「金」ランクとして条件を厳しくする。
- 名称は仮。「山の神」「花の2区」など通称由来のものは、商用化前に名称を要確認(§14)。

4.5 出身(出身県・出身高校・留学生)

- 全部員に**出身県(47都道府県)**と**出身高校(架空)**を設定する。選手カード・名簿・イベント・OB名簿に常に表示し、「どこから来た選手か」で愛着を持たせる。
- 留学生は出身県の代わりに**出身国**を持つ(§6.5)。国による能力補正は設けない(個人差のみ)。
- **県による能力の固定補正は原則設けない**(ステレオタイプを避ける)。例外は「強豪県の候補は能力の基準値が+3」のみ。

効果	内容
供給	県ごとの候補の出やすさ(人口区分 + 高校駅伝の強豪県補正)。関東22%・中部19%・近畿17%・九州15% など(`02_data_spec.md` §3.2)
距離帯	大学所在地との地方間の距離(地元 / 隣接 / 遠方)が、勧誘成功率・帰省イベント・視察コストに影響
同郷	同じ県の部員同士は絆の初期値+10、同グループでの絆の伸び ×1.1
県別OBネットワーク	その県のOBが増えるほど、同県のスカウト情報精度・勧誘成功率が上がる(§6.4)
郷土イベント	帰省・家族の応援・県人会・ホームシック(遠方出身の1年生。同郷の先輩がいると軽減)
方言(任意)	イベント台詞を地方別に差し替える。低優先で、デフォルメしすぎない(U-06)
記録・収集	出身県マップ(在籍・OBを日本地図に塗る)、「47都道府県出身の卒業生を揃える」実績

5. 育成システム(ゲームの中心)

5.1 操作は「月間計画 + 割り込み」(§3.4のプレイ時間から逆算)

階層	操作	頻度
① 年間方針	年度初めに、重点(バランス / スピード寄り / ロング寄り。推奨プランのテンプレートを選ぶ。08 R-04)と目標大会(見出しと予測の注目大会。規則には作用しない)を決める	年1
② **月間計画**	部員を A主力 / B準主力 / C育成 の3グループに分け、**グループごとに「月のテーマ」を選ぶ**と4週分のメニューが自動生成される	月1
③ 個別指導	**指導ポイント(月4)**を使い、特定の選手のメニュー上書き・面談・ケアを行う	月1
④ 割り込み対応	故障発生・疲労の危険域・イベント・大会週など、必要な時だけ介入	随時

月のテーマ(各テーマが4週分の週メニューを持つ。基本は「3週負荷 + 1週軽め」)

テーマ	内容	狙い
基礎ビルド	走行量 多、質は軽め(ペース走・閾値走)	STA / PAC
スピード強化	インターバル・レペティション中心	SPD / KCK
山対策	坂道走・下り走	CLB / DWN
夏合宿	走行量 極、ロング走中心、XP ×1.5	総合
回復月	補強・ケア・走行量 少	疲労↓
カスタム	週ごとに自由編集(上級者向け)	—

- **ピーキングは月のテーマではなく、大会連動で自動適用**される。エントリーした選手は、大会の3週前から大会週まで調整メニューになる(第一戦と第二戦のように間隔が短い場合は後ろから詰める)。モードは 標準 / 短め / なし(03 §5.3)。
- 自動生成された週メニューは、**疲労が故障寸前(F≥85)に入ると軽め週へ自動で差し替える**(設定で 75 / 85 / 無効)。放置してもすぐ壊れない安全網。
- 月間計画画面には**月末の疲労・調子の予測**を表示し、「これで大会に間に合うか」を判断できるようにする(08 R-05。疲労・能力・経験値は正確、故障と調子は確率で出す。実測と合うことを検算済み)。

メニューの最小単位 = ポイント練習2枠 + 走行量1つ(パワプロの練習コマンドに相当)。

5.2 メニュー一覧(確定した数値は `03_training_rules_spec.md` §4)

ポイント練習

練習	主な伸び(XP)	疲労負荷	故障リスク
ペース走	PAC↑ STA↑	9	低
閾値走(テンポ走)	STA↑↑ PAC↑	10	低
ロング走(30km走)	STA↑↑↑ PAC↑ MEN↑	15	中
インターバル	SPD↑↑ STA↑	14	中
レペティション	SPD↑↑↑ KCK↑	13	高
変化走・ビルドアップ	KCK↑↑ PAC↑	10	低
坂道走	CLB↑↑↑ STA↑	12	中
下り走	DWN↑↑↑	11	高(脚への衝撃)
補強・体幹	DUR↑↑ REC↑ 全能力微	4	故障率を下げる
調整(シャープニング)	調子+ 疲労↓	4	低
休養・ケア	疲労↓↓ 故障回復	0	—
タイムトライアル	MEN↑↑ 全体+ / **実力測定**	16	中

走行量(週間距離): 少 70km(負荷2)/ 標準 110km(6)/ 多 150km(10)/ 極 180km(15)。STA・PACの基礎XPと負荷、故障率が上がる。

5.3 疲労と成長効率

```
F(次週) = F × (1 − r) + L        r = 0.40 + 0.20 × REC/100 + ケア補正
L = Σ ポイント練習の負荷 + 走行量の負荷
```

週メニュー例	L	収束する疲労 F* (REC50)
軽め(補強+調整+少)	約10	約22
標準(中×2 + 標準)	約24	約48
強化(重+中 + 多)	約33	約66
追い込み(重×2 + 極)	約43	約86(長続き不可)

疲労 F	成長効率	故障率	状態
0〜39	×1.0	×1	良好
40〜69	×1.0→0.85	×1〜1.5	蓄積(攻めどころ)
70〜84	×0.7	×2	危険
85〜	×0.3	×4	故障寸前・不調化

成長式

```
XP(能力) = Σ メニューGain × K_XP_SCALE(3.0) × 効率補正 × 素質接近補正
効率補正 = 疲労効率 × やる気 × 成長タイプ(学年) × 施設 × コーチ × 絆(グループ練習) × 合宿
能力+1の閾値 T = 20 + 0.6 × 現在値      (50→50XP, 80→68XP)
素質接近補正 = (1 − 現在値/素質上限)^0.7   (上限に近づくほど伸びにくい)
```

故障判定(週): `p = 0.5% × 疲労係数 × 負荷係数 × (1.6 - DUR/100) × 特殊能力補正 × 施設`。確定した式と数値は `03_training_rules_spec.md` §6。故障中は「アクア/バイク」代替メニューでSTAの低下を半減できる。

5.4 調子とピーキング(三冠の肝)

```
調子スコア = (50 − F) × 0.5 + ピーキング補正 + 乱数(σ = 8 × (1.5 − CON/100))
絶好調 ≥ +20 / 好調 ≥ +8 / 普通 > −8 / 不調 > −20 / 絶不調
```

- 重い練習期は不調気味(実際の「脚が重い」感)。**エントリー後は、大会の3週前から自動で調整メニューに切り替わり、疲労が抜けて**好調〜絶好調になる(標準で大会週の約8割が好調以上。調整なしだと約4割)。
- ピーキング補正: 調整期にシャープニング(短く速い練習)を入れると +8。休みすぎ(4週以上)は −8(キレ落ち)。
- **レース出走で疲労が加算**される(`F += 区間距離km × 1.6`、20kmで約+32)。第一戦(10月2週)→第二戦(11月1週)は3週しかなく、第三戦(1月1週)までは約8週。→ **「第一戦は主力+若手でスピード編成、第二戦はスタミナ型を温存、第三戦にピーク」のように、層の厚さでピークをローテーションする**のが三冠への道。

5.5 練習グループと絆

- 同じグループで同じテーマを続けると「絆」が溜まり、練習効率+(最大 ×1.15)。イベント発生源にもなる。
- 主将(最上級生から指名)は士気と絆の伸びに影響する。

---

6. チーム運営と世代交代

6.1 スカウト(栄冠ナインの年1サイクル)

- 時期: 6〜11月に視察、12〜2月に勧誘、3月に確定、4月に入部。**スカウトポイントは6〜2月に毎月 +4 付与され、同じ年度の中では繰り越す**(3月に0。08 R-08)。
- 候補の**可視情報**: 5000m持ちタイム(能力から導出。08 R-03)、評価★・市場評価、性格タグ、競合度。(全国高校駅伝の区間成績は v1 では出さない。) **隠し情報**: 素質上限、成長タイプ、故障しやすさ。スカウト担当の能力に応じて段階的に判明。
- 行動: 視察(情報+)/ 声かけ(好感度+)/ 練習会招待 / 奨学金枠の使用。月ごとのスカウトポイントを配分する。

- 勧誘の成否は**確率ではなく競争で決まる**(05 §8): 提示スコア = f(部の名声, 寮・設備, 出場機会, 好感度, **他校との競合**, **距離帯(地元志向)**, **県別OBネットワーク**)で、**市場評価 V の高い順**に、定員10と奨学金枠の範囲で確定する。狙わなくても平均3人は自然に入るので、**狙う人数の推奨は 8人**(9番目以降はほぼ入れない。08 R-08)。画面には『競合度』(他校の狙い)と『確定の順番・枠のメーター』を出す。
- **地方 → 県の順に絞る**(日本地図UI。地方を選ぶと県のタイル 3×3 に切り替わる。県を直接タップさせない = 08 §4.3)。県ごとに候補数と注目選手が表示され、遠方ほど視察コストが高く、勧誘成功率が下がる。地元の県は逆に有利。

6.2 資金・施設・スタッフ

項目	内容
資金	収入: 大学補助、OB会寄付、スポンサー、成績ボーナス。支出: 合宿・遠征、施設投資、スタッフ給与
名声	駅伝成績・歴史で上昇。スカウト・資金に影響
施設(Lv1〜5)	寮(回復力・スカウト)/ トラック(SPD・KCK効率)/ 坂道コース(CLB・DWN)/ トレーニング室(DUR)/ 医療室(故障期間短縮)
スタッフ	コーチ(練習効率)/ トレーナー(故障)/ 栄養士(回復)/ 主務(事務)/ スカウト担当
拡張枠	監督ランクで施設上限・スタッフ枠・スカウト枠が増える(§3.3)

6.3 部の雰囲気(士気)

- 補欠で出場できない部員の不満、勝利の連鎖、4年生の最後の駅伝などで変動。
- 士気が高いと絆・やる気が上がり、低いと退部者が出る。

6.4 世代交代・OB・歴史(長期プレイの軸)

- **学年構成**: 各学年 8〜12名、全体で約40名(強豪は60名規模)。毎年およそ1/4が入れ替わる。
- **世代の波**: 学年別戦力グラフで「谷間の世代」を可視化し、スカウトで穴を埋める。黄金世代の卒業は大きな穴になるため、**次の世代を育て続けること自体が終わりのない遊び**になる。
- **卒業と進路**: 実業団 / 教員 / 一般就職 / 院進学。卒業生はOBとして登録され、軽量データ(名前・最終能力・主要記録・進路)で保存する。
- **OBの影響(世代を越えた資産)**: コーチ就任(練習効率+)/ スカウト協力(隠し情報の解放)/ 寄付 / スポンサー。有名OB(区間記録保持者・代表クラス)は名声を押し上げる。
- **県別OBネットワーク**: 出身県ごとにOBの人数を数え、`level = min(5, floor(√OB数))`（OB1人でLv1、4人でLv2、9人でLv3、…25人でLv5)。レベルが高い県は、候補の情報が見えやすく、勧誘も有利になる。**「あの県から毎年のように選手が来る」関係が長期で育つ**。
- **歴史**: 歴代優勝校、区間記録、歴代個人記録、OB殿堂。記録更新が長期目標になり、名声を通じてスカウトに効く。
- **主将と伝統**: 主将は毎年指名。部の伝統・気質(走り込み型/科学的/自主性)の長期形成は拡張で検討。

6.5 特別招聘(黄金世代プラン)と課金(確定した規則・検算は `07_special_recruit_spec.md`)

コンセプト:「次の年に、黄金世代の核になる選手を確定で迎える」。**物語や専用イベントは付けない**、シンプルな仕組みにする。本体は買い切りで完結し、**券を使わなくても三大駅伝制覇(クリア)・全実績・全モードに到達できる**。

仕組み

1. 「**特別招聘券**」を1枚使い、種別を選ぶ。種別は **留学生** または **スーパールーキー**。
2. 翌年度の入学者に**確定で1名**が加わる。勧誘判定はなく、他校に奪われない。宣言はスカウト期間(6〜2月)中に行い、3月の入学者発表で姿が分かる。
3. 能力・名前・外見・出身(留学生は出身国、スーパールーキーは出身県)は自動生成。入学後は通常の選手と同じ扱い(育成・起用・卒業・OB)。スーパールーキーは**出身県を指定できる**(任意案。愛着と県別OBネットワークの育成に使える)。
4. 種別の違いは**役割分担**: 留学生は入学時から強い即戦力だが、大会の留学生枠で出走が限られる。スーパールーキーは入学時はやや低いが伸びしろが大きく、出走枠の制限がない。**国による能力補正ではない**。
5. 券は**ゲーム内の節目でも獲得できる**(各大会の初優勝・初の表彰台・シード・三冠・クリア・監督ランク・継続の記念など。45年度で14枚。条件は 07 §4)。課金は獲得手段を足すだけにする。

6. 「何年度に使うか」自体が**世代設計の判断**になる(使える回数に上限があるため)。

#	原則
1	物語・専用イベントは作らない。券の効果は「確定入学」だけ
2	育成の核(練習・采配)は飛ばさない。飛ばすのは**1年度に1名ぶんの勧誘判定だけ**。入学後は通常の選手として育てる
3	確率課金(ガチャ)は作らない。種別を選べ、結果の範囲が事前に分かる
4	上限を設ける(年1回・所持2枚・同時在籍2名・能力帯)
5	シングルプレイ完結、オンラインランキングなし(他プレイヤーとの不公平を生まない)
6	購入はストアごとに独立(Steam / Apple / Google)。v1ではストア間で共有しない

ガードレール(初期値案。`02_data_spec.md` §3.9)

- 券の使用は**年度に1回まで**、所持は**最大2枚**、招聘枠の選手の同時在籍は**最大2名**。
- 入学時の能力は各能力 ≤ 80、10能力の平均 ≤ 68。素質は高めだが上限100。
- 留学生は大会の登録・出走上限に従う(§7.1)。国・地域による能力補正は設けない。留学生を人物として描き、ステレオタイプな描写を避ける。
- 入学後は通常の選手。購入状態が変わっても(別端末・返金など)**在籍は継続**する。
- **影響の目安(概算)**: 入学時に適性値72の選手が、適性値55のレギュラー枠に入ると、チーム適性値が +1.7 で、箱根の総合タイムが約2.5分短縮される。育って85になり、適性値60の枠に入ると +2.5 で約3.6分短縮される。中堅校と強豪校の差(約29分)に対して、1名で約9〜12%、2名在籍で約17〜25%。1名で戦力差を覆すほどではない。
- **公平性は実績の条件で分ける(決定)**: オンラインランキングがなく、課金する人の多くは『クリアしたい』ことが目的なので、**券の効果の大きさは制約にしない**(15%以内などの上限は設けない)。代わりに、特別招聘券を一度も使わないことを条件にした実績(『弱小・無招聘クリア』など。§3.3)を用意し、券を使わない挑戦を別に評価する。購入状態は実績の判定に使わない(券を使ったかどうかだけ)。
- **現状の測定(参考。07 §12.2)**: 有料プラン(4年度に2枚)の追加の効果は、弱小で クリア年度の平均 25.6 → 20.7(**−19%**)、中堅はほぼ出ない(12.2 → 12.6、誤差の範囲)。**プランの価値は、弱小・同好会のような難しい開始で大きい**。

課金方式(決定・U-09): 永続解放

- 「黄金世代プラン」を一度購入すると、決められた周期で券が付与される。Steam=DLC、スマホ=非消費型。購入の復元に対応する。サーバーは不要。
- 消費型(券の単品販売)と確率課金は採らない(管理が重く、Steamでは別の仕組みが要るため)。
- 購入しなくても、ゲーム内の節目で券を獲得できる。
- 開発中はプランなし(券はゲーム内の獲得のみ)で進め、プランの解放は後から有効にする。券の在庫は共通の1つの数値で、入手元(ゲーム内の獲得 / プランの付与)だけが違う(`02_data_spec.md` §3.9)。

配布周期(確定・U-12: 4年度に2枚、失効なし。07 §3・§5): 現在の上限(同時在籍2名・年度に1回)では、券を実際に使えるのは**平均で年0.5回(4年に2回)が上限**。それより速い周期で配っても使いきれず、余りは繰り延べで貯まる一方になる(失効はさせない)。40年度でのシミュレーション結果(失効ありで数えた当初案の値。使用の回数は繰り延べでも変わらない):

周期	配布	使用	余り(当初案では失効)	使用年度の例	特徴
毎年度 1枚	40	20	18	1, 2, 5, 6, 9, 10…	配布が使用上限を超え、半分近くが余る。使う時期を選べる柔軟さはある
2年度に 1枚	20	20	0	1, 3, 5, 7…	1年おき。連続入学の「黄金世代」は貯めないと作れない
4年度に 2枚(既定)	20	20	0	1, 2, 5, 6, 9, 10…	**世代ごとに2枚。2年連続で使うと黄金世代になり、次の2年は空く**
4年度に 1枚	10	10	0	1, 5, 9…	控えめ。1世代に1名

- 配布の多さを変えたい場合は、周期ではなく**同時在籍の上限**を動かす。**周期と上限は一緒に決めた**(同時在籍2名・年1回・周期4年度に2枚)。
- 付与の起点は、プランの購入を**初めて確認した年度**。その時点ですぐ最初の付与を受ける(購入後に何ヶ月も待たせない。07 §5.2)。

- 所持上限を超える入手は**失効させず、繰り延べる**(購入したものが無駄にならない。ゲーム内の獲得も同じ。07 §3.2)。上の表の『余り』は、当初案(失効あり)で失効していた枚数。**使用の回数は繰り延べでも変わらない**。

見た目・シナリオ(勝敗に影響しない追加コンテンツ。確定は 07 §9〜§10): 見た目パック(ユニフォーム・校章・髪型・髪色・BGM・応援歌。**表示の層だけ**に作用し、生成・乱数・結果には影響しない)、シナリオパック(追加の開始校。**開始状態だけ**を定め、世界の規則は変えない。発売後の拡張)。券とは独立。

本体との関係(U-08): 他校には留学生が在籍する(実大会に即した設定)。自校の候補にも一般の留学生が低頻度で出る。券は「確定」の手段を足すだけ。

懸念: 上限を設けても、券を使うと有利になる点は残る。「勧誘判定を1名ぶん飛ばす」以上の効果は持たせない。

---

7. 駅伝本番システム

7.1 3大会のルール(実大会を簡略化した初期値案)

	第一戦(出雲型)	第二戦(全日本型)	第三戦(箱根型)
時期	10月2週	11月1週	1月1週(往路・復路)
区間	6区間 / 45.1km	8区間 / 106.8km	10区間 / 217.1km
区間距離	5.8〜10.2km	9.5〜19.7km	20.8〜23.1km
登録 / 出走	10 / 6	14 / 8	16 / 10(補欠6)
当日変更(最大)	1	2	4
性格	スピード勝負。短区間	バランス。7・8区が長くアンカー勝負	スタミナ+山。5区上り・6区下り
効く能力	SPD, KCK	全般	STA, PAC, CLB, DWN
出場権	前年上位校	全校(簡略)	前年10位以内=シード + 予選会の通過13校(計23校。L-67)

- 表の見出しは**設計上の呼称(モデルとした大会)**。ゲーム内の大会名は差し替え可能な仮称テーブルで持つ(§0, §12)。
- 区間距離は実大会準拠。出場校数・登録人数はゲーム内ルール(調整可)。世界の大学は **36校**(自校含む)を想定。

7.2 本番までの流れ

1. **エントリー**(登録メンバー決定)
2. **他校情報**: 各校の上位選手の持ちタイムと前評判
3. **区間オーダー**: 各選手の区間適性(◎○△)と予想タイムを見て配置(自動編成も用意)
4. **当日**: 調子・天候・気温・風が判明 → 当日変更(第一戦1名 / 第二戦2名 / 第三戦4名まで)。各走者の**余裕の目安(◎○△)**を見て、区間ごとに指示(攻める / 維持 / 抑える)を決める
5. **レース進行**: 中継所ごとに順位表示、ハイライト(競り合い・ブレーキ・区間新)
6. **結果**: 総合・区間賞 → 評価ポイント、名声、選手経験(大舞台経験でMEN↑・特殊能力取得チャンス)

7.3 区間タイム計算(確定した式・手順・計算例は `04_race_spec.md`)

区間適性値 P = 区間タイプ別に能力を加重平均

区間タイプ	該当	重み
短距離高速	第一戦1〜5区、第二戦1〜4区	SPD .40 / STA .20 / PAC .15 / KCK .15 / MEN .10
中距離	第二戦5〜6区	SPD .25 / STA .30 / PAC .25 / KCK .10 / MEN .10
長距離	第二戦7〜8区、第一戦6区、第三戦の平地区間	STA .35 / PAC .25 / SPD .20 / MEN .10 / KCK .10
山登り	第三戦5区	CLB .45 / STA .25 / PAC .15 / MEN .10 / SPD .05
山下り	第三戦6区	DWN .45 / SPD .20 / STA .15 / PAC .10 / MEN .10

区間タイム

```
基準ペース(秒/km) pace50(d) = 176 + 6 × ln(d / 5)        … P=50 の選手
ペース = pace50(d) × 地形係数 − 0.4 × (P − 50)
地形係数: 区間ごとに、初期の区間記録(02 §4.8)から較正した値(02 §3.5・L-69。0.96〜1.05、第三戦 5区 ×1.127 / 6区 ×0.963)
区間タイム = d × ペース × (1 + 調子補正 + 環境補正 + ノイズ) × 特殊能力補正
```

- 調子補正(タイム側): 絶好調 −1.5% / 好調 −0.7% / 普通 0 / 不調 +1.2% / 絶不調 +3.0%
- 疲労 70以上は追加で +1%。
- ノイズ: `N(0, 0.8% × (1.5 - CON/100))`
- **ブレーキ**: 確率 `2% × (1.5 - CON/100) × 重圧係数(MEN が高いほど小)` で発生し、タイム +3〜10%。好走(良い側の裏返し)も 3% で発生。
- **失速(ペース配分)**: スタミナが「その区間の適性値に対して足りない」ほど、また距離が長いほど、区間の終盤に失速する確率が上がる(失速すると +2〜4%)。バランス型より持久型が長い区間に向き、スピード型は長い区間で失速しやすい。確率は画面の**余裕の目安(◎○△)**で見える(04 §7.4)。
- 展開補正: 集団走で追走側 −0.3%、単独走 +0.2%。アンカーは接戦時に KCK を加味。
- 環境補正は選手ごとに差が出る(暑さ=STA、向かい風=PAC、雨=CON)。

検算結果(この式で全区間の P を揃えたチームの総合タイム。実大会の水準と整合)

チームP平均	45.1km(第一戦)	106.8km(第二戦)	217.1km(第三戦)
35(下位)	2:18:44	5:34:45	11:34:16
50(中堅)	2:14:14	5:24:04	11:12:34
60	2:11:13	5:16:57	10:58:05
70(強豪)	2:08:13	5:09:50	10:43:37
78(圧倒的)	2:05:48	5:04:08	10:32:02

個人の目安: 第三戦5区 P85 → 1:07:27、6区 P75 → 56:02、2区 P85 → 1:05:54、第一戦2区(5.8km)P85 → 15:45。→ **強豪の目安は「チーム平均P=70前後」、能力90台のエースは歴代級**。

7.4 監督の介入(確定した規則は `04_race_spec.md` §9)

- **走行中の指示**(区間ごと): **攻める**(時間 −0.8%、失速率 ×2.5、ブレーキ率 ×1.5)/ **維持** / **抑える**(時間 +0.6%、失速率 ×0.3、ブレーキ率 ×0.5)。余裕の目安が◎なら攻める、△なら抑えると得になる。「いつでも攻める」は得にならない(失速の代償がある)。
- **檄(運営管理車からの声かけ)**: 1レースの使用回数に上限。ブレーキや失速が起きた区間の途中で使うと、その損失が 8割になる(効果は控えめ。やる気・絆が高い走者ほど効く)。
- **当日変更**: 調子が崩れた選手、暑さ・向かい風・雨に弱い選手を補欠と入れ替える(第一戦1名 / 第二戦2名 / 第三戦4名まで)。
- 観戦モード(フル / ダイジェスト / 結果のみ)に応じて、介入の機会数を変える(§3.4)。**おまかせ**(ダイジェスト・結果のみ)でも、指示は維持、当日変更は不調以下の置き換えを自動で行い、通常は十分に戦える。手動で指示・当日変更を最適に行うと、さらに有利になる(04 §14.4)。

---

8. ライバル校・世界シミュレーション

- 36校すべてが選手名簿(主力16名)と能力を持ち、**自校と同じレース式**でレースを処理する(特別扱いなし)。主力以外は「層の厚さ」の1パラメータで簡略化する。
- 他校の育成は簡略化: 年度更新ごとに「校格(強豪/中堅/下位)」を軸に成長、入学・卒業、ランダムなスーパールーキー。
- **王朝の興亡**: 強豪は続きやすいが永続せず、新興校が台頭する。プレイヤーのスカウトは他校の入部を奪う。
- 校の気質(例): 王者型 / 山特化型 / スピード型 / 新興型 / 古豪型。
- 大学名・選手名は架空。選手名は姓名テーブルから重複を避けて自動生成する。

長期安定性(無期限プレイの必須条件)

- 100年度以上続けても、校格の分布・平均能力・記録水準が発散しないこと。校格には平均回帰をかけ、**入学時の能力分布は固定**する(時代によるインフレなし)。記録の更新は「当たり年」でだけ起こるレアな出来事にする。

- セーブは在籍部員のみフルデータ。OB・他校の過去選手は要約のみとし、**年数に比例してセーブが膨らみ続けない**ように上限を設ける。

---

9. イベントシステム

構造: 発生条件(時期・状態・関係値)→ 本文 → 選択肢 → 効果(能力・やる気・絆・特殊能力・資金・補正)。データ駆動(確定した形式は `06_event_spec.md`)。**1年度あたり イベント約13件(選択あり約7・お知らせ約6)+ 覚醒約5件、所要時間 約130秒**(§3.4 の『イベント・成長演出 約3分』のうち約2分)。すべてのイベントに既定の選択肢があり、『おまかせ』で自動解決できる。効果は小さく(練習による成長の約1%)、勝敗を左右する仕掛けではなく、判断の手触りを足すもの。

種別	例
チーム	主将の指名(毎年4月)、OBの来訪、**OBの役割**(コーチ・スカウト協力・スポンサーを1年間)、チームの停滞
個人	スランプ、夏合宿の洗礼(1年生が限界。休ませる/続行/先輩に任せる)、ホームシック、故障からの復帰、先輩の背中、家族の応援
特殊能力の**覚醒**	取得条件(練習の累積・大会の実績)を満たした週に、確率で特殊能力を覚える(山の神、駅伝男、夏男…。負の能力も)
本番	**ブレーキからの再起**(励ます/叱咤/放置)、直前の軽傷(欠場か強行か)、4年生の最後の襷、補欠の意地、祝勝会
郷土	県人会、県のOBの来訪、同郷の先輩、留学生の一時帰国・生活の不安(個人の状況として描く)、年末年始の帰省
経営・卒業	大学の期待目標(達成で予算ボーナス)、寮の食事の改善案、海外遠征合宿(資金の使い道)、追いコン、殿堂入り

---

10. 体験イメージ(中堅校スタート)

- **1〜2年度目**: 春にエースを見極め、夏合宿で全体を底上げ。第一戦は様子見、第二戦で入賞を狙い、第三戦はシード権死守。
- **3〜5年度目**: スカウトした世代が伸び始め、層が厚くなる。第一戦と第二戦で別編成を組めるようになり、初優勝。
- **8〜14年度目**: 三大駅伝のすべてで優勝経験が揃い、**クリア演出**。ここからは王朝(連覇・同一年度の三冠・歴代記録)と、黄金世代の後の谷間をどう越えるかが遊びの中心になる。

---

11. UI/UX・アート方針(Steam + スマホ / 2D)

UI/UX

- **横画面基準**(16:9〜20:9、セーフエリア対応)。PC・スマホで同一レイアウトを使い、縦持ちは将来検討。
- 入力は**タッチ・マウス・ゲームパッド(Steam Deck含む)共通の「選択→決定」モデル**。ホバー依存は禁止し、詳細表示は長押し/タップ/ボタンで出す。
- いつでも中断できる。オートセーブはアプリ中断時・月境界・大会前。

画面は **42**(通常 22 / 進行 12 / 重なり 8)。一覧・遷移・各画面の仕様は `08_screen_spec.md` §5 と `08a_screen_catalog.md`。主な画面:

画面	役割
ホーム(SC-08)	状況、次の予定、通知、『続ける』
月間計画(SC-16)	グループ別のテーマ選択、月末の疲労・調子の予測(08 R-05)、伸びの見込み
部員一覧 / 選手詳細(SC-18, 19)	能力(ランク)・記録・疲労・調子・特殊能力・関係・出身、グループ編成、世代の戦力グラフ
スカウト(SC-27〜31)	**日本地図(地方→県)**、候補リスト、競合度と枠、ポイント配分、特別招聘の宣言
エントリー / オーダー(SC-21, 22)	区間適性◎○△、予想タイム、補欠、おまかせ登録・編成
大会(SC-23〜26)	前評判・予報、当日変更、余裕の目安、中継所順位、ハイライト、指示・檄、結果
歴史・殿堂(SC-34)	歴代優勝、記録、OB、**出身県マップ**(在籍・OBの出身県を日本地図に塗る)
運営(SC-32)	施設、スタッフ、資金

アート

- 世代交代で選手が無限に生まれるため、**キャラクターは個別の手描きではなく、パーツ合成(髪型・顔・体型・ユニフォーム色)で自動生成**する。ユニフォームは校色のパレットスワップ。
- **スタイルは「ドット絵のSD(ちびキャラ)」に決定**。量産・合成しやすく、小画面でも読みやすく、パワプロ的な雰囲気も出る。手描きデフォルメは、パーツ制作量が増えるため採用しない。

美術の調達方針(描き手がいない前提)

- **絵が必要な箇所を絞る**: UIは図形・色・フォントで構成し、ドット絵は ①選手アイコン(顔)②レース中のミニキャラ ③カットイン数枚 ④校章に限る。
- **パーツ合成なので描く枚数は少ない**: 髪型 約10種 × 顔 約5種 × 体型 約3種 を基本パーツにし、ユニフォームはパレットスワップで校色に対応する。組み合わせは数百通り以上になる。
- **開発順**: M1まではプレースホルダー(色付きの図形・仮アイコン)で進め、ゲームの面白さを先に検証する。美術はM1の手応えを見てから発注・調達する。
- **調達先の選択肢**: ①素材購入(Asset Store等。パーツ合成に対応したSDキャラ素材を探す)②外注(基本パーツのみ依頼すれば量は少ない)③生成AIの利用。③を使う場合は、Steamの生成AI利用の申告ルールや素材ライセンスを事前に確認する。どれにするかは §14 の未決事項。
- レース演出: 横スクロールのコースマップ上にミニキャラ/アイコンを走らせ、中継所でカットイン。山区間は高低差プロファイルを表示する。

---

12. 技術設計(Unity)

構成(最新のLTS系Unityを想定)

```
Assets/
  Scripts/
    Domain/        asmdef: Ekiden.Domain(noEngineReferences)… 純粋C#。 ゲームの全ルール
    Persistence/   asmdef: Ekiden.Persistence … セーブのコンテナ・世代・移行・検証と修復・同期の判定
    Platform/      Abstractions / Steam / Apple / Google / Fake … ストアとの境界(09 §15.4)
    Presentation/  asmdef: Ekiden.Presentation … UI、演出
    App/           asmdef: Ekiden.App … 起動・FlowController
  Master/ Strings/ … マスタ JSON・文字列テーブル(TextAsset。StreamingAssets は使わない)
Tests/             … Domain.Tests / Persistence.Tests(NUnit。Unity と dotnet の両方で実行)
tools/SimCli/      … .NET コンソール。Domain を参照して数千年度を高速に回す
tools/MasterGen/ StringsLint/ …
```

(詳細は `09_save_tech_spec.md` §15.2)

方針	内容
ロジック分離	Domain は `UnityEngine` を参照しない。EditModeテストと SimCli から同じコードを動かす
乱数	**鍵つきの PCG32**(`Pcg32(derive(master_seed, label, a, b))` 。状態を持たない。09 §8)。`UnityEngine.Random` / `System.Random` は使わない(環境差・セーブ再現性のため)。端末間のビット一致は求めず、実測で確かめた(09 §8.4)
時間	`DateTime.Now` 不使用。ゲーム内時計(年度・週)のみ
マスタ	JSON(`TextAsset`)で外部化。大会名・選手名テーブルも差し替え可能に。一度出した ID は再利用しない(09 §12)
UI	第一候補は **UI Toolkit**(リスト・表が多く、PC/スマホ共通に向く)。M0で**日本語フォント・モバイル性能・ゲームパッドのフォーカス移動**を検証し、難しければ uGUI
セーブ	コンテナ `EKSV` (ヘッダ + gzip の JSON + CRC32)・A/B の2世代・原子的な書き込み・マイグレーション・読み込み時の検証と修復(09 §4〜§7)。100年度で約 285 KB。クラウドは Steam Auto-Cloud / GameKit / Play Games の保存ゲーム。判定は『どちらが変わったか』で、削除しない(09 §9)
プラットフォーム	Steam: 実績・クラウド。スマホ: タッチ、アプリ中断対応、省電力。広告SDKは入れない。**永続解放**は Steam=DLC、スマホ=非消費型IAP(購入の復元に対応)。消費型は採らない。購入はストアごとに独立で、ストア間では共有しない(§6.5)
ローカライズ	日本語先行。文字列は最初からテーブル化

方針	内容
性能	世界シムは1年度あたり 36校×3大会+練習解決で軽い(Python 参照で、世界の更新 約 70 ms・自校の育成 約 50 ms・1レース 約 3 ms。09 §14)。スマホではホットパスの GC(LINQ・new)を避ける
決定性	Domain は純粋(時刻・乱数源・ファイルを知らない)。順序に意味のある走査は ID 昇順、整列は安定、丸めは `floor(x+0.5)`、並列化しない(09 §15.3 の D-1〜D-12。浮動小数の和は仕様の順に素朴に足す)

バランス目標と検算結果(`05_world_scout_spec.md` §14。04 v0.2 の本番式を組み込んだ参照モデルで検算済み。方針は『おまかせ』、特別招聘券はゲーム内の獲得のみ)

開始校	クリア(通算制覇)までの年度(中央値)	目安の時間	頭打ち後の優勝率(1大会)	備考
強豪	7	約3.5時間	0.57	開始時から優勝争い。全員がクリア
中堅	12.5	約6.5時間	0.51	順位は7位前後から 1.6位へ。45年度までに全員がクリア
弱小	23	約11.5時間	0.46	やり込み向け(45年度までに98%がクリア。上級の采配なら中央値18年度)
同好会	35.5	約18時間	0.14	部員5人で開始(通常の名簿と変わらない。07 §12.4)。45年度までに50%がクリア(券なしだと25%)

- **采配の差**(中堅、`05` §14.4 / 04 §14.4): 初心者(型に合わせた育成なし・変更なし・維持・檄なし)の優勝率 0.45、**おまかせ 0.51**、上級(当日変更と指示を最適化)0.58。クリア年度の中央値は 14 → **12** → 10。おまかせでも戦え、手動で詰めるほど有利になる。
- 頭打ちは戦力P(先発10人の平均)で約72。年度の1大会あたり優勝率は 0.51(おまかせ)〜0.58(上級)に収まる(目標A)。
- **長期安定性**: 自校なしの世界を80年度回しても、上位・中位・下位の戦力は ±0.8 以内で安定し、優勝校は平均16校に分散する(`05` §14.1)。**セーブは 100 年度で圧縮後 約 285 KB**(目標 1 MB・上限 3 MB。刈り込みあり。09 §5.4)。中堅を 300 年度回しても戦力 P は 72〜73 で頭打ちのまま(09 §16.3)。

---

13. 開発ロードマップ

マイルストーン	内容	完了条件
S(仕様確定)	仕様書一式(`00_spec_index.md`)を完成させ、各ルールに計算例を付ける。**Unity環境が不要な期間として、仕様の厳密化を優先する**	仕様完了の基準(`00_spec_index.md` §2)を全文書が満たす
M0	Domain の C# 実装(レース式 + 疲労モデル + 世界 + 永続化)+ テスト + SimCli(**Unity不要。.NET SDKのみで可**)。仕様の計算例を自動テストにする。作業の順序は 09 §17.3	§7.3 の検算が自動テスト化され、全区間で破綻しない。**09 §17.3: T1 ベクタ全件一致 / T2 性質テスト / T3 バイト一致の決定性 / T4 統計の帯(開始校ごとに 120 シード)/ T5 性能予算 / 300 年度連続で破綻しない**
M1	1年度の縦切りを Unity 上で(月間計画 → 3大会 → 結果。PC横画面)。**UI Toolkit の技術検証**(日本語・モバイル・パッド)を含む	1年度を通しで遊べ、約30分に収まる
M2	世代交代ループ(卒業・入学・スカウト・OB・主将)+ 長期シムの安定性テスト	50年度連続で破綻せず、セーブが膨らまない
M3	世界シミュレーション(36校 + 予選会)+ 歴史・殿堂	他校が盛衰し、記録が蓄積する
M4	イベント / 特殊能力 / 施設・スタッフ / クリア演出・実績	パワプロらしさが出る
M5	スマホ最適化(タッチUI・中断対応・クラウドセーブ)、Steam 対応、演出、難易度調整	製品品質
M6(拡張・未決)	**レジェンド保存(最大30人)とレジェンドオーダー**、共有コードによる出力・取り込み、非同期の対戦(`00_spec_index.md` §7 のメモ M-01・M-02)	レジェンドで組んだオーダーを走らせ、共有コードで友達と比べられる

---

14. 決定済み事項と未決事項

14.1 決定済み(v0.4)

項目	決定
クリア条件	三大駅伝それぞれでの優勝経験(別年度でも可)。同一年度の三冠は上位目標
大会名	仮称(秋雲駅伝 / 全国大学駅伝 / 新春大学駅伝)で進める。実名化は後で判断
アート	ドット絵のSD + パーツ合成(開発側に決定を委任)
画面向き	横画面基準。縦持ちは将来検討
収益モデル	本体は買い切り。**特別招聘券は永続解放(DLC・非消費型)で提供**(§6.5。券の配布周期は未決)
特別招聘	券を使うと翌年度に留学生またはスーパールーキーが確定入学する。物語・専用イベントは付けない(§6.5)
出身県	全部員に出身県(留学生は出身国)と出身高校(架空)を設定(§4.5)
開発方針	仕様を先に厳密化し、Unityの実装は後回し(§13 の S → M0 → M1)

実名化・通称の注意(商用化前に確認): 大会名・ロゴ・放送素材・通称(山の神・花の2区など)は権利対象になり得る。実名化や通称の使用は、主催者・法務へ確認してから判断する。名称はすべて差し替え可能なテーブルで持つ。

14.2 未決事項(既定値付きの一覧は `00_spec_index.md` §5)

1. **券の配布周期**(U-12): **確定** — 4年度に2枚、失効なし(07 §5)。ゲーム内の獲得条件(U-10)は 07 §4、留学生=即戦力型・スーパールーキー=育成型の役割分担(U-11)は 07 §6 で確定した。
2. **留学生の大会規定**(U-02): 実大会の登録・出走の上限を確認して反映する。既定値は全大会で 登録2 / 出走1。
3. **美術の調達方法**: 素材購入 / 外注 / 生成AI のどれにするか(§11)。M1まではプレースホルダーで進めるので、M1の手応えを見てから決めればよい。生成AIを使う場合は、Steamの申告ルールや素材ライセンスの確認が必要。
4. **言語**: 日本語のみか、英語対応も視野に入れるか。文字列は最初からテーブル化するので、後から追加は可能。
5. **価格帯**: 本体の価格と、追加コンテンツを出す場合はその価格。スコープ(特にM4以降)の見積もりに影響するので、M3のあたりで決めたい。

付録: パワプロ要素を落とさないためのチェックリスト

- ☐ 月間計画を決めて結果が出るまでの流れが気持ちいい(月末レポートで成長が見える)
- ☐ 能力が伸びる瞬間の演出(数値アップ・ランクアップ・特殊能力取得)
- ☐ 選択肢のあるイベントで性格やストーリーが分岐する
- ☐ 「調子」が上がった状態で本番を迎えるカタルシス
- ☐ 卒業時に育てた選手の「完成」を確認できる(最終能力・称号・進路)
- ☐ 黄金世代を送り出したあと、次の世代を育て直す手応えがある
- ☐ 新入部員の出身県・出身高校が見えて、同郷の絆や県別OBのつながりに愛着が持てる
- ☐ 出身県マップが年々埋まっていく収集の楽しみがある

第2部 データ仕様

原典: `02_data_spec.md`　　原典の版: v1.1　　改版の経緯: 付録 G.2

> 範囲: 列挙、マスタ(構造と初期値)、実行時エンティティ、不変条件、セーブ構造。数値の効果(練習のXP量、補正値など)は 03 / 04 で定義する。ここでは**構造・ID・範囲**を確定する。

1. 規約

項目	規約
整数	32bit整数。能力・疲労・やる気は `int`
時間(タイム)	**0.1秒単位の整数**(`ds`)。例: 1:05:54.3 = 39543 ds。表示時に変換
金額	万円単位の整数
浮動小数	計算中は `double` 。**状態に書き戻す前に整数へ丸める**(四捨五入、0.5は切り上げ)
再現性	同一プラットフォーム・同一シードで結果が一致すること。端末間のビット一致は求めない(セーブは状態を保存するため。端末差の実測は 09 §8.4)。基準ペース `pace50(d)` は**マスタの表の値**を使い(`log` を呼ばない。09 §8.4)、保存する `double` は**小数6桁に丸めて**書き戻す(`Q6` 。09 §5.3)
実行時ID	`int` 。1から単調増加、**再利用しない**
マスタID	文字列。小文字ドット区切り。例: `skill.mountain_god`
JSONキー	`snake_case`
コード識別子	`PascalCase` (型)/ `camelCase` (メンバ)
日付	`GameDate { year(年度, 1〜), month_index(0〜11, 0=4月), week(1〜4) }` 。通算週 = `(year-1)×48 + month_index×4 + (week-1)`
表示名	すべて**名称テーブル**から引く(コードに直書きしない)。大会名・大学名・選手名の差し替えを可能にする

2. 列挙(Enum)

列挙	値
`StatId`	`SPD` `STA` `PAC` `KCK` `CLB` `DWN` `CON` `MEN` `REC` `DUR` (この順で配列の添字 0〜9)
`GrowthType`	`Early` (早熟) `Normal` (標準) `Late` (晩成)
`OriginType`	`Japan` `Overseas`
`Region`	`Hokkaido` `Tohoku` `Kanto` `Chubu` `Kinki` `Chugoku` `Shikoku` `Kyushu`
`PopTier`	`A` `B` `C` `D` (県の人口区分)
`SchoolTier`	`Elite` (強豪) `Mid` (中堅) `Low` (下位)
`Group`	`A` (主力) `B` (準主力) `C` (育成)
`FormLevel`	−2 絶不調 / −1 不調 / 0 普通 / +1 好調 / +2 絶好調
`Motivation`	1〜5 の整数(1=最低)
`InjurySeverity`	`None` `Minor` (軽傷) `Moderate` (中傷) `Severe` (重傷)
`SkillRank`	`Blue` (通常) `Gold` (強力) `Red` (負)
`SkillCategory`	`Section` (区間) `Flow` (展開) `Peaking` (調整) `Mental` (精神) `Body` (体質)
`SectionType`	`Fast` (短距離高速) `Mid` (中距離) `Long` (長距離) `Climb` (山登り) `Descent` (山下り)
`RaceKind`	`Fall` (第一戦) `National` (第二戦) `NewYear` (第三戦) `Qualifier` (予選会) `Meet` (記録会)
`CareerPath`	`Corporate` (実業団) `Teacher` (教員) `Employment` (一般就職) `GradSchool` (院進学)

列挙	値
`PackKind`	`Cosmetic` (見た目) `Scenario` (シナリオ) `GoldenPlan` (特別招聘券の永続解放)
`TicketKind`	`Intl` (留学生) `Rookie` (スーパールーキー)
`RecruitSource`	`Normal` (通常のスカウト) `Ticket` (特別招聘)
`TicketSource`	`Earned` (ゲーム内で獲得) `PlanGrant` (GoldenPlanの周期付与)
`PeakingMode`	`Standard` (大会の3週前から4週) `Short` (直前2週) `None` (適用しない)。03 §5.3
`Difficulty`	`Elite` (強豪開始) `Mid` (中堅開始) `Low` (弱小開始) `Club` (同好会開始)。**新規ゲームの開始時にプレイヤーが選ぶ**(既定: `Mid`)。開始校は、その時点の**戦力順位**が `Elite` =3位 / `Mid` =7位 / `Low` =20位 / `Club` =33位 の学校(05 §5)
`Instruction`	`Attack` (攻める) `Keep` (維持) `Hold` (抑える)。04 §9.1
`Focus`	`Balanced` (バランス) `Speed` (スピード寄り) `Long` (ロング寄り)。年間方針の重点(08 §8 R-04)。月ごとのテーマのテンプレートを選ぶ
`EventCategory`	`Team` `Management` `Individual` `Race` `Hometown` `Graduation` `Awakening` (06 §4)
`ModKind`	`E_mult` `r_add` `inj_mult` `form_add` `funds_year` `scout_pts` (補正の種類。06 §6)
`AchCond`	実績の判定の種類。`RaceRank` `Clear` `Sweep` `Streak` `Count` (§3.10)
`StyleForm`	`Speed` (スピード型 `S`) `Balance` (バランス型 `B`) `Long` (ロング型 `L`)。個性の『型』。10 §2
`StyleRun`	`Pack` (集団走◎ `P`) `Free` (走法は普通 `F`) `Solo` (単独走◎ `S`)。個性の『走法』
`StyleFlow`	`Chase` (追い上げ型 `C`) `Even` (展開は普通 `E`) `Lead` (逃げ型 `L`)。個性の『展開』
`Gift`	`Climber` (登坂) `Descender` (下り) `Ace` (エース)。まれな隠しの素養。10 §2
`Trait`	`Serious` (真面目) `Competitive` (負けず嫌い) `MyPace` (マイペース) `Passionate` (情熱家) `Careful` (慎重) `Cheerful` (お調子者) `Quiet` (寡黙) `Caring` (面倒見がよい)
`StaffRole`	`Coach` `Trainer` `Nutritionist` `Manager` `Scout`
`FacilityId`	`Dorm` (寮) `Track` (トラック) `Hill` (坂道コース) `Gym` (トレーニング室) `Medical` (医療室)
`ScoutStatus`	`Unseen` `Seen` `Contacted` `Visited` `Committed` (入部確定) `Lost` (他校へ/辞退)

3. マスタデータ

3.1 Region(地方)と距離帯

隣接関係:

地方	隣接する地方
Hokkaido	Tohoku
Tohoku	Hokkaido, Kanto, Chubu
Kanto	Tohoku, Chubu
Chubu	Tohoku, Kanto, Kinki
Kinki	Chubu, Chugoku, Shikoku
Chugoku	Kinki, Shikoku, Kyushu
Shikoku	Kinki, Chugoku, Kyushu
Kyushu	Chugoku, Shikoku

```
distance_band(a, b):  a == b なら 0(地元) / 隣接なら 1 / それ以外 2(遠方)
                      b が海外(Overseas)なら 3
```

距離帯の効果(勧誘成功率・帰省イベント・視察コスト)は 05 で定義する。

3.2 Prefecture(都道府県)47件

項目	型	説明
`id`	int 1〜47	JISコード順
`name`	string	名称(名称テーブル)
`region`	Region	地方
`pop_tier`	PopTier	人口区分(A:500万以上 / B:150〜499万 / C:100〜149万 / D:100万未満)
`elite`	bool	高校駅伝の強豪県(初期値案。実績を参考にした仮置き)
`dialect_id`	string?	方言セット。未使用(U-06)

供給重み(候補の出身県の抽選)

```
supply_weight = { A:4.0, B:2.5, C:1.5, D:1.0 }[pop_tier] × (elite ? 1.3 : 1.0)
```

id	県	地方	人口区分	強豪		id	県	地方	人口区分	強豪
1	北海道	Hokkaido	A			25	滋賀	Kinki	C	
2	青森	Tohoku	C			26	京都	Kinki	B	★
3	岩手	Tohoku	C			27	大阪	Kinki	A	
4	宮城	Tohoku	B	★		28	兵庫	Kinki	A	★
5	秋田	Tohoku	D			29	奈良	Kinki	C	
6	山形	Tohoku	C			30	和歌山	Kinki	D	
7	福島	Tohoku	B	★		31	鳥取	Chugoku	D	
8	茨城	Kanto	B			32	島根	Chugoku	D	
9	栃木	Kanto	B			33	岡山	Chugoku	B	★
10	群馬	Kanto	B			34	広島	Chugoku	B	★
11	埼玉	Kanto	A	★		35	山口	Chugoku	C	
12	千葉	Kanto	A			36	徳島	Shikoku	D	
13	東京	Kanto	A			37	香川	Shikoku	D	
14	神奈川	Kanto	A			38	愛媛	Shikoku	C	
15	新潟	Chubu	B			39	高知	Shikoku	D	
16	富山	Chubu	C			40	福岡	Kyushu	A	★
17	石川	Chubu	C			41	佐賀	Kyushu	D	
18	福井	Chubu	D			42	長崎	Kyushu	C	
19	山梨	Chubu	D			43	熊本	Kyushu	B	
20	長野	Chubu	B	★		44	大分	Kyushu	C	
21	岐阜	Chubu	B			45	宮崎	Kyushu	C	
22	静岡	Chubu	B			46	鹿児島	Kyushu	B	
23	愛知	Chubu	A	★		47	沖縄	Kyushu	C	
24	三重	Kinki	B							

検算(自動テストに写す): 人口区分の件数は A=9 / B=15 / C=13 / D=10(計47)。地方別の件数は 北海道1 / 東北6 / 関東7 / 中部9 / 近畿7 / 中国5 / 四国4 / 九州8。供給重みの総和 = 112.3。

地方	供給比率(初期値案)
関東	22.0%
中部	18.7%
近畿	16.9%

地方	供給比率(初期値案)
九州	15.3%
東北	10.7%
中国	8.9%
四国	4.0%
北海道	3.6%

単県の最大は 福岡・愛知・埼玉・兵庫が各4.6%、最小は 人口区分Dの10県が各0.9%。

3.3 HighSchool(出身高校)

項目	型	説明
`id`	int	世界生成時に採番
`name`	string	架空名(語彙リスト × 接尾辞から生成。実在校名リストと照合して重複を避ける)
`prefecture_id`	int 1〜47	所在県
`strength`	int 0〜100	陸上部の強さ。高いほど上位の候補が出る

- 高校数: 人口区分 A=8 / B=5 / C=3 / D=2 校、**強豪県は +1 校**。合計 216校(72+75+39+20 + 強豪県10)。
- 高校はゲーム開始時に生成して固定する。年度ごとに増減しない。生成アルゴリズムは 05。

3.4 Country(出身国)

項目	型	説明
`id`	int	
`name`	string	名称テーブル

- 出身国は24か国(07 §6.3。名称テーブルで差し替え可能)。抽選は一様。
- **国・地域による能力・外見・成長の補正は設けない**(個人差のみ)。

3.5 Course / Section / RaceRules(大会)

Section(区間)

項目	型	説明
`index`	int 1〜	区間番号
`distance_km`	double	距離
`type`	SectionType	区間タイプ
`terrain_factor`	double	地形係数(基準ペースへの倍率。区間ごとの値は下の表。L-69)
`day`	int	開催日(第三戦のみ 1=往路 / 2=復路)

Course: `race_kind` と `Section[]`。

大会	区間(距離km:タイプ)	合計
Fall	1: 8.0 Fast / 2: 5.8 Fast / 3: 8.5 Fast / 4: 6.2 Fast / 5: 6.4 Fast / 6: 10.2 Long	45.1
National	1: 9.5 Fast / 2: 11.1 Fast / 3: 11.9 Fast / 4: 11.8 Fast / 5: 12.4 Mid / 6: 12.8 Mid / 7: 17.6 Long / 8: 19.7 Long	106.8
NewYear	1: 21.3 / 2: 23.1 / 3: 21.4 / 4: 20.9 / **5: 20.8 Climb** / **6: 20.8 Descent** / 7: 21.3 / 8: 21.4 / 9: 23.1 / 10: 23.0(5・6以外は Long。1〜5区は day=1、6〜10区は day=2)	217.1

地形係数(L-69。区間タイムの基準ペースへの倍率。04 §7.1): 平地の区間を一律 1.000 とはせず、**初期の区間記録(§4.8)から較正する**。区間 `i`(距離 `d`)の記録 `R`(秒)を、区間適性値が `P* = 85` の走者が標準の条件で出すタイムとみなし、`terr = (R / d + K_SLOPE × (P* - 50)) / pace50(d)`(小数 3 桁に丸める。`verify/calib_terrain.py`)。`P* = 85` は 04 の旧い注記『個人の区間記録は P≈85 に相当』(L-10)と同じ水準。

- **狙い**: 係数が一律だと、実在の記録が遅めの区間(第三戦 8・9・10 区など)は簡単に破られ、速めの区間(第三戦 3 区など)はほとんど破られない(24 区間のうち約 9 区間が年度 1 の大会で半分以上の確率で破られ、ほとんど破られない区間が 3 つ。L-68 ④)。較正すると、年度 1 で半分以上破られる区間は 5 に減り、ほとんど破られない区間もなくなる。記録が最初に破られるまでの年数の中央値は 4〜5 年度(自然にできた記録は 8〜10 年度)。
- **P* は『コース全体の速さの水準』でもある**: P* を上げると係数の平均が上がり(コースが遅くなり)、能力の差が時間に占める割合が小さくなって番狂わせが増え、優勝率が下がる(弱小開始・60 シードで P* = 88: 優勝率 0.43 → 0.36・クリア 24.5 → 28.8 年度。P* = 85 は旧と同じ 0.44・25.0)。記録の持ちは P* を上げると延びる(88 で 6〜9 年度)が、難しさを変えないために 85 とした。
- 5 区(登り)は旧 1.13 → 1.127、6 区(下り)は旧 0.93 → 0.963。下りは、実在の記録の速さに合わせると、旧い係数ほどには速くならない。

大会	区間	距離(km)	タイプ	地形係数
第一戦	1	8.0	Fast	1.022
第一戦	2	5.8	Fast	0.983
第一戦	3	8.5	Fast	0.982
第一戦	4	6.2	Fast	1.016
第一戦	5	6.4	Fast	1.008
第一戦	6	10.2	Long	1.001
第二戦	1	9.5	Fast	1.025
第二戦	2	11.1	Fast	1.005
第二戦	3	11.9	Fast	0.989
第二戦	4	11.8	Fast	1.005
第二戦	5	12.4	Mid	1.011
第二戦	6	12.8	Mid	1.026
第二戦	7	17.6	Long	0.996
第二戦	8	19.7	Long	0.994
第三戦	1	21.3	Long	0.998
第三戦	2	23.1	Long	0.989
第三戦	3	21.4	Long	0.978
第三戦	4	20.9	Long	1.009
第三戦	5	20.8	Climb	1.127
第三戦	6	20.8	Descent	0.963
第三戦	7	21.3	Long	1.002
第三戦	8	21.4	Long	1.043
第三戦	9	23.1	Long	1.019
第三戦	10	23.0	Long	1.031

RaceRules

項目	Fall	National	NewYear	備考
`entry_size`（登録）	10	14	16	
`run_size`（出走）	6	8	10	= 区間数
`swap_limit`（当日変更）	1	2	4	04 §5.3
`intl_reg_max`（留学生の登録上限）	2	2	2	初期値案。実大会の規定を確認して決める(U-02)
`intl_run_max`（留学生の出走上限）	1	1	1	同上

項目	Fall	National	NewYear	備考
`participants`（出場校数）	20	30	20	世界は36校
`qualification`	`YearRankTop`	`YearRankTop`	`SeedPlusQualifier`	04で定義
`entry_deadline_weeks`	4	4	4	エントリー締切は大会の4週前
`calendar`	10月2週	11月1週	1月1週	予選会は10月3週

3.6 TrainingMenu(練習)

項目	型	説明
`id`	string	下表
`kind`	`KeySession` / `Mileage` / `Theme`	種別
`gain`	map<StatId, int>	XP(0.1単位)。**値は 03 で定義**
`load`	int	疲労負荷。**値は 03 で定義**
`injury_risk`	enum	低 / 中 / 高。係数は 03

種別	ID一覧
ポイント練習(`ks.*`)	`ks.pace_run` `ks.threshold` `ks.long_run` `ks.interval` `ks.repetition` `ks.build_up` `ks.hill` `ks.downhill` `ks.core` `ks.sharpen` `ks.rest` `ks.time_trial` `ks.cross_train`（故障中の代替）
走行量(`mil.*`)	`mil.low`（70km） `mil.std`（110km） `mil.high`（150km） `mil.max`（180km）
月のテーマ(`th.*`)	`th.base_build` `th.speed` `th.mountain` `th.camp` `th.recovery` `th.custom`

Theme(月のテーマ)の構造: `weeks[4]` の配列。各要素は `{ key_sessions[2], mileage }`。基本は「3週負荷 + 1週軽め」。具体的な中身は 03。

3.7 Skill(特殊能力)

項目	型	説明
`id`	string	
`category`	SkillCategory	
`rank`	SkillRank	
`exclusive_with`	list<string>	同時に持てない能力(例: 正と負のペア)
`acquire`	条件式	取得条件(06 §3 の文法。覚醒は 06 §9。カウンターは 06 §10)
`effect`	効果定義	数値は 03 / 04

ID	名称(仮)	系統	ランク	効果の概要	数値の定義先
`skill.mountain_god`	山の神	Section	Gold	山登り区間の適性値+	04
`skill.downhill_master`	山下りの達人	Section	Blue	山下り区間の適性値+	04
`skill.ace_section`	エース区間〇	Section	Blue	各大会の最長区間の適性値+	04
`skill.anchor`	勝負のアンカー	Section	Blue	最終区の接戦補正が強まる	04
`skill.overtake`	ごぼう抜き	Flow	Blue	追う展開でタイム−	04
`skill.hold_lead`	逃げ切り	Flow	Blue	先頭走時のブレが小さい	04
`skill.pack_runner`	集団走〇	Flow	Blue	集団走の補正が大きい	04
`skill.solo_weak`	単独走×	Flow	Red	単独走の補正が悪化	04
`skill.peaking_master`	調整力〇	Peaking	Blue	ピーキング補正+	03
`skill.summer_boy`	夏男	Peaking	Blue	夏合宿のXP+、暑さの影響なし	03
`skill.summer_fade`	夏バテ	Peaking	Red	夏の回復力低下	03
`skill.big_stage`	大舞台〇	Mental	Blue	重圧係数が下がる	04

ID	名称(仮)	系統	ランク	効果の概要	数値の定義先
skill.ekiden_man	駅伝男	Mental	Gold	駅伝本番で調子が一段階上がる	04
skill.revenge	リベンジ	Mental	Blue	前回ブレーキの大会で適性値+	04
skill.stage_fright	本番弱い	Mental	Red	重圧係数が上がる	04
skill.brake_prone	ブレーキ癖	Mental	Red	ブレーキ率が上がる	04
skill.iron_legs	鉄の脚	Body	Gold	故障率が下がる	03
skill.recovery_plus	回復力〇	Body	Blue	疲労の減衰率+	03
skill.injury_prone	故障癖	Body	Red	故障率が上がる	03

排他: solo_weak と pack_runner、stage_fright と big_stage、injury_prone と iron_legs。

3.8 Facility(施設)・Staff(スタッフ)

項目	範囲
施設レベル	1〜5(FacilityId ごと)。初期値は開始校の難易度で決まる(05)
スタッフレベル	1〜5(StaffRole ごと)
スタッフ人数	役職ごとに最大1名(初期値案)。拡張枠は監督ランクで解放(01 §3.3)

3.9 特別招聘券(Ticket)と ContentPack

特別招聘券の規則(定数表。初期値案。確定した手順と検算は 07_special_recruit_spec.md)

定数	値	意味
K_TICKET_STOCK_MAX	2	手元の券(stock)の上限。超える入手は繰り延べる(bank 。失効しない)
K_TICKET_PER_YEAR	1	1年度に宣言できる回数
K_TICKET_ACTIVE_MAX	2	在籍する招聘選手(recruit_source = Ticket)と宣言済みの招聘の合計の上限
K_TICKET_DECLARE_FROM / K_TICKET_DECLARE_TO	month_index 2(6月)/ 10(2月)	宣言できる期間
K_PLAN_CYCLE_YEARS	4	GoldenPlan の付与の周期(年度)。**U-12 で確定**
K_PLAN_COUNT	2	1回の付与の枚数。**U-12 で確定**
K_OFFLINE_GRACE_DAYS	30	購入状態を確認できないときに、前回の確認を信じる日数
K_SPECIAL_MAX_STAT / K_SPECIAL_MAX_MEAN	80 / 68	招聘選手の入学時の能力の上限(各能力 / 平均)
K_SPECIAL_SKILL_P	0.30	招聘選手が入学時に特殊能力を1つ持つ確率

> 注: 使用は、年1回・同時在籍2名までなので、平均して年0.5回(4年に2回)が上限。入手がこれを超えても、券は**失効せず繰り延べる**(07 §3.2)。

流れ: 宣言期間中に「種別(TicketKind)」を選んで宣言(スーパールーキーは出身県も指定可能)→ 券を1枚消費し pending に記録(**取り消せない**)→ 3月の入学者確定で、pending の内容に応じた選手を1名生成して入学者に加える(自校の定員10のうち1人分。奨学金枠は使わない)。勧誘判定はなく、他校には奪われない。

入手経路: すべて同じ在庫(stock と bank)に入る。経路は TicketSource でログに残す。

- Earned : ゲーム内の節目(07 §4。14個 / 45年度。継続の記念で増え続ける)。
- PlanGrant : GoldenPlan の購入状態が True のとき、(year - plan_anchor_year) mod K_PLAN_CYCLE_YEARS = 0 の年度に K_PLAN_COUNT 枚(同じ年度に1回だけ)。**購入を初めて確認した年度を起点に、すぐ最初の付与**(07 §5.2)。権利を失った場合(返金など)は付与を止めるが、**所持済みの券・宣言済みの招聘・在籍する招聘選手は残る**。
- 消費型の購入による入手は採らない(U-09)。

招聘選手の生成帯(`SpecialRecruitBand`。07 §6.1 が確定値)

項目	留学生(`Intl`)	スーパールーキー(`Rookie`)
入学時の力 `A0`	`N(66,2)` を [64,68] に収める	`N(61,2)` を [58,64] に収める
ギャップ `gap`(`Q = A0 + 0.6×gap`)	`N(8,2)` を [4,12]。`Q` 平均 70.8	`N(25,3)` を [18,32]。`Q` 平均 76.0
能力の上限	各能力 ≤ 80、10能力の平均 ≤ 68(超えたら全能力を同じだけ下げる)	同左
素質	`stats[i] ≤ potential[i] ≤ 100`	同左
成長タイプ・型	通常の候補と同じ分布	同左
特殊能力	30% で1つ(Blue の10種から)	同左
出走の制約	留学生枠(登録2・出走1)	なし
出身	出身国(24か国から一様)	出身県(指定可)。高校は県内から抽選

- 差は**国による能力補正ではなく、種別の役割分担**(即戦力型と育成型)。外見・名前も出身国で変えない。

ContentPack(見た目・シナリオ・永続解放)

項目	型	説明
`pack_id`	string	例: `pack.golden_plan` / `pack.cosmetic.kit`
`kind`	PackKind	`Cosmetic` / `Scenario` / `GoldenPlan`
`title` / `description`	string	名称テーブル
`contents`	list<ContentRef>	`{ type, id }`。`type` = `Uniform` `Emblem` `HairStyle` `HairColor` `CheerSong` `Bgm` `Scenario`。`GoldenPlan` は空
`store_ids`	map	Steam DLC の AppID、App Store の商品ID、Google Play の商品ID
`min_game_version`	string	必要な本体バージョン

- 見た目は**表示の層だけ**に作用し、選手の生成・乱数・大会の結果に影響しない(07 §9.2)。

3.10 Achievement(実績)と ProfileData

実績は**プレイヤー(アカウント)単位**で、セーブデータ(`SaveData`)とは別の `ProfileData` に保存する。Steam 実績・Game Center・Google Play Games の実績に対応づける。解除の判定は本体の中で行い、各ストアへは解除を通知するだけにする(未接続のときは貯めておき、接続後にまとめて送る)。

AchievementDef(マスタ)

項目	型	説明
`id`	string	例: `ach.clear.low.no_invite`
`title` / `description`	string	名称テーブル(日本語先行)
`hidden`	bool	解除するまで内容を隠す
`cond`	{ kind: AchCond, params }	下表
`store_ids`	{ steam, apple, google }	各ストアの実績ID

判定の種類(`AchCond`)

kind	params	判定のタイミング
`RaceRank`	`race_kind?`, `rank_max`, `streak?`	大会の終了時。順位が `rank_max` 以内
`Clear`	`difficulty?`, `no_invite?`	クリア(3大会すべてで優勝経験が揃った大会の終了時)。`difficulty` を指定したら、そのセーブの開始校が一致すること。`no_invite = true` なら、クリア時点で `run.invites_declared = 0`
`Sweep`	`consecutive?`	同一年度に3大会すべてで優勝(`consecutive` 年連続)
`Streak`	`race_kind?`, `n`	同じ大会を `n` 年連続で優勝

kind	params	判定のタイミング
`Count`	`metric`, `n`	累計が `n` 以上(`metric`：卒業生数 / 区間賞 / 出身県の種類 / 殿堂入り / 年度数 など)

ProfileData(セーブとは別。Steam Cloud 等で同期)

項目	型	説明
`schema_version`	int	
`achievements`	map<string, { unlocked_at }>	解除した実績。**一度解除したら消えない**
`counters`	map<string, int>	累計の指標(複数のセーブにまたがる実績用)
`clears`	list<{ scenario_id, difficulty?, clear_year_index, no_invite, date }>	クリアの記録(開始校・シナリオごとの最短など)

- **購入状態のキャッシュ**(最後の確認の時刻・所有の有無)は `ProfileData` ではなく、**端末ごとの `LocalState`**(同期しない。09 §3.3)に持つ。確認の時刻と『動かした時間』が端末ごとの量だから(09 §10)。
- **`ProfileData` のマージ**: 競合の確認はせず、実績は和集合(解除日時は早いほう)・累計はキーごとに最大・クリアは和集合(09 §9.6)。
- **購入状態は実績に影響しない**: `GoldenPlan` を持っていても、券を使わなければ `invites_declared = 0` のまま。実績の違いは「招聘選手を使ったか」だけで決まる。
- `run.invites_declared` は、券を使って招聘を**宣言した時点で**加算する(入学前に取り消しはない)。クリア後に券を使っても、すでに解除した実績は消えない。
- 実績の一覧(初期案)は 01 §3.3。

3.11 ScenarioDef(開始状態)

開始状態だけを定めるデータ。世界の規則・他校・乱数は変えない(確定した形式と検算は 07 §10)。本体の4つの難易度も同じ形式で持つ。

項目	型	説明	範囲
`id`	string	例: `sc.low` / `sc.koki`	一意
`name` / `description`	string	名称テーブル	—
`strength_rank`	int	開始校を選ぶ戦力順位(先発10人の力の平均の順位)	1〜36
`prestige` / `fame_hist`	int / double?	名声と名声ポイントの履歴の上書き(省略なら、その学校のまま)	0〜1000 / 0〜40
`facilities` / `staff`	map<id, int>	施設・スタッフのレベル	各 1〜5
`funds`	int	資金(万円)	0〜5000
`members`	int?	部員数(省略なら、その学校の名簿のまま)	5〜40
`ob_level`	int?	各地方のOBの人数の初期値	0〜25
`difficulty_tag`	`Difficulty`?	本体の4つだけ設定。シナリオパックは null	—

本体の4つ: `sc.elite`(戦力3位)/ `sc.mid`(7位)/ `sc.low`(20位)/ `sc.club`(33位。部員5人)。

3.12 DeviceSettings(端末ごとの設定)と SaveMeta

DeviceSettings(見え方・音・操作。**端末ごと・クラウドに同期しない**。保存は `device.json`(09 §3.1)。08 §9)

項目	型	範囲・既定
`text_scale`	int	100 / 125 / 150(%。既定 100)
`color_mode`	enum	`Standard` / `Distinct`(色の区別を強める。既定 Standard)
`reduce_motion`	bool	動きを減らす(既定 false)
`kana`	bool	かな(ふりがな)の表示(既定 false)
`volume_bgm` / `volume_se` / `volume_cheer`	int	0〜10(既定 7 / 7 / 7)
`haptics`	bool	触覚(スマホ。既定 true)

項目	型	範囲・既定
`anim_speed`	int	1 / 2(演出の既定の速度。既定 1)
`hints_seen` / `hints_on`	list<screen_id> / bool	見たヒントの画面 / ヒントを出すか(既定 true)
`language`	string	`ja` (v1 は日本語のみ。U-04)
`cloud_sync`	bool	クラウド同期を使うか(既定 true。端末ごとに切れる。09 §9.5)

SaveMeta(スロットの一覧に出す要約。セーブの本体を読まずに表示する。**セーブのヘッダ(09 §4.2)の部分集合**で、ペイロードには持たない)

項目	型	説明
`slot`	int	1〜 `K_UI_SAVE_SLOTS` (3)
`saved_at`	datetime	保存の日時(表示用。ゲームの規則には使わない)
`year` / `month_index`	int	第N年度・月
`school_name` / `scenario_id` / `difficulty`	string / string / enum?	校名・開始の種類
`play_seconds`	int	プレイ時間

---

4. 実行時エンティティ

> 保存するときの鍵・型・順・既定値は **09 §5.6**(出身は `origin: { … }` に入れ子、故障は `injury: { severity, weeks_left, age }` など)。この節の表は項目の意味。

4.1 Runner(選手)

項目	型	範囲・説明
`id`	int	
`family_name` / `given_name`	string	姓 / 名
`family_kana` / `given_kana`	string	かな
`origin_type`	OriginType	
`prefecture_id`	int?	1〜47。`Japan` のとき必須
`country_id`	int?	`Overseas` のとき必須
`high_school_id`	int?	`Japan` のとき必須
`school_id`	int?	所属大学。候補・OBは null
`enroll_year`	int	入学年度
`grade`	int(派生)	`current_year - enroll_year + 1` 。1〜4
`growth_type`	GrowthType	
`stats`	int[10]	0〜100(`StatId` 順)
`potential`	int[10]	`stats[i]` 〜100。**隠し**
`xp`	int[10]	0.1単位。0以上、閾値未満
`skills`	list<string>	最大8個
`fatigue`	int	0〜100
`form`	FormLevel	−2〜+2(週ごとに更新)
`taper_weeks`	int	調整週の連続数(ピーキング補正の計算用)。0以上
`sharpen_bits`	int	直近3週に `ks.sharpen` を含んだかのビット列(0〜7)

項目	型	範囲・説明
motivation	int	1〜5
injury	{ severity, weeks_left }	severity = None ⇔ weeks_left = 0
group	Group?	自校のときのみ
bonds	map<int, int>	他の選手ID → 絆。**0.1単位(0〜1000)**。0の組は持たない(疎に保持)
traits	list<Trait>	最大3
style	StyleCode	**個性**。3文字(型 S/B/L + 走法 P/F/S + 展開 C/E/L 。例 SPC = スピード型・集団走◎・追い上げ型)。全員が持つ。練習・レースに影響(10)。見える
gifts	list<Gift>	**素養**。0〜2個(重複なし)。**隠し**。成長・素質の上限・特殊能力の取得に影響(10)
gift_revealed	bool	素養が判明したか(入部して K_GIFT_REVEAL_WEEKS 週後、またはスカウトで見た情報で。10 §8)。判明するまで画面には出さない
is_captain	bool	
is_overseas	bool(派生)	origin_type == Overseas
recruit_source	RecruitSource	通常のスカウトか、特別招聘か
ticket_kind	TicketKind?	recruit_source = Ticket のとき設定
pb	{ t5000, t10000, t_half }	**派生(保存しない)**。能力から調子0・環境なしで導出した記録(ds)。式は 08 §8 R-03。記録会は記録の更新に関与しない
career	{ races, section_wins, relay_wins, brakes, records }	通算成績。records = 区間記録を更新した回数(殿堂入りの条件。OBRecord.career.records に持ち越す)。**追加の項目**(ないセーブは 0)
ctr	map<string, int>	イベント・覚醒用のカウンター(06 §10)。自校の選手のみ
appearance	Appearance	4.2
look_seed	uint	外見・名前の再現用シード
appearance_override	{ hair_style?, hair_color? }	プレイヤーが後から変えた外見(表示の層だけ。購入状態で他の選手が変わらないよう、生成とは別に持つ。07 §9.2)。自校の選手のみ

4.2 Appearance(外見)

項目	型	説明
hair_style	int	約10種
hair_color	int	パレット番号
face	int	約5種
body	int	3種(細め・標準・がっしり)
skin_tone	int	パレット番号

ユニフォームの色は所属大学の校色から決まる(選手には持たせない)。

4.3 SchoolState(大学)

項目	型	説明
id	int	
name / name_short / kana	string	架空名
prefecture_id	int	所在地
region	Region(派生)	
tier	SchoolTier	開始時の校格(初期値の決定にだけ使う。以後の強さは名声で決まる)
prestige	double	0〜1000。世界の更新は連続値で動く(05 §3.2)ので double で保存し(Q6 。09 §5.3)、表示は floor(x + 0.5) (L-45)

項目	型	説明
`fame_hist`	double	名声ポイントの履歴(05 §3.2)。動く範囲は 0〜67.5(1 年度に取れる最大 = 3 大会とも優勝 3 × 12.5 + 三冠 30 を取り続けた極限。シナリオの上書き(§3 マスタデータ)の入力範囲 0〜40 とは別。L-45)
`colors`	{ primary, secondary }	RGB
`emblem_id`	int	校章
`is_player`	bool	
`roster`	list<int>	在籍選手のID(最大40)。自校は `Runner`、他校は `RivalRunner`
`year_points`	int	年度の評価ポイント(年度ランキング用。05 §3.1)
`cum_points`	int	評価ポイントの累計(監督ランク用。自校のみ)
`base_prestige`	double	他校のみ。基準名声(毎年ランダムに動く。05 §9.2)
`coach_q`	double	他校のみ。指導体制の良し悪し。±0.30(05 §9.2)

RivalRunner(他校の選手・軽量)

項目	型	説明
`id`	int	
`a0` / `gap`	double	入学時の力 / 伸びしろ(05 §1)
`prog`	double	前年度までの累計進捗率(05 §9.3)。0〜0.95
`growth_type`	GrowthType	
`profile`	enum	型(バランス / 持久 / スピード / 山)。レース時に10能力へ復元する(04)
`grade` / `enroll_year`	int	
`origin`	出身県または出身国	
`name` / `kana` / `appearance`		

4.4 PlayerTeamState(自校の運営状態)

項目	型	範囲・説明
`funds`	int	万円。0以上(マイナスは不可)
`facilities`	map<FacilityId, int>	1〜5
`staff`	list<Staff>	
`morale`	int	0〜100
`captain_id`	int?	
`manager_name`	string	監督名。8文字まで。既定『監督』(08 R-13)
`annual_plan`	{ focus, target_race, peaking_mode }	年度初めに決定。`focus`：`Focus`（推奨プランのテンプレート。08 R-04）。`target_race`：`RaceKind`（`Fall` / `National` / `NewYear`。**表示と予測の注目大会にだけ使い、規則には作用しない**。既定 `NewYear`）。`peaking_mode`：`map<RaceKind, PeakingMode>`（既定: 第一〜第三戦=Standard、予選会=Short、記録会=None）
`monthly_plan`	map<Group, ThemeId>	今月の計画
`custom_weeks`	map<Group, Week[4]>?	`th.custom` 選択時のみ
`guidance_points`	int	月4。月初に4へ戻る(繰り越さない)。個別指導(1人1ポイント。08 R-06)とイベントの `guide` が使う
`individual_menu`	map<runner_id, { until_abs_week, menu }>	個別指導で指定したメニュー(ポイント練習2つ + 走行量)。今週から月末まで有効(03 §5.7)。月をまたがない
`talked`	list<runner_id>	今月に面談した選手(1人月1回。03 §7.2 M3)。月初に空
`scout_points`	int	6〜2月の毎月 +4(ランクの追加分は6月)。**同じ年度の中では繰り越し**、3月に0(08 R-08)。使い方は 05 §7
`coach_rank`	int	監督ランク(評価ポイント累積で上昇)

項目	型	範囲・説明
`ob_network`	int[48]	添字1〜47 = 県ごとのOB人数。添字0 = 海外のOB人数
`tickets`	{ stock, bank, last_used_year, plan_anchor_year, plan_granted_years, milestones, pending, log }	07 §3.1。`stock`：0〜`K_TICKET_STOCK_MAX`。`bank`：繰り延べた券（`stock` に空きができたら移す。失効しない）。`last_used_year`：最後に宣言した年度（年1回の制限用）。`pending`：宣言済みの招聘 `{ kind, enroll_year, prefecture_id? }` または null。`plan_anchor_year`：`GoldenPlan` の付与の起点年度（未確認なら null）。`plan_granted_years`：付与を済ませた年度。`milestones`：獲得済みの節目（07 §4）。`log`：入手・使用の履歴（`TicketSource` 付き）

個性・素養から決まる派生値（保存しない）：`gm[10]`（能力別の XP の乗数）・`bond_k`（絆の効き）・`ind_mult`（個別指導の週の効率）。`style` と `gifts` から計算する（10 §5）。

県別OBネットワークのレベル（派生）：`level(p) = min(5, floor(sqrt(ob_network[p])))`。OB 1人でLv1、4人でLv2、9人でLv3、16人でLv4、25人でLv5。効果は 05。

4.5 Staff（スタッフ）

項目	型	説明
`id`	int	
`role`	StaffRole	
`name`	string	
`level`	int	1〜5
`salary`	int	万円/年

4.6 ScoutCandidate（スカウト候補）

候補は**軽量なデータ**（`PoolCandidate`）で毎年520人ほど生成し、入学が確定したときに `Runner`（自校）または `RivalRunner`（他校）へ展開する（05 §6）。次の項目を持つ。

項目	型	説明
`candidate_id`	int	
`a0` / `gap`	double	入学時の力 / 伸びしろ。`Q = a0 + 0.6×gap`、`V = a0 + 0.3×gap`（派生）
`growth_type` / `profile`		成長タイプ / 型
`origin`	出身県または出身国	出身高校を含む
`look_seed`	uint	外見・名前・10能力の展開・**個性・素養**用のシード（個性と素養は、`master_seed`・`look_seed`・`profile` から決定的に作る（10 §4）ので、候補には保存しない）
`candidate_year`	int	入学予定の年度
`status`	ScoutStatus	
`interest`	int 0〜100	自校への好感度
`rival_interest`	map<int, int>	他校ID → 好感度
`info_level`	map<InfoGroup, int>	0〜3。`InfoGroup` = `Basic`（持ちタイム等）/ `Stats`（能力）/ `Potential`（素質）/ `Body`（故障しやすさ・成長タイプ）
`rating`	int 1〜5	評価★

4.7 OBRecord（卒業生・軽量データ）

項目	型	説明
`runner_id`	int	
`name` / `kana`	string	
`origin`	出身県または出身国	

項目	型	説明
`high_school_name`	string	出身高校の**文字列テーブルのキー**（`hs.{id}.name`。高校がない・留学生は空）。表示のときに引く（09 §12.3）
`school_id`	int	在籍した大学
`enroll_year` / `grad_year`	int	
`final_stats`	int[10]	卒業時の能力
`skills`	list<string>	
`pb`	{ t5000, t10000, t_half }	卒業時の導出値（08 R-03）
`career`	{ race_wins, section_wins, records }	
`career_path`	CareerPath	
`hall_of_fame`	bool	殿堂入り
`role` / `role_until`	`None` / `Coach` / `ScoutHelper` / `Sponsor`、期限（絶対週）	OBとしての支援（06 §8。期限が来たら `None`）
`recruit_source`	RecruitSource	招聘選手の印（将来のレジェンド・対戦のルール用。07 §11）
`style` / `gifts`	StyleCode / list<Gift>	在籍中の個性と素養（殿堂・レジェンド用。素養は判明したものだけ表示）
`appearance`	Appearance	

- **保存するのは自校の OB だけ**。他校の卒業生は卒業した時点で破棄する（世界の更新は校ごとの要約で足りる。サイズの理由は 09 §5.2）。

4.8 RaceRecord（大会結果）

項目	型	説明
`race_kind`	RaceKind	
`year`	int	開催年度
`results`	list<TeamResult>	順位順

TeamResult: `{ school_id, rank, total_ds, kuriage, sections: [{ runner_id, time_ds, section_rank, cumulative_rank, instruction, events }] }`。`events`：区間内の出来事（`Brake` `Stall` `Geki` `Kuriage` `Boost`）。**保存するのは、自校は全項目、他校は `{ school_id, rank, total_ds }` だけ**（区間・個人の記録は、区間記録・大会記録 `Records` の更新分を除いて残さない。09 §5.2）。

Records（記録）: `course_records[race_kind][section_index] = { school_id, runner_id, time_ds, year }`（区間記録。他校の走者の `runner_id` は 0）。`team_records[race_kind] = { school_id, total_ds, year }`（大会記録）。実装（`WorldSim.History`。09 §17.7）：予熱の年度から全校の結果で更新する。**初期の記録がないとき、最初の 1 回は基準として黙って入れ（更新でない）、以後は厳密に速いときだけ更新**（同タイムは更新でない）。新規ゲームの区間記録は、年度 1 の前に下の『初期の区間記録』へ置き換わる（L-68）。保持者は、その区間の最速チームのうち順位の先頭。大会記録は繰り上げのない優勝チームの総合タイム。予選会は含めない。`race_records` は全校の『学校 ID・合計タイム・繰り上げ』を順位順に（優勝校 = 先頭）。

初期の区間記録（L-68。新規ゲームのみ）：予熱の 15 年度は自校と関係なく、そこでできた区間記録（他校のエースが出した記録）で最初から埋まっていると、現実味がなく、記録を狙う楽しみも減る。そこで**新規ゲームでは、年度 1 の最初の大会の前に、区間記録を次の表の値に置き換える**（予熱の中でできた区間記録は消える。大会記録 `team_records` と順位表 `race_records` は予熱のまま）。保持者は**『歴代の記録』**：`school_id = -1`（学校でも走者でもない）・`runner_id = 0`・`year = -100`（予熱の年度 −14〜0 と区別する印）。画面は `school_id < 0` のとき学校名の代わりに文字列 `record.legacy_holder`（『歴代の記録』）を出し、走者名と年度は出さない（View は `IsLegacy` / `PrevIsLegacy`）。以後は通常の規則（厳密に速いときだけ更新）で、年度 1 の大会から他校も自校も更新する。更新される前の保持者が『歴代の記録』のとき、`RecordUpdate.PrevSchoolId = -1`・`PrevYear = -100`。**既存のセーブは変えない**（履歴をそのまま読む。`school_id = -1` は保存の形として許される）。値は実在の公表値の水準（2026 年 10 月時点。資料により数秒〜2 分の差がある。**名前は載せない**。製品化の前に一次資料で確かめ直す）。区間の距離はゲームのコース（04 §2）と同じ。

大会	区間	距離(km)	初期値(ds)	表記
第一戦	1	8.0	13500	22:30
第一戦	2	5.8	9270	15:27
第一戦	3	8.5	13760	22:56
第一戦	4	6.2	10300	17:10
第一戦	5	6.4	10550	17:35
第一戦	6	10.2	16970	28:17
第二戦	1	9.5	16180	26:58
第二戦	2	11.1	18610	31:01
第二戦	3	11.9	19660	32:46
第二戦	4	11.8	19830	33:03
第二戦	5	12.4	21010	35:01
第二戦	6	12.8	22070	36:47
第二戦	7	17.6	29710	49:31
第二戦	8	19.7	33320	55:32
第三戦	1	21.3	36280	1:00:28
第三戦	2	23.1	39090	1:05:09
第三戦	3	21.4	35650	59:25
第三戦	4	20.9	36000	1:00:00
第三戦	5	20.8	40360	1:07:16
第三戦	6	20.8	34070	56:47
第三戦	7	21.3	36430	1:00:43
第三戦	8	21.4	38250	1:03:45
第三戦	9	23.1	40350	1:07:15
第三戦	10	23.0	40700	1:07:50

- **年度 1 の更新のしかた(測定。`SimCli records <world|Elite|Mid|Low> 60 45 --reset`)**: ゲームのコースは平地が基本(5 区の登りと 6 区の下りだけ補正)で、実在のコース固有の遅さを持たないので、実在の記録が遅めの区間は簡単に破られる。年度 1 の大会で破られる確率は、第三戦の 6・8・9・10 区(98〜100%)、第二戦の 6 区(100%)・1 区(72%)、第一戦の 1 区(98%)・6 区(65%)・4 区(50%)など約 9 区間が半分以上で、残りは数年かけて破られる(10 年度以内に 8 割前後)。**ほとんど破られないのは、第一戦の 2 区(45 年度間そのままが 98%)、第三戦の 3 区(100%)・1 区(67%)**。この『年度 1 の更新ラッシュ』は仕様(予熱の 15 年度は自校と無関係なので、初年度に消えるのは受け入れる)。45 年度の間の 1 区間あたりの更新は約 3 回(自校の走者は強豪で 0.5〜0.9 回・弱小で 0.1〜0.5 回)。
- **大会記録(総合タイム)**は初期値を置かない(**予熱のまま。**決定。L-68 ②)。区間記録だけが実在の水準で、総合記録はゲームの世界の水準のままになる。

4.9 SaveData

項目	型	説明
`schema_version`	int	1から。**破壊的な変更(意味の変更・削除・改名・型・列挙の読み替え)のとき増やし**、移行関数を用意する。**追加だけの変更**(既定値のある新しい項目)は版を上げず、既定値の表を読み込みのたびに適用する(09 §6)
`scenario`	{ id, difficulty?, player_school_id }	`id`：新規ゲーム開始時に選んだ `ScenarioDef`(3.11)。`difficulty`：本体の4つのとき `Elite` / `Mid` / `Low` / `Club`、シナリオパックなら null
`run`	{ invites_declared, cleared }	`invites_declared`：このセーブで招聘(特別招聘券の使用)を宣言した総数。`cleared`：クリアしたら `{ year, no_invite }`（`no_invite` はクリア時点の `invites_declared = 0`）。**セーブ単位**で、実績の判定に使う

項目	型	説明
`clock`	GameDate	
`rng`	{ algorithm, master_seed }	`algorithm = "pcg32-keyed-1"`。**乱数の状態は保存しない**(鍵つきストリーム。03 §3・09 §8)。`master_seed` は uint64(JSON では文字列)
`counters`	{ next_runner_id, next_staff_id, … }	
`schools`	list<SchoolState>	36校
`player_team`	PlayerTeamState	
`runners`	map<int, Runner>	在籍選手・候補・他校の主力
`ob_records`	list<OBRecord>	
`history`	{ race_records, course_records, team_records, hall_of_fame }	
`high_schools`	list<HighSchool>	世界生成時のもの
`event_state`	{ fired, year_counts, runner_ev, pair_cd, mods, bench, forced, log }	イベントの記録・補正・制限(06 §12)
`settings`	{ spectate_mode, auto_safety_f, event_auto, plan_auto, scout_auto, school_look, … }	プレイヤー設定(セーブごと。ゲームの進行に関わるもの)。`spectate_mode`：フル / ダイジェスト / 結果のみ(04 §9.3)。`auto_safety_f`：75 / 85 / 0(無効)。`event_auto`：イベントを『おまかせ』で解決(06 §7.4)。`plan_auto`：月間計画を推奨プランで自動に確定(既定 false。08 R-04)。`scout_auto`：スカウトをおまかせにする(既定 false。08 R-08)。`school_look`：{ `uniform_pattern, cheer_song, bgm_set` }(自校の見た目。表示の層だけ。07 §9.2。校章は `SchoolState.emblem_id`)。見え方・音・操作の設定は `DeviceSettings`(§3.12)
`pack_snapshot`	list<string>	セーブ時点で所持していたパック(参考情報)

- **サイズ上限(09 §5 で確定)**: 圧縮後 **100年度で約 285 KB**(目標 1 MB・上限 3 MB)。自校の OB のみ・他校の大会記録は順位と合計タイムのみ・ログは各50件に刈り込む(09 §5.2)。刈り込みなしでは 100 年度で 3.2 MB に達し、上限を超える。
- **保存の形式**: コンテナ `EKSV`(ヘッダ + gzip の JSON + CRC32)・A/B の2世代・原子的な書き込み(09 §4)。`SaveMeta`・`lineage_id`・`save_counter` はヘッダに持つ。JSON の規則(キーの順・整数・`double` の丸め)は 09 §5.3。

---

5. 不変条件(Invariants)

自動テストと、セーブ読み込み時の検証に使う。**読み込み時の扱い**(R 修復 / F 致命 / W 警告)と修復の内容は 09 §7.1。

ID	内容
I-01	`stats[i] ≤ potential[i] ≤ 100`、`stats[i] ≥ 0`
I-02	`origin_type = Japan` ⇒ `prefecture_id ∈ 1..47` かつ `high_school_id ≠ null` かつ `country_id = null`。`Overseas` ⇒ `country_id ≠ null` かつ `prefecture_id = null`
I-03	在籍中の選手は `grade ∈ 1..4`。年度末(3月)に `grade = 4` の選手は卒業して OBRecord に変換される
I-04	`injury.severity = None` ⇔ `injury.weeks_left = 0`
I-05	`fatigue ∈ [0, 100]`、`motivation ∈ [1, 5]`、`form ∈ [-2, +2]`
I-06	1人の選手が属する大学は高々1つ。`group` は自校の選手のみが持つ(保存される編成。毎週は決め直さない。08 R-04)
I-07	自校の在籍人数は最小5(同好会開始時)〜最大60
I-08	絆は対称。`bonds` は ID の小さい側にだけ保持する
I-09	`captain_id` は自校の在籍選手で、`grade ≥ 3`
I-10	エントリーした選手は自校の在籍選手。留学生の登録数・出走数は `RaceRules` の上限以下
I-11	実行時IDは一意で再利用されない

ID	内容
I-12	`funds ≥ 0`。施設レベルは1〜5
I-13	各校の `roster` は40人以下。他校の `RivalRunner` は `0 ≤ prog ≤ 0.95`。1学年の入学人数は定員(05 §2)以下
I-17	上位級(`V ≥ V_TOP`)の入学人数は、その年度の奨学金枠(05 §2)以下
I-14	`tickets.stock ∈ [0, K_TICKET_STOCK_MAX]`、`stock < K_TICKET_STOCK_MAX ⇒ bank = 0`。入手の合計 = 使用の合計 + `stock` + `bank`(失効しない)。券の使用は同一年度に `K_TICKET_PER_YEAR` 回まで。在籍する `recruit_source = Ticket` の数 + `pending`(0/1)≤ `K_TICKET_ACTIVE_MAX`
I-15	招聘選手の生成値は SpecialRecruitBand(3.9。07 §6.6 の S-1〜S-5)を満たす。`pending` は宣言期間内にだけ作られ、入学者確定時に必ず消化される。宣言は取り消せない
I-16	`ob_network` の合計 = 自校の OBRecord 件数(`career_path` が確定した卒業生)
I-18	`run.invites_declared` ≥ これまでに入学した `recruit_source = Ticket` の選手数(宣言して入学前のものを含む)。`run.cleared` は一度設定したら変更しない。`cleared.no_invite = true` ⇒ クリア時点で `invites_declared = 0`
I-19	`OBRecord.role ≠ None` ⇒ `role_until` が未来。OBの役割は1人1つ。`event_state.mods` の `until` は未来(切れた補正は消す)。`Runner.ctr` のキーは 06 §10 の一覧のみ。1人の選手が1週に当事者になるイベントは1件まで(06 §7.2)
I-20	`individual_menu` のキーは自校の在籍選手。`until_abs_week` はその月の最終週以下。**故障が起きた選手の指定は破棄する**(ポイントは戻らない。03 §5.1 で故障が最優先)。`talked` は自校の在籍選手。`guidance_points ∈ [0, K_UI_GUIDE_POINTS(4)]`。`annual_plan.focus ∈ Focus`、`target_race ∈ {Fall, National, NewYear}`。`scout_points ≥ 0`(3月の年度末処理で0)(08 §8)
I-21	`style` は 3 文字で、型 ∈ {S,B,L}・走法 ∈ {P,F,S}・展開 ∈ {C,E,L}。`gifts` は {Climber, Descender, Ace} の重複なしで 2 個以下(10 §2)

6. 出身県・留学生に関する生成規則(データ定義の範囲)

候補の出身県の抽選(入学候補を生成するとき)

1. 日本人の候補は、`supply_weight`(3.2)に比例して県を抽選する。スカウトの対象地方が指定されている場合は、その地方の県の中で重みを正規化して抽選する。
2. 県内の高校を `strength` に比例して抽選し、その高校を `high_school_id` にする。
3. 強豪県(`elite`)は、候補の能力の基準値を **+3** する(03・05で適用)。県による能力の固定補正は、これ以外に設けない。
4. 留学生の候補は、出身国を `Country` から抽選し、`prefecture_id` と `high_school_id` は設定しない。

表示

- 選手カード・名簿・イベント・OB名簿に、出身県(留学生は出身国)と出身高校を常に表示する。

7. 次の文書への引き継ぎ

引き継ぎ先	内容
03	練習メニューの `gain` / `load` の数値、月のテーマの週構成、疲労・故障・調子・成長の手順と計算例、乱数の使い方、強豪県の能力基準値補正
04(完了)	区間タイム計算、ペース配分(失速)、特殊能力の数値(区間・展開・精神)、順位決定・同着、留学生枠の検証、出場権(`YearRankTop`・`SeedPlusQualifier`)、当日変更・指示
05	距離帯と県別OBネットワークの効果、候補生成、スカウト行動、高校生成、入学・卒業・進路、他校の更新、施設・スタッフの効果
06(完了)	イベントの条件式と効果の形式、出身県・同郷・帰省のイベント、OBの役割、特殊能力の覚醒
07(完了)	特別招聘券の入手・使用・生成の手順、課金方式、購入状態の扱い、見た目・シナリオ、券あり/なしのクリア年数の測定
08(完了)	画面・入力モデル・進行制御(止まる場所)・画面のための規則(推奨プラン・月末の予測・個別指導・スカウトの競合度と枠・前評判・月末レポート)。`DeviceSettings` / `SaveMeta` / `individual_menu` を追加
10(完了)	個性(型・走法・展開)と素養(登坂・下り・エース)。`Runner.style` / `gifts` / `gift_revealed`、不変条件 I-21、生成・練習・レース・特殊能力の取得・見せ方

引き継ぎ先	内容
09(完了)	セーブの形式・世代・同期・マイグレーション・検証と修復・乱数の鍵・`LocalState`・刈り込み・整形・技術スタック・M0 の着手条件。`rng` / `schema_version` / `SaveMeta` / `ProfileData.entitlements` / `DeviceSettings.cloud_sync` を確定(v1.0)

第3部 育成ルール仕様

原典: `03_training_rules_spec.md`　　原典の版: v0.3　　改版の経緯: 付録 G.3

> 範囲: 自校の在籍選手の**週次の育成**(メニュー決定 → 経験値 → 疲労 → 故障 → 調子)と月次処理。前提: `02_data_spec.md` (データ構造)。他校の育成は 05、レース本体は 04。**検算**: 本書の数値と計算例は、参照モデル `verify/ref_model_03.py` (Python)で検証済み。計算例はそこから出力した値で、そのまま自動テストに写せる。

1. 用語と単位

項目	内容
XP の単位	**0.1XP = 1 `xu`**。コード上は整数の `xu`。表示は ÷10
週	1ターン。`abs_week = (year-1)×48 + month_index×4 + (week-1)`
疲労 F	整数 0〜100。`F_start` (週の初め)/ `F_end` (週の終わり)
負荷 L	その週のメニューの疲労負荷の合計
丸め	すべて「四捨五入(0.5は切り上げ)」`round_half_up(x) = floor(x + 0.5)`

2. 定数表(初期値案。`K_` 接頭辞)

定数	値	意味
`K_XP_SCALE`	3.0	XP全体の倍率(成長速度の調整つまみ)
`K_T_BASE` / `K_T_SLOPE`	20 / 0.6	能力+1に必要なXP `T(stat) = 20 + 0.6×stat` (50→50XP、80→68XP)
`K_ROOM_EXP`	0.7	素質接近補正の指数
`K_LEVELUP_MAX`	3	1能力が1週に上がれる最大値
`K_E_MAX`	2.0	効率補正 E の上限
`K_R_BASE` / `K_R_REC`	0.40 / 0.20	疲労の減衰率 `r = 0.40 + 0.20×REC/100 + …`
`K_R_MIN` / `K_R_MAX`	0.20 / 0.80	r の下限 / 上限
`K_P0`	0.005	故障の基礎確率(週)
`K_P_MAX`	0.5	故障確率の上限
`K_TAPER_L_MAX`	15	負荷がこの値以下の週を「調整週」と数える
`K_SAFETY_F`	85	安全網の発動閾値(設定: 75 / 85 / 無効)
`K_RACE_FATIGUE_PER_KM`	1.6	レース後の疲労加算(km あたり)
`K_CAMP_XP`	1.5	夏合宿(`th.camp`)の週のXP倍率
`K_CAMP_INJ`	1.3	夏合宿の週の故障確率倍率
`K_FORM_SIGMA`	`8.0 × (1.5 - CON/100)`	調子スコアの乱数の標準偏差
調子の閾値	+20 / +8 / −8 / −20	絶好調 ≥20 / 好調 ≥8 / 普通 >−8 / 不調 >−20 / 絶不調
`K_BOND_*`	§7.1	絆の増減

3. 乱数

3.1 アルゴリズム(整数演算のみ。全端末で同じ結果になる)

```
mix64(x):                       # SplitMix64 の最終化。すべて 2^64 の剰余
  x = (x ^ (x >> 30)) * 0xBF58476D1CE4E5B9
  x = (x ^ (x >> 27)) * 0x94D049BB133111EB
  return x ^ (x >> 31)

derive(master_seed, label, a, b):
  x = mix64(master_seed ^ (label * 0x9E3779B97F4A7C15))
  x = mix64(x ^ a)
  x = mix64(x ^ b)
  return x

PCG32(seed):                    # XSH-RR
  inc   = (mix64(seed ^ 0xDA3E39CB94B95BDB) << 1) | 1
  state = 0;  step();  state = state + seed;  step()
  step():  state = state * 6364136223846793005 + inc
  u32():   old = state;  step()
           xs  = (((old >> 18) ^ old) >> 27) & 0xFFFFFFFF
           rot = old >> 59
           return ((xs >> rot) | (xs << ((-rot) & 31))) & 0xFFFFFFFF

uniform()         = (u32() + 0.5) / 2^32              # 開区間 (0,1)
randint(a, b)     = a + floor(uniform() * (b - a + 1))
normal(mu, sigma) = mu + sigma × sqrt(-2 ln u1) × cos(2π u2)   # u1, u2 = uniform() を続けて2回
```

3.2 ストリームの鍵

選手・週ごとに独立のストリームを作る。**選手を処理する順序が結果に影響しない。**

用途	`label`	`a`	`b`
故障	1	`runner_id`	`abs_week`
調子	2	`runner_id`	`abs_week`
スカウト / レース / イベント / 世界の更新 / 世界の生成 / 外見	3 / 4 / 5 / 6 / 7 / 8	各文書で定義。**登録表は 09 §8.2**	

`master_seed` はセーブデータの `rng`（`{ algorithm: "pcg32-keyed-1", master_seed }`。乱数の状態は保存しない）に保持する(02 §4.9・09 §8.1)。

3.3 引く順序

- **故障ストリーム**: 週の初めに故障中なら引かない。そうでなければ ① `uniform()`（発生判定）→ 発生したら ② `uniform()`（重症度）→ ③ `randint(下限, 上限)`（期間。選ばれた重症度の分だけ1回)。
- **調子ストリーム**: `normal(0, K_FORM_SIGMA)` を1回(内部で `uniform()` を2回)。

---

4. マスタ値

4.1 ポイント練習

`gain` は `xu`（0.1XP)。`risk` は故障リスクの加算値、`rb` は疲労減衰率 r への加算値。

ID	gain(能力: xu)	load	risk	rb
`ks.pace_run`	PAC 40 / STA 30 / CON 20	9	0	0
`ks.threshold`	STA 55 / PAC 35 / CON 15	10	0	0
`ks.long_run`	STA 80 / PAC 35 / MEN 30	15	+0.15	0

ID	gain(能力: xu)	load	risk	rb
ks.interval	SPD 80 / STA 25	14	+0.15	0
ks.repetition	SPD 110 / KCK 45	13	+0.35	0
ks.build_up	KCK 65 / PAC 25	10	0	0
ks.hill	CLB 100 / STA 25	12	+0.15	0
ks.downhill	DWN 100	11	+0.35	0
ks.core	DUR 40 / REC 20 / SPD·STA·PAC·KCK 各4	4	−0.15	0
ks.sharpen	KCK 25 / SPD 15	4	0	0
ks.rest	REC 15	0	−0.10	+0.10
ks.time_trial	MEN 55 / SPD 25 / STA 15 / PAC 15 / CON 20	16	+0.15	0
ks.cross_train (故障中のみ)	STA 15	3	0	+0.05

4.2 走行量

ID	週間距離	gain(xu)	load	risk
mil.low	70km	STA 10	2	0
mil.std	110km	STA 18 / PAC 12	6	+0.10
mil.high	150km	STA 30 / PAC 20	10	+0.30
mil.max	180km	STA 42 / PAC 28	15	+0.80

4.3 月のテーマ(4週分の週メニュー)

形式は（ポイント練習1, ポイント練習2, 走行量）。

テーマ	第1週	第2週	第3週	第4週
th.base_build	pace_run, threshold, high	long_run, pace_run, high	long_run, threshold, high	core, pace_run, std
th.speed	interval, build_up, std	repetition, interval, std	time_trial, build_up, std	core, pace_run, low
th.mountain	hill, downhill, std	downhill, threshold, std	hill, downhill, std	core, pace_run, low
th.camp	long_run, threshold, **max**	long_run, hill, max	long_run, interval, max	core, pace_run, high
th.recovery	rest, core, low	rest, core, low	pace_run, core, low	pace_run, core, std
th.custom	プレイヤーが週ごとに指定(§5.5 の検証を通すこと)			

- 基本は「3週負荷 + 1週軽め」。th.camp の全週に K_CAMP_XP と K_CAMP_INJ を適用する。
- **ピーキングは月のテーマではない**(§5.3 の大会連動の自動適用)。

4.4 特別な週メニュー

名前	内容
ピーキング週 (TAPER[1..4])	T1: build_up, sharpen, std(L=20)/ T2: sharpen, pace_run, low(15)/ T3: sharpen, core, low(10)/ T4(大会週): sharpen, rest, low(6)
安全網の週(SAFE)	core, rest, low(L=6)
故障中の週(INJURED)	cross_train, rest, 走行量なし(L=3)

4.5 補正表

やる気 → 効率 **E_motiv**：1→0.85 / 2→0.92 / 3→1.00 / 4→1.08 / 5→1.15

学年 × 成長タイプ → **E_growth** (02/01 §4.3)

学年	Early	Normal	Late
1	1.3	1.0	0.7
2	1.1	1.1	0.9
3	0.8	1.0	1.2
4	0.6	0.9	1.3

疲労 F → 成長効率 `E_fatigue`: F<40: 1.0 / 40≤F<70: `1.0 - 0.15×(F-40)/29` (40→1.0、69→0.85)/ 70≤F<85: 0.7 / F≥85: 0.3

疲労 F → 故障倍率 `m_fatigue`: F<40: 1.0 / 40≤F<70: `1.0 + 0.5×(F-40)/29` (→1.5)/ 70≤F<85: 2.0 / F≥85: 4.0

4.6 施設・スタッフの効果(`Lv` は 1〜5。Lv1・不在は効果なし)

対象	効果
コーチ	全XP × `1 + 0.06×(Lv-1)`
トレーナー	故障確率 × `1 - 0.03×(Lv-1)` / 故障期間 × `1 - 0.03×(Lv-1)` / r + `0.02×(Lv-1)`
栄養士	r + `0.02×(Lv-1)`
寮	r + `0.02×(Lv-1)` / REC のXP × `1 + 0.12×(Lv-1)`
トラック	SPD・KCK のXP × `1 + 0.12×(Lv-1)`
坂道コース	CLB・DWN のXP × `1 + 0.15×(Lv-1)`
トレーニング室	DUR のXP × `1 + 0.15×(Lv-1)` / 故障確率 × `1 - 0.04×(Lv-1)`
医療室	故障期間 × `1 - 0.05×(Lv-1)` (トレーナーと合算して下限 ×0.5)

- **全施設・全スタッフがLv1→Lv5になると、平均的な新入生の卒業時の適性値が約 +3.7**、故障が 0.39 → 0.24 回/人年になる(05 の課題 L-01 への対応。以前の係数は約 +1.2 だった)。

4.7 個性・素養による乗数(10 §5 が確定)

項目	内容	式の位置
`gm[s]` (能力ごとの XP の乗数)	型: スピード型は SPD ×1.20・STA ×0.80 / ロング型は STA ×1.20・SPD ×0.80。展開: 追い上げ型は KCK ×1.20・PAC ×0.80 / 逃げ型は PAC ×1.20・KCK ×0.80。素養: 登坂は CLB ×1.15 / 下りは DWN ×1.15 / エースは MEN・CON ×1.15 と他の能力 ×1.05。**積**で合成(個性がなければ 1.0)	§6 手順 2(`add_s` に掛ける)
`bond_k` (絆の効き)	集団走◎ 0.30 / 普通 0.15 / 単独走◎ 0.05(個性がなければ 0.15)	§6 手順 1(`E_bond`)
`ind_mult` (個別指導の週の効率)	単独走◎ ×1.25 / それ以外 ×1.0	§6 手順 1(E_MAX の前)

- 型・展開の +20% と −20% は**平均すると中立**になるように対称にした(全員バランスの計画で、チームの適性値の個性なしとの差は、個性のみ +0.0〜+0.16 P・素養のみ +0.07 P・合わせて +0.09〜+0.34 P。10 §5.5・§9.1)。素養は『伸びる能力が伸びやすい』分だけ純粋に有利(まれ。10 §3)。
- 素養の**素質の上限**(登坂の CLB +6 など)は、入学時に `potential` へ加える(10 §4)。この章の式は変わらない。

---

5. 週メニューの決定

5.1 優先順位(上が優先)

1. **故障中** → `INJURED` (個別指定も無効)
2. **個別指定**(指導ポイントで上書き)
3. **ピーキング**(大会連動の自動適用。§5.3)

4. **安全網**（`F_start ≥ K_SAFETY_F` のとき `SAFE`。§5.4）
5. **月間計画**のテーマの該当週（グループ単位）

5.2 月間計画からの展開

- 月初に、各グループ `A/B/C` が選んだテーマ（`monthly_plan`）から、第 `week` 週の `(ks1, ks2, mil)` を取る。
- 選手が途中でグループを移った場合は、移った週から新しいグループのテーマを使う。

5.3 ピーキングの自動適用（大会連動）

- 対象: **その大会のエントリー（登録メンバー）に入っている自校の選手**。エントリー締切は大会の4週前。
- 大会週を `t` として、`t-3 … t` の4週に `TAPER[1..4]` を当てる。
- `peaking_mode`（大会ごとに設定）

モード	内容
`Standard`（既定。第一〜第三戦）	`TAPER[1..4]`
`Short`（予選会の既定）	`TAPER[3..4]` のみ（`t-1, t`）
`None`（記録会の既定）	適用しない

- **重なり**: 直前の大会週 `p` とある場合、`start = max(t-3, p+1)` から `t` まで、**後ろから詰めて**当てる（窓が3週なら `TAPER[2..4]`、2週なら `TAPER[3..4]`）。
  - 例: 第一戦（10月2週）と第二戦（11月1週）は3週差 → 第二戦は `TAPER[2..4]`。
  - **効果の実測**（L-55。05 §14.11）: 第二戦の出走メンバーの調子の平均は +0.9（他校の想定は +1.4。第一戦・第三戦は当日変更の効果もあって +1.9）。時間にして約 0.4%（力にして約 1.7）他校より不利で、第二戦の優勝率は第一戦・第三戦より 0.2〜0.4 低い。この不利を他校と同じにすると、実体の自校が抽象より強くなりすぎる（弱小のクリア 19 年度）ので、規則は変えない。
- エントリーから外れた選手（補欠で当日変更されなかった者を含む）も、登録メンバーである限り同じ週メニューになる。
- **予選会**（`Short`。04 §3.2 の 7）: 登録メンバーは第 26 週（予選会の週）に `TAPER[4]` だけ。第二戦の調整が同じ週（第 26 週は `T2`）から始まる選手でも、予選会のほうを先に当てる（第 27・28 週は第二戦の `T3・T4`）。

5.4 安全網

- `F_start ≥ K_SAFETY_F`（既定85）の選手は、その週のメニューを `SAFE` に置き換える。個別指定がある選手は対象外。
- 設定で「慎重（75）/ 標準（85）/ 無効」を選べる。
- 夏合宿の通常の経過（週末の疲労は最大でおおよそ76）では発動しない（§11）。

5.5 個別指定・`th.custom` の検証

ルール	内容
1	ポイント練習の2枠は**異なるID**にする
2	`ks.cross_train` は故障中の自動メニュー専用。指定不可
3	`ks.rest` は他のどのメニューとも組める
4	走行量は必須（故障中を除く）
5	検証に失敗した指定は無視して、テーマの週メニューに戻す（UI側で事前に弾く）

5.6 故障中

- `INJURED` を使う。故障中の選手は、XP は `INJURED` のメニュー分だけ得る。大会に出走できるかは 04。

5.7 個別指導（指導ポイントの使い方。08 R-06 が確定）

月の指導ポイント（`guidance_points`。月初に4。繰り越さない）を使って、次の2つを行える。

操作	費用	内容
メニューの指定	1ポイント / 人	今週から**月末まで**、その選手の週メニューを (ポイント練習2つ + 走行量) で指定する(§5.5 の検証を通す)。プリセット: ケア(`SAFE` と同じ)/ 調整(`ks.sharpen` + `ks.pace_run` + `mil.low`)/ 自由編集。 `PlayerTeamState.individual_menu` に持つ。最初の週が解決される前なら取り消せる(ポイントは戻る)。**指定したあとに故障が起きたら、指定は破棄**(ポイントは戻らない)
面談	1ポイント / 人	やる気 +1(§7.2 M3)。1人月1回(`talked`)

- §5.1 の優先順位のとおり、**個別指定はピーキングより優先**され、**安全網の対象外**(§5.4)。UI は上書き・安全網が働かないことを確認で知らせる。
- イベントの `guide` 効果(06 §5.2)も同じポイントを使う。
- **走法が『単独走◎』の選手は、個別指導の週の効率が ×1.25**(`ind_mult` 。§6 手順1。10 §5.2)。ポイントを誰に使うかの判断材料になる。

---

6. 週の解決手順(`WeekResolve` 、選手ごとに独立)

入力: 選手 `r` 、決定した週メニュー `(ks1, ks2, mil)` 、施設・スタッフのレベル、 `month_index` 、 `is_camp` (`th.camp` の週か)、 `abs_week` 、 `master_seed` 、 `bond_avg` (§7.1)。

手順

1. **効率 E**

```
E = E_fatigue(F_start) × E_motiv(motivation) × E_growth(grade, growth_type)
    × (1 + 0.06×(coach_Lv-1)) × E_bond
E_bond = 1 + bond_k × bond_avg / 1000      # bond_avg: 同グループの仲間との絆の平均(0〜1000)。bond_k: 走法で 0.30 / 0.15 / 0.05(個性なしは 0.15。§4.7)
合宿の週: E ×= K_CAMP_XP(夏男は更に ×1.1)
E ×= 補正 E_mult(06 §6。チームと本人の積。0.8〜1.2)
個別指導の週(§5.7): E ×= ind_mult          # 単独走◎ は 1.25、それ以外は 1.0(§4.7)
E = min(E, K_E_MAX)
```

2. **XP の付与**(能力ごと)

```
g[s]   = ks1.gain[s] + ks2.gain[s] + mil.gain[s]              # xu
room_s = (1 - stat_s / potential_s) ^ K_ROOM_EXP               # stat ≥ potential なら対象外
add_s  = round_half_up(g[s] × K_XP_SCALE × E × facility_mult(s) × room_s × gm[s])      # gm[s]: 個性・素養の乗数(§4.7。なければ 1.0)
xp_s  += add_s
```

`facility_mult(s)` : SPD・KCK=トラック、CLB・DWN=坂道、DUR=トレーニング室、REC=寮(§4.6)。それ以外は 1.0。 `ApplyXp` (§8。イベントなど)は `gm` を掛けない(E・room と同じ)。

3. **レベルアップ**(能力ごと、最大 `K_LEVELUP_MAX` 回)

```
while 回数 < K_LEVELUP_MAX かつ stat < potential:
    T = round_half_up(10 × (K_T_BASE + K_T_SLOPE × stat))     # xu
    if xp < T: break
    xp -= T;  stat += 1
stat == potential になったら xp = 0
```

4. **疲労**

```
r = K_R_BASE + K_R_REC×REC/100 + 寮・栄養士・トレーナーの加算 + ks1.rb + ks2.rb
    + (回復力○: +0.05) - (夏バテ かつ month_index 3〜5: -0.05)
r += 補正 r_add(06 §6。和。±0.06)
r = clamp(r, K_R_MIN, K_R_MAX)
L = ks1.load + ks2.load + mil.load                                # 走行量なしは0
F_end = clamp(round_half_up(F_start × (1 - r) + L), 0, 100)
```

5. **故障**(週の初めに故障中でないときだけ判定)

```
m_load = max(0.5, 1 + ks1.risk + ks2.risk + mil.risk)
m_dur  = 1.6 - DUR/100
m_fac  = (1 - 0.04×(gym_Lv-1)) × (1 - 0.03×(trainer_Lv-1))
m_sk   = (鉄の脚: 0.5) × (故障癖: 1.5)
p = clamp(K_P0 × m_fatigue(F_start) × m_load × m_dur × m_sk × m_fac × (合宿: K_CAMP_INJ) × 補正 inj_mult(06 §6。積。0.5〜2.0), 0, K_P_MAX)
u1 = uniform()  →  u1 < p なら故障発生
発生時: u2 = uniform()
  F_start < 85: u2<0.6 Minor / <0.9 Moderate / それ以外 Severe
  F_start ≥ 85: u2<0.4 Minor / <0.8 Moderate / それ以外 Severe
期間(基準): Minor randint(1,2) / Moderate randint(4,8) / Severe randint(12,24)
期間 = max(1, round_half_up(基準 × max(0.5, 1 - 0.05×(medical_Lv-1) - 0.03×(trainer_Lv-1))))
```

週の初めに故障中なら、`weeks_left -= 1`。0になれば `None` に戻る(期間は「`INJURED` のメニューで過ごす週数」)。

6. **調子**

```
調整週か: L ≤ K_TAPER_L_MAX かつ 故障中でない → taper_weeks += 1、そうでなければ 0
シャープニング履歴: 直近3週に ks.sharpen を含む週があるか(sharpen_recent)
peaking = peaking_bonus(taper_weeks, sharpen_recent, skills)
score   = (50 - F_end) × 0.5 + peaking + normal(0, K_FORM_SIGMA)
form    = 閾値表で -2〜+2。故障中は -2
```

`peaking_bonus`

taper_weeks	シャープニングあり	なし
0	0	0
1	+3	+2
2	+6	+3
3	+8	+4
4以上	−8(キレ落ち)	−8

調整力○は taper_weeks 1〜3 のとき +4。結果は [−8, +12] に収める。**06 の `form_add` は、この peaking に足してから [−8, +12] に収める**(08 L-28 で確定)。

7. **書き戻し**: `fatigue = F_end`、`form`、`taper_weeks`、`sharpen_hist`、`xp`、`stats`、`injury`。
8. **週レポート**: `{ runner_id, menu, E, xp_add[10], levelups[10], F_start, F_end, injury(発生した場合), form, 安全網の発動 }` をUI用に残す。

全体の順序

全選手の `WeekResolve` を実行 → 絆の更新(§7.1)→ 大会があれば 04 の手順 → 大会後処理(§8)→ **EventPhase(06 §7)**→ 第4週なら月次処理(§7)。

06 が追加するもの: ①補正(`E_mult` `r_add` `inj_mult`)を手順1・4・5で受け取る。`form_add`(和。±4)は手順6の peaking に足してから −8〜+12 に収める(08 L-28 で確定)。②選手ごとのカウンター(06 §10)を更新する: `hill_weeks`(手順1のあと。2つのポイント練習のどちらかが `ks.hill` で故障中でない週)、`injuries`・`injury_free`(手順5)、`camps`・`camp_overload`(8月の第4週の終わり。合宿の週の疲労の最大値が85以上なら後者)。

---

7. 月次処理(`MonthEnd`、各月の第4週の終わり)

7.1 絆(単位: 0.1。範囲 0〜1000)

規則	内容
週の増加	同じグループで、2人とも故障中でない組:**+10**。同じ出身県(日本)なら **+11**、同じ出身高校なら **+12**(毎週)
月末の減少	異なるグループの組で絆>0 のものは **−10**(下限0)
初期値(入学時)	同じ出身県の部員同士 **+100**、同じ出身高校は更に **+100**

規則	内容
保持	絆は ID の小さい側にだけ保持する(02 I-08)。0の組は持たない
`bond_avg`	選手と同じグループの仲間全員との絆の平均(未保持は0)

7.2 やる気(1〜5。すべて決定的な規則)

ID	条件	変化	タイミング
M1	疲労85以上の週が2週連続	−1	その月の最終週(月1回まで)
M2	重傷が発生	−1	発生時
M3	個別指導の面談を受けた	+1	実行時(1人月1回)
M4	チームの士気 ≥ 70 かつ やる気 ≤ 2	+1	月末
M5	チームの士気 ≤ 30 かつ やる気 ≥ 4	−1	月末
M6	その月に能力が合計2以上アップ かつ やる気 ≤ 3	+1	月末

- 大会の結果(出走・順位)による増減は 04、イベントによる増減は 06 で定義する。
- すべて `AddMotivation(runner, delta, reason)` を通し、結果は 1〜5 に収める。

---

8. 他文書から呼ばれる操作(フック)

操作	内容
`ApplyXp(runner, stat, xu)`	04・06 から。`E・room` を掛けずに XP を足し、§6 手順3のレベルアップだけを実行する
`AddMotivation(runner, delta, reason)`	やる気を増減(1〜5)
`AddFatigue(runner, amount)`	疲労を加算して 0〜100 に収める
`ApplyRaceFatigue(runner, section_km)`	`AddFatigue(round_half_up(section_km × K_RACE_FATIGUE_PER_KM))`
`Injure(runner, severity)`	イベント用。期間は §6 手順5の式で決める

大会週の順序: ①週の練習を解決(§6。調子が決まる)→ ②大会(04)が `form` を使ってタイムを計算 → ③大会後に `ApplyRaceFatigue` と、04 が定めるXP・やる気の更新。

---

9. 境界・例外

状況	扱い
`stat ≥ potential`	その能力はXPを得ない。`xp` は 0
1週に複数回レベルアップ可能	能力ごとに最大 `K_LEVELUP_MAX`。残りの `xp` は次週に持ち越す
疲労の計算結果が範囲外	0〜100 に収める
走行量なし(故障中)	`L` に走行量を足さない。`mil.risk` も0
故障中の選手に故障判定	しない(乱数も引かない)
故障発生した週	その週のメニュー・XPはそのまま有効。次週から `INJURED`
故障期間	1以上(丸めて0になる場合は1)
同じ週に複数大会	想定しない(大会は同じ週に1つ)
新入部員	疲労20、やる気3、調子0、`taper_weeks` 0(05で確定)
在籍選手がいない	何もしない

---

10. 計算例(テストベクタ)

すべて `verify/ref_model_03.py vectors` の出力。浮動小数の比較は 1e-6 の許容誤差。

TV-1 乱数

入力	期待値
`derive(123456789, 1, 7, 100)`	`10206515190289864039`
上記シードの `PCG32` の `u32()` ×3	`2889221095`, `1724165226`, `319155939`
同 `uniform()` ×2	`0.672699207`, `0.401438499`
同 `normal(0, 1)`	`-0.725096155`
`derive(123456789, 2, 7, 100)`	`12557608959185654001`

TV-2 テーブル

項目	値
`E_fatigue(F)`	F=0:1.0 / 39:1.0 / 40:1.0 / 55:0.9224 / 69:0.85 / 70:0.7 / 84:0.7 / 85:0.3 / 100:0.3
`m_fatigue(F)`	F=0:1.0 / 39:1.0 / 40:1.0 / 55:1.2586 / 69:1.5 / 70:2.0 / 84:2.0 / 85:4.0 / 100:4.0
`T(stat)` (xu)	30:380 / 50:500 / 55:530 / 80:680 / 99:794
調子の境界	score=20→+2 / 19.9→+1 / 8→+1 / 7.9→0 / −7.9→0 / **−8→−1** / −19.9→−1 / **−20→−2**
`peaking_bonus` (taper_weeks, シャープニングあり/なし)	0:0/0 / 1:3/2 / 2:6/3 / 3:8/4 / 4:−8/−8 / 5:−8/−8
`peaking_bonus` (調整力○, あり)	0:0 / 1:7 / 2:10 / 3:12 / 4:−8

TV-3 1週の解決

入力: stats=58,60,56,52,45,44,55,50,52,54、potential=[80,82,78,74,70,68,75,72,70,74]、xp=すべて0、F_start=40、やる気3、2年生、Normal、特殊能力なし、施設・スタッフはすべてLv2、`bond_avg`=0。メニュー=`th.base_build` 第1週(pace_run, threshold, high)、非合宿、`month_index`=0、`master_seed`=2026、`runner_id`=1、`abs_week`=0。

項目	期待値
E	1.166(= 1.0 × 1.0 × 1.1 × 1.06 × 1.0 × 1.0)
r	0.564(= 0.40 + 0.20×0.52 + 0.06)
L	29(= 9 + 10 + 10)
F_end	46(= round(40×0.436 + 29) = round(46.44))
XP 付与(xu)	STA 160 / PAC 137 / CON 49、他は0
レベルアップ	なし(STA の閾値は 60→T=560xu、PAC は 56→T=536xu)
故障確率 p	0.006416(= 0.005 × 1.0 × 1.30 × 1.06 × 0.9312)。乱数は引くが発生しない
調子スコア	peaking=0、noise=5.672137、score=7.672137 → 調子=0(普通。好調の閾値 8 にわずかに届かない)

TV-4 疲労の推移(基礎ビルド4週、F=40開始、REC=52、Lv2)

週	L	r	F_end
1	29	0.564	46
2	34	0.564	54
3	35	0.564	59
4	19	0.564	45

TV-5 ピーキング(F=60開始、CON=50、REC=52、Lv2)

週	メニュー	L	r	F_end	taper_weeks	peaking	調子スコアの決定部分
T1	build_up, sharpen, std	20	0.564	46	0	0	2.0
T2	sharpen, pace_run, low	15	0.564	35	1	+3	10.5
T3	sharpen, core, low	10	0.564	25	2	+6	18.5
T4	sharpen, rest, low	6	0.664	14	3	+8	26.0

(決定部分 = `(50-F_end)×0.5 + peaking`。これに `normal(0, 8.0)` が加わる)

TV-6 故障確率(%/週。DUR=54、施設Lv2、通常週)

週メニュー	F=30	F=50	F=66	F=86
標準(threshold + pace_run + std)	0.54	0.64	0.79	2.17
強化(interval + long_run + high)	0.79	0.93	1.14	3.16
追い込み(repetition + downhill + max)	1.23	1.45	1.79	4.94

TV-7 安全網(夏合宿第3週 long_run, interval, max)

F_start	週メニュー	F_end
85	`SAFE` (core, rest, low)に置換	35
84	そのまま(long_run, interval, max)	81

---

11. 検算結果(設計意図の確認)

参照モデルで、4年間・各1000人(施設とスタッフはLv2、全員が3大会すべてに出走)を試算した。入学時は平均50・素質は能力+15〜35の乱数。以下の「常に夏合宿」「夏合宿なし」「ピーキングなし」は各800人。

標準プランの4年間の平均上昇

SPD	STA	PAC	KCK	CLB	DWN	CON	MEN	REC	DUR
+11.6	+21.2	+17.3	+9.5	+5.1	+4.6	+5.7	+4.7	+4.1	+6.3

- 故障は **0.35回/人年**(重傷は10%)、離脱は **1.4週/人年**。

プランによる差(狙いどおり、重点を置いた能力が伸びる)

プラン	SPD	STA	PAC	KCK	CLB	DWN	故障/人年
標準	11.6	21.2	17.3	9.5	5.1	4.6	0.35
持久重視	3.6	23.3	20.0	5.1	1.4	0.0	0.35
スピード重視	15.8	20.3	16.7	12.7	1.4	0.0	0.34
山重視	3.7	21.1	16.5	5.2	10.9	13.6	0.35

重点ごとの適性値 P の差(08 R-04 / §11.2。区間タイプ別の適性値の増加の、標準プランとの差。1500人)

重点	Fast	Mid	Long	Climb	Descent
スピード寄り	+1.19	+0.27	+0.02	−0.40	+0.34
ロング寄り	+0.35	+0.54	+0.58	−1.27	−1.81
(持久重視。採用せず)	−3.19	−1.44	−0.96	+0.32	−1.13
(山重視。採用せず)	−2.34	−1.48	−1.23	+1.14	+1.25

→ 上の「プランによる差」で『狙いどおり、重点を置いた能力が伸びる』のは能力の値であって、**適性値 P で見ると、チーム全体を持久・山に振るのは標準プランに劣る**(素質の伸びしろが能力ごとにほぼ同じなため)。v1 の年間方針は、バランス・スピード寄り・ロング寄りの3つ(08 R-04)。山は、担当する数名のグループだけを山対策にするのが効率的(L-26)。

「鍛えすぎ」と「休みすぎ」のバランス

比較	結果
常に夏合宿(追い込み)	故障 **0.68回/人年**(標準の約2倍)、離脱 2.8週。不調・絶不調が全週の40%。STA +25.0(標準 +21.2)で、**伸びは約2割増に対し故障は約2倍**
夏合宿なし	故障 0.31回/人年。CLB +3.5、MEN +4.0(標準は +5.1 / +4.7)と少し伸びが落ちる
ピーキングなし	大会週の調子が 絶好調15%・好調24%・普通46%・不調以下15%(標準は 絶好調54%・好調30%・普通12%・不調以下3%)

月のテーマ別のXP(同一の選手、F=40開始、通常月=1.00)

テーマ	XP	4週の疲労	故障確率(4週計)
基礎ビルド	1.00	46 → 54 → 58 → 44	2.9%
スピード強化	0.98	47 → 53 → 55 → 39	2.8%
山対策	1.00	46 → 47 → 49 → 36	2.9%
夏合宿	**1.64**	57 → 67 → 73 → 54	**6.4%**
回復月	0.63	19 → 12 → 20 → 28	1.6%

→ **夏合宿は1か月で最も伸びる(通常月の約1.6倍)が、故障確率は約2.2倍。** 回復月は伸びが約6割に落ちる代わりに故障が約半分になる。

新入生のタイプ別(走力4能力の平均、標準プラン)

タイプ	入学時 → 卒業時	最高
平均的(平均50、素質+15〜35、施設Lv2)	50.0 → 64.9(+14.9)	71.0
有望株(60、+30〜40、Lv4)	60.0 → 79.0(+19.0)	83.5
怪物級(68、+25〜30、Lv4)	68.0 → 83.1(+15.1)	87.8
原石・晩成(45、+40〜50、Lv4)	45.0 → 71.3(+26.3)	75.8
伸びしろ小(55、+8〜15、Lv2)	55.0 → 63.1(+8.1)	68.8

(各400人)

→ 怪物級の持久型を育てると STA・PAC が80台後半〜90前後に届く(04のタイム式で歴代級のエース)。

---

12. 他文書への反映事項(本書の確定による変更)

文書	変更
01	夏合宿は「XP×1.5、故障率×1.3」(疲労負荷は高負荷メニュー自体で蓄積)。ピーキングは月のテーマではなく大会連動の自動適用。安全網はF≥85(設定で75/無効)。疲労効率の表、故障の基礎確率(0.5%)を更新
02	`th.peaking` を削除。絆は0.1単位(0〜1000)。`Runner` に `sharpen_bits` を追加。`PeakingMode` 列挙と `annual_plan.peaking_mode` を追加。`settings.auto_safety_f` に変更
00	03 を v0.1 にする

13. 未決・次の文書への引き継ぎ

項目	内容
04	大会後のXP・やる気の更新(`ApplyXp` / `AddMotivation` の呼び出し側)、出走可否と故障の関係

項目	内容
05	新入部員の初期値、退部、スタッフ・施設の取得と費用、指導ポイントの回復、士気の算出
06(完了)	やる気・絆・能力に作用するイベントの効果(06 §5 の `mot` `fat` `xp` `bond` `inj`)、補正とカウンターのフック
08(完了)	個別指導(§5.7)、`form_add` の読み方の確定、重点ごとの適性値の差(§11)。月末の予測(乱数を除いた週の解決)は 08 R-05
検算の対象外	絆(§7.1)とやる気(§7.2)の規則は参照モデルに実装していない。実装時にテストを足す

第4部 駅伝本番仕様

原典: `04_race_spec.md`　　原典の版: v0.2　　改版の経緯: 付録 G.4

> 範囲: 3大会(第一戦・第二戦・第三戦)と予選会の**出場権・登録・オーダー・当日・区間タイム・順位の決定・監督の介入・大会後の処理**。前提: `02_data_spec.md` (コース・ルール・列挙)、`03_training_rules_spec.md` (調子・疲労)、`05_world_scout_spec.md` (他校・名声)。**検算**: 本書の式と計算例は参照モデル `verify/ref_model_04.py` で検証した。05 の世界シミュレーションは、本書の本番式を組み込んで再検算した(`verify/ref_model_05.py`)。

1. 大会の流れ(週の時系列)

大会週を `t` とする(02 §3.5 のカレンダー)。

週	出来事
前年度末〜	出場権の確定(§3)。予選会は10月3週(シード外校のみ)
`t-4`	**エントリー締切**(登録メンバー、§4)。以後、登録者は 03 §5.3 のピーキングメニューになる
`t-3` … `t-1`	調整。登録メンバーの故障は**差し替え**できる(§5.3)
`t` の初め	**オーダー提出**(区間配置、§5)。環境の**予報**が出る(§6)
`t` の終わり	練習の解決が終わり、**調子が確定**(03 §6)
当日	環境が確定し、各走者の**余裕の目安(◎〇△)**が出る(§7.4)。**当日変更**(§5.3)→ 区間ごとの**指示**(§9.1)→ レース進行
終了	結果と後処理(§10)

2. 定数表(初期値案)

分類	定数	値
ペース	`K_SLOPE`	0.4 秒/km/適性値
	`pace50(d)`	`176 + 6×ln(d/5)` 秒/km(**実装ではマスタの表の値を使い、`log` を呼ばない**。距離は有限の集合。端末差の根拠は 09 §8.4)
調子の補正(時間)	絶好調 / 好調 / 普通 / 不調 / 絶不調	−1.5% / −0.7% / 0 / +1.2% / +3.0%
疲労	疲労70以上	追加で +1%
雑音	標準偏差	`0.8% × (1.5 - CON/100)`
ブレーキ	基礎確率	`2% × (1.5 - CON/100) × 重圧係数`
	大きさ	基準タイムの +3〜10%(一様)
	重圧係数	`clamp(1.5 - MEN/100, 0.5, 1.5) × 大会の重み`。大会の重み: 第一戦 0.9 / 第二戦 1.0 / 第三戦 1.2
好走	確率 / 補正	3%(ブレーキがないとき)/ −2%
失速(ペース配分)	確率 `q`	維持のとき `clamp(0.18 + 0.015 × (必要値 - 余裕), 0.03, 0.60)` (§7.4)
	大きさ	基準タイムの +2〜4%(一様)
展開	集団の幅 `G_PACK`	15秒
	追う展開の幅 `G_CHASE`	60秒
	集団走(追走側)/ 単独走	−0.3% / +0.2%
	個性(走法)の倍率 `K_RUN_SCALE`	集団走◎ ×1.5 / 普通 ×1.0 / 単独走◎ ×0.5(§7.3)
	個性(展開)の追う展開の補正 / 先頭走の雑音・ブレーキ率	追い上げ型 −0.2% / 逃げ型 +0.2% ・ 逃げ型 ×0.85・×0.8 / 追い上げ型 ×1.15・×1.2(§7.3)

分類	定数	値
指示	攻める / 維持 / 抑える	時間 −0.8%・雑音×1.3・ブレーキ率×1.5・失速率×2.5 / 0 / +0.6%・雑音×0.7・ブレーキ率×0.5・失速率×0.3
檄	ブレーキまたは失速時の使用	その損失(大きい方)× `(1 - 0.2 × f_k)`。`f_k = 1.0` で ×0.8(検算は `f_k = 1.0`)
	先行して使用	時間 −0.25% × `f_k` (§9.2)
	回数の上限	第一戦2 / 第二戦3 / 第三戦4
当日変更	`swap_limit`	第一戦 1 / 第二戦 2 / 第三戦 4
欠場	登録時 / 当日	5% / 3%
繰り上げ	先頭からの遅れの限度	第一戦 240秒 / 第二戦 600秒 / 第三戦 1200秒
予選会	距離 / 登録 / 合計に数える人数 / 通過	20km / 12名 / 上位10名 / **13校**(L-67)
調子の抽選(他校・調整あり)	絶好調 / 好調 / 普通 / 不調	55% / 31% / 12% / 2%

3. 出場権

3.1 3大会の参加校(世界は36校)

大会	参加校	決め方
第一戦	20校	**前年度の評価ポイント**の上位20校(同点は名声が高い順)
第二戦	30校	**名声**の上位30校
第三戦	**23校**	**シード10校**(前年度の第三戦の上位10校。繰り上げの出たチームは除く)+ **予選会の通過13校**(L-67)

- **第三戦の出場枠(L-67)**: モデルとした大会の通常大会の出場枠の拡大(単独校 20 → 23 校。予選会の通過が 10 → 13 校)に合わせ、第三戦を **20 → 23 校**にした(シードは 10 校のまま)。モデルの大会には、予選会に漏れた選手でつくる**オープン参加の学生連合チーム**(順位・シードの対象外)が加わるが、**ゲームには作らない**(従来の 20 校もこのチームを除いた数。作るなら 24 チーム目として、順位・名声・評価ポイントに入れない扱いになる = 未設計)。世界は 36 校のまま(予選会の出場校は 26 校で、そのうち 13 校が通過する)。モデルの大会は 4 年に 1 度の**記念大会**で出場枠をさらに広げ、予選会を全国の大学に開く予定だが、ゲームでは扱わない(00 §7 M-03)。
- 初年度は、シナリオデータの初期値(前年度の評価ポイント・前年度の第三戦の順位)を使う。
- 参加校でない自校は、その大会に出られない。結果は他校だけで決まる(自動)。

3.2 予選会(10月3週、シード外の26校)

1. 各校が12名を登録する(登録の検証は §4 の E2・E3。10名以上の健康な選手が要る)。
2. 全員が**20kmを同時に走る**(中継なし)。個人のタイムは §7 の式(区間タイプ `Long`、展開の補正なし、指示は維持、失速あり)で計算する。
3. **上位10名のタイムの合計**が小さい順に順位をつけ、**上位13校**が第三戦の出場権を得る(L-67。従来は10校)。同合計は、チームIDの小さい方が上。
4. 予選会は名声・評価ポイントに入れない(05 §3.1)。
5. 自校がシードなら予選会はない。シード外なら、登録 12 名を選び、結果を見る(「結果のみ」も選べる)。
6. **自校の登録 12 名(走った選手)**: 疲労は大会と同じ `ApplyRaceFatigue` (20km × 1.6 = **32**。上限 100)。経験値・やる気・通算成績・名声・評価ポイントは付かない(§10 は 3 大会のみ)。調子は週ごとの解決で決まる(下の 7)。(L-53 ①。検算: T4c の帯の内側のまま。00 L-53)
7. **予選会のピーキング**(03 §5.3 `Short`): 登録メンバーは予選会の週に `TAPER[4]` (sharpen, rest, low)。直前の第一戦の週(第 25 週)が `p` なので `start = max(t-1, p+1) = t`。第二戦の調整(第 26 週は `T2`)より先に当てる。登録メンバー = 手動の登録、なければ出られる選手の長距離の適性の上位 12 名。シード校には関係しない。(L-53 ②)

4. エントリー(登録)

4.1 検証ルール

ID	ルール
E1	登録は `entry_size` 人以下(第一戦10、第二戦14、第三戦16)。出走(`run_size`)人数に足りない場合は**出場できない**(その大会は棄権。名声・評価ポイントは0)
E2	留学生の登録は `intl_reg_max` (初期値2)以下
E3	重傷の選手は登録できない。中傷以下は登録できる
E4	登録できるのは、自校の在籍選手のみ。同じ選手を二重に登録できない

4.2 おまかせ登録(標準)

区間適性を見て、最適な `run_size` 人の配置(§5.2)を求め、残りの登録枠は**長距離の適性値**の高い順に埋める。留学生が登録上限・出走上限を超える場合は、貢献の小さい留学生から外す。

---

5. 区間オーダー

5.1 区間適性値 P

区間タイプごとに能力を加重平均する。**和は表の順に左から足す**(`P = ((w1×s1 + w2×s2) + w3×s3) + …`。浮動小数の和は足す順で最後のビットが変わるので、実装間で一致させるために固定する。09 §15.3 D-12)。

区間タイプ	該当	重み
`Fast`	第一戦1〜5区、第二戦1〜4区	SPD .40 / STA .20 / PAC .15 / KCK .15 / MEN .10
`Mid`	第二戦5〜6区	SPD .25 / STA .30 / PAC .25 / KCK .10 / MEN .10
`Long`	第一戦6区、第二戦7〜8区、第三戦の平地区間	STA .35 / PAC .25 / SPD .20 / MEN .10 / KCK .10
`Climb`	第三戦5区	CLB .45 / STA .25 / PAC .15 / MEN .10 / SPD .05
`Descent`	第三戦6区	DWN .45 / SPD .20 / STA .15 / PAC .10 / MEN .10

特殊能力による加算(02 §3.7)

特殊能力	効果
山の神	`Climb` で P +8
山下りの達人	`Descent` で P +5
エース区間〇	その大会の最長区間で P +3
勝負のアンカー	最終区間で P +2
リベンジ	前回の同じ大会でブレーキをした選手は P +4

- 表示用の**区間適性**は、P が 70以上=◎ / 60以上=〇 / 60未満=△(初期値案)。

5.2 オーダーの検証と自動配置

ID	ルール
O1	区間数と同じ人数を、登録メンバーから**重複なく**配置する
O2	留学生の出走は `intl_run_max` (初期値1)以下
O3	故障中(中傷以上)の選手は配置できない。軽傷は配置できるが調子は −2(03)

自動配置: 区間 × 選手の所要時間行列(基準タイム、調子0)に対して**ハンガリー法**で総時間が最小になる配置を求める。留学生が出走上限を超える場合は、1人ずつ外して再計算し、総時間が最小になる案を採る。

5.3 当日の欠場と変更

- **欠場**(登録時5%、当日3%): 配置された選手が出られない場合は、登録メンバーの中の**最適な代役**に自動で置き換える(`swap_limit` を消費しない)。登録メンバーの数が足りなければ、能力の低い補充要員で埋める。
- **当日変更**(`swap_limit`: 第一戦1名 / 第二戦2名 / 第三戦4名): オーダー提出時は調子が未確定で、**当日に調子と環境(気温・風・雨)が判明する**。当日、配置された選手と**登録メンバーの控え**を入れ替えられる。入れ替わった選手は**同じ区間**を走る。
  - 控えにも調子がある(調子は登録メンバー全員に付く)。変更の判断材料は、区間ごとの**期待タイム**(適性・調子・環境・ペース配分を含む。雑音・ブレーキ・展開は含まない)。
  - 控えの顔ぶれ(登録の16名のうち出走しない6名)が、変更の選択肢になる。山(登り・下り)の専門家、暑さ・向かい風・雨に強い選手、持久型を控えに入れておくと、当日の判断で効く。
- **おまかせ**(ダイジェスト・結果のみ、§9.3)の当日変更: 先発のうち**調子が不調以下**の選手を、その区間の期待タイムが最も小さくなる控えと入れ替える(上限まで。期待タイムが縮まる場合だけ)。

6. 環境

大会の日ごと(第三戦は往路・復路の2日)に、次を抽選する。

大会	気温(°C)平均・標準偏差	向かい風(m/s)平均・標準偏差	雨の確率
第一戦	18, 3	1.0, 1.5	20%
第二戦	11, 3	1.0, 2.0	15%
第三戦 往路	4, 2.5	1.5, 2.5	10%
第三戦 復路	5, 2.5	1.0, 2.5	10%

環境の補正(時間の割合。選手ごと)

```
暑さ = 0.0008 × max(0, 気温 − 15) × (1.4 − STA/100)          夏男は 0
寒さ = 0.0004 × max(0, 3 − 気温)
風   = 0.0030 × max(0, 向かい風) × (1.2 − PAC/100)
雨   = 雨なら 0.006 × (1.2 − CON/100) / 0.7
環境の補正 = 暑さ + 寒さ + 風 + 雨
```

- 環境は**全チームに共通**だが、**選手ごとの差**がある。暑さ(STA)・向かい風(PAC)・雨(CON)に弱い選手は、条件が厳しい日に最大で0.3% 前後(≒ 力にして 1.3)遅れる。これが当日変更の根拠になる。v0.1 の係数の2倍にした(選手間の差を当日変更の判断材料にするため)。
- 気温は、ペース配分の余裕にも影響する(20°Cを超えた分、実効スタミナが下がる。§7.4)。
- **予報**はオーダー提出時に出る。実際の値に誤差(気温 `N(0, 1.5)`、風 `N(0, 1.0)`)を加えたもので、雨は85%の的中率。
  - 引く順序(M1 準備で確定): 日ごとに 気温の誤差(`normal`)→ 風の誤差(`normal`)→ 雨の的中(`uniform` < 0.85 なら実際のまま、そうでなければ反転)。鍵は §12 の `race_id`・9998。

---

7. 区間タイム

7.1 手順(区間 `s`、距離 `d`、地形係数 `terr`、選手 `r`)

```
P        = 区間適性値(§5.1)
基準タイム = d × (pace50(d) × terr − K_SLOPE × (P − 50))                    [秒]   (terr は 02 §3.5 の表。初期の区間記録から較正。L-69)
倍率       = 1 + 調子 + 疲労 + 環境 + 雑音 + 好走 + 展開 + 指示(時間)+ 檄(先行)      (下限 0.85)
失速       = 発生時の大きさ(基準タイムに対する割合。§7.4)×(檄で ×0.8)
ブレーキ   = 発生時の大きさ(基準タイムに対する割合)×(檄で ×0.8)
区間タイム = 基準タイム × (倍率 + 失速) + 基準タイム × ブレーキ                  [秒]
区間タイム(ds) = round_half_up(区間タイム × 10)                              # 0.1秒単位の整数
```

- 倍率は 0.85 を下限とする(失速・ブレーキはその外側で加える)。

- 総合タイム = 区間タイム(ds)の合計。

7.2 補正の項

項	内容
調子	§2 の表(特殊能力「駅伝男」は調子を1段階上げる。最大は絶好調)
疲労	大会週の練習後の疲労が70以上なら +1%
雑音	`N(0, 0.8% × (1.5 - CON/100) × 指示の雑音倍率)`。「逃げ切り」で先頭走のときは ×0.8
ブレーキ	確率 `2% × (1.5 - CON/100) × 重圧係数 × 指示のブレーキ倍率`。「ブレーキ癖」×1.5、「大舞台○」で重圧 ×0.5、「本番弱い」で重圧 ×1.5。発生した場合、大きさは U(3%, 10%)、発生位置は区間の1/3か2/3(表示用)
好走	ブレーキが発生しなかった場合に、3% で −2%
失速	ペース配分(§7.4)。確率 `q × 指示の失速率`、大きさ U(2%, 4%)
展開	§7.3
指示	攻める −0.8% / 維持 0 / 抑える +0.6%(§9.1)

7.3 展開(集団走・単独走・追う展開)

区間の**スタート時刻**(前の区間のゴール時刻)で、各チームを分類する。1区(一斉スタート)は補正なし。

状況	補正
他のチームが `G_PACK` 秒以内にいて、自分より先にスタートしたチームがある(集団の追走側)	−0.3%(集団走○ならその2倍)
他のチームが `G_PACK` 秒以内にいて、自分が集団の先頭	0
`G_PACK` 秒以内に誰もおらず、全体の先頭でもない(単独走)	+0.2%(単独走×なら更に +0.3%、集団走○なら 0)
全体の先頭(2区以降)	0。「逃げ切り」で雑音 ×0.8
自分より前のチームが `G_CHASE` 秒以内にいる(追う展開)で「ごぼう抜き」	−0.5%

個性による補正(10 §6が確定。個性がない選手は従来どおり)

個性	内容
走法	『集団の追走側』と『単独走』の補正に、倍率 `K_RUN_SCALE` を掛ける: 集団走◎ ×1.5(追走側 −0.45%・単独走 +0.30%)/ 普通 ×1.0 / 単独走◎ ×0.5(追走側 −0.15%・単独走 +0.10%)。特殊能力(集団走○・単独走×)は、この倍率のあとに従来どおり重なる
展開	『追う展開』のとき: 追い上げ型 −0.2% / 逃げ型 +0.2%(普通は 0)。『全体の先頭(2区以降)』のとき: 逃げ型は雑音 ×0.85・ブレーキ率 ×0.8 / 追い上げ型は雑音 ×1.15・ブレーキ率 ×1.2(『逃げ切り』の ×0.8 とは別に重なる)
乱数	**引く回数は変わらない**(§12 の 8 回。雑音とブレーキ率の大きさだけが変わる)

- 区間タイムの 1% は適性値 P の約 4.6 点分。上の補正は、その状況で **±0.5〜0.9 P** に当たる(走法 ±0.69 / ±0.46 P・追う展開 ±0.92 P)。個性は**対称**に作ってあり(集団走◎と単独走◎、追い上げ型と逃げ型で正負が同じ)、全チームに個性をランダムに付けても勝率は変わらない(10 §6)。

7.4 ペース配分(失速)と「余裕の目安」

スタミナが「その区間を走る速さ」に対して足りないほど、また距離が長いほど、区間の終盤で失速する。**失速の確率** `q` は、各走者について区間ごとに次の式で決まる(当日、調子と気温が分かると確定する)。

```
実効スタミナ = STA + 3 × 調子段階(−2〜+2) − 5(疲労70以上) − 0.4 × max(0, 気温 − 20)
余裕 m       = 実効スタミナ − 区間適性値 P(§5.1 の重み付き平均。特殊能力の加算は含めない)
必要値 need  = −2.0 + 0.5 × 距離km
q            = clamp(0.18 + 0.015 × (need − m), 0.03, 0.60)          … 維持のときの失速確率
失速確率     = min(0.95, q × 指示の失速率)          攻める ×2.5 / 維持 ×1 / 抑える ×0.3
失速の大きさ = 基準タイムの +2〜4%(一様。平均 +3%)
```

- **力の水準に依存しない**: 全能力が同じ選手(バランス型)は `m = 3 × 調子段階` なので、強いチームでも弱いチームでも q は同じ。差が出るのは**能力の偏り**と**調子**と**距離**。持久型(STAが高くSPDが低い)は q が小さく、スピード型(SPDが高くSTAが低い)は長い区間で q が大きい。

- 失速の期待損失は平均で基準タイムの約 0.5〜0.7%(距離が長いほど大きい。全チームでほぼ共通)。これは基準タイムの検算(§14.1)には含めない(雑音・ブレーキなどと同じく、確率的な要素)。

余裕の目安(画面に出す)

目安	q(維持のとき)	意味	得になる指示
◎	q < 0.16	余裕がある	攻める
○	0.16 ≤ q < 0.26	ふつう	維持
△	q ≥ 0.26	余裕がない	抑える

- 切り替え点は、各指示の期待タイムが等しくなる q(攻める / 維持 ≒ 0.167、維持 / 抑える ≒ 0.262。TV-10)を丸めたもの。目安どおりに指示すると、期待タイムが最小の指示とほぼ同じ結果になる(§14.4)。
- **攻める**は、余裕があれば時間 −0.8% が得になるが、失速率が2.5倍になる。余裕がない走者に攻めさせると、時間の得より失速の損が大きい(**全区間を攻めると得にならない**。§14.4)。**抑える**は、余裕がない走者の失速とブレーキを減らす代わりに、時間が +0.6% かかる。
- 距離が短い大会ほど、◎(攻める)の走者が多い(第一戦・第二戦は約5割、第三戦は約2割)。第三戦は○〜△が中心で、指示の効果は小さい(目安どおりに指示して、チーム平均の力に換算して +0.2)。第一戦は +0.7、第二戦は +0.4。第三戦の手応えは、当日変更と山(5区・6区)の配置にある(§14.4)。

---

8. レース進行と順位

8.1 進行

1. 1区は全チームが同時にスタートする。以降、前の区間の到着が次の区間のスタート。
2. 各区間の到着時刻から、**中継所の通過順位**(累積タイム)と、先頭との差を求める。
3. **繰り上げスタート**: ある中継所で先頭の到着から限度(§2)を超えて遅れたチームは、次の走者が「先頭の到着 + 限度」でスタートする。そのチームの以後の累積タイムはこの時刻から数える。
4. 最後の区間のゴールで、総合タイムが決まる。

8.2 順位の決定

1. 繰り上げのなかったチームが上(繰り上げのあったチームは、それより下)。
2. 総合タイム(ds)が小さい順。
3. 同タイムなら、**最終区間から順にさかのぼって**区間タイムが小さい方が上。
4. それでも同じなら、チームIDの小さい方。

8.3 区間賞・記録

- **区間賞**: その区間のタイム(ds)が最小の選手(同タイムは全員)。
- **区間記録**: 同じコース・同じ区間の過去最速(02 §4.8)を**厳密に**更新したら記録更新(初期の記録がなければ、最初の 1 回は基準で、更新でない。新規ゲームの区間記録は年度 1 の前に『初期の区間記録』(02 §4.8。保持者 = 歴代の記録。L-68)へ置き換わるので、年度 1 の大会から更新が起こる。保持者は最速チームのうち順位の先頭)。**大会記録**(総合タイム)も同様(繰り上げのない優勝チームのタイム)。自校の走者の更新は `race.record`(06)・殿堂入り(05 §10.2)に効く。
- 区間の途中経過(表示用): ブレーキが区間の `u` の位置(1/3 か 2/3)で始まった場合、距離 `x` での通過時刻は `t(x) = t0 + (区間タイム - ブレーキの超過分) × (x/d) + ブレーキの超過分 × max(0, x/d - u) / (1 - u)`。

---

9. 監督の介入

9.1 走行中の指示(区間ごと)

- 当日、区間の前に、自校の走者に **攻める / 維持 / 抑える** を指示する。既定は維持。判断材料は**余裕の目安(◎〇△)**(§7.4)。

指示	時間	雑音	ブレーキ率	失速率	得になる場面
攻める	−0.8%	×1.3	×1.5	×2.5	余裕◎(q < 0.16)
維持	0	×1	×1	×1	余裕〇
抑える	+0.6%	×0.7	×0.5	×0.3	余裕△(q ≥ 0.26)

- 指示を決める画面(08 SC-25)は、フルの観戦で**区間の前**に自校の走者ごとに出す。余裕の目安どおりに指示を一括でそろえるボタン(SC-24)もある。乱数の引き方は変わらないので、レースの途中で中断しても再現できる(08 §6.4)。
- 余裕がある走者に攻めさせると期待タイムが縮み、余裕がない走者に抑えさせると期待タイムが縮む。**全区間を攻める(または抑える)のは得にならない**。
- 指示は区間の前に決める(走行中に変えられない)。乱数の引き方は指示で変わらない(§12)。
- 展開(順位・差)に応じて指示を変える余地もある(先頭を守るなら雑音とブレーキが小さい『抑える』、追うなら雑音が大きい『攻める』)。検算の方針には入れていない(上積みの余地。L-15)。

9.2 檄(運営管理車からの声かけ)

- 1大会の使用回数に上限(第一戦2 / 第二戦3 / 第三戦4)。
- **ブレーキまたは失速時**: 区間の途中(1/3 か 2/3 の地点)で「異変」が通知される。ここで檄を使うと、その損失(ブレーキと失速が両方起きたときは大きい方)が `× (1 - 0.2 × f_k)` になる(`f_k = 1.0` で ×0.8)。v0.2 の初版は 0.4(×0.6)だったが、効果が大きすぎた(L-14)ため 0.2 に下げた。`f_k` は走者のやる気と絆で決まり(下の先行使用と同じ式 `f_k = clamp(0.5 + 0.1×やる気 + 0.3×絆平均/1000, 0.3, 1.2)`)、やる気が高く絆が強い走者ほど効く。**実装済み**: 自校の走者は `RaceRunner.GekiF`(`RaceMath.GekiFactor`)、他校(抽象)は 1.0。参照モデルは `geki_f(mot, bond_avg)`、ゴールデンに式の検算と走者ごとの `geki_f` つきのレースを入れた。損失の軽減は `0.2 × f_k`(チーム一律の補正 `team.geki_f` は試験用に残し、掛け合わせる)。
  - 失速はブレーキより頻繁に起きるので、上限があるなかで『小さな失速に使うか、重いブレーキのために残すか』の判断がある。判断の価値は未検算(L-11)。
- **先行して使用**: 区間の前に使うと、時間 −0.25% × `f_k`(`f_k = clamp(0.5 + 0.1×やる気 + 0.3×絆平均/1000, 0.3, 1.2)`)。ただし1大会の3回目以降の使用は効果 ×0.5(乱用は逆効果)。※先行使用は検算対象外。
- 檄を使っても、乱数の引き方は変わらない(§12)。

9.3 観戦モード

モード	内容
フル	全区間を表示。指示・檄・当日変更を手動で行う
ダイジェスト(おまかせ)	中継所ごとの要約と見どころだけ。指示は維持、檄は**ブレーキ・失速時に自動で**上限まで使う、当日変更は**不調以下の先発を最適な控えに自動で**置き換える(全大会)
結果のみ(おまかせ)	結果を一覧で表示。自動の方針はダイジェストと同じ

- おまかせでも、他校の方針(§11)より有利に戦える(型に合わせた育成・区間適性を見た登録・当日変更・檄の分)。手動で指示と当日変更を最適にすると、さらに上積みがある(§14.4)。

見どころ(ハイライト)の判定: ①首位交代 ②先頭との差が20秒以上動いた区間 ③ブレーキ・失速・好走 ④区間賞・区間記録 ⑤順位が3つ以上動いた区間。ダイジェストでは、区間ごとに重要度の高いものから最大2つを出す。

判定の細部(C# の実装 `RaceHighlights`。表示の規則で、乱数は引かず結果にも影響しない):

- 判定は区間 s の終わりまでの記録だけを使う(途中経過で未来が漏れない)。首位 = 中継所の累積が最小(同じならチームIDの小さい方)。順位も同じ並び。

- **注目校** = 自校 と、その中継所の前後どちらかで上位 3 校に入っていたチーム。20 校すべての動きを並べると、差の変化・失速だけで区間ごとの枠が埋まる(試算: 全校だと 1 区間に差の変化が約 19 件)ので、② ③ ⑤ は注目校だけを見る(④ の区間賞・記録は全校)。
- ① 首位交代: 2区以降で、首位が前の中継所と違う(新しい首位と、それまでの首位を出す)。② 差の変化: 先頭との差が前の中継所に対して 200 ds(20秒)以上動いた、首位以外の注目校(1区は 0 からの変化。正 = 開いた / 負 = 縮めた)。**自校のほかは、動きの最も大きい 1 校だけ**。③ ブレーキ・失速(`Events` の `brake` / `stall`)・好走(`TeamResult.Boost`)の注目校。④ 区間賞(§8.3。同タイムは全員)。⑤ 順位の変動: 2区以降で、順位が前の中継所より 3 つ以上動いた注目校(正 = 上がった)。
- **区間記録・大会記録の更新(§8.3)は最重要**(① と同じ重要度。稀で、出れば最も目立つため。本来 ④ に並べていたものを引き上げた)。注目校でなくてもよい。大会記録は最終区間のゴールで出す。区間記録を出した区間では、同じチームの区間賞は出さない(記録が区間賞を含む)。
- 重要度 = ① 首位交代・記録 > ② > ③ > ④ > ⑤。同じ重要度の中は **自校が先** → (記録が先)→ 大きさの大きい方(首位交代 = 前の首位との差 / 差の変化 = 動いた量 / ③ = 時間の割合 / 区間賞 = 2位との差 / 記録 = 縮めた量 / 順位 = 動いた数)→ 種類の宣言順 → 区間の若い方 → チームIDの小さい方。
- ダイジェスト = 区間ごとに上位 2 件。結果画面の『大会の見どころ』= 全体の上位 5 件(**同じ種類は 2 件まで**。重要度の高い順に取り、時間順に並べる)。定数は `RaceConstants.Highlight*`(差 200 ds・順位 3 つ・注目校の上位 3・区間ごと 2 件・全体 5 件・種類ごと 2 件)。

---

10. 大会後の処理

対象	処理
名声・評価ポイント	05 §3.1。順位は §8.2 の最終順位(繰り上げは下位)。評価ポイントは繰り上げのないチームの上位10校に付く
疲労	走った選手は `ApplyRaceFatigue`(区間距離km × 1.6、03 §8)
経験値(MEN)	走った選手に `ApplyXp(MEN, 30 × 大会の重み)` xu。区間賞 +30、ブレーキを経験 +20(未検証の初期値)
やる気	区間賞の選手 +1。優勝チームの走者全員 +1。ブレーキの選手 −1。チームの順位が「期待順位」より3つ以上上なら走者全員 +1、3つ以上下なら −1(期待順位 = 参加校の中での出走メンバーの力の平均の順位)
通算成績	`career` の出場数、区間賞、優勝、ブレーキ回数。ブレーキは「リベンジ」の条件に使う
記録	区間記録・大会記録の更新(§8.3)
履歴	`RaceRecord`(02 §4.8)に保存
カウンター	出走した選手の `ctr`(06 §10)を更新: `peaked_races`(調整週が2以上で調子+1以上)、`overtakes`(中継所の順位が区間の前より3つ以上上がった)、`lead_sections` / `pack_sections` / `solo_sections`(§7.3 の分類)、`big_wins`(チーム3位以内かつ区間5位以内)、`ekiden_good`(調子+1以上、チーム5位以内、区間3位以内)
イベント	以上の処理のあとに EventPhase(06 §7)。`race.*` の変数(06 §3.2)が使える。イベントの `bench`(欠場)・`entry`(登録に追加)は次の大会の §4・§5.3 に反映する(06 §11)

---

11. 他校の方針と、検算用のプレイヤーの方針

項目	他校(AI)	おまかせ(標準)	上級(最適。検算用の上限)	初心者(検算用)
登録	長距離の適性値の上位。第三戦は登坂・下りの最適任者を各1名含める	区間適性を見た最適な配置の人数 + 長距離の高い順	おまかせと同じ	他校と同じ
配置(提出時)	ハンガリー法(登録メンバー内)	ハンガリー法(全在籍選手から)	同左	ハンガリー法(他校と同じ登録内)
調子	抽選(§2)	03 の調子	同左	同左
育成	型に合わせた重点なし	型ごとの重点プラン(03 §11 の差を能力に反映)	同左	型に合わせた重点なし
当日変更	なし	不調以下を最適な控えに置き換え	期待タイムが最も縮まる入れ替えを、上限まで繰り返す	なし

項目	他校(AI)	おまかせ(標準)	上級(最適。検算用の上限)	初心者(検算用)
指示	維持	維持	区間ごとに期待タイムが最小の指示(余裕の目安どおりとほぼ同じ)	維持
檄	使わない	ブレーキ・失速時に上限まで	同左	使わない

- 他校の選手の10能力は、力 P・型・固定の偏りから復元する(05 §6.2、`stat_i = P + 型の偏り_i + n10_i`)。
- 『上級』は、完全な情報で期待タイムを最小化する方針で、人が実際に到達できる上限に近い値(人は目安を見て判断するので、◎○△の通りに指示すればほぼ到達できる。当日変更は選択肢が多く、到達は難しい)。

---

12. 乱数のキー(03 §3 の PCG32)

`race_id = 年度 × 10 + 種別` (第一戦=1 / 第二戦=2 / 第三戦=3 / 予選会=4)。

用途	label	a	b
区間(1人・1区間)	4	`race_id`	`チームID × 100 + 区間番号(0始まり)`
予選会(1人)	4	`race_id`	`チームID × 100 + 選手の番号(0始まり)`
環境	4	`race_id`	9999
予報の誤差	4	`race_id`	9998
欠場の抽選	4	`race_id`	`8000 + チームID`
他校の調子の抽選	4	`race_id`	`9000 + チームID`

1人・1区間の引き方(固定の順序。分岐しても、指示を変えても消費は変わらない): ① `normal(0, σ)` (`uniform` ×2)→ ② ブレーキの有無の `uniform` → ③ ブレーキの大きさの `uniform` → ④ 位置の `uniform` (<0.5 なら 1/3)→ ⑤ 好走の `uniform` → ⑥ 失速の有無の `uniform` (`< 失速確率` なら失速)→ ⑦ 失速の大きさの `uniform` (2% + 2% × u)。

> **09 の反映**: ①本章の鍵の**登録表は 09 §8.2**(label 4 の b の範囲は他の用途と重ならない)。②区間タイムの `pace50` はマスタの表(09 §8.4)。③**総合タイムの画面表記は秒未満切り捨て**(09 §13 `FmtTotal`)。次章の表の総合タイムは四捨五入(`fmt_t`)なので、画面と 1 秒ずれることがある(L-36。画面が正)。④中断 → 再開で、完了した区間の結果は変わらない(09 §4.6。300 レースで検算)。

13. 計算例(テストベクタ。`verify/ref_model_04.py vectors`)

TV-1 区間適性値

能力 SPD62 STA70 PAC66 KCK58 CLB52 DWN55 CON60 MEN64 REC55 DUR58 の選手:

Fast	Mid	Long	Climb	Descent
63.8	65.2	65.6	60.3	60.65

山の神なら Climb 68.3。エース区間○で最長区間の Long 68.6。

TV-2 区間タイム(第三戦2区 23.1km、P=65.6)

`pace50(23.1) = 185.1824 秒/km`、基準タイム = **4133.569秒(1:08:54)**。調子+1(−0.7%)、雑音+0.4%、集団走の追走(−0.3%) → 倍率 0.9940 → **4108.767秒(1:08:29)= 41088 ds**。ブレーキ+5%が加わると 4315.446秒(1:11:55)、檄を使うと(ブレーキ ×0.8)4274.110秒。2区のタイム(補正なし): P=50 → 1:11:18 / 65 → 1:08:59 / 75 → 1:07:27 / 85 → 1:05:54 / 95 → 1:04:22。

TV-3 区間の乱数

`master=20260101, race_id=11, チーム1, 区間1` (第一戦)、CON60 MEN64 の選手、指示は維持: `σ_noise = 0.0072`、雑音 = **+0.003477**、ブレーキなし、位置 1/3、好走なし、失速なし。ブレーキ率(第三戦、CON60 MEN64)= 2% × 0.90 × 1.032 = **1.858%**。

TV-3b ペース配分の失速

第三戦2区(23.1km、Long)、気温4°C、能力 SPD62 STA70 PAC66 KCK58 CLB52 DWN55 CON60 MEN64 REC55 DUR58、調子+1。乱数は `master=20260101, race_id=13, チーム1, 区間番号 105` (同じ乱数で指示だけを変える)。

- 必要値 need = −2.0 + 0.5 × 23.1 = **9.550**。実効スタミナ = 70 + 3×1 = 73。区間適性 P(Long) = 65.6 → 余裕 m = **7.40**。
- q = 0.18 + 0.015 × (9.550 − 7.40) = **0.2122**(維持のときの失速確率)。

指示	失速確率	この乱数での失速
攻める	min(0.95, 0.2122 × 2.5) = 0.5306	+2.081%
維持	0.2122	なし
抑える	0.2122 × 0.3 = 0.0637	なし

TV-4 環境(同じシードの `race_id=11`)

大会	気温	向かい風	雨
第一戦	16.90	−1.29	なし
第二戦	9.90	−2.05	なし
第三戦 往路 / 復路	3.08 / 5.66	−2.31 / 0.92	なし / なし

環境の補正の例(気温25°C・向かい風3m/s・雨、STA70 PAC66 CON60): 暑さ 0.0056 + 風 0.00486 + 雨 0.005143 = **0.015603**(v0.1 の2倍)。

TV-5 ハンガリー法

所要時間行列 `[[10,9,8,7],[4,3,5,9],[8,7,6,2]]` → 割り当て(区間 → 選手)= `[2, 1, 3]`、合計 **13**。

TV-6 ミニ大会(第一戦6区間・3校、全員の能力は一定、調子は A=+1 / B=0 / C=−1)

A=70、B=66、C=62。乱数は `pcg_for(20260101, 11)`、指示は全員維持、檄なし。

順位	チーム	区間(ds)	合計	事象
1	A	13795, 9534, 14172, 10539, 10819, 17369	2:07:03	—
2	B	14542, 9727, 14460, 11109, 11066, 18469	2:12:17	1区(+3.52%)・4区(+2.02%)・6区(+2.11%)で失速
3	C	14460, 10189, 14613, 10985, 11241, 18107	2:12:17	2区でブレーキ(+4.76%、位置2/3)、5区の中継で繰り上げ

累積(ds): A = 13795, 23329, 37501, 48040, 58859, 76228。C は総合タイムでは B より 0.7 秒速い(79366 と 79373 ds。表示は同じ 2:12:17)が、繰り上げがあったため3位(§8.2)。

TV-7 繰り上げと順位の決定

同タイム(78000 ds)の2チームは、最終区間のタイムが小さい方が上。繰り上げのあったチームは、総合タイムが小さくても最下位に回る。

TV-8 予選会(3校×12名。20km、気温18°C、`race_id=14`)

順位	校	上位10名の合計	時間
1	1	360986 ds	10:01:39
2	2	362190 ds	10:03:39

順位	校	上位10名の合計	時間
3	0	362425 ds	10:04:02

TV-10 ペース配分の危険度 q と余裕の目安(気温10°C)

選手	調子	箱根1区 21.3km(Long)	第一戦1区 8.0km(Fast)
バランス型(全能力70)	0	q=0.310 △	q=0.210 ○
	+2	q=0.220 ○	q=0.120 ◎
持久型(STA76 SPD62 ほか)	0	q=0.218 ○	q=0.076 ◎
	+2	q=0.128 ◎	q=0.030 ◎
スピード型(STA64 SPD78 ほか)	0	q=0.386 △	q=0.334 △
	+2	q=0.296 △	q=0.244 ○

- 持久型は長い区間で q が小さく、スピード型は短い区間でも余裕がなくなりやすい。調子が+2だと q が約0.09下がる。
- 期待タイムが等しくなる q: 攻める / 維持 ≒ **0.167**、維持 / 抑える ≒ **0.262**(目安の境界 0.16 / 0.26)。

TV-11 当日変更(第一戦、`swap_limit` = 1)

登録10名(A〜J)。提出時の配置(調子は未確定)は H, E, I, F, G, **C**(C は力80でアンカーに配置)。当日、C が不調(−1)、控えの J(力75)が絶好調(+2)と分かった。

- おまかせ: C を J に置き換え(H, E, I, F, G, **J**)。指示は全区間維持。
- 上級: 同じ置き換え。指示は J(余裕◎)だけ攻める、他は維持。

---

14. 検算結果

14.1 基準タイムが実大会の水準と一致する

全能力が P のチーム(補正・雑音なし)の総合タイム。01 §7.3 の表と完全に一致する(本番式が基準を壊していない)。失速・ブレーキ・雑音・調子・環境は含まない。

P	第一戦 45.1km	第二戦 106.8km	第三戦 217.1km
35	2:18:44	5:34:45	11:34:16
50	2:14:14	5:24:04	11:12:34
60	2:11:13	5:16:57	10:58:05
70	2:08:13	5:09:50	10:43:37
78	2:05:48	5:04:08	10:32:02

14.2 力の差と優勝率(`ref_model_04.py odds`)

チーム平均の力の差 ΔP と、優勝率(1校との勝負 / 同格9校の中で)。調子・欠場・雑音・ブレーキ・失速・展開・環境をすべて入れている(全チーム維持の指示、当日変更・檄なし)。

大会	ΔP=0	+1	+2	+3	+5
第一戦	0.49 / 0.08	0.59 / 0.15	0.68 / 0.23	0.76 / 0.35	0.88 / 0.58
第二戦	0.52 / 0.12	0.61 / 0.20	0.70 / 0.30	0.79 / 0.39	0.90 / 0.70
第三戦	0.47 / 0.07	0.58 / 0.18	0.69 / 0.25	0.78 / 0.39	0.90 / 0.68

→ 同格の10校の中では優勝率は10％前後。**+3で約4割、+5で約7割**。v0.1(+3で約5割、+5で約8割)より、失速と環境で運の要素が増えた。短い第一戦ほど番狂わせが起きやすい。

14.3 登録・配置・型に合わせた育成・檄の大きさ(課題 L-07 の解消。`ref_model_04.py manage`)

同じ選手集団で、他校の方針に比べた差を「チーム平均の力」に換算:

大会	区間適性を見た登録・配置	+ 型に合わせた育成(効果70%)	+ 同(効果100%)
第一戦	+0.19	+0.67	+0.95
第二戦	+0.05	+0.25	+0.39
第三戦	+0.02	+0.18	+0.29

- 提出時の配置は他校もハンガリー法で最適化するので、配置だけでは差がつかない。差は**型に合わせた育成**と、**スカウト・名声・施設**(05)で生まれる。
- **注(08 L-24)**:『型に合わせた育成』は、**選手ごとに型別の重点プランを適用した前提**の検算用の値。グループ運用の v1 では再現できず、05 の長期の検算で外しても優勝率の差は ±0.016 以内(08 §11.8)なので、実際の効果はこれより小さい。`SPEC_SHIFT` の持久・山の行は、適性値 P で見るとマイナス(08 §11.2)。
- **檄**(ブレーキ・失速時に使用、第三戦4回まで、損失 ×0.8)の平均短縮は **60 秒/大会 = 力に換算して +0.69**(初版の ×0.6 では +1.39 と大きすぎたので弱めた。v0.1 はブレーキ時のみ ×0.6 で +0.20)。

14.4 采配の価値(課題 L-13 の解消。`ref_model_04.py skill`)

① 1大会での優勝率: 自校の力が他校より ΔP=+3 高いとき(他9校の中で)。同じ乱数・同じ選手で、方針だけを変える。1000回。

大会	初心者	檄なし	変更なし	おまかせ	全区間攻める	全区間抑える	指示のみ(目安どおり)	当日変更のみ(最適)	上級(両方)
第一戦	0.393	0.399	0.436	**0.457**	0.390	0.292	0.532	0.515	**0.646**
第二戦	0.407	0.425	0.459	**0.482**	0.408	0.302	0.534	0.567	**0.680**
第三戦	0.414	0.431	0.502	**0.529**	0.362	0.415	0.544	0.621	**0.674**

- 『初心者』= 型に合わせた育成なし・登録は他校と同じ・変更なし・維持・檄なし(検算用の下限)。『檄なし』= おまかせから檄を外したもの。『変更なし』= おまかせから当日変更の自動置き換えを外したもの。『指示のみ』『当日変更のみ』『上級』は、『変更なし』に指示の最適化(余裕の目安どおり)・当日変更の最適化・両方を足したもの。
- **全区間を攻める / 抑えるのは得にならない**(『変更なし』より下がる)。v0.1 では攻めるが無条件に得だった(優勝率 0.5 → 0.8〜0.9)。ペース配分の導入でその穴をふさいだ。
- **おまかせ → 上級**: 第一戦 +0.19、第二戦 +0.20、第三戦 +0.15(完全な情報で最適に行った上限)。内訳(『変更なし』との差): 指示の最適化は +0.10 / +0.08 / +0.04、当日変更の最適化は +0.08 / +0.11 / +0.12。**第一戦・第二戦はペース配分、第三戦は当日変更が効く。**
- 檄は(おまかせと檄なしの差): +0.06 / +0.06 / +0.10。使うだけで効き、上限に達することが少ないので、判断の余地は小さい(L-14)が、効果は初版の半分にした。

② 指示だけの効果(期待タイム。チーム平均の力に換算。`変更なし` との差)

大会	目安どおりの指示	全区間攻める	全区間抑える
第一戦	+0.69	−0.25	−0.75
第二戦	+0.44	−0.74	−0.64
第三戦	+0.19	−1.73	−0.27

③ 長期(世界シミュレーション。`ref_model_05.py`、40回×45年度): 方針ごとの頭打ち後(25年度以降)の優勝率とクリアまでの年度。特別招聘券はゲーム内の獲得のみ(07 §4)。

開始校	方針	優勝率(1大会)	三冠(年)	クリア年度の中央値 / 平均	20年度までにクリア	45年度までに未達
中堅	初心者	0.45	0.18	14 / 15.3	88%	1
	おまかせ	**0.51**	0.21	**12 / 12.2**	97%	0
	上級	0.58	0.30	10 / 10.1	100%	0
弱小	初心者	0.40	0.16	26 / 28.1	28%	5
	おまかせ	**0.46**	0.20	**24 / 25.6**	30%	1

開始校	方針	優勝率(1大会)	三冠(年)	クリア年度の中央値 / 平均	20年度までにクリア	45年度までに未達
	上級	0.60	0.32	18 / 19.6	62%	1

→ **おまかせ → 上級で、頭打ち後の優勝率が +0.07〜+0.14**。クリアまでの年数は中堅で 12 → 10、弱小で 24 → 18。『上級』は完全な情報で最適化した上限で、実際の人は『おまかせ』と『上級』の間になる(初期案の目標 +0.05〜+0.10 の上端付近)。初心者(檄・当日変更なし)でもクリアでき、手動で采配を詰めるほど有利になる。

14.5 M1 準備: 手動の当日(C# `RaceDay`・`GameSession` のオーダー)

画面なしの C#(`m0/Assets/Scripts/Domain/Game/RaceDay.cs` ほか)で、区間オーダー(SC-22)・当日変更(SC-24)・指示と檄(SC-25)の入口を作った。

項目	仕様どおりの動き
オーダー	`SetOrder(kind, 区間 → 選手 ID)`。エントリーの締切から大会の週の初めまで何度でも。O1(区間数と同じ人数・重複なし・登録メンバーから)・O2(留学生の出走は `intl_run_max` まで)・O3(故障中は配置できない)を理由つきで検証。登録を変えて外れた選手がいるオーダーは取り消す。出さなければおまかせ(最適配置)
欠場	登録時 5%・当日 3% で出られない先発の区間は、残りの登録メンバーの中の最適な代役(基準タイムが最小)に自動で置き換わる(当日変更の回数は使わない)
当日(`RaceDay`)	初期値は『おまかせ』(不調以下の先発を最適な控えに自動で置き換え・指示は全区間維持・檄は自動)。確定の前なら何度でも変えられる。当日変更は先発と控えの入れ替え(同じ区間を走る。上限 第一戦 1 / 第二戦 2 / 第三戦 4)、`ResetSwaps` で提出時の配置に戻せる。指示(攻める / 維持 / 抑える)と『目安どおりに指示』
レース進行	`Start` 以降、区間の前に指示、異変(ブレーキ・失速。区間の 1/3 か 2/3 の地点)が出たら檄(`UseGeki`)、`Advance` で次の中継所へ(順位表)。通過した区間の決定は変えられない。**乱数の引き方は指示・檄・当日変更で変わらない**ので、いまの決定で全チームの結果を何度でも再計算する(`Preview`)
予報・前評判	`Forecast`(§6 の誤差)・`RacePreview`(08 R-09)

- **檄の手動**: `RaceTeam.GekiUse`(区間ごと)を足した。null なら従来のおまかせ(異変が出れば上限まで自動)、非 null なら『使う』と決めた区間の異変にだけ使う。異変の通知位置(1/3 か 2/3)は `TeamResult.IncidentPos`(ブレーキの位置の乱数をブレーキ・失速の両方に使う)。未決の区間は自動。
- **留学生の出走(O2)を最適配置に入れた**(`DayPlanner.BestLineup`): 超えるときは 1 人ずつ外して再計算し、総時間が最小の案)。自校の登録(`intl_reg_max`)に加えて出走も守る。他校の抽象の選手は留学生でないので影響なし。
- **故障と出走(L-59)**: 登録は重傷だけ不可(E3)・配置と出走は中傷未満(O3。軽傷は調子 −2 で出る)。`TeamRacing.CanRegister` / `CanPlace` / `CanRun`。出られる選手が出走人数ちょうどで 1 人が軽傷でも棄権にならない。
- **整合の検査**: 何も変えない当日 = 従来のおまかせの当日(先発・指示・檄が一致。3 開始校 × 3 大会 × 3 年度)/ 見込み(`Preview`)= 実際の結果(全チームの順位・タイム・区間)/ ある区間より後の指示を変えても、手前の中継所の順位は変わらない。
- **上級者の方針での長期**(`accept --real --skilled`): 当日変更を期待時間が最小の控えと、指示を期待時間が最小の指示にした場合の、おまかせとの差(同じ世界・同じシード。120 シード × 45 年度)は下の表(`accept all 120 45 --real --skilled`。優勝率 = 25 年度以降の、3 大会あたりの優勝の割合 / 三冠率 = 同じく年度あたり / クリア年度 = 3 大会の優勝経験が揃う年度(揃わなければ 46)の平均 / 戦力 P = 25 年度以降の平均。いずれも 09 §16.3 の定義)。

開始校	優勝率 おまかせ → 上級	三冠率 おまかせ → 上級	クリア年度 おまかせ → 上級	戦力 P
Elite	0.588 → 0.741(+0.153)	0.257 → 0.458(+0.201)	8.97 → 5.19(−3.78)	72.9 → 72.8
Mid	0.534 → 0.695(+0.161)	0.216 → 0.404(+0.188)	13.74 → 9.28(−4.46)	72.5 → 72.4
Low	0.456 → 0.619(+0.163)	0.147 → 0.298(+0.151)	23.48 → 15.64(−7.83)	72.0 → 71.8

- **読み方**: 戦力 P は変わらない(育成・スカウトは同じ『おまかせ』)ので、差はすべて**区間オーダーの後の当日変更と指示**から来る。優勝率は 3 つの開始校で +0.15〜+0.16、クリアまでの年数は 3 割前後短くなる(弱小で 23 → 16 年)。数値は L-53 ①②(予選会の疲労・ピーキング)を入れたあとに取り直した。
- §14.4 の抽象の見込み(+0.07〜+0.14)の上端を少し超えた(+0.15〜+0.17)。この実験の『上級』は、全選手の見込みタイムを知っている**情報が完全な上限**(当日変更は期待時間が最小の控え、指示は期待時間が最小の指示)で、実際の人は『おまかせ』と『上

級』の間になる。合格基準を緩めた T4c(09 §16.3。クリア年度の帯 +1 年度)は『おまかせ』が帯に入るかの確認で、人の操作の分はこの差の分だけ上積みがある、という前提。

- 『おまかせ』の指示は全区間『維持』なので、『目安どおりに指示』(§7 の末尾・08 SC-24)を押すだけでも指示の分は『上級』にほぼ近づく(§14.4)。**手動で詰めるほど有利だが、操作しなくても詰まない**(『おまかせ』でも平均で、強豪は約 9 年、弱小は約 24 年で 3 大会を勝つ)。

15. 課題・未決・他文書への反映

ID	状態	内容
L-07	**解決**	管理の巧拙の大きさを測り(§14.3)、05 の検算から加点を外した
L-08	**解決**	05 の世界シミュレーションに本番式を組み込んで再検算した
L-10	新規	区間タイム式は実測データ(実大会の区間記録・分布)との照合が未了。個人の区間記録(例: 2区 1:05:54 は P85)は整合するが、ブレーキ・失速の頻度と大きさは経験則の仮置き
L-11	新規	檄の先行使用・環境の予報・留学生の登録/出走上限(登録の自動調整を含む)は、規則は書いたが検算していない(影響は小さい想定)
L-12	**解決(09 §8.4)**	他端末で同じレース結果を再現するには `log`・`cos` を使う計算を表引きにする必要があるか → 実測の結果、端末間のビット一致は求めない。基準ペース `pace50` はマスタの表の値にして端末差を 0 にする(`normal()` の端末差は 1 年度で約 10^{-11})
L-13	**解決(v0.2)**	采配の手応えを強めた。ペース配分(失速)・当日変更の全大会化・環境の個人差を2倍にした。おまかせ → 上級で優勝率 +0.10 前後(§14.4)
L-14	**解決(効果を下げた)**	檄の価値が大きく(おまかせと檄なしの差 +0.12〜+0.21)、上限に達しにくいので判断の余地が小さい。対応: 損失の軽減を ×0.6 から ×0.8 に下げた(差は +0.06〜+0.10)。判断の余地の小ささは残る(使うだけで効く)。上限を絞る案は未採用
L-15	新規	展開(順位・差)に応じた指示(先頭を守る・追う)は未検算。上積みの余地として残す
L-16	新規	第三戦は指示の効果が小さい(+0.2 力)。第三戦の手応えは当日変更が中心。対応: 案: 第三戦の区間に『向かい風の区間』『起伏の区間』などの特性を足し、指示と配置の価値を上げる

反映先	内容
02	`swap_limit` を 1 / 2 / 4 に。`Instruction` 列挙、`TeamResult` に指示と出来事、実績(§3.10)
05	本番式で再検算(§14)。開始校を戦力順位で選ぶ。包囲の強さを調整(`K_RIVAL_CHASE` 58)
00	04 を v0.2 に。課題 L-13 を解決、L-14〜L-16 を追加
06	新規。大会後のカウンター更新・EventPhase の起動・`bench` / `entry` の消費
08	画面(エントリー・オーダー・当日・レース進行・結果)。前評判(R-09)・調子の見込み(R-05)を表示。『型に合わせた育成』の注記(L-24)

第5部 世界・スカウト・名声・資金・施設 仕様

原典: `05_world_scout_spec.md`　　原典の版: v0.3　　改版の経緯: 付録 G.5

> 範囲: 自校の**名声・資金・施設/スタッフ・監督ランク**、**スカウト**(候補の生成・活動・勧誘の確定)、**入学・卒業・OB**、**他校(世界)の更新**、そして**長期バランスの検算**。前提: `02_data_spec.md`、`03_training_rules_spec.md`、`04_race_spec.md`。特別招聘券の運用は 07 で定義する。**検算**: 本書の数値は参照モデル `verify/ref_model_05.py` で検証した。個人の成長は 03 の詳細モデルを「進捗率の近似式」に要約して使い(`verify/fit_progress_05.py` で再現)、**レースは 04 の本番式**(`verify/ref_model_04.py`)を組み込んである(v0.1 では簡易版だったものを、04 の確定後に再検算して差し替えた)。**統計的な検算**(乱数は Python の `random`)なので、ビット一致のテストベクタは §13 の「乱数を使わない部分」に限る。**C# の実装の乱数は、09 §8.2 の鍵つきストリーム**(label 3 スカウト・6 世界の更新・7 世界の生成・8 外見)。Python とビット一致しないので、**統計の受け入れ**(開始校ごとに 120 シード。優勝率・三冠率・クリア年度・戦力 P の帯。09 §16.3)で確かめる(L-33)。

1. 前提と用語

項目	内容
力 P	選手が全区間で出せる走力を1つの数値にしたもの。03 の `P_long`(STA .35 / PAC .25 / SPD .20 / MEN .10 / KCK .10)にほぼ等しい
入学時の力 `A0`	入学時点の P
伸びしろ `gap`	素質の P − `A0`
本当の価値 `Q`	`A0 + 0.6×gap`。卒業時の力の期待値にほぼ等しい
市場評価 `V`	`A0 + 0.3×gap`。記録中心の世間の評価。**伸びしろは一部しか見えない**
上位級	`V ≥ V_TOP`(51.5)の候補。獲得に**奨学金枠**が要る
戦力	学校の戦力 = 先発10人の力 P の平均
名声 `prestige`	0〜1000
年度	02 の `year`(4月始まり)。入学の確定は3月、入部は4月

2. 定数表(初期値案)

分類	定数	値
候補プール	`K_POOL_N`	520人/年
	`A0` の分布	平均37、標準偏差7.5、範囲 28〜72
	`gap` の分布	平均24、標準偏差7、`+0.25×(A0-37)`、範囲 8〜45
	成長タイプ	Early 25% / Normal 55% / Late 20%
	`K_V_GAP_W` / `K_V_TOP`	0.3 / 51.5(プールの約19%が上位級)
定員	1学年の入学定員	10人(名声150未満の他校は8人)。部員は最大40人
奨学金枠	他校	`clamp(round(1 + 名声/150), 1, 6)`
	自校	`min(5, 寮Lv + 1)`
提示スコア	`K_W_PRESTIGE`	0.04(名声1000で40点)
	距離帯の加点	0:+12 / 1:+4 / 2:−6 / 3(海外):−8
	寮の加点	`4 × (寮Lv - 1)`
	県別OBネットワークの加点	`6 × Lv`(Lv は 0〜5)
	出場機会	`clamp((Q - 自校のP10)/10, -1, +1) × 30`(P10 = 来年度の名簿の10番目の力)

分類	定数	値
	乱数	自校・他校とも `N(0, 10)`
好感度	自校の狙った候補	+90(3ポイント/人)
	他校の狙い撃ち	+63(+包囲)。狙われない候補は `U(0, 20)`
他校の動き	狙う人数	`round(1.5 × 奨学金枠)`
	狙いの重み	`exp((V + N(0,6) - V_TOP) / 6)`
	`K_RIVAL_CHASE` (包囲)	58
名声	`K_FAME_TOP` / `K_FAME_DECAY` / `K_FAME_ENTRY`	12 / 4 / 0.5
	`K_PRESTIGE_ALPHA` / `H` / `STEP`	0.40 / 14 / 0.50
他校	基準名声への回帰	0.08
	基準名声のランダムウォーク	標準偏差18/年、範囲 20〜850
	指導体制 `coach_q`	毎年 `×0.95 + N(0, 0.05)`、範囲 ±0.30
	`coach_q` の影響	施設Lv換算 `×15`、提示スコア `×60`
資金	収入	`800 + 名声 + 0.0012×名声² + 25×評価ポイント` (万円)
	支出	スタッフ給与 + 施設維持 + `15×部員数` (万円)
	強化費用	`300 + 250×現在Lv` (万円)
	予備費	500万円(これを残して強化する。検算用の方針)
監督ランク	累計評価ポイント	`[0, 6, 15, 30, 50, 80, 120, 170, 230, 300]` でランク1〜10
	施設・スタッフの強化上限	初期 Lv3。ランク3以上で Lv4、ランク5以上で Lv5
スカウト	年間ポイント	`36 + 4 × floor(ランク/3)`
	1人あたり	3ポイント(視察1・声かけ1・練習会招待1)
	情報の誤差	`σ_info = max(1, 8 - 1.2×(スカウト担当Lv - 1))`
レース	本番式	04(登録・配置・環境・ブレーキ・展開・繰り上げ・予選会)

3. 名声と評価

3.1 名声ポイント(順位に応じた滑らかな点)

大会ごとに、出場した全校が順位 `k` に応じて名声ポイントを得る。

```
F(k) = K_FAME_ENTRY + K_FAME_TOP × exp(-(k − 1) / K_FAME_DECAY)        # k=1:12.5, 2:9.85, 3:7.78, 5:4.92, 10:1.77, 20:0.60
年間の名声ポイント = Σ 3大会の F(k)  +  (同一年度の三冠なら +30)
```

- 予選会・記録会は名声に入れない。出場しなかった大会は0。
- **評価ポイント**(01 §3.3: 優勝10 / 2位6 / 3位4 / 4〜6位2 / 7〜10位1、三冠 +30)は別に集計し、監督ランクと実績に使う。

3.2 名声の年度更新(全校共通)

```
hist'  = (1 − K_PRESTIGE_ALPHA) × hist + K_PRESTIGE_ALPHA × 年間の名声ポイント
目標   = 1000 × (1 − exp(−hist' / K_PRESTIGE_H))
名声'  = 名声 + K_PRESTIGE_STEP × (目標 − 名声)
```

- 名声は 0〜1000 に漸近する(1000には届かない)。結果が悪くなれば自然に目減りする。
- **他校のみ**: 更新の後に `名声 += 0.08 × (基準名声 - 名声)` (平均回帰)。

3.3 監督ランク

- 評価ポイントの**累計**でランク1〜10(§2)。ランクが上がると、次が解放される。

ランク	解放
3	施設・スタッフの強化上限 Lv4 / スカウトポイント +4/年
5	強化上限 Lv5
6	スカウトポイント +4/年(計+8)
9	スカウトポイント +4/年(計+12)

- 解任はない(01 §3.3)。ランクは下がらない。

---

4. 資金

4.1 年度の収支(3月に精算。単位: 万円)

```
収入 = 800 + 名声 + 0.0012 × 名声² + 25 × 評価ポイント(その年度) + 補正 funds_year(06 §6。OBのスポンサーなど)
支出 = スタッフ給与 + 施設維持 + 15 × 部員数
  スタッフ給与 = Σ給与(Lv) for コーチ・トレーナー・栄養士・スカウト担当     給与 = [Lv1:0, Lv2:40, Lv3:90, Lv4:160, Lv5:260]
  施設維持      = 8 × Σ(Lv − 1) for 寮・トラック・坂道コース・トレーニング室・医療室
資金 += 収入 − 支出          # 資金は0未満にしない(足りない強化は実行できない)
```

4.2 強化(施設・スタッフ)

- 費用: Lv n → n+1 は `300 + 250×n` (Lv1→2: 550、2→3: 800、3→4: 1050、4→5: 1300)。
- 上限: 監督ランクによる(§3.3)。初期状態で上限を超えているレベルは下げない。
- 実行は3月の精算後。**予備費500万円を残す**(検算用の方針で、ゲームの規則ではない)。
- 対象: 寮・トラック・坂道コース・トレーニング室・医療室・コーチ・トレーナー・栄養士・スカウト担当(主務は 05 では扱わない)。

4.3 資金の使い道(課題 L-05)

- 施設・スタッフがすべてLv5になった後は、資金の使い道がなく、貯まり続ける(検算では40年度で3万〜6万)。後半の使い道(合宿の質の向上、海外遠征、記念事業など)は未設計。

---

5. 施設・スタッフ

施設・スタッフ	効果
コーチ・トレーナー・栄養士・トラック・坂道コース・トレーニング室・医療室	03 §4.6
寮	03 §4.6 の効果に加えて、**奨学金枠 = `min(5, 寮Lv + 1)`** (Lv2→3枠、Lv4以上→5枠)と、提示スコア `+4×(寮Lv-1)`
スカウト担当	情報の誤差 `σ_info = max(1, 8 - 1.2×(Lv-1))` (Lv1:8.0 → Lv5:3.2)

開始時のレベル(難易度): 新規ゲームの開始時にプレイヤーが選ぶ(既定: 中堅)。開始校は、その時点の**戦力順位**(先発10人の力Pの平均の順位。36校中)で選ぶ。

難易度	戦力順位	コーチ/トレーナー/栄養士	寮	トラック/坂道/トレーニング室/医療室	スカウト	資金
強豪	3位	4 / 3 / 3	4	4 / 3 / 4 / 3	3	1500
中堅	7位	2 / 2 / 2	2	2 / 2 / 2 / 2	2	1500
弱小	20位	1 / 1 / 1	1	1 / 1 / 1 / 1	1	1500
同好会	33位	すべて1	1	すべて1	1	1500

- 同好会は部員5人で始める(07 §10.3)。戦力順位33位の学校の名簿のまま始める場合と変わらないことを検算した(07 §12.4)。

---

6. 入学候補(プール)

6.1 生成(毎年5月に翌年度入学ぶん)

1. `K_POOL_N` 人を生成する。各候補:
   - `A0 ~ N(37, 7.5)` を [28, 72] に収める。
   - `gap ~ N(24, 7) + 0.25×(A0 - 37)` を [8, 45] に収める。
   - 成長タイプを抽選。出身を抽選する: **日本人97%**(出身県を 02 §6 の供給重みで抽選。強豪県は能力の基準値を **+3**(プール全体の `A0` の平均が変わらないよう、基準の平均を `3 × 強豪県の供給の割合(36%)` だけ下げてから足す。M0 の実装。L-44)。出身高校を県内から抽選)/ **留学生3%**(出身国を抽選。能力の分布は日本人と同じ。距離帯は3)。留学生の出走は 04 の留学生枠に従う(検算対象外)。
   - **型(プロフィール)**を抽選: バランス35% / 持久25% / スピード25% / 山15%。
2. `Q = A0 + 0.6×gap`、`V = A0 + 0.3×gap`、評価★ = `V<42:1 / <48:2 / <54:3 / <60:4 / それ以上:5`。
3. 候補は**軽量なデータ**(`PoolCandidate`)で持ち、入学したときに選手(02 `Runner`)へ展開する。

6.2 能力の展開(入学時)

10能力は型ごとの偏りを `A0` に足して作る(検算対象外。03 の詳細モデルの入力生成)。

型	SPD	STA	PAC	KCK	CLB	DWN	その他(CON・MEN・REC・DUR)
バランス	0	0	0	0	0	0	0
持久	−8	+6	+4	−6	−2	−3	0
スピード	+8	−6	−4	+6	−2	−2	0
山	−4	+2	0	−2	+10	+6	0

```
stat_i      = clamp(A0 + 型の偏り_i + N(0, 3), 20, 95)        # CON・MEN・REC・DUR は N(0, 4)
potential_i = min(100, stat_i + max(2, round(gap + N(0, 3))))
```

- 型の偏りを入れても、`P_long` の平均は `A0` から ±1 以内に収まる。

個性と素養(10 §4。入学が確定したとき `Runner` に付ける): 上の『型』(入学時の能力の偏り。`profile`)から、選手の**個性**(型・走法・展開)を決める。`master_seed`・`look_seed`・`profile` から決定的に作る(鍵は 09 §8.2 の label 8・b = 40 / 41)ので、**候補データには保存しない**(画面に出すときも同じ関数で作る)。

- 個性の『型』: 入学時の型がスピードなら 90% でスピード型、持久なら 90% でロング型、山なら 60% でロング型・40% でバランス型、バランスなら 70% でバランス型・15% ずつスピード型・ロング型(全体で スピード 29% / バランス 34% / ロング 37%)。走法は 集団走◎ 30% / 普通 40% / 単独走◎ 30%、展開は 追い上げ型 30% / 普通 40% / 逃げ型 30%(独立)。
- 素養: 0 個 80% / 1 個 18% / 2 個 2%(種類は 登坂 38% / 下り 31% / エース 31%)。素養は入学時の `potential`(該当する能力の上限)を上げる。
- **他校の選手(`RivalRunner`)は個性・素養を持たない**(世界の更新は従来の抽象のまま。個性は平均して中立で、自校の勝率の目標値は変わらない。10 §9)。

6.3 上位級と奨学金枠

- `V ≥ V_TOP` の候補を獲得するには、学校の**奨学金枠を1つ使う**。枠が尽きた学校は上位級を獲得できない。
- `V < V_TOP` の候補は枠が要らない。**`V` が低くて `Q` が高い候補(原石)は、強豪校に狙われにくく、枠も使わない**(プールの約6%が `V<51.5` かつ `Q≥58`)。

---

7. スカウト活動(自校)

7.1 ポイントと行動

- 年間ポイント = `36 + 4×floor(ランク/3)` (6〜2月に月4ポイント。ランクによる追加分は6月に一括付与)。06 の補正 `scout_pts` (OBのスカウト協力など)は、その年度の年間ポイントに足す。
- 1人の候補に3ポイントを使うと、次を行える。

行動	ポイント	効果
視察	1	能力・伸びしろの見え方を段階的に解放
声かけ	1	好感度 +30(視察済みであること)
練習会に招待	1	好感度 +60(声かけ済みであること)

→ 3ポイントで好感度 **+90**。年間の目標数 = `floor(年間ポイント / 3)` (ランク1〜2で12人、3〜5で13人、6〜8で14人、9以上で16人)。**これは検算用の方針の目標数**で、定員10(狙わなくても入る人が平均3.3人いる)を超えている。**ゲームの画面は 8人を推奨**する(V の高い順に確定するため、目標の9番目以降はほぼ入れない。08 R-08、§14.7)。

- **時期(08 R-08)**: 視察は 6〜11月、声かけ・練習会は 12〜2月。ポイントは**同じ年度の中では繰り越す**(3月に0)。

7.2 情報の見え方

- 市場評価 `V` と★は**最初から見える**。
- `Q` は視察後に `Q_obs = Q + N(0, σ_info)` として見える(`σ_info` はスカウト担当Lvによる)。
- 成長タイプ・故障しやすさ・素質の細部は、練習会に招待すると解放される(検算対象外)。
- **個性(型・走法・展開)は視察すると見える**(走り方は見ればわかる)。**素養**は、練習会に招待すると『素養あり』だけが見え、**種類はスカウト担当が Lv3 以上のとき**に見える(Lv2 以下は入部後に判明。10 §8)。

7.3 標準の目標選び(検算用の方針。ゲームの規則ではない)

> 注: この方針は『全候補の `Q_obs` を知っている』前提。実際の画面では視察(1ポイント)で `Q_obs` が見える。**視察して選ぶ現実的な方針でも、長期の結果は変わらない**(§14.7)。`pwin` は較正が悪い(0.9〜0.99 で実測 72%)ので、画面には使わない(08 L-29)。

```
各候補について:
  pwin  = ロジスティック((自校の提示スコア + 90 − 最強のライバルの見積もり) / 8)
  価値  = Q_obs × pwin
価値の高い順に、年間目標数まで選ぶ。ただし上位級(V ≥ V_TOP)は奨学金枠の数まで。
```

- 「最強のライバルの見積もり」= 他校の提示スコアの最大値 + (上位級なら 63×0.8、そうでなければ 20×0.8)。

---

8. 勧誘の確定(3月の割り当て)

8.1 提示スコア

学校 `s` が候補 `c` に出す提示スコア。

```
S(c, s) = K_W_PRESTIGE × 名声_s + 距離帯の加点 + 4 × (寮Lv_s − 1) + 出場機会 + 60 × coach_q_s
          + 好感度項 + 乱数 N(0, 10)
  出場機会 = clamp((Q_c − P10_s) / 10, −1, +1) × 30
  好感度項 = 自校: 狙った候補に +90 / 他校: 狙い撃ちした候補に +63 + 包囲、それ以外は U(0, 20)
  県別OBネットワーク(自校のみ): + 6 × Lv(候補の出身県)
```

- `P10_s` = 学校 `s` の来年度の名簿の力の10番目(卒業生を除く)。
- `coach_q` は他校のみ(自校は 0)。他校の「寮Lv」は施設Lv(§9.2)を最大5として使う。

- **包囲**: `K_RIVAL_CHASE × clamp((自校の名声 - 400)/600, 0, 1)`。自校の名声が高いほど、他校が狙い撃ちの好感度を積み増す(打倒王者)。名声400で0、1000で+58。

8.2 割り当て手順

1. 他校の狙い撃ちを決める(§9.2)。
2. 候補を `V` **の高い順**に処理する。
3. 各候補について、**空き定員があり、(上位級なら)奨学金枠が残っている学校**の中で、提示スコアが最大の学校に入る。
4. 入った学校の定員(と、上位級なら奨学金枠)を1減らす。誰も受け入れられない候補は入学しない。

→ 勧誘の成否は**確率ではなく、競争で決まる**(UIには「競合状況」と見込みを出す。08 R-08 は『競合度』と『確定の順番・枠のメーター』で見せる)。

- **注(08 L-30)**: V の高い順に確定し、定員10に達すると以降は入れない。V が低い目標(原石)は定員で入れないことが多い(V < 51.5 の目標の 78% が枠で入れない。§14.7)。練習会まで進めた候補を先に確定する『本命の確約』は検算していない(既定: 変更しない)。

---

9. 他校(世界)の更新

9.1 36校の構成

地方	強豪	中堅	下位	計
関東	3	7	10	20
近畿	1	2	3	6
中部	1	1	2	4
九州	0	1	1	2
東北	1	0	1	2
北海道	0	0	1	1
中国	0	1	0	1
計	**6**	**12**	**18**	**36**

初期の名声と基準名声: 強豪 780/720、中堅 300/300、下位 60/60。そこから15年度ぶんの自校なしの進行(予熱)を回して、名簿と名声を作る。予熱の区間記録は自校と無関係なので、**新規ゲームでは年度 1 の前に実在の水準の『初期の区間記録』へ置き換える**(02 §4.8。L-68。順位表・大会記録は予熱のまま。世界の進行・乱数には触れない = 世界の再現性の目印は変わらない)。

9.2 他校の毎年度の動き

1. **狙い撃ち**: 奨学金枠の1.5倍の人数の上位級候補に、重み `exp((V + N(0,6) - V_TOP)/6)` で好感度を積む(上位級以外には `U(0,20)`)。
2. **名声・基準名声・指導体制**の更新(§3.2、§2)。
3. **成長**: 選手は力 `A0` と進捗率で持つ(§9.3)。施設レベルは名声から決まる。

```
施設Lv(他校) = clamp(1 + 3.5×(名声/1000)^0.8 + 15×coach_q, 0.5, 6.5)
```

4. **入学**: §8 の割り当てに参加(自校と同じ規則)。

9.3 力の進捗(近似式)

03 の詳細モデルでは、卒業までの力は「素質との差(ギャップ)の何割を進んだか」で表せる(`verify/fit_progress_05.py`)。

```
累計進捗率(学年 g, 成長タイプ T, 施設Lv) = (a_T,g + b_T,g × (Lv − 1)) × gap_adj(gap)      # 上限 0.95
gap_adj(gap) = clamp(1.33 − 0.0125 × gap, 0.7, 1.3)
力(学年 g のレース時点) = A0 + gap × (前年度までの累計進捗率 + 0.75 × その年度の増分)
```

型	1年度末 (a, b)	2年度末	3年度末	4年度末
Early	0.24, 0.020	0.40, 0.030	0.50, 0.030	0.56, 0.033
Normal	0.19, 0.020	0.37, 0.030	0.49, 0.035	0.58, 0.035
Late	0.14, 0.010	0.31, 0.020	0.48, 0.025	0.60, 0.028

- 「0.75」= 大会(10〜1月)は年度の約75%が経過した時点。
- 他校の選手はこの近似で成長する。**自校の選手は 03 の詳細モデルで育つ**(進捗率の近似は、検算と他校用)。

9.4 王朝の興亡

- 他校の**基準名声**は毎年ランダムに動き(標準偏差18)、**指導体制** `coach_q` は±0.3の範囲で揺らぐ(`×0.95 + N(0, 0.05)`)。
- これにより、強豪が衰え、新興校が台頭する。自校なしの世界の検算(§14.1)で、80年度に平均14校が優勝し、上位6校のうち4〜5校が入れ替わる。

---

10. 入学・卒業・OB

10.1 入学(4月)

- 入学した候補を選手へ展開する(§6.2)。初期値は 03 §9: 疲労20、やる気3(練習会に招待した候補は4)、調子0。
- 絆の初期値: 同じ出身県の部員同士 +100、同じ出身高校は更に +100(03 §7.1)。
- 定員が埋まらない場合の補充は不要。§8.2 の割り当てでは、プールに候補が残っている限り、空き定員のある学校に必ず誰かが入る(プール520人に対して全校の定員は約340人)。

10.2 卒業(3月。進路と殿堂は検算対象外)

- 4年生は卒業して `OBRecord` (02 §4.7)に変換する。
- **進路**(乱数で決定)

```
p_実業団 = clamp((卒業時の力 − 62) / 14, 0, 0.9)
残りの (1 − p_実業団) を 教員 25% / 一般就職 55% / 院進学 20% で分ける
```

- 殿堂入り: 区間記録の更新、または大会での区間賞が通算3回以上、または卒業時の力が 85 以上。

10.3 県別OBネットワーク

- 出身県ごとにOBの人数を数え、`Lv = min(5, floor(√OB数))` (02 §4.4)。
- 効果: 提示スコア `+6×Lv` (候補の出身県が対象)。情報の見え方への効果(視察1回で解放される段階の増加)は検算対象外。
- 検算モデルは、候補の出身を**地方単位**で近似している(OBの人数も地方ごとに数える)。ゲームの規則は県単位。
- 海外のOBは別枠(02 の添字0)で数え、同様に海外の候補に効く。

10.4 退部と士気(検算対象外)

```
士気 = clamp(round(25 × (部員のやる気の平均 − 1)) + 結果加点, 0, 100)
結果加点 = 年度のいずれかの大会で優勝 +10 / 入賞(3位以内) +5 / 前年度より全大会で順位を下げた −5 / それ以外 0
```

- 月末に再計算する(03 §7.2 の M4・M5 が参照)。
- 年度末、**やる気が1の選手は40%で退部**する。自校の在籍人数は5人を下回らない(02 I-07)。
- OBの役割(コーチ・スカウト協力・スポンサー)は 06 §8 で定義した(イベントで与え、1年間だけ効く)。

---

11. 特別招聘券との関係

- 券で入る選手は、割り当て(§8)に参加せず、**自校の定員10のうち1人分**を占める(通常の入学者が10人に達していれば `Q` の最も低い1人を外す)。奨学金枠は使わない。
- 生成帯・手順・出身・検証は **07 §6**(`Rookie` / `Intl`。入学時の力 `A0` は `Rookie` [58,64]、`Intl` [64,68]。平均の `Q` は 76.0 / 70.8、`V` は 68.5 / 68.4)。
- **検算で仮定した入手と使用**(07 §4・§5 が確定): ゲーム内の獲得は、大会の節目7(各大会の初優勝・初の表彰台・初のシード・初の三冠・クリア)+ 監督ランク3 / 6 / 9 + 継続の記念(10年度ごと)で、45年度に14枚。券は失効させず繰り延べ、使える年度(年1回、招聘枠の選手は同時在籍2名まで)になったら**すぐ使う**。種別は留学生とスーパールーキーを交互にする。有料プランは「4年度に2枚」(年度1・5・9…に付与)。**年度末に決まる獲得(ランク・記念)は、使用の窓(6〜2月)のあとなので、次の年度から使える**扱いにしている。
- 獲得・使用・課金の詳細は 07。検算結果は §14.5 と 07 §12。

---

12. 年度末処理の順序(3月)

1. 大会結果から、名声ポイントと評価ポイントを集計する(04)。
2. 全校の名声を更新する(§3.2)。他校は基準名声と指導体制も更新する(§9.2)。
3. 自校の資金を精算し、強化を実行する(§4)。
4. 成長を確定する(自校は 03、他校は §9.3)。
5. 卒業・進路の決定・退部判定を行う(§10)。学年を1つ進める。
6. 特別招聘券の獲得・使用を処理する(07)。
7. 割り当てを行い(§8)、入学を確定する。

---

13. 計算例(乱数を使わない部分。`verify/ref_model_05.py vectors`)

TV-1 市場評価と本当の価値

`A0=45, gap=30` → `Q = 63.0`、`V = 54.0`。`V ≥ 51.5` なので**上位級**(奨学金枠が要る)。

TV-2 名声ポイント `F(k)`

順位 k	1	2	3	5	10	15	20
F(k)	12.5	9.846	7.778	4.915	1.765	0.862	0.604

TV-3 名声の年度更新

`hist=10`、名声450、3大会で1位・2位・3位(年間の名声ポイント = 12.5 + 9.846 + 7.778 = 30.124) → `hist' = 0.6×10 + 0.4×30.124 = 18.0496`、目標 = `1000×(1-exp(-18.0496/14))` = **724.525**、名声' = `450 + 0.5×(724.525-450)` = **587.262**。他校で基準名声が600なら、続けて `+0.08×(600-587.262)` → **588.281**。

TV-4 提示スコア(乱数・好感度を除く)

自校(名声500、寮Lv3、関東)、候補 `A0=45, gap=36`(Q=66.6、関東)、`P10=60`: `0.04×500 + 12 + 4×(3-1) + clamp((66.6-60)/10, -1, 1)×30` = 20 + 12 + 8 + 19.8 = **59.8**。候補が九州(距離帯2)なら **41.8**、中部(距離帯1)なら **51.8**。

TV-5 資金の年度収支

名声600、評価ポイント20、部員40人、コーチ3・トレーナー2・栄養士2・スカウト3、寮3・他の施設2: 収入 = 800 + 600 + 0.0012×600² + 25×20 = **2332.0**。スタッフ = 90+40+40+90 = **260**、施設維持 = 8×(2+1+1+1+1) = **48**、部員 = 15×40 = **600** → 差引 **+1424.0**。

TV-6 強化の費用

Lv1→2: 550 / 2→3: 800 / 3→4: 1050 / 4→5: 1300。

TV-7 監督ランクと強化の上限

累計評価ポイント	0〜5	6〜14	15〜29	30〜49	50〜
ランク	1	2	3	4	5(300点でランク10)
強化の上限	Lv3	Lv3	Lv4	Lv4	Lv5

TV-8 進捗率(ギャップ30、`gap_adj = 0.955`)

型·Lv	1年度末	2年度末	3年度末	4年度末
Normal Lv1	0.181	0.353	0.468	0.554
Normal Lv3	0.220	0.411	0.535	0.621
Normal Lv5	0.258	0.468	0.602	0.688
Early Lv3	0.267	0.439	0.535	0.598
Late Lv3	0.153	0.334	0.506	0.626

`gap_adj`: gap=10→1.205 / 20→1.08 / 30→0.955 / 40→0.83 / 45→0.7675。例: `A0=45, gap=30, Normal, Lv3` の4年後の力 = `45 + 30×0.621` = **63.62**。

TV-9 他校の施設Lvと奨学金枠

名声	施設Lv	奨学金枠	定員
50	1.319	1	8
280	2.264	3	10
500	3.010	4	10
780	3.869	6	10
950	4.359	6	10

名声780で指導体制 +0.1 → 施設Lv = **5.369**(`3.869 + 15×0.1`)。

TV-10 券で入る選手の平均

`Rookie`: Q=76.0、V=68.5 / `Intl`: Q=70.8、V=68.4。

---

14. 検算結果

自校なしの世界と、自校あり(検算用の方針は 04 §11 の『おまかせ』。特別招聘券は**ゲーム内の獲得のみ**)を、乱数を変えて繰り返した。**レースは 04 v0.2 の本番式**(登録·配置·調子·環境·ブレーキ·失速·展開·繰り上げ·予選会·当日変更·指示·檄)。包囲 `K_RIVAL_CHASE` は 58。特別招聘券は 07 §4 の獲得(45年度で14枚、繰り延べ)で入手し、開始校は戦力順位で選ぶ(§5)。

14.1 自校なしの世界(80年度×20回)

名声順位	戦力P(1年度 / 40年度 / 80年度)
上位6校	68.2 / 68.6 / 68.9
中位12校	60.4 / 60.4 / 60.8
下位18校	55.1 / 55.0 / 55.0

- **80年度でも水準が発散・崩壊しない**。優勝校は80年度で平均15.9校、最多優勝校のシェアは22%、上位6校の入れ替わりは平均4.0校、第三戦の連覇は平均1.5年。

14.2 自校ありの推移(中堅開始、『おまかせ』、40回×45年度)

年度	戦力P	最強の他校のP	戦力順位	名声	施設Lv(平均)	監督ランク	優勝率(第一/第二/第三)	三冠
1	64.7	72.5	7.1	552	2.00	1.0	0.00 / 0.00 / 0.00	0.00
4	66.0	72.2	6.3	571	2.41	2.5	0.05 / 0.05 / 0.05	0.03
8	68.9	71.7	3.6	672	2.82	4.7	0.17 / 0.15 / 0.20	0.03
12	70.1	71.4	2.6	776	3.37	6.2	0.23 / 0.28 / 0.23	0.07
16	71.0	71.3	1.9	849	4.10	8.0	0.40 / 0.33 / 0.47	0.12
20	72.3	71.0	1.6	900	4.77	9.2	0.50 / 0.50 / 0.53	0.17
30	71.4	71.1	1.7	916	5.00	10.0	0.45 / 0.40 / 0.47	0.17
40	71.7	70.6	1.6	899	5.00	10.0	0.47 / 0.53 / 0.53	0.12

- **徐々に強くなり、頭打ちになる**: 戦力Pは 64.7 → 70.1(12年度)→ 72(20年度)で頭打ち。順位は7位前後から 1.6位へ。名声は 550 → 900 で漸近する。
- **頭打ち後(25年度以降)**: 1大会あたりの優勝率 **0.51**、三冠は年 0.21、順位の平均1.6(『上級』なら優勝率 0.58)。→ 目標A(優勝率5〜7割)の中ほどに収まった。
- 最強の他校のPは 70.6〜72.5 で推移し、自校が圧倒しすぎない(包囲の効果)。

14.3 開始校の違い(『おまかせ』、40回×45年度)

開始校	戦力P(1年度 / 12年度 / 40年度)	クリア(通算制覇)した割合	クリア年度の中央値(クリアした回)	25年度以降の優勝率	三冠(年)
強豪	68.4 / 72.9 / 72.3	40/40	**7**	0.57	0.28
中堅	64.7 / 70.1 / 71.7	40/40	**12.5**	0.51	0.21
弱小	57.1 / 63.9 / 71.0	39/40	**23**	0.46	0.20
同好会	52.8 / 60.5 / 66.1	20/40	35.5	0.14	0.06

> 第三戦を 23 校にした(L-67)あとの同好会は、参照モデル 120 シードで優勝率 0.164・クリア年度の平均 39.5 年・45 年度までの未クリア 50%(前: 0.161・39.5 年・53%)で、ほとんど変わらない(予選会ではなく、序盤の戦力と名声が壁)。強豪・中堅・弱小の最新の値は §14.8。上の表は 40 回の実験(L-67 の前)のまま残した。

→ 1年度が約30分なので、クリアまでの目安は 強豪 約3.5時間 / 中堅 約6.5時間 / 弱小 約11.5時間 / 同好会 約18時間(クリアした場合。同好会は45年度までに50%)。弱小・同好会は序盤の戦力が低く、名声と施設が育つまで時間がかかる(上級の采配なら弱小は中央値18年度)。

14.4 プレイヤーの方針の違い(40回×45年度)

04 §14.4 ③ に同じ表を載せている。要点:

開始校	方針	25年度以降の優勝率	クリア年度の中央値	20年度までにクリア
中堅	初心者(型に合わせた育成なし・登録は他校と同じ・変更なし・維持・檄なし)	0.45	14	88%
	おまかせ	**0.51**	**12**	97%
	上級(当日変更と指示を最適化)	0.58	10	100%
弱小	初心者	0.40	26	28%
	おまかせ	**0.46**	**24**	30%
	上級	0.60	18	62%

→ **おまかせ → 上級で、頭打ち後の優勝率が +0.07〜+0.14**(完全な情報で最適化した上限)。手動で采配を詰めるほど有利になり、初心者がおまかせで戦っても詰まらない。差が出るのは、采配のほか、スカウト・名声・施設・育成(03)と、特別招聘券。

14.5 特別招聘券の影響(課題 L-06、各40回×45年度)

詳細は 07 §12.2。獲得は 07 §4 の14個(繰り延べ)、プランは「4年度に2枚」。

開始校	券の入手	クリア年度の平均	20年度までにクリア	25年度以降の優勝率
中堅	券なし	15.7	78%	0.47
中堅	ゲーム内の獲得のみ	12.2	97%	0.51
中堅	有料プランあり	12.6	93%	0.53
弱小	券なし	28.6	17%	0.39
弱小	ゲーム内の獲得のみ	25.6	30%	0.46
弱小	有料プランあり	20.7(獲得のみ比 **−19%**)	55%	0.56

- 有料プランの追加の効果は弱小で大きく、中堅ではほぼ出ない(獲得だけで十分に早くクリアするため)。
- **方針(決定)**: オンラインランキングがなく、課金層は『クリアしたい』ことが目的なので、券の効果の大きさを公平性の制約にしない(01 §6.5 のガードレール『差15%以内』は廃止)。代わりに、**実績の条件で分ける**: 『弱小・無招聘クリア』のように、特別招聘券を一度も使わないことを条件にした実績を用意する(01 §3.3、02 §3.10)。券の上限(所持2・年1回・同時在籍2)は、効果を暴走させないために残す。

14.6 『型に合わせた育成』の有無(08 L-24。各40回×45年度、『おまかせ』)

04 §11 の『おまかせ』は、型ごとの重点プランを選手ごとに適用する前提(`SPEC_ON`)だった。グループ運用の v1 では再現できないので、**外した場合**を測った。

開始校	25年度以降の優勝率(外した / §14.3)	クリア年度の中央値(外した / §14.3)	備考
強豪	0.586 / 0.57	8 / 7	全員クリア
中堅	0.505 / 0.51	13 / 12.5	全員クリア
弱小	0.448 / 0.46	23 / 23	38/40 がクリア
同好会	0.153 / 0.14	—	22/40(55%)がクリア(§14.3 は 50%)

→ **差は優勝率で ±0.016 以内、中央値で ±1 以内**(ノイズの範囲)。v1 は型別の調整を持たず、§14.3 の数値はそのまま使える。

14.7 スカウトの枠と、操作できる方針(08 R-08)

中堅・弱小開始、12回×25年度(1年度に約13人狙う検算用の方針。狙った候補 8,141人)。

結果	割合
入学した	48.5%
枠(定員・奨学金枠)で入れなかった	**69.2%**
枠はあったが競り負けた	4.7%

- 入学者の内訳は、**狙った人 約6.7 + 自然に入る人 約3.3**(中堅。弱小は 6.6 + 3.4)。定員10がいつも埋まる。
- 目標を V の高い順に並べると、入学した割合は 1〜5位 0.74〜0.81(上位級は競合・奨学金枠)、6〜7位 0.82〜0.96、**8位 0.62、9位 0.31、10位 0.07、11位以降 0.00**。→ **推奨の目標は 8人**。
- 競合に勝った割合(枠があったとき。練習会まで進めた場合)は V の帯で、V < 51.5: 1.00 / 51.5〜55: 1.00 / 55〜59: 0.92 / ≥ 59: 0.65。

『視察して選ぶ』方針との比較(10回×25年度。視察 S = 年間ポイント − 2T 人、目標 T 人)

開始校	方針	入学者の Q 平均	15年度以降の戦力 P
中堅	検算用(全員の `Q_obs` を知る・約13人)	65.09	72.16

開始校	方針	入学者の Q 平均	15年度以降の戦力 P
	視察して選ぶ T=6 / 8 / 10	64.25 / 64.25 / 64.16	72.13 / 71.86 / 71.77
弱小	検算用	62.94	66.97
	視察して選ぶ T=6 / 8 / 10	62.56 / 62.90 / 63.27	67.66 / 68.24 / 69.31

→ 操作できる方針でも、**戦力 P は検算用の方針を下回らない**(下がる方向の差は 0.4 以内)。05 §14 の長期の結果はそのまま使える。

14.8 09 による再検算(シード別。C# の受け入れの参照値)

09 §16.3 で、『おまかせ』の世界をシード別に回した(中堅 120・弱小 80・強豪 80 シード。45 年度)。**40 回の値(§14.2〜14.3)とは、40 シードの標準誤差(優勝率 0.02〜0.03)の 1 倍前後の差**で、目標値は変えない。

第三戦の出場枠の拡大(L-67)のあと: 予選会の通過を 10 → 13 校(第三戦 20 → 23 校。04 §3)にしたので、参照モデルを 3 開始校とも **120 シード**(1〜120。`verify/band_05.py`)で取り直した。下の表がいまの値で、取り直す前の値(80・120・80 シード)は括弧の中。出場枠の拡大そのものの影響は、同じシードで比べると優勝率 +0.00〜+0.03・クリア年度 −1.3〜+0.1 年度(強豪 0.591 → 0.617 / 6.87 → 6.98・中堅 0.540 → 0.549 / 11.48 → 11.65・弱小 0.437 → 0.442 / 26.9 → 25.6)で、ばらつきの内。

開始校	優勝率(25年度〜)	三冠率	クリア年度の平均(中央値)	戦力 P(25年度〜)
強豪	0.617(0.589)	0.331(0.299)	7.0(7)	72.5(72.4)
中堅	0.549(0.535)	0.262(0.240)	11.7(11)	72.2(72.0)
弱小	0.442(0.441)	0.188(0.188)	25.6(23.5)	70.7(70.7)

地形係数の較正(L-69)のあと: 区間の地形係数を初期の区間記録から較正した(02 §3.5。`P* = 85`)ので、参照モデルを 3 開始校とも 120 シード(同じシード 1〜120)で取り直した。下の表がいまの値で、L-67 のときの値は括弧の中。差は優勝率 −0.03〜+0.00・三冠率 −0.03〜+0.01・クリア年度 −0.6〜+1.3(中堅)・戦力 P ±0.2 で、シード 120 回の標準誤差(優勝率 0.01〜0.02・クリア年度 0.3〜0.8)の 1〜2 倍(強豪の優勝率・三冠率が少し下がり、中堅のクリア年度が少し延びた)。`P*` を 88 にするとコース全体が遅くなって弱小開始の優勝率が 0.43 → 0.36・クリア 24.5 → 28.8 年度(60 シード)と難しくなるので、難しさを変えない 85 にした。

開始校	優勝率(25年度〜)	三冠率	クリア年度の平均(中央値)	戦力 P(25年度〜)
強豪	0.590(0.617)	0.305(0.331)	6.4(6.0)(7.0)	72.5(72.5)
中堅	0.553(0.549)	0.275(0.262)	13.0(12.0)(11.7)	72.2(72.2)
弱小	0.441(0.442)	0.185(0.188)	25.5(23.0)(25.6)	70.9(70.7)

- シード間の標準偏差は、優勝率で 0.12〜0.21・クリア年度で 3〜10。C# の 120 シードとの差の 3σ(優勝率 ±0.05〜0.09 など)を**受け入れの帯**にする(09 §16.3)。
- **300 年度**の長期(中堅・2 シード): 戦力 P は 72〜73、優勝率は 0.53〜0.71 で頭打ちのまま(年度 26〜300)。

14.9 M0 の C# 実装による再検算(自校なしの世界。`SimCli world`)

C# の世界(県単位の出身・留学生 3%・強豪県の +3・鍵つき乱数)を 80 年度 × 60 シード回し、参照モデルの 20 シード(`verify/world_baseline_05.py`)と比べた。

指標	参照モデル(20 シード)	C#(60 シード)
戦力 P 上位 6 校(1 / 40 / 80 年度)	68.0 / 68.3 / 68.7	68.6 / 69.1 / 68.9
中位 12 校	60.5 / 61.0 / 60.6	60.7 / 61.0 / 60.7
下位 18 校	55.2 / 54.7 / 55.0	55.1 / 55.0 / 55.1
80 年度で優勝した校の数	16.6	16.9
最多優勝校のシェア	20%	20%
上位 6 校の入れ替わり	4.2 校	4.8 校
第三戦の連続優勝(同じ校が続く年数の平均)	1.51 年	1.47 年

- 水準は発散・崩壊しない。上位 6 校だけ C# が +0.2〜+0.8 高いが、強豪県の +3(参照モデルは地方単位で、これを持たない)を外した C# は 68.3 / 69.0 / 68.4 で、差の大半はシード数の揺れ。**文書の目標値は変えない**。
- 乱数を使わない式(提示スコア・力の進捗・施設 Lv・奨学金枠・ランク・名声ポイント・収入)は、ランダムな入力 4,000 件で参照モデルと**完全一致**(`m0` の差分テスト)。乱数を使う部分は、鍵つきの乱数に置き換えたのでビット一致しない(L-33)。
- 候補プールの A0 の平均は 37.4(`[28, 72]` に収める分の上振れを含む。参照モデルも 37.4)。

14.10 M0 段階 10: 自校ありの受け入れと L-44 の決着(`SimCli accept`)

> **L-67 のあとの値**(第三戦 23 校。参照の帯は §14.8 の取り直した値)。取り直す前の値 — 強豪: T4a 0.606 / 0.316 / 7.28・T4b 0.615 / 0.322 / 7.44、中堅: T4a 0.545 / 0.251 / 12.89・T4b 0.558 / 0.278 / 12.40、弱小: T4a 0.435 / 0.188 / 26.70・T4b 0.427 / 0.179 / 27.00 — と比べて、差はばらつきの内。

C# の世界を、方針『おまかせ』(`PlayerAgent`)で開始校ごとに **120 シード × 45 年度**回した(開発機 4 コア・約 20 秒/回)。**強豪県の +3(02 §6)を外した世界(= 参照モデルと同じ構成。T4a 等価性)**と、**+3 ありの仕様の世界(T4b)**を、同じシードで対にして比べる。

開始校	世界	優勝率	三冠率	クリア年度(未クリア)	戦力 P(25 年度〜)
強豪	参照の帯	0.590 ± 0.044	0.305 ± 0.053	6.35 ± 1.07	72.49 ± 0.29
	T4a(+3 なし)	0.608	0.316	6.91(0%)	72.6
	T4b(+3 あり)	0.611	0.318	7.26(0%)	72.9(対との差 +0.32)
中堅	参照の帯	0.553 ± 0.058	0.275 ± 0.056	12.99 ± 2.21	72.18 ± 0.44
	T4a	0.549	0.260	12.51(0%)	72.1
	T4b	0.560	0.281	12.27(0%)	72.5(+0.37)
弱小	参照の帯	0.441 ± 0.071	0.185 ± 0.052	25.48 ± 3.44	70.90 ± 1.01
	T4a	0.408	0.171	27.21(11%)	70.4
	T4b	0.422	0.164	25.88(9%)	71.0(+0.58)

- **等価性は取れた**: +3 を外した C# は、3 開始校とも 4 指標のすべてが参照の帯の内側。式・手順の取りこぼしは(統計で見える大きさでは)ない。
- **+3 の効果**: 戦力 P が +0.2〜+0.4(強豪 +0.41 > 中堅 +0.32 > 弱小 +0.22)。プール全体の平均を変えずに強豪県だけ +3 にしたので、**上位校ほど強豪県の候補を取る分だけ、右の裾が厚くなる**効果。優勝率・三冠率・クリア年度は帯の内側のまま。強豪だけ素の P の帯(72.4 ± 0.4)を 0.1 超える。
- **決着(L-44)**: 仕様の +3 は残す(02 §6。県の個性は『強豪県』だけという設計の中身)。帯は参照モデルの値のまま、判定を 2 本立てにする — T4a: +3 を外した世界が帯の内側 / T4b: +3 ありの世界で優勝率・三冠率・クリア年度は帯の内側、**P は T4a との対の差が −0.3〜+1.0**(上限 +1.0 は、実測の最大 +0.41 の約 2.4 倍で、実装の取りこぼしを疑う目安)。文書の目標値は変えない(`Acceptance.L44_P_SHIFT_*`)。
- 弱小の未クリアの割合は 9〜10%(参照モデルは 4/80 = 5%。80 シードの誤差 ±2.4% の約 2 倍)。クリア年度の平均は帯の内側で、弱小の分布の右の尾(50〜60 年度まで上がらない)が少し重い。
- **保存する double の `Q6` 化(D-10)の影響**: 名声・基準名声・指導体制・a0・gap・prog・履歴・資金を代入のとき `Q6` に丸めるようにしたが、統計は変わらない(上の値は丸めたあとの実測)。

長期の安定(`SimCli stability 16 300 <開始校>`。16 シード × 300 年度。例外・不変条件(W-1〜W-7)の違反なし)

開始校	年度 26〜100 / 101〜200 / 201〜300 の戦力 P(シード平均)	優勝率(同)
強豪	73.05 / 73.11 / 73.20	0.654 / 0.653 / 0.660
中堅	72.33 / 72.30 / 72.52	0.539 / 0.547 / 0.555
弱小	71.77 / 71.94 / 71.93	0.496 / 0.483 / 0.484

- 3 開始校とも頭打ちのまま(区間の差は P 0.5・優勝率 0.06 以内)。参照モデルの 2 シード(中堅)の 72〜73・0.53〜0.71 と同じ水準。弱小は 50〜60 年度まで上がらないシードがあり(26〜100 年度の区間に上昇が入る)、頭打ちの比較は 101 年度以降で行う。

- **保存 → 読み込み → 続行**(T3 ②)が、世界 36 校と自校の抽象の状態(最終のセーブ 273〜286 KB の JSON)でバイト一致する(09 §17.5)。

14.11 M1 準備: 実体の自校 + 世界の受け入れ(`SimCli accept --real` 。T4c)

方針『おまかせ』の自校を、**抽象(`PlayerAgent`)ではなく実体の選手(`Runner` 。03 の週の解決・04 の大会・06 のイベント・05 §10 の卒業と退部)**で回した(`GameSession` 。09 §17.7)。世界・他校・名声・割り当ては同じ道で、同じシードの世界を使う。**120 シード × 45 年度**(開発機 4 コア・約 1.5 分/開始校)。グループは保存される編成(08 R-04。新規ゲームと年度初めに推奨の編成を適用)。

開始校	自校	優勝率	三冠率	クリア年度(中央値)	戦力 P(25 年度〜)
強豪	参照の帯	0.590 ± 0.044	0.305 ± 0.053	6.35 ± 1.07	72.49 ± 0.29
	抽象(同じ世界)	0.611	0.318	7.26	72.9
	実体	0.576	0.243	8.76(9)	72.9
中堅	参照の帯	0.553 ± 0.058	0.275 ± 0.056	12.99 ± 2.21	72.18 ± 0.44
	抽象	0.560	0.281	12.27	72.5
	実体	0.530	0.202	14.80(14)	72.5
弱小	参照の帯	0.441 ± 0.071	0.185 ± 0.052	25.48 ± 3.44	70.90 ± 1.01
	抽象	0.422	0.164	25.88	71.0
	実体	0.461	0.144	23.66(23)	72.0

- **判定(T4c)**: 優勝率・三冠率は帯の内側(**下側は 0.03 = `REAL_RATE_SLACK` まで許す**。L-67 で参照の帯を取り直したとき、強豪の三冠率が帯の下端を 0.009 下回った。実体は抽象より三冠率が 0.03〜0.06 低いのは既知(下)で、帯の幅はこの系統的な差を見込んでいない)、クリア年度は帯の幅 + 1.5 年度(`REAL_CLEAR_SLACK` 。L-69 で 1.0 → 1.5: 帯を取り直したとき強豪の帯の幅が 1.25 → 1.07 に狭まり、実体の既知の遅れ +1.5〜1.8 年度(L-55)を足すと 1.0 では足りなくなった。実体が抽象より遅いこと自体は変わらない)の内側、戦力 P は +3(L-44)を許した帯 [下端, 上端 + 1.0] の内側。3 開始校とも合格。この基準の緩め(クリア年度 +1 年度・P の上端 +1.0)は、『人が操作すれば多少は改善する』ことを前提に承認された。**帯そのものでは 5 か所が外れる**(L-69 のあと): 強豪の三冠率(0.243 < 下端 0.252)・中堅の三冠率(0.202 < 下端 0.219。ここまでは下側の余裕 0.03 の内)、強豪のクリア年度(8.76 > 上端 7.42。帯の幅 + 1.5 年度の内)、強豪・弱小の戦力 P(帯の上端を 0.1 / 0.1 超える。+3 の分で、抽象も同じ)。
- **実体は抽象より少し弱い**: 同じ世界で、優勝率 −0.03(強豪)・−0.02(中堅)・+0.02(弱小)、三冠率 −0.03〜−0.08、クリアは強豪・中堅で +1.4 年度遅い(弱小は −2.9)。戦力 P は抽象と同じ(±0.1。弱小だけ +1.1)なので、**力の水準ではなく、大会の当日の差**: 故障・疲労・調子のばらつき(抽象の自校は故障しない)、区間への配置が実際の 10 能力の形で決まること、型に合わせた育成を自動では行わないこと(08 L-24)。
- **原因の特定(L-55。 `SimCli diag-real <開始校> 40 30`)**: 大会ごとに、自校の出走メンバー 1 人あたりの調子(指数。他校の想定 = 絶好調 55% / 好調 31% / 普通 12% / 不調 2% で平均 +1.4)を測った(中堅・40 シード × 30 年度。他校は上位 5 校 / それ以外)。

大会	自校	他校(上位 5 校)	他校(それ以外)	調子の時間への効き(自校 / 他校)
第一戦	+1.86	+1.46	+1.37	−1.39% / −1.08%
第二戦	**+0.94**	+1.44	+1.38	**−0.68%** / −1.06%
第三戦	+1.87	+1.44	+1.38	−1.40% / −1.06%

強豪・弱小でも同じ形(第二戦だけ +0.9〜1.0、他は +1.8〜1.9)。疲労(平均 9〜13・70 以上は 0%)と出走メンバーの力(他校の上位 5 校より +1 前後)は差がない。**主因は第二戦の調子**: 第一戦の 3 週後で調整が短い(03 §5.3 の重なり = `TAPER[2..4]` 。予選会に出る選手は第 26 週が `TAPER[4]` だけでさらに短い)ため、他校の想定(全大会が満額の調整)より時間にして約 0.4%(力にして約 1.7)遅い。第二戦の優勝率は中堅で 0.21(第一戦 0.38・第三戦 0.60)。一方、第一戦・第三戦は当日変更(不調の先発を控えに替える。04 §5.3)で他校の想定より調子が高く、差し引きで『抽象より少し弱い』になる。**確かめ**: 実験として、自校の第二戦の調子だけを他校と同じ分布で引くと(120 シード。T4c と同じ条件)、実体は抽象を上回る: 強豪 0.630 / 0.333 / 6.62(抽象 0.611 / 0.318 / 7.26)・中堅 0.583 / 0.285 / 11.48(0.560 / 0.281 / 12.27)・弱小 0.509 / 0.209 / 19.35(0.422 / 0.164 / 25.88)。**弱小のクリア年度が帯の外(19.4 < 下端 22.0)になる**ので、第二戦の調整が短い規則はそのまま残す(実体の自校は当日変更・実際の育成の分だけ抽象より強く、第二戦の調整の短さがそれを打ち消して帯の内側に収まっている)。

- **グループを保存される編成にした影響**(08 R-04。毎週順位で決め直すのをやめた): 強豪 +0.00 / 中堅 +0.01(優勝率)で、統計は変わらない(三冠率とクリア年度は中堅で少し良くなった)。
- **見つけたこと**: ①資金は施設が上限(5)になる 18〜20 年度から積み上がり、45 年度で約 4 億円(L-05 のまま。実体の自校でも使い道がない)②招聘の券は常に 2 枚に近く手元に残る(同時在籍 2 名の制限が効く。抽象と同じ)③退部は『やる気 1』がほとんど出ないので 0 が多い(試験は強制した入力で確かめた)。
- **区間記録・大会記録を入れたあとの再実行**(02 §4.8。自校の走者の記録更新 → `race.record` のイベント(+100 万円・スカウトポイント +2)・殿堂入り): 強豪 0.580 / 0.249 / 8.88、中堅 0.524 / 0.200 / 13.65、弱小 0.454 / 0.151 / 24.25(優勝率 / 三冠率 / クリア年度)。入れる前と同じ範囲(±0.01〜0.015)で、3 開始校とも合格。
- **入れなかった理由の記録(08a SC-14『惜しかった候補』)**: 割り当て(§8)は、自校が狙った(声かけまで済ませた)候補が自校に入らなかったとき、その候補を処理した時点の自校の空きで理由を決める: ①自校の空き定員が 0 → **定員** ②上位級で自校の奨学金枠が 0 → **奨学金枠** ③それ以外(他校の提示スコアが上回った)→ **競り負け**(差 = 入った学校の提示スコア − 自校の提示スコア。小さいほど惜しい)。①と②が同時なら定員。特別招聘が定員を占めて外れた候補も定員。割り当ての結果・乱数は変わらない(記録だけ)。

15. 課題・未決・他文書への反映

15.1 課題トラッカーの更新(`00_spec_index.md` §6)

ID	状態	内容
L-01	**解決**	施設・スタッフの効果係数を引き上げた(03 §4.6)。全Lv1→Lv5で卒業時の力が約 +3.7
L-02	**解決**	自校の戦力曲線と頭打ちを設計し検算した(§14.2)
L-03	**解決**	頭打ちの水準を、奨学金枠・包囲・他校の指導体制で制御した(優勝率0.51。04 v0.2 の本番式で再検算)
L-04	一部解決	下振れ要因として「他校の指導体制の揺らぎ」「包囲」を追加。自校の名声・資金の目減りは名声の漸近のみ
L-05	**新規**	施設・スタッフが上限になった後の資金の使い道がない(§4.3)
L-06	**解決(方針)**	有料プランの追加の効果は、弱小 −19%、中堅はほぼなし(§14.5)。公平性の制約にはせず、『無招聘』の実績で分ける(01 §3.3)
L-07	**解決(04)**	加点を外し、本番式で再検算した(§14)
L-08	**解決(04)**	04 の本番式を組み込んで再検算した(§14)。留学生の登録/出走上限は未検算(04 の L-11)
L-09	**解決(07)**	同好会の「部員5人」開始は、通常の名簿で始める場合と変わらない(07 §12.4)。部員が少なくても、入学の定員・奨学金枠は通常どおり
L-23	**解決(08)**	年間の目標数 12〜16人は定員10を超え、狙っても約半分は枠で入れない(69%) → 画面は推奨 8人 + 確定の順番と枠のメーターで見せる(§14.7)。検算用の方針は変えない
L-24	**解決(08)**	『型に合わせた育成』は v1 のグループ運用では再現できない → 外しても長期の結果は変わらない(§14.6)。v1 は持たない
L-29	新規	`pwin` (§7.3)は較正が悪い → 画面には使わない(競合度と枠で見せる)。検算用の方針は影響なし
L-30	新規	V の高い順の確定のため、原石の目標が定員で入れない → 案: 本命の確約。既定: 変更しない(§8.2)

15.2 反映した文書

文書	変更
02	`RivalRunner` (軽量)・`PoolCandidate` を追加。`SchoolState` に `base_prestige`・`coach_q`・`fame_hist`・`cum_points` を追加。他校は `main16` / `depth_rating` ではなく名簿全体を軽量データで持つ
03	施設・スタッフの効果係数を引き上げ(§4.6 と計算例・検算結果を更新)
01	難易度の表と目標のクリア年数を検算結果に更新(本番式で再検算した値)
04	v0.2。本書の他校・名声・出場権の規則と整合。開始校を戦力順位で選ぶ。包囲 58
02	v0.6。開始校 `Difficulty` の選択、実績、`run`
10	§6.2 に個性と素養の付与(入学が確定したとき。他校の選手は持たない)、§7.2 に見え方(個性は視察、素養は練習会とスカウト担当 Lv3)
00	05 を v0.2 にし、課題を更新
09	§14.8(シード別の参照値と 300 年度の長期)/ 冒頭の注(C# の乱数は鍵つきストリーム。統計で受け入れる。L-33)

文書	変更
08	スカウトの時期・繰り越し・推奨の目標数(§7.1)/ §14.6・§14.7 の検算 / L-23・L-24・L-29・L-30

15.3 次の文書への引き継ぎ

引き継ぎ先	内容
04	(完了)本番式の組み込み。出場権・名声ポイントの集計・他校の選手の10能力の復元は 04 §3・§10・§11
06(完了)	OBの役割(06 §8)、郷土のイベント(06 §14.5)。イベントによる資金の増減は年 +90万円程度で、長期の結果は変わらない(06 §16.5)
07(完了)	券の獲得・使用・生成・購入状態・追加コンテンツ・シナリオ(`07_special_recruit_spec.md`)。有料プランの効果の調整は不要(方針決定、§14.5)
08(完了)	スカウト・世界(他校・出場権)・施設の画面。競合度・枠・前評判(08 R-08, R-09)、おまかせスカウト、強化の推奨(R-11)

第6部 イベント仕様

原典: `06_event_spec.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> 範囲: **イベント**(条件式・選択肢・効果のデータ形式、発生の手順、補正、OBの役割、特殊能力の覚醒、カウンター)と、**初期イベントの一覧**。前提: `02_data_spec.md` (選手・OB・特殊能力)、`03_training_rules_spec.md` (週の解決・フック)、`04_race_spec.md` (大会後の処理)、`05_world_scout_spec.md` (資金・OB・他校)。**検算**: 条件式の言語・検証・発生の手順・効果の適用を参照モデル `verify/ref_model_06.py` に実装し、イベントのカタログは `verify/events_06.py` に置いた(本書 §14 の表はここから生成)。発生頻度・所要時間・効果の大きさは、03 の週の解決を使った簡易チームシミュレーション(§16)で測った。

1. 目的と原則

イベントは**物語を語るためではなく、小さな判断と手触りを足すため**のもの(01 §0, §9)。数値の効果は小さく保ち(練習による成長の約1%)、チームの強さは 03・04・05 で決まる。

#	原則
1	**データ駆動**: イベントは「条件式・選択肢・効果」の組み合わせだけで定義する。個別のコードは書かない。
2	**決定的**: 同じセーブ・同じ選択なら、同じ結果になる。乱数は 03 §3 の PCG32(`label = 5`)だけを使い、引く順序を固定する。
3	**時間**: 1年度あたり 約13件 + 覚醒 約5件、合計 約130秒(01 §3.4 の「イベント・成長演出 約3分」のうち約2分)。
4	**おまかせ**: すべてのイベントは既定の選択肢を持ち、自動で解決できる(`settings.event_auto`)。
5	**能力を直接書き換えない**: 能力は XP(経験値)で動かし、03 のレベルアップの規則を通す(`ApplyXp`)。
6	**特別招聘の選手に専用イベントは付けない**(01 §6.5)。招聘選手も、通常の選手と同じ条件で通常のイベントの当事者になる。
7	**出身県・留学生**の描写に配慮する。国や人柄のステレオタイプは描かない。留学生のイベントは『言葉・食事・生活の不安』など、個人の状況に限る。
8	世界の他校・他校の選手には、イベントは起こらない(自校のみ)。

2. 定数表(初期値案。`K_EV_` 接頭辞)

定数	値	意味
`K_EV_WEEK`	2	1週に起こるイベントの上限
`K_EV_YEAR`	22	1年度に起こるイベントの上限(`exempt` と覚醒は数えない)
`K_EV_RUNNER_MONTH`	1	1人が1ヶ月に当事者になれる回数(§7.3)
`K_AWAKEN_P`	Blue 0.10 / Gold 0.05 / Red 0.08	取得条件を満たした週に覚醒する確率(§9)
`K_XP_MAX`	400 xu	1つの効果で1人に与える XP の上限。1つの選択肢の同じ対象への合計は 800 xu まで
`K_FUNDS_RANGE`	−1500 〜 +500 万円	1つの効果の資金の変化
`K_MOD_WEEKS_MAX`	48	補正の期間の上限(週)
`K_T_INTERACT` / `K_T_NOTICE`	12秒 / 4秒	選択のあるイベント / お知らせ(選択肢が1つ)の所要時間(見積り)
時間予算	150秒/年度	イベントと覚醒の所要時間の合計の目安(§16)
補正の範囲	`E_mult` 0.8〜1.2 / `r_add` ±0.06 / `inj_mult` 0.5〜2.0 / `form_add` ±4 / `funds_year` −300〜+600 / `scout_pts` −4〜+8	§6

3. 条件式(DSL)

条件式は、**副作用のない・乱数を使わない**式で、イベントの発生条件・選択肢の条件・特殊能力の取得条件に使う。**読み込み時に構文・変数・関数・引数の数・役割の束縛を検証し、実行時には例外を起こさない**(§13 V-02)。

3.1 文法

```
expr  := or
or    := and ('or' and)*
and   := not ('and' not)*
not   := 'not' not | cmp
cmp   := sum (('<' | '<=' | '>' | '>=' | '==' | '!=') sum)?
sum   := term (('+' | '-') term)*
term  := unary (('*' | '/') unary)*
unary := '-' unary | atom
atom  := 数値 | '文字列' | true | false | 変数 | 関数名 '(' [expr (',' expr)*] ')' | '(' expr ')'
変数   := 英数字と '_' と '.' の並び(例: r.stat.CLB)
```

- 優先順位は上の順(`not` > 比較 > `+ -` > `* /` の順に強い。`and` は `or` より強い)。`and`・`or` は左から評価して**短絡**する。
- 数値は整数または小数。真偽は比較・論理演算の結果。列挙は数値のコードで表す(§3.3)。
- ゼロ除算は 0 を返す。比較は数値どうし・文字列どうし(文字列は関数の引数にだけ使う)。
- **使える場所**(読み込み時に検証する。M0 の C# 実装で厳しくした): 文字列は `has_skill` の第 2 引数(特殊能力の ID)と `ctr` の第 2 引数(§10 のカウンター名)だけ。役割 `r` `p` は関数の役割の引数(`has_skill` `ctr` `bond` `same_pref_n` `senior_same_pref` `ob_level` `rival_same_hs` の `who`)だけで、値としては使えない。
- 役割: `r`(当事者の選手)と `p`(もう1人の選手。`pair` のイベントだけ)。`team` のイベントでは `r`・`p` を使えない。

3.2 変数

カレンダーと大会

変数	意味
`cal.year` / `cal.month` / `cal.week`	年度(0始まり)/ 月(0=4月〜11=3月)/ 月内の週(0〜3)
`cal.next_race`	次の大会までの週数(大会週は 0)。次が来年度なら翌年の値
`cal.next_race_kind`	次の大会(1=第一戦 / 2=第二戦 / 3=第三戦)
`race.kind`	その週に開催された大会(0=なし / 1〜3)。大会の後処理のあと(§7.1)だけ 0 以外
`race.team_rank` / `race.expected_rank`	自校の順位 / 期待順位(04 §10)。大会がなければ 99

選手(`r.` と `p.`)

変数	意味
`id`	選手ID。対称なペアの片方だけを選ぶのに使う(`r.id < p.id`)
`grade` / `weeks_in`	学年(1〜4)/ 入学からの週数
`stat.SPD` 〜 `stat.DUR`	10能力(0〜100)
`fatigue` / `form` / `motivation`	疲労(0〜100)/ 調子(−2〜+2)/ やる気(1〜5)
`injury` / `injury_weeks` / `injury_age`	故障の重さ(0=なし / 1=軽傷 / 2=中傷 / 3=重傷)/ 残り週 / 故障してからの週(故障がなければ 99)
`group` / `captain`	グループ(0=A / 1=B / 2=C。**保存される編成**: 新規ゲームと年度初めの『推奨の編成を適用』で決め、毎週は決め直さない。新入部員は C。08 R-04。参照モデルの `refresh` は毎週順位で決め直す = 差分テスト用)/ 主将(0/1)
`overseas` / `far` / `near`	留学生(0/1)/ 遠方出身(距離帯2以上または海外。0/1)/ 近郊出身(距離帯0。0/1)
`pref`	出身県(1〜47。留学生は 0)
`bond_avg`	同じグループの仲間との絆の平均(0〜1000)
`p_rank` / `regular`	力の順位(04 §5.1 の Long の区間適性値。1=最強)/ 力の上位10人で故障していない(0/1)

変数	意味
`gift.climber` / `gift.descender` / `gift.ace`	**素養**(10 §2)を持つか(0/1。**隠し**でも条件には使える)
`is_speed` / `is_long`	個性の型がスピード型 / ロング型か(0/1。バランス型は両方 0)
`is_pack` / `is_solo`	個性の走法が集団走◎ / 単独走◎か(0/1)
`is_chase` / `is_lead`	個性の展開が追い上げ型 / 逃げ型か(0/1)
`taper`	調整週の連続数(03)
`races` / `section_wins` / `brakes` / `skills`	通算の出走数 / 区間賞 / ブレーキ / 特殊能力の数
`gap_avg`	**隠し**: 素質と能力の差の平均。画面には出さない
`race.in_lineup` / `race.brake` / `race.section_rank` / `race.section_win` / `race.record`	その週の大会で、出走したか(0/1)/ ブレーキしたか / 区間順位(出走しなければ 0)/ 区間賞 / 区間記録の更新(0/1)。大会のあと以外は 0

チーム・施設・OB

変数	意味
`t.funds` / `t.prestige` / `t.morale`	資金(万円)/ 名声 / 士気(05 §10.4)
`t.size` / `t.n_g1` 〜 `t.n_g4` / `t.n_g3plus`	部員数 / 学年ごとの人数 / 3年生以上の人数
`t.coach_rank` / `t.scout_pts` / `t.guidance`	監督ランク / スカウトポイント(補正を含む)/ 今月に残る指導ポイント
`t.rank.fall` / `t.rank.national` / `t.rank.newyear`	今年度の順位(その大会がまだなら 99)
`t.last_rank.fall` / `.national` / `.newyear`	前年度の順位(なければ 8)
`fac.dorm` / `.track` / `.hill` / `.gym` / `.medical`	施設のレベル(1〜5)
`staff.coach` / `.trainer` / `.nutri` / `.scout`	スタッフのレベル(1〜5。不在は 1)
`ob.total` / `ob.corp`	OBの人数 / うち実業団に進んだ人数

3.3 関数

関数	戻り値
`has_skill(who, 'id')`	`who`（`r` または `p`）が特殊能力 `skill.<id>` を持つか
`ctr(who, 'name')`	カウンター(§10)の値
`bond(a, b)`	2人の絆(0〜1000)
`same_pref_n(who)`	自校の、同じ出身県の他の選手の数(留学生は 0)
`senior_same_pref(who)`	同じ出身県で学年が上の選手がいるか
`ob_level(who)`	その選手の出身県の県別OBネットワークのレベル(0〜5。留学生は海外のレベル。02 §4.4)
`rival_same_hs(who)`	その週に出場する他校の先発に、同じ出身高校の選手がいるか(世界の他校の選手の出身から判定。検算では確率 0.40 で近似)
`min(a,b)` / `max(a,b)` / `abs(a)` / `clamp(x, lo, hi)`	数値の関数

---

4. イベント定義（`EventDef`）

イベントは**データ**（`StreamingAssets/Master/events/*.json`）で定義する。文言は名称テーブルのキーで持つ(日本語先行。変数は `{r.name}` `{p.name}` `{pref}` などを差し込む)。

4.1 項目

項目	型	説明
`id`	string	例: `ev.ind.homesick`。一意

項目	型	説明
`category`	enum	`Team` `Management` `Individual` `Race` `Hometown` `Graduation` `Awakening`
`scope`	enum	`team`（当事者なし）/ `runner`（`r` が1人）/ `pair`（`r` と `p` の2人）
`months` / `weeks`	list?	発生する月（0=4月〜11=3月）・月内の週（0〜3）。null なら通年
`phase`	enum	`normal`（毎週）/ `race`（大会週の後処理のあと。`race.*` が使える）
`cond`	条件式	発生の条件（当事者ごとに評価）
`p`	double (0,1]	条件を満たす週に発生する確率（1つのイベントにつき週に1回判定。当事者の数に比例しない）
`cooldown`	int	同じ当事者（`pair` は同じ2人、`team` はイベント全体）で再び起こるまでの週数
`once`	enum?	`runner` なら、同じ選手の当事者としては一生に1回まで
`max_year`	int?	1年度に起こる回数の上限
`priority`	int	大きいほど先に処理する（同じなら ID 順）
`exempt`	bool	true なら年度の上限と、（覚醒以外は）1人1ヶ月の制限を受けない。行事・故障・大会のイベント向け
`choices`	list	選択肢（§4.2）
`default`	list<string>	『おまかせ』で選ぶ選択肢の優先順。条件を満たす最初のものを選ぶ。**最後の選択肢には条件を付けない**（必ず選べる）

4.2 選択肢

項目	説明
`id` / `label`	ID と、画面に出す文言。**数字は書かない**（効果の数値は `fx` から画面が要約を作る = 08 R-15。本文に書くと効果とずれる。L-63 ①）。以下の表の『★…→ …』は、左が `label`、右が `fx` の説明
`req`	条件式（省略可）。偽なら選べない（例: 資金が足りない、指導ポイントがない）
`fx`	効果のリスト（§5）。空なら『何もしない』

- 選択肢が1つだけのイベントは**お知らせ**（選ばずに効果が起こる）。2つ以上あれば**選択あり**。
- 選択の内容は結果に影響するが、**乱数の引き方は変えない**（§7.5）。

---

5. 効果

5.1 対象（`to`）

値	対象
`r` / `p`	当事者の選手 / もう1人の選手
`all`	在籍する全員
`g1` `g2` `g3` `g4`	学年ごと
`top10`	力の上位10人（`p_rank ≤ 10`）
`same_pref`	`r` と同じ出身県の選手（`r` を含む。留学生は `r` のみ）
`same_group`	`r` と同じグループの選手
`far` / `near` / `overseas`	遠方出身 / 近郊出身 / 留学生
`captain`	主将
`team`	チーム（補正の対象）

5.2 操作

操作	引数	内容	範囲（検証 V-07）
`mot`	`to`, `d`, `p?`	やる気を `d` 変える（1〜5に収める。03 `AddMotivation`）	`d` は ±1

操作	引数	内容	範囲(検証 V-07)
`fat`	`to`, `d`, `p?`	疲労を変える(0〜100。03 `AddFatigue`)	±20
`xp`	`to`, `stat`, `xu`, `p?`	XP を与え、03 §6 手順3のレベルアップを行う(03 `ApplyXp`。`E`・素質補正は掛けない)。`stat` は能力名 / `weakest`(最も低い能力。同じなら添字の小さい方)/ `random`(乱数)	`xu` ≤ 400
`bond`	`d`	`r` と `p` の絆を変える(0〜1000。`pair` のみ)	±100
`bondg`	`to`, `d`	対象の中のすべての組の絆を変える	±100
`inj`	`to`, `sev`	故障させる(故障中でなければ。期間は 03 §6 手順5)	初期イベントでは未使用
`skill`	`to`, `id`, `p?`	特殊能力を付ける。すでに持つ・排他の能力を持つ・8個に達していれば何も起きない	—
`funds`	`d`	資金(万円)を変える(0未満にしない)	−1500〜+500
`scout`	`d`	その年度のスカウトポイントに加える(05 §7.1)	—
`guide`	`d`	今月の指導ポイントを変える(0以上)	—
`mod`	`kind`, `to`, `value`, `weeks`	補正を付ける(§6)	§6 の範囲、`weeks` ≤ 48
`captain`	`select`	主将を決める。`top_bond`(絆が最も高い)/ `top_p`(力が最も高い)/ `top_men`(MEN が最も高い)。候補は3年生以上(いなければ全員)	—
`bench`	`to`	次の大会で欠場扱いにする(04 §5.3。控えが自動で入り、`swap_limit` は消費しない)	—
`entry`	`to`	次の第三戦の登録に追加する(04 §4。登録に入っていない選手1人ぶん。登録枠を超えるときは、出走しない登録メンバーの中で力の最も低い選手と入れ替える)	—
`ob_role`	`role`, `weeks`	OBに役割を与える(§8)	—

- **確率付き効果**(`p`): `to` が1人なら乱数を1回、複数なら**対象1人ごとに1回**(対象の ID の小さい順)引き、`< p` の対象にだけ適用する。
- 効果は**定義の順**に適用する。同じ選択肢の中の効果は、先の効果の結果を後の効果が見る(例: `xp` のあとの能力)。
- `to = p` は `pair` のイベントだけ、`bond` も `pair` だけ(V-06)。

---

6. 補正(`TimedModifier`)

一定の期間だけ効く補正。イベントの `mod` と、OBの役割(§8)が作る。

種類	値	合成	効く場所
`E_mult`	練習の効率の倍率	積(0.8〜1.2)	03 §6 手順1の `E` に掛ける(`E_MAX` の前)
`r_add`	疲労の回復率の加算	和(±0.06)	03 §6 手順4の `r` に足す(`R_MIN` 〜 `R_MAX` の前)
`inj_mult`	故障率の倍率	積(0.5〜2.0)	03 §6 手順5の `p` に掛ける(`m_sk` と同じ位置)
`form_add`	調子のスコアの加算	和(±4)	03 §6 手順6の `score` に足す(結果は −8〜+12 に収める)。大会の週の調子にだけ意味がある
`funds_year`	年間収入(万円)	和(−300〜+600)	05 §4.1 の年度の収入に足す(期間中の年度末に1回)
`scout_pts`	スカウトポイント	和(−4〜+8)	05 §7.1 のその年度のスカウトポイントに足す

- 対象は `team`(全員)か、1人の選手。**同じ(種類、対象、発生源のイベント)の補正は上書き**する(重ね掛けしない)。
- `until`(絶対週)を過ぎたら消える。期間は 48週まで。選手が卒業・退部したら、その選手への補正は消える。
- 補正の値はイベントごとに検証する(V-08)。

---

7. 発生の手順(週ごと)

7.1 週の順序(03 §6「全体の順序」に EventPhase を加える)

①全選手の `WeekResolve` (03 §6)→ ②絆の週の更新(03 §7.1)→ ③大会があれば 04 の手順と大会後処理(`race.*` が決まる)→ ④**EventPhase**(本章)→ ⑤第4週なら月次処理(03 §7)→ ⑥3月の第4週は年度末処理(05 §12)。

7.2 EventPhase

```
fired = []
stream = PCG32(derive(master_seed, 5, abs_week, 0))
for e in イベントを priority の大きい順、同じなら id の順に:
    if len(fired) ≥ K_EV_WEEK: break
    if e.phase == race かつ 今週は大会週でない: continue
    if e.months / e.weeks に今が含まれない: continue
    if e.max_year ≠ null かつ 今年度の回数 ≥ e.max_year: continue
    if not e.exempt かつ 今年度の回数(exempt を除く)≥ K_EV_YEAR: continue
    subjects = 当事者の候補(§7.3)
    if subjects が空: continue
    u1 = stream.uniform();  u2 = stream.uniform()        # 候補がいるイベントだけ、必ず2回引く
    if u1 < e.p:  fired.append((e, subjects[floor(u2 × len(subjects))]))
for k, (e, subject) in enumerate(fired):                 # 発生したものを順に解決
    estream = PCG32(derive(master_seed, 5, abs_week, 1000 + k))
    choice = 選択(§7.4)
    効果を定義の順に適用(estream を使う)
    記録を更新(回数・最後の週・当事者の制限・クールダウン)
```

- 上限(週 `K_EV_WEEK`)に達すると、それ以降のイベントは処理しない(乱数も引かない)。発生しなかったイベントの確率は、翌週にまた判定される。**固定の行事**(主将の指名など)は、発生する週を2週に広げて、上限で落ちても翌週に回るようにする(§14)。
- 選手が当事者のイベントは、**その週に1人1回まで**(1つのイベントで当事者になった選手は、同じ週の別のイベントの候補から外れる)。

7.3 当事者の候補

scope	候補
`team`	`cond` を満たし、`cooldown` が過ぎていれば、1つ(当事者なし)
`runner`	在籍の各選手 `r` で `cond(r)` が真、`cooldown` が過ぎている、`once = runner` なら未経験、かつ制限(下)に当たらない選手
`pair`	すべての (r, p)(r ≠ p。順序あり)で `cond(r, p)` が真、同じ2人の `cooldown` が過ぎている、かつ制限に当たらない組。対称にしたいときは `cond` に `r.id < p.id` を入れる

- **1人1ヶ月の制限**(`K_EV_RUNNER_MONTH` = 1): 同じ月にすでにイベントの当事者になった選手は、`exempt = false` のイベントと覚醒の候補から外れる(`exempt` で覚醒でないイベントは制限を受けない)。
- クールダウンは、そのイベントが最後に起こった週から数える(当事者ごと。`team` はイベントごと)。

7.4 選択の解決

- **プレイヤーが選ぶ**: 条件(`req`)を満たす選択肢から選ぶ。
- **おまかせ**(`settings.event_auto` が真、またはフル観戦以外の画面で時間を省くとき): `default` の先頭から、`req` を満たす最初の選択肢を選ぶ(最後の選択肢は必ず選べる)。
- 検算では『おまかせ』と『すべての選択肢を一様に選ぶ』の2通りを測った(§16)。

7.5 乱数

`label = 5` (03 §3.2 の予約)。

用途	a	b	引く順序
発生の判定	`abs_week`	0	§7.2 のとおり(候補のあるイベントごとに `u1` , `u2`)

用途	a	b	引く順序
効果(k番目に発生したイベント)	`abs_week`	`1000 + k`	定義の順に、確率付き効果(対象ごと)・`xp` の `random`・`inj` の期間

- 乱数の消費は**状態だけで決まる**(選択や選手の処理順に依存しない)。発生したイベントの選択が違っても、その週の発生の判定は変わらない(効果の乱数だけが選択で変わる)。

---

8. OBの役割(05 §10.4)

OB(`OBRecord`。02 §4.7)は、イベントを通じて**1年間だけ**チームを支える。役割は `OBRecord.role`(`None` / `Coach` / `ScoutHelper` / `Sponsor`)で表し、期限(`role_until`)が来たら `None` に戻る。

役割	発生するイベント(§14)	選ばれるOB	効果(補正。§6)	期間
`Coach`(コーチ)	OBがコーチを申し出る(実業団に進んだOBが3人以上、コーチのレベルが3以下)	実業団に進み、卒業時の力が最も高いOB(役割なし)	`E_mult` ×1.06(コーチのレベル +1 ぶん。03 §4.6 の `coach` 0.06/Lv)	48週
`ScoutHelper`(スカウト協力)	OBがスカウトに協力する(OBが10人以上。6月〜2月)	卒業時の力が最も高いOB(役割なし)	`scout_pts` +4	48週
`Sponsor`(スポンサー)	OBがスポンサーを申し出る(OBが12人以上、名声500以上)	卒業時の力が最も高いOB(役割なし)	`funds_year` +200 万円	48週

- 1人のOBが同時に持てる役割は1つ。役割のないOBがいなければ、イベントは起こっても効果はない(検算では十分な数のOBがいる)。
- 役割のあるOBの名前と顔は、画面(OB名簿・チーム画面)に出す。

---

9. 特殊能力の覚醒

特殊能力(02 §3.7)は、**取得条件(`acquire`)を満たした週に、ランクごとの確率で覚醒する**。覚醒は `category = Awakening` のイベントとして、特殊能力ごとに自動で作る(`id = ev.awk.<skill>`、`scope = runner`、`once = runner`、`priority = 95`、`exempt = true`、選択肢は『覚える』の1つ = お知らせ)。

- 条件式は `not has_skill(r, '<id>') and (<acquire>)`。確率は `K_AWAKEN_P`(Blue 0.10 / Gold 0.05 / Red 0.08。1ヶ月以内に覚醒する確率は 34% / 19% / 28%)。
- **排他の能力**(02 §3.7: 単独走× と 集団走○、本番弱い と 大舞台○、故障癖 と 鉄の脚)は、すでに片方を持っていれば付かない。能力は最大8個。
- 『リベンジ』だけは取得条件を持たず、『ブレーキからの再起』(§14)で『励ます』を選んだとき 35% で付く。『本番弱い』は、同じイベントで『叱咤する』を選んだとき 20% でも付く。
- 覚醒はイベントの上限(年度の上限)を数えないが、1人1ヶ月の制限と、1週2件の上限には従う。

取得条件(初期案。10 §7 で、素養・個性を使う形に改めた。素養・個性がない選手は、従来より厳しい側の条件になる)

ID	名称	ランク	取得条件(条件を満たした週に、ランクごとの確率で覚醒)
`skill.mountain_god`	山の神	Gold	`(r.gift.climber == 1 and r.stat.CLB >= 64 and ctr(r, 'hill_weeks') >= 24 and r.grade >= 2) or (r.stat.CLB >= 70 and ctr(r, 'hill_weeks') >= 32 and r.grade >= 3)`
`skill.downhill_master`	山下りの達人	Blue	`(r.gift.descender == 1 and r.stat.DWN >= 62 and ctr(r, 'hill_weeks') >= 16) or (r.stat.DWN >= 68 and ctr(r, 'hill_weeks') >= 28)`
`skill.ace_section`	エース区間○	Blue	`r.grade >= 3 and ((r.gift.ace == 1 and r.p_rank <= 4 and r.races >= 2) or (r.p_rank <= 2 and r.section_wins >= 4))`
`skill.anchor`	勝負のアンカー	Blue	`r.stat.KCK >= 64 and r.stat.MEN >= 60 and r.races >= 3`
`skill.overtake`	ごぼう抜き	Blue	`ctr(r, 'overtakes') >= 4 - r.is_chase`

ID	名称	ランク	取得条件(条件を満たした週に、ランクごとの確率で覚醒)
skill.hold_lead	逃げ切り	Blue	ctr(r, 'lead_sections') ≥ 2 - r.is_lead
skill.pack_runner	集団走〇	Blue	ctr(r, 'pack_sections') ≥ 6 - 2 * r.is_pack
skill.solo_weak	単独走×	Red	ctr(r, 'solo_sections') ≥ 5 and r.stat.MEN ≤ 52
skill.peaking_master	調整力〇	Blue	ctr(r, 'peaked_races') ≥ 8
skill.summer_boy	夏男	Blue	ctr(r, 'camps') ≥ 3 and r.stat.REC ≥ 61
skill.summer_fade	夏バテ	Red	ctr(r, 'camp_overload') ≥ 2
skill.big_stage	大舞台〇	Blue	ctr(r, 'big_wins') ≥ 3 and r.stat.MEN ≥ 60
skill.ekiden_man	駅伝男	Gold	ctr(r, 'ekiden_good') ≥ 4 and r.stat.MEN ≥ 62
skill.revenge	リベンジ	Blue	『ブレーキからの再起』の励ましでのみ取得
skill.stage_fright	本番弱い	Red	r.brakes ≥ 2 and r.stat.MEN ≤ 50
skill.brake_prone	ブレーキ癖	Red	r.brakes ≥ 3
skill.iron_legs	鉄の脚	Gold	r.stat.DUR ≥ 64 and ctr(r, 'injury_free') ≥ 96
skill.recovery_plus	回復力〇	Blue	r.stat.REC ≥ 64 and r.grade ≥ 2
skill.injury_prone	故障癖	Red	ctr(r, 'injuries') ≥ 4 and r.stat.DUR ≤ 54

- 能力の水準の目安: 検算用チームの4年生の能力は、CLB 平均56(上位10% 62)、DWN 55(61)、MEN 55(62)、DUR 57(63)、REC 54(62)、KCK 60(66)、STA 70(78)。『山の神』のような Gold は、**坂道を重点的に鍛える(03 §11 の型に合わせた育成)ことで届く**水準に置いてある。検算用チーム(全員が標準の練習計画)では Gold が出にくい(§16)。
- 条件の閾値は 03・04 の確定値に合わせて調整する(L-17)。

---

10. カウンター(ctr)と更新のフック

条件式から参照する**累計の値**。選手ごとに Runner.ctr (整数のマップ)に持ち、次のフックで更新する。

カウンター	意味	更新する場所
hill_weeks	坂道練習(ks.hill)を行った週の数(故障中を除く)	03 §6 手順1のあと(週ごと)
camps	夏合宿を過ごした回数	03 §7 月次処理(8月の第4週の終わり)
camp_overload	夏合宿の月に疲労が85以上になった回数	同上(合宿の週の疲労の最大値で判定)
injuries	故障した回数	03 §6 手順5(故障の発生時)
injury_free	故障していない連続週数	03 §6 手順5(故障の発生・継続中は 0、そうでなければ +1)
peaked_races	調整週が2以上で、調子が+1以上で出走した大会の数	04 §10(出走した選手)
overtakes	区間で順位を3つ以上上げた回数	04 §10(中継所の順位が区間の前より3つ以上上がった)
lead_sections	先頭で走った区間の数	04 §7.3(2区以降で、先頭でスタートした)
pack_sections	集団走の追走側になった区間の数	04 §7.3
solo_sections	単独走になった区間の数	04 §7.3
big_wins	チームが3位以内で、本人の区間順位が5位以内だった大会の数	04 §10
ekiden_good	調子+1以上で出走し、チームが5位以内、本人の区間順位が3位以内だった大会の数	04 §10

- 通算の races section_wins brakes は Runner.career (02 §4.1)から引く。
- カウンターは減らない(取得条件に使うだけ)。選手が卒業したら OBRecord には持ち越さない。

---

11. 04 との接続

- `race.*`（§3.2）は、04 の大会後処理（順位・区間順位・ブレーキ・記録）の**直後の EventPhase** で使える。大会週以外は 0 / 99。
- `race` フェーズのイベントは、大会週の EventPhase だけで評価する。
- `bench`（欠場）は、**次の大会**に有効。選手は欠場として扱われ、控えが自動で入る（04 §5.3。`swap_limit` は消費しない）。大会が終わったらクリアする。
- `entry`（登録に追加）は、**次の第三戦のエントリー締切**（大会の4週前）のあとから大会までの間に、1回だけ有効（04 §4）。『4年生の最後の欅』は、この週（`cal.next_race == 4` かつ次が第三戦）に起こる。
- 04 §10 のやる気・XP・やる気の更新（区間賞 +1、ブレーキ −1 など）は、イベントとは別に先に行う。イベントの効果は**その上に**重ねる（たとえば『祝勝会』にやる気の効果は付けない。優勝の +1 がすでにある）。

---

12. データ（02 への追加）

マスタ: `EventDef`（§4）を `StreamingAssets/Master/events/*.json` に置く。文言は名称テーブルのキー `ev.<id>.title` `.body` `.c.<choice_id>`（日本語先行）。特殊能力の覚醒イベントは `SkillDef.acquire` と `rank` から自動で作る（§9）。

セーブ（`SaveData.event_state`）

項目	型	説明
`fired`	map<id, { count, last_week }>	イベントごとの回数と最後に起こった週（`team` のクールダウン用）
`year_counts`	map<id, int>	今年度の回数（4月第1週に消す）。`_total` は `exempt` と覚醒を除く合計
`runner_ev`	map<runner_id, { last_month, once[], cooldown{id→week} }>	選手ごとの制限（1ヶ月に1件・一生に1回・クールダウン）
`pair_cd`	map<"id|a|b", week>	`pair` のクールダウン
`mods`	list<TimedModifier>	有効な補正。`{ kind, to('team' または runner_id), value, until, src }`
`bench` / `forced`	list<runner_id>	次の大会で欠場 / 次の第三戦に追加（§11）
`log`	リング（最新50件 = `K_EVENT_LOG_MAX`。09 §2）	起こったイベントと選択（画面の履歴用）

- `Runner.ctr`: map<string, int>（§10）。`OBRecord.role_until`: int（§8）。`settings.event_auto`: bool。
- サイズは小さい（年数に比例して増えるのは `log` のリングと `fired` のイベント数のみ）。

---

13. 検証ルール（データを読み込むときとテストで確認する）

ID	ルール
V-01	`id` が一意
V-02	すべての条件式（`cond`・`req`）が構文・変数・関数・引数の数・役割の束縛に適合する
V-03	選択肢が1つ以上あり、選択肢の `id` が一意
V-04	`default` のすべてが選択肢の `id` で、**最後の選択肢には `req` がない**
V-05	`0 < p ≤ 1`
V-06	効果の操作・対象が既知。`to = p` と `bond` は `pair` のイベントだけ。`xp` の `stat` は能力名 / `weakest` / `random`
V-07	効果の大きさが範囲内（やる気 ±1、疲労 ±20、`xu` ≤ 400、同じ対象への1つの選択肢の XP 合計 ≤ 800、資金 −1500〜+500）
V-08	補正の種類・値・期間（≤ 48週）が範囲内
V-09	`pair` に `once = runner` は使えない
V-10	`phase = race` のイベントに月の指定は不要

初期カタログ(§14 の52件 = 手書き34件 + 覚醒18件(取得条件のないものを除く))は、すべて適合する(`ref_model_06.py validate`)。

---

14. 初期イベント一覧

凡例: ID は先頭の `ev.` を省略している。「週の確率」はその週に条件を満たす当事者がいるときの発生確率(§4.1 `p`)。「★」は『おまかせ』の既定。`xu` は 0.1 XP(03 §1)。

14.1 チーム(7件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
`team.captain`	主将の指名	チーム	4月 第1・2週	`t.n_g3plus ≥ 1`	1(年1回まで)	★皆に慕われる選手を指名(絆が最も高い) → 主将(絆が最も高い3年以上)、やる気+1(主将) エースを指名(力が最も高い) → 主将(力が最も高い3年以上)、やる気+1(主将) 精神力が高い選手を指名 → 主将(MENが最も高い3年以上)、やる気+1(主将)
`team.holiday`	年末年始の帰省	チーム	12月 第3・4週	`t.size ≥ 5`	1(年1回まで)	★遠方の選手が帰省する → 疲労-15(遠方出身)、やる気+1(遠方出身)
`team.ob_visit`	OBの来訪	チーム	通年	`ob.total ≥ 6`	0.015(間隔48週)	★練習を見てもらう → 練習の効率×1.03(チーム、4週) 食事会を開く〔t.funds >= 150〕 → 資金-50万円、絆+25(全員の全員の間) お断りする → なし
`team.ob_coach`	OBがコーチを申し出る	チーム	通年	`ob.corp ≥ 3 and staff.coach ≤ 3`	0.01(間隔144週)	★お願いする → OBに役割『Coach』(48週) お断りする → なし
`team.ob_scout`	OBがスカウトに協力する	チーム	6月・7月・8月・9月・10月・11月・12月・1月・2月	`ob.total ≥ 10`	0.02(間隔48週、年1回まで)	★お願いする → OBに役割『ScoutHelper』(48週) お断りする → なし
`team.sponsor`	OBがスポンサーを申し出る	チーム	通年	`ob.total ≥ 12 and t.prestige ≥ 500`	0.01(間隔144週)	★お受けする → OBに役割『Sponsor』(48週) お断りする → なし
`team.slump`	チームの停滞	チーム	通年	`t.morale ≤ 35`	0.15(間隔12週)	★全体ミーティング〔t.guidance >= 1〕 → 指導ポイント-1、やる気+1(全員)(50%) 休養日にする → 疲労-10(全員)、練習の効率×0.95(チーム、1週) そのまま続ける → なし

14.2 経営(3件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
team.univ_bonus	期待目標の達成(予算ボーナス)	チーム	3月 第2・3週	t.rank.newyear ≤ max(1, t.last_rank.newyear - 1)	1(年1回まで)	★予算ボーナスを受け取る(目標: 前年度の第三戦より上の順位。首位なら維持) → 資金+200万円
team.dorm_meal	寮の食事の改善案	チーム	通年	fac.dorm ≤ 3 and t.funds ≥ 250	0.02(間隔96週)	改善する〔t.funds >= 250〕 → 資金-150万円、疲労の回復率+0.02(チーム、24週) ★見送る → なし
team.camp_abroad	海外遠征合宿	チーム	6月・8月・2月	t.funds ≥ 1500 and fac.track ≥ 4	0.1(間隔40週)	遠征する〔t.funds >= 1500〕 → 資金-800万円、STAに XP +100xu(主力(力の上位10人))、MENに XP +100xu(主力(力の上位10人))、やる気+1(主力(力の上位10人))、練習の効率×1.03(チーム、8週) ★見送る → なし

14.3 個人(12件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
ind.camp_hell	夏合宿の洗礼	選手	8月 第2・3週	r.grade == 1 and r.fatigue ≥ 60	0.6(間隔48週、年1回まで)	★休ませる → 疲労-15(本人) 続行させる → STAに XP +150xu(本人)、故障率×1.5(本人、2週) 先輩に任せる → 疲労-8(本人)、やる気+1(本人)(50%)
ind.slump	スランプ	選手	通年	r.motivation ≤ 2 and (r.fatigue ≥ 60 or r.form ≤ -1) and r.injury == 0	0.1(間隔16週)	個別面談〔t.guidance >= 1〕 → 指導ポイント-1、やる気+1(本人) ★休養させる → 疲労-20(本人)、練習の効率×0.9(本人、1週) 練習で乗り切らせる → 最も低い能力に XP +100xu(本人)、故障率×1.5(本人、4週)、やる気-1(本人)(40%)
ind.awakening	覚醒	選手	通年	r.grade ≥ 3 and r.motivation ≥ 3 and r.form ≥ 0 and r.gap_avg ≥ 12 and r.injury == 0	0.006(間隔96週)	★能力が急に伸びた → 最も低い能力に XP +250xu(本人)、やる気+1(本人)
ind.family_cheer	家族の応援	選手	通年	cal.next_race ≤ 2 and r.regular == 1 and r.injury == 0	0.06(間隔40週)	★家族が応援に来る → やる気+1(本人)、次の大会の調子+2(本人、3週)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
ind.homesick	ホームシック	選手	通年	r.grade == 1 and r.far == 1 and r.overseas == 0 and r.weeks_in ≤ 16	0.04(1人1回)	★同郷の先輩に任せる〔senior_same_pref(r)〕→ 絆+30(同郷の仲間の全員の間) 個別面談〔t.guidance >= 1〕→ 指導ポイント-1、やる気+1(本人) 様子を見る → やる気-1(本人)(60%)
ind.overseas_isolation	留学生の生活の不安	選手	通年	r.overseas == 1 and r.grade == 1 and r.weeks_in ≤ 20	0.1(1人1回)	★相談役をつける〔t.guidance >= 1〕→ 指導ポイント-1、やる気+1(本人)、絆+10(同じグループの全員の間) チームで食事会〔t.funds >= 100〕→ 資金-30万円、絆+20(全員の全員の間)、やる気+1(本人)(50%) 様子を見る → やる気-1(本人)(50%)
ind.overseas_return	留学生の一時帰国	選手	8月 第4週	r.overseas == 1	1(年1回まで)	★家族に会って戻る → やる気+1(本人)、疲労-10(本人)、資金-20万円
ind.rival_reunion	同じ高校のライバルとの再会	選手	通年	rival_same_hs(r) and cal.next_race ≤ 2 and r.regular == 1	0.055(間隔48週)	★切磋琢磨 → やる気+1(本人)、MENに XP +100xu(本人)、次の大会の調子+1(本人、3週)
ind.injury_return	故障からの復帰	選手	通年	r.injury == 3 and r.injury_weeks == 1	1	★慎重に復帰させる → 故障率×0.8(本人、8週) 早期復帰させる → 故障率×1.6(本人、8週)、STAに XP +50xu(本人)
ind.mentor	先輩の背中	2人	通年	r.grade ≤ 2 and p.grade ≥ r.grade + 2 and bond(r, p) ≥ 400 and r.injury == 0	0.011(間隔48週)	★先輩に学ぶ → 最も低い能力に XP +120xu(本人)、2人の絆+50、やる気+1(本人)
ind.anemia	貧血の疑い	選手	通年	r.fatigue ≥ 70 and r.stat.REC ≤ 52 and r.injury == 0	0.05(間隔48週)	★検査と栄養指導〔t.funds >= 100〕→ 資金-20万円、疲労-15(本人)、疲労の回復率 +0.02(本人、8週) 様子を見る → 故障率×1.3(本人、8週)
ind.pb_meet	記録会で自己ベスト	選手	5月·6月 第3週	r.motivation ≤ 4 and r.fatigue ≤ 50 and r.form ≥ 0 and r.injury == 0	0.12(間隔40週、年1回まで)	★自己ベストを更新 → やる気+1(本人)

14.4 本番(7件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
race.brake_recover	ブレーキからの再起	選手	大会週	r.race.brake == 1	1	★励ます → やる気+1(本人)、MENに XP +100xu(本人)、特殊能力『revenge』(35%) 叱咤する → MENに XP +250xu(本人)、やる気-1(本人)、特殊能力『stage_fright』(20%) 放っておく → やる気-1(本人)(50%)
race.win_party	祝勝会	チーム	大会週	race.kind > 0 and race.team_rank == 1	0.6	★祝勝会を開く〔t.funds >= 150〕 → 資金-60万円、絆+20(全員の全員の間)、疲労-5(全員) 簡単に済ませる → なし
race.record	区間記録の更新(自校の走者が、その大会で区間記録を更新した。04 §8.3)	選手	大会週	r.race.record == 1	1	★注目が集まる → 資金+100万円、スカウトポイント+2
race.media	取材の殺到	チーム	大会週	race.kind > 0 and race.team_rank == 1 and t.prestige ≥ 700	0.5	★取材に応じる → 資金+150万円、スカウトポイント+2、疲労+5(主力(力の上位10人))) お断りする → なし
race.sub_awaken	補欠の意地	選手	大会週	race.kind == 3 and r.grade ≥ 3 and r.race.in_lineup == 0 and r.p_rank ≥ 11 and r.p_rank ≤ 20 and r.motivation ≥ 3	0.15	★補欠から力をつけた → 最も低い能力に XP +200xu(本人)、やる気+1(本人)
race.last_relay	4年生の最後の襷	選手	通年	r.grade == 4 and r.regular == 0 and r.p_rank ≤ 24 and cal.next_race_kind == 3 and cal.next_race == 4	0.6(年2回まで)	★エントリーに入れる → エントリーに追加、やる気+1(本人)、絆+10(全員の全員の間) 入れない → やる気-1(本人)(50%)
race.late_injury	直前の軽傷	選手	通年	cal.next_race ≤ 1 and r.regular == 1 and r.injury == 1	1	★欠場させる(補欠が走る) → 欠場(控えが走る) 強行出走させる(大会後に故障しやすくなる) → 故障率×1.5(本人、4週)

14.5 郷土(3件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
home.pref_assoc	県人会	選手	通年	same_pref_n(r) ≥ 2 and ob_level(r) ≥ 1	0.01(間隔48週)	★参加する(差し入れをもらう) → 資金+30万円、絆+40(同郷の仲間の全員の間)、やる気+1(同郷の仲間) 見送る → なし
home.ob_visit	県のOBの来訪	選手	通年	ob_level(r) ≥ 2 and r.grade ≤ 2 and r.injury == 0	0.008(間隔48週)	★同郷のOBが激励に来る → 最も低い能力に XP +120xu(本人)、やる気+1(本人)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
`home.senior_reunion`	同郷の先輩	2人	通年	`r.pref > 0 and r.pref == p.pref and p.grade ⩾ r.grade + 1 and bond(r, p) < 200`	0.008(間隔144週)	★同郷の先輩と親しくなる → 2人の絆+80、やる気+1(本人)(50%)

14.6 卒業(2件)

ID	名称	範囲	時期	条件	週の確率	選択肢 → 効果(既定は★)
`team.send_off`	追いコン(卒業生を送る会)	チーム	3月 第2·3週	`t.n_g4 ⩾ 1`	1(年1回まで)	★卒業生を送る → 資金-30万円、やる気+1(3年生)、絆+15(全員の全員の間)
`grad.hall_of_fame`	殿堂入りの発表	選手	3月 第3週	`r.grade == 4 and (r.section_wins ⩾ 3 or r.stat.CLB + r.stat.STA ⩾ 170)`	1(年3回まで)	★卒業生が殿堂入りする → スカウトポイント+1

14.7 覚醒イベント(特殊能力。自動生成)

『覚醒: ○○』(`ev.awk.<skill>`)。取得条件は §9 の表。1ヶ月以内に覚醒する確率は Blue 34% / Gold 19% / Red 28%。

---

15. 計算例(テストベクタ。`verify/ref_model_06.py vectors`)

TV-1 条件式の構文と優先順位

`and` は `or` より強く、`not` は比較より弱い(比較のあとに `not` が付く)。`-` は単項。

式	構文木
`r.grade == 1 and r.far == 1 or r.overseas == 1`	`or( and( r.grade==1, r.far==1 ), r.overseas==1 )`
`not r.captain == 1 and r.fatigue ⩾ 60`	`and( not( r.captain==1 ), r.fatigue⩾60 )`
`max(1, t.last_rank.newyear - 1) + 2 * 3`	`+( max(1, -(t.last_rank.newyear, 1)), *(2, 3) )`
`-r.form < 1`	`<( neg(r.form), 1 )`

TV-2 検証(不適合の例)

式	役割	結果
`r.grade == 1 and q.x`	`r`	未知の変数 `q.x`
`p.grade ⩾ 2`	`r` のみ	束縛されていない役割 `p`
`max(1)`	—	引数の数が合わない
`r.stat.XXX > 1`	`r`	未知の変数
`r.grade ==`	—	構文エラー

TV-3 評価

選手 `r` : 学年1、遠方出身、留学生でない、疲労65、調子−1、やる気2、故障なし、主将でない、出身県13。`p` : 学年3、出身県13。`r` と `p` の絆 450。`cal.abs_week = 18` (8月の第3週)、入学から18週。前年度の第三戦の順位は 8。

式	結果
`r.grade == 1 and r.far == 1 and r.overseas == 0 and r.weeks_in ⩽ 16`	偽(`weeks_in` は18)
`r.motivation ⩽ 2 and (r.fatigue ⩾ 60 or r.form ⩽ -1) and r.injury == 0`	真
`bond(r, p) ⩾ 400 and p.grade ⩾ r.grade + 2`	真

式	結果
`cal.month == 4 and cal.week == 2`	真
`same_pref_n(r) ≥ 1`	真
`max(1, t.last_rank.newyear - 1)`	7

TV-4 イベントの選択(PCG32。`master_seed = 20260101`、`abs_week = 100`)

発生の判定の乱数 `derive(20260101, 5, 100, 0)` の最初の6つ: `0.428493, 0.560746, 0.236736, 0.974129, 0.569393, 0.713001`。

- イベント A(`p = 0.25`、当事者の候補3人): `u1 = 0.4285 ≥ 0.25` → **発生しない**(`u2` は引くが使わない)。
- イベント B(`p = 1.0`、当事者の候補2人): `u1 = 0.2367 < 1.0` → **発生**。当事者は `floor(0.9741 × 2) = 1` 番目(0始まり)。

TV-5 効果の適用

- 能力STA60、XP 500 xu の選手に `xp +200 xu`: 閾値 `T = round(10 × (20 + 0.6×60)) = 560 xu` → XP 700 ≥ 560 でレベルアップ、**STA 61、XP 140**(03 §6 手順3)。
- 補正の合成: `E_mult` = 1.03(OBの来訪)× 1.06(OBコーチ)× 0.9(個人)= **0.9826**。1.2 × 1.15 = 1.38 は上限で **1.2**。
- 排他:『集団走○』を持つ選手に『単独走×』は付かない。能力は8個まで。

TV-6 覚醒の確率

Blue: 条件を満たした週に 10%(1ヶ月以内 34.4%、期待 10週)/ Gold: 5%(18.5%、20週)/ Red: 8%(28.4%、12週)。

TV-7 検証

初期カタログ52件(手書き34件 + 覚醒18件)は V-01〜V-10 に適合する。

---

16. 検算結果(`ref_model_06.py sim`)

16.1 方法と限界

- 簡易チームシミュレーション: **03 の週の解決(`resolve_week`)をそのまま使い**、36人(各学年9人)を、3年のウォームアップのあと**10年度 × 12回**測った。毎年4月に9人が入学し、3月に4年生が卒業してOBになる(OBは72人から始める)。
- 選手の出身県は 05 の供給の比率(関東22%・中部19%・近畿17%・九州15% など)、留学生は3%、距離帯は 02 §3.1。絆は 03 §7.1、やる気は 03 §7.2 の M1・M2・M4〜M6 と 04 §10 に従う。
- **大会は乱数で結果を作る**(順位・区間順位・ブレーキ・集団/単独走・記録)。04 の本番式ではなく、イベントの条件に必要な事象だけを再現している。『他校に同じ出身高校の選手がいる』は確率 0.40 で近似。
- 入学生には**個性・素養を付ける**(10 §4。`Sim.individuality`。能力は 06 のまま)。練習計画は標準(03 §11)の1種類のみ。**素養のある選手に山対策を続けさせない(個性に合わせた育成をしない)ので、Gold の『山の神』は出ない**(§16.4。個性に合わせた育成は 10 §7 で別に検算した)。
- シナリオ: 中堅(資金1100で頭打ち、施設Lv2)/ 後半(資金が貯まり続ける、施設・スタッフがLv5)。選択の方針: おまかせ(既定)/ すべての選択肢を一様に選ぶ。

16.2 件数と所要時間(1年度あたり)

シナリオ・方針	イベント	うち選択あり	お知らせ	覚醒	所要時間	最大(件/年)
中堅・おまかせ	13.7	7.2	6.5	5.0	**133秒**	22
中堅・一様	13.5	7.1	6.4	5.1	131秒	22
後半・おまかせ	13.0	6.2	6.7	7.6	132秒	19

- 所要時間 = 選択あり 12秒 + お知らせ・覚醒 4秒(§2)。**予算150秒に収まる**(01 §3.4 の『イベント・成長演出 約3分』のうち約2分。残りは成長演出)。
- 件数の標準偏差は 2.8。週の上限(2件)で落ちた回数は 年に約3.1回(固定の行事は2週の窓で翌週に回る)。
- 種別(中堅・おまかせ): チーム 3.2 / 個人 5.2 / 本番 1.7 / 郷土 1.1 / 経営 0.8 / 卒業 1.7 / 覚醒 5.0。

16.3 イベントごとの回数(中堅・おまかせ、1年度あたり)

イベント	回数	イベント	回数	イベント	回数
故障からの復帰	1.12	主将の指名	1.00	年末年始の帰省	1.00
追いコン(卒業生を送る会)	1.00	夏合宿の洗礼	0.75	殿堂入りの発表	0.73
ホームシック	0.71	留学生の一時帰国	0.68	4年生の最後の襷	0.62
期待目標の達成(予算ボーナス)	0.47	家族の応援	0.47	OBの来訪	0.45
先輩の背中	0.45	ブレーキからの再起	0.44	同じ高校のライバルとの再会	0.41
県人会	0.41	県のOBの来訪	0.37	寮の食事の改善案	0.36
同郷の先輩	0.35	OBがスカウトに協力する	0.34	覚醒	0.30
祝勝会	0.25	OBがスポンサーを申し出る	0.22	留学生の生活の不安	0.19
OBがコーチを申し出る	0.19	補欠の意地	0.18	直前の軽傷	0.13
貧血の疑い	0.06	記録会で自己ベスト	0.04	区間記録の更新	0.03

- 中堅の 120 年度で、『取材の殺到』(名声700以上の優勝)・『スランプ』・『チームの停滞』は起こらなかった(『取材の殺到』は世界シミュレーションの頂点付近では起こる。『スランプ』『チームの停滞』が稀なのは、チームの士気が35以下になることが稀なため。安全網と M4 が働く)。『海外遠征合宿』は後半のシナリオで起こる(§16.6)。

16.4 特殊能力

- 卒業時の平均所持数: 中堅 **0.55個**(後半 0.82個)。1年度あたりの取得(チーム): Blue 4.4 / Gold 0.5 / Red 0.23(後半: 6.2 / 1.45 / 0.10)。
- 覚醒の多い順(中堅): 夏男 1.36 / 逃げ切り 0.62 / 調整力○ 0.50 / 鉄の脚 0.47 / エース区間○ 0.45 / 回復力○ 0.45 / 集団走○ 0.39 / 勝負のアンカー 0.36 / 故障癖 0.15 / 単独走× 0.07 / 山下りの達人 0.07 / 大舞台○ 0.06 / 駅伝男 0.02 / その他 0.01以下(『山の神』『ごぼう抜き』は 0)。
- **素養・個性を取得条件に入れた**(10 §7。§9 の表)。個性・素養のない従来の条件と比べて、**山下りの達人(0.60 → 0.07 /年)とエース区間○(0.59 → 0.45)は減り、逃げ切り(0.23 → 0.62)と集団走○(0.18 → 0.39)は増えた**。素養のない選手は、標準の練習では山下りの達人・エース区間○にほとんど届かない(素養が道を作る)。個性が合う選手は、逃げ切り・集団走○を取りやすい。合計は変わらない(卒業時の平均 0.55 個)。
- 『山の神』と『駅伝男』(Gold)は、**重点的に鍛えないと条件に届かない**水準にしてある。標準の練習だけの検算用チームでは出ない(山の神 0、駅伝男 年 0.02)。**素養(登坂)のある選手が山対策を続けると**、山の神は素養ありの約 18%(卒業生 10 人/年で約 6 年に 1 人)が取れる(10 §7 の K4。L-17 の一部を解決)。
- 『ごぼう抜き』は、検算では追い抜きの回数(`overtakes`)がほとんど増えず、取得率が測れていない(L-18 と同根。大会の事象を確率で近似しているため)。
- 負の能力(Red)は少ない(年 0.23)。ブレーキ・故障の累積が条件なので、順調なチームでは付かない。

16.5 効果の大きさ(選手1人・1年度あたり)

項目	おまかせ	一様	備考
イベントの XP(能力点に換算)	0.01	0.02	560 xu = 能力1点
補正による XP(能力点に換算)	0.29	0.18	`E_mult` の延長ぶん
練習による成長(能力点)	24.0	24.0	03(個性・素養あり)
イベント・補正の割合	**1.3%**	**0.8%**	練習の成長に対して
やる気の変化(合計)	+0.37	+0.36	負の変化も含む

項目	おまかせ	一様	備考
疲労の変化(合計)	−9.0	−8.4	週平均 −0.2
資金の変化(チーム・年度あたり)	+94万円	+24万円	収入は約2100万円

- **イベントによる強さの変化は、練習の成長の約1%。**05 のバランス(頂点の高さ・クリア年度)には影響しない。資金の +90万円/年を 05 の世界シミュレーションに足して確認した(中堅・弱小とも40回): 優勝率 0.466 → 0.490 / 0.411 → 0.414、クリア年度の中央値 13 → 12 / 26 → 24。**誤差の範囲**(検算の標準誤差は優勝率で ±0.03)。
- 選択の仕方で結果はほとんど変わらない(おまかせと一様で、件数・時間・効果の大きさがほぼ同じ)。**イベントは判断の手触りを足すもので、勝敗を左右する仕掛けではない。**

16.6 資金の使い道(課題 L-05)

後半のシナリオ(資金が貯まり続ける)で、『海外遠征合宿』(800万円)は 年に 0.65 回起こり、年に約 −520万円。収支の純増は +123万円/年(おまかせ)。05 §4.3 が見積もった後半の純増(年 1000万円前後)に対して、一部を使うだけで、**L-05 は一部の解決**にとどまる(資金の大きな使い道は未設計)。

---

17. 課題・未決・他文書への反映

17.1 課題トラッカー(`00_spec_index.md` §6)

ID	状態	内容
L-05	一部解決	『海外遠征合宿』を追加した(年に約 −520万円。効果は小さい)。大きな資金の使い道(記念事業・施設の新設など)は未設計
L-17	一部解決(10 §7)	取得条件に素養・個性を入れた。『山の神』は素養(登坂)のある選手が山対策を続けると取れる(素養ありの約 18%)。『駅伝男』(Gold)の水準と『ごぼう抜き』の取得率は未検証(04 の本番式を入れたシミュレーションで再調整する)
L-18	新規	`rival_same_hs` (出身高校が同じ他校の先発)・大会の結果の事象(集団走・単独走・追い抜き)は、検算では確率で近似した。本物は 05 の世界と 04 の本番式から取る
L-19	新規	イベントの文言(約100本)と立ち絵・演出は未作成。名称テーブルのキーだけを定義した

17.2 反映した文書

文書	変更
02	`Runner.ctr` 、`SaveData.event_state` 、`OBRecord.role_until` 、`settings.event_auto` 、`EventCategory` / `ModKind` 列挙、`TimedModifier` を追加(v0.7)
03	週の順序に EventPhase を追加。補正(`E_mult` など)を §6 の手順1・4・5・6 が受け取る。カウンターの更新フック(§10)を §8 に追加
04	大会後処理のあとに EventPhase を起動。カウンター(`peaked_races` など)の更新を §10 に追加。`bench` / `entry` の消費(§11)
05	年度の収入に `funds_year` の補正を足す(§4.1)。スカウトポイントに `scout_pts` の補正を足す(§7.1)。OBの役割は 06 §8(§10.4)
01	§9 のイベントシステムを、件数・時間・種別の確定値に更新
00	06 を v0.1 に、課題 L-05・L-17〜L-19 を更新
08	イベントの表示(SC-36)・効果の要約(R-15)・止まる規則(§6.3)
10	§3.2 に DSL の変数(`r.gift.*` `r.is_*`)、§9 の取得条件(素養・個性を使う形)、§16 の再検算(個性・素養を付けた簡易シミュレーション)

17.3 次の文書への引き継ぎ

引き継ぎ先	内容
07	特別招聘の選手は通常のイベントの当事者になる(専用イベントなし。§1)。『無招聘』の実績の判定は 02 §3.10
08(完了)	イベント画面(SC-36。選択あり・お知らせ・覚醒の3種類)、イベントの履歴(通知 SC-38)、『イベントをおまかせ』の設定、OBの役割の表示(SC-33)。**選択肢の2行目に効果の要約**を出す(08 R-15)。止まるのは選択ありのイベントだけで、お知らせ・覚醒は月末レポートに集約(08 §6.3)

引き継ぎ先	内容
09(完了)	`event_state` のセーブ形式(刈り込み: `log` 50 件。09 §5.2)・読み込み時の修復(切れた補正を捨てる・`Runner.ctr` のキーの一覧外を捨てる = I-19。09 §7.1)/ `label = 5` の乱数のキーは 09 §8.2 の登録表(`b = 0` と `1000 + k` 。重なりなし)/ イベントデータ(JSON)の読み込み時の検証(§13)は起動時に 1 回(09 §12.1)

17.4 M0 の C# 実装(段階 6)

- 場所: `m0/Assets/Scripts/Domain/Events/` (`Dsl/Parser.cs` 字句・構文、`Dsl/Compiler.cs` 検証と実行形、`EventContext.cs` 変数の評価、`EventDef.cs` 定義と定数、`EventPhase.cs` 発生の判定・選択・効果、`EventValidator.cs` V-01〜V-10、`EventCatalog.g.cs` カタログ。生成元は `verify/events_06.py`)と `Team/TeamState.cs` (自校の状態: 選手・絆・補正・OB・イベントの記録)。
- 条件式は読み込み時に一度だけ実行形(変数は種類と添字に解決済み)にする。条件式の評価は、実行時の例外も割り当てもない。`EventPhase.Select` (発生の判定。状態を変えない)と `Resolve` (選択・効果・記録)を分けてあり、プレイヤーが選ぶ画面は `IChoicePolicy` を実装して `Resolve` に渡す。選択肢の選び方は発生の判定を変えない(§7.5)。
- `rival_same_hs` は `IEventWorld` 越しに尋ねる。検算用の近似(確率 0.40。`ApproxEventWorld`)のまま(L-18)。
- 参照モデルとの差分テスト(`m0/tests/Domain.Tests/EventGoldenTests.cs`): 乱数を含むイベントの発生・選択・効果を 265 状態(52 種類すべて)で、条件式を 899 式 × 105 状態で、完全一致を確認した。まれな条件(『山の神』『ブレーキ癖』『チームの停滞』など)は、状態を乱数で少し乱して試している。
- 見つかったこと: 参照モデルの `p_rank` の元になる `long_p()` が `sum()` を使っていた(D-12)。素朴な左からの和に直した。

第7部 特別招聘・追加コンテンツ仕様

原典: `07_special_recruit_spec.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> 範囲: **特別招聘券**(状態・獲得・使用・入学)、**黄金世代プラン**(永続解放の付与)、**招聘選手の生成**、**購入状態の扱い**(ストア別の確認・猶予・返金)、**追加コンテンツ**(見た目・シナリオ)と**開始状態(シナリオ)の形式**。前提: `01_basic_system_design.md` §6.5(コンセプトと原則)、`02_data_spec.md` §3.9〜§3.10(定数・データ)、`05_world_scout_spec.md` §6・§11・§12(生成・入学・年度末の順序)、`06_event_spec.md` 。**検算**: 招聘選手の生成・券の状態遷移・購入状態の判定を参照モデル `verify/ref_model_07.py` に実装した。獲得の一覧・有料プラン・シナリオの長期の影響は、05 の世界シミュレーション(`verify/ref_model_05.py` 。券の獲得・繰り延べ・生成帯・開始状態の上書きを追加)で測った(§12)。

1. 原則

#	原則
1	**永続解放のみ**: 課金は「黄金世代プラン」と「見た目・シナリオのパック」の購入だけ。消費型・確率課金(ガチャ)・期間限定は採らない(01 §6.5)。
2	**本体で完結**: 券を使わなくても、クリア・すべての実績・すべてのモードに到達できる。有料で変わるのは**券の入手が増える**ことだけ(特別招聘の効果の大きさは制約にしない。公平性は『無招聘』の実績で分ける。01 §6.5)。
3	**券は失効しない**: 入手した券は、所持上限(2枚)を超えても失わず、**繰り延べて**空いたときに使えるようにする。購入したものが無駄になる経験をさせない。
4	**購入状態はシミュレーションに影響しない**: 乱数の引き方・選手の生成・大会の結果は、購入しているかどうかで変わらない。唯一の例外は、プランが**券の付与を通じて特別招聘の使用回数を増やす**こと。見た目・シナリオは、表示と開始状態だけに作用する。
5	**セーブは購入状態で壊れない**: 未所持・返金・別のストアで開いても、セーブは読み込める。在籍する選手・宣言済みの招聘・選んだ見た目は維持する(所持していない見た目は、既定の見た目で表示するだけ)。
6	**結果の範囲を事前に見せる**: 特別招聘は、種別を選ぶと、入学する選手の能力・見込みの**範囲**が画面に出る。確率の隠蔽はない(§7)。
7	**物語・専用イベントは付けない**: 招聘選手は、通常の選手と同じ条件で通常のイベントの当事者になる(06 §1)。
8	**サーバー不要・ランキングなし**: 購入状態の確認は各ストアの機能だけを使う。サーバーでの検証は行わない(U-09)。
9	**ストアごとに独立**: 購入は Steam / App Store / Google Play ごと。v1 ではストア間で共有しない。

---

2. 定数表(初期値案。`K_TICKET_` / `K_PLAN_` 接頭辞)

定数	値	意味
`K_TICKET_STOCK_MAX`	2	手元の券(`stock`)の上限。超える入手は繰り延べる(`bank`)
`K_TICKET_PER_YEAR`	1	1年度に宣言できる回数
`K_TICKET_ACTIVE_MAX`	2	在籍する招聘選手と宣言済みの招聘の合計の上限
`K_TICKET_DECLARE_FROM` / `_TO`	月 2(6月)/ 10(2月)	宣言できる期間(`month_index`)
`K_PLAN_CYCLE_YEARS`	4	黄金世代プランの付与の周期(年度)
`K_PLAN_COUNT`	2	1回の付与の枚数
`K_OFFLINE_GRACE_DAYS`	30	購入状態を確認できないときに、前回の確認を信じる日数(§8.2)
`K_SPECIAL_MAX_STAT` / `_MAX_MEAN`	80 / 68	招聘選手の入学時の能力の上限(各能力 / 平均)
`K_SPECIAL_SKILL_P`	0.30	招聘選手が入学時に特殊能力を1つ持つ確率
乱数	`label = 3`、`a = 入学年度`、`b = 9000 + index`	招聘選手の生成(03 §3.2 の予約のうち『スカウト』)

> 使用は、年1回かつ同時在籍2名までなので、**平均して年0.5回(4年度に2回)が上限**。これを超えて入手した券は、失効せず繰り延べる(§3.2)。周期と上限は一緒に決めた(U-12 の確定: **4年度に2枚**)。

---

3. 特別招聘券

3.1 状態(`SaveData.player_team.tickets`)

項目	型	説明
`stock`	int	手元の券。0〜 `K_TICKET_STOCK_MAX`
`bank`	int	繰り延べた券(0以上)。`stock` に空きができたら移す
`last_used_year`	int	最後に宣言した年度(年1回の制限用)
`pending`	{ kind, pref?, enroll_year }?	宣言済みで、まだ入学していない招聘。なければ null
`plan_anchor_year`	int?	黄金世代プランの付与の起点年度(§5.2)。未所持なら null
`plan_granted_years`	list<int>	プランの付与を済ませた年度
`milestones`	list<string>	獲得済みの節目(§4)
`log`	list	入手・使用の履歴 `{ year, delta, source }`（`TicketSource` = `Earned` / `PlanGrant`）

- **派生**: `active` = 在籍する招聘選手(`recruit_source = Ticket`)の数 + `pending` の数(0または1)。
- **不変条件**: `0 ≤ stock ≤ 2`。`stock < 2` ならば `bank = 0`(空いたらすぐ移す)。`active ≤ K_TICKET_ACTIVE_MAX`。**入手の合計 = 使用の合計 + `stock` + `bank`**(失効しない)。

3.2 入手と繰り延べ

入手は常に `add(n, source)`：`bank += n` のあと `release()`（`stock` に空きがあるだけ、`bank` から移す)。**失効はない**。入手の経路は2つ。

- `Earned`：ゲーム内の節目(§4)。1つの節目は、そのセーブで1回だけ。
- `PlanGrant`：黄金世代プラン(§5)。

3.3 使用(宣言)

条件(すべて満たすとき、宣言できる): 今が宣言期間(`month_index` 2〜10)/ `stock ≥ 1` / `pending = null` / `last_used_year ≠` 今年度 / `active + 1 ≤ K_TICKET_ACTIVE_MAX`。

手順

1. 種別を選ぶ: **留学生**(`Intl`)または**スーパールーキー**(`Rookie`)。スーパールーキーは**出身県を指定できる**(47県から選ぶ、または『おまかせ』)。
2. 画面に、入学する選手の**範囲**(§7)と、『宣言は取り消せない』ことを出して確認する。
3. 確定すると、`stock -= 1`、`last_used_year = 今年度`、`pending = { kind, pref?, enroll_year = 今年度 + 1 }`、`run.invites_declared += 1`(02 §3.10)。`release()` で繰り延べを `stock` へ。

- **宣言は取り消せない**(誤操作は確認画面で防ぐ)。`run.invites_declared` は『無招聘』の実績の判定に使う(02 §3.10)。
- 宣言のあと、実際に入学するまでの間も、`pending` は `active` に数える。

3.4 入学の確定(05 §12 の手順6〜7)

年度末(3月)、`pending` があれば、§6 の手順で招聘選手を1人生成し、入学者に加える。

- **定員**: 招聘選手は、自校の定員10のうち**1人分を占める**。通常の入学者が10人に達していれば、`Q` **の最も低い通常の入学者1人を外す**(その候補は空きのある他校に入る)。奨学金枠は使わない。

- 入学者発表(3月)で、姿・名前・出身・能力(`Basic` と `Stats`)が分かる。素質(`Potential`)と体質(`Body`)は、通常の選手と同じく隠す。
- 初期値: 疲労20、やる気3、調子0、特殊能力は §6.5、`recruit_source = Ticket`、`ticket_kind`(05 §10.1 の初期値と同じ。やる気の上乗せはない)。
- 入学後は**通常の選手と同じ扱い**(育成・起用・卒業・OB)。卒業して OB になっても `recruit_source` は残す(§11)。

---

4. ゲーム内の獲得(`TicketMilestone`)

節目を初めて満たしたとき、1枚を入手する(`Earned`)。

ID	節目	条件(そのセーブで初めて)	判定の時点
`tm.win.fall`	第一戦の初優勝	第一戦で優勝	その大会の終了直後
`tm.win.national`	第二戦の初優勝	第二戦で優勝	同上
`tm.win.newyear`	第三戦の初優勝	第三戦で優勝	同上
`tm.podium`	初の表彰台	いずれかの大会で3位以内	同上
`tm.seed`	初のシード	第三戦で10位以内	同上
`tm.sweep`	初の三冠	同一年度に3大会すべてで優勝	第三戦の終了直後
`tm.clear`	クリア	3大会すべての優勝経験が揃った	その大会の終了直後
`tm.rank.3` / `.6` / `.9`	監督ランク 3 / 6 / 9	ランクがその値になった	年度末の精算後(05 §12 の手順3)
`tm.anniv.N`	継続の記念	10年度・20年度・30年度……を終えた(上限なし)	年度末の精算後

- 大会の終了直後に入手した券は、その年度の宣言期間のうちに使える(第三戦の直後なら1〜2月)。**年度末に入手した券は、宣言期間(6〜2月)のあとなので、次の年度から使える。**
- 45年度で入手できるのは **14枚**(大会の節目7 + ランク3 + 継続の記念4)。長く続けるほど、記念で増え続ける(10年度ごとに1枚)。
- 券の効果の大きさは、特別招聘が**1年度に1人分の勧誘判定を飛ばす**以上のものではない(01 §6.5 の懸念)。獲得の時期と枚数の検算は §12。

---

5. 黄金世代プラン(`GoldenPlan`)

5.1 内容

- 一度購入すると、**永続的に**(返金しない限り)次の規則で券が付与される。**Steam = DLC、スマホ = 非消費型**。購入の復元に対応する(§8)。
- 付与は、`K_PLAN_COUNT`(2)枚を `K_PLAN_CYCLE_YEARS`(4)年度ごと。**付与された券は `Earned` と同じ在庫に入る**(§3.2。繰り延べるので失効しない)。
- プランがなくても、ゲームは最後まで遊べる(原則2)。開発中はプランなし(券はゲーム内の獲得のみ)で進め、プランの解放は後から有効にする(`GOLDEN_PLAN_ENABLED` のビルド設定)。

5.2 付与の規則(セーブごと)

```
TryPlanGrant(year, owned):                # owned: 購入状態(True / False / None=未確認。 §8.2)
    if owned ≠ True: return
    if plan_anchor_year == null: plan_anchor_year = year        # 初めて確認した年度。ここで最初の付与
    if (year − plan_anchor_year) mod K_PLAN_CYCLE_YEARS == 0 and year ∉ plan_granted_years:
        add(K_PLAN_COUNT, PlanGrant);  plan_granted_years.add(year)
```

- `TryPlanGrant` は、年度の4月第1週、ゲームの起動時、購入状態の更新に成功したとき、購入の完了時に呼ぶ。**同じ年度に何度呼んでも、付与は1回だけ**(`plan_granted_years`)。
- **途中購入**: 購入を確認した時点で、その年度を起点に**すぐ最初の付与**を受ける(購入後に何ヶ月も待たせない)。以後は 4年度ごと。

- **オフライン**: 4月に確認できなくても、**同じ年度のうちに確認できれば付与**する(取り戻し)。**年度をまたいだ取り戻しはしない**(返金・購入を繰り返して付与を増やす抜け道を作らないため)。
- **返金**: 購入状態が `False` になったら付与を止める。**所持済みの券(`stock` と `bank`)・宣言済みの招聘・在籍する招聘選手はそのまま残る。**再購入したときは、起点を変えず、次の周期の年度から付与を再開する(再購入の直後に付与する年度でなければ、待つ)。
- プランの起点の年度が違っても、1つのセーブの付与の間隔は4年度(初回のタイミングだけがセーブごとに違う)。

---

6. 招聘選手の生成

生成は、**種別と(スーパールーキーの)出身県の指定だけを入力**に、年度ごとに決まった乱数で行う(購入状態・他の乱数の消費に影響されない。原則4)。

6.1 帯(`SpecialRecruitBand`)

項目	留学生(`Intl`)	スーパールーキー(`Rookie`)
入学時の力 `A0`	`N(66, 2)` を **[64, 68]** に収める	`N(61, 2)` を **[58, 64]** に収める
ギャップ `gap`	`N(8, 2)` を [4, 12] に収める	`N(25, 3)` を [18, 32] に収める
`Q = A0 + 0.6×gap`	平均 70.8(5〜95%: 67.8〜73.8)	平均 76.0(71.9〜80.0)
役割	**即戦力型**: 入学時から強いが、伸びしろは小さい	**育成型**: 入学時はやや低いが、伸びしろが大きい
出走の制約	留学生枠(登録2・出走1。04 §4・§5.2)	なし
出身	出身国(24か国から一様)	出身県(指定、または一様)。高校は県内から `strength` に比例して抽選

- 差は**国による能力補正ではなく、種別の役割分担**(原則・01 §6.5)。能力・外見は出身国によって変えない(§6.3)。

6.2 手順(乱数の引く順序は固定)

乱数 `Pcg32(derive(master_seed, 3, 入学年度, 9000 + index))` (`index` = その年度の何人目の招聘か。通常は 0)。

順	引くもの	備考
①	`A0` (正規乱数。`uniform` ×2)	範囲に収める
②	`gap` (正規乱数。×2)	同上
③	成長タイプ(`uniform`)	早熟25% / 標準55% / 晩成20%(05 §2 と同じ)
④	型(`uniform`)	バランス35 / 持久25 / スピード25 / 山15(05 §6.1)
⑤	能力の偏り `n10` (正規乱数 ×10。最初の6個は `N(0,3)` 、CON・MEN・REC・DUR は `N(0,4)`)	
⑥	素質の揺れ(`N(0,3)` ×10)	
⑦	特殊能力の有無と種類(`uniform` ×2)	§6.5
⑧	出身(国または県)と高校(`uniform` ×2)	指定があっても引く(消費を変えない)
⑨	`look_seed` (`u32`)	外見・名前(§6.4)

```
stat_i   = clamp(round_half_up(A0 + 型の偏り_i + n10_i), 20, 95)          # 05 §6.2 と同じ
d        = max(0, max(stat) - 80, ceil(mean(stat) - 68))
stat_i  -= d                                                              # 上限を超えるときは、全能力を同じだけ下げる(型の形を保つ)
pot_i    = min(100, stat_i + max(2, round_half_up(gap + 素質の揺れ_i)))
```

6.3 出身

- **留学生**: 出身国を、次の24か国から**一様に**抽選する。**国による能力・外見・成長の補正は設けない。**人物を国の印象で描かない(06 §1 の原則7)。

ケニア / エチオピア / タンザニア / ウガンダ / モンゴル / 韓国 / 中国 / ベトナム / タイ / インド / インドネシア / トルコ / アメリカ / カナダ / ブラジル / メキシコ / オーストラリア / ニュージーランド / イギリス / フランス / ドイツ / スペイン / ポーランド / スウェーデン (初期案。実大会の留学生の出身を意識しつつ、特定の地域に偏らないようにした。名称・並びは差し替え可能な名称テーブルで持ち、人物描写への配慮の観点で見直す。)

- **スーパールーキー**: 出身県をプレイヤーが指定できる(愛着と県別OBネットワークの育成に使える)。**指定しても能力は変わらない**(強豪県の基準値 +3 も適用しない)。出身高校は県内から抽選する。
- 距離帯は 02 §3.1(留学生は3)。

6.4 外見・名前

- `look_seed` から、外見(02 §4.2)と名前を作る。**外見は出身国・出身県によって変えない**(髪型・髪色・顔・体型・肌の色を、すべての選択肢から一様に引く)。
- 名前は、スーパールーキーは日本人の名前テーブル、留学生は**国ごとの名前テーブル**から作る(テーブルはコンテンツとして用意する。L-20)。**M1 準備では、日本人の名前テーブルを実装し、留学生は国の印象と結びつけない架空の音の組み合わせの仮の名前にした**(09 §8.2。国別の表は L-20 のまま)。

6.5 特殊能力

- 確率 `K_SPECIAL_SKILL_P` (30%)で1つ持つ。種類は、次の Blue から**一様**に選ぶ: 山下りの達人 / エース区間〇 / 勝負のアンカー / ごぼう抜き / 逃げ切り / 集団走〇 / 調整力〇 / 夏男 / 大舞台〇 / 回復力〇。
- **Gold・Red・リベンジは付けない**(強すぎる・負の能力は入学時に持たせない・リベンジはイベント専用)。
- **個性と素養**(10 §4): 通常の入学生と**同じ関数・同じ分布**で付ける(鍵は 09 §8.2 の label 8・`look_seed`・b = 40 / 41)。個性の『型』は、④ で引いた型(バランス / 持久 / スピード / 山)から決める。招聘だからといって素養を保証・増やしたりしない(他の生成値と同じく『役割分担』は能力の帯だけ)。素養は入部から 12 週で判明する(10 §8)。

6.6 検証ルール(生成時・テスト時)

ID	ルール
S-1	各能力 ≤ 80、10能力の平均 ≤ 68
S-2	`stats[i] ≤ potential[i] ≤ 100`
S-3	特殊能力は0〜1個で、§6.5 の候補だけ
S-4	`A0・gap` が §6.1 の範囲内
S-5	`recruit_source = Ticket`、`ticket_kind` が設定されている

---

7. 宣言の画面に出す範囲

確率ではなく、**入学する選手の範囲**を出す(原則6)。値は §6.1 と §12.1 の検算による。

	留学生(即戦力型)	スーパールーキー(育成型)
入学時の力	64〜68	58〜64
卒業時の見込み(`Q`)	約68〜74	約72〜80
素質	小さい(平均 74)	大きい(平均 86)
型・成長タイプ	ランダム	ランダム
特殊能力	30% で1つ(Blue)	30% で1つ(Blue)
出走	留学生枠あり(登録2・出走1)	制限なし
出身	24か国からランダム	県を指定(またはランダム)

- 参考(05 §6.1 の候補、検算): 通常の候補の入学時の力は平均 37、卒業時の見込み `Q` は平均 52(上位級 `V ≥ 51.5` の候補でも平均 65、全体の上位5%で 67 以上)。招聘選手の `Q` は、留学生が約71(通常の候補の上位3%程度)、スーパールーキーが約

76(ほぼ最上位)。それでも、能力・素質の上限(§6.6)があり、1人で戦力差を覆すほどではない(§12.2)。

---

8. 購入状態(エンタイトルメント)

8.1 ストア別の確認

購入状態は、**ストアの機能だけ**で確認する(サーバー不要。原則8)。本体は `IEntitlementProvider` を通して `owned` / `not_owned` / `unknown` を受け取る。

ストア	商品の種類	確認の方法(実装時に最新のAPIを確認)	復元
Steam	DLC(AppID ごと)	`SteamApps.BIsSubscribedApp(dlcAppId)` (Family Sharing での所有を含む)	不要(アカウントの所有)
App Store	非消費型	StoreKit の現在の権利(`currentEntitlements`)/ Unity IAP	**『購入を復元』ボタンが必須**
Google Play	非消費型(買い切り)	Play Billing の購入の照会(`queryPurchasesAsync`)。購入の**承認(acknowledge)を3日以内**に行う	起動時に自動で再照会

- 確認のタイミング: **起動時**(待たせない。結果が来るまで `unknown` として進む)、新規セーブの開始時、**年度の4月第1週**、設定・商品の画面を開いたとき、購入の完了時。
- 結果は **`LocalState.entitlements[pack_id] = { owned, verified_at, verified_run }`** (端末ごと・同期しない。09 §3.3)に保存する。購入そのものはストアのアカウントに紐づくが、確認の時刻と『動かした時間』は端末ごとの量なので、`ProfileData` (同期する)には持たない(09 v0.1 で 02 §3.10 から移した)。問い合わせの待ちは `K_STORE_TIMEOUT_S` (5秒)で、待たずに進める(09 §10)。

8.2 判定と猶予

> **09 §10 の追加**: 端末の時計を操作して猶予を延ばす抜け道を塞ぐため、次の判定に ①**時計の巻き戻しの検知**(高水位 `clock_hw` 。24時間の許容)と ②**アプリを動かした時間の上限**(猶予のあいだに 120 時間。単調時計で数える)を足す。ゲームの結果・実績・既に得た券・選んだ見た目には影響しない(原則4・5)。下の判定は、時計が正しいときの 07 の規則そのもので、09 の `EntitlementGuarded` はこれと一致する(09 TV-10)。

```
EntitlementState(provider, cache, now):
    provider == owned      → True
    provider == not_owned  → False                    # 返金・未購入
    provider == unknown:   if cache.owned and now − cache.verified_at ≤ K_OFFLINE_GRACE_DAYS(30日): True
                           else None                  # 未確認
```

- `None` (未確認)のときは、付与も停止もしない(§5.2 の `TryPlanGrant` は何もしない)。画面の隅に『購入状態を確認できません』を出し、次の確認の機会に再度判定する。
- 初めて確認する前(キャッシュなし)にオフラインなら `None` 。購入済みでも、確認できるまで付与は待つ(同じ年度のうちに確認できれば付与。§5.2)。

8.3 返金・別の端末・別のストア

状況	扱い
返金(ストアが `not_owned` を返す)	プランの付与を止める。**所持済みの券・宣言済みの招聘・在籍する招聘選手・選んだ見た目はそのまま**(原則5)。見た目パックは、所持していない見た目を既定の見た目で表示する(選択は保持し、再購入で戻る)
別の端末(同じストア・同じアカウント)	クラウドセーブ(Steam Cloud / iCloud / Google Play Games)で同じセーブを使える。購入状態はそのアカウントのもの
別のストアへ移したセーブ	v1 では、ストア間のクラウド共有・購入の共有はしない。もしセーブのファイルを別のストア版に持ち込んで読み込めた場合、そのストアで未購入ならプランの付与は止まり、他は上記のとおり維持する
Family Sharing(Steam)	DLC の所有として `owned` (ストアの仕様に従う)

8.4 テストと開発

- ストアを使わないテスト用に、`FakeEntitlementProvider`(状態を設定できる)を用意する。`ref_model_07.py` の判定(TV-3)をそのまま自動テストに写す。
- 開発中は `GOLDEN_PLAN_ENABLED = false`(プランなし)。有効にするのは、ストアの商品を登録したあと。

8.5 ストア対応のチェックリスト(実装・申請の前に)

項目	内容
Steam	DLC ごとにストアページと AppID を作る。実績(02 §3.10)とクラウドセーブを有効にする
App Store	非消費型の商品を登録。『購入を復元』を設定画面に置く。年齢レーティング(ランダム要素のある課金がないことを説明できる)
Google Play	買い切り商品を登録。購入の承認(3日以内)。返金後の権利の取り消しに対応(再照会)
共通	価格は U-05 で本体と一緒に決める。地域別の価格・税表示はストアの設定に従う。購入の説明には、**『券を使わなくても、クリアまで遊べる』『確率で結果が変わらない』**を書く

---

9. 追加コンテンツ(`ContentPack`)

9.1 種類

PackKind	内容	勝敗への影響
`GoldenPlan`	特別招聘券の付与(§5)。`contents` は空	券の使用回数だけ(原則4)
`Cosmetic`	見た目(ユニフォーム柄・校章・髪型・髪色・応援歌・BGM)	**なし**(表示だけ)
`Scenario`	追加の開始状態(§10)	なし(開始状態だけ。世界の規則は変えない)

9.2 見た目(`Cosmetic`)

見た目は、**表示の層だけ**に作用する。選手の生成・乱数・大会の結果は、所持している見た目と無関係(原則4)。

項目	本体に含まれる数(初期案)	パックで増える数(初期案)	適用の範囲
ユニフォーム柄	4	+12	自校のユニフォーム(`SaveData.settings.school_look`)
校章	8	+20	自校(`SchoolState.emblem_id`)
髪型	10	+10	**自校の選手の外見の変更**(`Runner.appearance_override`)
髪色	12	+8	同上
応援歌	2	+4	自校の応援歌(大会画面)
BGM	4	+6	画面ごとの BGM セット

- **選手の外見の生成は、本体の部品だけを使う**(`look_seed` から02 §4.2 のとおり)。パックの髪型・髪色は、**プレイヤーが自校の選手の外見を後から変えるとき**に選べる(`appearance_override`。乱数と生成には入れない。購入状態で他の選手が変わらないようにするため)。
- **所持していない見た目**がセーブに残っていた場合(返金・別のストア)は、既定の見た目で表示し、選択は保持する。再び所持すれば元に戻る。セーブの読み込みは失敗しない(原則5)。
- 他校の見た目・世界の選手の外見は、パックの影響を受けない。

9.3 初期の商品一覧

pack_id	種類	内容	状態
`pack.golden_plan`	`GoldenPlan`	黄金世代プラン(§5)	v1
`pack.cosmetic.kit`	`Cosmetic`	ユニフォーム・校章パック(柄 +12、校章 +20)	v1
`pack.cosmetic.hair`	`Cosmetic`	髪型・髪色パック(髪型 +10、髪色 +8)	v1

pack_id	種類	内容	状態
pack.cosmetic.sound	Cosmetic	応援歌・BGMパック(応援歌 +4、BGM +6)	v1
pack.scenario.*	Scenario	追加の開始校(§10。例: 古豪復活)	発売後の拡張

- 価格は U-05(本体の価格帯)と一緒に決める。**初期案**: 黄金世代プランは本体価格の 40〜60%、見た目パックは各 15〜25%、シナリオは各 20〜30%(要検討)。
- U-01 の既定: v1 は『プラン + 見た目パック』。シナリオは発売後に追加する(§10 で形式と検算を済ませてある)。
- 商品は `ContentPack`(02 §3.9)で定義し、`store_ids` に各ストアの商品IDを持つ。`contents` は `ContentRef { type, id }` のリスト(`type` = `Uniform` `Emblem` `HairStyle` `HairColor` `CheerSong` `Bgm` `Scenario`)。

---

10. 開始状態(シナリオ)

10.1 形式(`ScenarioDef`)

シナリオは開始状態だけを定める。世界の規則・他校・乱数の使い方は変えない(世界は36校で共通)。本体の4つの難易度も、同じ形式のデータで持つ。

項目	型	説明	検証の範囲(SC-1〜)
id	string	例: `sc.low` / `sc.koki`	一意
name / description	string	名称テーブル	—
strength_rank	int	開始校を選ぶ**戦力順位**(先発10人の力の平均の順位)	1〜36
prestige / fame_hist	int / double?	名声と名声ポイントの履歴の上書き(省略なら、その学校のまま)	0〜1000 / 0〜40
facilities / staff	map<id, int>	施設・スタッフのレベル	各 1〜5
funds	int	資金(万円)	0〜5000
members	int?	部員数(省略なら、その学校の名簿のまま。多いときは無作為に減らす)	5〜40
ob_level	int?	各地方のOBの人数(県別OBネットワークの初期値)	0〜25
difficulty_tag	enum?	実績の分類。本体の4つだけ `Elite` `Mid` `Low` `Club`。シナリオパックは null	—

- 本体の難易度(05 §5): `sc.elite`(戦力3位。施設 コーチ4・トレーナー3・栄養士3・寮4・トラック4・坂道3・トレーニング室4・医療室3・スカウト3)/ `sc.mid`(7位。すべて2)/ `sc.low`(20位。すべて1)/ `sc.club`(33位。すべて1。部員数は §10.3)。資金はすべて 1500 万円。
- `SaveData.scenario = { id, difficulty?, player_school_id }`(02 §4.9)。**シナリオパックの開始校のクリアは、パックごとの専用の実績**で扱い、本体の難易度別の実績(強豪クリア〜同好会クリア。01 §3.3)には数えない。
- シナリオを選んで始めたセーブは、パックを所持していなくなっても続けられる(原則5)。新しく始めるときだけ、未所持のシナリオは選べない。

10.2 シナリオ例: 古豪復活(`sc.koki`)

戦力順位20位(弱小と同じ戦力)だが、**名声は強豪並み(700)、OBが各地方に多い(OBネットワークLv3)、施設は中堅(Lv2)。**名声とOBで入学者を集めやすいが、他校の包囲(名声400以上で強まる。05 §8.1)も早く受ける。

10.3 同好会の『部員5人』(課題 L-09)

01 §3.3 は同好会を『部員5人、設備なし』としていた。部員5人では、第一戦の出走人数(6人)にも足りず、最初の年度は大会に出られない(04 §4.1 E1)。**部員5人で始める場合と、通常の名簿(戦力順位33位の学校のまま)で始める場合を比べた結果は §12.4。**

---

11. 実績・共有との関係

- **無招聘**:『特別招聘券を一度も使っていない』は `run.invites_declared = 0`（宣言した時点で加算。取り消しなし。02 §3.10）で判定する。**購入状態は判定に使わない**。券を使わずプランを持っているだけなら『無招聘』のまま。
- **OB・レジェンド（01 §0 / 00 M-01・M-02）**: `OBRecord` に `recruit_source` を持たせる（招聘選手の印）。将来のレジェンドオーダーや、友人との対戦（ランキングなし）で、『招聘枠の選手は2人まで』のような**対戦ルール**に使える（課金の有無が勝敗に響くことを避けるため）。
- **イベント（06）**: 招聘選手は通常のイベントの当事者になる。専用のイベント・物語は付けない。

---

12. 検算結果

12.1 招聘選手の生成（`ref_model_07.py gen`。各20000人）

項目	留学生（Intl）	スーパールーキー（Rookie）
検証違反（S-1〜S-5）	0件	0件
上限で全能力を下げたもの	15%（平均 1.5 下げる）	0%
能力の平均（範囲）	65.7（59.8〜68.0）	60.9（54.2〜67.8）
最も高い能力の平均（最大）	73.3（80）	68.5（80）
`Q = A0 + 0.6×gap`（5% / 平均 / 95%）	67.8 / 70.8 / 73.8	71.9 / 76.0 / 80.0
素質の平均（最大）	73.7（99）	85.8（100）
特殊能力あり	30%	30%
型の比率（バランス / 持久 / スピード / 山）	35 / 25 / 25 / 15%	同左

→ 生成は検証ルールをすべて満たす。留学生は上限（平均68）に触れやすい（15% で調整が入る）。通常の候補（平均 `Q` 52、上位級で65）に対して、招聘選手は留学生が上位3%程度、スーパールーキーがほぼ最上位。

12.2 獲得の一覧とプランの影響（中堅・弱小、各40回×45年度。方針は『おまかせ』）

開始校	券の入手	頭打ち後の優勝率	クリア年度の中央値 / 平均	20年度までにクリア	45年度までに未達
中堅	券なし	0.47	15 / 15.7	78%	0
	獲得のみ（05 時点の6節目・失効あり）	0.48	12 / 12.8	95%	0
	獲得のみ（§4 の14節目・繰り延べ）	**0.51**	**12 / 12.2**	97%	0
	有料プランあり（4年度に2枚）	0.53	11 / 12.6	93%	0
弱小	券なし	0.39	27 / 28.6	17%	4
	獲得のみ（05 時点の6節目・失効あり）	0.41	26 / 26.9	28%	4
	獲得のみ（§4 の14節目・繰り延べ）	**0.46**	**24 / 25.6**	30%	1
	有料プランあり（4年度に2枚）	0.56	20 / 20.7	55%	1

- **ゲーム内の獲得だけで**、券なしより中堅は 15.7 → 12.2 年度、弱小は 28.6 → 25.6 年度に縮む。
- **有料プラン**の追加の効果は、弱小で大きい（獲得のみ 25.6 → 20.7 年度、**−19%**）。中堅は、獲得だけで十分に早く（12年度）クリアするため、プランの追加の効果はほとんど出ない（12.2 → 12.6、誤差の範囲。頭打ち後の優勝率は +0.02）。**プランの価値は、弱小・同好会のような難しい開始で大きい**。
- 方針（01 §6.5）: この差は公平性の制約にしない。券を使わない挑戦は『無招聘』の実績で評価する。
- 1回の標準誤差は、優勝率で ±0.03、クリア年度の平均で ±1 年度程度。

12.3 獲得の時期と使用（40回×45年度。『おまかせ』）

節目を初めて満たす年度の**中央値**:

節目	中堅	弱小
初のシード(`tm.seed`)	1	7
初の表彰台(`tm.podium`)	2	15
監督ランク 3 / 6 / 9	3 / 10 / 17	12 / 20 / 27
第一戦 / 第二戦 / 第三戦の初優勝	10 / 10 / 9	22 / 21 / 20
初の三冠 / クリア	17 / 12.5	27 / 23

項目	中堅・獲得のみ	中堅・プランあり	弱小・獲得のみ	弱小・プランあり
45年度までに使った券	14.0	23.0	13.4	23.0
うち20年度まで	9.2	10.0	5.1	10.0
最初に使う年度(中央値)	1	1	**7**	**1**
45年度の終わりの繰り延べ(`bank`)	0	14	0	14

- **序盤に入る券は少ない**: 中堅でも 初のシード・表彰台・ランク3 の3枚(年度1〜3)。弱小は **7年度まで最初の券が入らない**。プランは、この序盤の空白を埋める(最初の使用が 7 → 1 年度)。
- プランありでは、使用の上限(年0.5回)で頭打ちになり、**45年度の終わりに 14枚が使われずに残る**(付与と獲得の合計が使用の上限を上回るため。失効はない)。上限を動かせば、同じプランで使える枚数を増やせる(§2)。
- ゲーム内の獲得だけでも、45年度で 13〜14枚が使われる。**券を使わずにクリアできる**(原則2)ことは、『券なし』の行(§12.2。中堅 15.7 年度、弱小 28.6 年度)で確認できる。

12.4 開始状態の検算(40回×45年度。券なし。方針は『おまかせ』)

同好会の『部員5人』(課題 L-09)

開始	12年度の戦力順位	頭打ち後の優勝率	45年度までにクリア
通常の名簿(戦力順位33位の学校のまま)	16.3	0.09	10/40(25%)
部員5人	16.4	0.07	9/40(23%)
部員12人	16.6	0.07	9/40(23%)

→ **部員の数による差はない**(誤差の範囲)。最初の年度は大会に出られない(第一戦の出走は6人。04 §4.1 E1)が、最下位の学校は出ても評価ポイントが 0 に近く、翌年度に10人が入学してからは同じ推移になる。**『部員5人』のまま確定する**。券なしの同好会は 45年度までに 25% しかクリアしない(ゲーム内の獲得で 50% になる。05 §14.3)。

シナリオ例: 古豪復活(戦力順位20位、名声700、施設Lv2、各地方のOB 9人、資金1500万円)

開始	頭打ち後の優勝率	クリア年度の中央値 / 平均	20年度までにクリア	45年度までに未達
弱小(戦力順位20位。名声は弱小のまま)	0.39	27 / 28.6	17%	4
古豪復活(同じ戦力で、名声・OB・施設が上)	**0.48**	**14 / 17.5**	78%	2
中堅(参考)	0.47	15 / 15.7	78%	0

→ 古豪復活は、**中堅並みの難易度**(`Normal`)。名声とOBで入学者を集めやすい一方、戦力が低い序盤は苦しい。難しくしたいときは `prestige` を下げる(`ScenarioDef` の値だけで調整できる)。

---

13. 計算例(テストベクタ。`verify/ref_model_07.py vectors`)

TV-1 招聘選手の生成(PCG32。`master_seed = 20260101`、入学年度 6、`index = 0`)

乱数は `derive(20260101, 3, 6, 9000)`。

種別	A0	gap	成長 / 型	能力(SPD STA PAC KCK CLB DWN CON MEN REC DUR)	素質	出身	Q / V
スーパールーキー(県13を指定)	61.661	27.206	晩成 / バランス	57 58 62 54 64 63 63 56 64 62	84 85 89 81 89 92 94 83 90 92	13県	77.98 / 69.82
留学生	66.661	9.471	晩成 / バランス	62 63 67 59 69 68 68 61 69 67	72 72 77 68 76 79 81 71 78 79	アメリカ	72.34 / 69.50

どちらも検証(S-1〜S-5)に適合。特殊能力なし(`u = 0.5464`、確率 30% に届かない)。

上限の調整の例(入学年度 5・留学生): `A0 = 68.0`、スピード型。調整前の能力の平均 69.8 > 68 なので、全能力を **2** 下げて `76 65 57 71 66 67 63 68 71 74`(平均 67.8)。

TV-2 券の状態の推移(有料プランあり、年度1から。毎年、使えるなら使う)

獲得: 年3に初優勝、年5に初の表彰台、年8に初のシード。招聘選手は4年間在籍する。

年度	付与	獲得	使用	所持	繰り延べ	在籍(招聘)
1	2	0	宣言	1	0	1
2	0	0	宣言	0	0	2
3	0	1	—	1	0	2
4	0	0	—	1	0	2
5	2	1	—	2	**2**	2
6	0	0	宣言	2	1	2
7	0	0	宣言	2	0	2
8	0	1	—	2	1	2
9	2	0	—	2	**3**	2
10	0	0	—	2	3	2
11	0	0	宣言	2	2	2
12	0	0	宣言	2	1	2

→ 12年度までに使った券は6枚(使用は4年度に2回が上限)。**失効は0枚**。年度5・9は、入手が使用を上回り、繰り延べ(`bank`)にたまる。

TV-3 購入状態の判定(07 §8.2)と付与

状況	状態	付与
オンラインで所有	`True`	2枚
オフライン・10日前に確認	`True`	2枚
オフライン・40日前に確認(猶予30日を超える)	`None`	0枚
オフライン・確認なし	`None`	0枚
返金(`not_owned`)	`False`	0枚

- 4月にオフライン → 同じ年度のうちに確認できたら付与(0 → 2)。同じ年度に2回確認しても付与は1回(二重に付与しない)。
- 起点が年度1で、年度2はオフライン(周期の年ではない)、年度5に確認 → 付与 2枚(周期の年度 1, 5, 9…)。
- 年度5にずっと未確認のまま年度6になると、年度5ぶんは付与しない(取り戻しは同じ年度のうちだけ)。

TV-4 検証(S-1〜S-5)

生成20000人×2種別で違反は0件(§12.1)。

14. 課題・未決・他文書への反映

14.1 課題トラッカー（`00_spec_index.md` §6）

ID	状態	内容
L-06	**解決(方針)**	有料プランの影響(中堅はほぼなし、弱小 −19%。§12.2)は公平性の制約にしない。『無招聘』の実績で分ける
L-09	**解決**	同好会の『部員5人』開始は、通常の名簿で始める場合と変わらない(§12.4)。本体の定義を『部員5人』のままとする
L-20	新規	留学生の国別の名前テーブルと、24か国の一覧の見直し(人物描写への配慮の観点)は未作成・未確定
L-21	新規	価格(U-05)・各ストアの商品登録・年齢レーティングの申請は、発売前に行う(§8.5)
L-22	新規	購入状態の確認は、ストアのAPIの最新の仕様で実装時に確認する(§8.1 の API 名は設計時の想定)

14.2 未決の解決（`00_spec_index.md` §5）

ID	決定
U-01	v1 は『黄金世代プラン + 見た目パック3種』。シナリオパックは発売後に追加(形式と検算は §10)
U-10	ゲーム内の獲得は §4 の14個(45年度)。継続の記念で、長く遊ぶほど増える
U-12	**4年度に2枚、失効なし(繰り延べ)**。使用は年1回・同時在籍2名・所持2枚(§2)
U-05	未決(本体と一緒に M3 で決める)。商品の価格の目安は §9.3

14.3 反映した文書

文書	変更
02	v0.8: `tickets`（`bank` `plan_granted_years` `milestones`）、`ScenarioDef`（§3.11）、`ProfileData.entitlements`、`Runner.appearance_override`、`OBRecord.recruit_source`、`SaveData.scenario.id`、`settings.school_look`、国は24か国、I-14・I-15 の更新
01	§6.5 を確定した規則に更新(失効なし・初回の付与・見た目とシナリオの扱い・U-12 の確定)
05	§11 を 07 に更新、§14 を新しい獲得の一覧(14個・繰り延べ)で再検算
04	§14.4 ③ の長期の表を再検算した値に更新
00	07 を v0.1 に、U-01・U-10・U-12・L-06・L-09 を更新、L-20〜L-22 を追加
08	券・購入状態・見た目・シナリオの画面(SC-31, SC-05, SC-02, SC-04)
09	購入状態のキャッシュを `ProfileData` から `LocalState` へ移す(§8.1)/ 時計対策(§8.2 の注)/ 引き継ぎの完了(§14.4)
10	§6.5 に個性・素養の付与(通常の入学生と同じ。型は ④ の型から)

14.4 次の文書への引き継ぎ

引き継ぎ先	内容
08(完了)	宣言の画面は SC-31(種別の選択・県の指定・『範囲』の表示・取り消せない確認。『無招聘』の実績の注意を確認に明記)、追加コンテンツは SC-05。以下は引き継ぎの内容: 宣言の画面(種別の選択・県の指定・『範囲』の表示・取り消せない確認)、券の所持・繰り延べ・次の付与の表示、購入・復元の画面、シナリオの選択(未所持は選べない)、見た目の変更の画面、購入状態を確認できないときの表示
09(完了)	購入状態のキャッシュは `LocalState.entitlements`（09 §3.3）/ 時計対策(09 §10)/ ストアごとの `IEntitlementProvider` の形(09 §15.4)/ `tickets` のセーブ形式(刈り込み: `tickets.log` 50 件。09 §5.2)/ 乱数のキー（`label = 3, b = 9000 + index`。09 §8.2 の登録表）/ `FakeEntitlementProvider` を使った自動テスト(09 TV-10 を写す)

14.5 M0 の C# 実装(段階 6〜7 のあと)

- 場所: `m0/Assets/Scripts/Domain/Tickets/`（`TicketState` 状態遷移・`TicketMilestones` §4 の節目の判定）・`Recruit/SpecialRecruit.cs`（§6 の生成。個性・素養は `Individuality.GenerateStyle` / `GenerateGifts` で通常の入学生と同じ関数）・`World/Scenario.cs`（§10 の `ScenarioDef` と本体の 4 難易度・古豪復活）、`m0/Assets/Scripts/Persistence/Store/Entitlement.cs`（§8.2 と 09 §10 の判定）。

- 参照モデルとの差分テスト（`m0/tests/Domain.Tests/SpecialGoldenTests.cs`・`Persistence.Tests/EntitlementTests.cs`）: 招聘選手 3,000 人・券の操作列 200 本・購入状態 4,000 件が完全一致。節目（§4）は参照モデルに実装がなく（世界シミュレーションの近似だけ）、文書の表から実装して 45 年度 14 枚を確認した。
- 節目の入力: `EarnAfterRace(kind, rank, yearWins)`（大会の直後）・`EarnAtYearEnd(managerRank, yearsCompleted)`（年度末の精算後）。監督ランクと年度の勝利の集合は、呼ぶ側（段階 10 の統合）が持つ。

第8部 画面仕様 — 本編

原典: `08_screen_spec.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> 範囲: **画面(表示項目・操作・遷移)**、入力モデル、レイアウト規則、進行制御(どこで止まるか)、共通部品、そして**画面が表示・提案する値の規則**(月末の予測・推奨プラン・個別指導・スカウトの見込み・前評判など)。前提: 01〜07。画面は仕様の状態を**表示して操作する窓**で、規則(乱数・計算)は Domain にある(01 §12)。画面ごとの詳細は **`08a_screen_catalog.md`** (本書は規則・流れ・検算)。表示項目と Domain の値・API の対応は **`08b_screen_data.md`** 。**検算**: 参照モデル `verify/ref_model_08.py` + カタログ `verify/screens_08.py` 。①画面カタログの整合(到達性・コマンドと情報の網羅・パッドの移動)②月末の予測(03 の週の解決と一致)③推奨プランと重点ごとの違い④割り込みの回数⑤1年度の操作時間(30分の予算)⑥レイアウトの収まり⑦前評判・スカウトの競合度と枠の較正⑧仕様書とカタログの突き合わせを検算した。**本書で新しく決めた規則**(02〜07 が未定義だったもの)は §8 の R-01〜R-17。他文書への反映は §13。

1. 原則

#	原則
1	**週ごとの入力を求めない**。操作は「節目」(年間方針・月間計画・エントリー・オーダー・当日・年度末)だけ。それ以外は『続ける』で進む(01 §3.4)。
2	**すべての節目に、推奨の初期値がある**(推奨プラン・おまかせ登録・おまかせ編成・おまかせスカウト・おまかせの采配)。何も変えずに1タップで確定できる。
3	**判断の材料を同じ画面に出す**。表示する予測・適性・余裕・前評判・見込みは、仕様の式から計算し、**実測と合うことを検算した値**にする(◎が出たら、実際にそうなる)。
4	**取り消せない操作は確認する**: 特別招聘の宣言・購入・セーブの上書きと削除・施設の強化。それ以外は、確定するまで何度でも変えられる。
5	**選択 → 決定**。タッチ・マウス・ゲームパッドで同じ操作が成り立つ。**ホバーに依存しない**。詳細は長押し / ボタンで出す(01 §11)。
6	**色だけで区別しない**。◎○△・調子・故障は、記号・文字・形を併用する。文字の大きさを変えられる。
7	**待たせない**。ネットワークに依存しない。購入状態の確認は裏で行い、結果が来るまで『確認中』のまま進む(07 §8)。
8	**いつでも中断できる**。どの画面でもポーズでき、月境界・大会前・中断時に自動でセーブする(01 §11)。
9	**文言は名称テーブルから引く**(コードに直書きしない。U-04)。数値・時間・単位の整形は共通部品が行う(§7)。
10	**画面は規則を持たない**。可否の判断・計算は Domain の関数を呼び、画面は結果を表示する。本書の規則(R-xx)も Domain に置く。

2. 定数表(初期値案。`K_UI_` 接頭辞)

分類	定数	値	意味
基準キャンバス	`K_UI_REF_H`	360 u	基準の高さ。1 u = 高さ360dpの機種での1dp。実機では高さに合わせて拡大する(PC の 1080p は 1 u = 3 px)
	`K_UI_REF_W`	640 / 720 / 800 u	幅(16:9 / 18:9 / 20:9)。これより広い画面は余白を足す
	`K_UI_SAFE`	24 u	左右の安全域(切り欠き・角丸の機種)。PC は 0
	`K_UI_HEADER` / `K_UI_FOOTER`	40 u / 56 u	ヘッダ / 操作バー(フッタ)の高さ。内容の高さ = 360 − 40 − 56 = **264 u**
	`K_UI_MARGIN` / `K_UI_GAP`	12 u / 6 u	余白 / 要素の間隔
操作	`K_UI_TOUCH_MIN`	48 u	押せるものの最小の大きさ(幅・高さ)
	`K_UI_ROW` / `K_UI_ROW_COMPACT`	48 u / 24 u	操作できる行 / 読むだけの行の高さ
	`K_UI_LONGPRESS_MS`	450 ms	長押しの時間(= 詳細・ヘルプ)

分類	定数	値	意味
文字	`K_UI_TEXT_BODY` / `K_UI_TEXT_MIN` / `K_UI_TEXT_TITLE`	14 u / 12 u / 20 u	本文 / 最小(注記)/ 見出し
	`K_UI_TEXT_SCALE`	100 / 125 / 150 %	設定で選べる文字の倍率(DeviceSettings。§9)
進行	`K_UI_WEEK_ANIM_S`	1.2 秒	1週の経過の演出(スキップ可)
	`K_UI_SEC_ANIM_S` / `K_UI_DIGEST_SEC_S`	6.0 秒 / 2.0 秒	1区間の演出(フル / ダイジェスト)。速度 ×1 / ×2 / スキップ
	`K_UI_STOP_CAP_WEEK`	3	1週に止まる回数の上限(超えた分は月末レポートへ。§6.3)
	`K_UI_REPORT_MAX` / `K_UI_REPORT_PER_KIND`	7 / 3	月末レポートに出す項目の数(全体 / 種別ごと)
	`K_UI_WARN_F` / `K_UI_WARN_F_CAMP`	70 / 80	コンディション警告の疲労の閾値(夏合宿の月は 80)
育成	`K_UI_GROUP_A` / `K_UI_GROUP_B`	10 / 10	推奨のグループ編成の人数(A=力の上位10、B=次の10、C=残り)
	`K_UI_GUIDE_COST` / `K_UI_GUIDE_POINTS`	1 / 4	個別指導の費用(1人1回)/ 月の指導ポイント(繰り越さない。02 §4.4)
	`K_UI_HINT_LATE` / `K_UI_HINT_CAP`	0.85 / 0.95	素質のヒントの境界(能力 ÷ 素質)
スカウト	`K_UI_SCOUT_SURVEY_M`	6〜11月(`month_index` 2〜7)	視察ができる期間
	`K_UI_SCOUT_PURSUE_M`	12〜2月(`month_index` 8〜10)	声かけ・練習会ができる期間
	`K_UI_SCOUT_TARGET_REC`	8 人	推奨の目標人数(定員10 − 自然に入る約3人 + 余裕1。R-08)
	`K_UI_SCOUT_NATURAL`	3 人	狙わなくても入る人数の目安(実測 3.3。§11.7)
	競合度	V < 51.5: ほぼなし / < 55: 少 / < 59: 中 / それ以上: 多	市場評価 V の帯。枠があれば競り勝つ割合の実測は 約100% / 約100% / 約92% / 約65%(R-08)
大会	前評判の印	◎ 1位 / ○ 2〜3位 / ▲ 4〜6位 / △ 7〜10位	前評判の順位(R-09)
ヒント	`K_UI_HINT_MAX_PER_SCREEN`	1	初めて開いた画面に出すヒントのカードの数(§6.6)
セーブ	`K_UI_SAVE_SLOTS`	3	スロットの数。各スロットに自動セーブ2世代 + 手動1(09 §3・§4)
年度の予算(01 §3.4)	年度初め 180 / 月間計画 480 / スカウト 240 / 大会 720 / イベント 180 / **年度末 120(本書で追加)**秒		合計 1,920 秒 = 32 分(機械的な時間は 21 分。§11.3)

3. 入力モデル

3.1 操作の種類(3つの入力で共通)

操作	タッチ	マウス	ゲームパッド	キーボード
移動(フォーカス)	—	—	十字キー / 左スティック	矢印キー
選択	1タップ	左クリック	A(フォーカス中の項目)	Enter / Space
戻る	左端からのスワイプ / 戻るボタン	右クリック / 戻るボタン	B	Esc

操作	タッチ	マウス	ゲームパッド	キーボード
詳細・ヘルプ(その項目の説明)	長押し(450 ms)	右クリック(長押し相当)	X	F1
タブの切り替え	タブをタップ / 横スワイプ	タブをクリック	LB / RB	Q / E
ページ送り・大きなスクロール	縦スワイプ	ホイール	LT / RT	PageUp / PageDown
『おまかせ』(その画面の推奨を適用)	ボタン	ボタン	Y	A キー相当(ボタンの文字に表示)
ポーズ	画面端のメニュー	メニュー / Esc	Start	Esc

- **ホバーに依存しない**。ツールチップは作らない。説明は長押し / X / F1 で出す。
- **選択 → 決定**: リストの行は1タップで『開く』(副作用なし)。**状態を変える操作はボタン**で行い、確定の前なら何度でも変えられる。取り消せない操作は確認ダイアログ(§7.6)を通す。
- 十字キーは**領域(ヘッダ・一覧・詳細・操作バー)の間を移動し、領域の中では項目を移動する**。画面ごとの領域の並び(`grid`)は 08a に書き、**すべての領域に十字キーだけで届く**ことを検算した(§11.1)。
- 操作バーのボタンは、左から『戻る系』『補助』『主たる操作(右端、強調)』の順に置く。パッドでは主たる操作に A、補助に Y を割り当てる。

3.2 フォーカスと確認の規則

ID	規則
F1	画面を開いたとき、フォーカスは `default_focus` の領域の先頭(08a)。ダイアログは既定のボタン。
F2	取り消せない確認ダイアログの既定のフォーカスは『戻る』。『実行』は右にあり、色と文言で区別する(誤操作の防止)。
F3	戻ったとき、直前のフォーカス位置と一覧のスクロール位置を復元する。
F4	無効な操作(条件を満たさない)のボタンは**消さずに無効表示**にし、フォーカスは当たる。選ぶと**理由**を1行で出す(例:『今年度の宣言は済んでいます』)。
F5	長い一覧は先頭・末尾で止まる(循環しない)。LT / RT で1画面ぶん動く。

---

4. 画面の基準

4.1 キャンバスと領域

- **横画面のみ**(縦持ちは将来。01 §14.1)。基準の高さ 360 u(§2)。PC・タブレットは拡大して同じレイアウトを使い、広い画面は左右に余白を足す(中身を広げて詰めない)。
- 画面は **ヘッダ(40 u)/ 内容(264 u)/ 操作バー(56 u)** の3段。ヘッダに画面名・戻る・資金や日付などの要約、操作バーに主たる操作を置く。
- 内容は最大 **3つの領域**(操作バーを除く)に分ける(左右2列 + 上の帯など)。領域ごとにスクロールできる。1つの画面に操作の種類を詰め込まない。

4.2 文字・色・記号

項目	規則
文字	本文 14 u、最小 12 u(これより小さくしない)。表の数字は 14 u、固定幅の数字。倍率 100 / 125 / 150% は**本文の大きさだけ**を変え、列の幅が足りないときは折り返さずに省略(…)して長押しで全文を出す。
色	意味のある色は **記号と必ず併用**する。疲労ゲージ: 0〜39 緑 / 40〜69 黄 / 70〜84 橙 / 85〜 赤(数字も出す)。調子: ↑↑ ↑ → ↓ ↓↓。やる気: 5段階の顔。故障: 軽 / 中 / 重 の文字。
◎○△	**意味ごとに直前に語を付ける**:『適性◎』(区間適性。P ≥70 / ≥60 / それ未満)、『余裕○』(余裕の目安。q < 0.16 / < 0.26 / それ以上)。前評判は ◎○▲△(順位の印)、競合度は語(ほぼなし / 少 / 中 / 多)。記号だけを単独で出さない。
色覚の設定	『色の見え方』(標準 / 色の区別を強める)。後者では、色の代わりに塗りのパターン(斜線・点)を足す。
動き	『動きを減らす』で、画面の揺れ・フラッシュ・カットインの動きをなくす(演出は静止画に置き換える。結果は変わらない)。

項目	規則
言語	v1 は日本語のみ(U-04)。文字列はすべて名称テーブルのキー。かな(ふりがな)の表示は設定で切り替える(姓名のかなは 02 §4.1 の `family_kana` / `given_kana`)。

4.3 画面の大きさに対する収まり(検算済み。§11.4)

項目	基準(W=640 / 720 / 800 u)で収まる条件
区間オーダーの区間カード	10区間を横に並べて、1枚が 48 u 以上(W=640 で 51.4 u)
年間予定	半年(6か月)×2段(1か月 94.7 u 以上)
部員一覧の列	名前 96 + 学年 28 + グループ 28 + 力 36 + 疲労 36 + 調子 28 + やる気 24 + 故障 24 + 特殊能力 36 + 出身 72 = 408 u(右の詳細 160 u を除いた幅に収まる)
月間計画のグループ行	3グループ × 56 u = 168 u(内容 264 u に収まる)
スカウトの県のタイル	地方で絞ってから県を選ぶ。1地方の最大は中部の9県 → 3×3 のタイル(1つ 64 u)= 204 u
レースの順位表	読むだけの行 24 u で 7行。**先頭3校 + 自校の前後1校 + 区切り**の7行を出す
エントリー / オーダーの一覧	操作できる行 48 u で、概要 40 u を除いて4行。**スクロールが前提**で、先頭に『登録枠 n/16』『出走 n/10』の要約を固定する

- 日本地図そのものは、**47県を直接タップさせない**(最小の県は 12 u 程度で 48 u に満たない)。地方(8)→ 県のタイルの2段で選ぶ(01 §6.1 の『地方 → 県の順に絞る』)。地図は地方・県の位置を見せる飾りと、候補数・OBレベルの塗り分けに使う。

---

5. 画面の一覧と遷移

画面は42(通常 22 / 進行 12 / 重なり 8)。『進行』は、進行制御(§6)が節目で開く画面(ホームからも開ける)、『重なり』は画面の上に重ねるダイアログ。画面ごとの詳細は `08a_screen_catalog.md`。一覧は `verify/screens_08.py` から出力した(`python3 -I ref_model_08.py tables`)。

SC	画面ID	画面	区分	分類	入口	出口	戻る
SC-01	`scr.title`	タイトル	通常	起動・設定	起動	SC-03・SC-02・SC-04・SC-05・SC-06・SC-07	—
SC-02	`scr.newgame`	新規ゲーム	通常	起動・設定	SC-01	SC-08・SC-01・SC-37	SC-01
SC-03	`scr.load`	セーブの選択	通常	起動・設定	SC-01・SC-40	SC-08・SC-01・SC-37	SC-01
SC-04	`scr.settings`	設定	通常	起動・設定	SC-01・SC-40・SC-08	SC-05・SC-37	SC-01
SC-05	`scr.store`	追加コンテンツ	通常	起動・設定	SC-01・SC-04・SC-08・SC-02	SC-37	SC-01
SC-06	`scr.achv`	実績	通常	起動・設定	SC-01・SC-40		SC-01
SC-07	`scr.credits`	クレジット・権利表示	通常	起動・設定	SC-01・SC-04		SC-01
SC-08	`scr.home`	ホーム	通常	ホームと進行	SC-02・SC-03・進行・SC-40	SC-16・SC-18・SC-27・SC-32・SC-34・SC-35・SC-09・SC-31・SC-21・SC-22・SC-10・SC-38・SC-40・進行	SC-40
SC-09	`scr.calendar`	年間予定	通常	ホームと進行	SC-08	SC-21・SC-22・SC-10	SC-08
SC-10	`scr.annual`	年間方針	進行	ホームと進行	進行・SC-08・SC-09	SC-16・SC-08	SC-08
SC-11	`scr.report.month`	月末レポート	進行	ホームと進行	進行	SC-08・SC-19・進行	SC-08
SC-12	`scr.year.summary`	年度の結果と精算	進行	ホームと進行	進行	SC-32・進行	—
SC-13	`scr.year.grad`	卒業・進路	進行	ホームと進行	進行	SC-19・進行	—

SC	画面ID	画面	区分	分類	入口	出口	戻る
SC-14	scr.enroll	入学者発表・入部確認	進行	ホームと進行	進行	SC-19・進行	—
SC-15	scr.clear	クリア	進行	ホームと進行	進行	SC-08・SC-34	—
SC-16	scr.plan	月間計画	通常	育成・部員	SC-08・SC-10・進行	SC-17・SC-18・SC-20・SC-08・SC-37	SC-08
SC-17	scr.plan.custom	カスタム週の編集	通常	育成・部員	SC-16	SC-16	SC-16
SC-18	scr.roster	部員一覧	通常	育成・部員	SC-08・SC-16	SC-19・SC-37	SC-08
SC-19	scr.runner	選手詳細	通常	育成・部員	SC-18・SC-33・SC-34・SC-11・SC-13・SC-14・SC-22・SC-21・SC-29・SC-24・SC-26	SC-20・SC-37	SC-18
SC-20	scr.guidance	個別指導	重なり	育成・部員	SC-16・SC-19・SC-41・SC-42	SC-19・SC-16	SC-19
SC-21	scr.entry	エントリー	進行	大会	進行・SC-08・SC-09・SC-42	SC-22・SC-19・SC-08	SC-08
SC-22	scr.order	区間オーダー	進行	大会	進行・SC-08・SC-21・SC-09	SC-23・SC-19・SC-08・SC-21	SC-08
SC-23	scr.preview	他校情報・予報	進行	大会	SC-22・進行・SC-08	SC-24・SC-22・SC-35	SC-22
SC-24	scr.raceday	当日(調子・環境・当日変更)	進行	大会	進行・SC-23	SC-25・SC-26・SC-19	—
SC-25	scr.live	レース進行	進行	大会	SC-24	SC-26・SC-40	—
SC-26	scr.result	大会の結果	進行	大会	SC-25・SC-24・進行	SC-34・進行・SC-19	—
SC-27	scr.scout.map	スカウト(地図)	通常	スカウト	SC-08	SC-28・SC-30・SC-31	SC-08
SC-28	scr.scout.list	候補リスト	通常	スカウト	SC-27・SC-30	SC-29	SC-27
SC-29	scr.scout.cand	候補の詳細	通常	スカウト	SC-28・SC-30	SC-19・SC-37	SC-28
SC-30	scr.scout.target	目標とポイント	通常	スカウト	SC-27・SC-28・SC-29	SC-29・SC-28	SC-27
SC-31	scr.ticket	特別招聘の宣言	通常	スカウト	SC-27・SC-08	SC-37	SC-27
SC-32	scr.ops	運営	通常	運営・歴史	SC-08・SC-12	SC-37	SC-08
SC-33	scr.ob	OB名簿	通常	運営・歴史	SC-08・SC-34	SC-19	SC-08
SC-34	scr.history	歴史・殿堂	通常	運営・歴史	SC-08・SC-26・SC-15	SC-19・SC-33	SC-08
SC-35	scr.world	全国の大学	通常	運営・歴史	SC-08・SC-23		SC-08
SC-36	dlg.event	イベント	重なり	ダイアログ・割り込み	進行	進行	—
SC-37	dlg.confirm	確認	重なり	ダイアログ・割り込み	どこからでも	どこからでも	—
SC-38	dlg.notify	通知	重なり	ダイアログ・割り込み	SC-08	SC-08	SC-08
SC-39	dlg.help	ヘルプ・用語	重なり	ダイアログ・割り込み	どこからでも	どこからでも	—
SC-40	dlg.pause	ポーズ	重なり	ダイアログ・割り込み	SC-08・SC-25・進行	SC-04・SC-03・SC-06・SC-01・SC-37	—
SC-41	dlg.warn	コンディション警告	重なり	ダイアログ・割り込み	進行	SC-20・SC-16・進行	—
SC-42	dlg.injury	故障の知らせ	重なり	ダイアログ・割り込み	進行	SC-21・SC-22・SC-20・SC-19・進行	—

- 入口・出口の『進行』は、進行制御(§6)が開く / へ進むこと。『どこからでも』は、すべての画面から開ける(確認・ヘルプ)。
- 戻る = 『戻る』ボタン / B / Esc / 右クリックで、表の『戻る』の画面へ行く。ダイアログは閉じて、元の画面に戻る。ポーズ(SC-40)はホーム・レース進行・進行の画面から開き、閉じると同じ場所に戻る。

5.1 遷移の全体像

```
起動 → SC-01 タイトル ─┬─ SC-03 セーブの選択 ─┐
                       ├─ SC-02 新規ゲーム ──────┼→ SC-08 ホーム(ハブ)
                       ├─ SC-04 設定 / SC-05 追加コンテンツ / SC-06 実績 / SC-07 クレジット
                       └─(続きから = 直近のセーブ)┘

SC-08 ホーム ─┬─ SC-16 月間計画 ― SC-17 カスタム週 / SC-20 個別指導
              ├─ SC-18 部員一覧 ― SC-19 選手詳細 ― SC-20 個別指導
              ├─ SC-27 スカウト(地図)― SC-28 候補リスト ― SC-29 候補の詳細 / SC-30 目標とポイント / SC-31 特別招聘
              ├─ SC-32 運営 / SC-33 OB名簿 / SC-34 歴史・殿堂 / SC-35 全国の大学 / SC-09 年間予定
              ├─ SC-21 エントリー ― SC-22 区間オーダー(エントリーの確定後から週 t の初めまで編集でき、提出は週 t の初め。ホームの『大会』のカードから開く)
              └─ 『続ける』→ 進行制御(§6)

進行制御が開く画面(節目):
  年度初め   SC-14 入部確認 → SC-10 年間方針 → SC-16 月間計画(主将の指名は週0の終わりのイベント SC-36)
  月         SC-16 月間計画 →(4週が進む)→ SC-11 月末レポート
  大会       SC-21 エントリー(4週前)→ SC-22 区間オーダー → SC-23 他校情報・予報 →(練習の解決)
             → SC-24 当日 → SC-25 レース進行 → SC-26 結果 →(大会後のイベント SC-36)
  割り込み   SC-36 イベント / SC-42 故障の知らせ / SC-41 コンディション警告
  年度末     SC-11(3月)→ SC-12 年度の結果と精算 → SC-32 運営(強化)→ SC-13 卒業・進路 → SC-14 入学者発表 → 次の年度
  クリア     SC-26 結果の直後に SC-15 クリア(『続ける』で元の流れへ)
```

5.2 遷移の規則

ID	規則
N1	**ホームがハブ**。育成・スカウト・運営・歴史の画面は、ホームから開き、戻るとホームに戻る。戻る操作の深さは、通常の画面で3回以内(検算済み)。
N2	『進行』の画面は、進行制御が開いたときは**最後の画面で『続ける』**を押すと次の節目へ自動で進む(ホームを経由しない)。途中で『ホームへ』を押すと、ホームに出る。ホームの『続ける』で、止まっていた節目から再開する。
N3	重なり(ダイアログ)は**1段まで**。ダイアログの上に開けるのは確認(SC-37)だけ(検算済み)。
N4	進行の画面では、大会・年度末のように**後戻りできない処理**の前に、自動セーブを行う(§6.5)。
N5	年度末のフロー(SC-12 → SC-32 → SC-13 → SC-14)は**この順で固定**(05 §12 の手順3〜7に対応)。強化(SC-32)は手順3の精算のあと、入学の割り当て(手順7)の前に行う(寮のレベルが同じ年度の奨学金枠に効くため)。

---

6. 進行制御(FlowController)

6.1 年度の節目(止まる場所)

週は年度内の番号 0〜47(0 = 4月1週)。大会週は 第一戦 25(10月2週)/ 予選会 26(10月3週)/ 第二戦 28(11月1週)/ 第三戦 36(1月1週)。エントリー締切は大会の4週前(02 §3.5)。

いつ	止まる場所	画面	既定の操作	スキップ
週0(4月1週)の初め	年度初め: 入部確認 → 年間方針 → グループ編成の確認(そのあと月間計画)。**主将の指名は、週0の終わりのイベント**(06 `team.captain`。4月の第1・2週に発生。SC-36)	SC-14 → SC-10 → SC-18	推奨(グループは推奨の編成、方針は前年度のまま)	年間方針・グループは前年度のままなら1タップ。`plan_auto` でも止まる
週 `w % 4 = 0`(月の初め)	**月間計画**	SC-16	推奨プランを適用	`plan_auto = true` なら止まらない

いつ	止まる場所	画面	既定の操作	スキップ
週21 / 24 / 32(大会の4週前)	**エントリー**(第一戦 / 第二戦 / 第三戦)。出場権のある大会のみ	SC-21	おまかせ登録	しない(登録しないと棄権 = 04 §4.1 E1。必ず止まる)
週22	**予選会のエントリー**(シード外の年度のみ。12名)	SC-21	おまかせ登録	しない
週25 / 28 / 36(大会週の初め)	**区間オーダー** → 他校情報・予報	SC-22 → SC-23	おまかせ編成	しない
大会週の終わり(練習の解決のあと)	**当日**: 調子と環境の確認 → 当日変更 → レース進行 → 結果	SC-24 → SC-25 → SC-26	観戦モードに従う(『結果のみ』は SC-25 を飛ばす)	観戦モードで省略
週26	予選会の当日と結果(シード外の年度のみ)	SC-26	同上	結果のみ可
毎週の終わり	割り込み(§6.3)	SC-36 / SC-42 / SC-41	イベントは既定の選択肢	`event_auto` で選択のあるイベントも止まらない
週 `w % 4 = 3`(月の終わり)	**月末レポート**	SC-11	確認	しない(1タップ。長押しで詳細)
週47(3月4週)の終わり	**年度末**: 結果と精算 → 強化 → 卒業・進路 → 入学者発表	SC-12 → SC-32 → SC-13 → SC-14	強化は推奨(予備費500万円を残して、効果の大きい順)	しない
クリアしたとき	クリアの演出	SC-15	続ける	しない(初回のみ。2回目以降は通知だけ)

- スカウト・特別招聘の宣言・運営の強化は**止まらない**(プレイヤーがホームから開く)。ただし期限があるものは**ホームの『次の予定』と通知**で知らせる(宣言期間・スカウトの時期。§6.3)。
- 週の経過の演出は `K_UI_WEEK_ANIM_S`(1.2秒 × 4週)。『動きを減らす』のときは演出なしで進む。

6.2 進行のループ

```
loop:
    for stop in stops_before_week(week, qualifies, seed_school, plan_auto):   # §6.3 の表
        show(stop)                                                            # 確定するまで進まない
    resolve_training()                  # 03 §6 の週の解決(練習 → 疲労 → 故障 → 調子の確定)
    if race_week:
        run_race_flow()                 # 当日(SC-24)→ レース進行(SC-25)→ 結果(SC-26)。04 の計算と大会後処理を含む
    resolve_after()                     # 03 §6 の絆の更新 → 06 の EventPhase(大会週は大会後処理のあと)
    stops, deferred = stops_after_week(week, events, injuries, warn, race_kind, event_auto)
    for stop in stops: show(stop)       # RaceDay は run_race_flow で済んでいる。1週に最大 K_UI_STOP_CAP_WEEK 件。超えたぶんと『お知らせ』は deferred へ
    if week % 4 == 3: show(MonthReport(deferred を含む))                      # 月末レポートに集約
    if week == 47: show(YearEnd)
    week += 1
    autosave_if_needed()                # §6.5
```

- **『続ける』(C-FLOW-01)は唯一の前進の操作**。ホームの『続ける』は、次の節目の名前を表示する(『10月の計画へ』『第一戦のエントリーへ』『今月を進める』)。進行の画面の主たる操作は、同じ『続ける』(文言は画面ごと)。
- 大会週は、初めにオーダーを出してから週の練習を解決し、そのあとで当日になる(03 §8 の大会週の順序と 04 §1)。

6.3 停止の決定表(`stops_before_week` / `stops_after_week`)

週の初め(上から順に。同じ週に複数あるときはこの順で止まる)

停止	条件	備考
AnnualStart	`week = 0`	年度初め(§6.1)
MonthPlan	`week % 4 = 0` かつ `plan_auto = false`	年度初めの週は AnnualStart のあと
Entry:{大会}	出場権のある大会の `t - 4`	第一戦 21 / 第二戦 24 / 第三戦 32

停止	条件	備考
Entry:Qualifier	シード外の年度の週22	予選会の12名
Order:{大会}	出場権のある大会の週 `t`	第一戦 25 / 第二戦 28 / 第三戦 36

週の終わり(練習・大会・EventPhase のあと。順序は上から)

停止	条件	備考
RaceDay:{大会}	大会週	当日の画面。**大会週の故障は、この画面で扱う**(当日変更・自動の代役。04 §5.3)
Event:{id}	発生したイベントのうち選択肢が2つ以上、かつ `event_auto = false`	選択肢が1つ(お知らせ)と `event_auto` のときは止まらず、月末レポートへ
Injury:{選手}	新しい故障が **重傷**、または **中傷で A グループ / 登録メンバー**(大会週を除く)	軽傷は止まらず月末レポートへ。登録メンバーの重傷・中傷は差し替えを促す(04 §1)
Warn	`week % 4 = 2` (3週目の終わり)で、故障していない選手に疲労が `K_UI_WARN_F` (夏合宿の月は 80)以上 85 未満の人がいる	**月に1回まで**。標準プランでは起きない(下記)
MonthReport	`week % 4 = 3`	常に
YearEnd	`week = 47`	常に

- **1週に止まる回数の上限は 3**(`K_UI_STOP_CAP_WEEK` 。MonthReport・YearEnd を除く)。超えたぶんは月末レポートの『お知らせ』に回す。
- 疲労の警告は、**夏合宿の通常の経過では起きない**(合宿の月の閾値を 80 にした。標準プランで 100チーム×4年度に **0回/年度**。§11.5)。夏合宿の洗礼(06 `ind.camp_hell`)や個別指定・イベントで疲労が溜まったときだけ出る安全弁。
- 止まる回数(標準プラン・20人の A+B グループ): 故障の停止 約 **1.7件/年度**、選択のあるイベント 約 7件/年度(06 §9)。

6.4 大会の流れ(画面の順序と観戦モード)

1. **SC-21 エントリー**(大会の4週前): おまかせ登録を初期値に出す。登録メンバーの故障は、大会の3週前〜前週に**差し替え**できる(C-RACE-02)。登録するとピーキング(03 §5.3)の対象になる(登録メンバーは大会の3週前から調整メニュー)。
2. **SC-22 区間オーダー**(週 `t` の初め): おまかせ編成(ハンガリー法。04 §5.2)を初期値に出す。各区間の**区間適性(◎〇△)・予想タイム(基準タイム)・調子の見込み**(R-05)を見て、人を入れ替える。検証 O1〜O3 に反する配置は確定できない(理由を表示)。
3. **SC-23 他校情報・予報**: 前評判(R-09)と上位選手の持ちタイム、環境の予報(04 §6)を見る。見るだけの画面(1タップで進む)。
4. (週の練習の解決。調子が決まる。)
5. **SC-24 当日**: 調子と環境が確定。区間ごとの**余裕の目安(◎〇△)**と、控えとの**期待タイムの差**を見て、**当日変更**(上限 1 / 2 / 4名)をする。観戦モード(フル / ダイジェスト / 結果のみ)を選ぶ。『目安どおりに指示』で、全区間の指示を余裕の目安どおりにそろえられる。
6. **SC-25 レース進行**:
   - **フル**: 区間の前に、自校の走者の**指示(攻める / 維持 / 抑える)**を決める(既定は維持。目安どおりのボタンが強調される)。区間の途中(1/3 か 2/3 の地点)に『異変』が出たら**檄**(残り回数を表示)。中継所ごとに順位と差、見どころを出す。速度 ×1 / ×2 / スキップ。
   - **ダイジェスト(おまかせ)**: 指示は維持、檄はブレーキ・失速時に自動で上限まで、当日変更は不調以下を自動で置き換える(04 §9.3)。中継所ごとの要約と、区間ごとに**見どころ最大2つ**(04 §9.3 の判定の優先順)。
   - **結果のみ(おまかせ)**: SC-25 を開かず、SC-26 へ進む。
7. **SC-26 結果**: 総合(順位・タイム・繰り上げ)・区間賞・自校の区間(順位・タイム・出来事)・評価ポイント・名声・選手の経験・記録更新。そのあと**大会後のイベント**(06 §14.4)。

- 乱数の引き方は指示・檄・当日変更で変わらない(04 §12)ので、**レースの途中で中断しても再現できる**。中断セーブは『次の区間の指示の前』の状態(完了した区間・指示・檄の回数・当日変更の回数)を持つ(§6.5)。

6.5 自動セーブと中断

時点	内容
月末レポートの確認後(月境界)	自動セーブ(2世代を交互)
オーダーの確定後(大会前)	自動セーブ。ここまで戻ってオーダーなどを変えてやり直せる
年度末の各節の後(精算 / 卒業 / 入学者発表)	自動セーブ
アプリの中断(バックグラウンド・省電力・終了)	**中断セーブ**を1つ書く。再開時に1回だけ使い、使ったら破棄する。レース進行中は『次の区間の指示の前』に戻る。週の途中(練習の解決のあとの当日・結果・イベント・月末レポート)で中断したときは、その週の練習を解決する直前の状態に戻って、同じ結果でやり直す(選んだイベントの選択肢は選び直す。09 §4.6)
手動	ポーズから(スロットごとに1つ)

- レースの乱数は、大会・チーム・区間をキーにして決まる(04 §12)ので、**ロードしても、オーダー・当日変更・指示を変えなければ同じ結果になる**(結果を引き直すためのセーブ・ロードはできない)。決定を変えれば結果が変わりうるが、これは通常の試行錯誤として許容する。
- セーブの形式・世代管理・クラウド同期は 09 §4・§9。ここでは**いつ書くか**だけを定める。

6.6 ヒント(はじめての人向け)

- 各画面を**初めて開いたとき**、ヒントのカードを1枚だけ出す(`K_UI_HINT_MAX_PER_SCREEN` = 1。『とじる』『ヒントをすべて消す』)。見たヒントは DeviceSettings に記録する(§9)。
- 強制のチュートリアルは作らない。新規ゲームの既定は『中堅』で、最初の1年度のヒントは『やること』の順(年間方針 → 月間計画 → 登録 → オーダー → 結果)に出す。

---

7. 共通部品

7.1 選手の行(`RunnerRow`。部員一覧・オーダー・エントリーなど)

列	内容	幅(u)
名前	顔のアイコン(32 u)+ 姓名(かな表示の設定で上にかな)+ 学年(『2年』)+ 主将の印 + **2行目に個性チップ**(R-17。幅は変えない)	96
グループ	A / B / C(自校のみ)	28
力	総合力 P(`P_long`)の数値 + ランク(R-01)	36
疲労	ゲージ + 数値	36
調子	5段階のアイコン(↑↑ ↑ → ↓ ↓↓)	28
やる気	5段階の顔	24
故障	なし / 軽 / 中 / 重(残り週数は長押し)	24
特殊能力	持っている数(金・青・赤の色 + 数)	36
出身	県名(留学生は国名)+ 高校名の頭	72

- 合計 408 u(§4.3)。狭い場所(右の詳細・ダイアログ)では、名前 + 力 + 状態の3列に省略する。
- **出身(県・高校)は常に出す**(01 §4.5、02 §6)。同郷(同じ県)は印(●)、同じ高校は印(◆)を、選択中の選手と同じ印の選手に付ける。
- 長押し(X / F1)で、その選手の詳細カードをポップアップする。1タップで選手詳細(SC-19)を開く(副作用なし)。

7.2 能力の表示

- 10能力を『SPD 62 C』の形(略称・数値・ランク文字)で並べる。レーダーチャートは、走力6(SPD STA PAC KCK CLB DWN)と精神・体質4の2つに分ける(1つの6角形に10軸を詰めない)。

- 数値の横に、前月からの増減(+1 など)を小さく出す。レベルアップした週は、数値を一瞬強調する(『動きを減らす』のときは強調なし)。
- **素質は数値で出さない**。自校の選手の能力には、素質のヒント(R-02)の印を付ける(◆ 伸びしろあり / ◇ 終盤 / ▼ 頭打ち間近 / ■ 限界)。
- 区間適性(R-01 の P とは別。04 §5.1)は、Fast / Mid / Long / Climb / Descent の5つを『適性◎○△』で並べる(◎ ≥70 / ○ ≥60 / △)。

7.3 状態の表示

状態	表示
疲労	ゲージ(0〜100)と数値。0〜39 緑 / 40〜69 黄 / 70〜84 橙(危険)/ 85〜 赤(故障寸前)。安全網が働く見込みの週は、ゲージに印を付ける(R-05)
調子	絶好調 ↑↑ / 好調 ↑ / 普通 → / 不調 ↓ / 絶不調 ↓↓。**大会の調子は週の終わりに決まる**ので、それまでは『見込み』(R-05)を出す(『好調以上 82%』)
やる気	1〜5の顔。2以下は注意の色
故障	軽傷 / 中傷 / 重傷の文字 + 残り週数。故障中は行を暗くし、メニューは『アクア・バイク代替』(03 §4.4)と表示

7.4 数値・時間の整形

> **正本は 09 §13**(`FmtTime FmtTotal FmtDiff FmtKm FmtMoney FmtYear FmtGrade FmtPct FmtRank`。整数だけで計算し、59 件のベクタと全探索で検算した)。下の表はその要約。タイム差の例は、規則(1分未満は `±s.s`)に合わせて直した(v0.1 の例 `+0:12.3` は規則と食い違っていた)。

種類	規則	例
タイム	`ds`(0.1秒)から。1時間以上は `h:mm:ss.s`、未満は `m:ss.s`(区間・記録は小数1桁)。総合タイムは `h:mm:ss`(**秒未満は切り捨て**。同タイム判定は `ds` で行う)	39543 → 1:05:54.3 / 総合 2:06:48
タイム差	0 は `±0.0`。**1分未満は `±s.s`、1分以上は `±m:ss`**(秒未満は切り捨て)。先頭との差は `+`、区間の優劣は `-`(速い)。符号は `+` と `-`(U+2212)	+12.3 / −7.1 / +1:05
距離	km(小数1桁)	21.3 km
金額	万円(3桁区切り)。1億円以上は『1.2億円』	1,500万円
年度	『第12年度』+ 月・週(『10月2週』)。大会の年度は『第12年度の第三戦』	第12年度 10月2週
学年	『1年』〜『4年』。OB は卒業年度	3年
割合	整数 %(予測の確率は 0〜100%。5%未満は『5%未満』、95%超は『95%超』)	82%
順位	『3位』。繰り上げのあったチームは『繰上』を付ける(04 §8.2)	8位 繰上

7.5 通知・バッジ・トースト

- 通知は `{ kind, text, week, target? }` を履歴(最大30件)に残す。ホームのベルに未読の数を出す。
- トースト(画面の隅に3秒)は、**状態を変えなかった**ときの知らせ(券の獲得・購入状態の確認・自動セーブ)だけに使う。判断が必要なものは、止まる画面(§6.3)で出す。
- ホームのバッジ: 券(『宣言できます』)・購入状態不明(『購入状態を確認できません』)・未読の通知・次の予定(エントリー締切までの週数)。

7.6 確認ダイアログ(SC-37)

種類	使う操作	作り
通常	セーブの上書き、タイトルへ戻る、グループの一括変更	本文 +『はい』『いいえ』。既定は『いいえ』
取り消せない	特別招聘の宣言、購入、セーブの削除、施設の強化	本文に**起きること**を書く(例:『券を1枚使います。取り消せません』)。『戻る』(既定のフォーカス・左)と『実行』(右・強調色)。長押しの誤操作を防ぐため、実行はタップ・A のみ

7.7 空・エラーの表示

場面	表示
一覧が空	理由と次の行動を1行で出す(『視察した候補はまだいません。地図から地方を選んでください』)
購入状態が不明	ホーム・設定・追加コンテンツの隅に『購入状態を確認できません(接続を確認してください)』。進行は止めない(07 §8.2)
セーブの読み込みに失敗	最新の世代が壊れていれば**自動で1つ前から再開**して知らせる。すべて失敗したときだけ『このセーブは読み込めません』+ 手動セーブ・退避の案内(09 §4.5)。ゲームを落とさない・ファイルを消さない
条件を満たさない操作	ボタンは無効表示で残し、選ぶと理由を出す(F4)

---

8. 画面のための規則(R-01〜R-17)

02〜07 が定義していなかった『画面に出す値・提案する値』の規則。**Domain に置き、画面は結果を表示する**(原則10)。

R-01 能力のランク表記(G〜S)

`rank(v)` = S: v ≥ 90 / A: ≥ 80 / B: ≥ 70 / C: ≥ 60 / D: ≥ 50 / E: ≥ 40 / F: ≥ 30 / G: それ未満(01 §4.1 の『数値 + ランク G/F/E/D/C/B/A/S』の閾値を確定)。10能力・力 P・適性値のすべてに使う。標準プラン4年間(03 §11 と同じ設定)の卒業時は、C 33% / D 47% / E 9% / B 10% / A 1% で、**能力の最大値の中央値は 73(B)、上位10%が 80(A)**(§11.9)。S は歴代級のエースの最高の能力にだけ付く。

R-02 素質のヒント(自校の選手)

素質(`potential`)の数値は出さない(01 §4.3)。代わりに、能力ごとに『能力 ÷ 素質』から3段階のヒントを出す: 比 < 0.85 = **伸びしろあり** / < 0.95 = **終盤** / ≥ 0.95 = **頭打ち間近** / 能力 = 素質 = **限界**(`K_UI_HINT_LATE` / `K_UI_HINT_CAP`)。成長タイプ(早熟 / 標準 / 晩成)は、**自校の選手には入部時から表示する**(入学後は練習の伸び方で分かるため。候補のときは隠し情報 = 05 §7.2)。

R-03 記録(PB)は能力から導出する(保存しない)

自己ベストは、能力から**調子 0・環境なし**で導出する(01 §4.1『記録は能力から導出して表示する』)。

```
P5000  = Fast の重み(04 §5.1)の適性値   P10000 = Mid の重み   P半 = Long の重み
記録(ds) = round_half_up( 10 × 距離km × ( pace50(距離km) − K_SLOPE × (P − 50) ) )        # pace50(d) = 176 + 6×ln(d/5)、K_SLOPE = 0.4
距離: 5000m = 5.0 km / 10000m = 10.0 km / ハーフ = 21.0975 km
```

- 全能力 90 で 10000m = 27:21.6(01 の『27:22 ≒ 能力90相当』と一致)。5000m = 13:20.0、ハーフ = 59:17.8(TV-3)。
- 能力は下がらない(03)ので、導出した記録は**単調に良くなる**。`Runner.pb` は**保存しない**(02 の `pb` は派生に変更)。`OBRecord.pb` は卒業時の導出値を保存する。
- 記録会(5〜6月の第3週)は**カレンダー上の行事**で、記録の更新には関与しない(06 の `ind.pb_meet` が『自己ベスト更新』の知らせを出す)。`RaceKind.Meet` は v1 では使わない。
- 候補の『持ちタイム』(5000m)は、**候補の現在の能力**(05 §6.2 の展開。素質は含まない)から同じ式で出す。入学すると同じ能力から導出するので、**持ちタイムは入学の前後で連続する**。

R-04 推奨プランと年間方針・グループ編成

年間方針(`annual_plan`): `focus` ∈ {`Balanced`(バランス)/ `Speed`(スピード寄り)/ `Long`(ロング寄り)}(01 §5.1 の4種から変更。理由は下)、`target_race` ∈ {第一戦 / 第二戦 / 第三戦}(**画面の見出しと予測の注目大会にだけ使い、規則には作用しない**。既定は第三戦)、`peaking_mode`(03 §5.3 のまま)。

推奨プラン: 重点ごとの12か月のテンプレートから、その月のテーマを**全グループ共通**で返す(`auto_plan(focus, month)`)。動的な調整はしない(標準プランそのものが基準で、検算済みの成長曲線と一致する)。

月	4	5	6	7	8	9	10	11	12	1	2	3
バランス(標準プラン。03 §11)	基礎	スピード	基礎	山	合宿	スピード	基礎	基礎	山	回復	スピード	基礎

月	4	5	6	7	8	9	10	11	12	1	2	3
スピード寄り	基礎	スピード	**スピード**	山	合宿	スピード	基礎	**スピード**	山	回復	スピード	基礎
ロング寄り	基礎	スピード	基礎	**基礎**	合宿	スピード	基礎	基礎	**基礎**	回復	スピード	基礎

重点ごとの違い(4年間・1500人・施設Lv2。標準プランとの適性値 P の差。§11.2)

重点	Fast	Mid	Long	Climb	Descent	故障/人年
バランス(基準)	—	—	—	—	—	0.349
スピード寄り	+1.19	+0.27	+0.02	−0.40	+0.34	0.344
ロング寄り	+0.35	+0.54	+0.58	−1.27	−1.81	0.349

- **ロング寄りは『山の月』をなくす代わりに、登坂・下りが伸びない**(CLB +1.4、DWN +0.0。標準は +5.1 / +4.6)。スピード寄りは、短い区間(第一戦・第二戦)に強くなり、山登りがやや落ちる。**どれも『標準より常に良い』『常に悪い』にならず、山区間をどうするかのトレードオフになる。**年間方針の画面は、この表(区間タイプ別の適性値の差)をそのまま出す(I-14)。
- **『スタミナ型』『山重視』のチーム全体の重点は作らない**: 検算で、チーム全体を持久・山に振ると、**狙った区間タイプ以外が大きく下がり、3大会のどれでも標準プランに劣る**ことが分かった(持久重視: Fast −3.2 / Mid −1.4 / Long −1.0 / Descent −1.1、Climb だけ +0.3。山重視: Fast −2.3 / Mid −1.5 / Long −1.2、Climb +1.1 / Descent +1.3。第三戦で換算しても −0.7〜−0.9。素質の伸びしろが能力ごとにほぼ同じなので、SPD を削って STA を足しても得にならない。§11.2)。山区間は、**山を担当する数名のグループ(例: C)だけを山対策のテーマにする**ほうが効率的で、これはグループ運用の工夫(手動)に残す。
- **型に合わせた育成**(04 §11 の `SPEC_SHIFT`)は、グループ単位の推奨プランでは再現できない(選手ごとの個別指導が要る)。検算で、これを外しても長期の結果は変わらない(優勝率の差が ±0.016 以内。§11.8)ので、**v1 の『おまかせ』は型別の調整を持たない。**個性(10)が加わっても同じ: 型でグループ分けしても期待値の上積みは +0.2 P 程度で、同じ最良の方針に揃える方が効く(10 §5.5)ので、推奨プランと自動編成は個性を見ない(選手の個性は R-17 のチップと相性の印で見せ、判断はプレイヤーに任せる)。

グループの自動編成(`auto_groups`): 力 `P_long` の高い順(同値は ID の小さい方が上)に、上位 `K_UI_GROUP_A` (10)人を A、次の `K_UI_GROUP_B` (10)人を B、残りを C。**新規ゲームの開始時は、この編成を自動で適用する。**以後は自動で動かさず、4月(年度初め)に**『推奨の編成を適用』ボタン**を出す(1タップ)。新入部員は C に入れる。グループを変えると、別グループの組の絆が月に −10 ずつ下がる(03 §7.1)ので、変更の確認で知らせる。

`plan_auto` **(設定)**: true のとき、月間計画では止まらず、毎月推奨プランを自動で確定する(年度初めの年間方針とグループ編成の確認は止まる)。

R-05 月末の予測(Forecast)

月間計画・オーダー・個別指導の画面に、**これから起きること**を出す。計算は 03 §6 の週の解決から**乱数を除いたもの**(`expected_week`):

```
入力: 選手の状態、この先の週メニュー(個別指定 > 故障 > ピーキング > 安全網 > テーマ。03 §5.1)、施設、有効な補正(06 §6)
手順(1週ごと): 03 §6 の 手順1(E)→ 2(XP とレベルアップ)→ 4(疲労 F_end)を、故障は『起きない』として進める。
  手順5: 故障の確率 p だけを求める(乱数は引かない)。手順6: 調子スコアの中心 mu = (50 − F_end) × 0.5 + clamp(peaking + form_add, −8, +12) と、標準偏差 sigma を求める。
出力: F_end の推移 / 能力の伸び(能力の変化 + XP の端数)/ 安全網が働く週 / 期間の故障の確率 1 − Π(1 − p) / 最後の週の調子の分布
調子の分布: スコア ~ N(mu, sigma) として、絶好調 ≥ 20 / 好調 ≥ 8 / 普通 > −8 / 不調 > −20 / 絶不調 の確率(03 §2 の境界)
```

- **疲労・能力・XP・安全網は『正確』**(乱数を含まないので、故障が起きなければ実際の結果と一致する)。**故障と調子は確率**で出す。検算: 2,977件(個性・素養つき 1,500 件・個別指導の週 600 件を含む)で F・能力・XP・調整週・故障確率が実際の週の解決と一致、調子の中心の差 3.6e-15。調子の分布は、20000回の実測と ±0.003 で一致(§11.1)。
- **画面に出す項目**(月間計画。グループごと): ①月末の疲労(平均・最大)と週ごとの推移 ②要注意の人数(月内に F ≥ 70 になる人 + 故障者)③故障の見込み(Σ p =『月に約0.3件』)④安全網が働く週の数 ⑤**伸びの見込み**(区間タイプ別の適性値 P の増減。テーマごとの比較 = TV-10)⑥**次の大会の調子の見込み**(『好調以上 82%』= 故障しない確率 × スコア ≥ 8 の確率。大会が8週以内にあるとき)。
- **実装(`GameSession.FormOutlook` 。 `Views/PlanViews.cs` の `FormOutlookView`)**: 今週から大会の週(練習の解決まで)を、週ごとに 個別指導(月末まで)> 調整(登録した大会。未登録ならおまかせ登録の見込み。エントリーの画面は『登録した場合』)> 今月は月間計画・来月以降は推奨プラン で進める。大会の週の練習がすでに解決されたあとは確定した調子(`Determined`)。安全

網の閾値は設定（`auto_safety_f`）に従う。実際の週との突き合わせ（試験）: 疲労は故障しなかった選手の 98% が ±3 以内（間に大会のない第一戦は 96% が完全一致。外れの多くは、登録が締切までに入れ替わった選手）、『好調以上』の平均 0.593 に対して実際 0.595（2,652 人 × 大会）。

- 未設定の月は**推奨プラン**で、登録されていない大会は**登録見込み（おまかせ登録）**で計算する。補正（`TimedModifier`）は期間中なら含める。個別指定は含める。
- 例（TV-6）: 03 TV-3 の入力で基礎ビルド4週 → F = 46 → 54 → 59 → 45（03 TV-4 と一致）、故障 2.91%、4週目の調子は 好調以上 23.5%。

R-06 個別指導（指導ポイント）

月に `K_UI_GUIDE_POINTS` (4)ポイント（月初に4へ戻る。**繰り越さない**。02 §4.4）。操作は2つ。

操作	費用	内容
メニューの指定	1ポイント / 人	その選手に、**今週から月末まで**、（ポイント練習2つ + 走行量）を指定する。03 §5.5 の検証（2枠は異なる / `ks.cross_train` は不可 / 走行量は必須）を通す。プリセット: **ケア**（`ks.core` + `ks.rest` + `mil.low` = 03 の SAFE）/ **調整**（`ks.sharpen` + `ks.pace_run` + `mil.low`）/ 自由編集。**月をまたがない**。確定前、および最初の週が解決される前なら取り消せる（ポイントは戻る）
面談	1ポイント / 人	やる気 +1（03 §7.2 M3）。1人につき月1回

- 故障中の選手には指定できない（03 §5.1 の最優先）。指定したあとに故障が起きたら、**指定は破棄する**（ポイントは戻らない）。
- **個別指定は、ピーキング（大会の調整メニュー）より優先され、安全網の対象外**（03 §5.1, §5.4）。大会の調整期間にある登録メンバーに指定するとき、『調整メニューを上書きします』、疲労が 85 以上の選手に指定するとき『安全網が働きません』と**確認で知らせる**。
- データ（02 に追加。§10）: `PlayerTeamState.individual_menu: map<runner_id, { until_abs_week, menu }>` と `talked: list<runner_id>`（今月に面談した選手。月初に空）。
- イベントの効果 `guide`（06 §5.2）と**同じポイントを使う**。足りないとき、イベントの選択肢は `req`（06 §4.2）で選べない。
- 検算（`can_guide`、TV-11）: ポイント不足 / 故障中 / 2枠が同じ / cross_train / 走行量なし / 面談の2回目 を、すべて理由つきで拒否する。1か月に指定できるのは最大4人（部員の1割）で、成長への影響は、03 §5.1 が既に許す範囲の個別化に収まる（検算対象外）。

R-07 進行の停止

§6.3 の決定表。`stops_before_week` / `stops_after_week`（`ref_model_08.py`）がそのまま実装の仕様（TV-8）。

R-08 スカウトの画面

時期とポイント（05 §7.1 のポイントを、時期で区切る）

項目	規則
視察	**6〜11月**（`month_index` 2〜7）。1ポイント。未視察の候補に
声かけ・練習会	**12〜2月**（`month_index` 8〜10）。各1ポイント。声かけは視察済み、練習会は声かけ済みの候補に（05 §7.1）
確定	3月（05 §12 手順7）。4月に入部
ポイントの付与	6〜2月の毎月 +4（計36）。ランクの追加分（`4 × floor(ランク/3)`）は6月に一括。06 の補正 `scout_pts` は付与された時点で加算（6〜2月以外なら6月に加算）
繰り越し	**同じ年度の中では繰り越す**（使わなかったポイントは、あとの月で使える）。3月に残りは消える（次年度へは繰り越さない）

→ 視察で多く見るか、勧誘に絞るかが**ポイントの配分の判断**になる（視察 S 人 + 2 × 目標 T 人 ≤ 年間ポイント）。

候補の表示（I-19〜I-21）

- 最初から見える: 名前・出身（県・高校・距離帯）・評価★（V から。05 §6.1）・**市場評価 V**（数値）・持ちタイム（5000m。R-03）・性格タグ（R-16）・競合度。
- **視察で見える**: 卒業時の見込み `Q_obs ± σ_info`（1σの範囲『64〜72』。`σ_info` は スカウト担当のレベルで 8 → 3.2。05 §2）。
- **練習会で見える**: 成長タイプ・故障しやすさ・素質の細部（05 §7.2）。

競合度・枠・確定の順番（検算済み。§11.7）

- **確定は市場評価 V の高い順に行われ、定員 10 に達すると、それ以降の候補は入れない**(05 §8.2)。狙っていない(自然に入る)候補も同じ列に並び、**平均 3.3 人**が自然に入る(『狙わなくても入る人数の目安』`K_UI_SCOUT_NATURAL = 3`)。→ 目標のうち **V の高い順に 7番目までは 74〜96%、8番目は 62%、9番目は 31%、10番目は 7%、11番目以降は 0%**(実測)。**推奨の目標は 8人**(`K_UI_SCOUT_TARGET_REC`)。
- **競合度**(他校の狙い撃ち)は V の帯で決まる: V < 51.5 **ほぼなし** / < 55 **少** / < 59 **中** / それ以上 **多**。**練習会まで進めて、枠があれば競り勝つ割合の実測**は それぞれ 約100% / 約100% / 約92% / 約65%。上位級(V ≥ 51.5)は**奨学金枠**(自校 `min(5, 寮Lv + 1)`)を1つ使い、枠が尽きると入れない。
- **目標画面の『枠のメーター』**: 定員 10 − 特別招聘の予約(あれば1)= 使える枠 / 自然に入る目安 3 / 推奨の目標 8 / 上位級の目標数と奨学金枠。目標が推奨を超えたら黄色、使える枠 − 自然に入る目安 を超えたら注意を出す。各目標に『確定の順番: n番目』を表示する(n ≥ 9 は『ほぼ入れません』)。
- 競合に負ける割合は、枠があった場合でも全体で 4.7% しかない(競り負けより、定員・奨学金枠で入れないほうが 69% と大きい)。**『見込み』の確率は表示しない**: 05 §7.3 の `pwin` は 0.9〜0.99 の帯で実測 72% と較正が悪かったため、競合度と枠の見通しで見せる(§11.7)。

おまかせスカウト(C-SCOUT-01 の自動版。`settings.scout_auto`、または『おまかせスカウト』ボタン)

```
T = 8(目標)、S = 年間ポイント − 2×T(視察する人数。ランク1〜2で 20、3〜5で 24)
視察(6〜11月): 事前の価値 (V + 7.2) × pwin の高い順に S 人を視察する(7.2 = 伸びしろの平均 24 × 0.3)
勧誘(12月): 視察で見えた Q_obs × pwin の高い順に T 人を選ぶ(上位級は奨学金枠の数まで)。声かけ + 練習会を行う
```

- 05 §7.3 の検算用の方針(全員の Q_obs を知る・約13人を狙う)と比べて、**入学者の Q は 中堅で 0.8〜0.9 低い**(64.3 / 65.1)が、**15年度以降の戦力 P は 中堅で −0.4〜0.0、弱小で +0.7〜+2.3**。下がる方向の差は 0.4 以内で、検算済みの長期の結果を下回らない(§11.7)。**C# の実体の自校でも確かめた**(L-58。進行制御を通して月ごとにボタン。T4c の条件): 優勝率 0.590 / 0.533 / 0.452・クリア 8.33 / 12.86 / 22.98 年(強豪 / 中堅 / 弱小)で、検算用の方針(0.588 / 0.534 / 0.456・8.97 / 13.74 / 23.48 年)と同等。

R-09 前評判(他校情報)

大会の画面(SC-23)に、**出場校の『前評判』**を出す。

```
前評判の値 = 登録メンバー(他校はおまかせ登録)の上位 run_size 人の『区間適性 Long』(04 §5.1)の平均
前評判の順位 = 値の大きい順(自校を含む)。印: ◎ 1位 / ○ 2〜3位 / ▲ 4〜6位 / △ 7〜10位 / それ以外は印なし
各校の上位選手: 区間適性 Long の上位3人の名前・学年・5000m / 10000m の持ちタイム(R-03。他校の能力は 04 §11 の復元)
```

- 較正(1440大会): **◎ の優勝率 54〜56%、3位以内 84〜87% / ○ 17〜18%、60〜63% / ▲ 3〜4%、25〜26% / △ 0%、3〜4%**。前評判の順位と結果の順位の相関(スピアマン)は 0.83〜0.84(§11.6)。**印は『当たりやすさ』として実測と合っている**。画面には印と一緒に『◎の優勝率は約55%』と注記する。

R-10 月末レポート

- 構成: ①**今月の結果**(チームの平均の疲労・調子・士気の変化)②**注目の項目**(最大 `K_UI_REPORT_MAX` = 7件。下の優先度)③次の月への注意(警告の数・来月の予定)。③のあとに『ほか n件』(長押しで一覧)。
- **注目の項目の選び方**(`select_report_items`): 種別の優先度の順(重傷の故障 > 中傷の故障 > 覚醒 > 素養の判明 > ランクアップ > 大きな成長 > 復帰 > お知らせ > 調子の上昇)に、同じ種別は score の大きい順に最大3件、全体で7件。残りは『ほか n件』。

種別	優先度	内容	score
`injury_severe` / `injury_moderate`	100 / 90	新しい故障	力 P
`awaken`	80	特殊能力の覚醒(06 §9)	ランク(金 > 青 > 赤)
`gift_reveal`	75	素養の判明(入部から `K_GIFT_REVEAL_WEEKS` 週。10 §8。『◯◯に登坂の素養があることが分かった』)	0
`rank_up`	60	能力のランク文字が上がった(R-01)	新ランクの数値
`big_growth`	50	月に能力が合計 +3 以上	合計の増加

種別	優先度	内容	score
`return`	40	故障からの復帰	力 P
`notice`	30	イベントのお知らせ(§6.3)	0
`form_up`	20	調子が上がった選手	調子の差

- 材料の量(20人・標準プラン・1か月): レベルアップ 約 28件、ランク文字が変わった 約 2.8件、月に能力 +3 以上 約 3.0人(夏合宿の8月は 48件 / 4.6件 / 9.1人)。**全件は出せないので、選んで出す**(§11.5)。
- 例(TV-12): 材料15件(重傷1・ランクアップ5・成長4・お知らせ3・覚醒1・復帰1)→ 重傷1・覚醒1・ランクアップ3・成長2 の7件 +『ほか8件』。

R-11 施設・スタッフの強化

- **いつでも実行できる**(資金が足りるとき)。**標準の実行は年度末の精算(05 §12 手順3)のあと**: 寮のレベルは同じ年度の奨学金枠と提示スコアに効く(05 §5、§8.1)ので、入学の割り当て(手順7)の前に強化する。年度末のフロー(§5.2 N5)に SC-32 を組み込む。
- **推奨の強化**(検算用の方針と同じ。`recommended_upgrades`): 寮 → スカウト担当 → コーチ → トラック → 坂道 → トレーニング室 → トレーナー → 栄養士 → 医療室 の順に、**上限(監督ランクによる。05 §3.3)に達していない最初のもの**を、資金から**予備費 500 万円**を残せる間、強化する(費用 `300 + 250 × 現在Lv`)。画面は『推奨をすべて実行』と『1件ずつ』を出す。
- 主務(`StaffRole.Manager`)は効果が定義されていない(03・05)ので、**v1 では表示しない**(データには残す。課題 L-25)。

R-12 セーブ

§6.5。スロット数 `K_UI_SAVE_SLOTS` = 3、自動セーブ2世代 + 手動1。形式と同期は 09 §4・§9。

R-13 新規ゲーム

3ステップ。①**開始校**(強豪 / 中堅[既定]/ 弱小 / 同好会 + 所持しているシナリオ。各カードに戦力順位・施設・資金・部員数・**クリアまでの目安**(強豪 約3.5時間・中堅 約6.5時間・弱小 約11.5時間・同好会 約18時間。05 §14.3)・難易度の説明)→ ②**自校**: 世界の生成(自校の名前・所在地・校色を表示。**『別の世界にする』で引き直せる**。生成の時間が1秒を超えるときは進捗を出す)、見た目(校章 8 / ユニフォーム柄 4 / 応援歌 2 / BGM 4 + 所持するパック)、**監督名**(8文字まで。既定『監督』)→ ③確認して開始。スロットは①で選ぶ。同好会は『部員5人。最初の年度は第一戦に出られません(6人必要)』を明示する(07 §10.3)。

R-14 レース演出の時間

1区間の演出 `K_UI_SEC_ANIM_S`(6.0秒、フル)/ `K_UI_DIGEST_SEC_S`(2.0秒、ダイジェスト)。速度 ×1 / ×2 / スキップ(いつでも。スキップは結果へ)。大会ごとの機械的な時間(§11.3): 第一戦 134秒(予算 150)/ 第二戦 160(210)/ 第三戦 188(360)。

R-15 イベントの効果の要約(選択肢の2行目)

イベント(06 §4.2)の選択肢は、`label` の下に**効果の要約**を1行で出す: `fx` から ≤ 3項目(1項目 ≤ 14文字)を作り、あふれたら『ほか n件』。変換: やる気 +1 →『やる気+1』/ 疲労 −15 →『疲労−15』/ XP(`xu`)→ 能力名 + `xu ÷ 10`(『MEN経験値+10』)/ 資金 →『−50万円』/ 特殊能力 →『特殊能力 ○○(35%)』/ 補正 →『効率×1.03(4週)』/ 対象は『(全員)』『(主力)』『(遠方出身)』など `to` の名称(イベントの当事者 `r`・チーム全体・資金・スカウト・指導ポイントは自明なので書かない)。確率つきは『やる気+1(50%・全員)』のように括弧の中に **確率 → 期間 → 対象**の順。OB の役割は『OBコーチ:効率×1.06』(役割が作る補正。1 項目だけの選択肢なので 20 文字まで)。**選択肢の本文(`label`)には数字を書かない**(数値はこの要約が構造から作る。本文と効果がずれるのを防ぐ。L-63 ①)。実装: `App/EffectText.cs`(文字列は `efx.*`)。選べない選択肢(`req` が偽)は暗くし、理由(『指導ポイントが足りません』『資金が足りません』)を1行で出す。

R-16 候補・選手の性格タグ

`traits`(最大3。02 §4.1)は、**`look_seed` から決める**(候補のときも入学後も同じ値になる): `n = 1 + (u < 0.5)`(1〜2個)、`Trait`(8種)から重複なしで一様に選ぶ。**乱数は固定の 3 回**(① 個数 ② 1 つ目 ③ 2 つ目。個数が 1 のとき ③ は使わない。残りの種類から `floor(u × 残り)` 番目を選ぶ。鍵は 09 §8.2 の label 8・`a = look_seed`・`b = 20`。参照モデル `ref_model_08.traits_of`)。**性格タグに規則上の効果はない**(表示とイベント文言の差し込みだけ。3個目は拡張用)。

R-17 個性と相性の表示(10 §2・§5.4・§8)

`Runner.style`(型・走法・展開)と `gifts`(素養)を、画面に**見せるだけ**で、規則には作用しない(規則は 03 §6・04 §7.3)。

- **個性チップ**(`RunnerRow` の名前の列の2行目。選手詳細のヘッダ。候補は視察後): **普通でない軸だけ**を出す。型 =『スピード』『ロング』/ 走法 =『集団』『単独』/ 展開 =『追』『逃』。三つとも普通なら『標準型』。色だけに頼らない(§4.2)。長押しで意味を出す(例:『ロング型: STA が伸びやすく、SPD が伸びにくい』『単独走◎: 仲間との絆の効きが小さいが、個別指導の週は効率が高い』)。
- **相性の印**(月間計画のグループの行・選手詳細・個別指導): `fit(r, theme)`(10 §5.4。型・展開と**判明した素養だけ**で計算)を、◎(≥ `K_FIT_GOOD` 1.05)/ ○(≥ `K_FIT_OK` 0.99)/ △ で出す。語を付ける(『相性◎』。§4.2)。月間計画では、選んだテーマについて『相性 ◎ n人 / ○ n人 / △ n人』(故障者を除く)。個別指導では、選んだメニューの XP の配分から同じ式で出す。**印は『合う』を出すだけで、推奨プランは個性を見ない**(上の R-04)。
- **素養の判明**: 入部から `K_GIFT_REVEAL_WEEKS`(12)週後の週の解決で `gift_revealed = true` にし、次の月末レポートに `gift_reveal` の項目を出す(R-10)。スカウトで種類まで見えていた選手は、入部と同時に判明。判明するまで、画面に素養を出さない(月末の予測 R-05 の数字には効果が出る。10 L-43)。
- **山対策のグループへの案内**: 登坂・下りの素養が判明した選手には、部員一覧のグループの表示と選手詳細に『山対策のグループ(C)で伸ばせます』と『Cへ移す』(C-PLAN-02)を出す。**自動では移さない**(グループを変えると絆が下がる確認は従来どおり。03 §7.1)。山の神・山下りの達人に届くには、坂道の練習を続けるのが条件(06 §9)。
- 候補(SC-28・29): 個性は視察後、『素養あり』は練習会の後、種類はスカウト担当 Lv3 以上のとき(05 §7.2)。他校の選手には出さない。

---

9. 設定項目と保存の場所

設定は**保存の場所が3つ**に分かれる(同じ設定を2か所に持たない)。

場所	内容	同期
`SaveData.settings`(セーブごと。ゲームの進行に関わる)	`spectate_mode` / `event_auto` / `plan_auto`(新)/ `scout_auto`(新)/ `auto_safety_f` / `school_look`(`uniform_pattern`・`cheer_song`・`bgm_set`)	セーブと一緒
`DeviceSettings`(端末ごと。見え方・音・操作)	文字の倍率 / 色の見え方 / 動きを減らす / かな表示 / 音量(BGM・効果音・応援歌)/ 触覚 / 演出の既定の速度 / ヒントの既読 / 言語	**同期しない**(画面の大きさ・耳の好みは端末で違う)
`ProfileData`(アカウント。02 §3.10)	実績・累計・クリアの記録	クラウド(Steam Cloud など。**マージ**。09 §9.6)
`LocalState`(端末ごと。09 §3.3)	購入状態の確認のキャッシュ・同期の基準点・時計の高水位・実績の保留。利用者には見せない	**同期しない**

設定画面(SC-04)のタブ

タブ	項目	保存先
ゲーム	観戦モード(フル / ダイジェスト / 結果のみ。既定: フル)/ イベントをおまかせ(既定: オフ)/ 月間計画をおまかせ(既定: オフ)/ スカウトをおまかせ(既定: オフ)/ 安全網(慎重 75 / 標準 85 / 無効。既定: 85)/ 自校の見た目(校章・ユニフォーム柄・応援歌・BGM。所持している範囲。校章は `SchoolState.emblem_id`、ほかは `school_look`)	SaveData
表示	文字の大きさ(100 / 125 / 150%)/ 色の見え方(標準 / 色の区別を強める)/ 動きを減らす / かな表示 / 演出の速度(×1 / ×2)/ ヒントを出す	DeviceSettings
音	BGM / 効果音 / 応援歌の音量(0〜10)/ 触覚(スマホ)	DeviceSettings
操作	パッド・キーボードの割り当ての一覧(表示のみ。v1 は変更不可)	—
データ・購入	セーブの日時と同期の状態 / **クラウド同期(オン・オフ。`DeviceSettings.cloud_sync`。既定オン。09 §9.5)**・『今すぐ同期』/ 購入状態と『購入を復元』(07 §8)/ 追加コンテンツ(SC-05)/ クレジット(SC-07)/ 診断情報のコピー(09 §3.1)	DeviceSettings(クラウド同期)

- 観戦モード・イベントおまかせ・計画おまかせ・スカウトおまかせは、**ホームやレースの画面からも切り替えられる**(同じ設定を指す)。

- 設定を変えても**結果は変わらない**(見た目・音・演出だけ)ものと、**進行の止まり方が変わる**もの(おまかせ)を、設定画面の項目の説明に書く。観戦モードは**ゲームの結果に影響する**(おまかせの采配と手動の差。04 §14.4)ので、変更の説明にそう書く。

---

10. データへの追加(`02_data_spec.md` へ反映する内容)

対象	追加・変更
`PlayerTeamState`	`manager_name: string`(8文字まで。既定『監督』)/ `individual_menu: map<runner_id, { until_abs_week, menu: (ks1, ks2, mil) }>`(R-06。月末に消える)/ `talked: list<runner_id>`(今月に面談した選手。月初に空)/ `scout_points`:**同じ年度の中で繰り越す**(3月に0。R-08)/ `guidance_points`:月初に 4(繰り越さない)
`annual_plan`	`focus: Focus`(新しい列挙。`Balanced` / `Speed` / `Long`。以前の案の `Endurance` / `Mountain` は作らない = R-04)/ `target_race: RaceKind`(`Fall` / `National` / `NewYear`。**表示のみ**。既定 `NewYear`)
`Runner.pb`	**保存しない**(能力からの導出。R-03)。`OBRecord.pb` は卒業時の導出値を保存
`SaveData.settings`	`plan_auto: bool`(既定 false)/ `scout_auto: bool`(既定 false)を追加
`DeviceSettings`(新。02 §3.12)	`text_scale`(100 / 125 / 150)/ `color_mode` / `reduce_motion` / `kana` / `volume_bgm` / `volume_se` / `volume_cheer`(各 0〜10)/ `haptics` / `anim_speed`(1 / 2)/ `hints_seen: list<screen_id>` / `hints_on` / `language`(`ja`)
`Focus` 列挙	`Balanced`(バランス)/ `Speed`(スピード寄り)/ `Long`(ロング寄り)
中断セーブ	`suspend: { week?, race?: { race_kind, race_id, next_section, started, touched, instructions[], geki_used[], geki_spent, swaps[] } }`(形式は 09 §4.6。通常のセーブには入れず `suspend.eksv` にだけ付く。再開は『オーダー確定後』の状態から区間を再計算。R-12 / §6.5)
`SaveMeta`(スロットの表示用。09)	`{ slot, saved_at, year, month_index, school_name, scenario_id, difficulty?, play_seconds }`

---

11. 検算結果(`verify/ref_model_08.py`)

11.1 月末の予測の厳密性と較正(R-05。`forecast`)

- **週ごとの厳密性**: ランダムな状態・メニュー・月・学年・成長タイプ 2,977件(故障が起きなかった週。うち 1,500 件は個性・素養つき、600 件は個別指導の週)で、`expected_week` が 03 の `resolve_week` と**一致しない件数 = 0**(F・能力・XP・調整週・シャープニング履歴・故障確率・調子の中心。調子の中心の最大差 3.6×10^{-15})。
- **調子の分布の較正**: 月初に立てた予測と、乱数で4週を20000回実際に進めた結果(故障が起きなかった経路)の比較。

ケース(4週)	故障なし 予測 / 実測	好調以上 予測 / 実測	普通 予測 / 実測	不調以下 予測 / 実測
基礎ビルド4週(F=55)	0.970 / 0.970	0.289 / 0.288	0.663 / 0.665	0.048 / 0.047
ピーキング T1〜T4(F=60)	0.980 / 0.980	0.995 / 0.994	0.005 / 0.006	0.000 / 0.000
夏合宿4週(F=45)	0.938 / 0.936	0.106 / 0.105	0.729 / 0.730	0.165 / 0.165

→ **『好調以上 82%』と画面に出したら、実際にその割合になる**。補正(06 §6)の有無は、式(03 §6)のとおりに含める(E_mult ×1.1 で XP が増える / r_add +0.06 で F が下がる / inj_mult ×2 で故障確率が約2倍 / form_add +4 で調子の中心が +4 になることを確認)。

11.2 推奨プランと重点ごとの違い(R-04。`autoplan`)

4年間の育成(03 §11 と同じ設定: 入学時の能力は平均50、素質は能力 +15〜35、施設・スタッフは Lv2、大会に3回とも出走)。標準プラン(バランス)との**適性値 P の差**と、能力の4年間の上昇。

重点	人数	SPD	STA	PAC	KCK	CLB	DWN	Fast	Mid	Long	Climb	Descent	故障/人年
バランス(基準)	1500	11.6	21.1	17.3	9.5	5.1	4.6	(13.35)	(14.98)	(15.46)	(11.21)	(9.77)	0.349
スピード寄り	1500	15.0	19.6	15.9	11.8	5.1	4.7	**+1.19**	+0.27	+0.02	−0.40	+0.34	0.344

重点	人数	SPD	STA	PAC	KCK	CLB	DWN	Fast	Mid	Long	Climb	Descent	故障/人年
ロング寄り	1500	11.6	21.8	18.5	9.5	1.4	0.0	+0.35	+0.54	+0.58	**−1.27**	**−1.81**	0.349
(持久重視。採用せず)	600	3.7	22.8	19.2	5.2	5.1	4.6	−3.19	−1.44	−0.96	+0.32	−1.13	0.352
(山重視。採用せず)	600	6.8	21.4	17.2	7.0	8.2	9.6	−2.34	−1.48	−1.23	+1.14	+1.25	0.352

- バランスの行は 03 §11 の標準プランの表(SPD +11.6 / STA +21.2 / PAC +17.3 / KCK +9.5 / CLB +5.1 / DWN +4.6)と一致する。故障・大会週の調子(好調以上 0.85)は、重点によらず同じ。
- **持久重視・山重視(チーム全体)は、狙った区間タイプ以外が大きく落ち、第三戦で換算しても −0.7〜−0.9**(『Long の8区間 + Climb 1 + Descent 1』の平均)。**素質の伸びしろが能力ごとにほぼ同じ**(05 §6.2 `potential = stat + gap`)ので、STA は4年で上限に近づき、それ以上の STA の練習より SPD の伸びのほうが適性値に効く。これは 03 の設計が許す結果で、**3つの重点(バランス・スピード寄り・ロング寄り)に絞った**(R-04)。
- **疲労ガード**(月内に F ≥ 70 の人が3割以上なら回復月)は入れなかった: 標準のテーマでは、REC 30・F=72 から始めても月内の最大が 66 で、一度も発動しない(安全網 F ≥ 85 と、警告 R-07 で足りる)。

11.3 1年度の操作時間(R-14。`budget`)

機械的な最小時間(タップ・読み取り・演出。考える時間は含まない)。定数: タップ 1.0 秒 / 読み取り1項目 2.0 秒 / 画面遷移 0.5 秒 / 週の演出 1.2 秒 / 1区間の演出 6.0 秒(ダイジェスト 2.0 秒)。

区分(秒)	年度初め	月間計画	スカウト	大会	イベント	年度末	合計
予算(01 §3.4 + 年度末 120 秒)	180	480	240	720	180	120	**32.0 分**
A フル観戦・手動(既定の設定)	30	382	158	492	128	80	**21.2 分**
B ダイジェスト・手動の計画	30	382	158	324	128	80	18.4 分
C ダイジェスト・おまかせ(計画・イベント自動)	30	220	158	279	72	80	**14.0 分**
D 結果のみ・おまかせ(最短)	30	220	158	183	72	80	**12.4 分**

- **A の余白**(予算 − 機械的な最小時間 = 考える時間に使える秒数): 年度初め +150 / 月間計画 +98 / スカウト +82 / 大会 +228 / イベント +52 / 年度末 +40。**どの区分も予算に収まる。**合計 21.2 分に考える時間 約 10 分を足して 約 31 分(01 §3.4 の約30分。年度末の 120 秒を加えて予算は 32 分)。
- 最短(C・D)は **12.4〜14.0 分**で、01 §3.4 の『最短なら 12〜15分』と一致する。
- 大会ごと(A。秒): 第一戦 134(予算 150)/ 第二戦 160(210)/ 第三戦 188(360)。D: 54 / 58 / 62。

11.4 レイアウトの収まり(`layout`)

基準の高さ 360 u、内容 264 u。

幅 W	W'(安全域を除く)	10区間のカード(≥48)	年間予定の1か月(6×2段)	部員一覧の列 / 余裕	月間計画の行 / 内容	エントリーの見える行	順位表の見える行	県のタイル 3×3
640	592	51.4 u ○	94.7 u	408 / 408 ○	168 / 264 ○	4	7	204 u ○
720	672	59.4 u ○	108.0 u	408 / 488 ○	168 / 264 ○	4	7	204 u ○
800	752	67.4 u ○	121.3 u	408 / 568 ○	168 / 264 ○	4	7	204 u ○

→ **すべて収まる**(W=640 の部員一覧の列は余裕 0 で、これが最小幅の下限)。年間予定は初め12列で組んだが、W=640 で 47.3 u(< 48 u)となったので、半年 × 2段に変えた。実機の日本語フォント・文字の倍率での見え方は未検証(M1 の技術検証。課題 L-31)。

11.5 割り込みと月末レポートの量(R-07, R-10。`stops`)

20人(A10 + B10)・標準プラン・100チーム × 4年度。

項目	結果
疲労の警告の停止	**0.00 回/年度**(夏合宿の月の閾値を 80 にした。閾値 70 のままだと、夏合宿の8月に毎年 約 17.6 人が該当して、毎年止まる)

項目	結果
故障	全体 6.4 件/年度(20人)。停止(重傷、または A の10人の中傷)= **1.66 件/年度**
月末レポートの材料(1か月)	レベルアップ 28.2件 / ランク文字の変化 2.8件 / 能力 +3 以上の選手 3.0人(8月: 48件 / 4.6件 / 9.1人)

→ 止まる回数は、故障 約 1.7 + 選択のあるイベント 約 7(06 §9)+ 大会・年度末の節目で、**1か月に平均 1 回未満**。月末レポートは材料が多い(約28件)ので、**優先度で7件に絞る**(R-10)。

11.6 前評判の較正(R-09。`preview`)

中堅開始・12回 × 25年度 = 1440大会。前評判の順位ごとの実績。

大会	順位相関	◎(1位)優勝 / 3位以内	○(2〜3位)優勝 / 3位以内	▲(4〜6位)優勝 / 3位以内	△(7〜10位)優勝 / 3位以内
第一戦	0.831	0.56 / 0.84	0.17 / 0.60	0.03 / 0.26	0.00 / 0.04
第二戦	0.843	0.54 / 0.85	0.18 / 0.62	0.03 / 0.25	0.00 / 0.03
第三戦	0.831	0.54 / 0.87	0.17 / 0.63	0.04 / 0.25	0.00 / 0.03

→ **印は『当たりやすさ』として実測と合っている**(◎の優勝率は約55%)。順位の相関は 0.83〜0.84。

11.7 スカウトの画面(R-08。`scout`、05 の世界シミュレーション)

中堅・弱小開始、12回 × 25年度の自校の目標 8,141人(1年度に約13人狙う方針)。

結果	割合
入学した	48.5%
枠(定員・奨学金枠)で入れなかった	**69.2%**
枠はあったが、競り負けた	4.7%

市場評価 V の帯	競合度	目標の人数	競合校数の平均	枠があったとき競り勝った割合	枠で入れなかった割合
< 51.5	ほぼなし	5,773	0.13	**1.000**	0.79
51.5〜55	少	157	0.54	1.000(枠があった17人)	0.90
55〜59	中	426	0.78	**0.922**	0.59
≥ 59	多	1,785	2.83	**0.649**	0.40

- **目標を V の高い順に並べたときの入学の割合**(8回 × 25年度 × 2開始校 = 400年度。1年度あたり目標 13.5人):

目標の中での V の順位	1	2	3	4	5	6	7	8	9	10	11〜
入学した割合	0.75	0.78	0.81	0.74	0.76	0.96	0.82	**0.62**	**0.31**	**0.07**	**0.00**

→ 1〜5位は上位級(競合・奨学金枠で 0.74〜0.81)、6〜7位は枠不要の候補で 0.82〜0.96。**8位以降は急に落ち、11位以降は1人も入れない**。入学者の内訳は、**狙った人 約 6.7 + 狙っていない(自然に入る)人 約 3.3**(中堅。弱小は 6.6 + 3.4)で、定員 10 がいつも埋まる。→ **推奨の目標は 8人**(R-08)。05 §7.1 の『年間の目標数 12〜16人』は、検算用の方針が定員を超えて狙っていた(狙っても半分は入れない)。

- **競合度の表示**: V の帯で決める(上の表)。05 §7.3 の `pwin`(提示スコアの差のロジスティック)は、**0.9〜0.99 の帯で実測 72%、0.99 以上で 95%** と較正が悪く、画面には**使わない**。

『視察して選ぶ』方針との比較(おまかせスカウトの根拠。10回 × 25年度)

開始校	方針	入学者の Q 平均	15年度以降の戦力 P	優勝率(参考。ノイズ大)
中堅	検算用(全員の Q_obs を知る・約13人)	65.09	72.16	0.539
	視察して選ぶ T=6 / 8 / 10	64.25 / 64.25 / 64.16	72.13 / 71.86 / 71.77	0.606 / 0.500 / 0.464
弱小	検算用	62.94	66.97	0.236
	視察して選ぶ T=6 / 8 / 10	62.56 / 62.90 / 63.27	67.66 / 68.24 / 69.31	0.297 / 0.230 / 0.261

→ 実際に操作できる『視察して選ぶ』方針の、検算用の方針に対する差: **入学者の Q は 中堅で −0.8〜−0.9、弱小で −0.4〜+0.3。15年度以降の戦力 P は 中堅で −0.4〜0.0、弱小で +0.7〜+2.3。下がる方向の差は 0.4 以内**なので、操作できる方針でも検算済みの長期の結果(05 §14)を下回らない。優勝率は 10回 × 11年度の標本でばらつきが大きく(±0.05 程度)、方針の優劣は読み取れない。

11.8『型に合わせた育成』の有無(L-24。`ref_model_05.py` の `SPEC_ON`)

04 §11 の『おまかせ』は、型ごとの重点プランを選手ごとに適用した前提(`SPEC_SHIFT`)だったが、グループ運用の v1 では再現できない。**外した場合の長期の結果**(各40回 × 45年度、方針は『おまかせ』):

開始校	25年度以降の優勝率(外した / 05 §14.3)	クリア年度の中央値(外した / 05 §14.3)	備考
強豪	0.586 / 0.57	8 / 7	全員クリア
中堅	0.505 / 0.51	13 / 12.5	全員クリア
弱小	0.448 / 0.46	23 / 23	45年度までに38/40がクリア
同好会	0.153 / 0.14	—	45年度までに22/40(55%)がクリア(以前は 50%)

→ **差は優勝率で ±0.016 以内、クリア年度の中央値で ±1 以内**(40回 × 45年度のノイズの範囲)。**型別の調整は長期の結果にほとんど効かない**ので、v1 は持たず、05 §14 の数値はそのまま使える。04 §14.3 の『型に合わせた育成』の項は、**チーム全体の重点プランの効果を選手ごとに仮定した検算用の値**として残し、実際の効果は小さいと注記する。なお、`SPEC_SHIFT` の持久・山の行は、適性値 P で見るとマイナスになる(§11.2)。

11.9 ランク表記(R-01。`rank`)

03 §11 の標準プラン・4年間・600人(入学時の能力は平均50)。

ランク	S	A	B	C	D	E	F	G
入学時	0.0%	0.0%	0.0%	5.6%	48.1%	42.7%	3.6%	0.0%
卒業時	0.0%	1.2%	9.8%	32.5%	47.2%	9.2%	0.1%	0.0%

卒業時の最高の能力: 中央値 73(B)、上位10% 80(A)、最大 88。→ **A は上位の選手の最高の能力、S は歴代級だけ**(01 §4.1 の『卒業時のエースは 85〜95』と整合)。

11.10 画面カタログ(`graph`)

画面 42(通常 22 / 進行 12 / 重なり 8)、コマンド 29、情報要件 74。検査して**違反 0**:

- 画面IDの重複なし / 入口・出口・戻る先がすべて定義済み / 起点(タイトル)から**すべての画面に到達できる**(進行の疑似ノードを含め 43/43)。
- **戻る操作は通常の画面で3回以内**。ダイアログの上に開けるのは確認だけ。
- **02〜07 と 10 の『画面に出す』の記述 74項目すべて**が、いずれかの画面に割り当てられている(情報要件 I-01〜I-74。I-72〜I-74 は 10 の個性チップ・相性の印・素養の判明)。**プレイヤーの操作 29個すべて**が、主たる画面を1つずつ持つ(C-xxx)。コマンドも情報も持たない画面はない(タイトルの『メニューを開く』C-META-06、クレジットの権利表示 I-71 を足して解消)。
- パッドの方向キーで、**すべての画面のすべての領域に届く**(領域の隣接が連結)。
- **仕様書とカタログの突き合わせ**(`doccheck`): 08a の各画面の節に、カタログで割り当てた情報要件(I-01〜I-74 × 画面)とコマンド(29)がすべて書かれている(記載漏れ 0)。

---

12. 計算例(テストベクタ。`verify/ref_model_08.py vectors`)

TV-1 ランク表記(R-01)

値	0	29	30	39	40	49	50	59	60	69	70	79	80	89	90	100
ランク	G	G	F	F	E	E	D	D	C	C	B	B	A	A	S	S

TV-2 素質のヒント(R-02)

能力 / 素質	50 / 70	60 / 70	66 / 70	68 / 70	70 / 70	82 / 86
比	0.714	0.857	0.943	0.971	1.000	0.953
ヒント	伸びしろあり	終盤	終盤	頭打ち間近	限界	頭打ち間近

TV-3 記録の導出(R-03。調子0・環境なし)

全能力	5000m	10000m	ハーフ
50	14:40.0	30:01.6	1:04:55.4
60	14:20.0	29:21.6	1:03:31.0
70	14:00.0	28:41.6	1:02:06.6
80	13:40.0	28:01.6	1:00:42.2
90	13:20.0	**27:21.6**	59:17.8

例: SPD62 STA66 PAC64 KCK58 CLB54 DWN52 CON60 MEN58 REC56 DUR58 → 5000m 14:15.8(8558 ds)/ 10000m 29:10.0(17500 ds)/ ハーフ 1:03:04.9(37849 ds)。

TV-4 グループの自動編成(R-04)

26人(力 = 72.0 − 0.5 × 番号。ただし11番は 67.0 で、10番と同値)→ A = 1〜10番(10人)/ B = 11〜20番(10人)/ C = 21〜26番(6人)。A と B の境目の同値は ID の小さい方が上(10番が A、11番が B)。

TV-5 推奨プラン(R-04。4月 → 3月)

重点	4	5	6	7	8	9	10	11	12	1	2	3
バランス	基礎	スピード	基礎	山	合宿	スピード	基礎	基礎	山	回復	スピード	基礎
スピード寄り	基礎	スピード	スピード	山	合宿	スピード	基礎	スピード	山	回復	スピード	基礎
ロング寄り	基礎	スピード	基礎	基礎	合宿	スピード	基礎	基礎	基礎	回復	スピード	基礎

TV-6 月末の予測(R-05)

入力: 03 TV-3 の選手(SPD58 STA60 PAC56 KCK52 CLB45 DWN44 CON55 MEN50 REC52 DUR54、素質 80/82/78/74/70/68/75/72/70/74、2年・標準、F=40、やる気3、施設Lv2、`bond_avg` =0)、基礎ビルド4週(4月)。

項目	期待値
F の推移(週1〜4)	**46 → 54 → 59 → 45**(03 TV-4 と一致)
4週の故障の確率	2.91%
安全網の週	0
レベルアップ	STA +1 のみ
4週目の調子の分布(故障なしの前提。mu = 2.500、sigma = 7.600)	絶好調 0.011 / 好調 0.224 / 普通 0.682 / 不調 0.082 / 絶不調 0.002(好調以上 0.235)

TV-7 調子の分布(R-05)

mu	sigma	絶好調	好調	普通	不調	絶不調	好調以上
0.0	8.0	0.0062	0.1524	0.6827	0.1524	0.0062	0.1587
7.672137	7.6	0.0524	0.4304	0.4976	0.0195	0.0001	0.4828
18.5	8.0	0.4256	0.4797	0.0942	0.0005	0.0000	0.9053
26.0	8.0	0.7734	0.2144	0.0122	0.0000	0.0000	0.9878

(03 TV-5 のピーキング T3・T4 の決定部分 18.5 / 26.0 に対応。)

TV-8 進行の停止(R-07。§6.3)

週の初め(出場権は3大会すべて)

週	シード外・手動	シード・計画おまかせ
0	AnnualStart, MonthPlan	AnnualStart
4	MonthPlan	—
21	Entry:Fall	Entry:Fall
22	Entry:Qualifier	—
24	MonthPlan, Entry:National	Entry:National
25	Order:Fall	Order:Fall
28	MonthPlan, Order:National	Order:National
32	MonthPlan, Entry:NewYear	Entry:NewYear
36	MonthPlan, Order:NewYear	Order:NewYear

週の終わり

状況	止まる(順)	月末レポートへ
週10(選択ありイベント1・お知らせ1・重傷C・中傷B・軽傷A登録・警告あり)	Event:ev.a, Injury:1, Warn	Notice:ev.b, Injury:2, Injury:3
週35(`event_auto`・中傷A)	Injury:5, MonthReport	Notice:ev.a
週47	MonthReport, YearEnd	Notice:team.send_off
週25(第一戦・選択イベント2・重傷2)	RaceDay:Fall, Event:ev.a, Event:ev.b	Injury:1・2(当日の画面に表示)
週9(選択イベント3・重傷1。上限3)	Event:ev.a, Event:ev.b, Event:ev.c	Injury:1(上限)

TV-9 競合度と前評判の印(R-08, R-09)

V	48.0	51.4	51.5	54.9	55.0	58.9	59.0	66.0
競合度	ほぼなし	ほぼなし	少	少	中	中	多	多

前評判の順位	1	2	3	4	6	7	10	11
印	◎	○	○	▲	▲	△	△	—

TV-10 テーマごとの月の伸び(I-10)

全能力58・素質+25・F=40・2年・標準・施設Lv2の選手が、1か月そのテーマを続けたときの**適性値 P の増加**(能力 + XP の端数)と月末の疲労。

テーマ	月末の F	Fast	Mid	Long	Climb	Descent
基礎ビルド	44	+0.40	+0.61	+0.67	+0.46	+0.29
スピード強化	38	+0.59	+0.51	+0.49	+0.22	+0.30
山対策	36	+0.16	+0.24	+0.27	+0.46	+0.53
夏合宿	54	+0.72	+0.97	+1.05	+0.90	+0.50
回復月	27	+0.14	+0.19	+0.21	+0.13	+0.09

TV-11 個別指導の検証(R-06)

入力	結果
ポイント4・メニュー(`ks.hill`, `ks.threshold`, `mil.std`)	可(今週から月末まで有効)
同(`ks.hill` が2つ)	不可(メニューが不正 = `err.guide.menu_invalid`)

入力	結果
同(故障中)	不可(故障中の選手には指定できない = `err.guide.injured`)
ポイント0・面談	不可(指導ポイントが足りない = `err.guide.points`)
ポイント2・面談(今月に済み)	不可(面談は1人につき月1回 = `err.guide.talked`)
ポイント2・面談	可(やる気 +1)
(`ks.cross_train`, `ks.rest`, `mil.low`)/(`ks.core`, `ks.rest`, 走行量なし)	不可(メニューが不正)

TV-12 月末レポートの項目の選択(R-10)

材料 15件(重傷1・ランクアップ5・成長4・お知らせ3・覚醒1・復帰1)→ 表示 7件 = 重傷 r12 / 覚醒 a1 / ランクアップ r0, r1, r2 / 成長 g3, g2(**種別ごとに最大3件、全体で7件**)、『ほか 8件』。

---

13. 課題・未決・他文書への反映

13.1 課題トラッカー(`00_spec_index.md` §6)

ID	状態	課題	対応・方針
L-23	**解決(08)**	05 §7.1 の『年間の目標数 12〜16人』が定員10(自然に入る約3人)を超え、狙っても約半分は枠で入れない(69%)	推奨を **8人**にし、画面で『確定の順番』と枠のメーターを出す(R-08)。『視察して選ぶ』方針でも長期の結果は変わらない(§11.7)。05 の検算用の方針は変えない
L-24	**解決(08)**	04 §11 の『型に合わせた育成』(`SPEC_SHIFT`)は、グループ運用の v1 では再現できない	外した場合の差は 優勝率 ±0.016 以内(§11.8)。**v1 は持たない**。04 §14.3 に注記
L-25	新規	主務(`StaffRole.Manager`)に効果がない(03・05)	v1 は表示しない。効果を足すか、役職をなくすかを決める
L-26	新規	チーム全体を持久・山に振ると標準プランより劣る(素質の伸びしろが能力ごとにほぼ同じ。§11.2)	03・05 の設計の結果。v1 は3つの重点に絞った。将来、型ごとに伸びしろに偏りを持たせると『型に合わせた育成』が意味を持つ(03 / 05 の拡張案)
L-27	新規	`RaceKind.Meet`(記録会)は v1 で使わない(記録は導出。R-03)	記録会を実際の測定にするなら、03 に測定の週を足す
L-28	**解決(08)**	03 §6 手順6 と 06 §6 の `form_add` の上限の読み方が曖昧	**peaking に form_add を足してから −8〜+12 に収める**と確定。03・06 の文言を直す
L-29	新規	05 §7.3 の `pwin` は較正が悪い(0.9〜0.99 で実測 72%)	画面には使わない(競合度と枠で見せる)。05 の検算用の方針は影響なし
L-30	新規	05 §8.2 の『V の高い順の確定』のため、V < 51.5 の目標(原石)の 78% が定員で入れない	案: 練習会まで進めた候補を先に確定する『本命の確約』。**既定: 変更しない**(検算済みの値を保つ)。画面の『確定の順番』で緩和
L-31	新規	実機での文字・タップ領域・パッド操作・日本語フォント・性能は未検証	M1 の UI Toolkit の技術検証に含める(01 §13)
L-32	新規	画面の文言・アイコン・演出は未作成(名称テーブルのキーだけ)	実装時に作成。M1 はプレースホルダー

13.2 未決の解決(`00_spec_index.md` §5)

ID	内容	結論
U-04	言語	既定のまま(v1 は日本語のみ。文字列はテーブル化済み。かな表示の設定あり)
U-15(新)	縦持ちへの対応	既定: 将来(v1 は横画面のみ)
U-16(新)	セーブスロットの数	**確定(09 §3・§4)**: 3 スロット。各スロットに自動2世代(A / B)+ 手動1

13.3 反映した文書

文書	変更
01	§3.4 に年度末(120 秒)を追加(合計 32 分)/ §5.1 の年間方針の重点を バランス·スピード寄り·ロング寄り に / §6.1 スカウトの時期(視察 6〜11月·勧誘 12〜2月)·年内の繰り越し·推奨の目標 8人 / §11 の画面一覧を 08 の42画面に置き換え
02	v0.9: §10 の変更(`manager_name` / `individual_menu` / `talked` / `Focus` / `plan_auto` / `scout_auto` / `Runner.pb` の派生 / `DeviceSettings` (新 §3.12)/ `SaveMeta`)
03	§5.7『個別指導』を追加(R-06)/ §6 手順6 の `form_add` の読み方を確定(L-28)/ §11 に重点ごとの適性値の差を追記
10	個性·素養の画面: R-17(チップ·相性の印·素養の判明·山対策への案内)/ `RunnerRow` の名前の列の2行目 / R-10 に `gift_reveal` / 情報要件 I-72〜I-74 / SC-11·14·16·18·19·20·21·22·29 の表示
04	§14.3 に『型に合わせた育成』の注記(L-24)/ §9.1 に『指示は区間の前に決める画面』の注記
05	§7.1 に時期·繰り越し·推奨の目標数(L-23)/ §14 に『型に合わせた育成の有無』『視察して選ぶ方針』の検算を追記 / §8.2 に L-30 の注記
06	§17 の反映と引き継ぎ(イベントの表示·効果の要約 R-15·止まる規則)
07	§14 の反映と引き継ぎ(宣言の画面は SC-31、追加コンテンツは SC-05)
00	08 を v0.1(`08_screen_spec.md` + `08a_screen_catalog.md` 、 `ref_model_08.py` / `screens_08.py`)/ U-04 解決、U-15·U-16 / L-23〜L-32 / 用語集

13.4 次の文書(09 セーブ·技術仕様)への引き継ぎ(09 で完了。確定した場所を右列に付けた)

内容	要点
セーブ	スロット 3 + 自動2世代 + 手動1 / 中断セーブ(レースの『次の区間の指示の前』を含む)/ `SaveMeta` / 読み込みの失敗の扱い(1つ前の世代へ)/ 容量(100年度後も数MB)。→ **09 §3·§4·§5**(100 年度で約 285 KB。中断は `suspend.eksv` ·再計算で再開)
設定の保存	`DeviceSettings` (同期しない)/ `ProfileData` (Steam Cloud など)/ セーブごとの設定。→ **09 §3**(`device.json` · `profile.*.eksv` · `LocalState` 。クラウドの対象は 09 §3.1)
乱数	レースの乱数は大会·チーム·区間をキーに決まる(ロードしても同じ結果)/ 世界生成のシード(新規ゲームの『引き直し』)。→ **09 §8**(登録表·端末差の実測·引き直し)
文字列テーブル	キーの規則 / 日本語先行 / かな / 数値·時間の整形(§7.4)。→ **09 §12.3·§13**
UI の技術	UI Toolkit の検証(日本語·モバイル性能·パッド)/ 基準キャンバス 360 u の拡大 / 世界生成の所要時間(新規ゲームの進捗表示)。→ **09 §14·§15.5**(合格の基準。世界生成は 3 秒以内·進捗表示)
実績·購入	ストアへの通知の保留(U-14)/ 購入状態の確認の時点(07 §8.1)。→ **09 §10·§11**

13.6 M1 準備: 操作と進行制御の入口(`GameSession` の操作· `GameFlow`)

画面が呼ぶ入口を、画面なしの C#(`m0/Assets/Scripts/Domain/Game/`)として作った。**状態を書き換える前に規則で検証し、無効なら理由(F4)を返す**(`OpResult` : `Ok` / `NeedsConfirm` (確認ずみで再度呼ぶ。§7.6)/ `Error` · `Code` (理由のキー。`OpErr` の `err.*`)/ `Warnings` (`warn.*`)。**理由と警告は日本語の本文ではなくキーと差し込みの値(`Msg`)で、App の `OpText` が文字列テーブルから文にする**。L-63 ③)。

画面	操作(`GameSession`)	規則
SC-10 年間方針	`SetAnnualPlan(focus, targetRace, peaking[5])` / `ApplyRecommendedPlan()`	R-04。`plan_auto` なら今月の推奨プランを引き直す
SC-18 部員一覧(グループ編成)	`ApplyRecommendedGroups()` (4 月だけ)/ `MoveToGroup(runner, g)`	R-04。**グループは保存される編成**(毎週は決め直さない。新規ゲームと、おまかせの年度初めに推奨の編成を適用。新入部員は C)
SC-16 月間計画	`SetMonthlyPlan(g, theme)` / `SetCustomWeeks(g, 4 週)`	03 §5.5 の検証
SC-20 個別指導	`GuideMenu(runner, menu, confirmed)` / `CancelGuide` / `Talk(runner)`	R-06。調整メニューの上書き·安全網の対象外は確認を求める。今週の指定の取り消しはポイントが戻る
SC-21 エントリー	`SetEntries(kind, ids, confirmed)` / `AutoEntries(kind)`	04 §4。締切(大会の 4 週前)から大会の週まで。出走人数に足りないと確認(棄権)
SC-31 特別招聘の宣言	`DeclareTicket(kind, pref)`	07 §3.3。理由: 宣言済み / 今年度は済み / 券なし / 在籍が上限 / 期間外

画面	操作(GameSession)	規則
SC-22 区間オーダー	`SuggestOrder(kind)` / `SetOrder(kind, 区間 → 選手)` / `ClearOrder(kind)` / `SectionFit(kind, 選手, 区間)`	04 §5.2 O1〜O3。エントリーの締切から大会の週の初めまで。出さなければおまかせ
SC-23 他校情報・予報	`Forecast(kind)` / `RacePreview(kind)`	04 §6(予報の誤差)・R-09(前評判)
SC-24 当日	`OpenRaceDay(kind)` → `RaceDay`: `View`(区間)(適性・余裕の目安・期待時間)/ `Swap`(区間, 控え) / `ResetSwaps` / `SetInstruction` / `ApplyGauge`(目安どおり)/ `UseRecommended`(おまかせ)/ `Preview()`	04 §5.3・§9.1。初期値はおまかせ。確定の前なら何度でも
SC-25 レース進行	`RaceDay.Start` → 区間ごとに `SetInstruction` → `NextIncident` → `UseGeki` → `Advance`(中継所の順位表)/ `Complete`	04 §9.1・§9.2。通過した区間は変えられない
予選会の登録	`SetQualifierEntries(12 名)`	04 §3.2。出さなければおまかせ
SC-32 強化	`UpgradeFacility(key)` / `UpgradeRecommended()`	R-11。年度末の精算(`SettleYear`)のあと、入学の割り当て(`TurnoverYear`)の前
SC-27〜SC-30 スカウト	`ScoutObserve` / `ScoutTalk` / `ScoutInvite`(`ViewCandidate` で見え方)/ `AutoScoutObserve` / `AutoScoutRecruit`	R-08・05 §7。時期・ポイント・順序・競合度・情報の段階

進行制御(`GameFlow`):『続ける』(`Advance`)が唯一の前進で、止まる場所は §6.3 の決定表(`FlowRules`)のとおり。`Stop` の種類は AnnualStart / MonthPlan / Entry / QualifierEntry / Order / RaceDay / Event / Injury / Warn / MonthReport / YearEndSettle / YearEndTurnover。イベントは『発生の判定 → 停止で選択(`ChooseEvent`)→ 選択を反映して効果』の二段(選択は乱数を引かない。06 §7.5)。選ばずに進むと既定の選択肢。**何も操作しない流れは、おまかせ(`StepWeek`)とバイト一致する**(強化と招聘の宣言は画面の操作なので除く)。保存できるのは週の解決の前・年度末の精算のあと・入学者発表のあと(`CanSave`)。復元して作り直した `GameFlow` は、その週の最初の停止から再開する。

観戦モード(`settings.spectate_mode`): `Full` は大会の前に `RaceDayPrep` の停止(`Stop.RaceDay` に当日)が出る。画面は当日変更・指示・檄を決め、レース進行は `RaceDay` を直接進める。『続ける』で残りは決定のまま走る(未決の檄は自動)。`Digest` / `ResultOnly` は止まらず、提出したオーダーがあればそれを基に、当日変更・指示・檄は自動(04 §9.3)。何も操作しない流れは、`Full` でもおまかせとバイト一致する。

App 層の入口(`m0/Assets/Scripts/App/`。09 §17.7): 画面は `GameHost` を持ち、`Advance()` で進め、`Session` の操作と Views で見せる。自動セーブ(§6.5 の表: 月境界・オーダー確定後・年度末の各節)・手動セーブ・アプリの中断のセーブ・読み込み(回復の順・中断セーブの再開)・保存の失敗の通知は `GameHost` / `SaveService` が受け持つ。画面が呼ぶのは `OnAppPause()` / `OnAppResume()` / `Tick()` / `SaveManual()` と、スロットの見出し `SlotCatalog.List()` だけ。

読み取り API(Views。`m0/Assets/Scripts/Domain/Views/` + `GameSession.Views*.cs`): 画面は Domain の状態を直接読まず、規則(R-01〜R-17)で組み立てた表示用の型を受け取る。状態を変えず、語は文字列テーブルのキーで返し、隠す情報(素質・体質・成長タイプ・未判明の素養)は型に入れない。実装済み: `Home` / `RosterRows` / `RunnerRow` / `RunnerDetail` / `Plan` / `Entry` / `Order` / `OrderCandidates` / `RaceResult` / `MonthReportView` / `YearSummary` / `Graduation` / `Enrollment` / `Ops` / `WorldOverview` / `Obs` / `History` / `ScoutCandidates` / `ScoutCandidate` / `ScoutMap` / `ScoutTargets` / `AnnualPlanView` / `Preview` / `EventDialog` / `ConditionWarning` / `InjuryDialog` / `RaceDayView` / `RaceLive`。画面ごとの対応(○ △ ×)と足りないもの(G-01〜G-17)は `08b_screen_data.md`。

未実装(M1 の続き): 観戦モードの演出(中継所ごとの要約。見どころの判定は実装済み = `RaceHighlights`・`RaceLive().Highlights`・`RaceResult().Highlights`)/ 檄の先行使用と走者ごとの効き `f_k`(04 §9.2。L-60)/ 新規ゲームの画面(R-13: 名前・難易度の選択)/ 予選会の当日(SC-26 のみ。当日変更・指示なし)。月末レポートの `rank_up` `form_up` `big_growth` `gift_reveal` は R-10 どおりに実装した(L-57・L-64 解決)。

13.5 M0 の C# 実装(段階 9)

- 場所: `m0/Assets/Scripts/Domain/Screens/`(`DisplayRules.cs` R-01・R-02・R-03・R-08 の競合度・R-09 の印、`Forecaster.cs` R-05 の予測と調子の分布・テーマごとの月の伸び、`PlanRules.cs` R-04 の推奨プラン・グループ編成と R-06 の個別指導の検証、`FlowRules.cs` R-07 の停止と R-10 の月末レポート、`Presentation.cs` R-16 の性格タグと R-17 の個性チップ・相性の印)。

- 画面は結果を表示するだけ(原則 10)。表示の文言(個性チップの語・R-15 の効果の要約・理由の文)は名称テーブルのキーで持つ(09 §12.3)ので、C# は**キー**を返す(個性チップ = `form.speed` など)。文言の組み立て(R-15)は UI(M1)で行う。
- 参照モデルとの差分テスト(`m0/tests/Domain.Tests/ScreenGoldenTests.cs`): 上の各規則の乱数ケースが完全一致。予測(`Forecaster.ExpectedWeek`)は、実際の週の解決(`WeekResolver.Resolve`)と、補正つきで 1,484 件一致(故障が起きなかった週)。

第8部 画面仕様 — 画面カタログ

原典: `08a_screen_catalog.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> `08_screen_spec.md` の付属。**画面ごとの目的・領域・表示・操作・状態**を、SC 番号の順に書く。規則・流れ・検算は本体(08)、数値は 02〜07。書式: 各画面は『目的 / 領域 / 表示 / 操作 / 状態・例外』。領域の名前は `verify/screens_08.py` の `grid` と同じで、**十字キーの移動はこの並び**(上から下、同じ行は左から右)。『操作バー』はフッタ(56 u)で、主たる操作は右端(強調)。表示の `I-xx`、操作の `C-xxx` は 08 の情報要件・コマンドの ID(`ref_model_08.py tables`)。共通: すべての画面のヘッダに、戻る(ある場合)・画面名・要約(日付や資金)・ポーズ(SC-40)・ヘルプ(SC-39、?)を置く。長押し / X / F1 でその項目の説明を出す(08 §3)。

A. 起動・設定

SC-01 タイトル(`scr.title`)

項目	内容
目的	ゲームの入口
領域	`menu` (縦のメニュー)
表示	題名・版 /『続きから』(直近のセーブの校名・年度・日時。なければ非表示)/『はじめから』/『セーブの選択』/『実績』/『追加コンテンツ』/『設定』/『クレジット』
操作	C-META-06: 項目を選んで画面を開く。『続きから』は直近のセーブを読み込んでホームへ
状態・例外	セーブがないとき『続きから』『セーブの選択』は無効(理由を出す)。起動時に購入状態の確認を裏で始める(結果を待たない。07 §8.1)

SC-02 新規ゲーム(`scr.newgame`)

項目	内容
目的	開始校・見た目・監督名を決めて始める(R-13)
領域	`steps` (①開始校 ②自校 ③確認の進み)/ `body` / `actions`
表示	① 開始校のカード: 強豪 / 中堅(既定)/ 弱小 / 同好会 + 所持しているシナリオ(古豪復活など)。各カードに 戦力順位・施設・資金・部員数・クリアまでの目安(約3.5 / 6.5 / 11.5 / 18時間)・説明(I-65)/ ② 世界の生成結果(自校の名前・所在地・校色)、見た目(校章・ユニフォーム柄・応援歌・BGM。所持していない有料の見た目は鍵付きで、タップで SC-05)、監督名(8文字まで)(I-66)/ ③ 設定の確認と保存先のスロット
操作	C-META-01: ①カードを選ぶ ②『別の世界にする』(再生成)・見た目・監督名を決める ③『開始』。『戻る』で1つ前のステップへ。確認のダイアログはなし(開始前に変更できるため)。スロットに既存のセーブがあるときは上書きの確認(取り消せない)
状態・例外	同好会:『部員5人。最初の年度は第一戦に出られません』を表示。未所持のシナリオは選べない(購入への案内)。世界の生成が1秒を超えるときは進捗の表示。生成に失敗したら『もう一度』

SC-03 セーブの選択(`scr.load`)

項目	内容
目的	セーブの読み込み・削除
領域	`slots` (3スロット。縦)/ `actions`
表示	スロットごと: 校名・開始校 / シナリオ・第N年度の月・直近の成績・プレイ時間・保存日時・クラウド同期の状態(I-64)。自動セーブの世代(最新 / 1つ前)を展開して選べる
操作	C-META-02:『読み込む』(主)/『削除』(補助。取り消せない確認)
状態・例外	空のスロットは『空』。最新の世代が壊れていたときは**自動で1つ前から読み込み**、『ひとつ前から再開しました』を知らせる。すべて読み込めないときだけ『読み込めません』(ファイルは消さない。手動セーブ・退避を案内。09 §4.5)。**同期の競合**は SC-37 の形式で『この端末を使う / クラウドを使う / あとで決める』(既定は『あとで決める』。09 §9.4)。『退避』から『別の歴史として復元』。ポーズから開いたときは『読み込む』の前に『現在の進行は失われます』の確認

SC-04 設定（`scr.settings`）

項目	内容
目的	設定の変更（08 §9）
領域	`tabs`（ゲーム / 表示 / 音 / 操作 / データ・購入）/ `list`
表示	項目ごと：名前・現在の値・説明（結果に影響するか。観戦モードは影響する）。ゲームのタブに**自校の見た目**（校章・ユニフォーム柄・応援歌・BGM。所持している範囲。I-66）。データ・購入のタブに**セーブの日時とクラウド同期の状態**（I-64）。購入状態が不明なときは『購入状態を確認できません』（I-57）
操作	C-META-03：項目を選んで値を変える（即時に反映。確認は不要）。データ・購入のタブから『購入を復元』『追加コンテンツ』『クレジット』へ
状態・例外	セーブを開いていないとき（タイトルから）、ゲームのタブの項目は無効（DeviceSettings だけ変えられる）

SC-05 追加コンテンツ（`scr.store`）

項目	内容
目的	購入・復元・購入状態の確認（07 §9）
領域	`list`（商品）/ `detail` / `actions`
表示	商品：黄金世代プラン / 見た目パック（ユニフォーム・校章 / 髪型・髪色 / 応援歌・BGM。内容のプレビュー付き。I-66）/ シナリオ（古豪復活ほか）。各商品に価格（ストアが返す表示）・内容・所持の状態（未所持 / 所持 / 確認中。確認できないときは『購入状態を確認できません』。I-57）。**説明文に『券を使わなくても、クリアまで遊べます』『確率で結果が変わることはありません』を必ず書く**（07 §8.5）。黄金世代プランは『4年度に2枚、失効しません。購入した時点で最初の2枚』を明記
操作	C-META-04：『購入』（ストアの購入画面へ）/『購入を復元』/『購入状態を確認』
状態・例外	購入状態が不明なら『確認できません』（購入済みでも付与は待つ。07 §8.2）。ストアに接続できないときは購入ボタンを無効にして理由を出す。**購入の完了時に確認を行い、結果をトーストで知らせる**

SC-06 実績（`scr.achv`）

項目	内容
目的	実績の一覧と進捗（I-58）
領域	`tabs`（大会 / クリア / 育成 / 継続）/ `list`
表示	実績ごと：名前・条件・解除日（解除済み）。隠し実績は『???』（解除するまで内容を隠す。02 §3.10）。進捗のある実績（区間賞100 など）は現在の値 / 目標。『無招聘』の実績には『特別招聘券を一度も使っていない』と条件を明記（01 §3.3）
操作	閲覧のみ（スクロール・タブの切り替え・長押しで条件の全文）
状態・例外	ストアに未接続でも本体で判定した状態を出す（通知は後で送る。U-14）

SC-07 クレジット・権利表示（`scr.credits`）

項目	内容
目的	クレジット・素材とライセンスの表示
領域	`body`
表示	制作・素材・ライセンス。**『大学名・選手名・大会名はすべて架空です』**（01 §0）。大会名の実名化・通称の使用は権利確認が済むまで行わない（01 §14.1）（I-71）
操作	スクロールのみ
状態・例外	—

B. ホームと進行

SC-08 ホーム（`scr.home`）

項目	内容
目的	状況の確認と、各画面への入口（ハブ）。『続ける』で進行を再開する
領域	`status`（要約）/ `next`（次の予定）/ `menu`（入口のタイル）/ `actions`
表示	**status**: 校名・第N年度 月 週・監督ランクと次のランクまでの累計評価ポイント・名声・資金・士気・戦力順位・クリアの状態（I-48, I-49, I-60）。**next**: 次の節目（『第一戦 あと 4週（10月2週）。エントリー締切 あと 0週』など。I-61）・期待目標（『第三戦 6位以内で予算ボーナス 200万円』。I-15）・バッジ（特別招聘『宣言できます』・購入状態不明・未読の通知。I-24, I-57）。**menu**: 月間計画 / 部員 / スカウト / 運営 / 歴史・殿堂 / 全国の大学 / 年間予定 / OB名簿 /（大会の2〜4週前は『エントリー』『オーダー』）/（宣言できるときは『特別招聘』）
操作	C-FLOW-01: 『続ける』（主。文言は次の節目の名前）。タイルを選んで各画面へ。ベルで通知（SC-38）。メニューでポーズ（SC-40）
状態・例外	**期限が迫るもの**を next に出す: エントリー締切の週（登録しないと棄権）、特別招聘の宣言期間の最後の月（2月）、スカウトの勧誘期の始まり（12月）。年間方針が未設定の年度は『年間方針を決める』を主たる操作にする

SC-09 年間予定（`scr.calendar`）

項目	内容
目的	1年度の予定の一覧（I-61）
領域	`months`（4月〜3月。半年 × 2段）/ `detail`
表示	月ごと: 大会（第一戦 10月2週 / 予選会 10月3週 / 第二戦 11月1週 / 第三戦 1月1週）・エントリー締切（大会の4週前）・夏合宿（8月）・記録会（5・6月の第3週）・スカウトの時期（視察 6〜11月 / 勧誘 12〜2月）・特別招聘の宣言期間（6〜2月）・年度末（卒業・精算・入学）。選んだ月の詳細に、その月の月間計画のテーマと、ピーキング（調整）の期間
操作	月を選んで詳細を見る / 大会の月から『エントリー』『オーダー』へ（出場権があり、期間内のとき）/ 年間方針へ（C-PLAN-01 の副の入口）
状態・例外	出場権のない大会は『出場なし（予選会 / 前年度の順位）』と理由を出す（I-60）

SC-10 年間方針（`scr.annual`）

項目	内容
目的	年度の重点・目標大会・ピーキングを決める（R-04）
領域	`focus`（重点）/ `target`（目標大会）/ `peaking`（大会ごとの調整）/ `actions`
表示	**重点**（バランス / スピード寄り / ロング寄り）。選んだ重点の説明と、**標準プランとの適性値 P の差**（区間タイプ別。08 §8 R-04 の表。I-14）。**目標大会**（第一戦 / 第二戦 / 第三戦。既定は第三戦。『見出しと予測の注目大会に使います』）。期待目標（I-15）。**ピーキング**: 大会ごとに 標準 / 短め / なし（03 §5.3。既定: 3大会は標準、予選会は短め）
操作	C-PLAN-01: 重点・目標大会・ピーキングを選んで『決定』。変更しなければ前年度のまま『決定』で進む
状態・例外	年度初めには必ず止まる（§6.1）。年度の途中でもホーム・年間予定から開いて変えられる（次の月間計画から反映）

SC-11 月末レポート（`scr.report.month`）

項目	内容
目的	1か月の成長とコンディションの確認（I-54）
領域	`summary` / `highlights` / `actions`
表示	**summary**: 月の名前・チームの平均の疲労・調子・士気とその変化・今月のテーマ。**highlights**: 注目の項目を**最大7件**（08 §8 R-10: 重傷 > 中傷 > 覚醒 > 素養の判明 > ランクアップ > 大きな成長 > 復帰 > お知らせ > 調子の上昇）。各項目は選手の名前・内容・数値の変化。『ほか n件』（長押し・選択で一覧）。覚醒は金・青・赤の枠で強調（I-53）。**素養の判明**（『◯◯に登坂の素養があることが分かった』。I-74。選手詳細へ移動できる）。大会のあった月は大会のまとめ。**次の月への注意**（警告・来月の予定）
操作	項目を選んで選手詳細へ（副）/『ホームへ』（補助）/『続ける』（主。次の節目へ）
状態・例外	『動きを減らす』では、成長の強調・カットインを静止画にする。レポートに出ないほかの変化は、部員一覧・選手詳細で見られる

SC-12 年度の結果と精算（`scr.year.summary`）

項目	内容
目的	年度の成績の確認と資金の精算（05 §12 手順1〜3。I-47, I-55）
領域	`results`（成績）/ `funds`（収支）/ `actions`
表示	**results**: 3大会の順位・評価ポイント・名声ポイント・年間の名声の変化・三冠・期待目標の達成（予算ボーナス）・監督ランクの上昇と解放（05 §3.3）・**特別招聘券の獲得**（節目。07 §4）。**funds**: 収入（800 + 名声 + 0.0012 × 名声² + 25 × 評価ポイント + 補正）と支出（スタッフ給与・施設維持・15 × 部員数）の内訳・年度末の資金（I-47）
操作	『強化へ』（補助。SC-32 を『強化だけ』の形で開く）/『続ける』（主。卒業・進路へ）
状態・例外	強化をしないで『続ける』を押すと、推奨の強化が残っていることを1行で知らせる（確認はしない。R-11）

SC-13 卒業・進路（`scr.year.grad`）

項目	内容
目的	4年生の卒業を見送る（05 §10.2。I-55）
領域	`list`（卒業生）/ `detail` / `actions`
表示	卒業生ごと: 名前・出身（県・高校）・最終の能力（ランク付き。I-01）・特殊能力・主な成績（区間賞・出場数）・**進路**（実業団 / 教員 / 一般就職 / 院進学）・殿堂入りの印。殿堂入りした選手を先頭に出す。招聘で入った選手には印
操作	選手を選んで詳細（副）/『続ける』（主。入学者発表へ）
状態・例外	卒業生がいない年度はこの画面を飛ばす。やる気1で退部した選手は『退部』として別枠に出す（05 §10.4）

SC-14 入学者発表・入部確認（`scr.enroll`）

項目	内容
目的	**3月**: 入学が確定した新入生の発表（07 §3.4）/ **4月**: 入部の確認（I-56）
領域	`list` / `detail` / `actions`
表示	新入生ごと: 名前・出身（県・高校。I-07）・距離帯・評価★・**能力（Basic と Stats。素質と体質は隠す。I-01）**・**個性チップ**（I-72）・**素養**（スカウトで見えていたものだけ。ほかは入部から12週後に判明。I-74）・成長タイプは入部後に表示・特殊能力・**特別招聘の印**（留学生 / スーパールーキー）。狙った（声かけ・練習会をした）候補と、自然に入った候補を分けて表示。狙ったが入れなかった候補（定員・奨学金枠・競り負け）と理由を『惜しかった候補』として下に出す（I-23）
操作	選手を選んで詳細（副）/『続ける』（主）。4月の確認では、**『推奨のグループ編成を適用』**（C-PLAN-02。新入部員は C）を併せて出す
状態・例外	招聘がなかった年度は、招聘の印なし。部員が少ない年度（同好会）は、4月に『第一戦まで あと n人必要』を知らせる（04 §4.1 E1）

SC-15 クリア（`scr.clear`）

項目	内容
目的	三大駅伝制覇の演出と、続けるかの選択（01 §3.3。I-59）
領域	`body` / `actions`
表示	エンディングの演出（静止画 + 文）・クリアした年度・開始校・各大会の初優勝の年度・**『無招聘』かどうか**（その場で解除された実績を一覧）
操作	C-CLEAR-01:『続ける』（主。同じセーブで継続）/『歴史を見る』（補助。SC-34）
状態・例外	クリアは1つのセーブで1回だけ（`run.cleared`。02 §4.9）。『動きを減らす』は静止画のみ

C. 育成・部員

SC-16 月間計画（`scr.plan`）

項目	内容
目的	グループごとに月のテーマを決め、結果の予測を見る（R-04, R-05）。**月に約40秒**（08 §11.3）

項目	内容
領域	`groups` (A / B / C の3行)/ `forecast` (予測)・`detail` (選んだテーマの説明)/ `actions`
表示	**groups**: 各行に グループ名・人数・故障者数・平均の疲労(I-03, I-11)・**テーマの選択**(基礎ビルド / スピード強化 / 山対策 / 夏合宿 / 回復月 / カスタム)。推奨プランのテーマに『推奨』の印。選んだテーマの**相性**(『相性 ◎ 4人 / ○ 3人 / △ 1人』。グループの選手の個性と、テーマの練習の配分から。R-17。I-73)。選択中のテーマの 4週のメニュー(ポイント練習2つ + 走行量)。**forecast**(選んだグループ。R-05): ①月末の疲労(平均・最大)と週ごとの推移 ②要注意の人数 ③故障の見込み(『月に約0.3件』)④安全網が働く週の数 ⑤**伸びの見込み**(区間タイプ別の適性値 P の増減。各テーマを並べて比較できる = TV-10。I-10)⑥次の大会の調子の見込み(『第一戦 好調以上 82%』。8週以内にあるとき)(I-09, I-13)。**指導ポイント**の残り(I-12)。夏合宿の月は『故障率 ×1.3・経験値 ×1.5』を表示
操作	C-PLAN-03: テーマを選ぶ / 『推奨を適用』(全グループを推奨プランに)/ 『確定して進める』(主)。『カスタム』を選ぶと SC-17 へ。補助: 『個別指導』(SC-20)・『部員一覧』(SC-18)
状態・例外	推奨プランを変えずに『確定』でき、**既定のフォーカスは『確定して進める』**にはしない(グループの行から始める。誤って流さないため。ただし `plan_auto` ではこの画面を出さない)。故障者はグループの人数と別に数え、予測の平均から除く。大会週を含む月は、登録メンバーの調整(ピーキング)が自動で入ることを予測に反映して表示。個別指定のある選手は『個別』の印

SC-17 カスタム週の編集(`scr.plan.custom`)

項目	内容
目的	`th.custom` (週ごとに自由に決める。上級者向け)の編集(03 §4.3, §5.5)
領域	`weeks` (4週の列)/ `menu` (ポイント練習2・走行量の選択)/ `actions`
表示	週ごとの (ポイント練習1, ポイント練習2, 走行量)・負荷の合計 L・その週の予測の疲労(R-05)。選べるメニューは 03 §4.1〜4.2 の一覧(ポイント練習12種 + 走行量4種。`ks.cross_train` は出さない)
操作	C-PLAN-04: 週を選んでメニューを選ぶ。検証(2枠は異なる / 走行量は必須)に反するものは確定できず、理由を出す。『推奨から始める』(標準のテーマの週メニューをコピー)・『確定』(主)
状態・例外	検証に失敗した指定は無視される(03 §5.5 の5)ので、画面で必ず事前に弾く

SC-18 部員一覧(`scr.roster`)

項目	内容
目的	部員の状態の把握・グループ編成・世代の確認
領域	`tabs` (一覧 / グループ / 世代)/ `list`・`side` (選んだ選手の要約)/ `actions`
表示	**一覧**: `RunnerRow` (08 §7.1。力とランク I-01・疲労・調子・やる気・故障 I-03・特殊能力の数 I-05・出身 I-07。名前の列の2行目に**個性チップ**。I-72)。選んだ選手の要約(`side`)に記録(5000m / 10000m。I-02)。並べ替え(力 / 学年 / 疲労 / 調子 / 出身)と絞り込み(グループ / 学年 / 故障 / 特殊能力あり)。**グループ**: A / B / C の3列に分けて表示し(人数・平均の疲労・故障者数。I-11。選手にはチップ)、選手を選んで移す(C-PLAN-02)。素養(登坂・下り)が判明した選手には『山対策のグループ(C)で伸ばせます』の案内を付ける(R-17)。『推奨の編成を適用』(R-04)。グループを変えると絆が下がる注記。**世代**(I-08): 学年別の戦力グラフ(学年ごとの力 P の平均と上位の人数)。『谷間の世代』を強調。上のヘッダに士気(I-49)
操作	行を選んで選手詳細(SC-19)/ グループの移動(C-PLAN-02。一括変更は通常の確認)/ 『推奨の編成を適用』/ 並べ替え・絞り込み
状態・例外	故障中の選手は暗く表示。同郷・同じ高校の印(●・◆)を選択中の選手に合わせて付ける(I-07)。部員が0人の絞り込みでは『該当する選手がいません』

SC-19 選手詳細(`scr.runner`)

項目	内容
目的	1人の全情報の確認と、個別の操作
領域	`header` (顔・名前・学年・グループ・主将の印・出身)/ `stats` (能力)/ `side` (状態・関係・経歴)/ `actions`
表示	**header**: 顔・姓名とかな・学年・グループ・出身県(留学生は国)と出身高校・性格タグ(R-16)・成長タイプ(R-02)(I-07)・**個性チップ**(型・走法・展開。長押しで説明。I-72)・**素養**(判明したものだけ。『登坂の素養』。I-74)。**stats**: 10能力(『SPD 62 C』+ 前月の増減 + 素質のヒント)・区間適性5つ(『適性◎○△』)・総合力 P・**記録**(5000m / 10000m / ハーフ。R-03。I-02)(I-01, I-04)。**side**: 疲労・調子・やる気・故障(残り週数)(I-03)/ 特殊能力(名前・金青赤・説明・取得の経緯)(I-05)/ **絆**(同グループの仲間・同郷・同じ高校の上位。I-06)/ 経歴(出場数・区間賞・ブレーキの回数・出場した大会の成績)/ **この月の予測**(R-05。個別指定があるときは指定を反映)と、**このテーマとの相性**(◎○△。I-73。R-17)/ 招聘の印(留学生 / スーパールーキー)。素養が判明した選手には『Cへ移す』(C-PLAN-02。登坂・下りの素養)
操作	『個別指導』(SC-20。C-GUIDE-01・02)/ 見た目の変更(髪型・髪色。所持している範囲。C-LOOK-01。確定前に変更できる)/ 前後の選手へ(LB / RB)

項目	内容
状態・例外	OB・候補・入学者を開くときは同じ画面を『閲覧のみ』で使う(操作を隠す)。OB は最終の能力・進路・OBとしての役割(I-50)。**他校の選手は開かない**(他校は SC-23 の要約だけ)

SC-20 個別指導(`scr.guidance`)

項目	内容
目的	1人に、メニューの指定か面談をする(R-06)
領域	`target` (選手の要約)/ `options` (メニューの指定 / 面談)/ `actions`
表示	対象の選手(力・疲労・調子・やる気・故障。単独走◎は『個別指導の週は効率が高い』)・指導ポイントの残り(『残り 3 / 4』。I-12)・**面談**: やる気 +1(今月に済みなら無効)・**メニューの指定**: プリセット(ケア / 調整)と自由編集(ポイント練習2つ + 走行量)。選んだメニューの**予測**(R-05: 疲労の推移・伸び)と**相性の印**(I-73。R-17)。**注意の表示**: 調整(ピーキング)期間の登録メンバー『調整メニューを上書きします』/ 疲労85以上『安全網が働きません』/ 故障中『指定できません』
操作	C-GUIDE-01: 『指定する』(1ポイント。今週から月末まで)・指定の取り消し(最初の週の解決前ならポイントが戻る)/ C-GUIDE-02: 『面談する』(1ポイント)。確定前は何度でも変えられる(確認なし)
状態・例外	ポイントが足りないときは操作を無効表示にして『指導ポイントが足りません』。コンディション警告(SC-41)・故障の知らせ(SC-42)から、対象の選手を指定して開く

D. 大会

SC-21 エントリー(`scr.entry`)

項目	内容
目的	大会の登録メンバーを決める(04 §4)。予選会は12名(04 §3.2)
領域	`summary` (登録枠・出走枠・締切)/ `roster` (在籍選手の一覧)・`entered` (登録メンバー)/ `actions`
表示	**summary**: 『登録 11 / 16』『出走に必要 10』・締切(大会の4週前)・留学生の枠(登録2・出走1)。**roster**: `RunnerRow` (名前の列に個性チップ。I-72)+ 各選手の**区間適性(◎〇△の最高のタイプ)**・調子の見込み(R-05: 登録すると大会の3週前から調整)・疲労・故障(重傷は登録不可の理由を表示)(I-03)。**entered**: 登録済みの選手(登録順ではなく力順)。検証 E1〜E4 の結果(I-28)
操作	C-RACE-01: 選手を登録 / 外す・『おまかせ登録』(初期値。04 §4.2)・『確定』(主)。C-RACE-08: 予選会(12名)。C-RACE-02: **差し替え**(大会の3週前〜前週。故障した登録メンバーを外して別の選手を入れる)。確定するとピーキングの対象
状態・例外	出走人数に満たない登録は確定できない(『出場できません』。04 §4.1 E1)。重傷は登録できない(E3)。留学生が登録上限(2)を超えるときは、外す選手を選ばせる。**締切を過ぎる前の週の初めに止まる**ので、登録せずに進めない(§6.3)

SC-22 区間オーダー(`scr.order`)

項目	内容
目的	区間ごとの配置を決める(04 §5)
領域	`sections` (区間のカードの帯)/ `candidates` (選んだ区間の候補)・`detail` (選んだ選手)/ `actions`
表示	**sections**: 区間ごとのカード(区間番号・距離・タイプ・地形(山登り / 山下り)・配置された選手の名前・『適性◎〇△』・**個性チップ**(I-72)・**予想タイム**(基準タイム = 調子0・環境なし。I-27)。10区間でも 48 u 以上(08 §4.3)。**candidates**: 登録メンバーを**その区間の適性値 P の高い順**に並べ、P・適性・予想タイム・疲労・**調子の見込み**(R-05)・故障(I-03)。**detail**: 選手の能力と区間適性、特殊能力による加算(山の神 +8 など。04 §5.1)。下段: 総合の予想タイム(基準タイムの合計)・前年度の優勝タイム・補欠(登録のうち出走しない人)・コースの高低差(山区間。I-37)・環境の予報(SC-23 と同じ。I-30)・登録・配置の検証の結果(I-28)
操作	C-RACE-03: 区間を選び、候補を選んで配置(同じ選手を別の区間に置くと入れ替わる)・『おまかせ編成』(初期値。ハンガリー法。04 §5.2)・『確定』(主)
状態・例外	検証 O1〜O3 に反する配置は確定できず、理由を出す(留学生の出走 1 を超える / 中傷以上の選手 / 人数の不足)。軽傷の選手は配置できるが『調子 −2』を表示。**予想タイムは基準のタイムで、調子・環境・ブレーキ・失速は含まない**と注記する

SC-23 他校情報・予報（`scr.preview`）

項目	内容
目的	ライバルの把握と環境の予報（01 §7.2。R-09）
領域	`schools`（出場校）/ `forecast`（予報）/ `actions`
表示	**schools**: 出場校を前評判の順に。**前評判の印（◎〇▲△）**・校名・戦力順位・上位3選手（名前・学年・5000m / 10000m の持ちタイム）。自校を強調。『◎の優勝率は約55%』の注記（I-29）。**forecast**: 気温・向かい風・雨の予報（実際の値に誤差を加えた値。雨の的中率 85%。04 §6）。**予報の誤差**（気温 ±1.5°C など）を注記（I-30）
操作	『続ける』（主）。学校を選んで SC-35 の詳細へ（補助）
状態・例外	見るだけの画面（1タップで進む）。予選会には出さない

SC-24 当日（`scr.raceday`）

項目	内容
目的	当日の調子と環境を見て、当日変更をする。観戦モードを選ぶ（04 §5.3, §9）
領域	`env`（当日の環境）/ `lineup`（先発）・`bench`（控え）/ `actions`
表示	**env**: 気温・向かい風・雨（確定した値。第三戦は往路と復路）（I-31）。**lineup**: 区間ごとに 先発の名前・**調子（確定）**・**余裕の目安（◎〇△）**（I-32）・適性◎〇△・期待タイム（適性・調子・環境・ペース配分を含む。雑音・ブレーキ・展開は含まない）・故障・欠場の自動代役の通知（『〇〇は欠場。控えの△△が走ります』）。**bench**: 控えの選手の調子・疲労・暑さ / 向かい風 / 雨への強さの印（STA / PAC / CON）・**入れ替えたときの期待タイムの差**（I-33）。当日変更の**残り回数**（『残り 1 / 2』）。観戦モードの選択（フル / ダイジェスト / 結果のみ）
操作	C-RACE-04: 先発の選手を選び、控えを選んで入れ替える（**入れ替わった選手は同じ区間を走る**）。『目安どおりに指示』（C-RACE-05 の一括。全区間の指示を余裕の目安どおりにそろえる）。C-RACE-07: 観戦モード。『スタート』（主）
状態・例外	当日変更の上限（1 / 2 / 4）に達したら入れ替えを無効表示。ダイジェスト・結果のみ（おまかせ）では、不調以下の先発が自動で最適な控えに置き換わる（04 §5.3）ので、その旨を表示し、**置き換えの内容は結果の『自校』タブに出す。大会週の故障者はこの画面で『故障』と表示**し、先発に入っていれば代役（自動）と、手動の入れ替えの候補を示す

SC-25 レース進行（`scr.live`）

項目	内容
目的	レースの進行と、区間ごとの指示・檄（04 §9）。フルとダイジェスト
領域	`course`（コース）/ `standings`（順位表）・`lineup`（今の走者）/ `actions`
表示	**course**: 横スクロールのコース（区間・距離・中継所・山の高低差）と走者のミニキャラ（自校を強調）（I-37）。**standings**: 7行（先頭3校 + 自校の前後1校 + 区切り）の順位と先頭との差（繰り上げのあった校は『繰上』）（I-34）。**lineup**: 今の走者（名前・調子・余裕の目安（I-32）・指示）。**見どころ**（首位交代・差の変化・ブレーキ・失速・好走・区間賞・区間記録。ダイジェストは区間ごとに最大2つ。I-35）。檄の残り回数（I-36）
操作	**フル**: 区間の前に**指示のパネル**（攻める / 維持 / 抑える。既定は維持。余裕の目安どおりのボタンを強調。C-RACE-05）。**異変**（区間の1/3か2/3の地点でブレーキ・失速が起きたとき）に**檄**ボタン（C-RACE-06）。区間の前に**先行の檄**も使える（効果は小さく、3回目以降は半分。04 §9.2）。速度 ×1 / ×2 / スキップ（C-FLOW-02）。ポーズ。**ダイジェスト**: 操作なし（指示は維持・檄は自動）で、見どころを見て『続ける』
状態・例外	中断したら『次の区間の指示の前』に戻る（§6.5）。『動きを減らす』ではミニキャラの動きを止め、中継所ごとの静止した表示で進む。檄の上限を超えたらボタンを無効表示

SC-26 大会の結果（`scr.result`）

項目	内容
目的	順位・成績・成長の確認（04 §8, §10。予選会の結果も）
領域	`tabs`（総合 / 区間 / 自校 / 記録）/ `table`・`detail` / `actions`
表示	**総合**: 順位・校名・総合タイム・先頭との差・繰り上げ（『繰上』）・**大会の見どころ**（最大5件。04 §9.3 の判定。I-35）。自校を強調し、**20校でも自校と上位は必ず見える**（自校の位置へスクロール）。**区間**: 区間ごとの区間賞（区間タイム最小）と順位（I-38）。**自校**: 区間ごとの走者・区間タイム・区間順位・累積順位・指示・出来事（ブレーキ・失速・檄・好走）・**自動で行われた当日変更と檄の使用・評価ポイント・名声ポイント・名声の変化・選手の経験（MEN）・やる気の変化**（I-39）。**記録**: 区間記録・大会記録の更新（I-40）。予選会: 12名のタイムと上位10名の合計・通過校（I-69）
操作	行を選んで選手詳細 /『続ける』（主。大会後のイベントへ）/『歴史を見る』（補助。SC-34）

項目	内容
状態・例外	出場できなかった大会(棄権)は『棄権(出走人数が足りません)』と理由を表示(04 §4.1 E1)。優勝のときは演出(静止画に切り替え可)

E. スカウト

SC-27 スカウト(地図)(`scr.scout.map`)

項目	内容
目的	地方 → 県の順に絞って候補を探す入口(01 §6.1。R-08)
領域	`points` (時期とポイント)/ `map` (日本地図)・`side` (選んだ地方・県の要約)/ `actions`
表示	(I-16)**points**: 今の時期(『視察の時期(6〜11月)』『勧誘の時期(12〜2月)』『3月: 確定』)・スカウトポイントの残りと、今年度に付与される総数(I-22)。**map**: 日本地図(8地方を色分けし、選ぶと県のタイル 3×3 に切り替わる。地図は飾りと塗り分けで、**県をタップさせない**。08 §4.3)。地方・県ごとに: **候補の数**・**注目選手**(評価★5)・**県別OBネットワークのレベル**(Lv0〜5。I-17)・**距離帯**(地元 / 隣接 / 遠方 / 海外。視察コストと勧誘の有利・不利。I-18)。**side**: 選んだ地方・県の候補数・視察済み・目標にした数
操作	地方 → 県を選ぶ → 『候補を見る』(SC-28)。『目標とポイント』(SC-30)/『特別招聘』(SC-31。宣言できるとき)/『おまかせスカウト』(主。R-08)
状態・例外	視察の時期以外は『視察は 6〜11月』で視察系の操作を無効表示。留学生の候補は『海外』のタイル(1つ)にまとめる

SC-28 候補リスト(`scr.scout.list`)

項目	内容
目的	候補の一覧と絞り込み
領域	`filters` / `list`・`side` (選んだ候補の要約)/ `actions`
表示	候補ごと(48 u の行): 評価★・名前・出身(県・高校)・**市場評価 V**・**5000m 持ちタイム**(R-03)・性格タグ・**競合度**(ほぼなし / 少 / 中 / 多)・状態(未視察 / 視察済み / 声かけ済み / 練習会済み / 目標)(I-19)。並べ替えの既定は **V の高い順**(= 確定の順番。05 §8.2)。絞り込み: 地方・県・★・上位級・競合度・状態・目標のみ
操作	行を選んで候補の詳細(SC-29)。並べ替え・絞り込み
状態・例外	該当なしは理由と次の行動を出す(『視察した候補はまだいません』)。**520人の全員を一覧に出さない**(地方・県で絞った後の候補だけ。1県あたりの数は 最大でも数十人)

SC-29 候補の詳細(`scr.scout.cand`)

項目	内容
目的	1人の情報の確認と、視察・声かけ・練習会(05 §7)
領域	`header` / `info` (見える情報)・`side` (競合・枠)/ `actions`
表示	**header**: 名前・かな・★・出身(県・高校・距離帯。I-18)・性格タグ(I-07, I-19)・**個性チップ**(視察後に表示。I-72)。**info**: 持ちタイム(5000m。I-02)・市場評価 V・**視察後**: 卒業時の見込み `Q_obs ± σ_info` (『64〜72』。σ はスカウト担当のレベルで 8 → 3.2)・**練習会後**: 成長タイプ・故障しやすさ・素質の細部・**素養**(『素養あり』。種類はスカウト担当が Lv3 以上のとき。I-74)(未解放の項目は『視察で分かります』『練習会で分かります』)(I-20)。好感度は『声かけ済み / 練習会済み』の進み具合で示し、**数値は出さない**(他校の好感度も出さない)。**side**: 競合度(『多: 平均 2.8校が狙っています。枠があれば約65%で競り勝てます』)・上位級なら『奨学金枠を1つ使います(残り n / m)』・**確定の順番**(『あなたの目標の中で 3番目』。9番目以降は『ほぼ入れません』)(I-21)
操作	C-SCOUT-01: 『視察』(1ポイント。6〜11月)/『声かけ』(1ポイント。12〜2月。視察済み)/『練習会に招待』(1ポイント。12〜2月。声かけ済み)。ポイントと時期と前提が足りないボタンは無効表示で理由を出す。『目標にする』(目標の一覧に入れる)
状態・例外	取り消しはない(ポイントは戻らない)ので、ポイントを使う操作は**確認なしで実行する代わりに、費用と効果を事前に表示**する(視察は結果を見て判断するための操作。確認が多いと時間予算を超える)。他校に確定した候補は『他校に決まりました』(3月以降)

SC-30 目標とポイント(`scr.scout.target`)

項目	内容
目的	目標の一覧と、ポイント・枠の計画(I-22, I-23。R-08)

項目	内容
領域	`points`（ポイントの計画）/ `targets`（目標）・`side`（枠のメーター）/ `actions`
表示	**points**: ポイントの内訳のバー（視察 n / 声かけ n / 練習会 n / 残り n）と、年間の総数・これから付与される数。**競合状況**（競合度）と枠の見通し（I-21）。**targets**: 目標を **V の高い順（=確定の順番）**に並べ、各行に 順番・名前・★・V・競合度・進み具合（視察 / 声かけ / 練習会）・上位級の印。**side（枠のメーター）**: 定員 10 − 特別招聘の予約（あれば1）= 使える枠 / 自然に入る目安 約3人 / **推奨の目標 8人** / 上位級の目標数と奨学金枠 / **目標の数が推奨を超えたら黄色、（使える枠 − 自然に入る目安）を超えたら注意**（『9番目以降は定員で入れない見込みです』）
操作	目標の行を選んで詳細（SC-29）/ 目標から外す /『おまかせスカウト』（C-SCOUT-01 の自動版）/『候補を探す』（SC-28）
状態・例外	目標が0のとき『12月までに目標を決めましょう（推奨 8人）』。3月の確定後は結果（入学・惜しかった候補）を SC-14 で見る

SC-31 特別招聘の宣言（`scr.ticket`）

項目	内容
目的	特別招聘券を使って、翌年度に確定で入学する選手の種別を宣言する（07 §3.3, §7）
領域	`tickets`（券の状況）/ `options`（種別・出身県）・`range`（入学する選手の範囲）/ `actions`
表示	**tickets**: 手元の券（0〜2）・繰り延べの券・今年度に宣言できるか（年1回）・在籍の招聘選手 + 予約の数（上限2）・宣言期間（6〜2月）・黄金世代プランの付与の予定（次の付与の年度）（I-24）。**options**: 『留学生』『スーパールーキー』（スーパールーキーは出身県を指定できる = 47県から選ぶ。『おまかせ』も可）。**range**（07 §7 の表）: 入学時の力 64〜68 / 58〜64、卒業時の見込み Q 約68〜74 / 約72〜80、素質、特殊能力 30%、出走の制限（留学生は枠あり）、出身（I-25）
操作	C-SCOUT-02: 種別（と出身県）を選んで『宣言する』（主）→ **取り消せない確認**（SC-37。『券を1枚使います。取り消せません』。『実行』は右、既定は『戻る』。I-26）
状態・例外	宣言できない理由を1行で出す（『今年度の宣言は済んでいます』『宣言期間は 6〜2月です』『券がありません』『在籍する招聘選手が上限です』）。**『無招聘』の実績の注意**: 宣言すると、そのセーブでは『無招聘』の実績が取れなくなることを確認のダイアログに明記（01 §3.3, 07 §11）。宣言後は『宣言済み: 3月に姿が分かります』を表示

F. 運営・歴史

SC-32 運営（`scr.ops`）

項目	内容
目的	施設・スタッフの強化と、資金の確認（05 §4。R-11）
領域	`tabs`（施設 / スタッフ / 資金・収支）/ `list`・`detail` / `actions`
表示	**施設**（寮 / トラック / 坂道コース / トレーニング室 / 医療室）と **スタッフ**（コーチ / トレーナー / 栄養士 / スカウト担当。主務は表示しない = L-25）: 現在のレベル（1〜5）・**効果**（03 §4.6。例:『コーチ Lv3: 全経験値 +12%』）・次のレベルの費用（`300 + 250 × Lv`）・**上限**（監督ランクによる。初期 Lv3、ランク3で Lv4、ランク5で Lv5）・推奨の順位。**資金・収支**: 資金・年度の収入の見込み（`800 + 名声 + 0.0012 × 名声² + 25 × 評価ポイント`）・支出の見込み（スタッフ給与 + 施設維持 + 15 × 部員数）・資金の履歴（I-46, I-47, I-48）
操作	C-OPS-01:『強化』（1件ずつ。**取り消せない確認**）/『推奨をすべて実行』（予備費 500万円を残せる間。R-11。確認あり）。年度末の精算から開いたときは**強化だけ**を出す
状態・例外	資金が足りない・上限に達している強化は無効表示で理由を出す。寮の強化には『奨学金枠と提示スコアに効きます』の注記（05 §5）

SC-33 OB名簿（`scr.ob`）

項目	内容
目的	卒業生の一覧と、OBの役割・県別ネットワーク（I-50, I-17）
領域	`tabs`（OB一覧 / 役割のあるOB / 県別ネットワーク）/ `list`・`side`
表示	OB: 名前・卒業年度・出身（県・高校。I-07）・最終の能力・特殊能力・進路・殿堂入りの印・招聘の印。**役割のあるOB**（コーチ / スカウト協力 / スポンサー。**名前と顔**を出す。06 §8）と期限。**県別ネットワーク**: 県ごとのOBの人数と Lv（`min(5, floor(√人数))`）・次のレベルまでの人数。海外のOBは別枠
操作	OB を選んで選手詳細（閲覧のみ）。並べ替え・絞り込み
状態・例外	OB がいないとき『卒業生はまだいません』

SC-34 歴史・殿堂（`scr.history`）

項目	内容
目的	歴代の記録と収集（01 §6.4, §11）
領域	`tabs`（歴代優勝 / 記録 / 殿堂 / 出身県 / 自校の戦績）/ `body`・`side`
表示	**歴代優勝**: 大会ごと・年度ごとの優勝校（自校を強調）・連覇（I-41）。**記録**: 区間記録（大会・区間ごとの最速。選手・年度）・大会記録（総合タイム）（I-42）。直近で更新された記録を強調（I-40）。**殿堂**: 殿堂入りの卒業生（名前・出身（県・高校。I-07）・年度・成績）（I-43）。**出身県**: 日本地図を、在籍・OB の出身県で塗り分ける（人数の段階）と、『47県の達成 n / 47』（実績に連動。I-44）。**自校の戦績**: 年度ごとの3大会の順位・評価ポイント・名声・三冠・クリアの年度（I-45）
操作	行を選んで選手詳細・OB名簿へ / タブの切り替え
状態・例外	記録がまだないときは『まだ記録はありません』

SC-35 全国の大学（`scr.world`）

項目	内容
目的	他校の状況と、出場権の見通し（I-60, I-63）
領域	`tabs`（戦力順位 / 名声 / 出場権）/ `list`・`detail`
表示	36校の一覧: 戦力順位（先発10人の力の平均）・名声・校格・所在地・前年度の評価ポイント・前年度の第三戦の順位。**出場権**（04 §3.1）: 第一戦（前年度の評価ポイントの上位20校）/ 第二戦（名声の上位30校）/ 第三戦（シード = 前年度の第三戦の上位10校 + 予選会の上位13校 = 23校。L-67）に対する自校の位置と、『あと n位 / 圏内』。選んだ学校の詳細: 上位選手（名前・学年・持ちタイム。R-03）
操作	学校を選んで詳細 / タブの切り替え
状態・例外	他校の選手の能力の数値は出さない（持ちタイムと前評判まで。R-09）

G. ダイアログ・割り込み

SC-36 イベント（`dlg.event`）

項目	内容
目的	イベントの本文と選択（06 §4。I-51）
領域	`body`（題・本文）/ `choices`（選択肢）
表示	題・本文（3行まで。変数の差し込み）・**当事者**（選手の顔と名前。2人のイベントは両方）・選択肢（2〜3個。各ボタン 48 u 以上。ラベル + **効果の要約**（R-15）。『おまかせの既定』に印）。選べない選択肢（`req` が偽）は暗くして理由を1行（I-52）。**お知らせ**（選択肢が1つ）は『OK』だけ。覚醒（特殊能力の取得）は金・青・赤の枠で表示（I-53）
操作	C-EVENT-01: 選択肢を選ぶ（1タップで確定。**取り消せないが、確認は出さない**: 効果は小さく、1年度に約7回あるため）。『長押し』で効果の詳細
状態・例外	`event_auto` のときは出さず、結果を月末レポートに回す。同じ週に2件のとき、1件ずつ順に出す

SC-37 確認（`dlg.confirm`）

項目	内容
目的	操作の確認（08 §7.6）
領域	`body` / `buttons`
表示	本文（**起きること**を具体的に。取り消せない操作では『取り消せません』を必ず含める。I-26）・『戻る』『実行』（取り消せない確認は『戻る』が既定の位置）
操作	選ぶ・閉じる（B / Esc = 戻る）
状態・例外	確認の上に別のダイアログは開かない（N3）

SC-38 通知（`dlg.notify`）

項目	内容
目的	通知の履歴(I-62。C-FLOW-03)
領域	`list` / `actions`
表示	最大30件(新しい順): 日付(第N年度 月 週)・種別・本文。未読を強調。種別: 故障 / 警告 / イベントのお知らせ / 券の獲得 / 購入状態 / 大会の予定 / 自動セーブの失敗
操作	項目を選んで関連の画面へ(選手詳細・エントリーなど)/『すべて既読』
状態・例外	空のとき『通知はありません』

SC-39 ヘルプ・用語（`dlg.help`）

項目	内容
目的	用語と規則の説明(I-70)
領域	`tabs` (この画面 / 用語 / 規則)/ `body`
表示	**この画面**: いま開いている画面の説明(ヒントの再表示)。**用語**: 疲労・調子・余裕の目安・区間適性・ピーキング・安全網・競合度・特別招聘など(用語集と同じ)。**規則**: 年度の流れ・大会の仕組み・育成の流れ(短い図解)
操作	タブ・スクロール。長押し / X / F1 から**その項目の説明へ直接**開く
状態・例外	どの画面の上からでも開き、閉じると元の位置に戻る

SC-40 ポーズ（`dlg.pause`）

項目	内容
目的	中断・保存・設定への入口
領域	`menu`
表示	『再開』『セーブ』『ロード』『設定』『実績』『ヘルプ』『タイトルへ』。最後の自動セーブの日時と同期の状態(I-64)
操作	C-META-05: 『タイトルへ』(通常の確認。最後の自動セーブ以降の進行は失われる旨)。『セーブ』(手動。上書きは確認)
状態・例外	レース進行中(SC-25)に開いたときは、『再開』で同じ位置に戻る。アプリの中断(§6.5)はポーズを経ずに中断セーブを書く(同期で 200 ms 以内。再開は区間の再計算。09 §4.6)。同期の状態は『未接続』『最後の同期: 日時』『競合(未解決)』

SC-41 コンディション警告（`dlg.warn`）

項目	内容
目的	疲労が危険域の選手を知らせる(R-07。I-67)
領域	`list` / `actions`
表示	対象の選手(名前・グループ・疲労・次の週の予測・安全網が働く見込み)。『標準のプランでは起きません。夏合宿の洗礼・個別指定・イベントで疲労が溜まっています』の説明
操作	選手を選んで『個別指導』(SC-20。ケアのプリセット)/『月間計画へ』(SC-16。グループのテーマを回復月にする)/『このまま続ける』(主)
状態・例外	月に1回まで(3週目の終わり)。安全網(F ≥ 85)が自動で働くことを併記

SC-42 故障の知らせ（`dlg.injury`）

項目	内容
目的	重傷・中傷の故障の知らせと、差し替えの案内(R-07。I-68)
領域	`body` / `actions`
表示	選手(名前・グループ・学年)・故障の程度と**残り週数**・登録メンバーなら『○○のエントリーに入っています』・次の大会までの週数・**この選手が先発なら 影響**(適性値・代役の候補)
操作	『エントリーを差し替える』(SC-21。大会の3週前〜前週)/『オーダーを直す』(SC-22)/『個別指導』(ケア。故障中は指定できないので、復帰後)/『続ける』(主)

項目	内容
状態・例外	**大会週の故障はこの画面を出さず、当日の画面(SC-24)に統合**する(§6.3)。軽傷は止めない(月末レポートへ)

第8部 画面仕様 — 画面ごとの必要データと読み取り API

原典: `08b_screen_data.md`　　原典の版: M1 準備　　改版の経緯: なし(初版のまま)

> 前提: 08(規則・流れ)・08a(画面カタログ)・M0 の C#(`m0/Assets/Scripts/Domain/`)。08a の『表示』を 1 項目ずつ、**どの Domain の値・API から取れるか**に対応づけ、足りないものを洗い出した。画面(Unity)の層は、ここの『読み取り API』を呼んで**表示するだけ**にする(08 原則 10)。状態: 2026-10。実装は `m0/Assets/Scripts/Domain/Views/` (表示用のデータ型)と `Domain/Game/GameSession.Views*.cs` (組み立て)。

1. 読み方

記号	意味
○	読み取り API(`Views`)がそろっている。試験あり
△	値と規則は Domain にあるが、画面用にまとめた型が無い(`State` や既存の API から画面の層が組み立てることになる)。または一部の項目が足りない
×	Domain に状態も規則も無い(足りない)。§4 のギャップ(G-xx)に載せた
—	Domain の外(App / Unity 層の設定・ストア・端末の機能)で完結する。Domain からは要らない

Views の約束

1. **状態を変えない**(呼んでも乱数・時刻・ファイルに触れない)。同じ状態なら同じ結果。**全 View を年度のいろいろな時点で呼び、前後の保存の形(`GameSnapshot`)が同じ**ことを試験で確かめる(`ViewPurityTests`)。
2. 表示する語は**文字列テーブルのキー**で返す(`school.{id}.name` / `hs.{id}.name` / `th.*.name` / `skill.*.name` / `problem.*` / `tm.*.name` など。09 §12.3)。校名・高校名・選手名(姓・名)・県名・国名だけは値(文字列)で持つ。
3. 規則(ランク R-01・素質のヒント R-02・記録 R-03・予測 R-05・競合度 R-08・前評判 R-09・月末レポート R-10・個性チップ R-17)は、既存の `DisplayRules` / `Forecaster` / `PlanRules` / `FlowRules` / `Presentation` を通して作る。画面で計算し直さない。
4. 隠す情報(素質・体質・成長タイプ・未判明の素養)は View に**入れない**。画面が間違って出すことができない。

---

2. 実装済みの読み取り API

API(`GameSession`)	返す型	画面	内容	試験
`Home()`	`HomeView`	SC-08	校名・年度・月週・監督ランクと次のランクの累計・名声・資金・士気・戦力順位・クリアと初優勝の有無・次の大会(あと週数・登録と配置の受付)・期待目標と予算ボーナス・招聘の券と宣言の可否・指導ポイント	`ViewTests`
`RosterRows(selected?)` / `RunnerRow(id, selected?)`	`RunnerRowView`	SC-18・21・22 ほか(選手の行)	名前とかな・学年・主将・グループ・力 P とランク・力の順位・レギュラー・疲労・調子・やる気・故障と残り週数・特殊能力の数(金青赤)・出身(県・高校・国)・個性チップ・性格タグ・招聘の印・山対策の案内(登坂 / 下りの素養が判明)・同郷 / 同じ高校の印	`ViewTests`
`RunnerDetail(id)`	`RunnerDetailView`	SC-19・20	10 能力(値・ランク・素質のヒント)・区間適性 5 つ(◎○△)・記録(5000 / 10000 / ハーフ)・特殊能力・絆(上位 8)・出場数・区間賞・ブレーキ・成長タイプ・個性・判明した素養・今月のテーマとの相性・個別指導のメニュー・面談済み	`ViewTests`

API(GameSession)	返す型	画面	内容	試験
Plan()	PlanView	SC-16(SC-17 の一部)	グループごとの人数・故障者・平均疲労・テーマの選択肢(推奨・相性 ◎〇△の人数・4 週のメニュー・月末疲労の平均と最大・要注意・故障の見込み・安全網の週・区間タイプ別の伸び)	PlanEntryOrderViewTests
Entry(kind)	EntryView	SC-21	登録枠・出走に必要な人数・締切・留学生枠・選手ごとの区間適性と調子の見込み・検証の問題(problem.*)	同上
Order(kind) / OrderCandidates(kind, section)	OrderView / OrderCandidateView	SC-22	区間カード(距離・タイプ・地形・配置・適性・予想タイム)・候補を適性順・補欠・総合の予想タイム・検証の問題	同上
RaceResult(outcome, own?)	RaceResultView	SC-26	総合(順位・タイム差・繰り上げ・自校)・区間賞・自校の区間ごと(走者・区間順位・累積順位・指示・ブレーキ / 失速 / 檄 / 繰り上げ)・評価ポイント	ResultViewTests
MonthReportView(report)	MonthReportView	SC-11	月・平均の疲労 / 調子 / 士気とその月内の変化・テーマ・注目の項目(R-10 の種別: 重傷 / 中傷 / 覚醒 / ランクアップ / 大きな成長 / 素養の判明 / 復帰 / お知らせ / 調子の上昇)と『ほか n 件』・大会のまとめ・次の月への注意・次の大会	MonthViewTests
YearSummary()	YearSummaryView	SC-12	3 大会の順位と評価ポイント・三冠と加点・名声の前後・期待目標の達成と予算ボーナス・監督ランクの上昇と強化の上限・累計・招聘券の獲得(これまで / このあと入る節目)・収支の内訳(収入の式 / 補正 / スタッフ / 維持 / 部員費 / 強化の前後の資金 / 施設のレベル)	YearViewTests
Graduation()	GraduationView	SC-13	卒業生(殿堂入りを先頭に): 名前・出身・最終の力とランク・能力ランク・特殊能力・区間賞・出走数・記録・進路・招聘の印 / 退部した選手を別枠	同上
Enrollment()	EnrollmentView	SC-14	新入生(狙った / 自然に入った):行・評価★・能力とランク・距離帯(遠方 / 近郊)。素質・体質・成長タイプは入れない	同上
EventDialog(f, own)	EventDialogView	SC-36	題のキー・当事者・選択肢(キー・選べるか・おまかせの既定・効果の構造 3 つまで + ほか n 件)・覚醒(特殊能力と等級)	DialogViewTests
ConditionWarning()	ConditionWarnView	SC-41	警告の値(70。合宿の月は 80)以上で安全網(85)未満の選手・次の週の疲労と故障の確率・個別指導の有無	同上
InjuryDialog(id)	InjuryDialogView	SC-42	程度・残り週数・次の大会・間に合うか・登録 / 先発(提出したオーダー)か・代役の候補	同上
RaceDayView(rd)	RaceDayView	SC-24	確定した環境・区間ごとの先発(調子・適性 ◎〇△・余裕の目安・目安どおりの指示・いまの指示・期待時間・当日変更の印)・控え(調子・疲労・STA / PAC / CON の印・区間ごとの期待時間の差)・当日変更と檄の残り・いまの決定での自校の予想順位	RaceDayViewTests
RaceLive(rd)	RaceLiveView	SC-25	通過した区間数・これから走る区間・異変(ブレーキ / 失速・損失・通知の位置)・檄の残り・最後に通過した中継所の順位表(先頭 3 校 + 自校の前後 1 校・区切り)	同上
AnnualPlanView()	AnnualPlanView	SC-10	重点・目標大会・ピーキング・期待目標と予算ボーナス・重点 3 種の 12 か月のテーマ / 標準プランとの適性値 P の差 / 故障の回数	PlanOverviewViewTests

API(GameSession)	返す型	画面	内容	試験
`Preview(kind)`	`PreviewView`	SC-23	出場校(前評判の順位・値・印・戦力順位)と区間適性 Long の上位 3 人(名前・学年・持ちタイム。他校は 04 §11 の復元)・予報(第三戦は往路と復路)	同上
`ScoutCandidates()` / `ScoutCandidate(i)`	`ScoutCandidateView`	SC-28・29	名前・★・V・出身(県・高校・距離帯)・持ちタイム・性格タグ・競合度・段階ごとの見え方(視察: 見込み ± σ・個性チップ / 練習会: 成長タイプ・素養の有無 / 担当 Lv3: 素養の種類)・いま行動できるか	`ScoutViewTests`
`ScoutMap()`	`ScoutMapView`	SC-27	時期・ポイント・地方 / 県ごとの候補数・視察済み・目標・★5・OB ネットワーク Lv・距離帯	同上
`ScoutTargets()`	`ScoutTargetsView`	SC-30	目標(V の高い順 = 確定の順番・9 番目以降は『ほぼ入れません』)・枠のメーター(定員 10 − 招聘の予約・自然に入る目安 3・推奨 8・上位級と奨学金枠・黄色 / 注意)	同上
`Ops()`	`OpsView`	SC-32	施設・スタッフ 9 行(レベル・上限・次の費用・上げられない理由 cap / funds・推奨の順位と『推奨をすべて実行』のあとのレベル・効果と次のレベルの効果 `fx.*`)・資金・予備費・収入 / 支出の見込み・年度末の資金の履歴	`ClubViewTests`
`WorldOverview()`	`WorldView`	SC-35	36 校(戦力順位・名声・校格・地方・前年度の評価ポイントと第三戦の順位)・出場権に対する自校の位置(第一戦 20 / 第二戦 30 / 第三戦のシード 10 または予選会)	同上
`Obs()`	`ObView`	SC-33	OB(卒業年度の新しい順)・役割のある OB と残り週数・県別ネットワーク(人数・Lv・次のレベルまであと何人)	同上
`History()`	`HistoryView`	SC-34	年度ごとの戦績・各大会の初優勝の年度・優勝の通算・三冠・クリアと無招聘・殿堂入り・出身県ごとの人数と『47 県の達成 n』	同上

上の表の型に後から足した項目(G-02〜G-17。同じ API の返す型の中。試験は `RaceHighlightTests` `RecordTests` `OutlookTests` `MissedCandidateTests` `StatDeltaTests` ほか)

API / 型	足した項目	画面
`RaceResult` → `RaceResultView`	`Highlights` (大会の見どころ。最大 5 件・同じ種類は 2 件まで)・`Records` (記録の更新。区間の順、最後に大会記録)・`OwnLegs[].Record`	SC-26
`RaceLive` → `RaceLiveView`	`Highlights` (通過した区間のダイジェスト。区間ごとに最大 2 件。見込みの記録も含む)	SC-25
`Plan()` → `PlanView`	`OutlookRace` / `OutlookWeeks` (8 週以内で出場できる次の大会)・`ThemeOptionView.GoodChance` (そのテーマで『好調以上』の見込み)	SC-16
`Entry(kind)` / `Order(kind)` / `OrderCandidates(kind, section)`	各行の `Outlook` (`FormOutlookView` 。登録した場合の調子の分布・故障の確率・『好調以上』・大会の週の疲労)。1 人分は `FormOutlook(runnerId, kind)`	SC-21・22
`RunnerDetail(id)` → `StatView`	`DeltaThisMonth` / `DeltaLastMonth` (能力の増減)	SC-19
`Enrollment()` → `EnrollmentView`	`Missed` (惜しかった候補。名前・★・出身 + 理由 `miss.*` + 入った学校 + 競り負けの差)	SC-14
`Graduation()` / `Obs()` → `GraduateRowView`	`CountryName` (留学生の出身国)	SC-13・33
`History()` → `HistoryView`	`Champions` (歴代の優勝校。予熱の年度 = ゲーム開始前の歴史を含む `PreGame`)・`CourseRecords`・`TeamRecords`	SC-34
`Home()` → `HomeView`	`UnreadNotifications` (ベルの未読数)	SC-08
`Notifications()` → `NotificationsView`	通知(新しい順・最大 30 件・未読)。`MarkNotificationRead` / `MarkAllNotificationsRead` / `Notify` (App 層から)	SC-38

API / 型	足した項目	画面
`EventDialog(f)` → `ChoiceView.Effects` の `EffectView`	OB の役割の `Mod` / `ModValue`(役割が作る補正)。要約の文は App 層の `EffectText.Summary`(R-15。`efx.*`)。選択肢の本文に数字は書かない(L-63 ①)	SC-36
`QualifierResult()` → `QualifierResultView`	予選会の結果(順位表・通過ライン・自校の差・自校の 12 名のタイム)。保存しない	SC-26
`Entry(kind)` / `Order(kind)` の判定	登録は重傷だけ不可・配置は中傷以上が不可(軽傷は出られる。`TeamRacing.CanRegister` / `CanPlace`。L-59)。`FormOutlook` は大会の週にまだ故障中なら調子 −2 で確定	SC-21・22

見どころ・記録・調子の見込みの文は `hl.*` / `miss.*` のキーに `{school}` `{sec}` などを当てはめる(文字列表)。

既に画面向きの読み取りがある(Views の型は無い)もの

API	画面	備考
`ScoutPool` / `StageOf(i)` / `ViewCandidate(i)` → `ScoutView`	SC-28・29	評価 V・上位級・競合度と競り勝ちの割合・視察後の見込み `QObs ± σ`・個性 / 素養 / 成長タイプの見える段階
`ScoutPointsAvailable` / `ScoutTargetRecommended` / `ScoutStages`	SC-27・30	ポイントの残りと推奨の目標数(8)
`RacePreview(kind)` → `PreviewRow` / `Forecast(kind)` → `Env[]`	SC-23	前評判の印(◎○▲△)・戦力順位・予報
`OpenRaceDay(kind)` → `RaceDay`: `View(section)` / `Lineup` / `Bench` / `BenchExpectedSec` / `SwapsLeft` / `GekiLeft` / `Standings(section)` / `NextIncident`	SC-24・25	適性・余裕の目安・期待時間・当日変更と檄の残り・中継所の順位表・ブレーキ / 失速の通知
`Stats`(`PlayerYearStats`)/ `State.Obs`(`OBRecord`)/ `State.Cleared`	SC-34・33・15	自校の年度ごとの戦績 / OB / クリアの年度
`Catalog`(`EventCatalog`)・`Master`(`TrainingMaster`)	SC-36・32	イベント・特殊能力・練習・テーマの定義

3. 画面ごとの対応表

(画面番号は 08 §5。『取得元』は Domain の API / 状態。状態は §1 の記号)

3.1 起動・設定

SC	表示項目(08a)	取得元	状態
SC-01 タイトル	題名・版 / 続きから(直近のセーブの校名・年度・日時)/ 各ボタン	`SlotStore`(保存の一覧)と `HeaderInfo`(見出しの JSON)	○ `SlotCatalog.List()`(ヘッダだけから)
SC-02 新規ゲーム	開始校のカード(戦力順位・施設・資金・部員数・目安)/ 世界の生成結果 / 監督名 / スロット	`Scenarios.For(difficulty)` / `GameSession.New` の結果(`School`・`Home()`)/ `TeamState.ManagerName`	△ 世界の生成(予熱 15 年度)を先に回してから見せる。クリアまでの目安時間は固定の文言
SC-03 セーブの選択	スロットごとの校名・開始校・年度月・直近の成績・プレイ時間・日時・同期の状態	`SlotStore` / `HeaderInfo`	○ `SlotCatalog.List()`(プレイ時間は `GameHost` が数える)
SC-04 設定 / SC-05 追加コンテンツ / SC-07 クレジット	設定値・商品・購入状態・権利表示	`TeamSettings`(設定値)。購入状態・ストアは App 層	― App 層
SC-06 実績	名前・条件・解除日・進捗	―	× 実績の定義と判定が無い(G-08)

3.2 ホームと進行

SC	表示項目	取得元	状態
SC-08 ホーム	status / next / バッジ / menu	`Home()`	○。バッジのうち『購入状態不明』は App 層(`GameSession.Notify(Entitlement, "unknown")`)、『未読の通知』は `HomeView.UnreadNotifications`

SC	表示項目	取得元	状態
SC-09 年間予定	月ごとの大会・締切・合宿・記録会・スカウトの時期・招聘期間・年度末	`Calendar.RaceWeeks` / `TeamWeek.EntryDeadlineWeeks` / `TicketK` ほかの定数	△ 年間予定の View は無い(定数の並べ替えだけ。小さい)
SC-10 年間方針	重点・標準プランとの P の差(R-04)・目標大会・期待目標・ピーキング	`AnnualPlanView()`	○
SC-11 月末レポート	summary / highlights / 大会のまとめ / 次の月への注意	`MonthReportView(report)`	○(`gift_reveal` を含む。判明した素養は `ReportItemView.Gifts`)
SC-12 年度の結果と精算	results / funds	`YearSummary()`	○
SC-13 卒業・進路	卒業生・退部	`Graduation()`	○。留学生の出身国は OB に残らない(G-17)
SC-14 入学者発表・入部確認	新入生・狙った / 自然	`Enrollment()`	○。『惜しかった候補』は G-05
SC-15 クリア	演出・年度・開始校・各大会の初優勝の年度・無招聘	`State.Cleared`(`ClearInfo`)/ `Stats[].Wins` / `State.Scenario`	△ View は無い。実績の解除(その場で解除された一覧)は G-08

3.3 育成・部員

SC	表示項目	取得元	状態
SC-16 月間計画	groups / テーマの選択 / 相性 / forecast ①〜⑥	`Plan()`	○。⑥『次の大会の調子の見込み』は G-15
SC-17 カスタム週の編集	週ごとのメニュー・負荷 L・予測の疲労	`Plan()` のテーマの週 / `Forecaster` / `TrainingMaster`	△ カスタム週専用の View は無い(選べるメニューの一覧は `Master`)
SC-18 部員一覧	一覧(`RunnerRow`)・要約・グループ列・世代グラフ	`RosterRows()`	○。並べ替え・絞り込みは画面側。世代グラフ(学年別の P 平均と上位の人数)は集計の View が無い(△ 小さい)
SC-19 選手詳細	header / stats(前月の増減)/ side / 絆 / 経歴 / この月の予測	`RunnerDetail()`	○。**前月の増減**は状態に無い(G-14)
SC-20 個別指導	対象の選手・指導ポイント・面談・メニューの指定・予測・注意	`RunnerDetail()` / `State.GuidanceLeft` / `PlanRules.CanGuide` / `GuideMenu` の理由	△ 専用の View は無い(`RunnerDetail().IndividualMenu`・`Talked` と上の組み合わせ)

3.4 大会

SC	表示項目	取得元	状態
SC-21 エントリー	summary / roster / entered / 検証	`Entry(kind)`	○
SC-22 区間オーダー	sections / candidates / detail / 下段	`Order(kind)` / `OrderCandidates()`	○。前年度の優勝タイム・環境の予報(`Forecast`)・コースの高低差は別の API(△)
SC-23 他校情報・予報	出場校の前評判の印・戦力順位・**上位 3 選手(名前・学年・持ちタイム)** / 予報	`Preview(kind)`(`RacePreview` + `Forecast` を画面向けにまとめたもの)	○
SC-24 当日	env / lineup(調子・余裕の目安・適性・期待タイム)/ bench / 残り回数	`RaceDayView(rd)`	○(控えの暑さ / 向かい風 / 雨への強さの印と、区間ごとの期待時間の差を含む)

SC	表示項目	取得元	状態
SC-25 レース進行	course / standings(7 行)/ lineup / 見どころ / 檄	`RaceLive(rd)`	○ 順位表(先頭 3 校 + 自校の前後 1 校)・次の区間・異変と檄の残り・**見どころ**(`Highlights`。通過した区間ごとに最大 2 件。G-02 済)。記録も見どころに出る(最重要)。『繰上』の印は中継所の表に未対応
SC-26 大会の結果	総合 / 区間賞 / 自校 / 記録 / 予選会	`RaceResult()`	○『大会の見どころ』(`Highlights`。最大 5 件。G-02 済)・『記録の更新』(`Records`・`OwnLegs[].Record`。G-03 済)。自動で行われた当日変更と檄の使用は `RaceDayRecord`(△)。予選会の結果は `QualifierResult()` → `QualifierResultView`(○。順位表・通過ライン(10 位の合計)・自校の差(通過 = 次点との余裕 / 落選 = 足りなかった差)・自校の 12 名のタイムと『合計に数える 10 名』。保存しない = 予選会の直後の停止で使う)

3.5 スカウト

SC	表示項目	取得元	状態
SC-27 地図	時期・ポイント / 地方・県ごとの候補数・★5・OB ネットワーク Lv・距離帯	`ScoutMap()`	○
SC-28 候補リスト	★・名前・出身・V・5000m 持ちタイム・性格タグ・競合度・状態	`ScoutCandidates()`(V の高い順)	○(見えていないものは型に入らない)
SC-29 候補の詳細	header / info(視察後の見込み・練習会後の細部)/ 競合度 / 枠	`ScoutCandidate(index)`	○。『故障しやすさ』は候補に状態がない(未実装。05 §7.2)
SC-30 目標とポイント	points の内訳 / targets / 枠のメーター	`ScoutTargets()`	○
SC-31 特別招聘の宣言	tickets / options / range	`Home()` の券 / `State.Tickets` / `TicketK`	△ 07 §7 の範囲の表は固定の文言

3.6 運営・歴史

SC	表示項目	取得元	状態
SC-32 運営	施設とスタッフのレベル・効果・次の費用・上限・推奨の順位 / 資金・収支の見込み・履歴	`Ops()`	○(効果の文は `fx.*`。履歴は年度末の資金)
SC-33 OB 名簿	OB の一覧 / 役割のある OB / 県別ネットワーク	`Obs()`	○。留学生の出身国は G-17
SC-34 歴史・殿堂	歴代優勝 / 区間記録・大会記録 / 殿堂 / 出身県の地図 / 自校の戦績	`History()`(自校の戦績・初優勝・通算・三冠・殿堂・出身県・**歴代の優勝校(全校。`Champions`)・区間記録 / 大会記録**(`CourseRecords` `TeamRecords`))	○
SC-35 全国の大学	36 校の戦力順位・名声・校格・所在地・前年度の評価ポイント・第三戦の順位 / 出場権の位置 / 上位選手	`WorldOverview()`	△ 上位選手(名前・持ちタイム)は G-09

3.7 ダイアログ

SC	表示項目	取得元	状態
SC-36 イベント	題・本文・当事者・選択肢とその効果の要約(R-15)・選べない理由	`EventDialog(FiredEvent, own)`	○ 効果は構造(`EffectView`)で 3 つまで +『ほか n件』。文は UI が `ev.*` と効果の語から組み立てる。選べない理由の文は未(`req` の式から理由を作る規則がない。小さい)
SC-37 確認	本文・既定の位置	`OpResult.NeedsConfirm` / `Warnings`(`warn.*`。`App/OpText.Warnings` で文にする。L-63 ③)	—

SC	表示項目	取得元	状態
SC-38 通知	最大 30 件(日付・種別・本文・未読)	`Notifications()` → `NotificationsView`(新しい順。`Unread`)/ `MarkNotificationRead(index)` / `MarkAllNotificationsRead()`	○ 文は `TextKey`(`nt.*`)に `{runner}` `{n}` `{arg}`(`ArgKey` の文字列)を当てはめる。`Target` = 選んだとき開く画面
SC-39 ヘルプ・用語	用語・規則	文字列テーブル	— App 層
SC-40 ポーズ	再開・セーブ・ロード・設定…	`GameFlow.CanSave` / `SuspendCheckpoint`	— App 層(最後の自動セーブの日時は `SlotStore`)
SC-41 コンディション警告	対象の選手・次の週の予測・安全網	`ConditionWarning()`	○(次の週の疲労・故障の確率は `Forecaster` の正確な予測)
SC-42 故障の知らせ	選手・程度と残り週数・登録メンバーか・影響	`InjuryDialog(runnerId)`	○(次の大会までの週数・大会に間に合うか・提出したオーダーの先発か・代役の候補 3 人)

4. 足りないもの(ギャップ)

優先度: **高** = M1 の縦切り(1 年度が通る)の前に要る / **中** = 画面ができる段階で要る / **低** = 後でよい。

ID	画面	内容	必要な Domain の変更	優先	関連
G-01	SC-11・18・19	~~素養の判明が未実装~~ **解決**	週の解決で入部 12 週後に判明(`WeekReport.GiftRevealed`)・スカウトで種類を見た選手は入部と同時・月末レポートの `gift_reveal`・部員一覧の山対策の案内(`RunnerRowView.MountainAdvice`)	済	L-64
~~G-02~~	SC-25・26	~~見どころ~~(**済**: `RaceHighlights`。04 §9.3。首位交代・差の変化・ブレーキ・失速・好走・区間賞・順位の変動。区間記録は G-03 待ち)	結果からの判定(乱数なし。`TeamResult.Boost` を足しただけで結果は変わらない)	—	L-60 ③ 解決
~~G-03~~	SC-26・34	~~区間記録・大会記録~~(**済**: `WorldSim.History`・`Runner.Records`・`RaceResult().Records`・`History().CourseRecords` / `TeamRecords`・殿堂入り・`race.record`)	状態・保存・判定・View	—	L-53 ③ 解決。L-68: 新規ゲームの区間記録は年度 1 の前に『初期の区間記録』(保持者 = 歴代の記録)に置き換わる。保持者が学校でないとき `SchoolId = -1`・`SchoolNameKey = record.legacy_holder`・`CourseRecordView.IsLegacy` / `RecordUpdateView.PrevIsLegacy` = true(走者名・年度は出さない)
~~G-04~~	SC-34	~~歴代の優勝校~~(**済**: `History().Champions`。予熱の年度 = ゲーム開始前の歴史を含む。`PreGame`)	世界の履歴	—	—
~~G-05~~	SC-14	~~惜しかった候補~~(**済**: `Enrollment().Missed`。`Allocator` が自校の狙った候補の入れなかった理由 = 定員 / 奨学金枠 / 競り負け(差)を返す。招聘枠に押し出された候補は定員。保存しない)	割り当ての記録(乱数・結果は不変)	—	—
G-06	SC-36	~~イベントの効果の構造~~ **解決**(`EventDialog` の `EffectView`)。L-63 ① **済**(本文から数値を外し、`App/EffectText.Summary` が構造から要約を作る)。残り: 選べない理由の文	効果から本文の数値を作る	低	L-63

ID	画面	内容	必要な Domain の変更	優先	関連
~~G-07~~	SC-08・38	~~通知のログ~~(**済**: `TeamState.Notifications`(最大 `K_NOTIFY_MAX` = 30・新しい順・既読フラグ)。足す箇所 = 故障(`ResolveTraining` の `NewInjury`)・疲労の警告(月の第 3 週に 1 回)・イベント(`ResolveWeekEvents`)・特別招聘券(週の大会の後・年度末)・大会の予定(エントリー開始・大会の週・予選会の週。出場権があるときだけ)、外から `GameSession.Notify`(App 層: 自動セーブの失敗 = `GameHost.OnSaveNotice`・購入状態の不明)。保存 = `player_team.notifications`(追加の項目。ないセーブは空))	通知の生成元の集約・保存	―	L-65
~~G-08~~	SC-06・15	~~実績~~(**済**: `AchievementMaster`(01 §3.3 の 23 件。`{id}.title` / `.desc`)・`AchievementFacts.From(gs, race)` → `AchievementRules.Satisfied`。App の `AchievementTracker` が `ProfileData`(解除の日時・クリア)とストアへの通知待ち(`Pending`)に記録し、`GameHost.Advance` が呼ぶ。**プロフィールのファイル(`profile.a/b.eksv`)の読み書きとストアへの通知は Unity 側**)	実績の定義・判定・解除の記録	―	―
G-09	SC-23・35	~~他校の上位選手~~ **解決**(`Preview` の `Top`。名前は `look_seed` から。SC-35 の一覧には入れていない)	`NameGenerator` と 04 §11 の復元	済	―
G-10	SC-27・30	~~スカウトの地図の集計と枠のメーター~~ **解決**(`ScoutMap` / `ScoutTargets`)	残り:『故障しやすさ』(候補に状態がない)	済	―
G-11	SC-10	~~年間方針の重点ごとの P の差~~ **解決**(`AnnualPlanView`。表は `PlanRules.FocusDeltaP`、仕様の表と試験で突き合わせる)	―	済	―
G-12	SC-32	~~運営~~ **解決**(`Ops()`)	施設の効果は `FacilityEffectView`(コード + 値)、文は `fx.*`	済	―
G-13	SC-33・34・35	~~OB 名簿・歴史・全国の大学~~ **解決**(`Obs()` `History()` `WorldOverview()`)。残りは G-03・G-04・G-09	OB の View は `GraduateRowView` と共通	済	―
~~G-14~~	SC-19	~~能力の前月の増減~~(**済**: `Runner.MonthStartStats` / `PrevMonthStats`(`monthly.stats_start` / `stats_prev`。追加の項目)。`StatView.DeltaThisMonth` / `DeltaLastMonth`。月の最初に `RollMonthStats`)	選手に月の初めの能力を持たせる(保存が増える)	―	L-62 ②
~~G-15~~	SC-16・21・22	~~次の大会の調子の見込み~~(**済**: `FormOutlook(runnerId, kind)` → `FormOutlookView`。`Plan()` の `OutlookRace` / `ThemeOptionView.GoodChance`・`Entry()` の `Outlook`・`Order()` / `OrderCandidates()` の `Outlook`)	`Forecaster` の調子の分布を大会の週まで	―	―

ID	画面	内容	必要な Domain の変更	優先	関連
G-16	SC-01・03	~~セーブの見出し・プレイ時間~~ **解決**	`SlotCatalog.List` = `SlotSummary` (校名のキー・開始校・難易度・年度・月・プレイ時間・保存日時・手動の有無・『続きから』が中断した場所からか)。`GameHost` がプレイ時間を数えてヘッダに入れる	済	—
~~G-17~~	SC-13・33	~~卒業した留学生の出身国~~(**済**: `OBRecord.Country` (`origin_country` 。追加の項目)。`GraduateRowView.CountryName`)	OB に国を足す	—	—

4.1 この作業で直したもの

内容	変更
月末レポートの項目の作り方が仕様と違った(L-57)	`rank_up` (能力のランク文字が上がった)・`form_up`・`big_growth` (月に能力が合計 +3 以上。以前は『レベルアップ 2 回以上』の暫定)を R-10 どおりに作る。故障 / 復帰の score = 力 P、覚醒の score = 金 > 青 > 赤。月の初めの能力・調子・疲労・士気を覚える(`GameFlow`)
年度末の精算の内訳が捨てられていた	`TeamYear.Settle` が `Settlement` を返す(収入の式・補正・スタッフ・維持・部員費・強化の前後)。`GameSession.LastSettlement` (保存しない: 年度末の画面は中断できない。09 §4.6)
卒業生の出身高校が残らなかった	`OBRecord.HighSchoolName` に `hs.{id}.name` (文字列のキー)を入れる(卒業時)
入学者の評価★・狙った候補が残らなかった	`LastIntakeStars` / `LastIntakeTargeted` (保存しない。入学者発表の画面用)。評価★ = `DisplayRules.Stars(V)` (05 §6.1)
特別招聘券の節目の名前	`tm.*.name` (継続の記念は `{years}`)を文字列テーブルに
招聘券の『このあと入る節目』	`TicketMilestones.PeekAtYearEnd` (状態を変えない)

---

5. 次の順序(提案)

1. ~~G-01 素養の判明~~(済。L-64)。
2. **App 層の入口**(自動セーブ・中断セーブの呼び出し位置 = `SuspendCheckpoint` の接続。L-62 ③)。
3. M1 の縦切りに要る View の残り: `Scout*` (G-10)・`Ops` (G-12)・`World` / `OB` / `History` (G-13)・`RaceDay` の画面向けの型。
4. ~~G-02(見どころ)・G-03(記録)・G-04(歴代)~~(済)。
5. ~~G-14(前月の増減)~~(済。追加の項目で済んだ)・~~G-07(通知のログ)~~(済)。残りは G-06 の残りだけ。

第9部 セーブ・技術仕様

原典: `09_save_tech_spec.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> 範囲: **セーブの形式**(コンテナ・世代・原子的な書き込み・中断セーブ)、**バージョニングとマイグレーション**、**読み込み時の検証と修復**、乱数の鍵と再現性、**クラウド同期**、**購入状態の時計対策**、**実績・購入の通知**、**マスタデータと文字列テーブル**、**数値・時間の整形(正本)**、**性能とスレッド**、**技術スタックとプロジェクト構成**、**テスト・統計の受け入れ基準**、そして**仕様完了のチェックと M0 の着手条件**。前提: 01〜08。特に `02_data_spec.md` §4.9・§5(セーブ構造・不変条件)、`03` §3 / `04` §12(乱数)、`07` §8(購入状態)、`08` §6.5・§13.4(保存のタイミングと引き継ぎ)。検算: `verify/ref_model_09.py`（`python3 -I ref_model_09.py all`）。仕様書の数値・ベクタはここから出力した値。位置づけ: これで **S(仕様確定)の最後の文書**。§17 で『仕様完了の基準(00 §2)』を全文書について確かめ、M0 の着手条件を示す。

1. 設計の原則

ID	原則	具体化
P-1	**セーブは壊れない**	書き込みは原子的(一時ファイル → 読み戻して検証 → 置き換え)で、2世代を交互に使う。電源断・強制終了のどの時点でも、直前か新しい世代のどちらかが読める(§4。134 通りで検算)
P-2	**セーブは古くならない**	`schema_version` + 連鎖するマイグレーション。古い版は読めて、新しい版は『アプリを更新してください』で断る(§6)
P-3	**セーブは消えない**	同期は**決して削除しない**。競合は両方を残す。破損は修復するか、1つ前の世代へ戻す(§7・§9)
P-4	**状態だけを保存する**	乱数は鍵つきで状態を持たない(§8)。導出できる値(PB など)は保存しない。ロードしても同じ結果になる
P-5	**端末間のビット一致は求めない**	02 §1 のとおり。ただし**実測で見積もり**(§8.4)、一致が安く得られる箇所(基準ペースの `log`)だけは表で一致させる
P-6	**Domain は純粋**	Unity・時刻・ファイル・ストア・乱数源を知らない。境界は抽象インターフェース(§15)
P-7	**オフラインで完結する**	ストアとの通信は待たせない。失敗しても遊べる(§10・§11)
P-8	**測れるものは測ってから決める**	数値の根拠は `ref_model_09.py`。実機でしか測れないもの(UI・ストア API)は M1・M5 の検証項目に回す(§17)

2. 定数表(初期値案。`K_` 接頭辞。`ref_model_09.py` の `K` と一致させる)

定数	値	意味
`K_SLOTS`	3	セーブスロットの数(08 の `K_UI_SAVE_SLOTS` と同じ)
`K_AUTO_GENERATIONS`	2	自動セーブの世代数(A / B)。手動セーブはスロットごとに別に1つ(§3)
`K_MAGIC`	`EKSV`	コンテナの先頭4バイト(ASCII)
`K_CONTAINER_VER`	1	コンテナ形式の版(§4.1)
`K_SCHEMA_VER`	1	セーブ(ペイロード)の構造の版。製品の初版が 1(§6)
`K_GZIP_LEVEL`	6	ペイロードの gzip の圧縮レベル
`K_SIZE_TARGET`	1048576	100年度後の圧縮後サイズの目標(1 MB)
`K_SIZE_HARD`	3145728	圧縮後サイズの上限(3 MB。Google Play Games の保存ゲームの上限に合わせる)
`K_TICKET_LOG_MAX`	50	`tickets.log` の保持件数(新しい順)
`K_EVENT_LOG_MAX`	50	`event_state.log` の保持件数
`K_NOTIFY_MAX`	30	通知の履歴の件数(08 §7.5)
`K_CLOCK_BACK_TOL_S`	86400	時計の巻き戻しを許す幅(24時間。§10)
`K_GRACE_DAYS`	30	購入状態を確認できないときの猶予(07 §8.2 の `K_OFFLINE_GRACE_DAYS` と同じ)

定数	値	意味
`K_GRACE_RUN_H`	120	猶予のあいだに『アプリを動かした時間』の上限(時間。§10)
`K_HEADER_MAX`	65536	ヘッダ JSON の上限(バイト)
`K_PAYLOAD_MAX`	67108864	伸長後のペイロードの上限(バイト。64 MB。圧縮爆弾の対策)
`K_BACKUP_MAX`	5	競合の退避の最大数(スロットごと。§9.4)
`K_LOG_MAX_BYTES`	1048576	診断ログの上限(バイト。古い行から捨てる)
`K_STORE_TIMEOUT_S`	5	ストアへの問い合わせの待ち時間(秒。§10・§11)
`K_SAVE_RETRY_S`	1	書き込みの失敗後に、再試行するまでの待ち(秒。§4.6)
`K_ACH_RETRY_S`	60	実績の通知の再送の最短間隔(秒。§11)
`K_SYNC_MIN_S`	60	同じ記録のアップロードの最短間隔(秒。§9.3)
`K_DOMAIN_CALL_MS`	100	Domain の 1 呼び出しの上限(下位のスマホで。§14)

3. 保存するものと置き場所

3.1 ファイルの構成

ルート = `Application.persistentDataPath/EkidenSaves/` (以下、**root**)。ファイルの名前は `.eksv` (コンテナ)・`.json` (平文)。

パス(root から)	内容	形式	クラウド	備考
`saves/slot{n}.a.eksv` / `slot{n}.b.eksv`	自動セーブ(2世代)。n = 1〜`K_SLOTS`	コンテナ(§4)	○ 新しい世代の1つ	交互に書く(§4.4)
`saves/slot{n}.m.eksv`	手動セーブ(スロットごとに1つ)	コンテナ	○	ポーズの『セーブ』(08 §6.5)。上書きは確認する
`saves/suspend.eksv`	中断セーブ(アプリの中断時に1つ)	コンテナ	×	再開に1回だけ使う(§4.6)
`profile.a.eksv` / `profile.b.eksv`	`ProfileData` (実績・累計・クリアの記録。02 §3.10)	コンテナ	○(マージ。§9.6)	2世代
`device.json`	`DeviceSettings` (02 §3.12)	JSON	×	壊れたら既定値で起動(致命にしない)
`local_state.json`	`LocalState` (§3.3)	JSON	×	同期の基準点・購入状態のキャッシュ・時計の高水位・実績の保留
`backups/`	競合の退避 `conflict_slot{n}_{UTC時刻}.eksv`、移行前の控え `premigrate_slot{n}_v{k}.eksv`	コンテナ	×	§9.4・§6.1
`logs/diag.log`	診断ログ(上限 `K_LOG_MAX_BYTES`)	平文	×	外へ送らない。『診断情報をコピー』でだけ出す

- 書き込みはすべて**一時ファイル(`.tmp`)→ 置き換え**で行う(§4.4)。`.tmp` が残っていたら、起動時に削除する(読み込みには使わない)。
- **クラウドに載せるもの**は、スロットの先頭(自動の A / B のうち保存番号が大きい有効な1つ)・手動セーブ・プロフィールだけ。中断セーブ・端末の設定・退避・ログは載せない。
- 同期の単位(**記録 record**)は `slot{n}` (自動の先頭)・`slot{n}m` (手動)・`profile` の3種類(§9.1)。

3.2 3つの保存先の使い分け

名前	何を持つか	単位	同期	破損のとき
`SaveData` (セーブ)	1つの歴史のすべて(02 §4.9)	スロット	○	修復 → 1つ前の世代(§7)

名前	何を持つか	単位	同期	破損のとき
`ProfileData`	実績・累計・クリアの記録	アカウント	○(マージ)	1つ前の世代 → 空から作り直し(実績はストアに残っているので、解除済みの実績をストアから読み戻せるなら読み戻す。§11)
`DeviceSettings` / `LocalState`	見え方・音・操作 / 端末ごとの内部状態	端末	×	既定値

3.3 LocalState(新設。端末ごと・同期しない・利用者には見せない)

項目	型	説明
`schema_version`	int	
`clock_hw`	int	端末の時刻(UTC 秒)の**高水位**。時計の巻き戻しの検知に使う(§10)
`run_total_s`	int	アプリを動かした累計の秒(**単調時計**から。時刻の変更に影響されない。§10)
`entitlements`	map<pack_id, { owned, verified_at, verified_run }>	購入状態の最後の確認(07 §8)。**02 §3.10 の `ProfileData.entitlements` をここへ移す**(確認は端末ごと。`verified_run` = そのときの `run_total_s`)
`sync`	map<record, { lineage_id, counter, deleted }>	各記録の**最後に同期した時点**(§9.1)。`deleted` = 墓石
`pending_unlocks`	list<{ ach_id, queued_at }>	ストアへまだ通知していない実績の解除(§11)
`install_id`	string	端末の識別(診断のためだけ。ランダム。外へ送らない)

> 02 §3.10 の `ProfileData.entitlements`(購入状態のキャッシュ)を `LocalState` へ移した理由: ①確認の時刻と『動かした時間』は**端末ごとの量**で、同期すると意味が変わる ②購入そのものはストアのアカウントに紐づき、端末は起動時にストアへ問い合わせ直す ③ `ProfileData` は『実績・累計・クリア』だけにするとマージの規則が単純になる(§9.6)。07 §8.1・02 §3.10 を直す(§18)。

---

4. コンテナ形式 `EKSV`

4.1 バイトの並び(すべてリトルエンディアン)

オフセット	長さ	内容
0	4	`K_MAGIC = 45 4B 53 56`（`EKSV`）
4	2	`container_ver`(uint16)。未知の新しい版は**開かずに断る**
6	2	`flags`(uint16)。bit0 = ペイロードは gzip(v1 は常に 1)。その他は 0
8	4	`header_len` H(uint32。`K_HEADER_MAX` 以下)
12	H	ヘッダ JSON(UTF-8。§4.2)
12+H	4	`header_crc32`(先頭から ここまでの全バイトの CRC32)
16+H	4	`payload_len` P(uint32。`K_PAYLOAD_MAX` 以下)
20+H	P	ペイロード = **セーブ本体の JSON(UTF-8)を gzip**（`K_GZIP_LEVEL`、`mtime = 0`）
20+H+P	4	`file_crc32`(先頭から ここまでの全バイトの CRC32)

- **CRC32** は IEEE 802.3(多項式 0xEDB88320 の反転形。初期値・最終 XOR とも 0xFFFFFFFF)。検査値: `CRC32("123456789") = 0xCBF43926`。
- **ヘッダだけ読める**: スロットの一覧(SC-03)は、先頭 `16+H` バイトを読んで `header_crc32` を確かめれば表示できる(ペイロードを伸長しない)。ヘッダ部(先頭から `header_crc32` まで)は約 260 バイト。
- 最小のコンテナ(TV-1)は 301 バイト。100年度のセーブは約 288 KB(§5.4)。

4.2 ヘッダ JSON

キー	型	説明
`kind`	string	`auto` / `manual` / `suspend` / `profile`
`slot`	int	1〜 `K_SLOTS` (`profile` は 0)
`lineage_id`	string	**歴史の識別子**(UUID v4)。新規ゲームの開始時に作り、そのスロットの一生のあいだ変えない(§9.1)
`save_counter`	int	そのスロットの**単調増加の保存番号**(1 から。§4.4)
`base_counter`	int	`suspend` のみ。中断した時点のスロットの先頭の `save_counter` (§4.6)
`schema_version`	int	ペイロードの構造の版(§6)
`app_version`	string	書いたアプリの版(診断用)
`master_hash`	string	マスタの内容のハッシュ(先頭8桁。診断用。§12.2)
`saved_at`	int	保存の日時(UTC 秒。**表示用**。ゲームの規則には使わない)
`year` / `month_index`	int	第N年度・月
`school_name` / `scenario_id` / `difficulty`	string / string / string?	校名・開始の種類(02 §3.12 `SaveMeta` と同じ)
`play_seconds`	int	プレイ時間(読み込んだあと、ここから数え続ける)

- `SaveMeta` (02 §3.12)は**このヘッダの部分集合**(`slot` `saved_at` `year` `month_index` `school_name` `scenario_id` `difficulty` `play_seconds`)。画面(SC-03・SC-40)はヘッダだけを読んで出す。
- 未知のキーは**読み飛ばして、書き戻すときも保つ**(同じ版のあいだの前方互換)。

4.3 読み込みの手順と失敗の種類

手順	確かめること	失敗のコード	利用者への見せ方(08 §7.7)
1	長さ ≥ 16、`magic`	`truncated` / `magic`	読み込めないセーブ
2	`container_ver ≤ K_CONTAINER_VER`	`container_too_new`	『アプリを更新してください』(開かない・消さない)
3	`header_len ≤ K_HEADER_MAX` 、`header_crc32` 、ヘッダの JSON	`header_len` / `header_crc` / `header_json`	読み込めないセーブ
4	`payload_len ≤ K_PAYLOAD_MAX` 、ファイルの長さ = `20+H+P+...` 、`file_crc32`	`payload_len` / `truncated` / `length` / `crc`	読み込めないセーブ
5	gzip の伸長(**伸長後が `K_PAYLOAD_MAX` を超えたら止める**)、JSON の解析	`payload_len` / `payload`	読み込めないセーブ
6	`schema_version ≤ K_SCHEMA_VER` (新しければ断る)、必要ならマイグレーション(§6)	`schema_too_new`	『アプリを更新してください』
7	検証と修復(§7)	`fatal:…`	読み込めないセーブ

- 手順1〜5 はヘッダ・全体の整合(CRC)で、**切り詰め・ビット反転・書き換えを必ず見つける**(検算: 切り詰め 127,682 通り・ビット反転 20,000 回・8バイトの書き換え 20,000 回で見逃し 0。CRC32 の見逃しは約 2^{-32})。
- 「読み込めないセーブ」になったときは、§4.5 の**回復の順**で次の候補へ進む。利用者に見せるのは、すべて失敗したときだけ。

4.4 A / B の2世代と原子的な書き込み

保存番号: `save_counter` = そのスロットの `{a, b, m}` の最大値 + 1。読み込んだセーブを続けて遊ぶときも、番号は**戻らない**(古いセーブを読み込んで進んだ先は、新しい番号になる)。

書き込み先: 自動セーブは A / B のうち、**壊れている・無い・保存番号が小さいほう**(同じなら A)。直前の世代(もう一方)は触らない。

手順(`IFileStore.WriteAtomic`)

1. 状態を DTO に写す(メインスレッド。安全な地点でだけ。下の注)。
2. ヘッダとペイロードをエンコードする(バックグラウンド可)。

3. `{target}.tmp` に書く → フラッシュ（`FileStream.Flush(true)` = fsync 相当）。
4. `.tmp` を**読み戻して全体を検証**する（§4.3 の手順1〜5）。失敗したら `.tmp` を消して失敗として扱う（§4.6）。
5. `.tmp` を `target` へ**置き換える**（Windows: `File.Replace`、その他: `File.Move(src, dst, overwrite: true)` = 同じボリュームの `rename`）。
6. `.tmp` が残っていれば消す。

読み込み: A / B のうち、**全体の検証（手順1〜5）に通るもので `save_counter` が最大のもの**。両方通らなければ回復の順（§4.5）。

注（安全な地点）: 状態の取り出し（手順1）は、Domain の呼び出しの**途中では行わない**。自動セーブのきっかけ（08 §6.5）はすべて『ある操作が終わった直後』なので、その地点で取り出す。書き込み（手順2〜6）の最中に次のきっかけが来たら、**待っている1件に畳む**（新しいほうだけを書く。キューの深さ 1）。

電源断の検算（TV-2）: 書き込みの各手順（`.tmp` の作成・半分・全部・検証後・置き換え後）で止めた 5 通りと最後まで書いた 1 通り、置き換え先を途中で切った 128 通り（長さ 0 から 997 バイトおき）の計 134 通りで、**直前の世代か新しい世代のどちらかが必ず読めた**（読めなかった = 0）。5回書いたあとの読み込みは、保存番号 5（世代 A）・壊れた世代 0。

4.5 手動セーブ・削除・回復の順

- **手動セーブ**: `slot{n}.m.eksv` に同じ手順（§4.4）で書く。保存番号は A / B と**同じ数列**（`{a,b,m}` の最大 + 1）。上書きは確認する（08 §7.6）。
- **スロットの削除**（確認つき。取り消せない）: `a` `b` `m` と、そのスロットの `suspend`（あれば）を消し、`LocalState.sync` に**墓石**を記録する（§9.1）。
- **回復の順**（スロットを開くとき。上から試し、最初に成功したものを使う）: ① A / B の最新の有効な世代 → ② もう一方の世代 → ③ 手動セーブ（利用者に確認して）→ ④ `premigrate_slot{n}_*`（最新）→ ⑤ `conflict_slot{n}_*`（`saved_at` の新しい順）。①で成功したが**最新の世代が壊れていた**ときは、『最新のセーブが壊れていたため、ひとつ前（第N年度 M月）から再開しました』を知らせる（トースト + 通知）。すべて失敗したら『このセーブは読み込めません』とし、**ファイルは消さない**（利用者が手で取り出せるように）。
- **バックアップからの復元**は**別の歴史として複製**する: 新しい `lineage_id` を振り、空きスロットへ書く（元の歴史と再び競合しないため）。

4.6 書き込みの失敗と中断セーブ

書き込みの失敗（ディスクの空き不足・権限・I/O エラー・検証の失敗）

1. `K_SAVE_RETRY_S` 秒待って、**1回だけ**再試行する。
2. だめなら、ゲームは続けたまま（メモリの状態は保つ）、トースト + 通知『自動セーブに失敗しました』（08a の通知の種類）を出し、**次のきっかけでもう一度**試す。
3. ほかのセーブ・退避を**自動で消して空きを作ることはしない**（空き不足は『空き容量が足りません』と案内する）。

中断セーブ（`suspend.eksv`。08 §6.5）

- 書くとき: アプリの中断（バックグラウンド・省電力・終了）の通知を受けた時点。**同期で**書く（iOS の猶予は短い。100年度でも Python 参照で 70〜85 ms。§14）。**圧縮は最速（zlib level 1。`Eksv.Encode(fast: true)`）**にする: 形式は同じで通常の読み込みで読める。M0 の実測（100年度の合成セーブ）は、通常 73〜114 ms・最速 31 ms（コンテナ 387 KB。通常 288 KB）。下位のスマホ（開発機の 5 倍遅いと見る）で、通常の圧縮は 200 ms を超えるが最速なら収まる（L-47）。
- 中身: ヘッダ（`kind = suspend`、`slot`、`lineage_id`、`base_counter`）+ ペイロード = **通常のセーブ本体**（中断の時点の状態）+ `suspend`。

```
suspend = { week?: true,                          # 本体は『練習が解決される直前』の状態で、練習の解決のあとの週の途中で中断した
            race?: { race_kind, race_id, next_section, started, touched,
                     instructions[区間ごと], geki_used[区間ごと: -1 未決 / 0 / 1], geki_spent,
                     swaps[[区間, 選手 ID], …] } }          # 08 §6.5 の『次の区間の指示の前』（自校の当日の決定。他校は固定の方針）
```

  週の途中（練習の解決のあと）の本体は、**『オーダー確定後』の自動セーブと同じ状態**（その週の練習を解決する直前。`GameFlow.ResolvingWeek` の時点）にしておく。再開時は、Domain がその週の練習を**同じ結果で解決し直し**（乱数は鍵つきで決

まる)、記録された指示・当日変更・檄で当日を戻して**区間 0 から `next_section - 1` までを再計算**する(乱数は大会・チーム・区間の鍵で決まり、指示や処理順で引き方が変わらない。04 §12)。

- **再開の条件**: `suspend.eksv` が有効で、`lineage_id` が一致し、`base_counter` が**そのスロットの現在の先頭の `save_counter` と等しい**。等しくなければ(他の端末で進んだ・自動セーブが先に進んだ)**破棄**してスロットの先頭を使う。
- **使い切り**: 再開した後の**最初の自動セーブが成功してから** `suspend.eksv` を消す(読み込み直後に落ちても、失わない)。
- 同期しない(端末ごとの一時的なもの)。

実装(M1 準備。`Persistence/Snapshot/SuspendSnapshot.cs`・`Domain/Game/RaceDayRecord.cs`・`GameFlow.ResumeWeek` / `ResumeRace`)

- 本体の取り方: アプリが `GameFlow.ResolvingWeek` で毎週 `GameSnapshot.Capture`(約 4 ms。JSON の文字列にするのは中断のときだけ)。同じ状態を『オーダー確定後』の自動セーブにも使える。`SuspendCheckpoint` がこの部品(`Attach` / `BuildPayload` / `Resume`)。
- どこで中断してよいか: ①週の初め・年度末の精算のあと・入学者発表のあと(`GameFlow.CanSave`)は、そのときの状態をそのまま(`suspend` は空)②練習の解決のあとの週の途中(`GameFlow.MidWeek` = 当日・レース進行中・走ったあとの結果・イベント・月末レポート)は、取っておいた本体 + `week` + 大会の週なら `race`。**停止のない週は『続ける』の中で進むので、中断する場面がない。**
- 再開: 週の初めの停止(年間方針・月間計画・エントリー・オーダーなど)は中断の前に済んでいるので出さず、練習の解決の直前の通知(自動セーブの時点)も出さない。走り終わったあとの記録なら当日の画面は出さず、結果とイベントの停止から続ける。
- 戻らないもの(L-62。**受け入れ**): その週に選んだイベントの選択肢(選び直す)・月末レポートの今月の前の週の分(流れの側の状態)。通常の読み込みも同じ(月の途中から復元したときは月末レポートの『変化』を出さない。`MonthReport.StartKnown`)なので、中断だけ特別に保存しない。状態(選手・資金・乱数)は変わらない。
- 検算: 当日の画面・区間 0〜最終・走ったあと・2 回続けての中断のどこで中断 → 再開しても、中断せずに続けた結果と週の終わりの状態がバイト一致(`SuspendRaceTests`。3 大会)/ 1 年度ぶんの週の途中の停止すべて(イベント・月末レポート・当日)で一致 / 当日の記録の各項目(檄の回数・指示・当日変更・通過した区間・檄の決定)を 1 つずつ壊すとテストが落ちる(変異の確認)/ コンテナ(`SlotStore.WriteSuspend`)経由で 200 ms 以内・再開の条件(`CanResume`)・自動セーブが進んだら破棄。

再開の検算(TV-3): 3大会 × 300 レース(12 チーム)で、区間 k の前に中断 → 再開後に k 以降の指示を**変えても**、完了した区間(14,196 区間)の ds・累積は**1つも変わらなかった**。

---

5. セーブの中身とサイズ

5.1 構造(02 §4.9)に対する変更

項目	02 v0.9	09 での確定
`rng`	`{ algorithm, state }`	**`{ algorithm: "pcg32-keyed-1", master_seed }`**。鍵つきで状態を持たない(§8)。`master_seed` は uint64(JSON では文字列で書く。§5.3)
`SaveMeta`	別の項目	**ヘッダの部分集合**(§4.2)。ペイロードには持たない
`lineage_id` / `save_counter`	なし	**ヘッダ**に持つ(ペイロードには持たない)。同期・世代の判定はヘッダだけで行う
`suspend`	08 §13.4 の形(要点のみ)	§4.6 の形。**通常のセーブには入れない**(`suspend.eksv` のペイロードにだけ付く)
`history.race_records`	全チームの区間	**自校の結果は全区間、他校は順位・合計タイムだけ**(§5.2)
`ob_records`	他校の OB も	**自校の OB のみ**(§5.2)
`tickets.log` / `event_state.log`	上限なし	各 50 件(`K_TICKET_LOG_MAX` / `K_EVENT_LOG_MAX`)
`pb`	派生	保存しない(08 R-03。02 §4.1 のとおり)
`Runner.style` / `gifts` / `gift_revealed`(10)	なし(02 v1.1 で追加)	在籍中の選手と OB の記録に持つ。`style` は 3 文字(例 `SPC`)、`gifts` は文字列の配列。**入学前の候補には保存しない**(`master_seed`・`look_seed`・`profile` から決定的に作れる。10 §4)。派生値(`gm` など)は保存しない

5.2 刈り込みの規則(サイズを年数に比例させない)

規則	内容	理由
他校の OB は保存しない	他校の卒業生は、卒業した時点で破棄する(世界の更新は **校ごとの要約**(`prestige`・`ob` の人数など)だけで足りる。05 §9)	100年度で他校 OB は 3.5万人。保存すると 25 MB になる
他校の大会記録は要約	`race_records` の他校は `{ school_id, rank, total_ds }` だけ。区間・個人の記録は、**自校の結果と、区間記録・大会記録(`course_records` / `team_records`)の更新分**だけ残す	履歴の増え方を 1 年あたり約 7 KB に抑える
ログは新しい順に上限	`tickets.log` 50 件、`event_state.log` 50 件、通知 30 件(`K_NOTIFY_MAX`)	増え続けない
自校の OB は保存	能力・外見・名前・出身・主な記録(約 200 バイト/人。02 §4.7)。殿堂入りも残す	歴史・殿堂・レジェンド(M-01)のため
候補は 1 年度ぶんだけ	`candidates` は入学の確定後に破棄(翌年度の候補は、世界の更新で作る。05 §6)	

> **刈り込みなしの場合**(検算): 100年度で JSON 25,136 KB・gzip 3,213 KB、200年度で JSON 49,765 KB・gzip 6,306 KB。**上限 3 MB を超える**ので、刈り込みは必須。

5.3 JSON の規則(決定的にするため)

項目	規則
エンコード	UTF-8、BOM なし。空白・改行なし(コンパクト)
キーの順	**宣言順**(DTO のプロパティの順)で固定。辞書(`map`)は**キーの昇順**(整数キーは数値順、文字列は序数順)
整数	10進。`ds` ・ID・カウンターは 32bit 整数。`master_seed` と `look_seed` は **uint64 / uint32** → JSON では**文字列**(`"123456789012345"`)で書く(JavaScript 系のパーサで丸められないため)
double	**状態に書き戻す時点で小数 6 桁に丸める(`Q6`)**。書式は小数 6 桁まで・末尾の 0 を省く・指数表記なし(例 `0.5` / `12.345679` / `0`)。対象は `fame_hist` `base_prestige` `coach_q` `a0` `gap` `prog` と、`prestige` (世界の更新は連続値で動かす。02 §4.3・L-45)
文字列	名前・かなは UTF-8 のまま(`\u` エスケープは使わない)。制御文字は禁止
真偽・null	`true` / `false` / `null` 。値が既定のものも**省略しない**(読み込み側の既定に頼らない)
版	ペイロードの先頭のキーは `schema_version`

Q6: `q6(x) = floor(x × 10⁶ + 0.5) / 10⁶` (0.5 は切り上げ。02 §1 の丸め)。整数を 10^6 で割る IEEE の除算は最近接に丸められるので、結果は『10進6桁の表記の最近接の double』と一致し、**書式 → 読み戻しで必ず同じ値に戻る**。Domain は上の6項目に代入するとき `Q6` を通す(読み込み後の状態が、メモリの状態と一致する = ロードしても結果が変わらない)。検算(TV-4): 乱数 1,000,000 個(大きさ 1〜10^9、正負)で、書式 → 読み戻しの不一致 0・`Q6` が冪等でない 0。端の値: `5e-07 → 0.000001` / `1e-07 → 0` / `-0.0 → 0` / `0.1+0.2 → 0.3` 。

5.4 サイズの見積もり(`ref_model_09.py size`)

02 §4.9 の構造に沿った合成データ(自校の部員 40、他校 35 校 × 36 人、入学候補 520、**名前・能力はランダム = 圧縮に不利な側**)。

年度	JSON (KB)	gzip (KB)	比	刈り込みなしの gzip (KB)
1	598	123	4.9	153
10	710	138	5.1	—
25	899	163	5.5	—
50	1,224	205	6.0	—
100	**1,852**	**288**	6.4	3,316
200	3,145	456	6.9	6,509

- 100年度の内訳(JSON): `history` 741 KB・`ob_records` (自校のみ)524 KB・`rival_runners` 363 KB・`candidates` 139 KB・`runners` 41 KB・`schools` 20 KB・`high_schools` 13 KB・`event_state` 7 KB。

- 判定: 100年度の gzip **288 KB は目標 1 MB(`K_SIZE_TARGET`)・上限 3 MB(`K_SIZE_HARD`)のどちらも下回る**。増え方は 1 年度あたり約 1.7 KB(gzip)。200年度でも 456 KB。個性・素養のフィールド(10)を入れた分は、100 年度で +3 KB(gzip)。
- 実データは名前・能力が偏るので、圧縮はこの見積もりより効く見込み。**M2 で 50 年度連続のセーブを実測して確定する**(01 §13)。

5.5 容量の予算

項目	見積もり(100年度)	備考
1スロット = A + B + 手動	約 0.87 MB	288 KB × 3
3スロット + 中断 + プロフィール	約 2.9 MB	プロフィールは数 KB
退避(競合・移行前)	最大 5 件 × 3 スロット = 約 4.3 MB	`K_BACKUP_MAX`
合計の上限の目安	約 7.2 MB(8 MB で見積もる)	端末の空きが 50 MB 未満のときは、設定の『データ』に警告を出す(§4.6)

クラウドへ載せるのは、スロットごとに **先頭の1世代 + 手動 = 約 0.57 MB**(3スロットで約 1.7 MB)。1つの記録は `K_SIZE_HARD` (3 MB) に収まる。

5.6 自校の保存の形(M0 の追加。`TeamSnapshot`・`ref_model_09.py team_save`)

02 §4 は項目の意味、ここは**保存するときの鍵・型・順・既定値**の正本。自校(育成の側)の保存が持つ鍵と、世界の側の鍵は次のとおり分ける(キーの順はこの表の順)。

鍵	持つ側	内容
`schema_version` `scenario` `run` `clock` `rng` `counters`	自校	下の『小さな鍵』
`schools` `rival_runners` `candidates` `history` `high_schools`	世界(M3)	自校は持たない。検証器に通すときは呼ぶ側が `schools`(自校の名簿)と `rival_runners` を足す
`player_team` `runners` `ob_records` `event_state` `settings`	自校	下の各表
`pack_snapshot`	アプリ	セーブ時に所持していたパック(参考)

取り出しのタイミングと派生値: 取り出しは**週の境目**(1 週の処理が終わり、次の週の `Refresh` の前)。派生値は保存せず、復元でも作り直さない(**次の週の最初に `Refresh` を呼ぶのが約束**)。例外として、`far` / `near`(出身県と `player_team.school_prefecture_id` から)と素養の乗数 `gm` / `bond_k` / `ind_mult`(`style` と `gifts` から)は復元で作る。保存しない派生値: 力の順位・グループ内の絆の平均・レギュラー・素質との差・`pb`(02 §4.1)・1 週の途中にだけ意味を持つ大会の結果(`race_flags`)。

小さな鍵

鍵	形
`scenario`	`{ id, difficulty(Elite / Mid / Low / Club / null), player_school_id }`
`run`	`{ invites_declared, cleared(null / { year, no_invite }) }`
`clock`	`{ year(1〜), month_index(0〜11), week(1〜4) }`。通算週 = `(year-1)×48 + month_index×4 + (week-1)`(02 §1)
`rng`	`{ algorithm: "pcg32-keyed-1", master_seed(文字列) }`
`counters`	`{ next_runner_id, next_staff_id }`
`settings`	`{ spectate_mode(Full / Digest / ResultOnly), auto_safety_f, event_auto, plan_auto, scout_auto, school_look: { uniform_pattern, cheer_song, bgm_set } }`

`runners`(自校の在籍選手。キーは ID を 10 進で書いた文字列、**ID の昇順**)の 1 人

`id` `family_name` `given_name` `family_kana` `given_kana` `origin` `school_id` `enroll_year` `grade` `growth_type` `stats[10]` `potential[10]` `xp[10]` `skills` `fatigue` `form` `taper_weeks` `sharpen_bits` `motivation` `injury` `group` `bonds` `traits` `is_captain` `recruit_source` `ticket_kind` `career` `ctr` `monthly` `appearance` `appearance_override` `look_seed` `style` `gifts` `gift_revealed`(この順)。

項目	形・規則
`origin`	**入れ子**: `{ origin_type, prefecture_id, high_school_id, country_id }`（Japan: 県と高校、country は null / Overseas: country だけ）。02 §4.1 は平坦に書いたが、保存は入れ子（検証器 I-02 がこの形を読む。L-48）
`enroll_year` / `grade`	どちらも保存（`grade` は検証器が読む）。入学は年度の最初の週なので、復元の入学週 = `(enroll_year-1)×48`
`skills`	文字列の配列、**序数順**（獲得順は持たない）
`sharpen_bits`	直近 3 週に `ks.sharpen` を含んだか。bit0 = 3 週前、bit1 = 2 週前、bit2 = 前週
`injury`	`{ severity, weeks_left, age }`。`age` = 故障してからの週（故障がなければ 99。06 §3.2 の `injury_age`。追加項目）
`group`	`A` / `B` / `C`
`bonds`	`[[相手の ID, 絆], …]`。**ID の小さい側の選手にだけ**持ち、相手の ID の昇順。0 の組は持たない（I-08）
`traits`	文字列の配列（最大 3）
`is_captain`	主将の写し。**正は `player_team.captain_id`**（検証の修復は `captain_id` だけを直すため）
`ticket_kind`	`recruit_source = Ticket` のとき `Intl` / `Rookie`、それ以外は null
`career`	`{ races, section_wins, relay_wins, brakes, records }`（`records` は追加の項目。ないセーブは 0。09 §6）
`ctr`	`{ 名前: 値 }`、**名前の序数順**。名前は 06 §10 の一覧のみ
`monthly`	`{ gain, high_fatigue_weeks, overload_pending, camp_max_fatigue, stats_start, stats_prev }` = 月の途中経過と夏合宿の記録（03 §7.2 のやる気の規則 M1・M6、06 §10 の `camp_overload`）。月末に使って 0 に戻す。月の途中の週の境目で保存すると要る（追加項目。M1 準備）。`stats_start` / `stats_prev` = 今月の初めの能力と前の月の初めの能力（10 要素。null = まだ月を送っていない。画面の『増減』= 08b G-14。**追加の項目**: ないセーブは null）
`appearance` / `appearance_override`	`{ hair_style, hair_color, face, body, skin_tone }` / null または変えた項目だけの `{ hair_style?, hair_color? }`
`look_seed`	uint32 を 10 進の数で
`style` / `gifts`	3 文字 / 素養の配列（並びは保つ）。`style` が読めなければ中立 `BFE`（I-21）

`player_team`

`funds` `facilities` `staff` `morale` `captain_id` `manager_name` `annual_plan` `monthly_plan` `custom_weeks` `guidance_points` `individual_menu` `talked` `scout_points` `coach_rank` `ob_network` `tickets` `prestige` `year_rank` `last_year_rank` `year_points` `school_prefecture_id` `entries` `notifications`（この順）。

項目	形・規則
`facilities`	`{ Dorm, Track, Hill, Gym, Medical }`
`staff`	`[{ id, role(Coach / Trainer / Nutritionist / Scout), name, level, salary }]`（この 4 つ。`level` が施設・スタッフの 8 項目のうちの 4 つ、`salary` は `level` から導出 = 05 の `EXP_STAFF[level-1]`。読み込みでは無視）
`annual_plan`	`{ focus(Balanced / Speed / Long), target_race(Fall / National / NewYear), peaking_mode: { Fall, National, NewYear, Qualifier, Meet } }`（値は Standard / Short / None）
`monthly_plan`	`{ A, B, C }` = 今月のテーマ
`custom_weeks`	null または `{ A: [週 ×4], B, C }`。週 = `[ks1, ks2, 走行量]`
`individual_menu`	`{ 選手 ID: { until_abs_week, menu: [ks1, ks2, 走行量] } }`（ID の昇順）
`talked`	選手 ID の昇順
`entries`	`{ Fall, National, NewYear }` = 次の大会の登録メンバー（選手 ID の配列。登録の並びを保つ。04 §4。締切は大会の 4 週前、大会が終わったら空にする）。ピーキング（03 §5.3）は登録メンバーに掛かるので、締切から大会までの間の保存で要る（追加項目。M1 準備）
`notifications`	通知の履歴（新しい順・最大 `K_NOTIFY_MAX` = 30 件。08 §7.5・08a SC-38）: `{ kind, abs_week, code, runner_id, n, unread }` の配列。`kind` = `Injury` `Warning` `Event` `Ticket` `Schedule` `Entitlement` `AutosaveFailed`。文は `code` から画面が文字列表（`nt.*`）で組む。**追加の項目**: ないセーブは空

項目	形・規則
`ob_network`	長さ 48(添字 0 = 海外、1〜47 = 県)。**OB の記録から数えた派生値**(`career_path` が確定したものだけ。I-16)
`tickets`	07 §3.1。`{ stock, bank, last_used_year, plan_anchor_year, plan_granted_years, milestones(序数順), pending(null / { kind, enroll_year, prefecture_id? }), log(最新 K_TICKET_LOG_MAX 件) }`。`invites_declared` は `run` に置く
`morale` `scout_points` `prestige`	数。Q6 (整数のときは整数で書く)。02 の int は実際には小数にもなる値
`year_rank` `last_year_rank` `year_points` `prestige` `school_prefecture_id`	**暫定**: 履歴(`history`)や世界の `schools[自校]` ができるまで、自校が持つ写し(L-49)

`ob_records` (自校の卒業生。卒業した順)の 1 人

`runner_id` `name` `kana` `origin_index` (1〜47 = 県、0 = 海外。`ob_network` の添字と同じ) `origin_country` (留学生の出身国。1〜24 = 国の表の添字 + 1、なければ 0。**追加の項目**: ないセーブは 0) `high_school_name` `school_id` `enroll_year` `grad_year` `final_stats[10]` `skills` `pb: { t5000, t10000, t_half }` `career: { race_wins, section_wins, records }` `career_path` (Corporate / Teacher / Employment / GradSchool) `hall_of_fame` `role` (None / Coach / ScoutHelper / Sponsor) `role_until` `recruit_source` `appearance` `style` `gifts`。

- 06 §8 の『卒業時の力』は `P_long(final_stats)`(`power`)。保存せず、`final_stats` から作る。`career_path` が null(確定前)の古い記録は一般就職として読む。
- **期限の切れた役割**(`role_until` ≤ 現在の通算週)は `None` として書く(次の週の `ExpireTimed` と同じ。I-19)。

`event_state`: `fired` (ID の序数順) `year_counts`(`_total` を含む。序数順) `runner_ev` (選手 ID の昇順。**全員**ぶん) `pair_cd` (キー `"イベントID|小さい側の選手ID|大きい側"`、序数順) `mods`(`{ kind(E_mult / r_add / inj_mult / form_add / funds_year / scout_pts), to("team" または選手 ID), value, until, src }`。**期限の切れたものは書かない**。I-19) `bench` `forced` (ID の昇順) `log` (最新 `K_EVENT_LOG_MAX` 件。古い順)。

検算: ① `team_save`(`ref_model_09.py`)が書いた自校の保存 150 件が、参照モデルの検証器を**修復なし**で通り、C# が**復元 → 取り出しで鍵の順も含めて同じ JSON** に戻す(`PersistGoldenTests`)② 試験用の週ループ(40 人の新規ゲーム → 6 年度。イベント・故障・個別指導・面談・欠場・追加登録・招聘・卒業と入学を通す)で、3 週ごとの取り出しが検証器を修復なしで通り、**どの週(40 か所)で保存 → 読み込み → 続行しても、続行のみとバイト一致**する(コンテナ経由・毎週の読み込み直しも。`TeamSnapshotTests`)③ 復元で項目を 1 つ落とす変異(故障後の週・直近のイベントの月・士気・欠場・追加登録・面談・券の使用年度・性格タグ・ペアのクールダウン)は、試したものすべてでテストが落ちる。

ゲーム本体の保存(`GameSnapshot`。M1 準備): 実体の選手を持つ自校 + 36 校の世界(`GameSession`)は、`{ schema_version, world, team, session }` の 1 つの JSON にする。`world` = `WorldSnapshot`(自校は `is_player` の学校として施設・資金・累計ポイントだけを持ち、名簿は空)、`team` = `TeamSnapshot`(上の形)、`session` = `{ year_wins(今年度の優勝の bit)、race_power(第三戦の週の戦力。Q6)、next_ticket_kind、year_settled(年度末の精算まで済んだ = 強化の画面)、scout(今年度のスカウトの行動 [[候補の番号, 段階]]。段階 1 視察 / 2 声かけ / 3 練習会)、guided(今週、個別指導のメニューを指定した選手 = 取り消しでポイントが戻る)、orders(提出された区間オーダー。第一〜第三戦の 3 要素で、各々 null = おまかせ / 区間 → 選手 ID の配列)、qualifier_entries(予選会の登録 12 名。null = おまかせ)、stats(年度ごとの記録)、year_result(null / 年度の途中の { year, qualified, order: { fall, national, newyear } }。終わった大会だけ。**予選会も大会もまだ行われていない年度は null**: 画面が先に年度の準備 PrepareYear を呼んだかどうかで、保存の中身が変わらないようにする)}`。

- 取り出しは**週の境目**。`GameSession` の週の境目は、**次の週の最初の処理(`BeginWeek`: 前年度の記録を送る・月初のポイント・期限切れの補正・派生値・締切の週のおまかせ登録)まで済んだ状態**(画面の登録・月間計画はここから開く)。復元は派生値だけ作り直す(`Refresh`。L-51)。
- 年度末は、精算のあと(`year_settled` = true。強化の画面)と入学者発表のあとでも保存できる。精算のあとは学校の状態が変わっているので、復元は `BeginYear` をやり直さず、後半(成長 → 卒業 → 入学)に要る最小の年度の状態だけ作る。
- **年度の途中の出場権・他校の大会用の選手は保存しない**(年度の途中では変わらない)。復元は `BeginYear` をやり直して `year_result` を戻す。

- 自校の資金・施設は `team` が正。`world` の自校の写し（`funds` `facilities` `scout_lv`）は週の終わりごとに揃えてあり、復元でも `team` から揃え直す。
- **試験**: 4 開始校 × 2 シード × 6 年度で、年度の途中（大会の前後・予選会・年度末・入学の前後）を含む『保存 → 読み込み → 続行』が『続行のみ』とバイト一致（JSON 経由・コンテナ経由・毎週の読み込み直し・並列）。45 年度で JSON 約 550 KB（OB 450 人を含む）。

---

6. バージョニングとマイグレーション

6.1 規則

項目	規則
版の置き場所	ペイロードの `schema_version`（先頭のキー）とヘッダの `schema_version`。アプリは定数 `K_SCHEMA_VER` を持つ。両者が食い違うセーブは致命（`header_json`）
破壊的な変更（意味の変更・削除・改名・型の変更・列挙の読み替え）	**版を上げ**、`vN → vN+1` の**移行関数**（JSON → JSON の純関数）を書く。移行は連鎖して適用する（v0 → v1 → v2 …）。飛ばさない
追加だけの変更（既定値のある新しい項目）	版は**上げない**。版ごとの**既定値の表**（`DEFAULTS_Vn`）を、読み込みのたびに最後に適用する（足りない項目だけ補う。既存の値と未知の項目はそのまま）。書き込みは常にすべての項目を書く
新しい版を読んだとき	`schema_too_new`。**開かない・直さない・消さない**（『アプリを更新してください』）
同じ版のあいだの前方互換	**未知のキーは読み飛ばして保ち、書き戻す**（DTO の拡張データ。`[JsonExtensionData]`）。新しいアプリの版（同じ `schema_version`）で保存したセーブを、古いアプリの版で開いて保存し直しても、項目が消えない
移行前の控え	移行が必要なセーブを読み込んだとき、**最初の自動セーブの前に**元のファイルを `backups/premigrate_slot{n}_v{k}.eksv` へ複製する（スロットごとに最新の1つ）
移行後の書き込み	新しい版は**もう一方の世代**に書かれる。古い版のファイルは『1つ前』として残る（アプリを戻しても、1つ前は読める）
検証	移行の直後に、§7 の検証と修復を行う

> 02 §4.9 の『変更のたびに増やし』を、上の『破壊的な変更のとき増やし、追加は既定値の表』に改める（§18）。

6.2 v0 と v1

- **v1 = 製品の初版。v0 = 製品の前の開発中の形**（02 v0.8 までの構造。M1 の試作版のセーブを読むために持つ。製品の利用者には v0 のセーブは存在しない）。
- 移行関数と既定値の表は**データとして文書・コードの両方に残す**（§6.3）。次に版を上げるときも、この形に足す。

6.3 v0 → v1 の内容（08 の確定に伴う変更）

対象	変更	種類
`player_team.annual_plan.focus`	`Balanced → Balanced` / `Speed → Speed` / `Endurance → Long` / `Mountain → Balanced`（重点を3つに絞った。08 R-04）	破壊的（読み替え）
`player_team.annual_plan.target_race`	無ければ `NewYear`	追加
`runners[*].pb`	削除（派生で保存しない。08 R-03）	破壊的（削除）
`player_team.manager_name`	無ければ `監督`（08 R-13）	追加
`player_team.individual_menu` / `talked`	無ければ `{}` / `[]`（08 R-06）	追加
`settings.plan_auto` / `scout_auto`	無ければ `false`（08 R-04・R-08）	追加
`schema_version`	`1`	—

検算(TV-5): ① `Endurance → Long` ② `target_race` の既定 ③ `pb` の削除 ④ `manager_name` の既定 ⑤ `individual_menu` / `talked` の既定 ⑥ `plan_auto` / `scout_auto` の既定 ⑦**既存の値(`event_auto`)を保つ** ⑧**未知の項目を保つ** ⑨ `schema_version = 1` ⑩**冪等**(v1 を再度移行しても変わらない。適用ログ `['v0→v1']` / `[]`)⑪新しい版(v2)を `schema_too_new` で拒否 ⑫ランダムな旧データ 2000 件が例外なく v1 になり、不整合 0。すべて ○。

---

7. 読み込み時の検証と修復

セーブは自分のコードが作るので、不整合は**自分のバグ・版の差・手での編集**のどれかでしか起きない。読み込みのたびに、移行(§6)の直後に検証する。方針: **直せるものは決定的に直して続ける。直すと『データを作ってしまう』ものは致命にして、1つ前の世代へ進む(§4.5)。**

7.1 検証の分類(02 §5 の不変条件ごと)

分類	意味	処理
R 修復	範囲外・派生値の食い違いなど、決定的に正しい値へ直せる	直して続ける。修復の一覧を診断ログに残す。利用者には**知らせない**(ただし 1 件でも直したら、次の自動セーブは**修復後の状態**を書く)
F 致命	直すと選手・所属・出身などを『作る』ことになる	読み込みを失敗にし、回復の順(§4.5)で次の候補へ。ファイルは消さない
W 警告	生成時の検証や、一時的な状態の整合。読み込み時には直さない	診断ログに残すだけ(規則の側で、次の機会に整う)

ID	検査	分類	修復の内容
I-01	`0 ≤ stats[i] ≤ potential[i] ≤ 100`	R	`stats` を 0〜100 に収め、`potential` を `[stats, 100]` に収める(素質を**引き上げる**。能力を下げない)
I-02	出身の整合(日本 ⇒ 県1〜47・高校あり・国なし / 海外 ⇒ 国あり・県なし)	**F**	—
I-03	在籍中の選手の学年 = `current_year - enroll_year + 1` が 1〜4(派生。保存は `enroll_year`)	**F**	—
I-04	`injury.severity = None` ⇔ `weeks_left = 0`	R	重症度ありで残りが 0 以下 → `None`・0 / `None` で残りが 0 でない → 0
I-05	`fatigue ∈ [0,100]`、`motivation ∈ [1,5]`、`form ∈ [-2,+2]`	R	範囲に収める
I-06	1人は高々1つの大学に所属	**F**	—
I-07	自校の在籍 ≤ 60	**F**	—
I-08	絆は ID の小さい側にだけ・相手が存在・0 < 値 ≤ 1000	R	向きが逆なら小さい側へ移す・存在しない相手は捨てる・値を 0〜1000 に収める(0 は捨てる)・重複は大きいほう
I-09	`captain_id` は自校の在籍選手で `grade ≥ 3`	R	`null` (主将なし。`int?` なので有効な状態)
I-10	エントリーした選手は自校の在籍・留学生の上限以内	W	エントリーを破棄し、次のエントリー画面でやり直す(保存されるのは大会の前後だけ)
I-11	実行時 ID は一意・再利用されない(辞書のキー = `id`、他校の主力との重複なし)	**F**(重複・不一致)/ R(カウンター)	`next_runner_id` を `max(ID) + 1` 以上に引き上げる
I-12	`funds ≥ 0`、施設レベル 1〜5	R	0 に・範囲に収める
I-13	他校の `prog ∈ [0, 0.95]`、`roster ≤ 40`	R(`prog`)/ **F**(人数)	`prog` を収める
I-14	券: `stock ∈ [0,2]`、`stock < 2 ⇒ bank = 0`、合計を保つ	R	**合計(`stock + bank`)、負は 0)を保ったまま**振り分け直す(失効させない)
I-15	招聘選手の生成値が帯(07 §6.6)を満たす	W	生成済みの選手は直さない(規則の側で、生成時にだけ検証する)

ID	検査	分類	修復の内容
I-16	`ob_network` の合計 = 自校の OB で `career_path` が確定した人数	R	OB の記録から**数え直す**(派生値)
I-17	上位級の入学人数 ≤ 奨学金枠	W	年度末の処理の中だけの整合。保存の時点では存在しない
I-18	`invites_declared` ≥ 招聘で入学した人数。`cleared` は変えない	R	`invites_declared` を人数まで引き上げる(`cleared` は触らない)
I-19	切れた補正(`until` が過去)は消す。`Runner.ctr` のキーは 06 §10 の一覧のみ	R	切れた補正を捨てる・一覧にないキーを捨てる
I-20	`guidance_points` ∈ [0,4]・`scout_points` ≥ 0・`individual_menu` と `talked` は自校の在籍選手(個別指導の対象は故障なし)	R	範囲に収める・存在しない/故障中のキーを捨てる
I-21	`style` は 3 文字(型 ∈ {S,B,L}・走法 ∈ {P,F,S}・展開 ∈ {C,E,L})。`gifts` は {Climber, Descender, Ace} の重複なしで 2 個以下(在籍中の選手と OB の記録)	R	`style` が不正なら中立(`BFE`)に・`gifts` は有効なものだけを重複なしで残し、3 個以上は先頭の 2 個に

- **必須の項目が欠けている**(`schema_version` `clock` `player_team` `runners` `schools` `scenario` `rng` のどれか)・辞書でない・`rng.algorithm` が未知 ⇒ **致命**。
- **修復は冪等**: 修復後のセーブをもう一度検証しても、修復は入らない。**正常なセーブは1項目も変わらない**。

7.1b 実際のセーブの形への掛け方(L-66 ①)

実際のセーブ(`GameSnapshot` = `world` / `team` / `session`)は、仕様の `SaveData` の形を**自校の半分**(`team` = `clock` `rng` `scenario` `run` `counters` `player_team` `runners` `ob_records` `event_state` `settings`)と**世界の半分**(`world` = 他校の名簿・候補・履歴)に分けて持つ。検証もそれに合わせる(`GameLoader.Load`)。

1. `team` に `SaveValidator.ValidateAndRepairTeam`(I-01〜I-21 のうち自校に関わるもの。必須項目は `schema_version` `clock` `player_team` `runners` `scenario` `rng`。他校の項目 `schools` `rival_runners` がなければ、その検査と修復は省く。自校の在籍が 60 人を超えれば致命 = I-07)。致命 → 失敗(回復の順で次の候補へ)/ 修復あり → 修復後の `team` で続ける。
2. 復元(`GameSnapshot.Restore`)。形が合わず例外になるものは失敗。
3. 世界の半分は復元してから `WorldInvariants`(W-1〜W-4: 学校の数・名声の範囲・名簿の人数と学年・選手の範囲)で検査し、違反があれば失敗(世界は『直す』と選手を作ることになるので修復しない)。
4. 修復したら、次の自動セーブは**修復後の状態**を書く(§7.1)。中断セーブからの再開(`SuspendCheckpoint.Resume`)も同じ関門を通り、再開した流れがもう一度中断しても修復後の本体を使う。

正常なセーブは 1 項目も変わらない(複数の開始校・シード・6 年度の約 350 の時点で、修復 0・本体が同じ)。修復の一覧は `OpenResult.Repairs`(診断ログ用)。

7.2 検算(TV-6。`ref_model_09.py validate`)

独立に書いた検査(`check_invariants`)と、修復(`validate_and_repair`)を、ランダムなセーブ(自校 40 人・2 校・OB 30・補正・券)で確かめた。

試験	結果
正常なセーブ 300 件	修復が入った件 = **0**(内容も変わらない)
壊したセーブ 3,000 件(1〜6 か所。修復で足りる操作 24 種 + 致命にすべき操作 6 種を混ぜた)	修復で足りる 1,506 件: 修復後も不変条件に違反 = **0**、2 度目の修復で変化 = **0**
同上(致命にすべき 1,494 件)	致命にした = **1,494**、通してしまった = **0**、修復で足りるのに致命にした = **0**
必須項目の欠落 7 種	致命として拒否 = **7 / 7**
修復の内訳(件数)	I-01 390 / I-04 340 / I-05 549 / I-08 534 / I-09 128 / I-12 338 / I-13 186 / I-14 155 / I-16 181 / I-18 191 / I-19 171 / I-20 425 / I-21 320 / カウンター 150

> 検算の範囲: 上の表の I-10・I-15・I-17(W)は**規則の側の整合**なので、参照モデルでは扱わない(M0 の Domain のテストで、各規則の手順の中で検証する。§16)。I-02・I-03・I-06・I-07・I-11・I-13 の F は、**実装でも『検出して致命』だけ**を満たせばよい(直し方を持たない)。

8. 乱数と再現性

8.1 方式

- 乱数は **03 §3.1 の PCG32 を、鍵から作るストリームごとに使う**(状態を持たない): `stream = Pcg32(derive(master_seed, label, a, b))`。`mix64`・`derive`・`Pcg32` は**整数演算だけ**で、全端末で同じ結果になる(03 §3.1、適合ベクタは TV-7)。
- `SaveData.rng = { algorithm: "pcg32-keyed-1", master_seed }`。**乱数の状態は保存しない**。`master_seed` は uint64(JSON では文字列。§5.3)。`algorithm` が未知なら読み込みは失敗(§7)。
- 帰結: ①**どのタイミングで保存・再開しても結果が変わらない**(保存すべき乱数の状態がない)②**ロードし直しても、入力(オーダー・メニュー・指示)を変えなければ、同じ結果になる**(結果を引き直すためのセーブ&ロードはできない。08 §6.5)③ユニット(選手・週・区間・候補・学校)ごとにストリームが独立なので、**処理の順序や人数が変わっても、他のユニットの結果は変わらない**。
- 整数の `a` `b` は **int64** として渡し、`& (2⁶⁴-1)` で符号なしにする(負の年度を使うため。§8.2 の予熱)。

8.2 鍵の登録表

`race_id = 年度 × 10 + 種別`(第一戦 1 / 第二戦 2 / 第三戦 3 / 予選会 4。04 §12)。『予熱の年度』(新規ゲームの世界生成で最初に回す 15 年度)は**年度を −14〜0** として、通常と同じ鍵の形を使う(自校がないので、ラベル 1・2・5 は使わない)。

label	a	b の範囲	用途	出典
1	`runner_id`	0〜2^{31}	週ごとの故障(b = 通算週)	03 §3.2
2	`runner_id`	0〜2^{31}	週ごとの調子(b = 通算週)	03 §3.2
3	入学年度	0〜99	入学候補の年ごとの共通の抽選(0: 地方・型の割合 / 1: プールの出身県の抽選)	09
3	入学年度	100〜199	他校の狙い撃ち(100 + 学校ID)	09
3	入学年度	1000〜1999	候補1人の生成(1000 + 候補の番号)	09
3	入学年度	9000〜9999	特別招聘の選手(9000 + index)	07 §6.2
3	入学年度	10000〜59999	割り当ての提示スコアの乱数(10000 + 候補の番号 × 64 + 学校ID。候補 ≤ 780)	09
4	race_id(第一〜第三戦)	0〜3599	区間(チームID × 100 + 区間番号)	04 §12
4	race_id(第一〜第三戦)	8000〜8035	欠場の抽選(8000 + チームID)	04 §12
4	race_id(第一〜第三戦)	9000〜9035	他校の調子の抽選(9000 + チームID)	04 §12
4	race_id(第一〜第三戦)	9998	予報の誤差	04 §12
4	race_id(第一〜第三戦)	9999	環境	04 §12
4	race_id(予選会)	0〜3599	予選会の選手(チームID × 100 + 選手の番号)	04 §12
5	通算週	0	イベントの発生の判定	06 §7.5
5	通算週	1000〜1099	k 番目に発生したイベントの効果(1000 + k)	06 §7.5
6	年度	0	世界の更新の共通の抽選	09
6	年度	100〜199	学校ごとの更新(基準名声のゆらぎ・指導体制。100 + 学校ID)	09
6	年度	1000〜1099	学校ごとの入学・名簿の調整(1000 + 学校ID)	09

label	a	b の範囲	用途	出典
7	段階(0 学校 / 1 高校 / 2 初期の名簿)	0〜999	新規ゲームの世界の生成(0: b = 学校ID / 1: b = 高校ID / 2: b = 学校ID × 4 + 学年 − 1)	09
8	`look_seed`	0〜3	名前(姓・名・姓のかな・名のかな)	09
8	`look_seed`	4〜8	外見(髪型・髪色・顔・体型・肌)	09
8	`look_seed`	10〜19	10能力の展開の偏り `n10`(05 §6.2。b = 10)・視察の誤差(b = 11。`Q_obs` の雑音。05 §7.2)・入学時の素質の雑音(b = 12。10 個の N(0,3) を能力の順に。05 §6.2)	09
8	`look_seed`	20〜29	性格タグ(20: 個数 / 21〜: 選択。08 R-16)	09
8	`look_seed`	30〜39	名前の重複を避ける再抽選(30 + 回数)	09
8	`look_seed`	40〜49	個性と素養(40: 型・走法・展開の 3 回 / 41: 素養の個数・種類の 3 回)	10 §4
9	年度	0	自校(実体の選手)の年度末の抽選: 卒業生を ID の昇順に 2 回ずつ(実業団か・その他の進路の分け)、続けて やる気 1 の選手を ID の昇順に 1 回ずつ(退部)。年度 0 = 新規ゲームで部員数を指定したとき、名簿から残す人を選ぶ入れ替え	05 §10.2・§10.4

- **規則**: ①同じ `label` で `a` の意味が同じなら、`b` の範囲は**重ならない** ②新しい用途は、**この表に足してから**使う(表にない鍵は使わない)③`label` 9 以上は予約(拡張)。
- ラベル 3(スカウト)・6(世界の更新)・7(世界の生成)・8(外見)の**ストリームの中の引く順序**は、**該当の手順(05 §6〜§12)の記述順**とする(候補番号・学校ID・学年の昇順)。05 の参照モデルは Python の `random` で統計的に検算してあるので、**C# とビット一致のベクタはない**(統計の受け入れで確かめる。§16.3)。ラベル 8 の b = 40 / 41(個性・素養)は 3 回ずつ引く順序が 10 §4 に書いてあり、`ref_model_10.py vectors` の TV-1 がビット一致のベクタになる(Python の `Pcg32` を使うため)。
- **名前の引き方(label 8・b = 0〜3 と 30〜39。M1 準備で実装。`Domain/Text/NameGenerator.cs`)**: 姓の添字 = b = 0 の uniform ×1、名の添字 = b = 1 の uniform ×1(添字 = `min(n - 1, floor(u × n))`)。かな(姓のかな・名のかな)は表の組なので引かない(**b = 2・3 は予約**)。いまの部員と同じ姓名になるときは、再抽選 k = 0〜9 回目に b = 30 + k のストリームから姓・名の順に uniform ×2 で引き直す(10 回引き直しても重複なら、そのまま使う)。留学生は、国別の表(L-20)ができるまで、国の印象と結びつけない架空の音の部品を、列ごとに uniform ×1 でつなぐ(姓 = b = 0・名 = b = 1 のストリームから、姓の列 → 名の列の順)。表は `Assets/Master/names_ja.json` で、`verify/gen_names_cs.py` が C#(`NameTable.g.cs`)にする。**表の並びを変えると生成される名前が変わる**(足すときは末尾に)。
- **検算(TV-7)**: 登録表 26 区間で重なり **0**。予熱 15 年度 + 3 年度ぶんの鍵 **520,798 個**の導出値(64bit)がすべて異なる(衝突 **0**)。適合ベクタ(`derive(123456789,1,7,100)`・PCG32 の `u32` ×3・`uniform` ×2・`normal(0,1)`・`derive(123456789,2,7,100)`・04 TV-3 の雑音 `+0.003477`)がすべて一致。

8.3 引く順序の規約

1. **1つのユニット = 1つのストリーム**。ユニットが増減しても、他のストリームに影響しない。
2. **プレイヤーの入力で分岐するストリームは、分岐しても消費数を変えない**。04 §12 の『1人・1区間』は**常に 8 回**(`uniform` ×2 の `normal`・ブレーキの有無・大きさ・位置・好走・失速の有無・失速の大きさ)。指示・特殊能力・距離・気温を変えた 20,000 通りで、8 でなかった = **0**。イベントも状態だけで消費が決まる(06 §7.5)。
3. 入力と無関係なストリーム(候補・学校の更新・世界の生成)は、分岐で消費数が変わってよい(同じ状態から同じ順に引くので結果は決まる)。

8.4 端末差(ビット一致を求めない根拠)

02 §1 は『同一プラットフォーム・同一シードで一致。端末間のビット一致は求めない』。L-12 を、**測って**確かめた。端末によって `log`・`cos`・`pow` の最後のビット(1 ulp ≈ 2.2×10^{-16})が違い得る。

発生源	感度(区間タイム `ds` が変わる確率 / 相対誤差 eps)	1 ulp のとき(1区間)	1 年度(約 560 区間)
`normal()` の `log`・`cos`(雑音は標準偏差が小さいので誤差が薄まる)	約 58 × eps(eps = 10^{-4}・10^{-5}・10^{-6} の実測の平均)	1.3×10^{-14}	7×10^{-12}

発生源	感度(区間タイム ds が変わる確率 / 相対誤差 eps)	1 ulp のとき(1区間)	1 年度(約 560 区間)
pace50(d) = 176 + 6 ln(d/5) (区間タイムの**大きさそのもの**にかかるので、ds の丸めを跨ぎやすい)	約 16,000 × eps(eps = 10^{-6}・10^{-7})	3.6×10^{-12}	**2×10^{-9}**

- 1 ulp の実際のずれ(nextafter)を normal() に与えた実測: 300 レース・36,000 区間で ds が変わった = **0**、順位が変わった = **0**。
- 規則: ①**基準ペース pace50(d) は、距離が有限の集合(区間・記録の距離)なので、マスタに『倍精度の値の表』として持つ**(tools/MasterGen が参照の式から生成し、起動時に |表 − 式| ≤ 10^{-9} を診断で確かめる。使うのは表の値)。これで pace50 の端末差は **0**。②残る normal() の端末差は 1 年度で約 10^{-11} — **無視できる**。③**セーブは状態を保存する**ので、端末間の違いが積み上がることはない(同じセーブを別の端末で開いても、そこから先の乱数は同じ鍵で決まる)。
- **SimCli(.NET)と Unity(IL2CPP・Mono)のあいだ**でも、double の演算のまとめ方(FMA の融合・x87 かどうか)で最後のビットが違い得る。**同じ端末・同じビルドで一致すればよい**とし、実装間は**統計で比べる**(§16.3)。
- **M-02(他人のデータとの対戦)**: ゴースト対戦(②)は、**両方のチームを同じ端末で走らせる**設計なら、端末間の一致は要らない(結果の異なる確率は 1 レースあたり 10^{-12} 程度)。端末間で**必ず同じ**結果が要る場合(③ランキング)だけ、log・cos を**基本演算(+ − × ÷ √)だけで書いた決定的な数学関数**に置き換える(IDeterministicMath。FMA の融合を禁じる設定が要る)。v1 では持たない。**L-12 はこれで解決(実測 + 方針)**。

8.5 世界の生成・引き直し・共有コードの予約

- **新規ゲーム**: master_seed は**永続化層**が OS の乱数(RandomNumberGenerator)から 8 バイト取って Domain に渡す(Domain は乱数源を持たない)。NewGame(scenario, master_seed, look_seed) が世界(ラベル 7)と予熱 15 年度を作る。
- **『別の世界にする』(08 SC-02)** = 新しい master_seed で作り直す。プレビューは 3 秒以内(§14)。
- **共有コード(M-02 の予約)**: 形式は EK1-{scenario_id}-{master_seed の Base32}-{CRC8}。v1 では**実装しない**。『同じコードで同じ世界』が成り立つのは**同じプラットフォームの同じルール版**に限る(世界の生成は長い連鎖なので、端末差が増幅し得る)。

---

9. クラウド同期

9.1 モデル

同期は『**どちらが新しいか**』ではなく『**どちらが変わったか**』で決める(時計に頼らない。端末の時刻は信用しない)。

用語	意味
記録 record	同期の単位。slot{n}(自動の先頭)・slot{n}m(手動)・profile
lineage_id	歴史の識別子(§4.2)。新規ゲームで作る。**別の新規ゲームは別の系譜**
counter	save_counter(§4.4)。同じ系譜の中で単調に増える
last_synced	**この端末が最後にクラウドと一致させた時点**の { lineage_id, counter, deleted }。LocalState.sync[record](§3.3)
墓石	スロットを削除した記録。kind = tombstone のヘッダだけのコンテナ(ペイロードは {})。**系譜を保ち、counter を 1 進める**。端末にも LocalState.sync に deleted = true で残す

- 端末が変わった = L.counter > last_synced.counter。クラウドが変わった = C.counter > last_synced.counter。**番号が同じでも、両方が last_synced から進んでいれば競合**(両方が 5 → 6 に進んだ場合など)。
- **削除しない**: 同期のどの経路でも、競合の敗者は**退避**(§9.4)に残る。墓石は『利用者が削除した』ときにだけ作る。

9.2 判定 SyncDecide(L, C)

L = 端末の記録(なければ none。墓石を含む)、C = クラウドの記録(同)。戻り値は noop / upload / download / conflict。

```
SyncDecide(L, C):
  L=none ∧ C=none                → noop
  L=none                         → C が墓石なら noop、そうでなければ download      # 初めての端末
```

```
C=none                          → L が墓石なら noop、そうでなければ upload        # 初めての同期
L.lineage ≠ C.lineage:
    L が墓石                    → C が実体なら download                          # 墓石は別の系譜の実体に負ける
    C が墓石                    → upload                                         # 新しい歴史は墓石を上書きする
    どちらも実体                → conflict
同じ系譜:
    lc = L.counter > last_synced.counter,  cc = C.counter > last_synced.counter
    lc ∧ cc → conflict ;  lc → upload ;  cc → download ;  どちらでもない → noop
```

判定の表(TV-8。`ref_model_09.py sync` が 15 件を検算)

#	状況	結果
1	端末なし・クラウドなし	noop
2	端末なし・クラウドあり(初めての端末)	download
3	端末あり・クラウドなし(初めての同期)	upload
4	同じ系譜・端末だけ進んだ(5 → 7)	upload
5	同じ系譜・クラウドだけ進んだ(5 → 8)	download
6	同じ系譜・どちらも変わらない	noop
7	同じ系譜・どちらも進んだ(5 → 6 と 5 → 6。**番号が同じでも競合**)	conflict
8	同じ系譜・端末が削除(墓石)、クラウドは変わらず	upload(墓石)
9	同じ系譜・端末が削除、クラウドは進んだ	conflict
10	同じ系譜・クラウドが墓石、端末は変わらず	download(端末も削除)
11	同じ系譜・クラウドが墓石、端末は進んだ	conflict
12	別の系譜・どちらも実体	conflict
13	別の系譜・クラウドが墓石(新しいゲームが勝つ)	upload
14	別の系譜・端末が墓石(クラウドの実体が勝つ)	download
15	端末なし・クラウドが墓石	noop

9.3 実行の手順

同期の機会(待たせない。画面は止めない。08 §6.5・SC-40 の『同期の状態』):

1. **起動時**(裏で。記録ごとに)。
2. **自動セーブの直後**(同じ記録につき `K_SYNC_MIN_S` 秒に1回まで。月境界・大会前・年度末の節)。
3. **アプリの中断時**(ベストエフォート。待たない)。
4. 設定の『今すぐ同期』・ポーズの『同期』。

1つの記録の同期

1. クラウドの記録を読む(`C`、読んだ時点の状態を `base` として覚える)。
2. `SyncDecide(L, C)`。
3. `download`: クラウドのファイルを取得 → **§4.3 の検証(手順1〜6)に通す**。通らなければ捨てる(診断ログに残す。端末側は変えない。次の機会に再試行)。通ったら、**まず退避の要否**(`L` が実体で、競合の解決として置き換えるとき)を行い、ローカルへ置き(A / B の次の世代)、`last_synced = C` にする。
4. `upload`: ローカルの先頭の有効な世代を**そのまま**(再エンコードせず)送る。**前提つき書き込み**: クラウドの記録が `base` から変わっていたら書かず、**最初から判定し直す**(最大 3 回。それでも変わるなら次の機会へ)。成功したら `last_synced = L`。
5. `conflict`: §9.4。
6. `noop`: 何もしない。

- ダウンロードで置き換える先は、**必ず検証済みのファイル**。`last_synced` は**書き込みが成功してから**更新する(途中で落ちても、次の機会に同じ判定になる)。
- 同期の失敗(通信なし・認証なし・タイムアウト `K_STORE_TIMEOUT_S`)は**遊びに影響しない**。SC-03・SC-40 の『同期の状態』に『未接続』『最後の同期: 日時』を出す。

9.4 競合の解決(利用者に選んでもらう)

SC-37(確認)の形式で、次の内容を出す(本文は名称テーブル。08 §7.6)。

要素	内容
タイトル	『別の端末のセーブと食い違っています』(別の系譜のときは『別の歴史のセーブです』)
2枚のカード	『この端末』と『クラウド』。それぞれ 校名・開始校・第N年度 M月・プレイ時間・保存日時。**保存番号は出さない**。`saved_at` の新しいほうに『新しい』の印(ただし印は参考で、選択を誘導しない)
選択肢	『この端末を使う』『クラウドを使う』『あとで決める』。**既定のフォーカスは『あとで決める』**(何も変えない)
退避の注記	選ばれなかったほうは**消さずに退避に残す**(`backups/conflict_slot{n}_{UTC}.eksv`)。`K_BACKUP_MAX` を超えるときは、この選択を確定するときに**最も古い退避**(第N年度・日付を本文に書く)を削除する

- 『この端末を使う』: 保存番号を `max(L.counter, C.counter) + 1` に進め、クラウドの内容は**この端末の退避**へ。そのうえで `upload` (前提つき)。
- 『クラウドを使う』: クラウドを検証して置き換え、ローカルの内容は退避へ。
- 『あとで決める』: そのスロットの同期を**止める**(SC-03 に『競合(未解決)』の印)。ローカルの遊びと自動セーブは続ける。**次の起動で再び尋ねる**。
- **退避の復元**: SC-03 の『退避』から、『別の歴史として復元』(新しい `lineage_id` を振って、空きスロットへ。§4.5)。
- 墓石との競合(#9・#11)は、『削除』と『続きがある』の食い違いなので、本文を『別の端末でこのセーブが削除されましたが、この端末では続きがあります』(または逆)にする。**既定は『あとで決める』**。

9.5 プラットフォーム別(v1)

	Steam	iOS	Android
転送	**Steam Auto-Cloud**。root を Steamworks で登録し、対象を `saves/*.eksv` と `profile.*.eksv` に限る(除外: `suspend` `device.json` `local_state.json` `backups/` `logs/`)	**GameKit の Saved Games**(`GKLocalPlayer` 。名前 = `slot1` `slot1m` `profile` など記録名)	**Play Games Services の Saved Games**(スナップショット。名前は同じ)
競合の検出	Steam がファイル単位で解決する(起動前のダイアログ)。**どの世代のファイルが残っても**、読み込みは『検証に通る最大の保存番号』なので壊れない(§4.4)。起動後に `lineage_id` ・ `counter` を `last_synced` と比べ、§9.2 で判定する	取得時に同名の複数(`conflicts`)が返る → §9.2 に渡して解決を返す	スナップショットの競合(`conflictId`)→ §9.2 に渡して解決を返す
前提つき書き込み	Auto-Cloud は持たない。**書き込みの直前に読み直す**(窓は狭い)+ Steam 自身の競合検出	同名の上書きは最後が勝つ → **取得時の競合の検出**で補う	スナップショットの確定が競合を検出する
サイズ	Steam のファイルのクォータに収まる(実装時に確認)	実装時に確認	**3 MB**(`K_SIZE_HARD`)
認証なし	クラウド未接続で遊べる	〃	〃
利用者が止める	設定の『クラウド同期』(新設。 `DeviceSettings.cloud_sync` 。既定 true)で端末ごとに切る	〃	〃

- **ストア間(Steam ↔ スマホ)の共有はしない**(07 §8.3)。同じプラットフォームの端末どうしだけ。
- ログイン・認証の失敗・クラウドの無効化は、**すべて『未接続』として扱い、遊べる**(P-7)。
- **実機での確認は M5**(L-34): 上の API の挙動(競合の返し方・サイズ・オフライン時)は仕様書の時点では未検証。 `ICloudSave` (§15)の背後に閉じ込め、§9.2 の判定は**プラットフォームに依存しない純粋な関数**として Domain の外(永続化層)に置く。

9.6 ProfileData のマージ

`ProfileData` は**競合の確認をしない**。端末のものとクラウドのものを**マージ**する(読み込み時と同期時)。

項目	規則
`schema_version`	大きいほう

項目	規則
`achievements`	**和集合**。同じ実績の `unlocked_at` は**早いほう**(一度解除したら消えない)
`counters`	キーごとに**最大**(累計は単調に増える)
`clears`	**和集合**(`scenario_id` + `difficulty` + `date` が同じものは1件)

- 検算: 3,000 組で、**可換**(マージの順に依らない)・**結合的**・**冪等**・実績が消えない。すべて違反 0。
- **限界**(L-35): `counters` は最大を取るので、2 台で別々に増えた分(10 → 12 と 10 → 13)は合計されず 13 になる(15 より少ない)。『累計で解除する』実績の達成が**遅れる側**にだけずれる(早まらない)。必要なら、将来『端末ごとの加算』に変える。

9.7 検算(TV-9。`ref_model_09.py sync`)

版の木(各更新が節点・親は更新前)を作り、**どこにも残っていない葉(= 失われた更新)**を数える。端末 2〜3 台が、乱数でプレイ・同期の開始・同期の確定・中断・削除・新規を 300 手繰り返し、同期の**読み取りと書き込みのあいだに他の端末が割り込む**。競合の解決は、利用者の気分(50% ずつ)で選ぶ。最後に、『クラウドを使う』で同期を 3 巡させて収束を見る。

試験	結果
判定の表(§9.2)15 件	不一致 **0**
前提つき書き込みあり(400 回 × 300 手。更新 55,966・同期 28,181 = アップロード 6,991・ダウンロード 5,710・競合 9,425・前提の失敗 1,062)	更新が失われた回 = **0**、終了時に失われた葉 = **0**、収束しなかった回 = **0**
前提なし(最後の書き込みが勝つ。同じ条件)	更新が失われた回 = **36 / 400**、失われた葉 165、収束しなかった 2 → **前提つき書き込みは必須**
ふつうの使い方(2 台を 7:3 で行き来。36,000 セッション)で、起動時に同期できない 15%・終了時に同期できない 10%	競合 3,202 回(8.9%)、失われた更新 **0**
同 5% / 3%	競合 1,160 回(3.2%)、失われた更新 **0**
同 2% / 1%	競合 449 回(1.2%)、失われた更新 **0**
同 0% / 0%(いつも同期できる)	競合 **0** 回

- **競合は同期できなかったときだけ起きる**(いつも同期できれば 0 回)。同期の失敗が 2% 程度なら、100 セッションに 1 回ほど。
- 競合の解決で増える退避(§9.4)は、`K_BACKUP_MAX` = 5 を超える前に、利用者が選ぶ時点で古いものを整理する。

---

10. 購入状態の時計対策と `LocalState`

07 §8.2 の判定(確認できないときは、30 日までキャッシュを信じる)に、**端末の時計を操作して猶予を延ばす**抜け道を塞ぐ規則を足す。**ゲームの結果・実績・既に得た券・選んだ見た目には一切影響しない**(影響するのは『黄金世代プランの付与』の可否だけ。07 原則4・5)。

10.1 二つの物差し

物差し	内容	操作されるか
①**壁時計**	端末の時刻(UTC 秒)。`LocalState.clock_hw`(**高水位** = これまでに見た最大の時刻)と比べて、`now + K_CLOCK_BACK_TOL_S < clock_hw` なら**巻き戻された**	操作できる(止める・戻す)
②**動かした時間**	`LocalState.run_total_s` = **単調時計**(`Stopwatch` / `Time.realtimeSinceStartup` 相当。端末の時刻の変更に影響されない)で数えた、**アプリを動かした累計の秒**。確認の時点を `verified_run` に覚え、`run_total_s - verified_run` が `K_GRACE_RUN_H`(120 時間)を超えたら猶予を終える	操作できない(動かせば増える)

10.2 判定 `EntitlementGuarded`

```
EntitlementGuarded(provider, cache, now, hw, run_total):
  provider が owned / not_owned(ストアに確認できた)
      → (owned か, hw = now, cache = { owned, verified_at = now, verified_run = run_total })   # 基準を取り直す
  new_hw = max(hw, now)
  now + K_CLOCK_BACK_TOL_S < hw                                  → None(巻き戻された)
```

```
cache なし、または cache.owned = false                    → None
run_total − cache.verified_run > K_GRACE_RUN_H × 3600     → None
now − cache.verified_at ≤ K_GRACE_DAYS × 86400            → True
それ以外                                                  → None
```

- **`None` は『未確認』**: 付与も停止もしない(07 §8.2)。次の確認の機会に再判定する。購入済みの人が**不利益を受けるのは、ネットに繋がずに 30 日以上(または 120 時間以上)遊び続けたときだけ**で、繋げば戻る。
- 確認の機会は 07 §8.1 のとおり(起動時・新規ゲーム・年度の 4 月第 1 週・設定・商品を開いたとき・購入完了)。問い合わせの待ちは `K_STORE_TIMEOUT_S` (5 秒)で、**待たずに進める**。
- **抜け道の限界**(正直に): 端末を一度も起動しないまま時計を過去に設定する(高水位が更新されていない)と、壁時計の巻き戻しは見つからない。しかし②**動かした時間**の上限があるので、**猶予は延びない**(最大 30 日 / 120 時間)。時計を進める・戻すを繰り返しても同じ。

10.3 検算(TV-10。`ref_model_09.py clock`)

状況	結果
オンラインで所有	`True`
返金(`not_owned`)	`False`
オフライン・10日前に確認(動かした時間 20 時間)	`True`
オフライン・**30日ちょうど** / 30日と1秒	`True` / `None`
オフライン・確認なし / 以前に `not_owned`	`None` / `None`
時計を 5 日戻した(10日目 → 5日目)	`None`
時計を **24 時間ちょうど** / 24 時間と 1 秒戻した	`True` / `None`
時計を止めたまま(5日目の表示)、動かした時間が **120 時間ちょうど** / 120 時間と 1 秒	`True` / `None`
巻き戻したまま、オンラインで所有を確認	`True` (基準を取り直す)

- 13 件すべて一致。07 TV-3 の 5 件(オンライン所有・10日前・40日前・確認なし・返金)とも一致。
- 性質の検査: ①**正直な利用者**(時計の揺れ ±12 時間・確認から 1〜29 日・動かした時間 100 時間以内)4,000 人で、猶予の間に『未確認』になった = **0** ②**時計を操作する利用者**(確認の機会なし。固定・巻き戻し・実時間をまぜる)4,000 人で、動かした時間が 120 時間を超えても『有効』になった = **0**(『有効』と判定された起動 10,347 回)。

---

11. 実績・購入の通知のタイミング(U-14)

11.1 実績

- **判定と記録は本体で行い、`ProfileData.achievements` が正**(02 §3.10)。ストアへの通知は**ミラー**で、失敗しても解除は取り消さない。
- **通知の経路**: 解除 → `ProfileData` に記録 → `LocalState.pending_unlocks` に追加 → `IAchievementSink.Unlock(store_id)` を呼ぶ → 成功したら `pending_unlocks` から外す。
- **未接続・失敗**: `pending_unlocks` に残す。再送は**起動時・接続の回復時・解除のたび**。同じ実績の再送は `K_ACH_RETRY_S` (60 秒)以上あける。**冪等**(ストアは同じ実績の再解除を無視する)。
- **初回・端末の入れ替え**: 端末に `pending_unlocks` がなく、`ProfileData` に解除済みがあるとき(新しい端末・再インストール)、**解除済みの実績をまとめて一度だけ送る**(ストア側の欠けを埋める)。ストア側にだけ解除があって `ProfileData` に無いときは、`ProfileData` **へ読み戻す**(ストアが読めるとき)。
- 解除時の画面(トースト)は、ストアの接続に関係なく出す(08 §7.5)。

11.2 購入状態

- 確認のタイミングと保存(`LocalState.entitlements`)は §10・07 §8.1。**起動時は待たない**(結果が来るまで `unknown`)。

- 購入の完了は、`IEntitlementProvider` が返したら**すぐに** `LocalState.entitlements` を更新し、07 §5.2 の `TryPlanGrant` を呼ぶ(同じ年度のうちに確認できれば付与)。
- **復元**(App Store の『購入を復元』)は SC-05 のボタンから `RestorePurchases()` を呼び、結果を同じ経路で反映する。

---

12. マスタデータと文字列テーブル

12.1 マスタデータ

項目	規則
形式	種類ごとに1ファイルの JSON: `Master/{kind}.json` = `{ "kind", "master_version", "items": [...] }`。種類 = 02 §3 のマスタ(`region` `prefecture` `high_school` `country` `course` `race_rules` `training_menu` `skill` `facility` `staff` `ticket` `content_pack` `achievement` `scenario`)と、06 のイベント(`event`)・05 の学校(`school`)・名称などの表
置き場所	Unity では **`TextAsset`**(`Assets/Master/`)として組み込み、`IMasterSource` から読む。SimCli は**同じディレクトリのファイル**を読む。**`StreamingAssets` は使わない**(Android では APK の中にあり、直接ファイルとして読めないため。01 §12 を直す)
ID の規則	02 §1 のとおり(小文字ドット区切り)。**一度出した ID は再利用・変更しない**。廃止は `deprecated: true`(データは残す。セーブの参照が切れないように)。改名は移行関数の別名表で
読み込み時の検証	ID の一意・参照の解決・値の範囲(02・06 §13)。**起動時に1回**。失敗は致命(開発中は例外で止め、製品ではエラー画面)
派生値の表	`course.json` の各区間に **`pace50` の倍精度の値**を持つ(§8.4)。PB の導出に使う 5000m・10000m・ハーフの `pace50` も同様。`tools/MasterGen` が参照の式から生成する

12.2 マニフェストとハッシュ

- `Master/manifest.json` = `{ "master_version", "files": { "skill.json": "sha256:…", … } }`。
- `master_hash` = (ファイル名の昇順に、名前と各ファイルの SHA-256 を連結した文字列の SHA-256)の**先頭 8 桁**。セーブのヘッダ(§4.2)に書く。
- **用途は診断だけ**(バグの報告で『どのマスタで保存したか』が分かる)。読み込みの可否には使わない(互換は `schema_version` と ID の規則で保つ)。

12.3 文字列テーブル

項目	規則
形式	`Strings/ja.json` = `{ "キー": "本文" }`(平らな辞書)。言語を足すときは `Strings/{lang}.json`。v1 は `ja` のみ(U-04)
キーの名前	`scr.{画面}.{要素}`(画面) / `ev.{イベントID}.{部分}` / `skill.{ID}.name` `skill.{ID}.desc` / `menu.{ID}.name` / `ach.{ID}.title` `ach.{ID}.desc` / `race.{種別}.name` / `err.{コード}`(操作の失敗の理由)/ `warn.{コード}`(確認が要る警告)/ `hint.{コード}`(可能なときの一言)/ `ui.{共通}`
差し込み	`{名前}` の**名前つき**。本文の `{school}` は呼び出し側が文字列で渡す(数値・時間は §13 の関数で整形してから渡す)
学校・選手の名前	**表ではなくデータ**(セーブに入る。`SchoolState` / `Runner`)。**大会名・施設名・メニュー名**などの固定の語は表
欠落	開発ビルド: 赤字で『キー』を表示して警告。製品: ？ を表示して診断ログに1回だけ残す。**CI で、マスタ・コード・画面から参照されるキーがすべて表にあること**と、本文の `{名前}` が呼び出しの引数と一致することを検査する(`StringsLint`)
ID から決まるキー(M1 準備で実装)	ID が `ev.team.captain` なら `ev.team.captain.title` / `ev.team.captain.choice.{選択肢}`(**イベント ID の接頭辞 `ev.` を含める**)。特殊能力は `skill.{ID}.name`・練習 / テーマ / 走行量は `{ks.* / th.* / mil.*}.name`・シナリオは `{sc.*}.name` `.desc`・施設 `fac.{dorm 等}.name`・能力 `stat.{SPD 等}.name` `.abbr`・大会 `race.{fall / national / newyear / qualifier}.name`・指示 `instr.{attack / keep / hold}.name`・故障 `injury.{minor / moderate / severe}.name`・調子 `form.{2〜-2}.name`・成長 `growth.*.name`・個性 `style.{form / run / flow}.*.name`・素養 `gift.*.name`・性格 `trait.*.name`・校 `school.{0〜35}.name`・高校 `hs.{1〜216}.name`(OB の `high_school_name` に入るキー)・検証の理由 `problem.{コード}`・特別招聘券の節目 `tm.{win.fall 等}.name`(継続の記念は `tm.anniv.name` の `{years}`)・地方 `region.{kanto 等}.name`・校格 `tier.{elite / mid / low}.name`・進路 `career.*.name`・OB の役割 `obrole.*.name`・施設の効果 `fx.{コード}`(`{value}`)
かな	選手の姓名・校名のかなは**データ**(`family_kana` ほか)で、設定『かな表示』で上に出す(08 §7.1)。ほかの本文に振り仮名は付けない

項目	規則
文字の範囲	製品のフォント資産に、**表・マスタ・名前の生成で使うすべての文字**が入っていること（`StringsLint` が文字の集合を出し、フォントの対応と突き合わせる。M1 の検証。L-31）

実装（M1 準備）

- 部品: `Platform/Abstractions/ILocalization.cs`（`Get(key, 名前, 値, …)`・`Has`）・`TableLocalization.cs`（差し込み `{名前}`・`{{ }}`・欠落は開発 `〖キー〗` / `〖名前〗`、製品 `？`・キーの欠落は診断ログに 1 回）・`Persistence/Strings/StringTable.cs`（厳格な読み書き。キーの重複・空のキー・文字列でない値・末尾の余計な文字は不正。書くときはキーの昇順）・`Verification/StringsLint.cs`（キーの書式・本文が空でない・差し込みの書式・呼び出しの引数との一致 `CheckCalls`・文字の集合 `Charset`・フォントにない文字 `MissingFrom`）。
- 表: `Assets/Strings/ja.json`（738 件。イベントの題名 52 件・選択肢 80 件・校名 36 件・高校名 216 件ほか）。`SimCli strings seed` が、ID から決まるキー（`Verification/StringsSeed.cs`。イベント・特殊能力・シナリオは Domain の名称から、練習・施設・能力・大会などは仮の名称）のうち**表にないものだけ**を足す（既にある本文は変えない）。以後は **ja.json が正**で、本文の編集は ja.json に直接行う。
- 検査: `SimCli strings lint` = 書式 / `strings chars [ja.json] [フォントの文字のファイル]` = 使われている文字（表の本文 + 名前の表 + 整形が出す文字）とフォントにない文字（L-31）。`StringsCoverageTests` が、**ID から決まるキーがすべて表にあること**・書式の指摘が 0 であることを CI で確かめる（新しいイベント・特殊能力・練習・施設などの ID を足すと、表に足すまで落ちる）。
- 表に入れないもの: `EventDef.Hint`（設計メモ。覚醒イベントでは特殊能力の等級）。呼び出し側（画面）の `CheckCalls` の登録は、画面ができてから（M1）。

---

13. 数値・時間の整形（08 §7.4 の正本）

画面の整形は**この規則に従う**（08 §7.4 は要約）。純関数として `Ekiden.Domain` に置き（SimCli・テストから使う）、**整数だけで計算**する（浮動小数に頼らない）。数値・文字の変換は `CultureInfo.InvariantCulture`。

関数	入力	規則	例
`FmtTime(ds)`	`ds`（整数・0 以上）	1時間以上は `h:mm:ss.s`、未満は `m:ss.s`（区間・記録）	39543 → `1:05:54.3` / 8558 → `14:15.8` / 599 → `0:59.9`
`FmtTotal(ds)`	同	**秒未満は切り捨て**。1時間以上は `h:mm:ss`、未満は `m:ss`（総合タイム。順位・同タイムの判定は `ds`）	76089 → `2:06:48` / 76090 → `2:06:49`
`FmtDiff(dds)`	`ds` の差（整数）	0 は `±0.0`。1分未満は `+s.s` / `-s.s`、1分以上は `+m:ss` / `-m:ss`（秒未満は切り捨て）、1時間以上は `±h:mm:ss`。**先頭との差は `+`、区間の優劣は `-`（速い）**。符号は `+` と `-`（U+2212）	123 → `+12.3` / −71 → `-7.1` / 600 → `+1:00`
`FmtKm(km)`	double	小数 1 桁（0.5 は切り上げ）+ `km`	21.0975 → `21.1 km`
`FmtMoney(man)`	万円（整数）	3 桁区切り + `万円`。**10,000 万円以上は億円**で小数 1 桁（`.0` は省く）。負は `-`	1500 → `1,500万円` / 12000 → `1.2億円` / 99999 → `10億円`
`FmtYear(y, week?)`	年度・通算の週（0〜47）	`第{y}年度`。週があれば `{月}月{週}週`（月は 4 月始まりで 4 週ずつ）	(12, 26) → `第12年度 10月3週`
`FmtGrade(g)`	1〜4	`{g}年`	3 → `3年`
`FmtPct(p)`	0〜1	`x = p × 100`。**`x < 5` は `5%未満`、`x > 95` は `95%超`**、それ以外は四捨五入（0.5 は切り上げ）の整数 + `%`	0.82 → `82%` / 0.0499 → `5%未満` / 0.95 → `95%` / 0.9501 → `95%超`
`FmtRank(n, 繰上?)`	整数	`{n}位`。繰り上げは `繰上` を付ける（04 §8.2）	(8, 繰上) → `8位 繰上`

- 四捨五入は、すべて **`floor(x + 0.5)`**（02 §1。`Math.Round` の既定（偶数丸め）は使わない）。
- **04 の表の総合タイムとの差**: ref_model_04 の `fmt_t` は四捨五入で、`FmtTotal`（切り捨て）と **1 秒ずれる**ことがある（`ds` が 60,000〜89,999 の範囲で 45%）。**画面が正**で、04 の表は計算例の参考値（L-36）。

検算(TV-11。`ref_model_09.py fmt`): ベクタ 59 件すべて一致。`FmtTime` は `ds` 0〜99,999 で**往復**(読み戻して同じ)・08 の浮動小数の実装と一致(不一致 0)。`FmtPct` は p を 0.0001 刻みで動かしたとき**表示が逆戻りしない**(0 回)。`FmtDiff` は `+d` と `-d` で符号以外が同じ(不一致 0)。

---

14. 性能とスレッド

14.1 実測(Python 参照。`ref_model_09.py timing`)

処理	実測	条件
世界の生成(初期の名簿 4 学年 + 予熱 15 年度)	1.1〜1.3 秒	36 校・1 校あたり平均 約 36 人
1 年度の世界の更新(全校の成長・入学・卒業・簡易な大会結果)	約 70 ms	他校のみ(自校の育成は別)
1 レース(第一戦 20 校 × 6 区間)	3〜4 ms	乱数 960 回(120 区間 × 8)
自校の 1 年度の育成(40 人 × 48 週 = 1,920 件の週の解決)	約 50 ms	1 件あたり 約 25 µs
セーブ(100 年度)	70〜85 ms	直列化 25〜35 + gzip 44〜48 + CRC 0.1(JSON 1,852 KB → 288 KB)
読み込み(100 年度)	約 35 ms	gzip の伸長 6 + JSON の解析 28〜30
検証と修復(自校 40 人の簡易セーブ)	2〜3 ms	

→ **1 年度の遊びに必要な計算は、すべて足しても 0.3 秒に満たない**(世界 70 + 育成 50 + 大会 10 + セーブ 85)。重い処理は『世界の生成』だけ。

14.2 予算(規則)

C# の実装は、上の Python 参照の値を『上限』とする(C# + IL2CPP は、この種の数値ループで CPython より 1〜2 桁速い。下位のスマホで数倍遅くても、余裕がある)。実機で守る数値:

項目	予算	備考
Domain の 1 呼び出し	`K_DOMAIN_CALL_MS` **(100 ms)以内**(下位のスマホ)	年度末の世界の更新・卒業と入学・自動セーブの状態の取り出し。超える見込みのものは分割して呼ぶ
世界の生成(新規ゲーム・『別の世界にする』)	**3 秒以内**。バックグラウンドで作り、進捗を出す(`IProgress<float>`)	08 SC-02 のプレビュー。**完成後に差し替える**(作りかけを Domain に見せない)
セーブ: 状態の取り出し(メインスレッド)	30 ms 以内	取り出し = DTO への写し
セーブ: エンコード + 書き込み	バックグラウンド。中断セーブは**同期で 200 ms 以内**	
読み込み(読み取り → 伸長 → 解析 → 移行 → 検証 → Domain の復元)	500 ms 以内	画面は『読み込み中』を出す
レースの 1 区間の計算	1 ms 以内	
ホットパスの割り当て(GC)	**0**(週の解決・レースの計算・世界の更新の内側)	SimCli の `bench` で、1,000 レース・10 年度の前後の割り当て量の差を測る(閾値は M0 で実測して確定)
診断ログ	`K_LOG_MAX_BYTES`	古い行から捨てる

14.3 スレッド

- **Domain は単一スレッド**。内部で `Parallel`・`async` を使わない。状態はすべて**インスタンスに閉じ**、静的な可変状態を持たない。→ SimCli が世界を**並列に**回しても(世界ごとに別インスタンス)、結果は世界ごとに決まり、スレッドの順序に依らない。
- Domain を呼ぶのは **Unity のメインスレッド**だけ。バックグラウンドに回すのは、①新規ゲームの世界の生成(**別のインスタンス**を作る)②セーブのエンコード・書き込み(取り出した **DTO の写し**だけを触る。Domain の状態を触らない)③クラウド同期 ④ストアへの問い合わせ。

- DTO の写しは**不変**(取り出した後に Domain が状態を進めても影響しない)。

14.4 M0 の実測(C#。`SimCli bench` / `save`)

開発機(クラウドのコンテナ。Xeon 2.1 GHz・4 コア・.NET 8)の実測。**予算は下位のスマホの値**なので、開発機では **1/5 以内を合格**とする(下位のスマホを開発機の 5 倍遅いと見る。実機の確認は M1・L-38)。

処理	実測	予算	余裕	参照 Python
世界の生成(初期の名簿 4 学年 + 予熱 15 年度)	95〜125 ms	3,000 ms	×24〜32	1,100〜1,300 ms
1 年度の更新(3 大会 + 予選会 + 成長 + 入学。自校あり)	平均 5.8〜7.3 ms・最大 13〜17 ms	100 ms(`K_DOMAIN_CALL_MS`)	×6〜17	約 70 ms(他校のみ)
1 レース(第一戦 20 校 × 6 区間)/ 1 区間	0.04〜0.07 ms / 0.01 ms	— / 1 ms	×84〜163	3〜4 ms
週の解決 1 件	1.7〜2.1 µs	25 µs(Python の値を上限)	×12〜14	約 25 µs
自校 1 年度の育成(40 人 × 48 週)	3.3〜4.0 ms	—	—	約 50 ms
状態の取り出し(世界 36 校 + 自校の抽象。100 年度)	3.1〜3.2 ms	30 ms	×9〜10	—
100 年度の合成セーブ: 直列化 / エンコード全体(gzip level 6)/ 読み込み(伸長 + 解析)	11〜17 ms / 73〜114 ms / 60〜95 ms	読み込み 500 ms	×5〜8	25〜35 / 70〜85 / 35 ms
同 最速の圧縮(**中断セーブ**。同期)	31〜32 ms(387 KB)	200 ms	×6	—

- **世界の履歴(`history`)の取り出し**(R1a で足した): 100 年度で JSON 約 116 KB・取り出しの約 3 割(2.3 ms)。大会の順位表は大会が終わると変わらない(追記だけ)ので、作った JSON を `ConditionalWeakTable` で使い回す(`WorldSnapshot.CaptureHistory`。使う側は読むだけ)。取り出し(100 年度)は 11.8 ms → 4.7〜5.0 ms(この開発環境。履歴の前は 3.1〜3.2 ms = 環境の速さの差 ×1.5 を含む)。
- **この環境(クラウドのコンテナ)の実測の注意**: `bench` の『最速の圧縮のエンコード』(40〜47 ms)と『読み込み(伸長 + 解析)』(200 ms 前後)は、上の表(31 ms・60〜95 ms)の 2〜3 倍遅く、判定(予算の 1/5)を外す。`Eksv`・`Json`・合成セーブはこの作業で変えていない(初版から同じ)ので、環境の差。手元の開発機で確かめる。
- **セーブの大きさ**: 合成セーブの大きさは Python の見積もり(§5.4)と一致する(100 年度で JSON 1,857 KB・コンテナ 288 KB。参照 1,852 / 288。1・10・25・50・200 年度も ±1% 以内)。目標 1 MB・上限 3 MB を下回る。
- **gzip は C# の方が遅い**: 圧縮は zlib が支配し、同じ内容で Python の 44〜48 ms に対し C# は 60〜95 ms 前後(§14.2 の『Python の値を上限とする』が成り立たないのは、ここだけ)。C# が速いのは自前のループ(世界・レース・週の解決)で、Python の 1/10〜1/100。→ 同期で書く中断セーブだけ最速の圧縮にした(§4.6・L-47)。通常の自動セーブは背景なので時間の予算なし。
- **ホットパスの割り当て(GC)は『0』に届かない(L-46)**。M0 は実測を基準値にして退行を止める: 週の解決 488 B/回(基準 640)・第一戦 1 レース 43 KB(基準 57 KB)・世界の 1 年度 2.4 MB(基準 3.1 MB。候補プール 520 人の生成が大半)。0 に近づける作業は M1 の実機プロファイル(IL2CPP・GC の発生回数)のあと。

---

15. 技術スタックとプロジェクト構成

15.1 スタック

項目	決定	備考
エンジン	**Unity 6 の LTS 系の最新**(M1 の開始時に版を固定する)	2D(URP の 2D レンダラ)。ドット絵は Pixel Perfect の設定(点のフィルタ・圧縮なし・整数倍の拡大)で描く(M1 で設定)
スクリプトの実行	IL2CPP(スマホは必須。PC も揃える)	SimCli(.NET)とは `double` の最後のビットが違い得る(§8.4)
言語	C# 9 の範囲(Unity 6 の言語版)。Domain は `netstandard2.1` の API だけ	SimCli(.NET 8)と Unity で**同じソースを共有**する
入力	**Input System**(新)	ゲームパッド・タッチ・キーボード・マウス。08 §3 の入力モデル

項目	決定	備考
UI	**UI Toolkit**(第一候補)。M1 で §15.5 の基準を確かめ、満たせなければ uGUI	01 §12 のとおり
JSON	**Newtonsoft.Json**(UPM: `com.unity.nuget.newtonsoft-json`)。**DTO クラスだけ**で使い、Domain には持ち込まない	`[JsonExtensionData]` で未知のキーを保つ(§4.2・§6.1)。数値は §5.3 の書式
圧縮・CRC	`System.IO.Compression.GZipStream`・自前の CRC32(表引き。検査値 `0xCBF43926`)	
テスト	Unity Test Framework(NUnit)+ `dotnet test`(Domain・Persistence のテストを両方で実行)	
ローカライズ	**自前の文字列テーブル**(§12.3)。Unity の Localization パッケージは使わない	v1 は日本語のみ。言語を足す時点(U-04)で見直す
広告・解析	**入れない**(01 §12)。診断ログは端末の中だけ	

15.2 プロジェクトの構成(01 §12 を具体化)

```
Assets/
  Scripts/
    Domain/          asmdef: Ekiden.Domain            noEngineReferences / 参照なし。ゲームの全規則・DTO に依存しない
    Persistence/     asmdef: Ekiden.Persistence       コンテナ・世代・移行・検証と修復・同期の判定・DTO・JSON(Domain を参照)
    Platform/
      Abstractions/  asmdef: Ekiden.Platform.Abstractions   §15.4 のインターフェース(UnityEngine なし)
      Steam/ Apple/ Google/   各ストアの実装(条件付きコンパイル / プラットフォームごとの asmdef)
      Fake/          テスト・SimCli 用の実装
    Presentation/    asmdef: Ekiden.Presentation      UI Toolkit・演出・入力
    App/             asmdef: Ekiden.App               起動・依存の組み立て・FlowController(08 §6)
    Verification/    asmdef: Ekiden.Verification      T1 のベクタ・T3 の決定性の実行(SimCli と Unity の確認で共有。UnityEngine なし。M1 のゲート B)
    DevTools/        asmdef: Ekiden.DevTools          Unity 専用の確認の入口(実機・IL2CPP の自己検査 `VerifyBootstrap`。Editor/ にメニューとビルド)
  Master/            マスタ JSON(TextAsset。§12.1)
  Strings/           文字列テーブル(§12.3)
tools/
  SimCli/            .NET 8 コンソール。Domain・Persistence・Fake を参照(ソースを共有)
  MasterGen/         表の生成(pace50 の表ほか)
  StringsLint/       文字列・文字の検査
Tests/
  Domain.Tests/  Persistence.Tests/   NUnit(Unity と dotnet の両方で実行)
verify/              参照モデル(Python)。仕様の正本の検算
```

15.3 Domain の決定性の規則

ID	規則
D-1	乱数は `IRngFactory.Stream(label, a, b)` の鍵つきストリームだけ(§8)。`System.Random`・`UnityEngine.Random`・`Guid.NewGuid` を使わない
D-2	**時刻を読まない**(`DateTime.Now`・`Stopwatch` 禁止)。ゲーム内の時計(年度・週)だけ
D-3	順序に意味のある走査は **ID の昇順の配列**。`Dictionary` / `HashSet` の列挙順に依存しない(キーを整列してから回す)
D-4	整列は**安定**にする。同値のタイブレークに ID を使う(`List.Sort` は不安定)
D-5	丸めは `floor(x + 0.5)`。`Math.Round` の既定(偶数丸め)を使わない。`double` → `int` は明示的に
D-6	数値 ⇄ 文字列は `CultureInfo.InvariantCulture`
D-7	`float` を使わない(`double` のみ)。`Math.Log / Exp / Pow` は状態に効く計算で許す(§8.4)が、**基準ペースは表の値**を使う
D-8	静的な可変状態を持たない(読み取り専用のマスタのみ)
D-9	Domain の内側で並列化・`async` をしない(§14.3)
D-10	保存する `double`(`fame_hist` `base_prestige` `coach_q` `a0` `gap` `prog` `prestige`。M0 の自校の抽象では `funds` も)は代入のときに `Q6` を通す(§5.3。L-45)。M0 の統合は『状態の取り出し → 復元で全部の double が閉じていること』を `WorldSnapshot.NotQ6Closed` と T3 ② で確かめる

ID	規則
D-11	Domain は不正な入力で例外を投げてよい(Presentation が防ぐ)。**セーブの読み込みの経路は例外ではなく** `LoadError` (Persistence)で返す
D-12	**浮動小数の和は、仕様に書いた順に左から素朴に足す**(区間適性値の重みつきの和など)。補正つきの和・並べ替え・並列の和を使わない。Python の組み込み `sum()` は 3.12 以降は補正つきで、版によって最後のビットが変わるため、参照モデルも明示的なループで足す(M0 段階 4 の差分テストで見つかった)

15.4 プラットフォームの抽象

インターフェース	主なメンバ	実装
`IFileStore`	`Exists` `ReadAllBytes` `WriteAtomic(path, bytes, verify)` `Delete` `List` `FreeSpace`	`DiskFileStore`（`System.IO`。置き換えは §4.4)/ `MemoryFileStore`（テスト。**電源断の注入**（`CrashAt(n)`）つき。参照モデルの `Store` と同じ)
`IClock`	`UtcNow()`（秒)/ `MonotonicSeconds()`	`SystemClock` / `FakeClock`
`IEntitlementProvider`	`QueryAsync(packId, ct)` / `RestoreAsync()` → `Owned` / `NotOwned` / `Unknown`	Steam(DLC の所有)/ Apple(StoreKit。Unity IAP)/ Google(Play Billing の購入の照会)/ `FakeEntitlementProvider`（07 §8.4)
`IAchievementSink`	`UnlockAsync(storeId)` / `GetUnlockedAsync()`	Steam / Game Center / Play Games / `Fake`
`ICloudSave`	`GetAsync(record)` / `DownloadAsync(record)` / `UploadAsync(record, bytes, base)`（前提つき)/ `Available`	Steam Auto-Cloud(転送はプラットフォーム。判定は §9.2)/ GameKit の Saved Games / Play Games の Saved Games / `FakeCloud`（参照モデルの `SyncWorld`）
`IMasterSource`	`Read(kind)`	`TextAssetMasterSource` / `FileMasterSource`（SimCli)
`ILocalization`	`Get(key, args)`	`TableLocalization`（§12.3)
`IRngFactory`（Domain 側)	`Stream(label, a, b)`	`KeyedRngFactory(master_seed)`

- すべて**呼び出しは待たせない**(非同期 + タイムアウト `K_STORE_TIMEOUT_S`）。失敗は例外ではなく結果（`Unknown` / `Unavailable`）で返す。
- 実装は**ストアごとの asmdef**に分け、Domain・Persistence から参照しない(条件付きコンパイルを散らさない)。

15.5 UI 技術の検証(M1。合格の基準)

項目	合格の基準	満たせないとき
日本語の表示	製品のフォント資産で、§12.3 の文字集合がすべて表示できる(豆腐なし)。`text_scale` 150% でも 08 のレイアウトが収まる	フォント資産の再構成(サブセット・動的)
モバイルの性能	基準端末(M1 の開始時に決める。下位の対象 = 5 年前後の中位機)で、最も重い 5 画面(SC-16・SC-18・SC-25・SC-27・SC-34)の**フレーム時間 p95 ≤ 16.7 ms**、メモリ ≤ 400 MB	uGUI へ切り替える
ゲームパッド	十字キーのフォーカス移動が 08a の `grid` のとおり(全画面)。A / B / Menu で操作が完結する	フォーカスの自前管理
基準キャンバス	高さ 360 u の基準で、W = 640 以上の範囲で 08 §10 の収まりの検算どおり。拡大は端末の解像度に応じて整数倍に近づける	レイアウトの再検討
世界の生成の進捗	§14.2 の 3 秒以内・進捗表示	生成の分割

→ 上の**どれか 1 つでも M1 の試作で満たせなければ uGUI**(U-17)。判断の材料(フレーム時間・メモリ・文字)は M1 で記録する。

16. テストと自動検証

16.1 テストの層

層	内容	実行
T1 ベクタ	03〜09 のテストベクタ(§19)を C# に写した値の一致	`dotnet test`(CI)・Unity EditMode
T2 性質・ファズ	コンテナの切り詰め・ビット反転・電源断(§4)、移行(§6)、修復(§7)、同期の乱数シミュレーション(§9)、`Q6`(§5.3)、整形(§13)の性質テストを C# に写す	同上
T3 決定性	①同じ `master_seed` と入力で 2 回実行して、最終のセーブ(§5.3 の正規形)が**バイト一致** ②**任意の年度で保存 → 読み込み → 続行**と、**続行のみ**がバイト一致 ③中断 → 再開(§4.6)の一致	同上(M0 は世界 + 自校の抽象の `WorldSnapshot` で、`SimCli determinism`・`DeterminismTests`)
T4 統計	世界シミュレーション(§16.3)が参照の帯に入る	夜間の CI・手動
T5 性能	§14.2 の予算	SimCli の `bench`
T6 画面	画面の到達性・コマンドと情報の網羅(08 の `graph` を C# のカタログで)、UI の目視	Unity Test Runner・手動
T7 実機	各ストアの API(購入・実績・クラウド)・フォント・性能	M1・M5 の検証表(L-31・L-34)

16.2 ベクタの移植

- 各 `ref_model_NN.py vectors`(09 は `vectors` と `all`)の**出力の行 = 自動テストの 1 ケース**。表(文書の TV)と出力は同じ値なので、**文書 → テスト**の写しは機械的にできる。数値は文書の桁で比べる(許容 10^{-6}。ds・整数は完全一致)。
- `simcli vectors` が C# の値を同じ書式で出し、`tools/compare_vectors.py`(Python)が参照の出力と**行ごとに突き合わせる**(浮動小数は桁を指定)。差があれば CI を落とす。
- 乱数を使う参照(05 の世界シミュレーション)は Python の `random` なので、ビット一致のベクタは『乱数を使わない部分』(05 §13)に限る。残りは §16.3 の統計で受け入れる。

16.3 統計の受け入れ(M0)

やり方: SimCli で、方針『おまかせ』(04 §11・08 R-04)・45 年度の世界を、開始校ごとに **120 シード**(1〜120)回す。各シードから ①25 年度以降の**1 大会あたりの優勝率** ②25 年度以降の**三冠率** ③**クリア(通算制覇)の年度**(45 年度までにクリアしなければ 46 として数える)④25 年度以降の**戦力 P の平均**を出し、シード間の平均を、参照(`ref_model_05.py`)の値と比べる。

参照の値と許容の幅(参照モデルを、シード別に 3 開始校とも 120 回した = `python3 -I verify/band_05.py <開始校> 120`。第三戦の出場枠の拡大(L-67)と地形係数の較正(L-69)のあとの値。シード間の標準偏差から、C# の 120 回との差の 3σ = √(1/n_参照 + 1/120) × 3 × SD)

開始校	優勝率	三冠率	クリア年度の平均	戦力 P(25年度〜)	参照のシード数
強豪(Elite)	0.590 ± 0.044	0.305 ± 0.053	6.35 ± 1.07	72.49 ± 0.29	120
中堅(Mid)	0.553 ± 0.058	0.275 ± 0.056	12.99 ± 2.21	72.18 ± 0.44	120
弱小(Low)	0.441 ± 0.071	0.185 ± 0.052	25.48 ± 3.44	70.90 ± 1.01	120

開始校	シード間の標準偏差(優勝率 / 三冠率 / クリア年度 / P)	40 シードの平均の標準誤差(優勝率 / 三冠率 / クリア年度)
強豪	0.114 / 0.136 / 2.75 / 0.76	0.018 / 0.022 / 0.43
中堅	0.150 / 0.143 / 5.70 / 1.14	0.024 / 0.023 / 0.90
弱小	0.183 / 0.135 / 8.87 / 2.60	0.029 / 0.021 / 1.40

- **強豪県の +3(02 §6)の扱い(L-44)**: 参照モデルは出身を地方単位で近似し +3 を持たない。C# は仕様どおり県単位で +3 を入れる(平均は変えない)。+3 ありの世界は戦力 P が +0.2〜+0.4 高い(強豪だけ素の帯を 0.1 超える)。そこで T4 を **2 本立て**にする — **T4a 等価性**: +3 を外した世界(= 参照と同じ構成)が 4 指標とも上の帯の内側 / **T4b 仕様の世界**: +3 ありの世界で優勝率・三冠率・クリア年度は上の帯の内側、**戦力 P は T4a と同じシードの対の差が −0.3〜+1.0**。`SimCli accept` が両方を判定する(`--no-equiv` は T4b だけ)。M0 の結果は 05 §14.10。

- **T4c 実体の自校**（`accept --real`。M1 準備）: 同じ世界で、自校を実体の選手（`GameSession`）にしたもの。優勝率・三冠率は上の帯の内側（下側は 0.03 = `REAL_RATE_SLACK` まで許す。L-67）、クリア年度は帯の幅 + 1.5 年度（`REAL_CLEAR_SLACK`。L-69 で 1.0 → 1.5）の内側、戦力 P は +3 を許した帯の内側（下端 = 帯の下端、上端 = 帯の上端 + 1.0）。実体は故障・疲労・調子のばらつきのぶん抽象より少し弱い（05 §14.11）。
- クリア年度の中央値: 強豪 6 / 中堅 12 / 弱小 23。45 年度までにクリアしない割合: 強豪 0 / 中堅 1/120（1%）/ 弱小 9/120（8%）。**『20 年度までにクリア』の割合**: 強豪 1.00 / 中堅 0.91 / 弱小 0.33。
- **01・05 の 40 回の値**（中堅の優勝率 0.51・中央値 12.5、弱小 0.46、強豪 0.57 など）は、この値との差が 40 シードの標準誤差（0.02〜0.03）の 1 倍前後に収まる。**文書の目標値は変えない**（統計の揺れの範囲）。
- **検出力の限界**: 120 回で検出できるのは、優勝率で約 ±0.05（相対で 10% 程度）のずれまで。**それより小さい実装の誤りは、統計では見つからない**。だからこそ、**式・手順の誤りは T1 のベクタ（完全一致）で潰す**。統計は『大きな取りこぼし（手順の抜け・順序の誤り）』を見つけるための最後の網。
- 帯の外れ: ①T1 のベクタ（特に 03 TV・04 TV・05 TV）を先に確かめる ②おまかせ（AutoPlan）の実装が 08 R-04 と同じか ③乱数の鍵（§8.2）の重なり・消費の順序 ④世界の更新の順序（05 §9〜§12）。
- 同じ帯を、**ビルド（Unity・IL2CPP）でも**少数のシード（20）で確かめる（端末差で結果の分布が変わらないことの確認。§8.4）。

長期の安定（M0 の C# の結果）: 3 開始校 × 16 シード × 300 年度（`SimCli stability 16 300 <開始校>`）で、例外なし・世界の不変条件（`WorldInvariants` W-1〜W-7）の違反なし。戦力 P と優勝率のシード平均が年度 26〜100 / 101〜200 / 201〜300 で動かない（区間の差は P 0.5・優勝率 0.06 以内。中堅 P 72.33 / 72.30 / 72.52。第三戦の出場枠の拡大（L-67）のあとの値）。1 シードごとには、区間の P の差 2.0 以内・65〜80 を崩壊の目安にする（優勝率は 1 校の運の揺れが大きいので、シードをならして見る）。弱小は 50〜60 年度まで上がらないシードがある（クリアしない 5〜10%）ので、頭打ちの比較は 101 年度以降の区間どうしで行う。

長期の安定（参照モデルで確認）: 中堅・『おまかせ』を **300 年度**回した（2 シード）。戦力 P と優勝率は年度 26〜300 で頭打ちのまま（シード 1: 年度 26〜100 / 101〜200 / 201〜300 で P 72.8 / 72.0 / 72.2、優勝率 0.56 / 0.53 / 0.54。シード 2: P 72.9 / 73.3 / 73.4、優勝率 0.66 / 0.71 / 0.71 — シード間の差の範囲）。**C# の M0 の完了条件も 300 年度**とする。

SimCli の命令（`m0/tools/SimCli`。すべて終了コード 0 = 合格）: `world`（自校なしの世界の指標。05 §14.1）/ `accept`（上の T4a・T4b。`--real` で T4c）/ `play`（実体の自校を 1 回プレイして年度ごとの経過を出す）/ `stability`（300 年度の安定）/ `determinism`（T3 の ①②③。最終のセーブの SHA-256 を出す。続けて実体の自校の T3: 年度の途中の保存 → 読み込みも）/ `bench`（§14.2・§14.4。`--quick` で回数を減らす）/ `save`（合成セーブの大きさと時間。§5.4）/ `fuzz-save`（コンテナの切り詰め・ビット反転・電源断・検証と修復のファズ。§4・§7）/ `sync-sim`（§9.7）/ `vectors`（T1 の主なベクタ 139 行。`python3 -I m0/tools/compare_vectors.py` が参照モデルと行ごとに突き合わせる）。

---

17. 仕様完了のチェックと M0 の着手条件

17.1 仕様完了の基準（00 §2）× 文書

`ref_model_09.py inventory` が出す数（参照モデルの実行・文書の行数・テストベクタの節・`K_` の定数・課題の数）に基づく。

基準	内容	結果
1	項目・型・範囲・必須/任意・不変条件	○ 02（全エンティティ・I-01〜I-21）。読み込み時の検証・修復は 09 §7（修復 R 14 件・致命 F 6 件・警告 W 3 件。I-11・I-13 は R と F の両方を含む。ファズで検算）
2	手順の順序・端数・上下限・境界	○ 03〜09。端数は 02 §1（`floor(x+0.5)`）、整形は 09 §13、保存の double は 09 §5.3
3	計算例（テストベクタ）	○ 03: 7 / 04: 11 / 05: 10 / 06: 7 / 07: 4 / 08: 12 / 09: 11 / 10: 6（計 68 節）。いずれも `ref_model_NN.py vectors` で出力でき、そのまま自動テストにできる。05 の乱数を使う部分は統計（§16.3）
4	数値は初期値として明記・調整可能な値は定数表かマスタに集約	○ **（ただし名前に注意）** 各文書の定数表と、参照モデルの `K` 辞書のキーは**名前が一致しない**（文書は `K_TICKET_STOCK_MAX`、モデルは `STOCK_MAX` など）。値は TV で担保されている。**M0 の最初に `K` 辞書から `Constants` を作り、対応表を付ける**（L-37。`inventory` が一致しない数を出す）。10 だけは名前を合わせてあり、`ref_model_10.py check` が文書の定数表・取得条件・テストベクタとの一致を確かめる
5	画面・イベント・商品などの一覧が漏れなく	○ 画面 42（08・`graph` で到達性）/ イベント 52（06・`validate`）/ 商品（07 §9）/ 実績（01 §3.3）/ マスタの種類（§12.1）

基準	内容	結果
6	未決は既定値つき	○ 00 §5 の U-01〜U-19 すべてに既定値(U-14 解決・U-17・U-18・U-19 を追加)

17.2 仕様のうち、未検証・未作成のもの(M0 を止めない)

項目	状態	検証の時期
実大会データとの照合(ブレーキ・失速の頻度)	L-10	M3 以降
ストアの API(購入・実績・クラウド)の実機の挙動	L-22・L-34	M5
実機での文字・タップ・パッド・日本語フォント・性能	L-31	M1
C#・IL2CPP での性能・GC・メモリ	L-38	M0(SimCli)・M1(実機)
画面の文言・イベントの本文(約 100 本)・演出・美術	L-19・L-32・U-03	M1〜M4
世界の更新・スカウトの乱数ストリームの引く順序(Python とビット一致しない)	L-33	M0 の統計(§16.3)

17.3 M0 の着手条件と作業の順序

着手条件(すべて満たす): ①全文書が §17.1 の基準を満たす(上表)②すべての計算例が参照モデルから再現できる(`ref_model_NN.py vectors` が全て終了コード 0。`inventory` で確認)③M0 を止める未決がない(U-xx はすべて既定値つき)。

M0 の作業の順序(01 §13 の M0 を具体化。各段階の完了 = その段階のベクタが全件一致)

段階	内容	完了の条件
0	ソリューションの雛形(§15.2: Domain・Persistence・Fake・Tests・SimCli)、CI(`dotnet test`)	空のテストが緑
1	`Constants`(L-37)・`Q6`・整形(§13)・CRC32	09 TV-1・TV-4・TV-11
2	`mix64` / `derive` / `Pcg32`・鍵の登録表	03 TV-1・04 TV-3・09 TV-7
3	週の解決(03)	03 TV-1〜7
4	レース(04)・`pace50` の表(MasterGen)	04 TV-1〜11・09 §8.4 の表
5	世界・スカウト・名声・資金(05)	05 TV-1〜10、世界の長期安定(自校なし 80 年度。05 §14.1)
6	イベント・条件式(06)	06 TV-1〜7(`validate` の C# 版)
7	特別招聘・購入状態(07・09 §10)	07 TV-1〜4・09 TV-10
8	永続化(コンテナ・世代・移行・検証と修復・同期の判定・マージ・中断セーブ)	09 TV-1〜6・TV-8・TV-9(`fuzz-save`・`sync-sim`)
9	画面のための規則(08 R-01〜R-17)	08 TV-1〜12
10	統合: SimCli の世界シミュレーション・決定性・性能	§16.3 の帯・T3・§14.2 の予算

M0 の完了条件: T1 が全件一致 / T2 が緑 / T3(バイト一致)/ T4(3 開始校が帯に入る)/ T5(予算内)/ 世界を **300 年度**連続で回して破綻しない(例外・不変条件の違反なし。戦力の頭打ちが保たれる。下の注)。

17.4 M0 の C# 実装(段階 8)

- 場所: `m0/Assets/Scripts/Persistence/`(`Json/` 最小の JSON・`Container/Eksv.cs` コンテナ・`Container/SlotStore.cs` A/B・回復の順・中断セーブの再開の条件・`Migration/SaveMigrator.cs` v0 → v1・`Validation/SaveValidator.cs` I-01〜I-21 の検証と修復・`Validation/SaveInvariants.cs` 独立の検査・`Sync/` 判定・ProfileData のマージ・乱数シミュレーション・`Store/` 購入状態と LocalState)と `m0/Assets/Scripts/Platform/`(`IFileStore`・電源断を注入できる `MemoryFileStore`)。
- 参照モデルとの差分テスト(`m0/tests/Persistence.Tests/PersistGoldenTests.cs`): コンテナ 414 件・マイグレーション 605 件・検証と修復 768 件・同期の判定 6,000 件・マージ 3,000 件が完全一致。
- 決めたこと: ヘッダ JSON がオブジェクトでないコンテナは `header_json` で断る(参照モデルは通す)。JSON の数は、読んだものを元の文字列のまま書き戻す(未知の項目を壊さない)。gzip のバイト列は実装で変わるので、コンテナの検査は『読み戻して一致』。

17.5 M0 の C# 実装(段階 10: 統合)

- 場所: `m0/tools/SimCli/`(命令は §16.3)、`m0/Assets/Scripts/Domain/World/`(`PlayerAgent` 自校の抽象・`Acceptance` 帯の判定・`WorldInvariants` 世界の不変条件・`WorldSim.Restore`)、`m0/Assets/Scripts/Persistence/Snapshot/`(`WorldSnapshot` 状態の取り出しと復元・`Determinism` T3 の実行器)。
- **T3 の結果**: 4 開始校(なし・Mid・Low・Elite)× 2 シード × 45 年度で、①同じ入力の 2 回 ②保存 → 読み込み → 続行(JSON の文字列経由・EKSV コンテナ経由・毎年度)③並列実行が、最終のセーブ(JSON 273〜286 KB)の**バイト一致**。復元の欠落(例: `last_newyear_rank` を復元しない)は `DeterminismTests` が落ちることを確かめた。端末・ビルドをまたいだ一致は求めない(§8.4)ので、SHA-256 は記録用。
- **T4 の結果**: 05 §14.10。**T5 の結果**: §14.4。**T2(ファズ)の結果**: コンテナの切り詰め・ビット反転・8 バイトの書き換え 45,282 回で見逃し 0、電源断 24 通りで読めなくなる例 0、検証と修復(破損 4,000 件: 修復 1,954・致命 2,046。修復後の違反 0・冪等・致命の取りこぼし 0)、同期(CAS ありで失われた更新 0・収束。なしだと 400 回中 39 回で失われる)。
- **T1(139 行)**: `compare_vectors.py` が参照モデルと全行一致。
- **統合で見つけたこと**: ①保存する double が `Q6` で閉じていない場所が世界の更新・資金にあった(D-10)— 状態の取り出しの検査(`NotQ6Closed`)で見つけた。名声(`prestige`)も連続値で保存する必要があり(L-45)、`fame_hist` の範囲は 0〜67.5 だった ②中断セーブの圧縮(L-47)③GC『0』は未達(L-46)④+3 の効果(L-44)。
- **未了(M0 の範囲外として記録)**: 自校の育成(Runner・イベント・日程)を含む保存の DTO(`WorldSnapshot` の隣に足す。M1)/ `ICloudSave`・`IEntitlementProvider` の実装(M5)/ Unity・IL2CPP での T3・T4・T5(同じソースを `vectors` と少数シードの `accept` で。§16.3 の最後の項)/ おまかせスカウト(R-08)の別の方針(T = 8)と推奨の施設強化(R-11)は `PlayerAgent` の方針のまま。

17.6 M1 準備: 自校の保存(`TeamSnapshot`)

- 場所: `m0/Assets/Scripts/Persistence/Snapshot/TeamSnapshot.cs`(取り出し `Capture` と復元 `Restore`。形は §5.6)。Domain 側に足したデータ項目: `Runner`(名前・外見・性格タグ・`RelayWins`・`Country`・`DeriveDistance`)、`Team/TeamData.cs`(`Appearance` `AnnualPlan` `MenuSpec` `IndividualMenu` `TeamSettings` `EventLogEntry` `ClearInfo` `ScenarioRef`)、`TeamState`(計画・個別指導・面談・設定・券・採番・イベントの履歴 `LogEvent`)、`OBRecord`(卒業生の記録 `FromGraduate`。`Corp` は `CareerPath` から導く)。規則には作用しない項目で、既存の差分テスト(イベント 286 件ほか)はそのまま通る。
- **結果**: 参照モデルが書いた保存 150 件(検証器を修復なしで通る形)が往復で同じ JSON、試験用の週ループ(6 年度)の 40 か所で『保存 → 読み込み → 続行』が『続行のみ』とバイト一致(JSON 経由・コンテナ経由・毎週の読み込み直し)。試験は 307 件(Domain 221 + Persistence 86)。
- **統合で見つけたこと**: ① `far` / `near` は自校の所在県が要る(保存しないと、帰省イベントの対象が復元後に変わる)→ `school_prefecture_id` を保存し、復元で導く ②取り出した時点で期限が切れている補正は、書かないことで I-19 を満たす(L-50)③ `origin` の形の食い違い(L-48)④復元が派生値を作らない約束(L-51)⑤ `Corp`(実業団)と `career_path` の二重の持ち方 → `career_path` を持ち、`Corp` は導く。
- **未了**: 世界の側との合成(L-49。**§17.7 で合成した**: `GameSnapshot`。自校の写し `prestige` `year_rank` などの整理は残る)/ 保存のタイミング(08 §13.4 の自動セーブの時点)で `Capture` を呼ぶ側 / 試験用の週ループは試験にだけ置いた(M1 の本物のループは別に作る。ただし約束は L-51)。

17.7 M1 準備: 実体の自校 + 世界(`GameSession`)

- 場所: `m0/Assets/Scripts/Domain/Game/GameSession.cs`(1 回のプレイ。`New` `StepWeek` `PlayYears` `Restore`)、`Team/`(`TeamWeek` 週の処理・`TeamRacing` 大会の側・`TeamYear` 年度末)、`Recruit/RunnerFactory.cs`(候補 → 選手)、`World/PlayerSide.cs`(`IPlayerSide`: 世界の年度処理への差し込み)、`Persistence/Snapshot/GameSnapshot.cs`、`tools/SimCli/`(`play` `accept --real` `determinism`)。
- **週の順**(06 §7.1): 週の最初(年度の最初なら前年度の記録を送る・月の最初ならスカウトと指導のポイント・期限切れの補正・派生値 `Refresh`・エントリー締切なら登録)→ 全選手の週の解決(03 §6)と絆 → 予選会(10 月第 3 週)・大会(第 25・28・36 週)と大会後の処理(04 §10)→ イベント → 2 月の最初の週に招聘の宣言(おまかせ)→ 月末(絆の減衰・士気・やる気の規則 M1・M4〜M6)→ 年度末(05 §12: 名声・資金と強化・成長 → 卒業と退部 → 招聘の入学 → 割り当て)。

- **世界との関係**: 他校・名声・割り当ては抽象の自校と**同じ道**（`WorldSim.BeginYear` `PlayQualifier` `PlayRace` `EndYear`）。自校だけ `IPlayerSide` の差し込みで実体の選手が受け持つ。抽象の自校の結果は**ビット同一**（T3 の SHA-256 は変わらない）。
- **乱数**: 週の解決・イベント・大会は既存の鍵（label 1・2・4・5）。新しい鍵は label 9 の 1 本だけ（年度末の卒業・退部の抽選。新規ゲームの部員の間引きは年度 0）。
- **結果**: T3（実体の自校）バイト一致（上の §5.6）。T4c（実体の自校の統計）は 05 §14.11。性能: 45 年度の通しが約 3 秒（1 年度 0.05 秒）。試験は 624 件（Domain 474 + Persistence 125 + App 25）。読み取り API（`Views`。08b）と App 層（下）を加えた。
- **当日（04 §14.5）**: `GameSession.Race.cs`（オーダー・予報・前評判・当日を開く・予選会の登録）・`RaceDay.cs`（当日変更・指示・檄・レース進行）。当日の状態（当日変更・指示・檄）は**通常のセーブには入れない**（`GameFlow.CanSave` は練習が解決される前だけ。提出したオーダーは保存する）。アプリの中断では、当日の決定を `RaceDayRecord` にして**中断セーブ**に入れる（§4.6。K1）。
- **操作と進行制御（08 §13.6）**: `GameSession.Commands.cs`（年間方針・月間計画・個別指導・登録・招聘・施設）・`GameSession.Scout.cs`（スカウト）・`GameFlow.cs`（停止）。週は段階（`ResolveTraining` → `PlayWeekRaces` → `SelectWeekEvents` → `ResolveWeekEvents` → `FinishWeek` → 年度末は `SettleYear` → `TurnoverYear` → `AdvanceWeek`）に分け、おまかせの `StepWeek` と `GameFlow` が同じ段階を呼ぶ（何も操作しない流れ = おまかせ、バイト一致）。`WorldSim.EndYear` は精算（`EndYearSettle`）と後半（`EndYearTurnover`）に分けた（抽象の自校の結果はビット同一）。
- **App 層の入口（§4.4〜§4.6・08 §6.5。`Assets/Scripts/App/`・asmdef `Ekiden.App`・Unity なし）**: `GameHost`（`NewGame` / `Open` / `Advance` / `SaveManual` / `OnAppPause` / `OnAppResume` / `Tick`）が `GameSession` + `GameFlow` + `SlotStore` をつなぐ。`SaveService` が書き込みの段取り、`SlotCatalog` が SC-01・03・40 のスロットの見出し。
  - **保存の呼び出し位置**: 新規ゲームの開始（`Initial`）/ **月の最初の週の練習が解決される直前**（`Month`。`GameFlow.ResolvingWeek`。月末レポートの確認のあと・その月の計画が決まったあと）/ **自校が出る大会（予選会を含む）の週の同じ時点**（`PreRace` = オーダー確定後。月の最初と重なる週は 1 回）/ 年度末の精算の停止（`YearEndSettle`）・卒業と入学の停止（`YearEndTurnover`）/ 手動 / アプリの中断（中断セーブ）。本体は `SuspendCheckpoint.Body`（約 4 ms の取り出しを共用）なので、復元するとその週の週の初めの停止からやり直す（決めたことは状態に残っている）。
  - **書き込み**: 要求は待っている 1 件に畳む（新しい要求は古い再試行も取り消す）。失敗は `K_SAVE_RETRY_S` 秒後に 1 回だけ再試行（`Tick`）、それも失敗なら通知（`AutosaveFailed` / 空き不足は `DiskFull`）してゲームは続け、次のきっかけでもう一度。保存番号は失敗で飛ばさない。`Synchronous = false` にすれば `Pump` を別のスレッドで呼べる。中断セーブは待っている自動セーブを先に書き、`base_counter` = スロットの先頭の保存番号。
  - **開く**: `SlotStore.RecoveryCandidates`（§4.5 の順の候補）を先頭から試し、**コンテナとして有効でも中身が復元できなければ次の候補**へ。最新が使えなければ `RecoveredFromOlder`。中断セーブは、先頭の世代を使え・同じ歴史・`base_counter` が先頭と等しいときだけ再開（`ResumedFromSuspend`。最初の自動セーブが成功してから消す）。合わなければ捨てて `SuspendDiscarded`（別のスロットの中断セーブには触らない）。すべて失敗なら Host は null でファイルは消さない。
  - **検算**: 1 年度の自動セーブのすべて（17 か所）から開き直して年度末まで続けても、続けたまま = 同じ状態 / 週の初め・月末レポート・大会の当日・イベントの選択での中断 → 再開が同じ年度末 / 書き込みの失敗（再試行・通知・次のきっかけ・古い状態を書き足さない）/ 回復の順 / プレイ時間（中断中は数えない）。呼び出し位置・再試行・使い切りなどを 1 つずつ壊すとテストが落ちる。
- **読み込み時の検証・修復（§7.1b。L-66 ①）**: `Persistence/Snapshot/GameLoader.cs`（`LoadOutcome` = 復元した `GameSession`・本体・致命の理由・修復の一覧）。`GameHost.Open` と `SuspendCheckpoint.Resume` が使う。`SaveValidator.ValidateAndRepairTeam` を足した（`schools` `rival_runners` がなければ省く。完全な `SaveData` の形では従来どおりで、参照モデルとの差分テストは変わらない）。試験（`GameLoaderTests`・`HostTests`）: 正常なセーブの修復 0 / 範囲外 21 種を直して続けられ・冪等・続けたあとも修復 0 / 致命（I-11・I-03・I-02・必須項目の欠落・型の違い・在籍 60 人超・世界の名声）で失敗 / 最新の世代が致命なら前の世代へ / 修復は利用者に知らせない / 中断セーブからの再開も同じ。
- **通知のログ（08 §7.5・08a SC-38。08b G-07）**: `Team/Notification.cs`・`TeamState.Notifications`（新しい順・最大 `K_NOTIFY_MAX` = 30・`Unread`）。足すのは `GameSession` の週の処理（故障 = `ResolveTraining`・疲労の警告 = 月の第 3 週に 1 回・イベント・特別招聘券・大会の予定 = 出場権のある大会のエントリー開始 / 大会の週 / 予選会の週）と、外から `GameSession.Notify`（`GameHost.OnSaveNotice` が自動セーブ・手動セーブの失敗と空き不足を `AutosaveFailed` で足す。購入状態の不明 `Entitlement` は呼ぶ側）。文は `code` から `nt.*` の文字列キーで組む（`NotificationsView.Items[i].TextKey / ArgKey / Target`）。保存 = `player_team.notifications`（§5.6。**追加の項目**: ないセーブは空）。読み取りは `Notifications()`、既読は `MarkNotificationRead` / `MarkAllNotificationsRead`、ホームの未読数は `HomeView.UnreadNotifications`。乱数を使わないので決定性の目印（抽象の自校）は変わらない。

- **実績(02 §3.10・01 §3.3。08b G-08)**: `Domain/Achievements/`（`AchievementMaster` 23 件・`AchievementFacts`・`AchievementRules`。事実から『いま満たしている ID』を返す純関数)と、`App/AchievementTracker.cs`（`ProfileData` に解除の日時を記録 = 一度解除したら消えない・ストアへの通知待ち `Pending` に積む・クリアの記録を 1 件)。`GameHost.Advance` が停止のたびに呼ぶ(山の神は走った大会の結果から)。`HostEnvironment.Profile` に Unity 側が読み込んだ `ProfileData` を渡す。**プロフィールのコンテナ（`profile.a/b.eksv`）の読み書き・マージ・ストアへの通知は Unity 側**（`ProfileData.Merge` と `LocalState.PendingUnlocks` は実装ずみ)。
- **調子の見込み(08 R-05 ⑥。08b G-15)**: `Game/GameSession.Views.Outlook.cs`。読み取りだけ(保存は増えない)。この作業で見つけた既存のずれ 2 つを直した: ① `Forecaster` が安全網の閾値を定数 85 に固定していた(設定 75 / 0 を無視。`safetyF` を引数に)②個別指導のメニューの期限（`UntilAbsWeek` = 翌月の第 1 週)を含めて数えていた(月をまたぐ見込みでずれた。`>` に)。
- **世界の履歴（`history`。02 §4.8・§4.9。08b G-03・G-04)**: `WorldSim.History`（`World/RaceHistory.cs`)。**予熱の年度から**、大会(第一〜第三戦。予選会は含めない)ごとの順位表(全校の『学校 ID・合計タイム・繰り上げ』を順位順。優勝校 = 先頭)と、区間記録（`Course[大会][区間]` = 学校・走者 ID(他校は 0)・タイム・年度)・大会記録（`Team[大会]`）を持つ。`WorldSim.PlayRace` が更新し（`RaceOutcome.Records` に更新分)、`WorldSnapshot` が `history { race_records, course_records, team_records }` として保存する(**追加の項目**: ないセーブは空の履歴で読み、版は上げない)。記録の規則: 最初の 1 回は基準として黙って入れる(更新でない)/ 以後は厳密に速いときだけ更新 / 保持者は最速チームのうち順位の先頭 / 大会記録は繰り上げのない優勝タイム。**再現性の目印（`m0/tools/determinism_anchors.txt`）は、この項目が保存 JSON に増えたので更新した**（`history` の出力だけを外すと従来の SHA と一致 = 世界の進行は不変。セーブの JSON は 45 年度で約 +61 KB、圧縮前)。自校の走者の区間記録の更新は `Runner.Records`（`career.records`。追加の項目)に数え、`RaceFlags.Record`（`race.record` のイベント)・殿堂入り・`OBRecord.career.records` に効く(自校の結果だけが変わる = 実体の自校の 9 件目以降の SHA は変わる。抽象の自校は不変)。**新規ゲームの区間記録の初期値(L-68)**: `GameSession.New` が予熱のあと・年度 1 の前に `History.SeedLegacyCourseRecords()` で区間記録を『初期の区間記録』(02 §4.8。`RecordSeeds`）に置き換える。保持者は `school_id = -1`・`runner_id = 0`・`year = -100`（『歴代の記録』)。`WorldSim.Create`（予熱)は変えないので、世界の進行・乱数・再現性の目印(抽象の自校)は不変。保存の形は変えない（`school_id = -1` を許すだけ。既存のセーブはそのまま読む = 版は上げない)。
- **試験の見つけたこと**: ①保存の境目を『週の最初の処理のあと』にしたので、復元で `Refresh` が要る(なければ、絆の平均が復元後の最初の週の解決に入らず、バイト一致が崩れた)②グループを毎週決め直すと、手動のグループが戻る(08 R-04 に合わせて、保存される編成にした。参照モデルの `refresh` は決め直すので、差分テストは `Refresh(assignGroups: true)`）③停止から保存 → 読み込みをすると、同じ週の停止がもう一度出る(週の初めの停止は、その週の最初から再開する)。
- **未了(L-52〜L-55)**: 名前の表(M1 のマスタ)/ 予選会の疲労と調子の反映 / 区間記録 / 他校の同じ高校の選手（`rival_same_hs`）は近似のまま / 資金の使い道(L-05)/ 型に合わせた育成の自動化(L-24)/ 檄の先行使用(L-60 ①)・予選会の当日(L-61)/ 観戦モードの演出・月末レポートの項目・新規ゲームの画面(Unity の層、M1)。手動の操作(月間計画・登録・個別指導・スカウト・オーダー・当日)の入り口と進行制御は M1 準備で作った(下の 2 項)。

17.8 M1 準備: Unity での確認(ゲート A・B)

- **Unity プロジェクト**: 根は `m0/`（`Assets/` の Domain・Persistence・Platform の asmdef をそのまま使う)。版は **Unity 6.3 LTS(6000.3.x)**。UI Toolkit の採否(U-17)は §15.5 の基準で M1 の試作のあとに決める。
- **ゲート A(済)**: Unity で Domain・Persistence・Platform.Abstractions がエラー 0。見つけたこと: `Ekiden.Persistence.asmdef` の `references` に `Ekiden.Platform.Abstractions` がなく（`.csproj` は Platform を参照していた)、`dotnet test` では出ずに Unity だけでコンパイルエラーになった。asmdef と `.csproj` の参照は**両方**で確かめる。
- **ゲート B**: IL2CPP のビルドで ①T1 のベクタが参照モデルと全行一致（`compare_vectors.py <出力ファイル>`）②そのビルドの中で T3 の ①②③ が一致（`determinism.txt` に `×` なし)。**SimCli(.NET)と IL2CPP の SHA-256 の一致は求めない**(§8.4)。違うときは記録に残し、統計で比べる(§16.3)。実行は `Assets/Scripts/DevTools`（`VerifyBootstrap`・`Editor/VerifyMenu`）と `README` の手順。コードは SimCli と共有（`Assets/Scripts/Verification`)。
- **ゲート B(合格。2026-10-28)**: Unity 6000.3.24f1・Windows 11・i5-12400F・IL2CPP の Windows ビルド（`EkidenVerify.exe -batchmode -nographics -ekidenVerify`）で、終了コード 0・`compare_vectors.py` が `139 行: 不一致 0`・`result.txt` が `OK`。Editor(Mono)の T1・T3 も合格。つまずいたこと: ①IL2CPP のモジュールを足したあとは Unity

の再起動が要る ②Visual Studio の C++ ツール(MSVC v143)だけでは足りず、**Windows SDK** も要る(『Could not set up a toolchain for Architecture x64』)。

- **観測**: IL2CPP のビルドで `BitConverter.IsLittleEndian` が `False` と出た(x64 は実際にはリトルエンディアン。`env.txt` の記録にだけ使っている)。ゲームのコード(Domain・Persistence)は `BitConverter` もエンディアンも使わない(`grep` で 0 件)ので影響はないが、**エンディアンに依存するコードを書かない**こと。
- **未了**: T5(性能・GC)の IL2CPP での実測(`bench`)と、§15.5 の UI 技術の検証は、ゲート B のあと。

---

18. 課題・未決・他文書への反映

18.1 この文書が解決・追加した課題

ID	状態	内容	対応
L-12	**解決(09 §8.4)**	他端末で同じレース結果を再現するには `log`・`cos` を表引きにする必要がある	**実測**で、`normal()` は 1 年度で約 10^{-11}・`pace50` の `log` は約 2×10^{-9} の確率でしか `ds` が変わらないと示した。`pace50` は**表の値**にして 0 にする。端末間のビット一致は求めない。M-02 は『両チームを同じ端末で走らせる』で足りる(必ず一致が要る場合のみ `IDeterministicMath`)
L-22	一部解決	各ストアの購入状態 API の挙動が未検証	インターフェース・猶予・時計対策・通知のタイミングは 09 §10・§11・§15.4 で確定。**実機の挙動は M5**(L-34)
L-33	新規	世界の更新・スカウト・生成の乱数ストリーム(ラベル 3・6・7・8)の引く順序は 05 の記述順で、Python の参照(`random`)とビット一致しない	統計で受け入れる(§16.3)
L-34	新規	ストアのクラウド(Steam Auto-Cloud・GameKit・Play Games)の競合の返し方・サイズ・オフライン時の挙動は未検証	`ICloudSave` の背後に閉じ込め、判定は純関数(§9.5)。M5 で実機検証
L-35	新規	`ProfileData.counters` のマージは最大を取るので、複数端末で別々に増えた分が合計されない(達成が遅れる側にずれる)	v1 は許容。必要なら端末ごとの加算に変える(§9.6)
L-36	新規	04 の表の総合タイム(四捨五入)と画面の総合タイム(切り捨て)が 1 秒ずれることがある	画面が正(§13)。04 の表は参考値
L-37	新規	定数の名前が、文書(`K_…`)と参照モデルの `K` 辞書のキーで一致しない	M0 の最初に `K` 辞書から `Constants` を作り、文書の定数表との対応表を付ける(§17.1)
L-38	新規	C#・IL2CPP・実機での性能・GC・メモリは未測定(Python の値を上限としている)	M0 の SimCli `bench`・M1 の実機(§14.2)

ID	状態	内容	既定値
U-14	**解決(09 §11)**	実績の取り扱い	本体で判定・記録し、ストアへは通知(ミラー)。未接続は保留して後で送る
U-16	**確定(09 §3・§4)**	セーブスロットの数	3 スロット。各スロットに 自動 2 世代(A / B)+ 手動 1
U-17(新)	未決	Unity の版と UI Toolkit の採否	Unity 6 LTS・UI Toolkit。M1 で §15.5 の基準を確かめ、満たせなければ uGUI
U-18(新)	未決	スマホのクラウド同期の実装方式	各プラットフォームの保存ゲーム(GameKit・Play Games)。自前のサーバーは持たない(01 §6.5 の原則)

18.2 用語(00 の用語集へ追加)

用語	意味
コンテナ(`EKSV`)	セーブのファイル形式。ヘッダ(JSON)+ 本体(gzip の JSON)+ CRC32(§4)

用語	意味
世代	自動セーブの A / B。交互に書く(§4.4)
保存番号(`save_counter`)	スロットの単調増加の番号。同期の判定に使う
系譜(`lineage_id`)	1 つの歴史の識別子。新規ゲームごとに別(§9.1)
記録(`record`)	同期の単位(`slot{n}`・`slot{n}m`・`profile`)
墓石	削除したことを表す記録(§9.1)
退避	競合・移行で置き換えられたセーブの控え。消さない(§9.4)
`LocalState`	端末ごと・同期しない内部状態(§3.3)
`Q6`	保存する `double` の小数 6 桁への丸め(§5.3)
高水位(`clock_hw`)	これまでに見た最大の時刻。時計の巻き戻しの検知(§10)
鍵つきストリーム	`Pcg32(derive(master_seed, label, a, b))`。状態を持たない乱数(§8)

18.3 反映した文書

文書	変更
00	09 を v0.1(`09_save_tech_spec.md`・`ref_model_09.py`)/ 検算の一覧に追加 / L-12 解決・L-22 一部解決・L-33〜L-38・U-14 解決・U-16 確定・U-17・U-18 / 用語集(§18.2)/ 仕様完了のチェックへの参照
01	§12 の技術設計の表(セーブ・乱数・マスタの置き場所(`StreamingAssets` をやめる)・Unity の版・プロジェクト構成)/ §13 の M0 の完了条件
02	v1.0: §1 再現性(`pace50` の表)/ §3.10 `ProfileData.entitlements` を `LocalState` へ(§3.3)/ §3.12 `DeviceSettings.cloud_sync` を追加・`SaveMeta` はヘッダの部分集合 / §4.9 `rng`・`schema_version` の規則・サイズ上限(確定)・刈り込みの規則 / §5 の検証の分類(R/F/W)への参照
03	§3.2 の鍵の予約 → 登録表は 09 §8.2 / L-12
04	§12 に鍵の登録表(09 §8.2)・`pace50` の表(09 §8.4)・総合タイムの表記(切り捨て。09 §13)の注
05	§13 の注に『乱数ストリームの鍵は 09 §8.2』/ §14 に『統計の受け入れの帯は 09 §16.3』
06	`event_state.log` の保持件数(`K_EVENT_LOG_MAX` = 50)の注
07	§8.1・§8.2: 購入状態のキャッシュは `LocalState`・時計対策は 09 §10 / §14 の引き継ぎの完了
08	§7.4 の整形の正本は 09 §13(タイム差の例を規則に合わせて修正)/ §13.4 の引き継ぎの完了 / SC-03・SC-40 の同期の状態と競合の確認(09 §9.4)/ SC-04 に『クラウド同期』/ U-16 確定

19. テストベクタ(`verify/ref_model_09.py`)

TV-1 CRC32 とコンテナ(§4.1〜§4.3。`vectors` / `container`)

`CRC32("123456789") = 0xCBF43926`。最小のコンテナ(ヘッダ: `kind=auto, slot=1, save_counter=7, schema_version=1, year=3, month_index=6, school_name=大和 ほか`、本体: `{"schema_version":1,"clock":{…}}`)は **301 バイト**。先頭 16 バイトは `45 4b 53 56 | 01 00 | 01 00 | c2 00 00 00 | 7b 22 73 6c`(`EKSV` / `container_ver 1` / `flags 1` / `header_len 194` / ヘッダ JSON の先頭 `{"sl`)。往復・ヘッダだけの読み取りが一致し、ヘッダ CRC32 の位置は 206、末尾 4 バイトがファイル CRC32。100 年度相当のセーブ(127,682 バイト)で、切り詰め 127,682 通り・ビット反転 20,000 回・8 バイトの書き換え 20,000 回の見逃し **0**、新しいコンテナ版・ヘッダ 64 KB 超・ペイロードの宣言 64 MB 超・圧縮爆弾(6,155 バイト → 6 MB。上限を 1 MB にして)をすべて拒否。

TV-2 A / B の世代と電源断(§4.4。`vectors` / `container`)

5 回書いたあとのファイルは `slot1.a` `slot1.b` の 2 つ。最新 = 保存番号 5(世代 A)。最新を壊す(末尾 3 バイトを欠く)と、保存番号 4(世代 B。1 つ前)で読み込め、壊れた世代 = 1。書き込みの各手順で止めた 5 通り + 完走 1 通り + 置き換え先を途中で切った 128 通り = **134 通りで、読めなかった = 0**。

TV-3 中断 → 再開(§4.6。`replay`)

3 大会 × 300 レース(12 チーム)で、区間 k の前に中断し、再開後に k 以降の指示を変えても、完了した 14,196 区間の ds・累積が変わった = **0**。

TV-4 `Q6` と書式(§5.3。`q6`)

乱数 1,000,000 個で、書式 → 読み戻しの不一致 0・`Q6` が冪等でない 0。端の値: `0.0 → 0`、`-0.0 → 0`、`1e-07 → 0`、`5e-07 → 0.000001`、`0.1+0.2 → 0.3`、`123456789.12345679 → 123456789.123457`、`2.5e-06 → 0.000003`、`-3e-06 → -0.000003`。

TV-5 マイグレーション v0 → v1(§6.3。`vectors` / `migrate`)

入力 `{schema_version:0, annual_plan.focus:"Endurance", runners["1"].pb:{…}, settings.event_auto:true}` → 適用 `['v0→v1']`、`focus = Long`、`target_race = NewYear`、`pb` なし、`event_auto = true`(保つ)、`schema_version = 1`。冪等・新しい版の拒否・未知の項目の保持・ランダムな旧データ 2,000 件(不整合 0)。

TV-6 読み込み時の検証と修復(§7.2。`validate`)

正常 300 件で修復 0。壊した 3,000 件: 修復で足りる 1,506 件は修復後の違反 0・冪等・致命にすべき 1,494 件はすべて致命・修復で足りるのに致命にした 0・必須項目の欠落 7 / 7 を拒否。

TV-7 乱数の鍵(§8。`rng` / `perturb`)

登録表 26 区間で重なり 0。予熱 15 年度 + 3 年度の鍵 520,798 個で衝突 0。1 人・1 区間の乱数の消費 = 8(20,000 通り)。適合ベクタ:

入力	期待値
`derive(123456789, 1, 7, 100)`	`10206515190289864039`(`0x8DA4D37D37DB1567`)
その `Pcg32` の `u32()` ×3	`2889221095`, `1724165226`, `319155939`
`uniform()` ×2	`0.672699`, `0.401438`
`normal(0, 1)`	`-0.725096`
`derive(123456789, 2, 7, 100)`	`12557608959185654001`
04 TV-3: `Pcg32(derive(20260101, 4, 11, 101))` で `draw_leg`(第一戦・維持・8.0 km・18°C・Fast)の雑音	`+0.003477`

端末差(`perturb 300`): `normal()` は 1 ulp で 1 区間あたり 1.3×10^{-14}(1 年度 7×10^{-12})、`pace50` は 3.6×10^{-12}(1 年度 2×10^{-9})。`nextafter` による実際の 1 ulp のずれ(300 レース・36,000 区間)で変化 0。

TV-8 同期の判定(§9.2。`sync`)

§9.2 の表の 15 件(不一致 0)。

TV-9 同期の乱数シミュレーション(§9.7。`sync`)

前提つき書き込み: 400 回 × 300 手で、失われた更新 0・収束しなかった 0(前提なしでは 36 / 400 回で失われる)。ふつうの使い方 36,000 セッション × 4 通りで、失われた更新 0(競合は同期できなかったときだけ。0% / 0% で 0 回)。`ProfileData` のマージは可換・結合的・冪等(3,000 組で違反 0)。

TV-10 購入状態と時計対策(§10.3。`clock`)

13 件(07 TV-3 の 5 件を含む判定と一致)+ 性質(正直な利用者 4,000 人で未確認 0 / 操作する利用者 4,000 人で 120 時間超でも有効 0)。

TV-11 数値・時間の整形(§13。`fmt`)

59 件の一致 + `FmtTime` の `ds` 0〜99,999 の往復と 08 の実装との一致(不一致 0)+ `FmtPct` の単調性(0.0001 刻みで逆戻り 0)+ `FmtDiff` の対称性。

検算の一括実行: `python3 -I verify/ref_model_09.py all` (約 1 分。`size` `container` `migrate` `validate` `rng` `perturb` `sync` `clock` `q6` `replay` `timing` `fmt` `inventory`)。個別は `... vectors|size|container|migrate|validate|rng|perturb|sync|clock|q6|replay|timing|fmt|inventory`。

第10部 個性・素養仕様

原典: `10_individuality_spec.md`　　原典の版: v0.1　　改版の経緯: なし(初版のまま)

> 範囲: 選手の**個性**(型・走法・展開。全員が持つ)と**素養**(登坂・下り・エース。まれで隠し)。定義、生成、練習への影響(能力別の成長・絆・個別指導・相性の印)、レースへの影響、特殊能力の取得への影響、見せ方(判明・視察・画面)、効果の大きさの検算。前提: `01` 〜 `09`。特に `03` §4.7・§6(成長の式)、`04` §7.3(区間タイム)、`05` §6.2〜§7(候補の生成・視察)、`06` §9(特殊能力の取得条件)、`08` R-17(画面)、`09` §8.2(乱数の鍵)。検算: `verify/ref_model_10.py`（`python3 -I ref_model_10.py vectors|gen|train|neutral|select|bond|race|skill|world|all`）。03・04・05・06 の参照モデルにある挙動不変のフック（`gm bond_k ind_mult / style / individuality`）を通して、**本物の式の中で**測った。仕様書の数値はここから出力した値。位置づけ: **後から足した文書**(番号は 09 の後)。01〜09 を書き終えたあと、『素養のない大多数の選手に育成の判断が無い』という指摘から足した。数値と規則は各文書に反映済みで、本書はその根拠と全体像をまとめる(§11)。

1. 要約

項目	内容
個性	全員が3つの軸を持つ。**型**(スピード型 / バランス型 / ロング型)・**走法**(集団走◎ / 普通 / 単独走◎)・**展開**(追い上げ型 / 普通 / 逃げ型)。見える情報。
素養	5人に1人弱(20%)が持つ隠しの才能。**登坂 / 下り / エース**から0〜2個。入部から12週で判明する。
練習への影響	能力ごとの成長の乗数(型・展開は ±20%、素養は ×1.15)・絆の効き(集団走◎は 2 倍、単独走◎は 1/3)・個別指導の効き(単独走◎は ×1.25)。
レースへの影響	走法は『集団の追走側』『単独走』の補正の倍率、展開は『追う展開』の補正と先頭走のブレ。**味付け**(勝率を動かさない。§6)。
特殊能力への影響	取得条件に素養・個性を使う。素養のある選手は**山の神・山下りの達人・エース区間○**に届く。素養がなければ、標準の練習では届かない水準(§7)。
何が変わるか	同じ入学生でも**同じ練習で別の選手に育つ**(速さ向きと長さ向きの差の標準偏差 +21%。型でグループ分けすると +46%)。『この子はどのグループで、どのテーマを多くするか』という判断に、**見える手がかり**(個性のチップ・相性の印)が付く。
何が変わらないか	チーム全体の強さの**期待値**(平均して中立。個性+素養で +0.1〜+0.3 P)。05 の長期バランス(優勝率・クリア年度)は変わらない。個別の選手の練習を最適にしても、同じ最良の方針で揃えたチームより +0.2〜+0.4 P にしかならない(登録16人から最適に選ぶため。§9.4)。

この文書の結論(正直に): 個性は『育成の判断を**増やす**』が、『判断の**価値**を大きくする』ものではない。チーム全体の強さに対する**構造的な上限**があるため、判断の価値を大きくするほど勝率が個性に支配される(05 の較正が崩れる)。したがって個性の主な働きは、**(1)選手の違いが見える・育ち方が違う(多様性)(2)グループの編成・個別指導・絆に意味が出る(3)素養が特殊能力への道を作る**、の3つ。

---

2. 定義

2.1 個性（`Runner.style`。3文字）

軸	値(コード)	日本語	意味
型 `StyleForm`	`S` / `B` / `L`	スピード型 / バランス型 / ロング型	スピード型は SPD が伸びやすく STA が伸びにくい。ロング型は逆。バランス型は偏りなし
走法 `StyleRun`	`P` / `F` / `S`	集団走◎ / 走法は普通 / 単独走◎	集団走◎は仲間との絆が練習に効きやすい。単独走◎は絆の効きが小さいが、個別指導の週の効率が高い
展開 `StyleFlow`	`C` / `E` / `L`	追い上げ型 / 展開は普通 / 逃げ型	追い上げ型は KCK が伸びやすく PAC が伸びにくい。逃げ型は逆

- コードは `型 + 走法 + 展開` の3文字(例: `SPC` = スピード型・集団走◎・追い上げ型)。**中立は `BFE`** (個性のない選手・他校の選手・不正値の修復後)。

- 画面に出すチップは**普通でない軸だけ**(全部普通なら『標準型』)。軸の意味は §5〜§6 のとおり。

2.2 素養(`Runner.gifts` 。0〜2個・重複なし。`gift_revealed`)

素養 Gift	日本語	成長	素質の上限	特殊能力
`Climber`	登坂	CLB の XP ×1.15	`potential[CLB]` +6	山の神への道(§7)
`Descender`	下り	DWN の XP ×1.15	`potential[DWN]` +6	山下りの達人への道
`Ace`	エース	MEN・CON の XP ×1.15 / ほかの能力 ×1.05	MEN・CON +6 / ほかの能力 +2	エース区間〇への道

- 素養は**隠し**。判明するまで画面に出さない(§8)。ただし**取得条件・成長には隠しのまま効く**。
- 素質の上限は、入学時に `potential` へ加える(100 を超えない)。入学後は 03 の式のまま(`potential` は能力の上限)。
- スピード・持久の素養は作らない(型が同じ役割を持つため。検討と却下は §3.3)。

2.3 派生値(保存しない。`style` と `gifts` から計算)

派生値	内容	使う場所
`gm[10]`	能力ごとの XP の乗数。型・展開・素養の**積**(§5.1)	03 §6 手順2
`bond_k`	絆の効きの係数。集団走◎ 0.30 / 普通 0.15 / 単独走◎ 0.05	03 §6 手順1
`ind_mult`	個別指導の週の効率。単独走◎ ×1.25 / それ以外 ×1.0	03 §6 手順1

2.4 定数表(初期値案。`K_` 接頭辞。`ref_model_10.py` の `K` と一致させる)

定数	値	意味
`K_FORM_UP` / `K_FORM_DOWN`	0.20 / 0.20	型: 得意の能力(スピード型は SPD、ロング型は STA)の XP ×1.20 / 苦手の能力 ×0.80(**対称**)
`K_FLOW_UP` / `K_FLOW_DOWN`	0.20 / 0.20	展開: 追い上げ型は KCK ×1.20・PAC ×0.80 / 逃げ型は PAC ×1.20・KCK ×0.80(対称)
`K_BOND_K`	集団走◎ 0.30 / 普通 0.15 / 単独走◎ 0.05	走法ごとの絆の効き(`E_bond = 1 + bond_k × bond_avg / 1000`)。普通は 03 の従来の値
`K_IND_MULT`	集団走◎ 1.0 / 普通 1.0 / 単独走◎ 1.25	走法ごとの、個別指導の週の効率
`K_GIFT_XP`	0.15	素養の成長の乗数(+15%)
`K_GIFT_ACE_OTHER`	0.05	エースの素養の、MEN・CON 以外の能力の乗数(+5%)
`K_GIFT_POT` / `K_GIFT_ACE_OTHER_POT`	6 / 2	素養の素質の上限の加算(エースの MEN・CON 以外は +2)
`K_GIFT_REVEAL_WEEKS`	12	入部から素養が判明するまでの週数(§8)
`K_FIT_GOOD` / `K_FIT_OK`	1.05 / 0.99	相性の印の境界(◎ ≥ 1.05 / 〇 ≥ 0.99 / △ それ未満。§5.4)
`K_RUN_SCALE`	集団走◎ 1.5 / 普通 1.0 / 単独走◎ 0.5	走法の、集団走・単独走の補正の倍率(04 §7.3)
`K_STYLE_CHASE_ADJ`	追い上げ型 −0.2% / 普通 0 / 逃げ型 +0.2%	追う展開のときの補正(04 §7.3)
`K_LEAD_NOISE` / `K_LEAD_BRAKE`	逃げ型 0.85 / 0.8、追い上げ型 1.15 / 1.2	先頭走のときの雑音とブレーキ率の倍率(04 §7.3)
`K_STYLE_RUN_P` / `K_STYLE_FLOW_P`	30 / 40 / 30 %	走法(集団走◎ / 普通 / 単独走◎)・展開(追い上げ / 普通 / 逃げ)の分布。独立(§4)
`K_STYLE_FORM_P`	§4 の表	入学時の『型』(能力の偏り)ごとの、個性の『型』の分布
`K_PTYPE_W`	バランス 35 / 持久 25 / スピード 25 / 山 15 %	入学時の『型』(能力の偏り)の分布(05 §6.1。検算用にモデルへ複製)
`K_GIFT_N`	80 / 18 / 2 %	素養の個数(0 / 1 / 2)
`K_GIFT_KIND_W`	登坂 38 / 下り 31 / エース 31 %	素養の種類

3. 設計の原則

3.1 原則

ID	原則	具体化
P-1	**全員が違う**	個性は全員が持つ(素養のある20%だけに判断を寄せない)。3軸 × 3値 = 27 通りの組み合わせ
P-2	**平均して中立**	型・展開の ±20% は対称。個性を付けても、チームの強さの期待値は +0.0〜+0.16 P しか動かない(§9.1)。05 の長期バランスの較正が保たれる
P-3	**見えて、方向がわかる**	個性は見える情報。『この子には、このテーマが合う』を相性の印(◎○△)で出す。数値の細部(乗数)は出さない
P-4	**レースでは味付けにとどめる**	勝敗を個性が決めない(全員に個性を付けても勝率は変わらない。§6)。選手の特徴を演出に使える
P-5	**素養はまれで、隠れていて、道を作る**	20% が持つ。判明までは隠し(発見の楽しみ)。効果は『伸びやすい』にとどめ、**特殊能力への道**として働く(§7)
P-6	**判断の価値の限界を認める**	チーム全体では登録16人から最適に選ぶので、個別の育成の最適化の価値は小さい(§9.4)。判断を増やしても、判断の価値は上げない

3.2 構造的な限界(なぜ、判断の価値を大きくできないか)

- 区間オーダーは、登録16人から大会ごとに最適に選ぶ(04 §4・§5)。ある選手の育て方が悪くても、**別の選手を使えば済む**ので、個別の最適化の価値は薄まる。
- 実測(§9.4): 全員を同じ最良の方針で育てる → 各選手の最適な方針で育てる、の差はチーム全体で **+0.2〜+0.4 P**(個性の有無でほぼ同じ)。個性の効果量を上げても、上限は大きくは動かない見込み(『合う方針を選ぶ価値』は上がるが、チームの強さは登録の中の最良の選手を使うため頭打ちになる。±30〜40% は未測定)。
- 勝率への換算: +0.5 P ≈ 優勝率 +0.035(§9.3)。現実的な使いこなし(+0.2〜+0.5 P)は、05 の優勝率の許容帯(中堅 0.553 ± 0.058。L-69 で取り直した値。個性の検算をしたときは 0.535 ± 0.051)の内側に収まる。全員の最適を知っているオラクルの水準(+1.0 P で +0.090)だけが帯を超える。

3.3 検討して採らなかった案

案	理由
素養を『成長率 +15〜25%』だけ(素質の上限なし)	最良の方針は素養なしと同じで、選手の P は +0.3〜+1.1 にとどまる。判断が変わらない(`exp_gift.py train`)
素養を『素質の上限』だけ	同上。さらに、上限に届くまでの差が見えにくい
スピード・持久の素養	型(スピード型・ロング型)と同じ役割。素養と個性が重なるので、型に一本化した
素養に『取得条件への影響』を持たせない	山の神が**誰にも取れない**(検算用チームで 0/1,224。標準の練習では条件の能力に届かない)問題が残る。素養を道にする(§7)
個性の効果を大きくする(±30〜40%)	測っていない(見込み)。構造的な上限(§3.2)は動かないので、判断の価値は上がらず、平均の効きのずれ(多様性が増す分)だけが大きくなる。対称な ±20% で、多様性 +21〜46% は出せている(§5.5)
個性を勝率の補正として大きくする(レース)	04 の較正が崩れる。『対称』を保つ範囲で味付けにとどめた(§6)
選手ごとの継続方針(『この子は1年間ずっと山重視』を選手単位で持つ)	個別の最適化の上積みは +0.2〜+0.4 P で(§5.5)、勝率への効果が小さい。月ごとの個別指導(月4人。03 §5.7)と、山対策のグループ(C)など**グループの使い分け**で足りるので、v1 は持たない(確認済み)
他校の選手にも個性・素養を持たせる	世界の更新は抽象(`RivalRunner`)。個性は平均して中立なので、持たせなくても自校の勝率の目標値は変わらない。実装も重くなる(L-41)

4. 生成

入学が確定したとき(05 §12)、`Runner` に個性・素養を付ける。候補の段階では**保存しない**(`master_seed`・`look_seed`・`profile` から決定的に作れるので、視察で見せるときも同じ関数で作る)。

乱数(09 §8.2。label 8、a = `look_seed`):

b	用途	引く回数と順序(固定)
40	個性	3 回: ① 型 ② 走法 ③ 展開
41	素養	3 回: ① 個数 ② 1つ目の種類 ③ 2つ目の種類(個数が 2 のときだけ使う。1つ目と同じなら、次の種類に送る)

個性の分布

項目	分布
型	入学時の『型』(05 §6.2 の能力の偏り `profile`)から決まる: **バランス** → スピード 15% / バランス 70% / ロング 15%。**持久** → スピード 5% / バランス 5% / ロング 90%。**スピード** → スピード 90% / バランス 10%。**山** → バランス 40% / ロング 60%(全体で スピード 29% / バランス 34% / ロング 37%)
走法	集団走◎ 30% / 普通 40% / 単独走◎ 30%
展開	追い上げ型 30% / 普通 40% / 逃げ型 30%(走法と独立)

素養の分布: 個数 0 個 80% / 1 個 18% / 2 個 2%。種類は 登坂 38% / 下り 31% / エース 31%。

- 招聘選手(07 §6.2)も同じ関数・同じ分布で作る(個性の『型』は 07 §6.2 ④ で引いた型から決める。素養を保証・増やしたりしない。07 §6.5)。
- 他校の選手(`RivalRunner`)は個性・素養を持たない。OB の記録(`OBRecord`)は、在籍中の `style` と `gifts` を保存する(02 §4.7。素養は判明していたものだけを表示)。
- **素養は能力を直接上げない**(上限だけを入学時に加える)。入学時の能力は 05 §6.2 の展開のまま。

検算(`ref_model_10.py gen` 。20,000 人): 型 バランス 0.340 / ロング 0.367 / スピード 0.292。走法 普通 0.397 / 集団走◎ 0.301 / 単独走◎ 0.302。展開 追い上げ 0.301 / 普通 0.401 / 逃げ 0.298。素養の個数 0 個 0.802 / 1 個 0.178 / 2 個 0.020(のべ 登坂 0.081 / 下り 0.069 / エース 0.067)。入学時の型 → 個性の型は表のとおり(許容 ±0.01)。

---

5. 練習への影響

03 §6 の週の解決に、3つの派生値(§2.3)を入れる。**個性・素養のない選手(従来)は `gm = 1`・`bond_k = 0.15`・`ind_mult = 1` で、従来と同じ値になる**(03 のテストベクタは不変)。

5.1 能力別の成長の乗数 `gm[s]`

型・展開・素養の**積**。

種類	乗数
型	スピード型: SPD ×1.20・STA ×0.80 / ロング型: STA ×1.20・SPD ×0.80 / バランス型: なし
展開	追い上げ型: KCK ×1.20・PAC ×0.80 / 逃げ型: PAC ×1.20・KCK ×0.80 / 普通: なし
素養	§2.2 のとおり

```
add_s = round_half_up(g[s] × K_XP_SCALE × E × facility_mult(s) × room_s × gm[s])      # 03 §6 手順2。ApplyXp(レベルアップ)は変わらない
```

- **型の苦手の能力も、練習すれば伸びる**(×0.80)。スピード型でも STA は伸びる。
- 型の乗数は、練習のテーマごとの XP 配分(03 §4)と掛け合わさるので、**テーマによって合う・合わないが出る**(§5.4)。例: ロング型は基礎ビルド(STA・PAC が中心)で ×1.16、スピード強化で ×0.96。

5.2 走法: 絆と個別指導

走法	bond_k	ind_mult	意味
集団走◎	0.30	1.0	仲間との絆が練習に効く。**同じグループで育てるほど有利**(グループを替えると絆が下がる)
普通	0.15	1.0	従来
単独走◎	0.05	1.25	絆に頼らない。**個別指導の週は効率が高い**(指導ポイントを使う価値が上がる)

```
E_bond = 1 + bond_k × bond_avg / 1000          # 03 §6 手順1。bond_avg: 同グループの仲間との絆の平均(0〜1000)
個別指導の週: E ×= ind_mult                    # E_MAX で頭打ちにする前
```

効果の大きさ(`ref_model_10.py bond`。30 人・4 年・バランス計画。個性の型・展開は普通にして走法だけを変えた P_Long の平均)

走法	同じグループ(安定)	半年ごとにグループを替える	毎週が個別指導
集団走◎	65.80(普通・安定との差 **+0.69**)	64.56(**−1.25**)	65.80(±0)
普通	65.12(基準)	64.48(−0.63)	65.12(±0)
単独走◎	64.61(**−0.50**)	64.44(−0.18)	66.50(**+1.89**)

- 表の括弧は、各走法の『同じグループ(安定)』との差(左端の列だけ、普通・安定との差)。
- 集団走◎は**グループを安定させるほど得**で、替えると大きく損をする(−1.25)。単独走◎は**個別指導をするほど得**(指導は 1 ポイントで月末まで。ポイントは月 4 なので、毎月 4 人まで続けられる)。上限の見積もり: 単独走◎ 4 人が毎月指導を受けて全員が先発に入ると、チーム平均で 4 × (66.50 − 65.12) / 10 ≈ **+0.55 P**(§9.3 の範囲)。
- 『グループを替えると絆が下がる』(03 §7.1)ことに**選手ごとの意味**が付く。

5.3 個別指導との関係

個別指導(03 §5.1)の指定は従来どおり。単独走◎の選手の指定週は `ind_mult` で効率が上がる。画面(SC-20)は、指定するメニューの**予測**(R-05)に `ind_mult` を含めて出す(予測は 03 §6 の式から乱数を除いたものなので、自動で含まれる)。

5.4 相性の印(画面に出す。規則には作用しない)

選手の `gm` と、テーマの XP 配分から、『このテーマは、この選手に合うか』を出す。

```
theme_gain[s] = そのテーマの4週で、能力 s に入る XP の合計(03 §4。走行量を含む)
fit(r, theme)  = Σ_s theme_gain[s] × gm_view[s] / Σ_s theme_gain[s]
gm_view        = 型・展開と、判明した素養だけを入れた gm(未判明の素養は入れない。§8)
印:  ◎ fit ≥ K_FIT_GOOD(1.05) / ○ fit ≥ K_FIT_OK(0.99) / △ それ未満        # 画面は『相性◎』のように語を付ける(08 §4.2)
```

- 1.00 = 標準。**素養だけでは ◎ にならない**(登坂の素養の山対策で 1.033 = ○)。◎ は型と展開の組み合わせが、テーマの配分に合うとき。
- 例(TV-4): ロング型・逃げ型は 基礎ビルド 1.156◎ / スピード強化 0.964△ / 山対策 1.067◎。スピード型・追い上げ型は 基礎ビルド 0.844△ / スピード強化 1.036○。
- **印は『合う』を出すだけで、『これを選べ』とは言わない**(個性のないグループの推奨プランは変えない。08 R-04)。

5.5 効果の大きさ(`ref_model_10.py train` / `neutral` / `select`。同じ入学生の対比較)

チーム適性値の換算 P(大会ごとに、全 16 人から最適に区間を選んだチームの強さ。大きいほど強い)。

(a) 平均の効き(`neutral`。全員バランス計画。40 スクワッド。個性なしとの差)

	第一戦(Fall)	第二戦(National)	第三戦(NewYear)
個性のみ	+0.14(標準誤差 0.04)	+0.16(0.04)	+0.01(0.03)
素養のみ	+0.07(0.02)	+0.07(0.02)	+0.07(0.02)
個性 + 素養	**+0.21**(0.04)	**+0.22**(0.04)	**+0.09**(0.03)

→ 平均して**ほぼ中立**(+0.1〜+0.2 P。多様な選手がいるほど区間の当てはめがよくなる分と、素養が純粋に有利な分)。入学生の組が違う 30 スクワッド(`train`)では、個性 + 素養の差は +0.32 / +0.34 / +0.09。**平均の効きは +0.1〜+0.3 P**。

(b) 育て方の違い(`train`。30 スクワッド × 16 人。個性ありの選手を、方針ごとに育てた場合)

比べるもの	第一戦	第二戦	第三戦
全員を相性の高いテーマで育てる − ランダムな 3 グループ(`g - r`)	+0.49	+0.35	+0.02
型でグループ分けして育てる − ランダムな 3 グループ(`m - r`)	+0.20	+0.18	−0.00
全員を同じ最良の方針で育てる − 全員バランス(`b - a`)	+0.97	+0.62	+0.30
相性の高いテーマ − 全員が同じ最良の方針(`g - b`)	+0.07	+0.15	−0.14
選手ごとの最適(上限)− 全員が同じ最良の方針(`o - b`)	+0.19	+0.31	+0.43
型でグループ分け − 全員バランス(`m - a`)	+0.76	+0.60	+0.13
相性の高いテーマ − 全員バランス(`g - a`)	+1.04	+0.76	+0.16
選手ごとの最適(上限)− 全員バランス(`o - a`)	+1.16	+0.93	+0.73

→ **個性を無視してランダムに分けると 0.2〜0.5 P 損をするが、一つの良い方針に揃える(年間方針。`b`)ことで大半が取れる**。全員バランスとの差は、選手ごとの最適(`o`)で +0.7〜+1.2 P だが、その大半(`b - a` = +0.3〜+1.0)は年間方針ひとつで取れ、**個性を使って上積みできるのは +0.2〜+0.4 P**(`o - b`)。個性なしのチームでも同じ(§3.2)。

(c) 多様性(速さ向きと長さ向きの差 P_Fast − P_Long の、卒業時の標準偏差)

	値
個性なし	1.98
個性あり(全員バランスで育てた)	2.39(**+21%**)
個性あり(型でグループ分けして育てた)	2.89(**+46%**)

→ 同じ入学生でも、**個性があると卒業時の姿が変わる**(同じ練習でも、速さ向きの子と長さ向きの子に分かれる)。型に合わせて分けて育てると、差はさらに広がり、**区間ごとの適任者**(速い区間・長い区間・山)がはっきりする。

(d) 登録の選び方の価値(`select`。『総合力の上位だけから選ぶ』と『登録 16 人から大会ごとに最適に選ぶ』の差。P)

育て方 / 個性・素養	第一戦	第二戦	第三戦	多様性
全員バランス / なし	0.34	0.16	0.04	1.98
全員バランス / あり	0.36	0.19	0.06	2.39
型でグループ分け / なし	0.35	0.16	0.09	2.41
型でグループ分け / あり	0.42	0.22	0.11	2.89

→ 個性があると、**オーダー(04 §4・§5)で『誰をどの区間に置くか』の価値が少し上がる**(+0.02〜+0.07 P)。

---

6. レースへの影響(04 §7.3 が確定)

個性	状況	補正
走法	集団の追走側(集団走の補正)	集団走◎ −0.45% / 普通 −0.30% / 単独走◎ −0.15%(`K_RUN_SCALE` を掛ける)
走法	単独走	集団走◎ +0.30% / 普通 +0.20% / 単独走◎ +0.10%
展開	追う展開(前のチームが `G_CHASE` 秒以内)	追い上げ型 −0.2% / 普通 0 / 逃げ型 +0.2%
展開	全体の先頭(2区以降)	逃げ型は雑音 ×0.85・ブレーキ率 ×0.8 / 追い上げ型は雑音 ×1.15・ブレーキ率 ×1.2
乱数	—	**引く回数は変わらない**(04 §12 の 8 回)

- 区間タイムの 1% = 適性値 P の約 4.62 点(21 km の基準ペース)。上の補正は、その状況で **±0.5〜0.9 P**(走法 ±0.69 / ±0.46 P、追う展開 ±0.92 P)。

- 特殊能力(集団走〇・単独走×・逃げ切り・ごぼう抜き)は、個性の補正のあとに従来どおり重なる(04 §7.3)。個性と特殊能力が一致するほど効果が重なる(例: 集団走◎ + 集団走〇)。

中立の確認(`ref_model_10.py race`。各 800 回、同じ乱数。全チームの全選手に個性を付ける。自校の力 ΔP = +3 / 0 / −3)

ΔP	大会	個性なし	個性あり・配置は力だけ	個性あり・配置も個性を見る
+3	第一戦	0.519	0.515(−0.004)	0.506(−0.009)
+3	第二戦	0.521	0.526(+0.005)	0.515(−0.011)
+3	第三戦	0.525	0.525(+0.000)	0.541(+0.016)
0	第一戦	0.176	0.172(−0.004)	0.171(−0.001)
0	第二戦	0.182	0.185(+0.003)	0.186(+0.001)
0	第三戦	0.144	0.146(+0.003)	0.146(+0.000)
−3	第一戦	0.020	0.021(+0.001)	0.022(+0.001)
−3	第二戦	0.024	0.020(−0.004)	0.026(+0.006)
−3	第三戦	0.014	0.015(+0.001)	0.014(−0.001)

→ 優勝率の変化は最大でも **±0.016**(標準誤差は約 ±0.017)で、**統計的に有意でない**。個性を見て区間を配置しても、利用できる差は出ない。**レース側の個性は味付け**(実況・演出・選手の特徴の語り)。

---

7. 特殊能力の取得への影響(06 §9 が確定)

素養・個性は、特殊能力の**取得条件(DSL)の変数**になる(06 §3.2: `r.gift.climber` `r.gift.descender` `r.gift.ace`、`r.is_speed` `r.is_long` `r.is_pack` `r.is_solo` `r.is_chase` `r.is_lead`)。

特殊能力	新しい取得条件(06 §9。どちらかを満たす)
山の神(Gold)	**素養(登坂)**: CLB ≥ 64・坂道練習 24 週以上・2年生以上 / **素養なし**: CLB ≥ 70・坂道練習 32 週以上・3年生以上
山下りの達人(Blue)	**素養(下り)**: DWN ≥ 62・坂道練習 16 週以上 / **素養なし**: DWN ≥ 68・坂道練習 28 週以上
エース区間〇(Blue)	3年生以上で、**素養(エース)**: チーム内の力の順位が4位以内・出場 2 回以上 / **素養なし**: 順位が2位以内・区間賞 4 回以上
ごぼう抜き(Blue)	追い抜きの回数 ≥ 4 − (追い上げ型なら 1)
逃げ切り(Blue)	先頭の区間の数 ≥ 2 − (逃げ型なら 1)
集団走〇(Blue)	集団走の区間の数 ≥ 6 − (集団走◎なら 2)

検算(`ref_model_10.py skill`。中堅・8 回 × 20 年度・卒業生 1,224 人。06 の簡易チームシミュレーション。K0 = 個性・素養なしで従来の条件 / K3 = 新しい条件・全員が標準の練習 / K4 = 新しい条件・**素養のある選手は山対策を続ける**)

特殊能力	K0(従来)	K3(標準の練習)	K4(個性に合わせた練習)
山の神	0.000	0.000(素養ありでも 0.000)	**0.017**(素養あり 0.183 / なし 0.000)
山下りの達人	0.064	0.007(素養あり 0.114)	0.043(素養あり **0.633** / なし 0.003)
エース区間〇	0.062	0.034(素養あり **0.482** / なし 0.002)	0.033(素養あり 0.470 / なし 0.002)
ごぼう抜き	0.000	0.001	0.001
逃げ切り	0.009	0.047(逃げ型 0.125 / それ以外 0.011)	0.050(逃げ型 0.138)
集団走〇	0.004	0.020(集団走◎ 0.061 / それ以外 0.002)	0.024(集団走◎ 0.075)
卒業時の特殊能力の平均	0.46 個	0.44 個	**0.50 個**

- **従来の条件では山の神が誰にも取れなかった**(0/1,224。標準の練習で CLB が 66 に届かない)。素養のある選手が山対策を続ければ、素養ありの 18% が取れる(卒業生 10 人/年で、約 **6 年に 1 人**。Gold にふさわしい頻度)。
- 素養のない選手は、**山対策を続けても標準の練習でも、山下りの達人・エース区間〇にほとんど届かない**(素養なしで 0.000〜0.003)。素養が『道』を作る。K0 では 6% が偶然取れていた(素養と無関係)ので、**取れる人が減って、取れる理由が明確になる**。

- 個性に合わせた練習(素養のある選手に山対策を続ける)で、卒業時の特殊能力の平均が 0.44 → 0.50 個に増える。**プレイヤーの判断が結果に出る。**
- 逃げ切り・集団走○は、個性があると取りやすい(逃げ型 12〜14%、集団走◎ 6〜8%)。**個性が合う選手が、合う場面を経験するうちに身につく。**
- **ごぼう抜き**: 検算用の簡易シミュレーションでは、追い抜きの回数(`overtakes`)がほとんど増えず、取得率が測れていない(06 の大会の事象は確率で近似。L-18)。条件の式は同じ形で、実装の本物のレースの結果(04)から数える。

---

8. 見せ方

8.1 個性は見える、素養は隠す

対象	個性(型・走法・展開)	素養
自校の選手	**入部した時から表示**(チップ)	**入部から `K_GIFT_REVEAL_WEEKS` (12)週で判明。**月末レポートの『お知らせ』に出る(R-10)。判明までは画面に出さない
候補(スカウト)	**視察すると見える**(走り方は見ればわかる。05 §6.3)	**練習会に招待すると『素養あり』だけ**が見える。**種類はスカウト担当が Lv3 以上のとき**に見える(Lv2 以下は入部後の判明まで分からない)。招待していない候補は『不明』
他校の選手	なし(`RivalRunner` は個性を持たない)	なし
OB	保存した個性	判明していた素養だけ

- スカウトの段階で素養の種類を見た選手は、**入部と同時に判明**(`gift_revealed = true`)。
- 素養は**隠しのまま成長・取得条件に効く。**判明の前後で挙動は変わらない(表示だけが変わる)。
- **実装(M1 準備。00 L-64)**: 週の解決(`TeamWeek.TrainAll`)で、`週 - 入部週 ≥ K_GIFT_REVEAL_WEEKS` の選手を `gift_revealed = true` にし、素養を持つ選手には `WeekReport.GiftRevealed` の印を付ける(月末レポートの `gift_reveal` の材料)。素養のない選手も『なし』が確定するが、知らせない。スカウトで種類まで見えた選手(練習会の招待 + スカウト担当 Lv3 以上)は、入学の確定(`GameSession.Intake`)で判明にする。新規ゲームの初期の部員は入部から 12 週以上経っているので、最初の週に判明する(最初の月末レポートに『素養が判明した』が並ぶ。1 種別は最大 3 件で、残りは『ほか n 件』)。判明の前後で能力の伸びが変わらないことは試験(`GiftRevealTests`)で確認した。
- 隠しの情報は**相性の印には入れない**(§5.4 の `gm_view`)。ただし月末の予測(08 R-05)の『伸びの見込み』は、03 の式をそのまま使うので、判明前の素養の効果が数字に出る(CLB の伸びが同じ練習の仲間より 15% 大きい、など)。**許容する**(判明は 12 週で来る。予測を『正確』に保つ方を優先。L-43)。

8.2 画面(08 R-17。08a SC-16 / SC-18 / SC-19 / SC-20 / SC-28 / SC-29 / SC-11)

画面	追加する表示
部員一覧・オーダー・エントリー(`RunnerRow`)	名前の列の2行目に**個性チップ**(普通でない軸だけ。例『ロ・単・逃』。行の幅は変えない)
選手詳細(SC-19)	ヘッダに個性(3つのチップ・長押しで説明)と、判明した素養(『登坂の素養』)。『この月の予測』の下に、**このテーマとの相性**(R-17)
月間計画(SC-16)	グループの行に『相性 ◎ n人 / ○ n人 / △ n人』(選んだテーマごと)。月間計画の『伸びの見込み』は従来どおり
部員一覧のグループ(SC-18)	選手にチップ。素養が判明した選手には『登坂 / 下りの素養は、山対策のグループ(C)で伸ばせます』の案内(月末レポートから『Cへ移す』で移動できる)
個別指導(SC-20)	選んだメニューの相性の印と、単独走◎の効率(『個別指導の週は効率が高い』)
候補(SC-28・SC-29)	視察後に個性チップ。練習会後に『素養あり』(Lv3 以上のスカウト担当なら種類)
月末レポート(SC-11)	『お知らせ』に**『素養が判明した』**(◯◯は登坂の素養がある)

- チップの言葉と意味は、ヘルプ(SC-39)の用語に置く(『型』『走法』『展開』『素養』『相性』)。
- チップは**色だけに頼らない**(08 §4.2): 型は文字(スピード / ロング)、走法は『集団』『単独』、展開は『追』『逃』。

9. 検算結果

9.1 平均して中立か(§5.5 (a)・§6)

- 練習: 個性 + 素養で **+0.09〜+0.34 P**(チームの適性値の換算。40 スクワッドで +0.09〜+0.22・標準誤差 0.03〜0.04、30 スクワッドで +0.09〜+0.34)。個性のみ +0.0〜+0.16、素養のみ +0.07。
- レース: 全員に個性を付けても優勝率の変化は ±0.016 以内(標準誤差の範囲)。

9.2 多様性(§5.5 (c))

個性なし 1.98 → 個性あり 2.39(+21%)→ 型でグループ分け 2.89(+46%)。

9.3 長期バランス(`ref_model_10.py world`。中堅・おまかせ・40 シード × 45 年度・同じシード)

自校の全員の力 P に上積みを足して、05 の世界シミュレーションの結果の変化を測った(`PLAYER_RACE_BONUS`)。

上積み(P)	優勝率	三冠(年)	クリア年度の平均
+0.00	0.507	0.205	12.2
+0.25	0.522(+0.015)	0.220(+0.015)	12.2(±0.0)
+0.50	0.542(+0.035)	0.240(+0.036)	11.9(−0.3)
+1.00	0.597(+0.090)	0.311(+0.106)	11.1(−1.1)

- **+0.5 P ≈ 優勝率 +0.035**。個性・素養の平均の効きは +0.1〜+0.3 P(優勝率 +0.02 以下)で、05 の許容帯(中堅の優勝率 0.535 ± 0.051・クリア年度 12.6 ± 1.9)の**内側**。
- **利用の上限**: 個性を使って上積みできるのは、同じ最良の方針で揃えたチームに対して +0.2〜+0.4 P(選手ごとの最適 `o - b`。§5.5 (b))+ 単独走◎の指導 +0.5 P 程度(§5.2)で、合わせて **+0.2〜+0.9 P = 優勝率 +0.015〜+0.06**。+0.5 P 以下は帯(±0.051)の内側。全員の最適を知っている**オラクル**でも、全員バランスとの差 +0.7〜+1.2 P のうち大半は年間方針ひとつで取れる分(個性と無関係)。+1.0 P の上積み(優勝率 +0.090)は帯を超えるが、個性だけでは届かない水準。

9.4 判断の価値の上限(§3.2 の裏づけ)

- 『全員同じ最良の方針』から『選手ごとの最適』への上積みは +0.2〜+0.4 P(個性の有無でほぼ同じ)。登録 16 人から最適に選ぶので、一部の選手の育て方が悪くても別の選手を使える。
- したがって、**個性は判断の価値を劇的には上げない**(§1 の正直な結論)。判断の**数と見える手がかり**(チップ・相性・素養の判明)と、**特殊能力への道**(§7)が個性の価値。

9.5 受け入れ(統計の帯)

05 の世界シミュレーションは個性・素養を持たない抽象のまま(自校の育成は 03 の期待値で表す)なので、**統計の受け入れの帯(09 §16.3。中堅: 優勝率 0.535 ± 0.051・三冠 0.240 ± 0.056・クリア年度 12.6 ± 1.9)は変わらない**。個性の効きは『上積み』に換算して、平均(+0.1〜+0.3 P)は帯の内側、オラクルの上限(+1.0 P)だけが帯を超えることを確かめた(§9.3)。実装後は、個性を付けた自校で 05 と同じ測定をやり直す(L-39)。

10. 計算例(テストベクタ。`verify/ref_model_10.py vectors`)

TV-1 生成(`master_seed = 20260101`、`look_seed = 1〜6`。PCG32。§4)

look_seed	入学時の型	型	走法	展開	素養	能力(SPD STA PAC KCK CLB DWN CON MEN REC DUR)	素質
1	Mountain	バランス	普通	追い上げ	なし	43 44 48 37 54 42 40 42 53 42	65 69 68 56 78 65 63 61 77 64

look_seed	入学時の型	型	走法	展開	素養	能力(SPD STA PAC KCK CLB DWN CON MEN REC DUR)	素質
2	Mountain	ロング	普通	普通	なし	42 54 45 47 58 53 48 51 43 50	56 69 61 64 72 74 60 58 54 59
3	Endurance	ロング	単独	普通	なし	38 58 53 44 51 46 50 44 52 48	58 77 75 63 72 64 76 72 72 73
4	Balanced	バランス	単独	追い上げ	なし	44 46 46 49 44 43 46 39 48 48	79 79 86 81 74 77 79 75 79 82
5	Mountain	ロング	単独	逃げ	なし	43 47 45 46 53 51 46 49 50 49	66 60 64 59 72 70 62 60 63 67
6	Balanced	スピード	単独	逃げ	**登坂**	55 57 56 56 57 54 55 48 51 54	82 88 86 81 85 80 89 77 74 84

TV-2 成長の乗数(§5.1。能力の順 SPD STA PAC KCK CLB DWN CON MEN REC DUR)

個性・素養	gm	bond_k	ind_mult
スピード型・集団走◎・追い上げ型	1.20 0.80 0.80 1.20 1.00 1.00 1.00 1.00 1.00 1.00	0.30	×1.0
ロング型・単独走◎・逃げ型 + 登坂	0.80 1.20 1.20 0.80 **1.15** 1.00 1.00 1.00 1.00 1.00	0.05	×1.25
バランス型・普通・普通 + エース	1.05 1.05 1.05 1.05 1.05 1.05 **1.15** **1.15** 1.05 1.05	0.15	×1.0

TV-3 1 週の解決(§5。03 §6。山対策テーマの第1週・施設 Lv2・`bond_avg` 500)

入学時 `stats = [50,55,52,48,45,44,50,48,46,50]`、`potential = [75,80,77,73,70,69,75,73,71,75]`、成長タイプ Normal。

選手	E	XP の追加(能力の順。xu)
個性なし	1.2535	0 72 21 0 210 212 0 0 0 0
スピード型・集団走◎・追い上げ型	1.3409	0 61 18 0 225 227 0 0 0 0
ロング型・単独走◎・逃げ型(個別指導の週)	1.4939	0 102 29 0 251 253 0 0 0 0
登坂の素養(バランス型・普通・普通)	1.2535	0 72 21 0 **265** 212 0 0 0 0

TV-4 相性の印(§5.4。テーマごとの平均の乗数と印)

個性・素養	基礎ビルド	スピード強化	山対策	合宿	回復
スピード型・追い上げ型	0.844 △	1.036 ○	0.933 △	0.877 △	0.930 △
ロング型・逃げ型	1.156 ◎	0.964 △	1.067 ◎	1.123 ◎	1.070 ◎
バランス型・登坂の素養	1.000 ○	1.000 ○	1.033 ○	1.013 ○	1.000 ○

画面の相性(素養は判明するまで入れない): 登坂の素養・判明前 山対策 1.000 ○ / 判明後 1.033 ○。

TV-5 レースの補正(§6。04 §7.3)

状況	集団走◎	普通	単独走◎
集団の追走側(集団走)	−0.45%	−0.30%	−0.15%
単独走	+0.30%	+0.20%	+0.10%

状況	追い上げ型	普通	逃げ型
追う展開(前に 60 秒以内)	−0.20%	0.00%	+0.20%

TV-6 特殊能力の取得条件(§7。06 §9)

特殊能力	条件式(DSL)
skill.mountain_god	(r.gift.climber == 1 and r.stat.CLB ≥ 64 and ctr(r, 'hill_weeks') ≥ 24 and r.grade ≥ 2) or (r.stat.CLB ≥ 70 and ctr(r, 'hill_weeks') ≥ 32 and r.grade ≥ 3)
skill.downhill_master	(r.gift.descender == 1 and r.stat.DWN ≥ 62 and ctr(r, 'hill_weeks') ≥ 16) or (r.stat.DWN ≥ 68 and ctr(r, 'hill_weeks') ≥ 28)
skill.ace_section	r.grade ≥ 3 and ((r.gift.ace == 1 and r.p_rank ≤ 4 and r.races ≥ 2) or (r.p_rank ≤ 2 and r.section_wins ≥ 4))

11. 課題・未決・他文書への反映

11.1 課題トラッカー(00_spec_index.md §6)

ID	状態	内容	対応
L-17	一部解決(10 §7)	特殊能力の取得条件の閾値。山の神が誰にも取れなかった問題は、素養を道にして解決(素養あり + 山対策で 18%)。Gold(駅伝男)の水準は未検証	06 §16.4。実装後に本物のレースの結果で再調整
L-39	新規	個性・素養の**効果量は検算用のモデルで較正**した。チップ・相性の印が**伝わるか、練習の判断に使われるか**は実機・実プレイでしか分からない	M1 の画面の検証・M2 のプレイテストで確認し、効果量(±20% など)を調整
L-40	新規	練習の**判断の価値は構造的に小さい**(個別の最適化で、同じ最良の方針で揃えたチームより +0.2〜+0.4 P)。判断を増やしても、価値は上がらない	判断の価値を上げるなら、チームの強さに効く別の軸(区間ごとの適性の見える化・合宿の種類の増加)を将来検討
L-41	新規	他校の選手は個性・素養を持たない(世界の更新は抽象)。他校の選手の個性は画面に出ない	個性は中立なので勝率には影響しない。他校の特徴の演出は将来
L-42	新規	『ごぼう抜き』の取得率が簡易シミュレーションで測れない(追い抜きの回数のカウンターがほとんど増えない)	L-18 と同根。本物のレース(04)の結果から数える実装で確認
L-43	新規	月末の予測の『伸びの見込み』は、判明前の素養の効果が数字に出る(隠しの情報が漏れる)	許容(判明は12週)。気になるなら、判明前は予測から素養を外す(予測が正確でなくなる)

11.2 未決の解決(00_spec_index.md §5)

ID	内容	既定値(決まらなくても進める値)
U-19	素養の出現率(20%)・判明の速さ(12 週)・効果量(×1.15・+6)	本書の値。M2 のプレイテストで『発見の楽しみ』と『育成の差』を確かめ、調整する

11.3 反映した文書

文書	変更
02	v1.1: Runner.style / gifts / gift_revealed、OBRecord.style / gifts、列挙 StyleForm StyleRun StyleFlow Gift、不変条件 I-21、派生値(gm bond_k ind_mult)の説明
03	§4.7 に個性・素養の乗数、§6 手順1・2 の式に bond_k ind_mult gm(個性なしは従来の値)
04	§7.3 に個性の補正(走法の倍率・追う展開・先頭走のブレ)
05	§6.2 に個性・素養の生成、§7.2 に見え方(個性は視察、素養は練習会とスカウト担当 Lv3)
06	§3.2 に DSL の変数、§9 の取得条件(素養・個性を使う形)、§16 の再検算
07	§6.5 に招聘選手の個性・素養の生成(通常の入学生と同じ関数・同じ分布)
08	R-17(個性と相性の表示)、RunnerRow の名前の列に個性チップ、SC-11 の『素養が判明した』、SC-16 / 18 / 19 / 20 / 28 / 29、情報 I-72〜I-74
09	I-21(修復 R)、乱数の登録表(label 8・b = 40〜49)、サイズ(100 年度で +3 KB)、TV-6・TV-7 の数

文書	変更
01	§4 に個性・素養の要約
00	10 を v0.1 に、L-17 の更新、L-39〜L-43・U-19 を追加、用語、参照モデルの一覧

11.4 次への引き継ぎ

引き継ぎ先	内容
M0(Domain)	`Style`・`Gift` の列挙と `gm` `bond_k` `ind_mult` の導出(純関数)、生成(§4。鍵つきの乱数)、相性の印(§5.4)。TV-1〜TV-6 をそのまま自動テストにする。`Constants` は `K` 辞書から作る(L-37)
M1(画面)	チップ・相性の印・『素養が判明した』の通知の見え方と、操作量(時間予算)を確認する(L-39)
M2(プレイテスト)	個性の体感(『育てた差』が分かるか)・素養の出現率と判明の速さ(U-19)・山の神の頻度(約 6 年に 1 人)を確認する

付録A 用語集

日本語	識別子(コード)	意味
年度	`year`	4月始まり。1年度 = 48週
週 / 月	`week` / `month_index`	1ヶ月 = 4週。`month_index` は 0=4月 〜 11=3月
月間計画	`MonthlyPlan`	グループごとに選ぶ月のテーマ(4週分のメニューを自動生成)
月のテーマ	`Theme`	基礎ビルド、スピード強化 など
ポイント練習	`KeySession`	週に2枠。質の高い練習
走行量	`Mileage`	週間距離の段階(少/標準/多/極)
疲労	`fatigue`	0〜100
調子	`form`	絶不調(−2)〜絶好調(+2)
やる気	`motivation`	1〜5
素質	`potential`	能力ごとの上限(隠し)
経験値	`xp`	能力アップに必要な蓄積
区間適性値	`P`	区間タイプ別に能力を加重平均した値
校格	`tier`	強豪 / 中堅 / 下位
名声	`prestige`	0〜1000
年度ランキング	`year_rank`	前年度の評価ポイントによる順位。出場権に使う
出身県	`prefecture_id`	1〜47
地方	`region`	北海道・東北・関東・中部・近畿・中国・四国・九州の8地方
距離帯	`distance_band`	0=地元 / 1=隣接 / 2=遠方 / 3=海外
県別OBネットワーク	`ob_network`	県ごとのOB数から決まるレベル(0〜5)
留学生	`origin_type = Overseas`	出身国を持つ選手
特別招聘券	`Ticket`	使うと翌年度に留学生またはスーパールーキーが確定入学する券
招聘選手	`recruit_source = Ticket`	特別招聘券で入学した選手
イベント	`EventDef`	条件式・選択肢・効果で定義するデータ(06)。1年度に約13件
覚醒	`ev.awk.*`	取得条件を満たした週に特殊能力を覚えるイベント(06 §9)
補正	`TimedModifier`	期間付きの効率・回復・故障率などの補正(06 §6)
追加コンテンツ	`ContentPack`	見た目・シナリオ・黄金世代プランの永続解放(07 §9。価格は未決)
繰り延べ	`bank`	所持上限(2枚)を超えて付与された券を、失効させず貯めておく枠。空きができたら取り出す(07 §3.2)
購入状態	`EntitlementState`	追加コンテンツを買っているかの判定。True / False / 不明(30日の猶予。07 §8)
古豪復活	`sc.koki`	追加シナリオ。弱小校に名声と施設を上乗せして始める(中堅と同程度の難しさ。07 §10)
失速	`stall`	区間の終盤でスタミナが足りず落ちること。確率はペース配分(04 §7.4)
余裕の目安	`gauge` (◎〇△)	走者ごと・区間ごとの失速の起きにくさ。指示を決める材料
指示	`Instruction`	攻める / 維持 / 抑える
当日変更	`swap`	当日、調子と環境が分かった後に先発と控えを入れ替える
おまかせ	—	ダイジェスト・結果のみのときの自動の采配(指示は維持、不調以下の置き換え、檄の自動使用)
実績	`Achievement`	アカウント単位の達成(Steam 実績ほか)。セーブとは別に保存
無招聘	`no_invite`	そのセーブで特別招聘券を一度も使っていないこと。実績の条件に使う
節目(停止)	`Stop`	進行が止まって入力を求める場所(月間計画・エントリー・オーダー・当日・イベントなど。08 §6)

日本語	識別子(コード)	意味
続ける	`Continue`	次の節目まで進める唯一の前進の操作(C-FLOW-01)
推奨プラン	`auto_plan`	重点ごとの12か月のテンプレートから、その月のテーマを返す。標準プラン(03 §11)が基準(08 R-04)
重点	`Focus`	年間方針の選択。バランス / スピード寄り / ロング寄り
月末の予測	`forecast`	乱数を除いた週の解決で、疲労・能力・故障の確率・調子の分布を先に見せる(08 R-05)
個別指導	`guidance`	指導ポイント(月4)で、1人のメニューを今週から月末まで指定する / 面談(やる気 +1)(08 R-06)
競合度	`rivalry`	他校の狙いの多さ。市場評価 V の帯で ほぼなし / 少 / 中 / 多(08 R-08)
前評判	`preview`	出場校の登録メンバーの区間適性(Long)の平均の順位。◎○▲△(08 R-09)
ランク(G〜S)	`rank`	能力の10刻みの文字。S ≥90 〜 G <30(08 R-01)
u	—	基準キャンバスの長さの単位。高さ360 u。1 u = 高さ360dpの機種での 1dp(08 §2)
コンテナ(EKSV)	`K_MAGIC`	セーブのファイル形式。ヘッダ(JSON)+ 本体(gzip の JSON)+ CRC32(09 §4)
世代	`a / b`	自動セーブの2世代。交互に書く(09 §4.4)
保存番号	`save_counter`	スロットの単調増加の番号。世代の選択と同期の判定に使う
系譜	`lineage_id`	1つの歴史の識別子。新規ゲームごとに別(09 §9.1)
記録(同期の単位)	`record`	`slot{n}` · `slot{n}m` · `profile` (09 §9.1)
個性	`Runner.style`	全員が持つ3軸(型・走法・展開)。練習の伸び方・絆の効き・レースの味付けに影響。コードは3文字(`BFE` = 中立。10 §2)
型	`StyleForm`	スピード型 / バランス型 / ロング型。得意の能力(SPD / STA)が +20%、苦手が −20%(10 §5.1)
走法	`StyleRun`	集団走◎ / 普通 / 単独走◎。絆の効き(0.30 / 0.15 / 0.05)と個別指導の効率(単独走◎ ×1.25)(10 §5.2)
展開	`StyleFlow`	追い上げ型 / 普通 / 逃げ型。KCK・PAC の伸び方と、追う展開・先頭走の補正(10 §5.1, §6)
素養	`Runner.gifts`	まれ(20%)で隠しの才能。登坂 / 下り / エース。成長 ×1.15・素質の上限 +6・特殊能力への道。入部から 12 週で判明(10 §2.2, §8)
相性の印	`fit_mark`	選手の個性とテーマの練習の配分から出す ◎○△。『この子にはこのテーマが合う』(10 §5.4, 08 R-17)
墓石	`tombstone`	スロットを削除したことを表す同期の記録(09 §9.1)
退避	`backups/`	競合・移行で置き換えられたセーブの控え。消さない(09 §9.4)
端末の内部状態	`LocalState`	同期しない。同期の基準点・購入状態のキャッシュ・時計の高水位・実績の保留(09 §3.3)
Q6	`Q6`	保存する double の小数6桁への丸め(09 §5.3)
高水位	`clock_hw`	これまでに見た最大の時刻。時計の巻き戻しの検知(09 §10)
鍵つきストリーム	`derive + Pcg32`	`master_seed` と (label, a, b) から作る、状態を持たない乱数(09 §8)

付録B 仕様完了の基準と書式規約

B.1 仕様完了の基準(各文書がすべて満たすこと)

1. 扱うデータの**項目・型・範囲・必須/任意・不変条件**が定義されている。
2. 計算式は**手順の順序・端数処理・上下限(クランプ)・境界条件**まで書かれている。
3. 主要な規則に**計算例(テストベクタ)**が付いている。入力と期待値が数値で書かれ、そのままテストに写せる。
4. 数値は**初期値**として明記され、調整可能な値は「マスタ」か「定数表」に集約されている。
5. 画面・イベント・商品などの**一覧が漏れなく**ある(「など」で終わらない)。
6. 未決事項は**既定値**が付いており、決まらなくても実装が止まらない。

B.2 書式規約

ルールの書き方(03以降の各ルールはこの順で書く)

項目	内容
入力	参照する状態・定数
前提	実行条件。満たさない場合の扱い
手順	番号付きの手順。順序が結果を変えるものは順序を固定する
出力	更新する状態と範囲
境界・例外	0、上限、空、同値などの扱い
計算例	入力値と期待値

数値の記述: 初期値は「初期値案」と書く。式中の定数には `K_` 接頭辞の名前を付け、定数表で一覧化する。

付録C 未決トラッカー(既定値付き)

ID	内容	決める時期	既定値(決まらなくても進める値)
U-01	追加コンテンツ(見た目・シナリオ)の商品範囲	**決定済み(07 §9)**	v1 は『黄金世代プラン』+ 見た目3パック(ユニフォーム・ヘア・応援歌/BGM)+ シナリオ。価格は U-05 と一緒に決める
U-02	留学生の登録・出走の上限(実大会の規定を確認)	04作成時	全大会 登録2 / 出走1
U-03	美術の調達方法	M1の後	プレースホルダー
U-04	言語	**決定済み(08 §13.2)**	v1 は日本語のみ(文字列はテーブル化。かな表示の設定あり)
U-05	本体の価格帯	M3	—
U-06	方言機能の採否	06作成時	採用しない(拡張)
U-07	大会名・大学名などの実名化	商用化前	仮称
U-08	留学生を本体に含めるか	決定済み(05 §6.1)	本体にも留学生は存在する(候補の3%。他校にも在籍)。特別招聘券は「確定入学」の手段を足す
U-09	特別招聘券の課金方式	決定済み	**永続解放(DLC・非消費型)**。開発中はプランなしで進め、解放は後から有効にする
U-10	券のゲーム内獲得条件	**決定済み(07 §4)**	14 の達成(各大会の初優勝3・初の3位以内・初シード・初の三冠・監督ランク3/6/9・10年度ごとの節目 ほか)に各1枚。使えるのは翌年度から
U-11	留学生=即戦力型 / スーパールーキー=育成型の役割分担	05作成時	`02_data_spec.md` §3.9 の生成帯(国による補正ではなく役割分担。描写に配慮)
U-12	券の配布周期・1回の枚数、所持上限を超える付与の扱い	**決定済み(07 §3.2, §5)**	黄金世代プランは 4年度に2枚。**失効なし**(上限を超えた分は繰り延べて貯め、空きができたら取り出す)。所持上限2・同時進行2は据え置き。購入を確認した時点で最初の2枚を付与する
U-13	実績の一覧と条件の確定	06〜08作成時	01 §3.3 の初期案。『弱小・無招聘クリア』『同好会・無招聘クリア』を含む。ストアごとの実績IDは 09
U-14	実績の取り扱い(未接続時の保留、解除の取り消しなし)	**決定済み(09 §11)**	本体で判定・記録(`ProfileData` が正)し、ストアへは通知(ミラー)だけを送る。未接続なら `pending_unlocks` に貯めて後で送る。冪等
U-15	縦持ちへの対応	決定済み(08 §13.2)	将来(v1 は横画面のみ)
U-16	セーブスロットの数	**確定(09 §3・§4)**	3 スロット。各スロットに自動2世代(A / B)+ 手動1
U-17	Unity の版と UI Toolkit の採否	M1	Unity 6 LTS・UI Toolkit。M1 で 09 §15.5 の基準(日本語・性能・パッド・基準キャンバス)を確かめ、満たせなければ uGUI
U-18	スマホのクラウド同期の実装方式	M5	各プラットフォームの保存ゲーム(GameKit・Play Games)。自前のサーバーは持たない(01 §6.5 の原則)
U-19	素養の出現率(20%)・判明の速さ(12週)・個性と素養の効果量	M2	10 の値(出現率 20%・判明 12 週・型/展開 ±20%・素養 ×1.15 / +6)。プレイテストで『発見の楽しみ』と『育てた差』を確かめて調整(10 §11.2)

付録D 課題トラッカーと拡張メモ

D.1 長期バランスの課題トラッカー

「何年も続けると徐々に強くなり、頭打ちになる」を満たすための課題。05 で L-01〜L-03 を解決し、L-04 を一部解決した。04 で L-07・L-08・L-13 を解決した(本番のレース式で再検算、v0.2 で采配を強化)。詳細は `05_world_scout_spec.md` §14〜15。

ID	状態	課題	対応・方針
L-01	**解決(05)**	施設・スタッフの効果が小さい(Lv1→5で卒業時の力が +1.2)	効果係数を引き上げた(03 §4.6)。Lv1→5で約 +3.7
L-02	**解決(05)**	自校の戦力曲線と頭打ちが未設計	名声・資金・施設・スカウト・他校を設計し、45年度の推移を検算した
L-03	**解決(05)**	頭打ちの水準が高すぎると王朝が自明になる	奨学金枠・包囲・他校の指導体制で制御。頭打ち後の優勝率 0.51(中堅開始『おまかせ』、04 v0.2 の本番式 + 07 の券で再検算)
L-04	一部解決	下振れ要因が少ない	他校の指導体制の揺らぎ・包囲を追加。自校のスタッフ引退・名声の目減りは未実装
L-05	一部解決	施設・スタッフが上限になった後の資金の使い道がない	『海外遠征合宿』(800万円)を06に追加した(後半で年に約 −520万円。効果は小さい)。大きな使い道(記念事業・施設の新設)は未設計
L-06	**解決(方針)**	有料プランの影響が弱小で大きい(中堅は ほぼなし、弱小 −19%。07 §12.1)	効果の大きさは公平性の制約にしない(ランキングなし・課金層は『クリア』が目的)。券を使わないことを条件にした実績(『弱小・無招聘クリア』など。01 §3.3)で分ける。券の上限は残す
L-07	**解決(04)**	検算は「管理の巧拙 +2」を仮定していた	本番式で測ると、登録・配置と型に合わせた育成の合計は +0.2〜+1.1、檄は +0.7(04 §14.3)。加点を外して再検算した
L-08	**解決(04)**	レースは簡易版(全区間で力P)で検算していた	05 の世界シミュレーションに 04 の本番式を組み込んで再検算した(05 §14)
L-09	**解決(07)**	同好会の「部員5人」開始は未検証	通常の名簿(戦力順位33位)と差がないことを検算した(07 §12.4)。部員5人のまま始めてよい
L-10	新規	区間タイム式は実大会の区間記録・分布との照合が未了(個人の区間記録は整合するが、ブレーキ・失速の頻度と大きさは仮置き)	実データで照合し、ブレーキ・失速の頻度と大きさを調整
L-11	新規	檄の先行使用・環境の予報・留学生の登録/出走上限は規則のみで未検算(影響は小さい想定)。当日変更と指示は 04 v0.2 で検算した	実装時にテストを足す
L-12	**解決(09 §8.4)**	他端末で同じレース結果を再現するには、`log`・`cos` を使う計算を表引きや固定小数点にする必要がある(M-02)	実測: `normal()` の端末差は1年度で約 10^{-11}、`pace50` の `log` は約 2×10^{-9}。`pace50` をマスタの表にして 0 にする。端末間のビット一致は求めない。M-02 は両チームを同じ端末で走らせればよい
L-13	**解決(04 v0.2)**	本番の采配が長期の結果に効きにくかった	ペース配分(失速)を追加して指示を『余裕の目安』を見る判断にし(全区間攻めるのは得にならない)、当日変更を全大会に拡大し、環境の個人差を2倍にした。おまかせ → 上級で優勝率 +0.07〜+0.14(04 §14.4)
L-14	**解決(効果を下げた)**	檄の価値が大きく、上限に達しにくいので判断の余地が小さい	損失の軽減を ×0.6 から ×0.8 に下げた(おまかせと檄なしの差 +0.06〜+0.10)。判断の余地の小ささは残る(04 §9.2)
L-15	新規	展開(順位・差)に応じた指示(先頭を守る・追う)は未検算	上積みの余地として残す(04)
L-16	新規	第三戦(箱根型)は指示の効果が小さい(+0.2力)。手応えは当日変更が中心	案: 第三戦の区間に特性(向かい風・起伏)を足す(04)

ID	状態	課題	対応・方針
L-17	一部解決(10 §7)	特殊能力の取得条件に素養・個性を入れた。『山の神』は素養(登坂)のある選手が山対策を続けると取れる(素養ありの約 18%)。『駅伝男』(Gold)の水準・『ごぼう抜き』の取得率は未検証	04 の本番式を入れて再調整(06 §9, §16.4)
L-18	新規	『他校に同じ出身高校の選手』・大会の事象(集団走・単独走・追い抜き)は、06の検算では確率で近似した	05 の世界と 04 の本番式から取る(06 §16.1)
L-19	新規	イベントの文言(約100本)・演出は未作成。名称テーブルのキーだけを定義した	実装時に作成(08・09)
L-20	新規	招聘選手の出身国(24か国)の名前表・国旗・描写の確認が未了。国による能力差は付けない(役割分担のみ)	実装前に名前表を作成し、描写をレビューする(07 §14)
L-21	新規	追加コンテンツの価格・ストア登録(商品ID・年齢区分・返金)が未決	U-05 と一緒に決める。商品ID一覧は 07 §9
L-22	一部解決(09)	各ストアの購入状態の取得 API(Steam / Apple / Google)の挙動(オフライン時・返金・ファミリー共有)が未検証	インターフェース・猶予・時計対策・通知は 09 §10・§11・§15.4 で確定。実機の挙動は M5(L-34)
L-23	**解決(08)**	年間の目標数 12〜16人が定員10を超え、狙っても約半分は枠で入れない(69%)	推奨 8人 + 確定の順番と枠のメーター(08 R-08。05 §14.7)
L-24	**解決(08)**	『型に合わせた育成』(04 `SPEC_SHIFT`)は v1 のグループ運用では再現できない	外しても優勝率 ±0.016 以内(08 §11.8)。v1 は持たない
L-25	新規	主務(`StaffRole.Manager`)に効果がない	v1 は表示しない。効果を足すか役職をなくすか決める(08 R-11)
L-26	新規	チーム全体を持久・山に振ると標準プランより劣る(素質の伸びしろが能力ごとにほぼ同じ)	v1 は3つの重点に絞った。将来、型ごとの伸びしろの偏りを検討(08 §11.2)
L-27	新規	記録会(`RaceKind.Meet`)は v1 で使わない(記録は能力から導出)	測定にするなら 03 に測定の週を足す(08 R-03)
L-28	**解決(08)**	`form_add` の上限の読み方が曖昧	peaking に足してから −8〜+12 に収めると確定(03 §6)
L-29	新規	05 §7.3 の `pwin` は較正が悪い(0.9〜0.99 で実測72%)	画面には使わない(競合度と枠で見せる)
L-30	新規	V の高い順の確定のため、原石(V<51.5)の目標の78%が定員で入れない	案: 本命の確約。既定: 変更しない(05 §8.2)
L-31	新規	実機での文字・タップ領域・パッド操作・日本語フォント・性能は未検証	M1 の UI Toolkit の技術検証に含める(08 §11.4)
L-32	新規	画面の文言・アイコン・演出は未作成(名称テーブルのキーのみ)	実装時に作成。M1 はプレースホルダー
L-33	新規	世界の更新・スカウト・生成の乱数ストリーム(ラベル 3・6・7・8)の引く順序は 05 の記述順で、Python の参照(`random`)とビット一致しない	統計で受け入れる(09 §16.3)
L-34	新規	ストアのクラウド(Steam Auto-Cloud・GameKit・Play Games)の競合の返し方・サイズ・オフライン時の挙動は未検証	`ICloudSave` の背後に閉じ込め、判定は純関数(09 §9.5)。M5 で実機検証
L-35	新規	`ProfileData.counters` のマージは最大を取るので、複数端末で別々に増えた分が合計されない(達成が遅れる側にずれる)	v1 は許容。必要なら端末ごとの加算に変える(09 §9.6)
L-36	新規	04 の表の総合タイム(四捨五入)と画面の総合タイム(切り捨て)が 1 秒ずれることがある	画面が正(09 §13)。04 の表は参考値
L-37	新規	定数の名前が、文書(`K_…`)と参照モデルの `K` 辞書のキーで一致しない(`inventory` が数を出す)	M0 の最初に `K` 辞書から `Constants` を作り、文書の定数表との対応表を付ける(09 §17.1)
L-38	新規	C#・IL2CPP・実機での性能・GC・メモリは未測定(Python の値を上限としている)	M0 の SimCli `bench`・M1 の実機(09 §14.2)

ID	状態	課題	対応・方針
L-39	新規	個性・素養の効果量は検算用モデルで較正した。チップ・相性の印が**伝わるか・練習の判断に使われるか**は実機・実プレイでしか分からない。平均の効き(+0.1〜+0.3 P)は、実装後に個性を付けた自校で 05 と同じ測定をやり直して確かめる	M1 の画面の検証・M2 のプレイテストで確認(10 §11)
L-40	新規	練習の**判断の価値は構造的に小さい**(個別の最適化で、同じ最良の方針で揃えたチームより +0.2〜+0.4 P。登録 16 人から最適に選ぶため)。個性は判断の数と手がかりを増やすが、価値は上げない	価値を上げるなら別の軸(区間適性の見える化・合宿の種類)を将来検討(10 §3.2, §9.4)
L-41	新規	他校の選手は個性・素養を持たない(世界の更新は抽象)。他校の特徴は画面に出ない	個性は中立なので勝率に影響しない。他校の特徴の演出は将来(10 §3.3)
L-42	新規	『ごぼう抜き』の取得率が簡易シミュレーションで測れない(追い抜きの回数がほとんど増えない)	L-18 と同根。本物のレース(04)の結果から数える実装で確認(10 §7)
L-43	新規	月末の予測の『伸びの見込み』は、判明前の素養の効果が数字に出る(隠しの情報が漏れる)	許容(判明は 12 週)。気になるなら判明前は予測から外す(予測が正確でなくなる。10 §8.1)
L-44	**解決(M0 段階 10)**	05 の参照モデルは、出身を地方単位で近似し、『強豪県は能力の基準値 +3』(02 §6)を実装していなかった。M0 の C# は県単位で実装し、プール全体の A0 の平均が変わらないよう基準を下げて足した。+3 ありの自校ありの世界は、参照の帯より戦力 P が +0.2〜+0.4 高い(Elite 72.9 / Mid 72.4 / Low 70.9。Elite だけ素の帯 72.4 ± 0.4 の外)	**仕様の +3 を残す**。T4 を『等価性(+3 を外した世界 = 参照と同じ構成が 4 指標とも帯の内側)』と『仕様の世界(+3 あり。優勝率・三冠率・クリア年度は帯の内側、P は同じシードの対との差が −0.3〜+1.0)』の 2 本立てにした(09 §16.3・05 §14.10)
L-45	**解決(M0 段階 10)**	02 §4.3 は `prestige` を int、`fame_hist` の範囲を 0〜40 と書いたが、世界の更新は名声を連続値で動かし、履歴は毎年 3 大会とも優勝を続けると 67.5 まで上がる(統合の 300 年度で 59 を観測。0〜40 はシナリオの上書きの入力の範囲)	`prestige` も double(`Q6`)で保存し表示で四捨五入、`fame_hist` の範囲を 0〜67.5 にした(02 §4.3・09 §5.3・D-10)。保存する double は代入のときに `Q6` を通す(自校の抽象の `funds`・記録も)
L-46	新規	ホットパスの GC『0』(09 §14.2)は M0 では未達。実測: 週の解決 488 B/回・第一戦 1 レース 43 KB・世界の 1 年度 2.4 MB(`WeekReport`・`Pcg32`・リストを毎回作る)	M0 は実測を基準値にして退行を止める(`bench`)。0 に近づける作業(`struct` 化・プール)は M1 の実機プロファイル(IL2CPP)のあと(09 §14.4)
L-47	**解決(M0 段階 10)**	中断セーブは同期で 200 ms 以内(09 §14.2)だが、100 年度の合成セーブの通常の圧縮(gzip level 6)は開発機で 73〜114 ms で、下位のスマホ(5 倍遅いと見る)では超える	中断セーブだけ**最速の圧縮**(zlib level 1)にした。31 ms(+4 倍の余裕)・387 KB(通常 288 KB)。形式は同じで通常の読み込みで読める(09 §4.6)
L-48	**解決(M1 準備)**	Runner の保存の形が文書で食い違っていた: 02 §4.1 は出身を平坦な項目(`origin_type` `prefecture_id` …)、検証器・`mini_save` は入れ子の `origin`、`build_save` は平坦	**入れ子で確定**(検証器 I-02 が読む形)。保存の正本を 09 §5.6 にした(鍵・型・順・既定値)。02 §4 に案内を足した
L-49	**一部解決(M1 準備)**	自校の保存は、世界の側(`schools` `rival_runners` `candidates` `history` `high_schools`)と**まだ合成していない**。自校が暫定で持つ写し: `prestige` `year_rank` `last_year_rank` `year_points`(本来は `schools[自校]` と `history` から導く)・`school_prefecture_id`(`schools[自校].prefecture_id`)	世界と `Runner` の自校は `GameSession` / `GameSnapshot` で**合成した**(09 §5.6・§17.7)。残り: 写し(`prestige` `year_rank` `last_year_rank` `year_points`)をやめて世界の側を正にする(M3。いまは世界の `School` が正で、自校が週の終わり・年度末に写している)
L-50	**解決(M1 準備)**	取り出しの時点で期限が切れた補正・OB の役割は、次の週の `ExpireTimed` が消すまで状態に残る(I-19 に反する)	取り出しでは**書かない**(`ExpireTimed` と同じ結果)。復元した状態は、live の状態が次の週の最初にたどり着く状態と同じ

ID	状態	課題	対応・方針
L-51	**解決(M1 準備)**	復元は派生値(力の順位・レギュラー・グループ内の絆の平均・素質との差)を作らない	**復元した側が `Refresh` を呼ぶ**(`GameSession.Restore` は呼ぶ)。`GameSession` の週の境目は『次の週の最初の処理(`BeginWeek`)まで済んだ状態』なので、約束は『復元で `Refresh` 』に変わった(09 §5.6)
L-52	新規	高校の強さの表(05 §6.2・07 §6.2 の『strength に比例して県内の高校を選ぶ』)の値が文書にない。C# は高校の ID の割り当て(`Geo.HighSchoolId`)を県内の高校数で均等に抽選している	同じ高校の絆(+100)・`rival_same_hs` は ID の一致で足りる。強さの差が要る機能(高校ごとの推薦枠など)を作るときに表を定義する
L-53	**解決(③④ 済・⑤のみ近似)**	実体の自校の大会で、仕様にあって M0 で未実装のもの: ①**済**: 予選会に出た 12 名の疲労(20km × 1.6 = 32。04 §3.2 の 6 で仕様の穴を埋めた)②**済**: 予選会へのピーキング(`peaking_mode.Qualifier` = `Short`。第 26 週に `TAPER[4]`。04 §3.2 の 7・03 §5.3。`Meet`(記録会)は大会がないので無関係)③区間記録・`race.record`(殿堂入りの条件のうち記録更新)**(解決: 世界の履歴 `WorldSim.History` + `Runner.Records`。02 §4.8・09 §17.7)**④留学生の出走枠 `IntlRun`(当日の出走は 1 人まで)は**自校の最適配置・オーダー・当日変更に適用した**(M1 準備。他校の抽象の選手は留学生でない)⑤`rival_same_hs` は近似(`ApproxEventWorld`: 確率 0.40)のまま	①②の T4c(3 開始校 × 120 シード × 45 年度)は帯の内側のまま: 強豪 0.588 / 0.257 / 8.97 年・中堅 0.534 / 0.216 / 13.74 年・弱小 0.456 / 0.147 / 23.48 年(前: 0.588 / 0.256 / 8.97・0.533 / 0.211 / 13.79・0.456 / 0.153 / 23.50。差はばらつきの内。ピーキングだけでは弱小 24.39 年・両方で 23.48 年。向きがそろわないので、差は 120 シードのばらつきの内)。強豪はほぼ毎年シードで影響がほぼない。試験 `QualifierTrainingTests`(調整の対象・シード校・疲労・保存)と変異 4 本。⑤は世界の他校の先発の出身高校を引く(未実装のまま)
L-54	**解決(M1 準備)**	選手の名前・かなの生成(look_seed → 姓・名。09 §8.2 label 8 の b = 0〜3)は名前の表(マスタ)が要るので M0 では未実装。`FamilyName` などは空文字のまま(OB の `name` も空)	名前の表(`Assets/Master/names_ja.json`。姓 150・名 110)・`NameGenerator`(引き方は 09 §8.2)・重複を避ける再抽選(b = 30〜)を実装し、新規ゲームの名簿・入学者・招聘選手のすべてに付けた。留学生は国別の表(L-20)ができるまで、架空の音の部品の仮の名前
L-55	**解決(原因を特定。受け入れ)**	実体の自校は抽象より少し弱い(優勝率 −0.04、クリア +1.5〜1.8 年度。05 §14.11)。原因の候補: 故障・疲労・調子のばらつき / 区間配置が実際の 10 能力で決まる / 型に合わせた育成を自動で行わない(L-24)。また資金は 18〜20 年度から積み上がり(L-05)	**原因 = 第二戦の調子**(`SimCli diag-real`。05 §14.11 の表): 第二戦は第一戦の 3 週後で調整が短く(03 §5.3 の重なり = `TAPER[2..4]`)、出走メンバーの調子が他校の想定 +1.4 に対し +0.9(時間にして約 0.4%・力にして約 1.7 の不利)。疲労・力・故障は他校と差がなく、第一戦・第三戦は当日変更の効果で他校の想定より調子が高い(+1.9)。**確かめ**: 第二戦の調子だけを他校と同じ分布で引くと実体は抽象を上回り(強豪 0.630 / 6.6 年・中堅 0.583 / 11.5 年・弱小 0.509 / 19.4 年)、弱小のクリア年度が帯の外(下端 22.0 未満)になる。**規則は変えない**(第二戦の調整の短さが、実体の自校の当日変更・実際の育成の強さを打ち消して帯の内側に収めている。T4c のクリア年度の余裕 1.5 年度 = この差を見込んだもの)。資金の使い道(L-05)は別件のまま
L-56	**解決(M1 準備)**	区間オーダー(SC-22)の編集・当日変更(SC-24)・指示と檄(SC-25)の操作は未実装だった	`RaceDay` と `GameSession` のオーダー・予報・前評判・予選会の登録を実装した(04 §14.5・08 §13.6)。何も変えない当日 = おまかせ、見込み = 実際の結果、通過した区間の決定は固定
L-57	**解決(M1 準備)**	月末レポート(SC-11)の項目の作り方が暫定だった(`rank_up` `form_up` は未、`big_growth` は『レベルアップ 2 回以上』)	R-10 どおりに作る: ランクアップ(能力のランク文字が上がった)・大きな成長(月に能力が合計 +3 以上)・調子の上昇・覚醒(金 > 青 > 赤)・故障と復帰(score = 力 P)。月の初めの能力・調子・疲労・士気を `GameFlow` が覚える。`gift_reveal` は素養の判明(L-64)で追加。08b

ID	状態	課題	対応・方針
L-58	**解決(揃えない)**	スカウトの『おまかせ』が 2 つある: `scout_auto` (年度末に検算済みの方針 `ChooseTargetsFor` を使う。約 13 人を狙う。T4c の元)と、画面のボタン (`AutoScoutObserve` `AutoScoutRecruit`。R-08 の目標 8 人)。後者の長期の結果(R-08 の『入学者の Q が 0.8〜0.9 低く、P は −0.4〜+2.3』)は C# では未検証だった	**C# の長期で測った**(`SimCli accept all 120 45 --real --scout-button`。進行制御を通し、月末レポートの停止ごとにボタンを押す = 視察は 6〜11 月・勧誘は 12〜2 月。比べる基準は `--flow-only` = 進行制御を通すだけで `scout_auto`。基準は通常の T4c とバイト一致): ボタン = 強豪 0.590 / 0.257 / 8.33 年(P 72.9)・中堅 0.533 / 0.215 / 12.86 年(72.5)・弱小 0.452 / 0.148 / 22.98 年(71.9)、`scout_auto` = 0.588 / 0.257 / 8.97 年・0.534 / 0.216 / 13.74 年・0.456 / 0.147 / 23.48 年 (P 72.9 / 72.5 / 72.0)。**ボタンは `scout_auto` と同等(優勝率・三冠率・P は差が誤差、クリアは 0.5〜0.9 年度早い)で、T4c の帯の内側。揃えない**: `scout_auto` (約 13 人・全員の見込みを知る)は『プレイヤーが操作しない』場合の検算用の方針として据え置く。人がボタンで十分に詰められる(R-08 の方針で長期が下がらない)ことが C# でも確かめられた。試験 `ScoutPolicyTests`
L-59	**解決(C# を仕様に合わせた)**	O3(04 §5.2)・E3(04 §4.1): 軽傷は配置・出走できる(調子は −2 固定。03 §6)/ 登録できないのは重傷だけ(中傷は登録できるが、大会までに治らなければ出られず差し替えになる)。M0 の C# は故障中を一律に外していた	`TeamRacing.CanRegister` (重傷でない)・`CanPlace` (中傷未満)・`CanRun` (`CanPlace` かつイベントの欠場でない)に分け、登録 (`SetEntries` / `SetQualifierEntries`)・オーダー (`SetOrder`)・おまかせ登録・当日・各 View(`CanEnter` / `entry.injured` / `order.injured`)を揃えた。見込み (`FormOutlook`)は、大会の週にまだ故障中なら調子 −2 で確定・それまでに治るなら治ったあとの分布。出られる選手が出走人数ちょうどで 1 人が軽傷でも棄権にならない(試験)。T4c 再実行: Elite 0.576 / 0.246 / 9.08 年・Mid 0.524 / 0.207 / 13.88 年・Low 0.450 / 0.147 / 24.42 年(すべて帯の内側。差は 0.01 未満)
L-60	一部解決(① 残り)	檄: ①先行使用(区間の前に使うと時間 −0.25% × `f_k`。検算対象外。**未実装**: 反応的な使用と回数を取り合うので、UI で観戦が実際にどう見えるかを見てから決める)②走者ごとの効き `f_k = clamp(0.5 + 0.1×やる気 + 0.3×絆平均/1000, 0.3, 1.2)` は**実装した**(参照モデル `geki_f`・`RaceMath.GekiFactor`・`RaceRunner.GekiF`。自校の走者はやる気と絆、他校の抽象は 1.0。チーム一律の補正 `RaceTeam.GekiF` は試験用に残し掛け合わせる。ゴールデンに式の検算と走者ごとの f_k つきのレース 450 件)③見どころ(ハイライト。§9.3)の判定は**実装した**(`RaceHighlights`。重要度の同順の決め方は §9.3 の末尾)	②で T4c をやり直した: Elite 0.588 / 0.256 / 8.97 年・Mid 0.533 / 0.211 / 13.79 年・Low 0.456 / 0.153 / 23.50 年(すべて帯の内側。②の前との差は 0.01 前後)。04 §14.4 の長期の結果(一律 1.0)は、平均の f_k が 1.0 に近いため据え置く
L-61	方針決定(当日の画面は作らない)	予選会(10 月第 3 週)は、登録 12 名の手動 (`SetQualifierEntries`)だけで、当日変更・指示・檄はない(04 §3.2 の予選会は 1 人 1 レースの順位)	画面は SC-26 の結果のみ(結果は `QualifierResult()`: 順位表・通過ラインとの差・自校の 12 名のタイム。実装済み)。当日の画面(当日変更・指示・檄)は**作らない**方針(1 人 1 レースで指示・檄の区間がなく、当日変更を入れると乱数の鍵(チーム × 選手番号)と TV-8 の検算に触れる)。遊んでみて単調なら再検討

ID	状態	課題	対応・方針
L-63	**解決(M1 準備)**	文字列を表(`Assets/Strings/ja.json`)へ移すときに見つけた整理の残り: ①**済**: イベントの選択肢の本文(`ChoiceDef.Label`)から数値を外した(32 件。06 §4.2 の `label` は数字を書かない)。効果の要約は `App/EffectText.Summary`(08 R-15。`efx.*`)が `EffectView` から作る。旧い本文は効果とずれていた(確率・隠れた効果の書き忘れ。例 `home.senior_reunion` は『やる気 +1』だが実際は 50%、`team.camp_abroad` は XP・補正が未記載) ② `EventDef.Hint` は設計メモ(例『資金の使い道(L-05)』)で画面の本文ではない。表には入れていない ③**済**: Domain の `OpResult` の理由・警告は日本語の直書きをやめ、文字列テーブルのキーと差し込みの値(`Msg`。`Domain/Game/Msg.cs`)にした。定義は `OpErr`(`err.*` 失敗 / `warn.*` 確認が要る警告 / `hint.*` 可能なときの一言。入れ子の理由は `{why}`)で、App の `OpText.Reason` / `Warnings` が表から文にする。個別指導の検証(`PlanRules.ValidateMenu` / `CanGuide`。08 R-06)も同じキーを返し、Python 参照(`ref_model_08.py`)とゴールデンはキーに合わせた。表は 77 件(開発者向けの中断セーブの記録の検証 11 件は本文なし)	試験 `OpTextTests`: すべての定義の本文があり差し込みの名前が引数と一致する・孤立したキーがない・旧い本文と同じ文が出る。変異 4 本で確認
L-62	**解決(M1 準備。①② は受け入れ)**	中断セーブ(09 §4.6)の戻らないもの: ①再開した週に選び直すイベントの選択肢 ②月末レポートの今月の前の週の分(`GameFlow` の月の状態を保存していない)③アプリ側(App 層)の実装	①②は**受け入れる**(決定): 通常の読み込みも同じ性質(月末レポートの月の途中の状態は保存せず、月の途中から復元したときは『変化』を出さない設計 = `MonthReport.StartKnown`)で、中断だけ特別に保存すると『中断 → 再開』と『保存 → 読み込み』の結果が食い違う。失うのは月末レポートの今月の前の週の項目(故障・覚醒・お知らせ)と、選び直すイベントの選択だけで、状態(選手・資金・乱数)は 1 バイトも変わらない(09 §4.6 の検算)。直すなら流れの月の状態(月の初めの能力・各週の項目)を保存の側に足す。**③は `GameHost`(`Assets/Scripts/App/`)で実装した**(09 §17.7)。Unity の層は `OnApplicationPause(true)` / `OnApplicationFocus(false)` / `OnApplicationQuit` で `OnAppPause()`、戻ったら `OnAppResume()`、毎フレームか定期的に `Tick()` を呼ぶだけ
L-64	**解決(M1 準備)**	素養の判明が未実装だった(`Runner.GiftRevealed` を立てる処理が無く、素養は画面に永久に出なかった)	実装した(10 §8): ①週の解決で入部から 12 週(`K_GIFT_REVEAL_WEEKS`)経った選手を判明にする(`WeekReport.GiftRevealed` = 素養を持つ選手だけ『判明した』印)②スカウトで種類まで見えた(練習会の招待 + スカウト担当 Lv3 以上)選手は入部と同時に判明 ③月末レポートに `gift_reveal`(`ReportItemView.Gifts`)④ `RunnerRowView.MountainAdvice`(登坂 / 下りの素養が判明 →『山対策のグループで伸ばせます』)。判明の前後で能力の伸びは変わらない(試験で確認)。新規ゲームの初期の部員は最初の週に判明する。08b G-01
L-65	一部解決	画面の読み取り API の残り(08b §4): **解決済み** G-01(L-64)・G-02(L-60 ③)・G-03・G-04(L-53 ③)・G-05 惜しかった候補・G-09〜G-13・G-08 実績・G-14 能力の増減・G-15 調子の見込み・G-16・G-17 留学生の出身国・G-07 通知のログ(`player_team.notifications`。追加の項目)。**残り**: G-06 の選択肢の本文に数値が書き込まれている問題と選べない理由の文(L-63 ①)。実績は判定と記録まで(プロフィールのファイルの読み書きとストアへの通知は Unity 側)	残りは M1 の縦切りの前には要らない(画面ができる段階か、後でよい)。保存の形が変わるものは G-07 で済んだ

ID	状態	課題	対応・方針
L-66	一部解決(① の検証・修復 済)	App 層(`GameHost`)がつないでいないもの: ① 読み込みでの**検証・修復**は**つないだ**(`Persistence/Snapshot/GameLoader.cs`。自校の半分 `team` に `SaveValidator.ValidateAndRepairTeam`(修復は直して続ける / 致命は次の候補へ)→ 復元 → 世界の半分を `WorldInvariants` で検査。通常の読み込みも中断セーブの再開も同じ。修復は `OpenResult.Repairs` に残し、利用者には知らせない)。**移行**(`SaveMigrator`、移行前の控え)は、スキーマの版をまだ上げていない(追加の項目だけ)ので未接続で、最初に版を上げるときにつなぐ ②クラウド同期(09 §9)・購入状態 ③書き込みを別のスレッドへ回す実装(`Synchronous = false` と `Pump` はある。Unity 側のワーカー)④セーブの暗号化・改ざん対策は不要の方針(01)のまま	①の移行は版を上げるときに。②は Unity の層(L-34)。③は Unity の層
L-67	**解決(M1 準備)**	モデルとした大会の通常大会の出場枠が拡大される(単独校 20 → 23 校。予選会の通過 10 → 13 校。シードは 10 校のまま。オープン参加の学生連合チームが別に 1 つ加わる)。製品のリリースは新しい枠の施行後になるので、ゲームの第三戦も合わせる	**第三戦 = 23 校(シード 10 + 予選会の通過 13)**にした(04 §2・§3)。学生連合はゲームに作らない(従来の 20 校も連合を除いた数。順位・シードの対象外なので、作るなら 24 チーム目の別扱い = 未設計)。世界は 36 校のまま(予選会は 26 校のうち 13 校が通る)。**数値の順序どおり**: 参照モデル(`ref_model_04.QUAL_PASS` = 13)→ 帯の取り直し(`verify/band_05.py`。3 開始校 × 120 シード)→ 仕様書 → C#(`RaceConstants.QualPass`・`Acceptance.Bands`)。ゴールデンは内容が変わらない(世界・レースの式は通過校の数に依存しない)。出場枠の拡大そのものの影響は小さい(同じシードで優勝率 +0.00〜+0.03・クリア −1.3〜+0.1 年度。同好会も 0.161 → 0.164・クリア 39.5 年のまま)。**T3 はバイト一致・決定性の目印(抽象の自校)は意図して更新**・T4a / T4b すべて帯の内側・T4c は帯の内側(強豪の三冠率が帯の下端を 0.009 下回ったため、実体の優勝率・三冠率の下側に 0.03 の余裕 `REAL_RATE_SLACK` を足した。実体は抽象より三冠率が 0.03〜0.06 低いのが既知(L-55)で、帯の幅はこの系統差を見込んでいなかった)・300 年度の安定 3 開始校とも合格・T1 ベクタ不一致 0。記念大会(4 年に 1 度。出場枠をさらに広げ予選会を全国に開く)は扱わない = 拡張メモ M-03

ID	状態	課題	対応・方針
L-68	**解決(①② 済。③値の確認は製品化の前・④ は別件)**	実在の区間記録を各大会の初期の記録にしたい(最初から架空の記録で埋まっていると現実味がなく、魅力が下がる)。予熱の 15 年度は自校と無関係なので、そこで他校のエースが出した区間記録は、年度 1 の前に実在の記録へリセットしてよい。初年度に更新ラッシュで消えるのは受け入れる(ユーザーの決定)	**①解決(新規ゲームの区間記録の初期値)**: `GameSession.New` が予熱のあと・年度 1 の前に区間記録を『初期の区間記録』(02 §4.8 の表。3 大会 24 区間。実在の公表値の水準・**名前は載せない**)に置き換える(`RecordSeeds`・`RaceHistory.SeedLegacyCourseRecords`)。保持者 = **『歴代の記録』**(`school_id -1`・`runner_id 0`・`year -100`。画面は `record.legacy_holder`・View は `IsLegacy` / `PrevIsLegacy`)。順位表・大会記録は予熱のまま。`WorldSim.Create` は変えない = 世界の進行・乱数・再現性の目印(抽象の自校 8 件)は不変・保存の形は変えない(`school_id -1` を許すだけ。既存のセーブはそのまま)。**測定**(`SimCli records <world か開始校> 60 45 --reset`): 年度 1 の大会で破られるのは約 9 区間(第三戦 6・8・9・10 区・第二戦 6 区・第一戦 1 区など 98〜100%、ほか半分程度)で、残りは数年かけて破られる。ほとんど破られないのは第一戦 2 区(98% が 45 年度そのまま)・第三戦 3 区(100%)・第三戦 1 区(67%)。理由: ゲームのコースは平地が基本(5 区の登りと 6 区の下りだけ補正)で、実在のコース固有の遅さを持たないため、実在の記録が遅めの区間は簡単に破られる。予熱の記録を初期値にする現行方式との比較: 記録が持つ中央値は 8〜10 年度(10 年度以内に破られる区間 42〜54%・一度も破られない 15〜23%・期末の保持者が自校の走者 7〜39%)。**②解決: 大会記録(総合タイム)は予熱のまま**(ユーザーの決定。3 大会とも最新の実在の値を確認できていないことも理由。予熱の総合記録は、ゲームの世界の水準に合っている)。**③未決: 値の確認**(いまの表は 2026 年 10 月時点の検索で拾った値で、資料により数秒〜2 分の差がある(5 区など)。製品化の前に一次資料で確かめ直す)。④解決(L-69): コース固有の補正(地形係数)を実在の記録から較正した。年度 1 で半分以上破られる区間は 9 → 5 になった

ID	状態	課題	対応・方針
L-69	**解決(M1 準備)**	L-68 ④: ゲームのコースは平地が基本(5 区の登りと 6 区の下りだけ補正)で、実在の区間記録を初期値にすると、実在の記録が遅めの区間は簡単に破られ、速めの区間はほとんど破られない(24 区間のうち約 9 区間が年度 1 の大会で半分以上の確率で破られ、ほとんど破られない区間が 3 つ)。L-10(区間タイム式と実在の記録との照合)の一部	**地形係数を、初期の区間記録から較正した**(02 §3.5 の表。`terr = (R / d + K_SLOPE × (P* - 50)) / pace50(d)`・`P* = 85`・小数 3 桁。`verify/calib_terrain.py`、C# は `RaceTables`・`RecordSeeds.TerrainPStar`)。24 区間とも 0.96〜1.05(5 区 1.13 → 1.127・6 区 0.93 → 0.963・第三戦 8 区 1.00 → 1.043・3 区 1.00 → 0.978 など)。**P* はコース全体の速さの水準でもある: 88 にすると記録は長持ちする(最初に破られるまで 6〜9 年度)が、能力の差が時間に占める割合が小さくなり、弱小開始の優勝率が 0.43 → 0.36・クリア 24.5 → 28.8 年度(60 シード)と難しくなる。85 は難しさを変えない(0.44・25.0)ので 85 にした。効果(世界・60 シード・年度 1 の前に実在の水準へ置き換え): 年度 1 で半分以上破られる区間 9 → 5・ほとんど破られない区間(45 年度間そのまま 90% 以上)3 → 0・最初に破られるまでの年数の中央値 1〜3 → 4〜5 年度。数値の順序どおり: 参照モデル(`ref_model_04.COURSE`・`ref_model_05.TERRAIN`)→ 帯の取り直し(`band_05.py`。3 開始校 × 120 シード。強豪 0.590 / 0.305 / 6.35・中堅 0.553 / 0.275 / 12.99・弱小 0.441 / 0.185 / 25.48。L-67 のときの値との差は誤差の 1〜2 倍)→ 仕様書(02 §3.5・04 §13 TV-6・05 §14・09 §16.3)→ C#(`RaceTables`・`Acceptance.Bands`・ゴールデン `race_cases`・TV-6 の試験)。T1 の 139 行は参照モデルと全行一致・T3 はバイト一致・再現性の目印(抽象の自校 8 件)は意図して更新・T4a / T4b すべて帯の内側・T4c は帯の内側。T4c のクリア年度の余裕を 1.0 → 1.5 年度にした**(`REAL_CLEAR_SLACK`): 帯を取り直したとき強豪のシード間の標準偏差が 3.2 → 2.8 に小さくなって帯の幅が 1.25 → 1.07 に狭まり、実体の既知の遅れ(同じ世界の抽象より +1.5〜1.8 年度。L-55)を足すと 1.0 では足りなくなった(強豪 8.76 > 6.35 + 1.07 + 1.0)。実体が抽象より遅いこと自体は変わっていない(今回 +1.5 年度)。世界に依存していた試験 2 件(当日変更の 2 回目・予選会のピーキング)は、シードが変わる世界の側の偶然(留学生の出走枠・予選会に出るか)に頼っていたので、頼らない書き方に直した。L-16 の『指示が効く特性(向かい風・起伏)』は別件のまま

自校の戦力曲線(検算結果)(中堅開始、『おまかせ』、先発10人の力Pの平均。本番のレース式 04 v0.2 + 07 の券。`05` §14.2)

期間	戦力	内容
1〜4年度	65 → 66	最初の世代が育つまでの充填期。順位は7位前後から6位へ
5〜12年度	66 → 70	名声と施設で入学者の質が上がる成長期。順位は3位前後へ。優勝が出始める
13〜20年度	70 → 72	上位に定着。クリア(通算制覇)の中央値は12.5年度
25年度〜	約72で頭打ち	1大会あたりの優勝率 0.51(上級なら0.58)、三冠は年0.21、順位の平均1.6

開始校ごとの頭打ち後の優勝率は、強豪 0.57 / 中堅 0.51 / 弱小 0.46 / 同好会 0.14(05 §14.3)。クリア年度の中央値は 7 / 12.5 / 23 / 35.5(同好会は45年度までに50%がクリア)。

D.2 拡張メモ(未設計。採用は未決)

M-01 レジェンド保存とレジェンドオーダー(メモ)

- **要望**: 卒業生を 20〜30 人ほど「レジェンド」としてデータに残し、**歴代の選手でオーダーを組む**(レジェンドオーダー)。
- **既存の設計で足りる部分**: 卒業生は `OBRecord`(02 §4.7)に、レースに必要なデータ(10能力 `final_stats`・特殊能力・外見・名前・出身・主な記録)をすでに保存する。殿堂入りの印 `hall_of_fame` もある。**能力の分布は時代で変わらない**(01 §8)ので、別のセーブや別の時代の選手でも同じ物差しで比べられる。
- **追加が必要なもの**: ①レジェンド枠の指定(最大30人。上限を超えたら入れ替え)②**レジェンドの調子の決め方**(大会ごとの調子の乱数ではなく、固定または調整済みの分布にする)③レジェンドだけで走る**エキシビションのレースモード**(コースを選び、16人を登録

して10人を出走させる)④UI(レジェンド一覧・オーダー画面)。

- **容量**: 1人あたり約200バイトなので、30人でも数KB。
- **未決**: 能力は「卒業時」か「最も強かった年度」か(既定: 卒業時)。招聘券で入学した選手や、有料の見た目パーツを使った選手の扱い。
- **依存**: 04(本番の区間タイム)が前提。ロードマップは 01 §13 の M6。

M-02 データの出力と、他人のデータとの対戦(メモ)

- **要望**: データを出力して、他人のデータと対戦できるか。
- **結論**: **可能**。ただし範囲によって難しさが大きく違う。

段階	内容	難しさ	必要なもの
① 出力・取り込み	レジェンドのオーダー(10〜16人)を**共有コード**(圧縮した文字列)・ファイル・QRコードで出し入れ	小	共有用の形式(版番号・チェックサム付き)。10人のオーダーなら1KB未満でQR1枚に収まる見込み(要検証)。30人の殿堂は文字列かファイルが現実的
② 非同期の対戦(ゴースト)	自分と相手のオーダーを**同じ条件(同じコース・同じ乱数の種)**で走らせて比べる。サーバー不要	中	04 のレース式を入力データだけで動かせること。種は両者のコードから決める
③ 結果の検証・ランキング	改ざんや不正のないランキング	大	サーバー(結果の再計算・署名)。01 §6.5 の原則「オンラインランキングなし」と相反するので、導入するなら方針を変える

注意点

1. **結果の一致**: 02 §1 は「端末間のビット一致は求めない」としている。対戦で両者に同じ結果を見せるには、①片方の端末で計算した結果を共有する(簡単だが検証できない)か、②**ログ・指数・三角関数を使う計算を、表引きか固定小数点にして端末間で一致させる**のどちらかが要る。乱数そのもの(PCG32)は整数演算だけなので、すでに一致する。**09 §8.4 の実測**: 端末差で結果が変わる確率は 1 レースあたり約 10^{-12}〜10^{-9} なので、**両方のチームを同じ端末で走らせる**設計なら一致は要らない。端末間で必ず同じ結果が要る場合(③)だけ、決定的な数学関数(`IDeterministicMath`)を使う。
2. **版の互換**: ルールや特殊能力、コースが更新で変わると、古いデータで結果が変わる。共有データに**ルールセットの版**を入れ、版が違うときは取り込みを断るか、移行する。
3. **不正なデータ**: 能力を書き換えたデータが持ち込める。友達同士の対戦なら許容できる。許容できない場合は、不変条件(02 §5)と上限(能力 ≤ 99 など)で検査し、サーバーの署名が要る。
4. **有料要素**: 招聘券で入学した選手が対戦に出ると、課金の有無が勝敗に響く。対戦では「招聘枠の選手は2人まで」などの**対戦ルール**を別に設ける。見た目パーツは、相手が持っていなければ標準の見た目に置き換える。
5. **名前**: 選手名・大学名は架空で生成されるため、個人情報の問題は小さい。ユーザーが付ける名前(チーム名など)を共有する場合は、オンラインではフィルタが要る。
6. **共有の手段**: Steam は Workshop が使える。スマホは共有コードのコピー/ペーストと、OSの共有機能。形式は共通にして、機種をまたげるようにする。

推奨の進め方: ① → ② を先に作り(サーバー不要)、③ は需要を見てから判断する。

未決: ランキングを作るか(作るなら 01 §6.5 の原則を改める)/ 対戦ルール(出走人数・留学生枠・招聘枠)/ 共有の手段(コード・QR・Workshop)/ 乱数以外の計算を端末間で一致させるか。

M-03 記念大会(メモ)

- **きっかけ**: モデルとした大会は、通常大会の出場枠を単独校 20 → 23 校に広げる(L-67。ゲームの第三戦は 23 校にした)。あわせて、4 年に 1 度(夏季の国際大会の年)の**記念大会**では、出場枠をさらに広げ(単独校 25 校)、**予選会を全国の大学に開く**と発表されている。ゲームでは記念大会を扱っていない。
- **案**: 年度 4 回に 1 回(例: 年度が 4 の倍数)を『記念大会の年度』にし、第三戦の出場校を +2(= 25 校)にする。予選会の出場校も広げる必要があるが、世界は 36 校で足りない(シード 10 を除く 26 校のうち 15 校が通る形になり、予選会の意味が薄れる)。世界の校数を増やす(全国の弱小校を追加)か、記念大会の年度だけ予選会に『地方の大学』を足す必要がある。
- **未決**: 世界の校数・歴史(殿堂・記録)との整合・記念の年度の名声・評価ポイントの重み・実績『記念大会優勝』の要否。継続の記念(券の節目 `Anniversary`。07 §4)と別に作るか。

- **依存**: 04 §3(出場権)・05(世界の更新)。ロードマップは 01 §13 の M6 以降。

付録E 検算ツールと実行方法

E.1 参照モデル(Python)と検算の実行

検算用の参照モデル: `verify/` に、仕様の数値を実装して検算するPythonスクリプトを置く(ゲーム本体のコードではない)。仕様書の計算例はここから出力した値を使い、後の実装ではそのまま自動テストに写す。

- `verify/ref_model_03.py` (03 用)。`python3 -I ref_model_03.py vectors` で計算例、`calib` で検算結果を出力する。
- `verify/ref_model_04.py` (04 用)。`python3 -I ref_model_04.py vectors` で計算例、`anchor` (基準タイム)・`odds` (力の差と優勝率)・`manage` (登録・型に合わせた育成・檄の大きさ)・`skill` (采配の価値。方針ごとの優勝率)で検算結果を出力する。
- `verify/ref_model_05.py` (05 用。レースは 04 の本番式を使う)。`python3 -I ref_model_05.py run Mid` で長期の推移(開始校は `Elite` / `Mid` / `Low` / `Club`。戦力順位 3 / 7 / 20 / 33 位の学校)、`vectors` で乱数を使わない計算例を出力する。
- `verify/ref_model_06.py` (06 用)+ `verify/events_06.py` (イベントのカタログ)。`python3 -I ref_model_06.py vectors` で計算例、`validate` でカタログの検証、`sim 12` で発生頻度・所要時間・効果の大きさ、`tables` で仕様書の表を出力する。
- `verify/ref_model_07.py` (07 用)。`python3 -I ref_model_07.py vectors` で計算例(招聘選手の生成・券の状態遷移・購入状態の判定)、`gen` で生成帯の検証、`timeline` で券の取得時期を出力する。世界シミュレーションの券・シナリオの設定は `ref_model_05.py` 側(`K.TICKET_EARN` / `K.SCENARIO`)。
- `verify/ref_model_08.py` + `verify/screens_08.py` (08 用。画面のカタログ)。`python3 -I ref_model_08.py vectors` で計算例、`graph` (画面の到達性・コマンドと情報の網羅・パッドの移動)、`doccheck` (08a とカタログの突き合わせ)、`forecast` (月末の予測の厳密性と較正)、`autoplan` (重点ごとの適性値の差)、`budget` (1年度の時間)、`layout` (収まり)、`stops` (割り込みと月末レポートの量)、`preview` (前評判の較正)、`scout` (競合度と枠。世界シミュレーション)、`rank`、`tables` (仕様書の表)、`all` (一括)。`ref_model_05.py` には較正用の記録(`TRACE` / `TRACE_PREVIEW`)を追加した。
- `verify/ref_model_09.py` (09 用)。`python3 -I ref_model_09.py vectors` でテストベクタ、`all` (約 1 分)で一括。個別: `size` (セーブの大きさ)・`container` (コンテナ・A/B世代・電源断・長さの上限)・`migrate`・`validate` (検証と修復のファズ)・`rng` (鍵の登録表・衝突・適合ベクタ)・`perturb` (端末差の摂動実験)・`sync` (同期の判定と乱数シミュレーション・プロフィールのマージ)・`clock` (購入状態の時計対策)・`q6`・`replay` (中断からの再開)・`timing` (所要時間)・`fmt` (整形)・`inventory` (他の参照モデルの実行と文書の数・定数の突き合わせ)。
- `verify/ref_model_10.py` (10 用)。`python3 -I ref_model_10.py vectors` でテストベクタ(生成・乗数・1週の解決・相性の印・レースの補正・取得条件)、`gen` (生成の分布)・`train` (練習への影響。約 4 分)・`neutral` (平均の効き)・`select` (登録の選び方の価値)・`bond` (走法と絆)・`race` (レースの中立の確認)・`skill` (特殊能力の取得率。約 10 分)・`world` (長期バランスの換算。約 4 分)・`all` (縮小版)。03・04・05・06・08 の参照モデルのフック(`gm` `bond_k` `ind_mult` / `style` / `PLAYER_RACE_BONUS` / `Sim.individuality`)を通して、本物の式の中で測る。
- `verify/fit_progress_05.py` (05 の進捗率の近似式を、03 の詳細モデルから再現する)。
- `verify/band_05.py` (09 §16.3 の『受け入れの帯』を、参照モデル 05 をシード別に回して取り直す。`python3 -I band_05.py <Elite|Mid|Low> [シード数 120] [予選会の通過校数]`。L-67)。
- `verify/gen_golden_03.py` (M0 の差分テスト用。03・10 の週の解決を 5,500 件・4 年間の育成 60 人ぶん解いて `m0/tests/Domain.Tests/Golden/` に出力する)。同様に `gen_golden_04.py` (レース)・`gen_golden_05.py` (世界の式)・`gen_golden_06.py` (イベント。条件式・派生値・発生と効果・検証違反。簡易チームシミュレーションの週の途中の状態を保存する)・`gen_golden_07.py` (招聘選手の生成・券の状態遷移・購入状態の判定)。`gen_golden_09.py` は永続化に加えて**自校の保存**(`ref_model_09.py team_save`。09 §5.6)を 150 件書く。`gen_events_cs.py` は `events_06.py` から C# のカタログ(`m0/Assets/Scripts/Domain/Events/EventCatalog.g.cs`)を作る。

E.2 C# の実装と実行コマンド

実装: `m0/` (M0 の C# 実装。Unity 不要。09 §15.2 の構成。進み具合は `m0/README.md`。段階 0〜10 が完了し、統合の命令 `SimCli accept / stability / determinism / bench / save / fuzz-save / sync-sim / vectors` が合格。

`m0/tools/compare_vectors.py` が T1 の主なベクタを参照モデルと突き合わせる。09 §17.5)。

実行コマンド(`m0/` で。原典 `CLAUDE.md` より):

```
cd m0
dotnet test Ekiden.sln -c Release          # 685 件(Domain 508 + Persistence 137 + App 40)。約 1.5 分。全部緑が前提
dotnet build tools/SimCli/SimCli.csproj -c Release
S="dotnet run --project tools/SimCli/SimCli.csproj -c Release --no-build --"
$S determinism 2 45                        # T3 + 再現性の目印(下)
$S accept all 120 45 --real                # T4c(約 5 分)。--skilled を足すと上級の方針との比較(約 12 分)/ --scout-button = スカウトを画面のボタンで(L-58)/ --flow-only = その基準
$S flow 1 Mid 2                            # 進行制御の停止の並び
python3 -I tools/compare_vectors.py        # T1 のベクタを SimCli(C#)と参照モデルで突き合わせる(自分で SimCli を走らせる)
python3 -I ../verify/ref_model_09.py inventory
python3 -I ../verify/lint_tables.py        # 仕様書の表の列数(0 が合格)
```

Python は必ず `python3 -I` (カレントのファイルを読み込ませない)。SimCli の全コマンドは `m0/README.md`。

仕様書の統合版 PDF: `python3 -I specbook/build_spec_pdf.py` (約 1 分。pandoc・Node の playwright・poppler の pdftotext・Python の pypdf と reportlab が要る。しくみは同スクリプトの先頭)。`front.md` の現況(第 0 部 §7)の数字は手で直す。

付録F 仕様と実装(C#)・試験の対応

F.1 仕様の各部と実装の対応

実装は `m0/`（.NET 8 / `netstandard2.1`。Unity プロジェクトの根を兼ねる）。パスは `m0/Assets/Scripts/` からの相対。試験は `m0/tests/` の NUnit（`Domain.Tests`・`Persistence.Tests`・`App.Tests`）。

部	仕様の範囲	実装(C#)	参照モデル・生成物（`verify/`）	主な試験
2 データ	列挙・マスタ・エンティティ・不変条件	`Domain/Entities/`（`Runner` `Enums` `Style` `Individuality`）・`Domain/Numerics/`（`Q6` `Rounding`）・`Persistence/Validation/SaveInvariants.cs`	`ref_model_09.py`（不変条件の検査）	`DocSyncTests`（文書の定数表と C# の一致）・`Q6Tests`・`GoldenTests`
3 育成	週メニュー・週の解決・月次・個別指導	`Domain/Training/`（`WeekResolver` `TrainingMaster` `Progression` `YearPlanner`）・`Domain/Team/TeamWeek.cs`・`Domain/Screens/Forecaster.cs`	`ref_model_03.py` / `gen_golden_03.py` → `week_cases` `careers`	`TrainingVectorTests`・`GoldenTests`（週の解決 5,500 件・4 年間の育成 60 人）・`OutlookTests`
4 レース	出場権・予選会・登録・オーダー・区間タイム・介入・大会後	`Domain/Race/`（`RaceSimulator` `RaceMath` `DayPlanner` `Hungarian` `Qualifier` `Pace50` `RaceHighlights`）・`Domain/Game/RaceDay.cs`・`Domain/Team/TeamRacing.cs`	`ref_model_04.py` / `gen_golden_04.py` → `race_cases` / `gen_master_pace50.py`	`RaceVectorTests`・`RaceGoldenTests`（区間 3,000・ハンガリー法 300・レース 450・当日変更 240・予選会 40）・`RaceDayTests`・`GekiFactorTests`・`MinorInjuryTests`・`RaceHighlightTests`
5 世界・スカウト	名声・資金・施設・候補・スカウト・勧誘・他校・入学卒業 OB	`Domain/World/`（`WorldSim` `PlayerAgent` `Allocator` `Scouting` `PoolGenerator` `School` `RaceHistory` `WorldInvariants` `Acceptance`）・`Domain/Recruit/RunnerFactory.cs`・`Domain/Team/TeamYear.cs`・`Domain/Game/GameSession.Scout.cs`	`ref_model_05.py` / `gen_golden_05.py` → `world_cases` / `world_baseline_05.py`	`WorldVectorTests`・`WorldGoldenTests`・`ScoutingTests`・`ScoutSessionTests`・`RaceHistoryTests`・`RecordTests`・`MissedCandidateTests`
6 イベント	条件式 DSL・効果・補正・覚醒・カウンター・カタログ	`Domain/Events/`（`EventPhase` `EventBuilder` `EventValidator` `EventContext` `EventDef` `EventCatalog.g.cs`）	`ref_model_06.py` + `events_06.py` / `gen_golden_06.py` → `events_cases` / `gen_events_cs.py`	`EventVectorTests`・`EventGoldenTests`（条件式 899・評価 105 状態・発生と効果 265 状態 52 種類）

部	仕様の範囲	実装(C#)	参照モデル・生成物 (verify/)	主な試験
7 特別招聘	券・黄金世代プラン・招聘選手・購入状態・追加コンテンツ	Domain/Tickets/TicketState.cs・Domain/Recruit/SpecialRecruit.cs・Persistence/Store/(Entitlement LocalState)	ref_model_07.py / gen_golden_07.py → special_cases	SpecialVectorTests・SpecialGoldenTests・EntitlementTests・LocalStateTests
8 画面	進行制御・画面のための規則 R-01〜R-17・読み取り API	Domain/Game/GameFlow.cs・Domain/Screens/(PlanRules FlowRules DisplayRules Presentation)・Domain/Game/GameSession.Views*.cs と Domain/Views/・App/EffectText.cs	ref_model_08.py + screens_08.py / gen_golden_08.py → screen_cases	ScreenGoldenTests・GameFlowTests・ViewTests・DialogViewTests・PlanEntryOrderViewTests・ResultViewTests・ClubViewTests ほか(画面ごと)・ViewSchemaTests (型の形)・ViewPurityTests (状態を変えない)・EffectTextTests
9 セーブ・技術	コンテナ・世代・移行・検証と修復・乱数・同期・購入状態・性能	Persistence/(Container Json Migration Validation Snapshot Store Sync Strings)・Domain/Rng/・Platform/・App/(GameHost SaveService SlotCatalog AchievementTracker)・Verification/	ref_model_09.py / gen_golden_09.py → persist_cases	ContainerTests・PersistGoldenTests・SyncTests・DeterminismTests・GameSnapshotTests・TeamSnapshotTests・SuspendRaceTests・GameLoaderTests・HostTests・StringsTests・RngTests
10 個性・素養	型・走法・展開・素養	Domain/Entities/(Individuality = 生成・乗数 GrowthMult BondK IndMult・Style)・Domain/Training/WeekResolver.cs (練習への影響)・Domain/Race/ (レースの補正)・Domain/Screens/Presentation.cs (相性の印)	ref_model_10.py	GiftRevealTests・GoldenTests (個性つきの週の解決)・RaceGoldenTests
(実績)	実績 23 件の定義と判定	Domain/Achievements/・App/AchievementTracker.cs・Persistence/Sync/ProfileData.cs	—	AchievementTests・AchievementHostTests
(通知)	通知のログ(SC-38)	Domain/Team/Notification.cs・Domain/Game/GameSession.Notifications.cs	ref_model_09.py (保存の形)	NotificationTests・TeamSnapshotTests

F.2 実装の層と依存

```
Presentation(Unity の UI。M1 以降)
      | 呼ぶ
      ▼
App ── GameHost / SaveService / SlotCatalog / AchievementTracker / EffectText
      |                          |
      ▼                          ▼
Domain(全規則・純 C#)     Persistence(コンテナ・世代・移行・検証と修復・同期)
Game(GameSession/GameFlow) Platform.Abstractions(IFileStore / IClock / ILocalization)
Team / Training / Race / World / Events / Tickets / Views / Screens …
```

- Domain は `UnityEngine`・時刻・ファイルを参照しない。同じコードを、EditMode テスト・SimCli(.NET コンソール)・Unity が動かす。
- 画面(Presentation)は `GameSession` の読み取り API(`Views`)を呼んで**表示するだけ**にし、操作は `GameSession` のコマンド(理由つきで拒否できる `OpResult`)を呼ぶ。
- 保存・読み込みは `GameHost`(App)が呼び出し位置(月の最初・大会の前・年度末の各節・手動・アプリの中断)を受け持つ。

F.3 主要な型と入口

入口	役割	仕様
`GameSession`	36 校の世界 + 実体の自校の 1 回のプレイ。週は段階(`ResolveTraining → PlayWeekRaces → … → FinishWeek`)、年度末は `SettleYear → TurnoverYear`。操作は `*.Commands.cs`・`*.Scout.cs`・`*.Race.cs`	第 3〜8 部
`GameFlow`	進行制御。`Advance()` だけが前に進み、`StopKind` の停止で止まる	08 §6
`RaceDay`	大会の当日(当日変更・指示・檄)。決定のたびに全チームを再計算(`Preview`)	04 §9・§14.5
`GameSnapshot` / `TeamSnapshot` / `WorldSnapshot`	状態の取り出しと復元。保存 → 読み込み → 続行 = 続行のみ	09 §5・§17
`GameLoader`	読み込み(自校の半分を検証・修復 → 復元 → 世界の不変条件)	09 §7.1b
`GameHost`	App 層の入口。自動セーブ・中断セーブ・読み込み・保存の失敗の通知	09 §4・§17.7
`WorldSim`	世界の年度処理(`BeginYear → PlayQualifier → PlayRace → EndYear`)。自校だけ `IPlayerSide` で差し込む	05
`SimCli`	検証用コンソール(`accept` `determinism` `bench` `save` `fuzz-save` `sync-sim` `vectors` `stability` `play` `flow` `world`)	09 §16.3

F.4 注意すべき実装上の約束

- **保存の境目**: 週の最初の処理(`BeginWeek`)は前の週の終わりに走る。復元後は派生値の再計算(`Refresh`)が要る。
- **生成物は手で編集しない**: ゴールデン(`tests/*/Golden/*.json.gz`)・`EventCatalog.g.cs`・`NameTable.g.cs`・`Pace50Table.g.cs` は `verify/gen_*.py` が作る。
- **仕様書の定数表はテストが読む**(`DocSyncTests`)。表の値を変えたらコードを合わせる。表を足したら `verify/lint_tables.py`(列数の検査)。
- **文字列**: `Assets/Strings/ja.json` が正(キーは約 740)。語は文字列テーブルのキーで返し、Domain に表示用の文を持たない(例外: `OpResult` の理由。L-63 ③)。

付録G 改版履歴

各文書の冒頭に書かれていた改版メモ(『vN の変更』『NN の反映』)を、原典の文言のまま集めた。第 1〜10 部の本文からは省いてある。文書の版は、その文書を大きく改めたときの版で、番号は文書ごとに独立している(本書の版とは別)。

G.1 第1部 基本システム設計書(01)

原典: `01_basic_system_design.md`

- v0.1 からの主な変更: ①任期なし・世代交代で無期限に継続 ②Unity / Steam+スマホ / 2D の確定 ③プレイ時間バジェットの追加と、それに伴う「月間計画」方式への操作設計の変更 ④世代交代・OB・歴史の設計を追加
- v0.2 からの主な変更: 未決事項への回答を反映(クリア条件・大会名の扱い・画面向き・買い切りを確定、アートはドット絵SD+パーツ合成に決定)。美術の調達方針を追加(§11)。
- v0.3 からの主な変更: ①**仕様優先**へ方針転換(Unityは後回し。仕様書一式を `00_spec_index.md` に整理)②**出身県**の導入(§4.5, §6.1, §6.4)③**特別招聘券(翌年度に留学生・スーパールーキーが確定入学)**と課金方式の方針案を追加(§6.5)。
- v0.4 からの主な変更: 課金方式を**永続解放**に決定(消費型は採らない)。券の配布周期を検算し、未決事項として整理(§6.5, U-12)。
- v0.5 からの主な変更: 03(育成ルール仕様)の確定に伴い、夏合宿・ピーキング(大会連動の自動適用)・安全網・疲労効率・故障確率の数値を更新(§3.2, §5)。
- v0.7 からの主な変更: 08(画面仕様)の確定に伴い、**年度の時間バジェットに年度末を追加**(§3.4)、年間方針の重点を **バランス / スピード寄り / ロング寄り** に変更(§5.1)、スカウトの時期・ポイントの繰り越し・推奨の目標 8人(§6.1)、画面一覧を 08 の42画面に置き換え(§11)。
- v0.8 からの主な変更: 09(セーブ・技術仕様)の確定に伴い、**§12 の技術設計**(プロジェクト構成・セーブ・乱数・マスタの置き場所・決定性の規則)と **§13 の M0 の完了条件**を具体化した。マスタは `StreamingAssets` ではなく `TextAsset` (Android で直接ファイルとして読めないため)。
- v0.9 からの主な変更: 10(個性・素養仕様)の追加に伴い、**選手の個性(型・走法・展開)と素養(登坂・下り・エース)**を §4.3a に追加。
- v0.6 からの主な変更: 04(駅伝本番仕様)の確定に伴い、采配(ペース配分・当日変更・檄)を強化(§7)。**実績(Steam 実績ほか)を搭載**し、開始校を新規ゲーム時に選ぶことを明記(§3.3)。券の効果は公平性の制約にせず、『無招聘』の実績で分ける(§6.5)。バランス表を再検算した値に更新(§12)。

G.2 第2部 データ仕様(02)

原典: `02_data_spec.md`

- **v1.1 の変更(10 の確定に伴う)**: 選手に**個性**(`style` : 型・走法・展開)と**素養**(`gifts` ・ `gift_revealed`)を追加(§4.1・§4.7)/ 列挙 `StyleForm` `StyleRun` `StyleFlow` `Gift` (§2)/ 不変条件 I-21(§5)/ 派生値 `gm` `bond_k` `ind_mult` は保存しない。
- **v1.0 の変更(09 の確定に伴う)**: `SaveData.rng` を鍵つき(状態なし)に(§4.9)/ `schema_version` の規則と刈り込みの規則・サイズ(§4.9・§4.7・§4.8)/ `ProfileData.entitlements` を `LocalState` (09 §3.3)へ移した(§3.10)/ `DeviceSettings.cloud_sync` を追加・ `SaveMeta` はセーブのヘッダの部分集合(§3.12)/ 再現性の注(`pace50` の表。§1)/ 不変条件の読み込み時の分類(§5)。

G.3 第3部 育成ルール仕様(03)

原典: `03_training_rules_spec.md`

- **v0.3 の変更(10 の確定に伴う)**: §4.7(個性・素養による乗数)と §6 手順1・2の式(`bond_k` `ind_mult` `gm`)を追加。個性・素養がない選手は従来と同じ値。

- **v0.2 の変更(08 の確定に伴う)**: 個別指導(§5.7)を追加。`form_add` の上限の読み方を確定(§6、§6 末尾)。重点ごとの適性値の差(§11)を追記。
- **10 の反映**: 選手の**個性**(型・走法・展開)と**素養**を、手順1・2の式に入れた(`E_bond` の係数 `bond_k`、個別指導の週の `ind_mult`、能力ごとの乗数 `gm`。§4.7・§6)。個性・素養がない選手(従来の検算)は、`bond_k = 0.15`・`ind_mult = 1`・`gm = 1` で**従来と同じ値**になる(テストベクタは不変)。
- **09 の反映**: §3.2 の鍵の予約は、09 §8.2 の**登録表**(label 1〜8。b の範囲の重なり 0・520,798 個の鍵で衝突 0)に集約した。端末差の実測は 09 §8.4(L-12 解決)。

G.4 第4部 駅伝本番仕様(04)

原典: `04_race_spec.md`

- **v0.2 の変更(課題 L-13)**: 采配の手応えを強めた。①**ペース配分(失速)**を追加し、指示(攻める / 維持 / 抑える)を「余裕の目安」を見て決める判断にした(v0.1 は『攻める』を全区間に指示するだけで優勝率が 0.5 → 0.8〜0.9 になる穴があった)。②**当日変更**を全大会に広げた(1 / 2 / 4名)。③**環境の個人差**を2倍にした。④檄をブレーキと失速の両方に使えるようにした(効果は損失 ×0.8)。

G.5 第5部 世界・スカウト・名声・資金・施設 仕様(05)

原典: `05_world_scout_spec.md`

- **v0.3 の変更(08 の確定に伴う)**: スカウトの時期・ポイントの繰り越し・推奨の目標数(§7.1)、`pwin` と V 順の確定の注記(§7.3, §8.2)、§14.6『型に合わせた育成』の有無と §14.7 スカウトの枠・操作できる方針の検算を追加。